



“十二五”普通高等教育本科国家级规划教材  
21 世纪高等学校计算机规划教材  
21st Century University Planned Textbooks of Computer Science

# 数据结构

(C语言版)(第2版)



Data Structure (2nd Edition)

严蔚敏 李冬梅 吴伟民 编著

符合新考研大纲要求

PPT电子教案

获高校电子课件评比

一等奖

- 32、48、64学时参考教学大纲
- 书中超过200道课后习题，另附赠电子版习题库
- 电子版实验指导书，满足平时实验和小学期使用要求
- 全书课后习题答案和C语言案例代码
- 书中算法的Flash动态演示



名家系列

人民邮电出版社  
POSTS & TELECOM PRESS



“十二五”普通高等教育本科国家级规划教材

21 世纪高等学校计算机规划教材

21st Century University Planned Textbooks of Computer Science

# 数据结构

(C语言版)(第2版)

Data Structure (2nd Edition)

严蔚敏 李冬梅 吴伟民 编著



名家系列

人民邮电出版社

北京

## 图书在版编目 (C I P) 数据

数据结构 : C语言版 / 严蔚敏, 李冬梅, 吴伟民编著. — 2版. — 北京 : 人民邮电出版社, 2015.2  
21世纪高等学校计算机规划教材. 名家系列  
ISBN 978-7-115-37950-4

I. ①数… II. ①严… ②李… ③吴… III. ①数据结构—高等学校—教材②C语言—程序设计—高等学校—教材 IV. ①TP311.12②TP312

中国版本图书馆CIP数据核字 (2015) 第004999号

## 内 容 提 要

本书在选材与编排上, 贴近当前普通高等院校“数据结构”课程的现状和发展趋势, 符合最新研究生考试大纲, 内容难度适度, 突出实用性和应用性。全书共 8 章, 内容包括绪论, 线性表, 栈和队列, 串、数组和广义表, 树和二叉树, 图, 查找和排序。全书采用类 C 语言作为数据结构和算法的描述语言。

本书可作为普通高等院校计算机和信息技术相关专业“数据结构”课程的教材, 也可供从事计算机工程与应用工作的科技工作者参考。

- 
- ◆ 编 著 严蔚敏 李冬梅 吴伟民  
责任编辑 刘 博  
责任印制 沈 蓉 彭志环
  - ◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路 11 号  
邮编 100164 电子邮件 315@ptpress.com.cn  
网址 <http://www.ptpress.com.cn>  
北京铭成印刷有限公司印刷
  - ◆ 开本: 787×1092 1/16  
印张: 17.5 2015 年 2 月第 2 版  
字数: 460 千字 2015 年 2 月北京第 1 次印刷
- 

定价: 35.00 元

读者服务热线: (010)81055256 印装质量热线: (010)81055316  
反盗版热线: (010)81055315

## 第 2 版前言

---

---

---

---

---

---

---

---

本书第一版自出版以来，深受广大读者欢迎，被百余所学校选为“数据结构”课程的教材，并被评为“‘十二五’普通高等教育本科国家级规划教材”。为了更好地满足广大高等院校的学生对数据结构知识学习的需要，编者结合近几年的教学改革实践、科研项目以及广大读者的反馈意见，并参考了大量文献资料，对教材进行了仔细的修订。这次修订的主要内容如下。

(1) 采用“案例驱动”的编写模式。书中结合实际应用，将各章按照“案例引入——数据结构及其操作——案例分析与实现”的案例驱动思路来展开。每章使用一个有趣的“问题案例”开头，由该案例逐步引入新的数据结构，然后给出该数据结构的存储表示及各种基本操作的实现，之后进一步分析此案例，最终利用该数据结构来实现此案例。这样，学生便能体会到从问题求解到程序设计的转换过程，深刻理解数据结构在程序设计中的作用。

(2) 算法讲解更加细致。学生学习数据结构最大的困难是不能将用文字表述的算法思想转换成程序。新教材中对每个算法思想进行详细阐述，将用文字描述的算法步骤与用类 C 语言表述的算法描述一一对应。尤其是对于有循环结构的算法，在算法步骤的描述中利用缩进的格式清晰地体现出了循环的执行过程。因为教材中的算法是由浅到深的，所以学生通过学习这些算法，在不知不觉中便逐步提高了将自然语言描述的算法转化为高级语言描述的程序的能力，真正提高了算法设计与算法实现的能力。

(3) 优化教材内容。参考计算机专业最新的全国统考考研大纲，增加了大纲近两年新增的考点内容，如分块查找、外部排序等，有助于考研学生复习备考使用。

本书共 8 章内容，其中第 1 章为绪论，综述数据、数据结构和抽象数据类型等基本概念；第 2 章至第 6 章从抽象数据类型的角度，分别讨论线性表、栈、队列、串、数组、广义表、树和二叉树以及图等基本类型的数据结构及其应用；第 7 章和第 8 章分别讨论查找和排序，除了介绍各种实现方法之外，还着重从时间上进行定性或定量的分析和比较。本书突出了抽象数据类型的概念。对每一种数据结构，都分别给出相应的抽象数据类型规范说明和实现方法。

全书中采用类 C 语言作为数据结构和算法的描述语言，在对数据的存储结构和算法进行描述时，尽量考虑 C 语言的特色，同时兼顾数据结构和算法的可读性。学生在实际上机操作时，可以很容易地将本书中的数据结构和算法转换成 C 或 C++ 程序。

为方便教师教学和学生学习，本书还提供了 PPT 教学课件、习题答案、源代码、教学大纲、实验指导，以及算法的 Flash 动态演示。读者可从人民邮电出版社教学服务和资源网（[www.ptpedu.com.cn](http://www.ptpedu.com.cn)）上免费下载。

本书可作为普通高等院校计算机和信息技术相关专业“数据结构”课程的教材使用，也可供从事计算机工程与应用工作的科技工作者参考。

---

---

---

---

在本书的编写过程中，北京林业大学信息学院的钟复之、陈忠富、金鑫、王岩琪、王帅、陈婷、朱秋昱、胡健行等同学参加了有关程序的调试工作和文字校对工作，苏翔、冯宁、侯丹、毕涛等同学参加了算法的 Flash 动态演示制作工作，在此表示衷心的感谢！

因编者水平有限，书中错误在所难免，恳请批评指正。

编 者

2014 年 11 月

# 目 录

|                              |                               |
|------------------------------|-------------------------------|
| <b>第1章 绪论</b> .....1         | 2.8 案例分析与实现.....45            |
| 1.1 数据结构的研究内容.....1          | 2.9 小结.....50                 |
| 1.2 基本概念和术语.....3            | 习题.....51                     |
| 1.2.1 数据、数据元素、数据项和数据对象.....3 | <b>第3章 栈和队列</b> .....54       |
| 1.2.2 数据结构.....4             | 3.1 栈和队列的定义和特点.....54         |
| 1.2.3 数据类型和抽象数据类型.....6      | 3.1.1 栈的定义和特点.....54          |
| 1.3 抽象数据类型的表示与实现.....7       | 3.1.2 队列的定义和特点.....55         |
| 1.4 算法和算法分析.....10           | 3.2 案例引入.....55               |
| 1.4.1 算法的定义及特性.....11        | 3.3 栈的表示和操作的实现.....56         |
| 1.4.2 评价算法优劣的基本标准.....11     | 3.3.1 栈的类型定义.....56           |
| 1.4.3 算法的时间复杂度.....11        | 3.3.2 顺序栈的表示和实现.....57        |
| 1.4.4 算法的空间复杂度.....15        | 3.3.3 链栈的表示和实现.....59         |
| 1.5 小结.....16                | 3.4 栈与递归.....61               |
| 习题.....16                    | 3.4.1 采用递归算法解决的问题.....61      |
| <b>第2章 线性表</b> .....19       | 3.4.2 递归过程与递归工作栈.....65       |
| 2.1 线性表的定义和特点.....19         | 3.4.3 递归算法的效率分析.....67        |
| 2.2 案例引入.....19              | 3.4.4 利用栈将递归转换为非递归的方法.....68  |
| 2.3 线性表的类型定义.....21          | 3.5 队列的表示和操作的实现.....69        |
| 2.4 线性表的顺序表示和实现.....23       | 3.5.1 队列的类型定义.....69          |
| 2.4.1 线性表的顺序存储表示.....23      | 3.5.2 循环队列——队列的顺序表示和实现.....69 |
| 2.4.2 顺序表中基本操作的实现.....25     | 3.5.3 链队——队列的链式表示和实现.....73   |
| 2.5 线性表的链式表示和实现.....29       | 3.6 案例分析与实现.....75            |
| 2.5.1 单链表的定义和表示.....29       | 3.7 小结.....82                 |
| 2.5.2 单链表基本操作的实现.....32      | 习题.....83                     |
| 2.5.3 循环链表.....38            | <b>第4章 串、数组和广义表</b> .....87   |
| 2.5.4 双向链表.....39            | 4.1 串的定义.....87               |
| 2.6 顺序表和链表的比较.....41         | 4.2 案例引入.....88               |
| 2.6.1 空间性能的比较.....41         | 4.3 串的类型定义、存储结构及其运算.....89    |
| 2.6.2 时间性能的比较.....41         | 4.3.1 串的抽象类型定义.....89         |
| 2.7 线性表的应用.....42            | 4.3.2 串的存储结构.....90           |
| 2.7.1 线性表的合并.....42          | 4.3.3 串的模式匹配算法.....91         |
| 2.7.2 有序表的合并.....43          |                               |

|                         |            |                      |            |
|-------------------------|------------|----------------------|------------|
| 4.4 数组.....             | 98         | 6.2 案例引入.....        | 151        |
| 4.4.1 数组的类型定义.....      | 98         | 6.3 图的类型定义.....      | 152        |
| 4.4.2 数组的顺序存储.....      | 99         | 6.4 图的存储结构.....      | 153        |
| 4.4.3 特殊矩阵的压缩存储.....    | 100        | 6.4.1 邻接矩阵.....      | 153        |
| 4.5 广义表.....            | 102        | 6.4.2 邻接表.....       | 155        |
| 4.5.1 广义表的定义.....       | 102        | 6.4.3 十字链表.....      | 158        |
| 4.5.2 广义表的存储结构.....     | 103        | 6.4.4 邻接多重表.....     | 159        |
| 4.6 案例分析与实现.....        | 105        | 6.5 图的遍历.....        | 160        |
| 4.7 小结.....             | 107        | 6.5.1 深度优先搜索.....    | 161        |
| 习题.....                 | 107        | 6.5.2 广度优先搜索.....    | 163        |
| <b>第 5 章 树和二叉树.....</b> | <b>111</b> | 6.6 图的应用.....        | 165        |
| 5.1 树和二叉树的定义.....       | 111        | 6.6.1 最小生成树.....     | 165        |
| 5.1.1 树的定义.....         | 111        | 6.6.2 最短路径.....      | 170        |
| 5.1.2 树的基本术语.....       | 112        | 6.6.3 拓扑排序.....      | 176        |
| 5.1.3 二叉树的定义.....       | 113        | 6.6.4 关键路径.....      | 178        |
| 5.2 案例引入.....           | 114        | 6.7 案例分析与实现.....     | 184        |
| 5.3 树和二叉树的抽象数据类型定义..... | 115        | 6.8 小结.....          | 185        |
| 5.4 二叉树的性质和存储结构.....    | 118        | 习题.....              | 187        |
| 5.4.1 二叉树的性质.....       | 118        | <b>第 7 章 查找.....</b> | <b>191</b> |
| 5.4.2 二叉树的存储结构.....     | 120        | 7.1 查找的基本概念.....     | 191        |
| 5.5 遍历二叉树和线索二叉树.....    | 121        | 7.2 线性表的查找.....      | 192        |
| 5.5.1 遍历二叉树.....        | 122        | 7.2.1 顺序查找.....      | 192        |
| 5.5.2 线索二叉树.....        | 128        | 7.2.2 折半查找.....      | 193        |
| 5.6 树和森林.....           | 132        | 7.2.3 分块查找.....      | 197        |
| 5.6.1 树的存储结构.....       | 133        | 7.3 树表的查找.....       | 198        |
| 5.6.2 森林与二叉树的转换.....    | 134        | 7.3.1 二叉排序树.....     | 198        |
| 5.6.3 树和森林的遍历.....      | 135        | 7.3.2 平衡二叉树.....     | 205        |
| 5.7 哈夫曼树及其应用.....       | 136        | 7.3.3 B-树.....       | 210        |
| 5.7.1 哈夫曼树的基本概念.....    | 136        | 7.3.4 B+ 树.....      | 218        |
| 5.7.2 哈夫曼树的构造算法.....    | 137        | 7.4 散列表的查找.....      | 219        |
| 5.7.3 哈夫曼编码.....        | 140        | 7.4.1 散列表的基本概念.....  | 219        |
| 5.8 案例分析与实现.....        | 143        | 7.4.2 散列函数的构造方法..... | 221        |
| 5.9 小结.....             | 145        | 7.4.3 处理冲突的方法.....   | 223        |
| 习题.....                 | 146        | 7.4.4 散列表的查找.....    | 225        |
| <b>第 6 章 图.....</b>     | <b>149</b> | 7.5 小结.....          | 228        |
| 6.1 图的定义和基本术语.....      | 149        | 习题.....              | 230        |
| 6.1.1 图的定义.....         | 149        | <b>第 8 章 排序.....</b> | <b>234</b> |
| 6.1.2 图的基本术语.....       | 149        | 8.1 基本概念和排序方法概述..... | 234        |

|                        |     |                      |     |
|------------------------|-----|----------------------|-----|
| 8.1.1 排序的基本概念.....     | 234 | 8.4.2 树形选择排序.....    | 248 |
| 8.1.2 内部排序方法的分类.....   | 235 | 8.4.3 堆排序.....       | 249 |
| 8.1.3 待排序记录的存储方式.....  | 235 | 8.5 归并排序.....        | 254 |
| 8.1.4 排序算法效率的评价指标..... | 236 | 8.6 基数排序.....        | 256 |
| 8.2 插入排序.....          | 236 | 8.6.1 多关键字的排序.....   | 256 |
| 8.2.1 直接插入排序.....      | 236 | 8.6.2 链式基数排序.....    | 257 |
| 8.2.2 折半插入排序.....      | 238 | 8.7 外部排序.....        | 260 |
| 8.2.3 希尔排序.....        | 239 | 8.7.1 外部排序的基本方法..... | 260 |
| 8.3 交换排序.....          | 241 | 8.7.2 多路平衡归并的实现..... | 261 |
| 8.3.1 冒泡排序.....        | 241 | 8.7.3 置换-选择排序.....   | 263 |
| 8.3.2 快速排序.....        | 243 | 8.7.4 最佳归并树.....     | 266 |
| 8.4 选择排序.....          | 246 | 8.8 小结.....          | 267 |
| 8.4.1 简单选择排序.....      | 246 | 习题.....              | 269 |

# 第 1 章

## 绪论

早期的计算机主要用于数值计算，现在，计算机主要用于非数值计算，包括处理字符、表格和图像等具有一定结构的数据。这些数据内容存在着某种联系，只有分清楚数据的内在联系，合理地组织数据，才能对它们进行有效的处理，设计出高效的算法。如何合理地组织数据、高效地处理数据，这就是“数据结构”主要研究的问题。本章简要介绍有关数据结构的基本概念和算法分析方法。

### 1.1 数据结构的研究内容

计算机主要用于数值计算时，一般要经过如下几个步骤：首先从具体问题抽象出数学模型，然后设计一个解此数学模型的算法，最后编写程序，进行测试、调试，直到解决问题。在此过程中寻求数学模型的实质是分析问题，从中提取操作的对象，并找出这些操作对象之间的关系，然后用数学语言加以描述，即建立相应的数学方程。例如，用计算机进行全球天气预报时，就需要求解一组球面坐标系下的二阶椭圆偏微分方程；预测人口增长情况的数学模型为常微分方程。求解这些数学方程的算法是计算数学研究的范畴，如高斯消元法、差分法、有限元法等算法。数据结构主要研究非数值计算问题，非数值计算问题无法用数学方程建立数学模型，下面通过三个实例加以说明。

#### 【例 1.1】 学生学籍管理系统。

高等院校教务处使用计算机对全校的学生情况作统一管理。学生的基本信息，包括学生的学号、姓名、性别、籍贯、专业等，如表 1.1 所示。每个学生的基本情况按照不同的顺序号，依次存放在“学生基本信息表”中，根据需要对这张表进行查找。每个学生的基本信息记录按顺序号排列，形成了学生基本信息记录的线性序列，呈一种线性关系。

表 1.1 学生基本信息表

| 学号        | 姓名  | 性别 | 籍贯 | 专业       |
|-----------|-----|----|----|----------|
| 060214201 | 杨阳  | 男  | 安徽 | 计算机科学与技术 |
| 060214202 | 薛林  | 男  | 福建 | 计算机科学与技术 |
| 060214215 | 王诗萌 | 女  | 吉林 | 计算机科学与技术 |
| 060214216 | 冯子晗 | 女  | 山东 | 计算机科学与技术 |

诸如此类的线性表结构还有图书馆的书目管理系统、库存管理系统等。在这类问题中，计算

机处理的对象是各种表, 元素之间存在简单一对一的线性关系, 因此这类问题的数学模型就是各种线性表, 施加于对象上的操作有查找、插入和删除等。这类数学模型称为“线性”的数据结构。

### 【例 1.2】 人机对弈问题。

计算机之所以能和人弈是因为已经将弈的策略在计算机中存储好。由于弈的过程是在一定规则下随机进行的, 所以, 为使计算机能灵活弈, 就必须把弈过程中所有可能发生的情况及相应的对策都加以考虑。以最简单的井字棋为例, 初始状态是一个空的棋盘格局。对弈开始后, 每下一步棋, 则构成一个新的棋盘格局, 且相对于上一个棋盘格局的可能选择可以有多种形式, 因而整个对弈过程就如同图 1.1 所示的“一棵倒长的树”。在这棵“树”中, 从初始状态(根)到某一最终格局(叶子)的一条路径, 就是一次具体的对弈过程。

人机对弈问题的数学模型就是如何用树结构表示棋盘和棋子等, 算法是博弈的规则和策略。诸如此类的树结构还有计算机的文件系统、一个单位的组织机构等。在这类问题中, 计算机处理的对象是树结构, 元素之间是一对多的层次关系, 施加于对象上的操作有查找、插入和删除等。这类数学模型称为“树”的数据结构。

### 【例 1.3】 最短路径问题。

从城市 A 到城市 B 有多条线路, 但每条线路的交通费不同, 那么, 如何选择一条线路, 使得从城市 A 到城市 B 的交通费用最少呢? 解决的方法是, 可以把这类问题抽象为图的最短路径问题。如图 1.2 所示, 图中的顶点代表城市, 有向边代表两个城市之间的通路, 边上的权值代表两个城市之间的交通费。求解 A 到 B 的最少交通费用, 就是要在有向图中 A 点(源点)到达 B 点(终点)的多条路径中, 寻找一条各边权值之和最小的路径, 即最短路径。

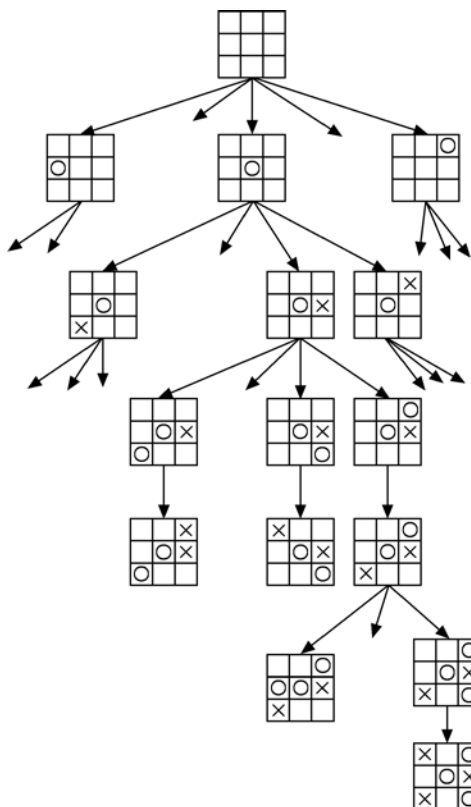


图 1.1 井字棋的对弈树

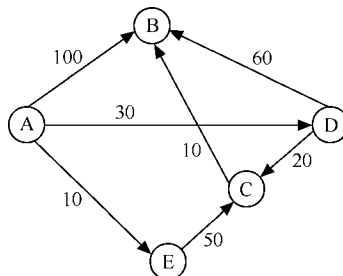


图 1.2 最短路径问题

最短路径问题的数学模型就是图结构，算法是求解两点之间的最短路径。诸如此类的图结构还有网络工程图和网络通信图等，在这类问题中，元素之间是多对多的网状关系，施加于对象上的操作依然有查找、插入和删除等。这类数学模型称为“图”的数据结构。

从上面三个实例可以看出，非数值计算问题的数学模型不再是数学方程，而是诸如线性表、树和图的数据结构。因此，简单地说，数据结构是一门研究非数值计算程序设计中的操作对象，以及这些对象之间的关系和操作的学科。

20 世纪 60 年代初期，“数据结构”有关的内容散见于操作系统、编译原理等课程中。1968 年，“数据结构”作为一门独立的课程被列入美国一些大学计算机科学系的教学计划，同年，著名计算机科学家 D.E.Knuth 教授发表了《计算机程序设计艺术》第一卷《基本算法》。这是第一本较系统地阐述“数据结构”基本内容的著作。之后，随着大型程序和大规模文件系统的出现，结构化程序设计成为程序设计方法学的主要研究方向，人们普遍认为程序设计的实质就是对所处理的问题选择一种好的数据结构，并在此结构基础上施加一种好的算法，著名科学家 Wirth 教授的《算法+数据结构=程序》正是这种观点的集中体现。

目前，数据结构在计算机科学中是一门综合性的专业基础课。数据结构的研究不仅涉及计算机硬件（特别是编码理论、存储装置和存取方法等）的研究范围，而且和计算机软件的研究有着密切的关系，无论是编译程序还是操作系统都涉及数据元素在存储器中的分配问题。在研究信息检索时也必须考虑如何组织数据，以使查找和存取数据元素更为方便。因此，可以认为数据结构是介于数学、计算机硬件和软件三者之间的一门核心课程。

有关“数据结构”的研究仍不断发展，一方面，面向各专门领域中特殊问题的数据结构正在研究和发展；另一方面，从抽象数据类型的观点来讨论数据结构，已成为一种新的趋势，越来越被人们所重视。

## 1.2 基本概念和术语

下列概念和术语将在以后各章节中多次出现，本节先对这些概念和术语赋予确定的含义。

### 1.2.1 数据、数据元素、数据项和数据对象

**数据 (Data)** 是客观事物的符号表示，是所有能输入到计算机中并被计算机程序处理的符号的总称。如数学计算中用到的整数和实数，文本编辑中用到的字符串，多媒体程序处理的图形、图像、声音及动画等通过特殊编码定义后的数据。

**数据元素 (Data Element)** 是数据的基本单位，在计算机中通常作为一个整体进行考虑和处理。在有些情况下，数据元素也称为元素、记录等。数据元素用于完整地描述一个对象，如前一节示例中的一名学生记录，树中棋盘的一个格局（状态），以及图中的一个顶点等。

**数据项 (Data Item)** 是组成数据元素的、有独立含义的、不可分割的最小单位。例如，学生基本信息表中的学号、姓名、性别等都是数据项。

**数据对象 (Data Object)** 是性质相同的数据元素的集合，是数据的一个子集。例如：整数数据对象是集合  $N = \{0, \pm 1, \pm 2, \dots\}$ ，字母字符数据对象是集合  $C = \{ 'A', 'B', \dots, 'Z', 'a', 'b', \dots, 'z' \}$ ，学生基本信息表也可以是一个数据对象。由此可以看出，不论数据元素集合是无限集（如整数集），或是有限集（如字母字符集），还是由多个数据项组成的复合数据元素（如学生表）的

集合，只要集合内元素的性质均相同，都可称之为一个数据对象。

## 1.2.2 数据结构

**数据结构** (Data Structure) 是相互之间存在一种或多种特定关系的数据元素的集合。换句话说，数据结构是带“结构”的数据元素的集合，“结构”就是指数据元素之间存在的关系。

数据结构包括逻辑结构和存储结构两个层次。

### 1. 逻辑结构

数据的**逻辑结构**是从逻辑关系上描述数据，它与数据的存储无关，是独立于计算机的。因此，数据的逻辑结构可以看作是从具体问题抽象出来的数学模型。

数据的逻辑结构有两个要素：一是数据元素；二是关系。数据元素的含义如前所述，关系是指数据元素间的逻辑关系。根据数据元素之间关系的不同特性，通常有四类基本结构，如图 1.3 所示。它们的复杂程度依次递进。

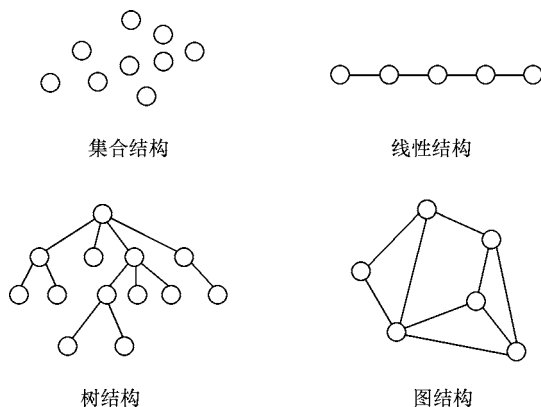


图 1.3 四类基本逻辑结构关系图

下面四种结构中所举的示例是以某班级学生作为数据对象（数据元素是学生的学籍档案记录），来分别考察数据元素之间的关系。

#### (1) 集合结构

数据元素之间除了“属于同一集合”的关系外，别无其他关系。例如，确定一名学生是否为班级成员，只需将班级看做一个集合结构。

#### (2) 线性结构

数据元素之间存在一对一的关系。例如，将学生信息数据按照其入学报到的时间先后顺序进行排列，将组成一个线性结构。

#### (3) 树结构

数据元素之间存在一对多的关系。例如，在班级的管理体系中，班长管理多个组长，每位组长管理多名组员，从而构成树形结构。

#### (4) 图结构或网状结构

数据元素之间存在多对多的关系。例如，多位同学之间的朋友关系，任何两位同学都可以是朋友，从而构成图状结构或网状结构。

其中集合结构、树结构和图结构都属于非线性结构。

线性结构包括线性表（典型的线性结构，如例 1.1 中的学生基本信息表）、栈和队列（具有特

殊限制的线性表，数据操作只能在表的一端或两端进行)、字符串(也是特殊的线性表，其特殊性表现在它的数据元素仅由一个字符组成)、数组(是线性表的推广，它的数据元素是一个线性表)、广义表(也是线性表的推广，它的数据元素是一个线性表，但不同构，即或者是单元素，或者是线性表)。非线性结构包括树(具有多个分支的层次结构)和二叉树(具有两个分支的层次结构)、有向图(一种图结构，边是顶点的有序对)和无向图(另一种图结构，边是顶点的无序对)。这几种逻辑结构可以用一个层次图描述，如图 1.4 所示。

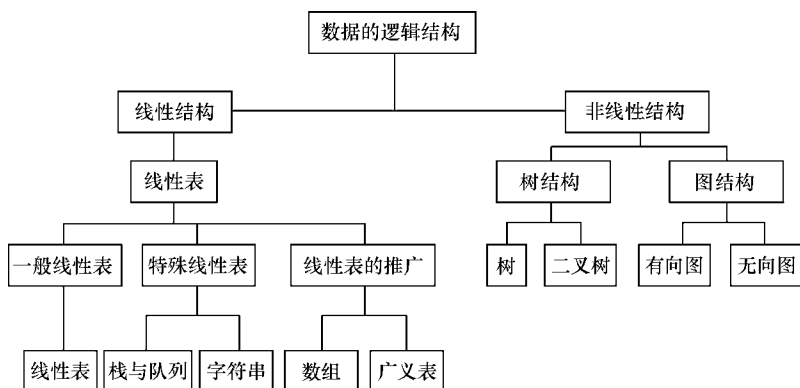


图 1.4 几种逻辑结构层次图

## 2. 存储结构

数据对象在计算机中的存储表示称为数据的**存储结构**，也称为**物理结构**。把数据对象存储到计算机时，通常要求既要存储各数据元素的数据，又要存储数据元素之间的逻辑关系，数据元素在计算机内用一个结点来表示。数据元素在计算机中有两种基本的存储结构，分别是顺序存储结构和链式存储结构。

### (1) 顺序存储结构

顺序存储结构是借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系，通常借助程序设计语言的数组类型来描述。

对于前面的“学生基本信息表”，假定每个结点(学生记录)占用 50 个存储单元，数据从 0 号单元开始由低地址向高地址方向存储，对应的顺序存储结构如表 1.2 所示。

表 1.2 顺序存储结构

| 地址  | 学号        | 姓名  | 性别 | 籍贯 | 专业       |
|-----|-----------|-----|----|----|----------|
| 0   | 060214201 | 杨阳  | 男  | 安徽 | 计算机科学与技术 |
| 50  | 060214202 | 薛林  | 男  | 福建 | 计算机科学与技术 |
| 100 | 060214215 | 王诗萌 | 女  | 吉林 | 计算机科学与技术 |
| 150 | 060214216 | 冯子晗 | 女  | 山东 | 计算机科学与技术 |

### (2) 链式存储结构

顺序存储结构要求所有的元素依次存放在一片连续的存储空间中，而链式存储结构，无需占用一整块存储空间。但为了表示结点之间的关系，需要给每个结点附加指针字段，用于存放后继元素的存储地址。所以链式存储结构通常借助于程序设计语言的指针类型来描述。

假定给前面的“学生基本信息表”中的每个结点附加一个“下一个结点地址”，即后继指针字

段，用于存放后继结点的首地址，则可得到如表 1.3 所示的链式存储结构。从表中可以看出，每个结点占用两个连续的存储单元，一个存放结点的信息，另一个存放后继结点的首地址。

表 1.3 链式存储结构

| 地址  | 学号        | 姓名  | 性别 | 籍贯 | 专业       | 后继结点的首地址 |
|-----|-----------|-----|----|----|----------|----------|
| 0   | 060214201 | 杨阳  | 男  | 安徽 | 计算机科学与技术 | 100      |
| 50  | 060214216 | 冯子晗 | 女  | 山东 | 计算机科学与技术 | ^        |
| 100 | 060214202 | 薛林  | 男  | 福建 | 计算机科学与技术 | 150      |
| 150 | 060214215 | 王诗萌 | 女  | 吉林 | 计算机科学与技术 | 50       |

为了更清楚地反映链式存储结构，可采用更直观的图示来表示，如“学生基本信息表”的链式存储结构可用如图 1.5 所示的方式表示。



图 1.5 链式存储结构示意图

## 1.2.3 数据类型和抽象数据类型

### 1. 数据类型

**数据类型**（Data Type）是高级程序设计语言中的一个基本概念，前面提到过顺序存储结构可以借助程序设计语言的数组类型描述，链式存储结构可以借助指针类型描述，所以数据类型和数据结构的概念密切相关。

一方面，在程序设计语言中，每一个数据都属于某种数据类型。类型明显或隐含地规定了数据的取值范围、存储方式以及允许进行的运算，**数据类型**是一个值的集合和定义在这个值集上的一组操作的总称。例如，C 语言中的整型变量，其值集为某个区间上的整数（区间大小依赖于不同的机器），定义在其上的操作为加、减、乘、除和取模等算术运算；而实型变量也有自己的取值范围和相应运算，比如取模运算是不能用于实型变量的。程序设计语言允许用户直接使用的数据类型由具体语言决定，数据类型反映了程序设计语言的数据描述和处理能力。C 语言除了提供整型、实型、字符型等基本类型数据外，还允许用户自定义各种类型数据，例如数组、结构体和指针等。

### 2. 抽象数据类型

抽象就是抽取出实际问题的本质。在计算机中使用二进制数来表示数据，在汇编语言中则可给出各种数据的十进制表示，它们是二进制数据的抽象，使用者在编程时可以直接使用，不必考虑实现细节。在高级语言中，则给出更高一级的数据抽象，出现了数据类型，如整型、实型、字符型等，可以进一步利用这些类型构造出线性表、栈、队列、树、图等复杂的抽象数据类型。

**抽象数据类型**（Abstract Data Type, ADT）一般指由用户定义的、表示应用问题的数学模型，

以及定义在这个模型上的一组操作的总称，具体包括三部分：数据对象、数据对象上关系的集合以及对数据对象的基本操作的集合。

抽象数据类型的定义格式如下：

```
ADT 抽象数据类型名{
    数据对象:〈数据对象的定义〉
    数据关系:〈数据关系的定义〉
    基本操作:〈基本操作的定义〉
}ADT 抽象数据类型名
```

其中，数据对象和数据关系的定义采用数学符号和自然语言描述，基本操作的定义格式为：

```
基本操作名 (参数表)
    初始条件:〈初始条件描述〉
    操作结果:〈操作结果描述〉
```

基本操作有两种参数：赋值参数只为操作提供输入值；引用参数以“&”打头，除可提供输入值外，还将返回操作结果。“初始条件”描述了操作执行之前数据结构和参数应满足的条件，若初始条件为空，则省略。“操作结果”说明了操作正常完成之后，数据结构的变化状况和应返回的结果。

## 1.3 抽象数据类型的表示与实现

运用抽象数据类型描述数据结构，有助于在设计一个软件系统时，不必首先考虑其中包含的数据对象，以及操作在不同处理器中的表示和实现细节，而是在构成软件系统的每个相对独立的模块上定义一组数据和相应的操作，把这些数据的表示和操作细节留在模块内部解决，在更高的层次上进行软件的分析 and 设计，从而提高软件的整体性能和利用率。

抽象数据类型的概念与面向对象方法的思想是一致的。抽象数据类型独立于具体实现，将数据和操作封装在一起，使得用户程序只能通过抽象数据类型定义的某些操作来访问其中的数据，从而实现了信息隐藏。在 C++ 中，我们可以用类的声明表示抽象数据类型，用类的实现来实现抽象数据类型。因此，C++ 中实现的类相当于数据的存储结构及其在存储结构上实现的对数据的操作。

抽象数据类型和类的概念实际上反映了程序或软件设计的两层抽象：抽象数据类型相当于在概念层（或称为抽象层）上描述问题，而类相当于在实现层上描述问题。此外，C++ 中的类只是一个由用户定义的普通类型，可用它来定义变量（称为对象或类的实例）。因此，在 C++ 中，最终是通过操作对象来解决实际问题的，所以我们将该层次看做是应用层。例如，main 程序就可看做是用户的应用程序。

由此可以看出，最终表示和实现抽象数据类型，最好用面向对象的方法，比如用 C++ 语言的类描述比较方便、有效，但本课程大都在大学低年级开设，用 C 语言的描述方法更符合学生的实际情况。另外，由于实际问题千变万化，数据模型和算法也形形色色，因此抽象数据类型的设计和实现，就不可能像基本数据类型那样规范和一劳永逸。本书所讨论的数据结构及其算法主要是面向读者的，故采用介于伪码和 C 语言之间的类 C 语言作为描述工具。这使得数据结构与算法的

描述与讨论简明清晰,不拘泥于 C 语言的细节,又容易转换成 C 或 C++ 程序。

本书采用的类 C 语言精选了 C 语言的一个核心子集,同时做了若干扩充修改,增强了语言的描述功能,以下对其做简要说明。

#### (1) 预定义常量及类型:

```
//函数结果状态代码
#define OK 1
#define ERROR 0
#define OVERFLOW -2
//Status 是函数返回值类型,其值是函数结果状态代码。
typedef int Status;
```

(2) 数据结构的表示(存储结构)用类型定义 typedef 描述;数据元素类型约定为 ElemType,由用户在使用该数据类型时自行定义。

#### (3) 基本操作的算法都用如下格式的函数来描述:

```
函数类型 函数名(函数参数表)
{
    //算法说明
    语句序列
} //函数名
```

当函数返回值为函数结果状态代码时,函数定义为 Status 类型。为了便于描述算法,除了值调用方式外,增加了 C++ 语言引用调用的参数传递方式。在形参表中,以“&”打头的参数即为引用参数。传递引用给函数与传递指针的效果是一样的,形参变化实参也发生变化,但引用使用起来比指针更加方便、高效。

#### (4) 内存的动态分配与释放。

使用 new 和 delete 动态分配和释放内存空间:

```
分配空间  指针变量=new 数据类型;
释放空间  delete 指针变量;
```

#### (5) 赋值语句:

```
简单赋值  变量名 = 表达式;
串联赋值  变量名 1 = 变量名 2 = ... = 变量名 n = 表达式;
成组赋值  (变量名 1, ..., 变量名 n) = (表达式 1, ..., 表达式 n);
结构赋值  结构名 1 = 结构名 2;
           结构名 = (值 1, 值 2, ..., 值 n);
条件赋值  变量名 = 条件表达式 ? 表达式 T: 表达式 F;
交换赋值  变量名 1 <-> 变量名 2;
```

#### (6) 选择语句:

```
条件语句 1 if (表达式) 语句;
条件语句 2 if (表达式) 语句;
           else 语句;
开关语句 switch (表达式)
{
    case 值 1: 语句序列 1 ;break;
    case 值 2: 语句序列 2 ;break;
```

```

...
case 值  $n$ : 语句序列  $n$ ; break;
default: 语句序列  $n+1$ ;
}

```

(7) 循环语句:

```

for 语句      for (表达式 1; 条件; 表达式 2) 语句;
while 语句    while (条件) 语句;
do-while 语句 do {
                语句序列;
            } while (条件);

```

(8) 结束语句:

```

函数结束语句  return 表达式;
                return;
case 或循环结束语句 break;
异常结束语句 exit (异常代码);

```

(9) 输入输出语句使用 C++ 流式输入输出的形式:

```

输入语句      cin >> 变量 1 >> ... >> 变量  $n$ ;
输出语句      cout << 表达式 1 << ... << 表达式  $n$ ;

```

(10) 基本函数:

```

求最大值      Max (表达式 1, ..., 表达式  $n$ )
求最小值      Min (表达式 1, ..., 表达式  $n$ )

```

下面以复数为例, 给出一个完整的抽象数据类型的定义、表示和实现。

(1) 定义部分:

```

ADT Complex {
    数据对象:  $D = \{e1, e2 \mid e1, e2 \in \mathbb{R}, \mathbb{R} \text{ 是实数集}\}$ 
    数据关系:  $S = \{ \langle e1, e2 \rangle \mid e1 \text{ 是复数的实部, } e2 \text{ 是复数的虚部} \}$ 
    基本操作:
        Creat( &C, x, y )
            操作结果: 构造复数 C, 其实部和虚部分别被赋以参数 x 和 y 的值。
        GetReal(C)
            初始条件: 复数 C 已存在。
            操作结果: 返回复数 C 的实部值。
        GetImag(C)
            初始条件: 复数 C 已存在。
            操作结果: 返回复数 C 的虚部值。
        Add( C1, C2 )
            初始条件: C1, C2 是复数。
            操作结果: 返回两个复数 C1 和 C2 的和。
        Sub( C1, C2 )
            初始条件: C1, C2 是复数。
            操作结果: 返回两个复数 C1 和 C2 的差。
} ADT Complex

```

在后面的章节中, 每定义一个新的数据结构, 都先用这种定义方式给出其抽象数据类型的定

义, 对于数据结构的表示方法, 则根据不同的存储结构相应给出, 同时用类 C 语言给出主要操作的实现方法。下面为了让读者对抽象数据类型有一个完整、正确的理解, 给出复数的存储表示和相应操作的具体实现过程。

(2) 表示部分:

```
typedef struct          //复数类型
{
    float Realpart;      //实部
    float Imagepart;     //虚部
}Complex;
```

(3) 实现部分:

```
void Create( &Complex C, float x, float y)
{ //构造一个复数
    C.Realpart=x;
    C.Imagepart=y;
}
float GetReal(Complex C)
{ //取复数 C=x+yi 的实部
    return C.Realpart;
}
float GetImag(Complex C)
{ //取复数 C=x+yi 的虚部
    return C.Imagepart;
}
Complex Add(Complex C1, Complex C2)
{ //求两个复数 C1 和 C2 的和 sum
    Complex sum;
    sum.Realpart=C1.Realpart+C2.Realpart;
    sum.Imagepart=C1.Imagepart+C2.Imagepart;
    return sum;
}
Complex Sub(Complex C1, Complex C2)
{ //求两个复数 C1 和 C2 的差 difference
    Complex difference;
    difference.Realpart=C1.Realpart-C2.Realpart;
    difference.Imagepart=C1.Imagepart-C2.Imagepart;
    return difference;
}
```

这样定义之后, 就可以在主程序中通过调用 Create 函数构造一个复数, 调用 Add 或 Sub 函数实现复数的加法或减法运算, 用户可以像使用整数类型那样使用复数类型了。通过上述实例, 读者可以对抽象数据类型的概念有更加深刻的理解。

## 1.4 算法和算法分析

数据结构与算法之间存在着本质联系, 在某一类型数据结构上, 总要涉及其上施加的运算, 而只有通过对所定义运算的研究, 才能清楚理解数据结构的定义和作用; 在涉及运算时, 总要联系到该算法处理的对象和结果的数据。在“数据结构”中, 将遇到大量的算法问题, 因为算法联

系着数据在计算过程中的组织方式，为了描述实现某种操作，常常需要设计算法，因而算法是研究数据结构的重要途径。

### 1.4.1 算法的定义及特性

**算法**（Algorithm）是为了解决某类问题而规定的一个有限长的操作序列。

一个算法必须满足以下五个重要特性。

- （1）**有穷性**。一个算法必须总是在执行有穷步后结束，且每一步都必须在有穷时间内完成。
- （2）**确定性**。对于每种情况下所应执行的操作，在算法中都有确切的规定，不会产生二义性，使算法的执行者或阅读者都能明确其含义及如何执行。
- （3）**可行性**。算法中的所有操作都可以通过已经实现的基本操作运算执行有限次来实现。
- （4）**输入**。一个算法有零个或多个输入。当用函数描述算法时，输入往往是通过形参表示的，在它们被调用时，从主调函数获得输入值。
- （5）**输出**。一个算法有一个或多个输出，它们是算法进行信息加工后得到的结果，无输出的算法没有任何意义。当用函数描述算法时，输出多用返回值或引用类型的形参表示。

### 1.4.2 评价算法优劣的基本标准

一个算法的优劣应该从以下几方面来评价。

- （1）**正确性**。在合理的数据输入下，能够在有限的运行时间内得到正确的结果。
- （2）**可读性**。一个好的算法，首先应便于人们理解和相互交流，其次才是机器可执行性。可读性强的算法有助于人们对算法的理解，而难懂的算法易于隐藏错误，且难于调试和修改。
- （3）**健壮性**。当输入的数据非法时，好的算法能适当地做出正确反应或进行相应处理，而不会产生一些莫名其妙的输出结果。
- （4）**高效性**。高效性包括时间和空间两个方面。时间高效是指算法设计合理，执行效率高，可以用时间复杂度来度量；空间高效是指算法占用存储容量合理，可以用空间复杂度来度量。时间复杂度和空间复杂度是衡量算法的两个主要指标。

### 1.4.3 算法的时间复杂度

算法效率分析的目的是看算法实际是否可行，并在同一问题存在多个算法时，可进行时间和空间性能上的比较，以便从中挑选出较优算法。

衡量算法效率的方法主要有两类：事后统计法和事前分析估算法。事后统计法需要先将算法实现，然后测算其时间和空间开销。这种方法的缺陷很显然，一是必须把算法转换成可执行的程序，二是时空开销的测算结果依赖于计算机的软硬件等环境因素，这容易掩盖算法本身的优劣。所以我们通常采用事前分析估算法，通过计算算法的渐近复杂度来衡量算法的效率。

#### 1. 问题规模和语句频度

不考虑计算机的软硬件等环境因素，影响算法时间代价的最主要因素是问题规模。**问题规模**是算法求解问题输入量的多少，是问题大小的本质表示，一般用整数  $n$  表示。问题规模  $n$  对不同的问题含义不同，例如，在排序运算中  $n$  为参加排序的记录数，在矩阵运算中  $n$  为矩阵的阶数，在多项式运算中  $n$  为多项式的项数，在集合运算中  $n$  为集合中元素的个数，在树的有关运算中  $n$  为树的结点个数，在图的有关运算中  $n$  为图的顶点数或边数。显然， $n$  越大算法的执行时间越长。

一个算法的执行时间大致上等于其所有语句执行时间的总和, 而语句的执行时间则为该条语句的重复执行次数和执行一次所需时间的乘积。

一条语句的重复执行次数称作**语句频度** (Frequency Count)。

由于语句的执行要由源程序经编译程序翻译成目标代码, 目标代码经装配再执行, 因此语句执行一次实际所需的具体时间是与机器的软、硬件环境 (如机器速度、编译程序质量等) 密切相关的。所以, 所谓的算法分析并非精确统计算法实际执行所需时间, 而是针对算法中语句的执行次数做出估计, 从中得到算法执行时间的信息。

设每条语句执行一次所需的时间均是单位时间, 则一个算法的执行时间可用该算法中所有语句频度之和来度量。

**【例 1.4】** 求两个  $n$  阶矩阵的乘积算法。

```
for(i=1;i<=n;i++)           //频度为 n+1
    for(j=1;j<=n;j++)       //频度为 n*(n+1)
    {
        c[i][j]=0;           //频度为 n^2
        for(k=1;k<=n;k++)    //频度为 n^2 * (n+1)
            c[i][j]=c[i][j]+a[i][k]*b[k][j]; //频度为 n^3
    }
```

该算法中所有语句频度之和, 是矩阵阶数  $n$  的函数, 用  $f(n)$  表示之。换句话说, 上例算法的执行时间与  $f(n)$  成正比。

$$f(n)=2n^3+3n^2+2n+1$$

## 2. 算法的时间复杂度定义

对于例 1.4 这种较简单的算法, 可以直接计算出算法中所有语句的频度, 但对于稍微复杂一些的算法, 则通常是比较困难的, 即便能够给出, 也可能是个非常复杂的函数。因此, 为了客观地反映一个算法的执行时间, 可以只用算法中的“基本语句”的执行次数来度量算法的工作量。所谓“基本语句”指的是算法中重复执行次数和算法的执行时间成正比的语句, 它对算法运行时间的贡献最大。通常, 算法的执行时间是随问题规模增长而增长的, 因此对算法的评价通常只需考虑其随问题规模增长的趋势。这种情况下, 我们只需要考虑当问题规模充分大时, 算法中基本语句的执行次数在渐近意义下的阶。如例 1.4 矩阵的乘积算法, 当  $n$  趋向无穷大时, 显然有

$$\lim_{n \rightarrow \infty} f(n)/n^3 = \lim_{n \rightarrow \infty} (2n^3 + 3n^2 + 2n + 1)/n^3 = 2$$

即当  $n$  充分大时,  $f(n)$  和  $n^3$  之比是一个不等于零的常数。即  $f(n)$  和  $n^3$  是同阶的, 或者说  $f(n)$  和  $n^3$  的数量级 (Order of Magnitude) 相同。在这里, 我们用“O”来表示数量级, 记作  $T(n) = O(f(n)) = O(n^3)$ 。由此我们可以给出下述算法时间复杂度的定义。

一般情况下, 算法中基本语句重复执行的次数是问题规模  $n$  的某个函数  $f(n)$ , 算法的时间量度记作

$$T(n) = O(f(n))$$

它表示随问题规模  $n$  的增大, 算法执行时间的增长率和  $f(n)$  的增长率相同, 称做算法的**渐近时间复杂度**, 简称**时间复杂度** (Time Complexity)。

数学符号“O”的严格定义为:

若  $T(n)$  和  $f(n)$  是定义在正整数集合上的两个函数, 则  $T(n) = O(f(n))$  表示存在正的常数  $C$  和  $n_0$ ,

使得当  $n \geq n_0$  时都满足  $0 \leq T(n) \leq Cf(n)$ 。

该定义说明了函数  $T(n)$  和  $f(n)$  具有相同的增长趋势, 并且  $T(n)$  的增长至多趋向于函数  $f(n)$  的增长。符号“O”用来描述增长率的上限, 它表示当问题规模  $n > n_0$  时, 算法的执行时间不会超过  $f(n)$ , 其直观的含义如图 1.6 所示。

### 3. 算法的时间复杂度分析举例

分析算法时间复杂度的基本方法为: 找出所有语句中语句频度最大的那条语句作为基本语句, 计算基本语句的频度得到问题规模  $n$  的某个函数  $f(n)$ , 取其数量级用符号“O”表示即可。具体计算数量级时, 可以遵循以下定理。

**定理 1.1** 若  $f(n) = a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$  是一个  $m$  次多项式, 则  $T(n) = O(n^m)$ 。

定理 1.1 说明, 在计算算法时间复杂度时, 可以忽略所有低次幂项和最高次幂的系数, 这样可以简化算法分析, 也体现出了增长率的含义。

若算法可用递归方法描述, 则算法的时间复杂度通常可使用递归方程表示, 此时将涉及递归方程求解问题。有关递归算法的时间复杂度分析方法将在第 3 章给出。

下面举例说明如何求非递归算法的时间复杂度。

#### 【例 1.5】常量阶示例。

```
{x++; s=0;}
```

两条语句频度均为 1, 算法的执行时间是一个与问题规模  $n$  无关的常数, 所以算法的时间复杂度为  $T(n) = O(1)$ , 称为常量阶。

实际上, 如果算法的执行时间不随问题规模  $n$  的增加而增长, 算法中语句频度就是某个常数。即使这个常数再大, 算法的时间复杂度都是  $O(1)$ 。例如, 上面的程序作如下改动:

```
for(i=0; i<10000; i++){x++; s=0;}
```

算法的时间复杂度仍然为  $O(1)$ 。

#### 【例 1.6】线性阶示例。

```
for(i=0; i<n; i++){x++; s=0;}
```

循环体内两条基本语句的频度均为  $f(n) = n$ , 所以算法的时间复杂度为  $T(n) = O(n)$ , 称为线性阶。

#### 【例 1.7】平方阶示例。

```
(1) x=0; y=0;
(2) for(k=1; k<=n; k++)
(3)     x++;
(4) for(i=1; i<=n; i++)
(5)     for(j=1; j<=n; j++)
(6)         y++;
```

对循环语句只需考虑循环体中语句的执行次数, 以上程序段中频度最大的语句是 (6), 其频度为  $f(n) = n^2$ , 所以该算法的时间复杂度为  $T(n) = O(n^2)$ , 称为平方阶。

多数情况下, 当有若干个循环语句时, 算法的时间复杂度是由最深层循环内的基本语句的频度  $f(n)$  决定的。

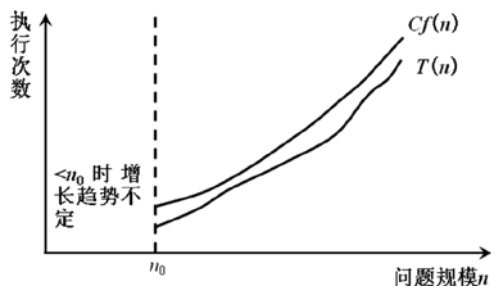


图 1.6 符号“O”的直观含义

【例 1.8】 立方阶示例。

```
(1) x=1;
(2) for(i=1;i<=n;i++)
(3)     for(j=1;j<=i;j++)
(4)         for(k=1;k<=j;k++)
(5)             x++;
```

显见, 该程序段中频度最大的语句是 (5), 这条最深层循环内的基本语句的频度, 依赖于各层循环变量的取值, 由内向外可分析出语句 (5) 的执行次数为:

$$\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n i(i+1)/2 = [n(n+1)(2n+1)/6 + n(n+1)/2]/2$$

则该算法的时间复杂度为  $T(n) = O(n^3)$ , 称为立方阶。

【例 1.9】 对数阶示例。

```
for(i=1;i<=n;i=i*2){x++;s=0;}
```

设循环体内两条基本语句的频度为  $f(n)$ , 则有  $2^{f(n)} \leq n$ ,  $f(n) \leq \log_2 n$ , 所以算法的时间复杂度为  $T(n) = O(\log_2 n)$ , 称为对数阶。

常见的时间复杂度按数量级递增排列依次为: 常量阶  $O(1)$ 、对数阶  $O(\log_2 n)$ 、线性阶  $O(n)$ 、线性对数阶  $O(n \log_2 n)$ 、平方阶  $O(n^2)$ 、立方阶  $O(n^3)$ 、……、 $k$  次方阶  $O(n^k)$ 、指数阶  $O(2^n)$  等。

不同数量级的时间复杂度性状如图 1.7 所示。一般情况下, 随着  $n$  的增大,  $T(n)$  的增长较慢的算法为较优的算法。显然, 时间复杂度为指数阶  $O(2^n)$  的算法效率极低, 当  $n$  值稍大时就无法应用。应该尽可能选择使用多项式阶  $O(n^k)$  的算法, 而避免使用指数阶的算法。

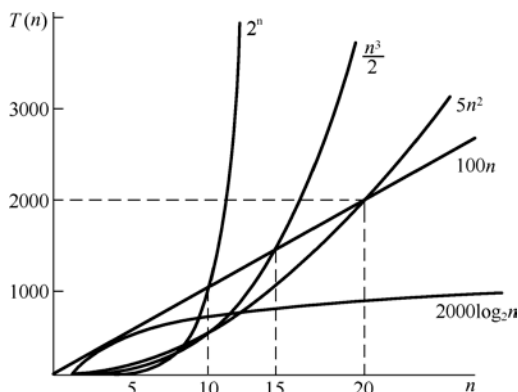


图 1.7 常见函数的增长率

#### 4. 最好、最坏和平均时间复杂度

对于某些问题的算法, 其基本语句的频度不仅仅与问题的规模相关, 还依赖于其他因素。在此仅举一例说明之。

【例 1.10】 在一维数组  $a$  中顺序查找某个值等于  $e$  的元素, 并返回其所在位置。

```
(1) for(i=0;i<n;i++)
(2)     if(a[i]==e) return i+1;
(3) return 0;
```

容易看出, 此算法中语句 (2) 的频度不仅与问题规模  $n$  有关, 还与输入实例中数组  $a[i]$  的各元素值及  $e$  的取值有关。假设在数组  $a[i]$  中必定存在值等于  $e$  的元素, 则查找必定成功, 且  $\text{for}$  循环内的语句的频度将随被找到的元素在数组中出现的位置不同而不同, 最好情况是, 每次要找的值与  $e$  相同的元素恰好就是数组中的第一个元素, 则不论数组的规模多大, 语句 (2) 的频度  $f(n) = 1$ , 最坏情况是, 每次待查找的都是数组中最后一个元素, 则语句 (2) 的频度  $f(n) = n$ 。而对于一个算法来说, 需要考虑各种可能出现的情况, 以及每一种情况出现的概率, 一般情况下, 可假设待

查找的元素在数组中所有位置上出现的可能性均相同,则可取语句(2)的频度在最好情况与最坏情况下的平均值,即  $f(n) = n/2$ , 作为它的度量。

此例说明,算法的时间复杂度不仅与问题的规模有关,还与问题的其他因素有关。再如某些排序的算法,其执行时间与待排序记录的初始状态有关。因此,有时会对算法有最好、最坏以及平均时间复杂度的评价。

称算法在最好情况下的时间复杂度为**最好时间复杂度**,指的是算法计算量可能达到的最小值;称算法在最坏情况下的时间复杂度为**最坏时间复杂度**,指的是算法计算量可能达到的最大值;算法的**平均时间复杂度**是指算法在所有可能情况下,按照输入实例以等概率出现时,算法计算量的加权平均值。

对算法时间复杂度的度量,人们更关心的是最坏情况下和平均情况下的时间复杂度。然而在很多情况下,算法的平均时间复杂度难于确定。因此,通常只讨论算法在最坏情况下的时间复杂度,即分析在最坏情况下,算法执行时间的上界。在本书后面内容中讨论的时间复杂度,除特别指明外,均指最坏情况下的时间复杂度。

### 1.4.4 算法的空间复杂度

关于算法的存储空间需求,类似于算法的时间复杂度,我们采用渐近**空间复杂度**(Space Complexity)作为算法所需存储空间的量度,简称空间复杂度,它也是问题规模  $n$  的函数,记作:

$$S(n) = O(f(n))$$

一般情况下,一个程序在机器上执行时,除了需要寄存本身所用的指令、常数、变量和输入数据外,还需要一些对数据进行操作的辅助存储空间。其中,对于输入数据所占的具体存储量取决于问题本身,与算法无关,这样只需分析该算法在实现时所需要的辅助空间就可以了。若算法执行时所需要的辅助空间相对于输入数据量而言是个常数,则称这个算法为原地工作,辅助空间为  $O(1)$ ,本节中前面的示例都是如此。有的算法需要占用临时的工作单元数与问题规模  $n$  有关,如第 8 章介绍的归并排序算法就属于这种情况。

下面举一简单示例说明如何求算法的空间复杂度。

**【例 1.11】** 数组逆序,将一维数组  $a$  中的  $n$  个数逆序存放到原数组中。

**【算法 1】**

```
for(i=0;i<n/2;i++)
{
    t=a[i];
    a[i]=a[n-i-1];
    a[n-i-1]=t;
}
```

**【算法 2】**

```
for(i=0;i<n;i++)
    b[i]=a[n-i-1];
for(i=0;i<n;i++)
    a[i]=b[i];
```

算法 1 仅需要另外借助一个变量  $t$ , 与问题规模  $n$  大小无关,所以其空间复杂度为  $O(1)$ 。

算法 2 需要另外借助一个大小为  $n$  的辅助数组  $b$ , 所以其空间复杂度为  $O(n)$ 。

对于一个算法,其时间复杂度和空间复杂度往往是相互影响的,当追求一个较好的时间

复杂度时,可能会导致占用较多的存储空间,即可能会使空间复杂度的性能变差,反之亦然。不过,通常情况下,鉴于运算空间较为充足,人们都以算法的时间复杂度作为算法优劣的衡量指标。

## 1.5 小 结

本章介绍了数据结构的基本概念和术语,以及算法和算法时间复杂度的分析方法。主要内容如下。

(1) 数据结构是一门研究非数值计算程序设计中操作对象,以及这些对象之间的关系和操作的学科。

(2) 数据结构包括两个方面的内容:数据的逻辑结构和存储结构。同一逻辑结构采用不同的存储方法,可以得到不同的存储结构。

① 逻辑结构是从具体问题抽象出来的数学模型,从逻辑关系上描述数据,它与数据的存储无关。根据数据元素之间关系的不同特性,通常有四类基本逻辑结构:集合结构、线性结构、树形结构和图状结构。

② 存储结构是逻辑结构在计算机中的存储表示,有两类存储结构:顺序存储结构和链式存储结构。

(3) 抽象数据类型是指由用户定义的、表示应用问题的数学模型,以及定义在这个模型上的一组操作的总称,具体包括三部分:数据对象、数据对象上关系的集合,以及对数据对象的基本操作的集合。

(4) 算法是为了解决某类问题而规定的一个有限长的操作序列。算法具有五个特性:有穷性、确定性、可行性、输入和输出。一个算法的优劣应该从以下四方面来评价:正确性、可读性、健壮性和高效性。

(5) 算法分析的两个主要方面是分析算法的时间复杂度和空间复杂度,以考察算法的时间和空间效率。一般情况下,鉴于运算空间较为充足,故将算法的时间复杂度作为分析的重点。算法执行时间的数量级称为算法的渐近时间复杂度, $T(n) = O(f(n))$ ,它表示随着问题规模  $n$  的增大,算法执行时间的增长率和  $f(n)$  的增长率相同,简称时间复杂度。

学完本章后,要求掌握数据结构相关的基本概念,包括数据、数据元素、数据项、数据对象、数据结构、逻辑结构、存储结构等;重点掌握数据结构所含两个层次的具体含义及其相互关系;了解抽象数据类型的定义、表示与实现方法;了解算法的特性和评价标准;重点掌握算法时间复杂度的分析方法。

## 习 题

1. 简述下列概念:数据、数据元素、数据项、数据对象、数据结构、逻辑结构、存储结构、抽象数据类型。

2. 试举一个数据结构的例子,叙述其逻辑结构和存储结构两个层次的含义及相互关系。

3. 简述逻辑结构的四种基本关系并画出它们的关系图。
4. 存储结构由哪两种基本的存储方法实现？
5. 选择题
- (1) 在数据结构中，从逻辑上可以把数据结构分成( )。
- A. 动态结构和静态结构                      B. 紧凑结构和非紧凑结构
- C. 线性结构和非线性结构                  D. 内部结构和外部结构
- (2) 与数据元素本身的形式、内容、相对位置、个数无关的是数据的( )。
- A. 存储结构                                  B. 存储实现
- C. 逻辑结构                                  D. 运算实现
- (3) 通常要求同一逻辑结构中的所有数据元素具有相同的特性，这意味着( )。
- A. 数据具有同一特点
- B. 不仅数据元素所包含的数据项的个数要相同，而且对应数据项的类型要一致
- C. 每个数据元素都一样
- D. 数据元素所包含的数据项的个数要相等
- (4) 以下说法正确的是( )。
- A. 数据元素是数据的最小单位
- B. 数据项是数据的基本单位
- C. 数据结构是带有结构的各数据项的集合
- D. 一些表面上很不相同的数据可以有相同的逻辑结构
- (5) 算法的时间复杂度取决于( )。
- A. 问题的规模                                  B. 待处理数据的初态
- C. 计算机的配置                              D. A 和 B
- (6) 以下数据结构中，( )是非线性数据结构。
- A. 树                                              B. 字符串
- C. 队列                                          D. 栈
6. 试分析下列各算法的时间复杂度。
- (1) 

```
x=90; y=100;
while(y>0)
    if(x>100)
        {x=x-10;y--;}
    else x++;
```
- (2) 

```
for(i=0; i<n; i++)
    for(j=0; j<m; j++)
        a[i][j]=0;
```
- (3) 

```
s=0;
for(i=0; i<n; i++)
    for(j=0; j<n; j++)
        s+=B[i][j];
sum=s;
```
- (4) 

```
i=1;
while(i<=n)
    i=i*3;
```

```

(5) x=0;
    for(i=1; i<n; i++)
        for(j=1; j<=n-i; j++)
            x++;
(6) x=n; //n>1
    y=0;
    while(x>=(y+1)*(y+1))
        y++;

```

# 第 2 章

## 线性表

从本章至第 4 章讨论的线性表、栈、队列、串和数组都属于线性结构。线性结构的基本特点是除第一个元素无直接前驱，最后一个元素无直接后继之外，其他每个数据元素都有一个前驱和后继。线性表是最基本且最常用的一种线性结构，同时也是其他数据结构的基础，尤其单链表，是贯穿整个数据结构课程的基本技术。本章将讨论线性表的逻辑结构、存储结构和相关运算，以及线性表的应用实例。本章所涉及的许多问题都具有一定的普遍性。因此，本章是整个课程的重点与核心内容，也是其他后续章节的重要基础。

### 2.1 线性表的定义和特点

在日常生活中，线性表的例子比比皆是。例如，26 个英文字母的字母表：

(A, B, C, ..., Z)

是一个线性表，表中的数据元素是单个字母。在稍复杂的线性表中，一个数据元素可以包含若干个数据项。例如在第 1 章中提到的学生基本信息表，每个学生为一个数据元素，包括学号、姓名、性别、籍贯、专业等数据项。

由以上示例可以看出，它们的数据元素虽然不同，但同一线性表中的元素必定具有相同的特性，即属于同一数据对象，相邻数据元素之间存在着序偶关系。

诸如此类由  $n$  ( $n \geq 0$ ) 个数据特性相同的元素构成的有限序列称为**线性表**。

线性表中元素的个数  $n$  ( $n \geq 0$ ) 定义为线性表的长度， $n = 0$  时称为**空表**。

对于非空的线性表或线性结构，其特点是：

- (1) 存在唯一的一个被称作“第一个”的数据元素；
- (2) 存在唯一的一个被称作“最后一个”的数据元素；
- (3) 除第一个之外，结构中的每个数据元素均只有一个前驱；
- (4) 除最后一个之外，结构中的每个数据元素均只有一个后继。

### 2.2 案例引入

案例 2.1：一元多项式的运算。

在数学上，一个一元多项式  $P_n(x)$  可按升幂写成：

$$P_n(x) = p_0 + p_1x + p_2x^2 + \cdots + p_nx^n$$

要求：实现两个一元多项式的相加、相减和相乘的运算。

实现两个多项式相关运算的前提是如何在计算机中有效地表示一元多项式，进而在此基础上设计相关运算的算法？这个问题看似很复杂，我们通过学习本章中线性表的表示及其相关运算便可以完成。

可以看出，一元多项式可由  $n+1$  个系数唯一确定，因此，可以将一元多项式  $P_n(x)$  抽象为一个由  $n+1$  个元素组成的有序序列，可用一个线性表  $P$  来表示：

$$P = (p_0, p_1, p_2, \cdots, p_n)$$

这时，每一项的指数  $i$  隐含在其系数  $p_i$  的序号中。

假设  $Q_m(x)$  是一元  $m$  次多项式，同样可用线性表  $Q$  来表示：

$$Q = (q_0, q_1, q_2, \cdots, q_m)$$

不失一般性，设  $m \leq n$ ，则两个多项式相加的结果  $R_n(x) = P_n(x) + Q_m(x)$  可用线性表  $R$  表示：

$$R = (p_0 + q_0, p_1 + q_1, p_2 + q_2, \cdots, p_m + q_m, p_{m+1}, \cdots, p_n)$$

在后面的叙述中将看到，对此类多项式的线性表只需要用数组表示的顺序存储结构便很容易实现上述运算。

然而，在通常的应用中，多项式的次数可能很高且变化很大，这种所谓的稀疏多项式如果采用上述表示方法，将使得线性表中出现很多零元素。下面给出稀疏多项式的例子。

#### 案例 2.2：稀疏多项式的运算。

例如，在处理形如

$$S(x) = 1 + 3x^{10000} + 2x^{20000}$$

的多项式时，就要用一个长度为 20001 的线性表来表示，而表中仅有 3 个非零元素，此时将会造成存储空间的一大浪费，这种对空间的浪费是应当避免的。由于线性表的元素可以包含多个数据项，由此可改变元素设定，对多项式的每一项，可用（系数，指数）唯一确定。

一般情况下的一元  $n$  次多项式可写成

$$P_n(x) = p_1 x^{e_1} + p_2 x^{e_2} + \cdots + p_m x^{e_m} \quad (2-1)$$

其中， $p_i$  是指数为  $e_i$  的项的非零系数，且满足

$$0 \leq e_1 < e_2 < \cdots < e_m = n$$

若用一个长度为  $m$  且每个元素有两个数据项（系数项和指数项）的线性表

$$((p_1, e_1), (p_2, e_2), \cdots, (p_m, e_m)) \quad (2-2)$$

便可唯一确定多项式  $P_n(x)$ 。在最坏情况下， $n+1 (=m)$  个系数都不为零，则比只存储每项系数的方案要多存储一倍的数据。但是，对于类似  $S(x)$  的稀疏多项式，这种表示将大大节省空间。

由上述讨论可以看出，如果多项式属于非稀疏多项式，且只对多项式进行“求值”等不改变多项式的系数和指数的运算，可采用数组表示的顺序存储结构。如果多项式属于稀疏多项式，虽然可以采用数组表示法，但这种顺序存储结构的存储空间分配不够灵活。因为事先无法确定多项式的非零项数，所以需要根据预期估计可能的最大值定义数组的大小，这种分配方式可能会带来两种问题：一种是实际非零项数比较小，浪费了大量存储空间；另一种是实际非零项数超过了最大值，存储空间不够。另外在实现多项式的相加运算时，还需要开辟一个新的数组保存结果多项式，导致算法的空间复杂度较高。改进方案是利用链式存储结构表示多项式的有序序列，这样灵活性更大些。

那么,如何利用链式存储结构表示由式(2-2)定义的多项式,并实现多项式的相关运算呢?本章2.8节将给出详细的介绍。

### 案例 2.3: 图书信息管理系统。

出版社有一些图书数据保存在一个文本文件 book.txt 中,为简单起见,在此假设每种图书只包括三部分信息:ISBN(书号)、书名和价格,文件中的部分数据如图2.1所示。现要求实现一个图书信息管理系统,包括以下6个具体功能。

(1) 查找:根据指定的 ISBN 或书名查找相应图书的有关信息,并返回该图书在表中的位置序号。

(2) 插入:插入一种新的图书信息。

(3) 删除:删除一种图书信息。

(4) 修改:根据指定的 ISBN,修改该图书的价格。

(5) 排序:将图书按照价格由低到高进行排序。

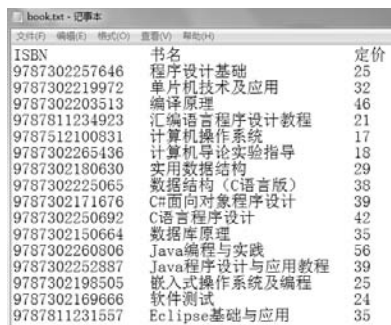
(6) 计数:统计图书表中的图书数量。

要实现上述功能,与以上案例中的多项式一样,我们首先根据图书表的特点将其抽象成一个线性表,每本图书作为线性表中的一个元素,然后可以采用适当的存储结构来表示该线性表,在此基础上设计完成有关的功能算法。具体采取哪种存储结构,可以根据两种不同存储结构的优缺点,视实际情况而定。

可以看出,在工作和生活中的许多实际问题都会涉及图书信息管理中用到的这些基本操作。这些问题中都包含由  $n$  个数据特性相同的元素,即可以表示为线性表。不同的问题所涉元素的数据类型不尽相同,可以为简单数据类型(如案例2.1所示的一元多项式表示),也可以为复杂数据类型(如案例2.2所示的稀疏多项式表示和案例2.3中的图书数据),但这些问题所涉的基本操作都具有很大的相似性,如果为每个具体应用都编一个程序显然不是一种很好的方法。解决这类问题的最好方法就是从具体应用中抽象出共性的逻辑结构和基本操作(即抽象数据类型),然后采用程序设计语言实现相应的存储结构和基本操作。

本章后续章节将依次给出线性表的抽象数据类型定义、线性表的顺序和链式存储结构的表示及实现、线性表应用实例的实现。

在学完本章后,案例2.3的基本操作读者很容易就能实现。



| ISBN          | 书名            | 定价 |
|---------------|---------------|----|
| 9787302257646 | 程序设计基础        | 25 |
| 9787302219972 | 单片机技术及应用      | 32 |
| 9787302203513 | 编译原理          | 46 |
| 9787811234923 | 汇编语言程序设计教程    | 21 |
| 9787512100831 | 计算机操作系统       | 17 |
| 9787302265436 | 计算机导论实验指导     | 18 |
| 9787302180630 | 实用数据结构        | 29 |
| 9787302225065 | 数据结构(C语言版)    | 38 |
| 9787302171676 | C#面向对象程序设计    | 39 |
| 9787302250692 | C语言程序设计       | 42 |
| 9787302150664 | 数据库原理         | 35 |
| 9787302260806 | Java编程与实践     | 56 |
| 9787302252887 | Java程序设计与应用教程 | 39 |
| 9787302198505 | 嵌入式操作系统及编程    | 25 |
| 9787302169666 | 软件测试          | 24 |
| 9787811231557 | Eclipse基础与应用  | 35 |

图 2.1 图书数据

## 2.3 线性表的类型定义

线性表是一个相当灵活的数据结构,其长度可根据需要增长或缩短,即对线性表的数据元素不仅可以进行访问,而且可以进行插入和删除等操作。为不失一般性,本书采用1.2节抽象数据类型格式对各种数据结构进行描述。下面给出线性表的抽象数据类型定义:

```
ADT List{
    数据对象:  $D=\{a_i | a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0\}$ 
```

数据关系:  $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2, \dots, n \}$

基本操作:

InitList(&L)

操作结果: 构造一个空的线性表 L。

DestroyList(&L)

初始条件: 线性表 L 已存在。

操作结果: 销毁线性表 L。

ClearList(&L)

初始条件: 线性表 L 已存在。

操作结果: 将 L 重置为空表。

ListEmpty(L)

初始条件: 线性表 L 已存在。

操作结果: 若 L 为空表, 则返回 true, 否则返回 false。

ListLength(L)

初始条件: 线性表 L 已存在。

操作结果: 返回 L 中数据元素个数。

GetElem(L, i, &e)

初始条件: 线性表 L 已存在, 且  $1 \leq i \leq \text{ListLength}(L)$ 。

操作结果: 用 e 返回 L 中第 i 个数据元素的值。

LocateElem(L, e)

初始条件: 线性表 L 已存在。

操作结果: 返回 L 中第 1 个值与 e 相同的元素在 L 中的位置。若这样的数据元素不存在, 则返回值为 0。

PriorElem(L, cur\_e, &pre\_e)

初始条件: 线性表 L 已存在。

操作结果: 若 cur\_e 是 L 的数据元素, 且不是第一个, 则用 pre\_e 返回其前驱, 否则操作失败, pre\_e 无定义。

NextElem(L, cur\_e, &next\_e)

初始条件: 线性表 L 已存在。

操作结果: 若 cur\_e 是 L 的数据元素, 且不是最后一个, 则用 next\_e 返回其后继, 否则操作失败, next\_e 无定义。

ListInsert(&L, i, e)

初始条件: 线性表 L 已存在, 且  $1 \leq i \leq \text{ListLength}(L) + 1$ 。

操作结果: 在 L 中第 i 个位置之前插入新的数据元素 e, L 的长度加 1。

ListDelete(&L, i)

初始条件: 线性表 L 已存在且非空, 且  $1 \leq i \leq \text{ListLength}(L)$ 。

操作结果: 删除 L 的第 i 个数据元素, L 的长度减 1。

TraverseList(L)

初始条件: 线性表 L 已存在。

操作结果: 对线性表 L 进行遍历, 在遍历过程中对 L 的每个结点访问一次。

ADT List

(1) 抽象数据类型仅是一个模型的定义, 并不涉及模型的具体实现, 因此这里描述中所涉及的参数不必考虑具体数据类型。在实际应用中, 数据元素可能有多种类型, 到时可根椐具体需要选择使用不同的数据类型。

(2) 上述抽象数据类型中给出的操作只是基本操作, 由这些基本操作可以构成其他较复杂的操作。例如, 2.2 节中的两个应用实例, 不论是一元多项式的运算还是图书的管理, 首先都需要将数据元素读入, 生成一个包括所需数据的线性表, 这属于线性表的创建。这项操作可首先调用基本操作定义中的 InitList(&L)构造一个空的线性表 L, 然后反复调用 ListInsert(&L, i, e)在表中插入元素 e, 就可以创建一个需要的线性表。同样, 对



说明

于一元多项式的运算可以看作是线性表的合并，合并过程需要不断地进行元素的插入操作。其他如求线性表的拆分、复制等操作也都可以利用上述基本操作的组合来实现。

(3) 对于不同的应用，基本操作的接口可能不同。例如，案例 2.2 的删除操作，如果要求删除图书表中 ISBN 为  $x$  的图书，首先需要根据  $x$  确定该图书在线性表中的位置，然后再利用  $ListDelete(\&L,i)$  基本操作将该种图书记录从表中删除。

(4) 由抽象数据类型定义的线性表，可以根据实际所采用的存储结构形式，进行具体的表示和实现。

## 2.4 线性表的顺序表示和实现

### 2.4.1 线性表的顺序存储表示

线性表的顺序表示指的是用一组地址连续的存储单元依次存储线性表的数据元素，这种表示也称作线性表的顺序存储结构或顺序映像。通常，称这种存储结构的线性表为顺序表 (Sequential List)。其特点是，逻辑上相邻的数据元素，其物理次序也是相邻的。

假设线性表的每个元素需占用  $l$  个存储单元，并以所占的第一个单元的存储地址作为数据元素的存储起始位置。则线性表中第  $i+1$  个数据元素的存储位置  $LOC(a_{i+1})$  和第  $i$  个数据元素的存储位置  $LOC(a_i)$  之间满足下列关系：

$$LOC(a_{i+1}) = LOC(a_i) + l$$

一般来说，线性表的第  $i$  个数据元素  $a_i$  的存储位置为：

$$LOC(a_i) = LOC(a_1) + (i-1) \times l$$

式中， $LOC(a_1)$  是线性表的第一个数据元素  $a_1$  的存储位置，通常称作线性表的起始位置或基地址，表中相邻的元素  $a_i$  和  $a_{i+1}$  的存储位置  $LOC(a_i)$  和  $LOC(a_{i+1})$  是相邻的。每一个数据元素的存储位置都和线性表的起始位置相差一个常数，这个常数和数据元素在线性表中的位序成正比（见图 2.2）。由此，只要确定了存储线性表的起始位置，线性表中任一数据元素都可随机存取，所以线性表的顺序存储结构是一种随机存取的存储结构。

由于高级程序设计语言中的数组类型也有随机存取的特性，因此，通常都用数组来描述数据结构中的顺序存储结构。在此，由于线性表的长度可变，且所需最大存储空间随问题不同而不同，则在 C 语言中可用动态分配的一维数组表示线性表，描述如下：

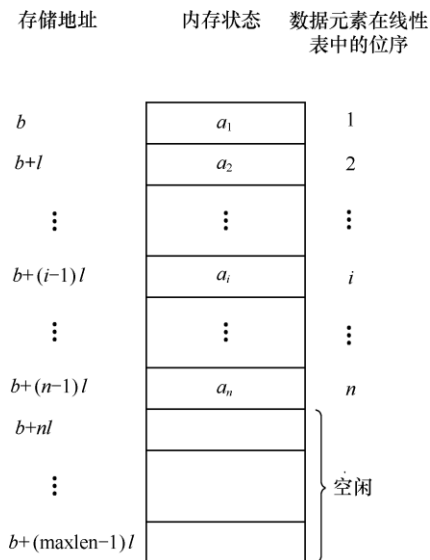


图 2.2 线性表的顺序存储结构示意图

```
// - - - - 顺序表的存储结构 - - - -
#define MAXSIZE 100           // 顺序表可能达到的最大长度
typedef struct
{
```

```
ElemType *elem;           //存储空间的基地址
int length;               //当前长度
}SqList;                  //顺序表的结构类型为 SqList
```

(1) 数组空间通过后面算法 2.1 初始化动态分配得到, 初始化完成后, 数组指针 `elem` 指示顺序表的基地址, 数组空间大小为 `MAXSIZE`。



说明

(2) 元素类型定义中的 `ElemType` 数据类型是为了描述统一而自定的, 在实际应用中, 用户可根据实际需要具体定义表中数据元素的数据类型, 既可以是基本数据类型, 如 `int`、`float`、`char` 等, 也可以是构造数据类型, 如 `struct` 结构体类型。

(3) `length` 表示顺序表中当前数据元素的个数。因为 C 语言数组的下标是从 0 开始的, 而位置序号是从 1 开始的, 所以要注意区分元素的位置序号和该元素在数组中的下标位置之间的对应关系, 数据元素  $a_1$ 、 $a_2$ 、 $\dots$ 、 $a_n$  依次存放在数组 `elem[0]`、`elem[1]`、 $\dots$ 、`elem[length-1]` 中。

用顺序表存储案例 2.2 的稀疏多项式数据时, 其顺序存储分配情况如图 2.3 所示。多项式的顺序存储结构的类型定义如下:

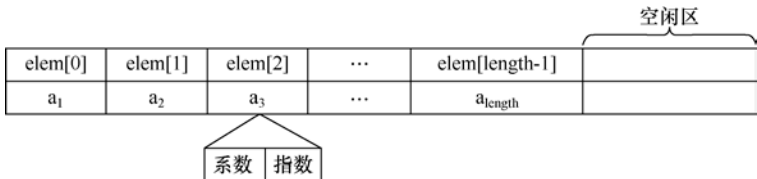


图 2.3 一元多项式的顺序存储结构示意图

```
#define MAXSIZE 100           //多项式可能达到的最大长度
typedef struct                //多项式非零项的定义
{
    float coef;               //系数
    int expn;                 //指数
}Polynomial;
typedef struct
{
    Polynomial *elem;         //存储空间的基地址
    int length;               //多项式中当前项的个数
}SqList;                      //多项式的顺序存储结构类型为 SqList
```

用顺序表存储案例 2.3 的图书数据时, 其顺序存储分配情况如图 2.4 所示。图书表的顺序存储结构的类型定义如下:

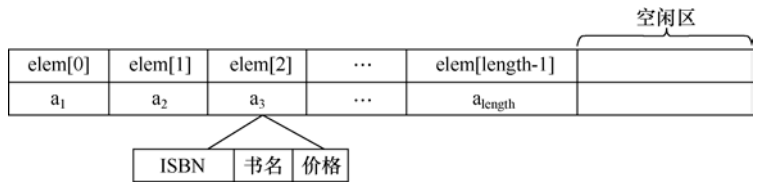


图 2.4 图书数据的顺序存储结构示意图

```

#define MAXSIZE 10000          //图书表可能达到的最大长度
typedef struct                 //图书信息定义
{
    char no[20];                //图书 ISBN
    char name[50];              //图书名字
    float price;                //图书价格
}Book;
typedef struct
{
    Book *elem;                 //存储空间的基地址
    int length;                 //图书表中当前图书个数
}SqList;                       //图书表的顺序存储结构类型为 SqList

```

在上述定义后，可以通过变量定义语句

```
SqList L;
```

将 L 定义为 SqList 类型的变量，便可以利用 L.elem[i-1]访问表中位置序号为 i 的的图书记录。

## 2.4.2 顺序表中基本操作的实现

可以看出，当线性表以上述定义的顺序表表示时，某些操作很容易实现。因为表的长度是顺序表的一个“属性”，所以可以通过返回 length 的值实现求表长的操作，通过判断 length 的值是否为 0 判断表是否为空，这些操作算法的时间复杂度都是  $O(1)$ 。下面讨论顺序表其他几个主要操作的实现。

### 1. 初始化

顺序表的初始化操作就是构造一个空的顺序表。

#### 算法 2.1 顺序表的初始化

##### 【算法步骤】

- ① 为顺序表 L 动态分配一个预定义大小的数组空间，使 elem 指向这段空间的基地址。
- ② 将表的当前长度设为 0。

##### 【算法描述】

```

Status InitList(SqList &L)
{//构造一个空的顺序表 L
    L.elem=new ElemType[MAXSIZE];    //为顺序表分配一个大小为 MAXSIZE 的数组空间
    if(!L.elem) exit(OVERFLOW);      //存储分配失败退出
    L.length=0;                       //空表长度为 0
    return OK;
}

```

动态分配线性表的存储区域可以更有效地利用系统的资源，当不需要该线性表时，可以使用销毁操作及时释放占用的存储空间。

### 2. 取值

取值操作是根据指定的位置序号 i，获取顺序表中第 i 个数据元素的值。

由于顺序存储结构具有随机存取的特点，可以直接通过数组下标定位得到，elem[i-1]单元存储第 i 个数据元素。

## 算法 2.2 顺序表的取值

### 【算法步骤】

- ① 判断指定的位置序号  $i$  值是否合理 ( $1 \leq i \leq L.length$ ), 若不合理, 则返回 ERROR。
- ② 若  $i$  值合理, 则将第  $i$  个数据元素  $L.elem[i-1]$  赋给参数  $e$ , 通过  $e$  返回第  $i$  个数据元素的传值。

### 【算法描述】

```
Status GetElem(SqList L, int i, ElemType &e)
{
    if (i < 1 || i > L.length) return ERROR;    //判断 i 值是否合理, 若不合理, 返回 ERROR
    e = L.elem[i-1];                            //elem[i-1] 单元存储第 i 个数据元素
    return OK;
}
```

### 【算法分析】

显然, 顺序表取值算法的时间复杂度为  $O(1)$ 。

## 3. 查找

查找操作是根据指定的元素值  $e$ , 查找顺序表中第 1 个与  $e$  相等的元素。若查找成功, 则返回该元素在表中的位置序号; 若查找失败, 则返回 0。

## 算法 2.3 顺序表的查找

### 【算法步骤】

- ① 从第一个元素起, 依次和  $e$  相比较, 若找到与  $e$  相等的元素  $L.elem[i]$ , 则查找成功, 返回该元素的序号  $i+1$ 。
- ② 若查遍整个顺序表都没有找到, 则查找失败, 返回 0。

### 【算法描述】

```
int LocateElem(SqList L, ElemType e)
{ //在顺序表 L 中查找值为 e 的数据元素, 返回其序号
    for (i = 0; i < L.length; i++)
        if (L.elem[i] == e) return i+1;        //查找成功, 返回序号 i+1
    return 0;                                    //查找失败, 返回 0
}
```

### 【算法分析】

当在顺序表中查找一个数据元素时, 其时间主要耗费在数据的比较上, 而比较的次数取决于被查元素在线性表中的位置。

在查找时, 为确定元素在顺序表中的位置, 需和给定值进行比较的数据元素个数的期望值称为查找算法在查找成功时的平均查找长度 (Average Search Length, **ASL**)。

假设  $p_i$  是查找第  $i$  个元素的概率,  $C_i$  为找到表中其关键字与给定值相等的第  $i$  个记录时, 和给定值已进行过比较的关键字数, 则在长度为  $n$  的线性表中, 查找成功时的平均查找长度为

$$ASL = \sum_{i=1}^n p_i C_i \quad (2-3)$$

从顺序表查找的过程可见,  $C_i$  取决于所查元素在表中的位置。例如, 查找表中第一个记录时,

仅需比较一次；而查找表中最后一个记录时，则需比较  $n$  次。一般情况下  $C_i$  等于  $i$ 。

假设每个元素的查找概率相等，即

$$p_i = 1/n$$

则式 (2-3) 可简化为式 (2-4)

$$ASL = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} \quad (2-4)$$

由此可见，顺序表按值查找算法的平均时间复杂度为  $O(n)$ 。

#### 4. 插入

线性表的插入操作是指在表的第  $i$  个位置插入一个新的数据元素  $e$ ，使长度为  $n$  的线性表

$$(a_1, \dots, a_{i-1}, a_i, \dots, a_n)$$

变成长度为  $n+1$  的线性表

$$(a_1, \dots, a_{i-1}, e, a_i, \dots, a_n)$$

数据元素  $a_{i-1}$  和  $a_i$  之间的逻辑关系发生了变化。在线性表的顺序存储结构中，由于逻辑上相邻的数据元素在物理位置上也是相邻的，因此，除非  $i = n+1$ ，否则必须移动元素才能反映这个逻辑关系的变化。

例如，图 2.5 所示为一个线性表在插入前后数据元素在存储空间中的位置变化。为了在线性表的第 5 个位置上插入一个值为 25 的数据元素，则需将第 5 个至第 8 个数据元素依次向后移动一个位置。

一般情况下，在第  $i$  ( $1 \leq i \leq n$ ) 个位置插入一个元素时，需从最后一个元素即第  $n$  个元素开始，依次向后移动一个位置，直至第  $i$  个元素（共  $n-i+1$  个元素）。

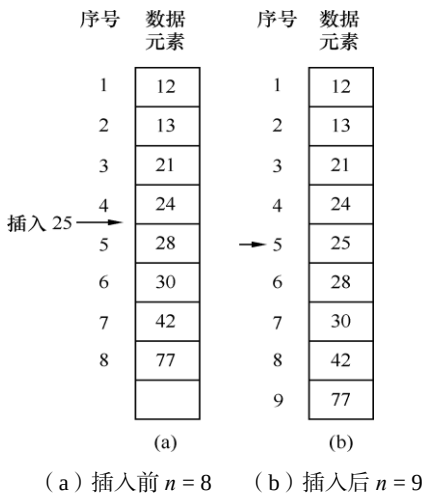


图 2.5 线性表插入前后的状况

#### 算法 2.4 顺序表的插入

##### 【算法步骤】

- ① 判断插入位置  $i$  是否合法 ( $i$  值的合法范围是  $1 \leq i \leq n+1$ )，若不合法则返回 ERROR。
- ② 判断顺序表的存储空间是否已满，若满则返回 ERROR。
- ③ 将第  $n$  个至第  $i$  个位置的元素依次向后移动一个位置，空出第  $i$  个位置 ( $i = n+1$  时无需移动)。

- ④ 将要插入的新元素  $e$  放入第  $i$  个位置。

- ⑤ 表长加 1。

##### 【算法描述】

```
Status ListInsert(SqList &L, int i, ElemType e)
{ // 在顺序表 L 中第 i 个位置之前插入新的元素 e, i 值的合法范围是 1 ≤ i ≤ L.length+1
  if ((i < 1) || (i > L.length+1)) return ERROR; // i 值不合法
  if (L.length == MAXSIZE) return ERROR; // 当前存储空间已满
  for (j = L.length-1; j >= i-1; j--)
```

```

        L.elem[j+1]=L.elem[j];           //插入位置及之后的元素后移
    L.elem[i-1]=e;                       //将新元素 e 放入第 i 个位置
    ++L.length;                          //表长加 1
    return OK;
}

```

上述算法没有处理表的动态扩充, 因此当表长已经达到预设的最大空间时, 则不能再插入元素。

### 【算法分析】

当在顺序表中某个位置上插入一个数据元素时, 其时间主要耗费在移动元素上, 而移动元素的个数取决于插入元素的位置。

假设  $p_i$  是在第  $i$  个元素之前插入一个元素的概率,  $E_{\text{ins}}$  为在长度为  $n$  的线性表中插入一个元素时所需移动元素次数的期望值 (平均次数), 则有

$$E_{\text{ins}} = \sum_{i=1}^{n+1} p_i (n-i+1) \quad (2-5)$$

不失一般性, 可以假定在线性表的任何位置上插入元素都是等概率的, 即

$$p_i = \frac{1}{n+1}$$

则式 (2-5) 可简化为式 (2-6)

$$E_{\text{ins}} = \frac{1}{n+1} \sum_{i=1}^{n+1} (n-i+1) = \frac{n}{2} \quad (2-6)$$

由此可见, 顺序表插入算法的平均时间复杂度为  $O(n)$ 。

## 5. 删除

线性表的删除操作是指将表的第  $i$  个元素删去, 将长度为  $n$  的线性表

$(a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$

变成长度为  $n-1$  的线性表

$(a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$

数据元素  $a_{i-1}$ 、 $a_i$  和  $a_{i+1}$  之间的逻辑关系发生了变化, 为了在存储结构上反映这个变化, 同样需要移动元素。如图 2.6 所示, 为了删除第 4 个数据元素, 必须将第 5 个至第 8 个元素都依次向前移动一个位置。

一般情况下, 删除第  $i$  ( $1 \leq i \leq n$ ) 个元素时需将第  $i+1$  个至第  $n$  个元素 (共  $n-i$  个元素) 依次向前移动一个位置 ( $i=n$  时无需移动)。

### 算法 2.5 顺序表的删除

#### 【算法步骤】

① 判断删除位置  $i$  是否合法 (合法值为  $1 \leq i \leq n$ ), 若非法则返回 ERROR。

② 将第  $i+1$  个至第  $n$  个的元素依次向前移动一个位置 ( $i=n$  时无需移动)。

③ 表长减 1。

| 序号 | 数据元素 | 序号 | 数据元素 |
|----|------|----|------|
| 1  | 12   | 1  | 12   |
| 2  | 13   | 2  | 13   |
| 3  | 21   | 3  | 21   |
| 4  | 24   | 4  | 28   |
| 5  | 28   | 5  | 30   |
| 6  | 30   | 6  | 42   |
| 7  | 42   | 7  | 77   |
| 8  | 77   |    |      |
|    |      |    |      |

(a) 删除前  $n=8$       (b) 删除后  $n=7$

图 2.6 线性表删除前后的状况

## 【算法描述】

```

Status ListDelete(SqList &L, int i)
{ // 在顺序表 L 中删除第 i 个元素, i 值的合法范围是  $1 \leq i \leq L.length$ 
  if ((i < 1) || (i > L.length)) return ERROR; // i 值不合法
  for (j = i; j <= L.length - 1; j++)
    L.elem[j - 1] = L.elem[j]; // 被删除元素之后的元素前移
  --L.length; // 表长减 1
  return OK;
}

```

## 【算法分析】

当在顺序表中某个位置上删除一个数据元素时, 其时间主要耗费在移动元素上, 而移动元素的个数取决于删除元素的位置。

假设  $p_i$  是删除第  $i$  个元素的概率,  $E_{\text{del}}$  为在长度为  $n$  的线性表中删除一个元素时所需移动元素次数的期望值 (平均次数), 则有

$$E_{\text{del}} = \sum_{i=1}^n p_i (n - i) \quad (2-7)$$

不失一般性, 可以假定在线性表的任何位置上删除元素都是等概率的, 即

$$p_i = \frac{1}{n}$$

则式 (2-7) 简化为式 (2-8)

$$E_{\text{del}} = \frac{1}{n} \sum_{i=1}^n (n - i) = \frac{n - 1}{2} \quad (2-8)$$

由此可见, 顺序表删除算法的平均时间复杂度为  $O(n)$ 。

顺序表可以随机存取表中任一元素, 其存储位置可用一个简单、直观的公式来表示。然而, 从另一方面来看, 这个特点也造成了这种存储结构的缺点: 在做插入或删除操作时, 需移动大量元素。另外由于数组有长度相对固定的静态特性, 当表中数据元素个数较多且变化较大时, 操作过程相对复杂, 必然导致存储空间的浪费。所有这些问题, 都可以通过线性表的另一种表示方法——链式存储结构来解决。

## 2.5 线性表的链式表示和实现

### 2.5.1 单链表的定义和表示

线性表链式存储结构的特点是: 用一组任意的存储单元存储线性表的数据元素 (这组存储单元可以是连续的, 也可以是不连续的)。因此, 为了表示每个数据元素  $a_i$  与其直接后继数据元素  $a_{i+1}$  之间的逻辑关系, 对数据元素  $a_i$  来说, 除了存储其本身的信息之外, 还需存储一个指示其直接后继的信息 (即直接后继的存储位置)。这两部分信息组成数据元素  $a_i$  的存储映像, 称为结点

(node)。它包括两个域：其中存储数据元素信息的域称为**数据域**；存储直接后继存储位置的域称为**指针域**。指针域中存储的信息称作**指针或链**。 $n$  个结点 ( $a_i (1 \leq i \leq n)$  的存储映像) 链结成一个**链表**，即为线性表

$$(a_1, a_2, \dots, a_n)$$

的链式存储结构。又由于此链表的每个结点中只包含一个指针域，故又称**线性链表或单链表**。

根据链表结点所含指针个数、指针指向和指针连接方式，可将链表分为单链表、循环链表、双向链表、二叉链表、十字链表、邻接表、邻接多重表等。其中单链表、循环链表和双向链表用于实现线性表的链式存储结构，其他形式多用于实现树和图等非线性结构。

本节先讨论单链表，例如，图 2.7 所示为线性表的单链表存储结构，整个链表的存取必须从头指针开始进行，头指针指示链表中第一个结点（即第一个数据元素的存储映像，也称首元结点）的存储位置。同时，由于最后一个数据元素没有直接后继，则单链表中最后一个结点的指针为空（NULL）。

| (ZHAO, QIAN, SUN, LI, ZHOU, WU, ZHENG, WANG)                                                 |      |       |      |
|----------------------------------------------------------------------------------------------|------|-------|------|
|                                                                                              | 存储地址 | 数据域   | 指针域  |
| 头指针 L<br><div style="border: 1px solid black; padding: 2px; display: inline-block;">61</div> | 1    | LI    | 85   |
|                                                                                              | 13   | QIAN  | 25   |
|                                                                                              | 25   | SUN   | 1    |
|                                                                                              | 37   | WANG  | NULL |
|                                                                                              | 49   | WU    | 73   |
|                                                                                              | 61   | ZHAO  | 13   |
|                                                                                              | 73   | ZHENG | 37   |
|                                                                                              | 85   | ZHOU  | 49   |

图 2.7 单链表示例

用单链表表示线性表时，数据元素之间的逻辑关系是由结点中的指针指示的。换句话说，指针为数据元素之间的逻辑关系的映像，则逻辑上相邻的两个数据元素其存储的物理位置不要求紧邻，由此，这种存储结构为非顺序映像或链式映像。

通常将链表画成用箭头相链接的结点的序列，结点之间的箭头表示链域中的指针。图 2.7 所示的单链表可画成如图 2.8 所示的形式，这是因为在使用链表时，关心的只是它所表示的线性表中数据元素之间的逻辑顺序，而不是每个数据元素在存储器中的实际位置。

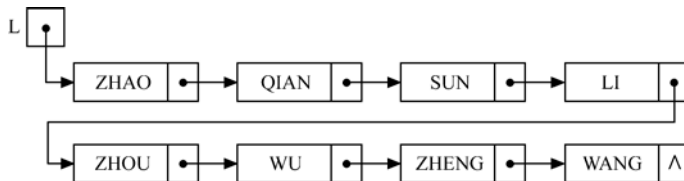


图 2.8 单链表的逻辑状态

由上述可见，单链表可由头指针唯一确定，在 C 语言中可用“结构指针”来描述：

```
// - - - - 单链表的存储结构 - - - -
typedef struct LNode
{
    ElemType data;           // 结点的数据域
```

```

struct LNode *next;           //结点的指针域
}LNode, *LinkList;           //LinkList 为指向结构体 LNode 的指针类型

```

(1) 这里定义的是单链表中每个结点的存储结构, 它包括两部分: 存储结点的数据域 data, 其类型用通用类型标识符 ElemType 表示 (例如, 用链表表示案例 2.1 中的图书信息时, 只需将 ElemType 替换为 2.4.1 定义的 Book 数据类型即可); 存储后继结点位置的指针域 next, 其类型为指向结点的指针类型 LNode \*。



说明

(2) 为了提高程序的可读性, 在此对同一结构体指针类型起了两个名称, LinkList 与 LNode \*, 两者本质上是等价的。通常习惯上用 LinkList 定义单链表, 强调定义的是某个单链表的头指针; 用 LNode \* 定义指向单链表中任意结点的指针变量。例如, 若定义 LinkList L, 则 L 为单链表的头指针, 若定义 LNode \*p, 则 p 为指向单链表中某个结点的指针, 用 \*p 代表该结点。当然也可以使用定义 LinkList p, 这种定义形式完全等价于 LNode \*p。

(3) 单链表是由表头指针唯一确定, 因此单链表可以用头指针的名字来命名。若头指针名是 L, 则简称该链表为表 L。

(4) 注意区分指针变量和结点变量两个不同的概念, 若定义 LinkList p 或 LNode \*p, 则 p 为指向某结点的指针变量, 表示该结点的地址; 而 \*p 为对应的结点变量, 表示该结点的名称。

一般情况下, 为了处理方便, 在单链表的第一个结点之前附设一个结点, 称之为头结点。图 2.8 所示的单链表增加头结点后如图 2.9 所示。

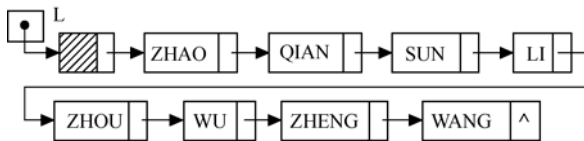


图 2.9 增加头结点的单链表的逻辑状态

下面对首元结点、头结点、头指针三个容易混淆的概念加以说明。

(1) 首元结点是指链表中存储第一个数据元素  $a_1$  的结点。如图 2.8 或图 2.9 所示的结点 “ZHAO”。



说明

(2) 头结点是在首元结点之前附设的一个结点, 其指针域指向首元结点。头结点的数据域可以不存储任何信息, 也可存储与数据元素类型相同的其他附加信息。例如, 当数据元素为整数型时, 头结点的数据域中可存放该线性表的长度。

(3) 头指针是指向链表中第一个结点的指针。若链表设有头结点, 则头指针所指结点为线性表的头结点; 若链表不设头结点, 则头指针所指结点为该线性表的首元结点。

链表增加头结点的作用如下。

(1) 便于首元结点的处理

增加了头结点后, 首元结点的地址保存在头结点(即其“前驱”结点)的指针域中, 则对链表的第一个数据元素的操作与其他数据元素相同, 无需进行特殊处理。

(2) 便于空表和非空表的统一处理

当链表不设头结点时, 假设 L 为单链表的头指针, 它应该指向首元结点, 则当单链表为长度  $n$  为 0 的空表时, L 指针为空 (判定空表的条件可记为:  $L = \text{NULL}$ )。

增加头结点后,无论链表是否为空,头指针都是指向头结点的非空指针。如图 2.10 (a) 所示的非空单链表,头指针指向头结点。若为空表,则头结点的指针域为空(判定空表的条件可记为:  $L \rightarrow next = NULL$ ),如图 2.10 (b) 所示。

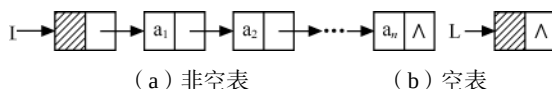


图 2.10 带头结点的单链表

在顺序表中,由于逻辑上相邻的两个元素在物理位置上紧邻,则每个元素的存储位置都可从线性表的起始位置计算得到。而在单链表中,各个元素的存储位置都是随意的。然而,每个元素的存储位置都包含在其直接前驱结点的信息之中。假设  $p$  是指向单链表中第  $i$  个数据元素(结点  $a_i$ , 即数据域为  $a_i$  的结点)的指针,则  $p \rightarrow next$  是指向第  $i+1$  个数据元素(结点  $a_{i+1}$ )的指针。换句话说,若  $p \rightarrow data = a_i$ ,则  $p \rightarrow next \rightarrow data = a_{i+1}$ 。由此,单链表是非随机存取的存储结构,要取得第  $i$  个数据元素必须从头指针出发顺链进行寻找,也称为**顺序存取**的存取结构。因此,其基本操作的实现不同于顺序表。

## 2.5.2 单链表基本操作的实现

### 1. 初始化

单链表的初始化操作就是构造一个如图 2.10 (b) 所示的空表。

#### 算法 2.6 单链表的初始化

##### 【算法步骤】

- ① 生成新结点作为头结点,用头指针  $L$  指向头结点。
- ② 头结点的指针域置空。

##### 【算法描述】

```
Status InitList(LinkList &L)
{ //构造一个空的单链表 L
    L=new LNode;                //生成新结点作为头结点,用头指针 L 指向头结点
    L->next=NULL;                //头结点的指针域置空
    return OK;
}
```

### 2. 取值

和顺序表不同,链表中逻辑相邻的结点并没有存储在物理相邻的单元中,这样,根据给定的结点位置序号  $i$ ,在链表中获取该结点的值不能像顺序表那样随机访问,而只能从链表的首元结点出发,顺着链域  $next$  逐个结点向下访问。

#### 算法 2.7 单链表的取值

##### 【算法步骤】

- ① 用指针  $p$  指向首元结点,用  $j$  做计数器初值赋为 1。
- ② 从首元结点开始依次顺着链域  $next$  向下访问,只要指向当前结点的指针  $p$  不为空( $NULL$ ),并且没有到达序号为  $i$  的结点,则循环执行以下操作:
  - $p$  指向下一个结点;
  - 计数器  $j$  相应加 1。

③ 退出循环时,如果指针  $p$  为空,或者计数器  $j$  大于  $i$ ,说明指定的序号  $i$  值不合法 ( $i$  大于表长  $n$  或  $i$  小于等于 0),取值失败返回 ERROR;否则取值成功,此时  $j=i$  时,  $p$  所指的结点就是要找的第  $i$  个结点,用参数  $e$  保存当前结点的数据域,返回 OK。

#### 【算法描述】

```
Status GetElem(LinkList L,int i,ElemType &e)
{ //在带头结点的单链表 L 中根据序号 i 获取元素的值,用 e 返回 L 中第 i 个数据元素的值
    p=L->next;j=1;                //初始化, p 指向首元结点,计数器 j 初值赋为 1
    while(p&& j<i)                //顺链域向后扫描,直到 p 为空或 p 指向第 i 个元素
    {
        p=p->next;                //p 指向下一个结点
        ++j;                      //计数器 j 相应加 1
    }
    if(!p||j>i) return ERROR;      //i 值不合法 i>n 或 i≤0
    e=p->data;                    //取第 i 个结点的数据域
    return OK;
}
```

#### 【算法分析】

该算法的基本操作是比较  $j$  和  $i$  并后移指针  $p$ , while 循环体中的语句频度与位置  $i$  有关。若  $1 \leq i \leq n$ , 则频度为  $i-1$ , 一定能取值成功;若  $i > n$ , 则频度为  $n$ , 取值失败。因此算法 2.7 的最坏时间复杂度为  $O(n)$ 。

假设每个位置上元素的取值概率相等,即

$$p_i = 1/n$$

则

$$ASL = \frac{1}{n} \sum_{i=1}^n (i-1) = \frac{n-1}{2} \quad (2-9)$$

由此可见,单链表取值算法的平均时间复杂度为  $O(n)$ 。

### 3. 查找

链表中按值查找的过程和顺序表类似,从链表的首元结点出发,依次将结点值和给定值  $e$  进行比较,返回查找结果。

#### 算法 2.8 单链表的按值查找

##### 【算法步骤】

- ① 用指针  $p$  指向首元结点。
- ② 从首元结点开始依次顺着链域  $next$  向下查找,只要指向当前结点的指针  $p$  不为空,并且  $p$  所指结点的数据域不等于给定值  $e$ ,则循环执行以下操作:  $p$  指向下一个结点。
- ③ 返回  $p$ 。若查找成功,  $p$  此时即为结点的地址值,若查找失败,  $p$  的值即为 NULL。

##### 【算法描述】

```
LNode *LocateElem(LinkList L, Elemtype e)
{ //在带头结点的单链表 L 中查找值为 e 的元素
    p=L->next;                    //初始化, p 指向首元结点
```

```

while(p && p->data!=e)    //顺链域向后扫描,直到 p 为空或 p 所指结点的数据域等于 e
    p=p->next;            //p 指向下一个结点
return p;                //查找成功返回值为 e 的结点地址 p,查找失败 p 为 NULL
}

```

#### 【算法分析】

该算法的执行时间与待查找的值  $e$  相关,其平均时间复杂度分析类似于算法 2.7,也为  $O(n)$ 。

#### 4. 插入

假设要在单链表的两个数据元素  $a$  和  $b$  之间插入一个数据元素  $x$ ,已知  $p$  为其单链表存储结构中指向结点  $a$  的指针,如图 2.11 (a) 所示。

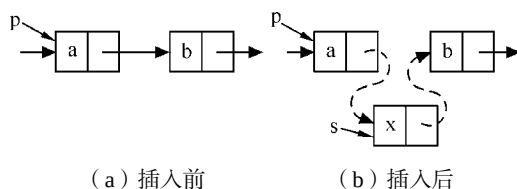


图 2.11 在单链表中插入结点时指针变化状况

为插入数据元素  $x$ ,首先要生成一个数据域为  $x$  的结点,然后插入到单链表中。根据插入操作的逻辑定义,还需要修改结点  $a$  中的指针域,令其指向结点  $x$ ,而结点  $x$  中的指针域应指向结点  $b$ ,从而实现 3 个元素  $a$ 、 $b$  和  $x$  之间逻辑关系的变化。插入后的单链表如图 2.11 (b) 所示。假设  $s$  为指向结点  $x$  的指针,则上述指针修改用语句描述即为

```
s->next = p->next; p->next = s;
```

#### 算法 2.9 单链表的插入

##### 【算法步骤】

将值为  $e$  的新结点插入到表的第  $i$  个结点的位置上,即插入到结点  $a_{i-1}$  与  $a_i$  之间,具体插入过程如图 2.12 所示,图中对应的 5 个步骤说明如下。

- ① 查找结点  $a_{i-1}$  并由指针  $p$  指向该结点。
- ② 生成一个新结点  $*s$ 。
- ③ 将新结点  $*s$  的数据域置为  $e$ 。
- ④ 将新结点  $*s$  的指针域指向结点  $a_i$ 。
- ⑤ 将结点  $*p$  的指针域指向新结点  $*s$ 。

##### 【算法描述】

```

Status ListInsert(LinkList &L,int i,ElemType e)
{ //在带头结点的单链表 L 中第 i 个位置插入值为 e 的新结点
    p=L;j=0;
    while(p && (j<i-1))
        {p=p->next;++j;} //查找第 i-1 个结点, p 指向该结点
    if(!p||j>i-1) return ERROR; //i>n+1 或者 i<1
    s=new LNode; //生成新结点*s
    s->data=e; //将结点*s 的数据域置为 e
    s->next=p->next; //将结点*s 的指针域指向结点 a_i
}

```

```

p->next=s;                                //将结点*p 的指针域指向结点*s
return OK;
}

```



和顺序表一样,如果表中有  $n$  个结点,则插入操作中合法的插入位置有  $n+1$  个,即  $1 \leq i \leq n+1$ 。当  $i=n+1$  时,新结点则插在链表尾部。

### 【算法分析】

单链表的插入操作虽然不需要像顺序表的插入操作那样需要移动元素,但平均时间复杂度仍为  $O(n)$ 。这是因为,为了在第  $i$  个结点之前插入一个新结点,必须首先找到第  $i-1$  个结点,其时间复杂度与算法 2.7 相同,为  $O(n)$ 。

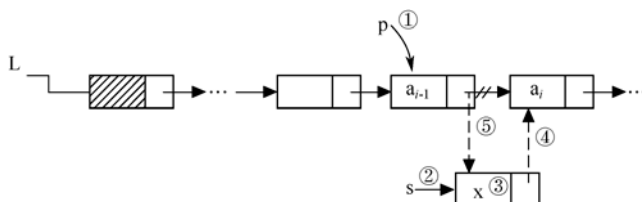


图 2.12 在单链表第  $i$  个位置上插入新结点的过程

### 5. 删除

要删除单链表中指定位置的元素,同插入元素一样,首先应该找到该位置的前驱结点。如图 2.13 所示,在单链表中删除元素  $b$  时,应该首先找到其前驱结点  $a$ 。为了在单链表中实现元素  $a$ 、 $b$  和  $c$  之间逻辑关系的变化,仅需修改结点  $a$  中的指针域即可。假设  $p$  为指向结点  $a$  的指针,则修改指针的语句为

```
p->next = p->next->next;
```

但在删除结点  $b$  时,除了修改结点  $a$  的指针域外,还要释放结点  $b$  所占的空间,所以在修改指针前,应该引入另一指针  $q$ ,临时保存结点  $b$  的地址以备释放。

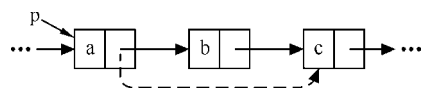


图 2.13 在单链表中删除结点时指针的变化

### 算法 2.10 单链表的删除

#### 【算法步骤】

删除单链表的第  $i$  个结点  $a_i$  的具体过程如图 2.14 所示,图中的对应的 4 个步骤说明如下。

- ① 查找结点  $a_{i-1}$  并由指针  $p$  指向该结点。
- ② 临时保存待删除结点  $a_i$  的地址在  $q$  中,以备释放。
- ③ 将结点  $*p$  的指针域指向  $a_i$  的直接后继结点。
- ④ 释放结点  $a_i$  的空间。

#### 【算法描述】

```

Status ListDelete(LinkList &L,int i)
{
    //在带头结点的单链表 L 中,删除第 i 个元素
    p=L;j=0;
    while((p->next) && (j<i-1))    //查找第 i-1 个结点, p 指向该结点
    {
        p=p->next; ++j;
    }
}

```

```

    if (!(p->next) || (j>i-1)) return ERROR;    //当 i>n 或 i<1 时, 删除位置不合理
    q=p->next;                                  //临时保存被删结点的地址以备释放
    p->next=q->next;                             //改变删除结点前驱结点的指针域
    delete q;                                    //释放删除结点的空间
    return OK;
}

```



删除算法中的循环条件( $p \rightarrow \text{next} \&\& j < i - 1$ )和插入算法中的循环条件( $p \&\& (j < i - 1)$ )是有所区别的。因为插入操作中合法的插入位置有  $n+1$  个, 而删除操作中合法的删除位置只有  $n$  个, 如果使用与插入操作相同的循环条件, 则会出现引用空指针的情况, 使删除操作失败。

### 【算法分析】

类似于插入算法, 删除算法时间复杂度亦为  $O(n)$ 。

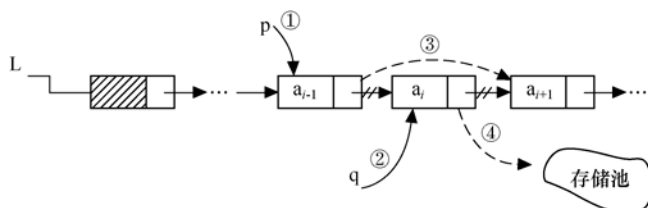


图 2.14 删除单链表第  $i$  个结点的过程

## 6. 创建单链表

算法 2.6 的初始化操作是创建一个只有一个头结点的空链表, 而上面链表的其他算法都是假定链表已经存在多个结点。那么, 如何建立一个包括若干个结点的链表呢? 链表和顺序表不同, 它是一种动态结构。整个可用存储空间可为多个链表共同享用, 每个链表占用的空间不需预先分配划定, 而是由系统按需即时生成。因此, 建立线性表的链式存储结构的过程就是一个动态生成链表的过程。即从空表的初始状态起, 依次建立各元素结点, 并逐个插入链表。

根据结点插入位置的不同, 链表的创建方法可分为前插法和后插法。

### (1) 前插法

前插法是通过将新结点逐个插入链表的头部 (头结点之后) 来创建链表, 每次申请一个新结点, 读入相应的数据元素值, 然后将新结点插入到头结点之后。

#### 算法 2.11 前插法创建单链表

#### 【算法步骤】

- ① 创建一个只有头结点的空链表。
- ② 根据待创建链表包括的元素个数  $n$ , 循环  $n$  次执行以下操作:
  - 生成一个新结点  $*p$ ;
  - 输入元素值赋给新结点  $*p$  的数据域;
  - 将新结点  $*p$  插入到头结点之后。

图 2.15 所示为线性表 (a,b,c,d,e) 前插法的创建过程, 因为每次插入在链表的头部, 所以应该逆位序输入数据, 依次输入 e、d、c、b、a, 输入顺序和线性表中的逻辑顺序是相反的。

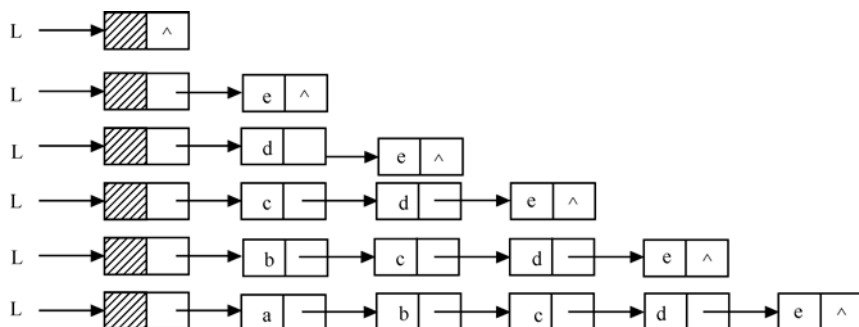


图 2.15 前插法创建单链表

## 【算法描述】

```
void CreateList_H(LinkList &L, int n)
{ // 逆序输入 n 个元素的值, 建立带表头结点的单链表 L
  L = new LNode;
  L->next = NULL; // 先建立一个带头结点的空链表
  for (i = 0; i < n; ++i)
  {
    p = new LNode; // 生成新结点 *p
    cin >> p->data; // 输入元素值赋给新结点 *p 的数据域
    p->next = L->next; L->next = p; // 将新结点 *p 插入到头结点之后
  }
}
```

显然, 算法 2.11 的时间复杂度为  $O(n)$ 。

## (2) 后插法

后插法是通过将新结点逐个插入到链表的尾部来创建链表。同前插法一样, 每次申请一个新结点, 读入相应的数据元素值。不同的是, 为了使新结点能够插入到表尾, 需要增加一个尾指针  $r$  指向链表的尾结点。

## 算法 2.12 后插法创建单链表

## 【算法步骤】

- ① 创建一个只有头结点的空链表。
- ② 尾指针  $r$  初始化, 指向头结点。
- ③ 根据创建链表包括的元素个数  $n$ , 循环  $n$  次执行以下操作:
  - 生成一个新结点  $*p$ ;
  - 输入元素值赋给新结点  $*p$  的数据域;
  - 将新结点  $*p$  插入到尾结点  $*r$  之后;
  - 尾指针  $r$  指向新的尾结点  $*p$ 。

图 2.16 所示为线性表 (a,b,c,d,e) 后插法的创建过程, 读入数据的顺序和线性表中的逻辑顺序是相同的。

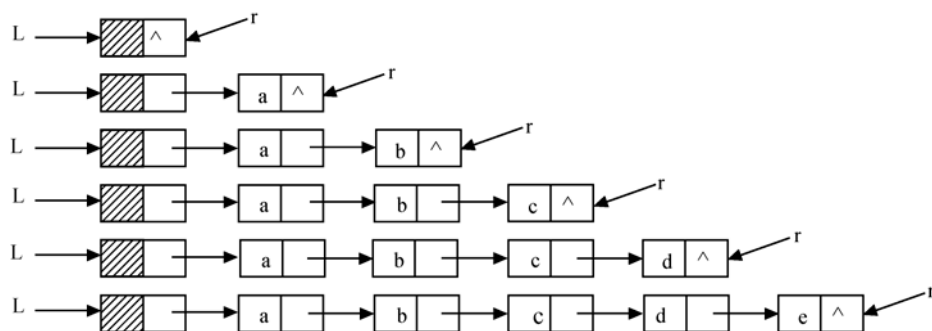


图 2.16 后插法创建单链表

### 【算法描述】

```
void CreateList_R(LinkList &L,int n)
//正位序输入 n 个元素的值, 建立带表头结点的单链表 L
{
    L=new LNode;
    L->next=NULL;                                //先建立一个带头结点的空链表
    r=L;                                          //尾指针 r 指向头结点
    for (i=0;i<n;++i)
    {
        p=new LNode;                            //生成新结点
        cin>>p->data;                            //输入元素值赋给新结点*p 的数据域
        p->next=NULL; r->next=p;                //将新结点*p 插入尾结点*r 之后
        r=p;                                    //r 指向新的尾结点*p
    }
}
```

算法 2.12 的时间复杂度亦为  $O(n)$ 。

## 2.5.3 循环链表

**循环链表** (Circular Linked List) 是另一种形式的链式存储结构。其特点是表中最后一个结点的指针域指向头结点, 整个链表形成一个环。由此, 从表中任一结点出发均可找到表中其他结点, 图 2.17 所示为单链的循环链表。类似地, 还可以有多重链的循环链表。

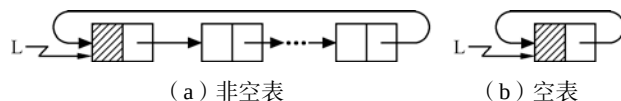


图 2.17 单循环链表

循环单链表的操作和单链表基本一致, 差别仅在于: 当链表遍历时, 判别当前指针  $p$  是否指向表尾结点的终止条件不同。在单链表中, 判别条件为  $p!=NULL$  或  $p->next!=NULL$ , 而循环单链表的判别条件为  $p!=L$  或  $p->next!=L$ 。

在某些情况下, 若在循环链表中设立尾指针而不设头指针 (见图 2.18 (a)), 可使一些操作简化。例如, 将两个线性表合并成一个表时, 仅需将第一个表的尾指针指向第二个表的第

一个结点，第二个表的尾指针指向第一个表的头结点，然后释放第二个表的头结点。当线性表以图 2.18 (a) 的循环链表作存储结构时，这个操作仅需改变两个指针值即可，主要语句段如下：

```
p = B->next->next;
B->next = A->next;
A->next = p;
```

上述操作的时间复杂度为  $O(1)$ ，合并后的表如图 2.18 (b) 所示。

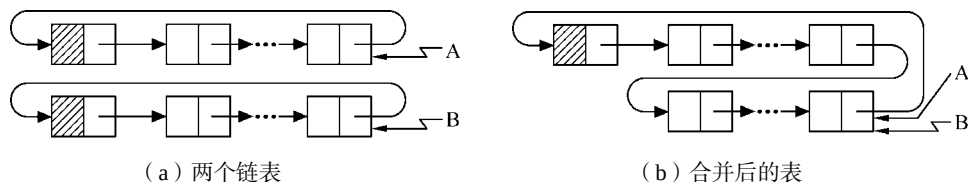


图 2.18 仅设尾指针的循环链表

## 2.5.4 双向链表

以上讨论的链式存储结构的结点中只有一个指示直接后继的指针域，由此，从某个结点出发只能顺指针向后寻查其他结点。若要寻查结点的直接前驱，则必须从表头指针出发。换句话说，在单链表中，查找直接后继结点的执行时间为  $O(1)$ ，而查找直接前驱的执行时间为  $O(n)$ 。为克服单链表这种单向性的缺点，可利用**双向链表** (Double Linked List)。

顾名思义，在双向链表的结点中有两个指针域，一个指向直接后继，另一个指向直接前驱，结点结构如图 2.19 (a) 所示，在 C 语言中可描述如下：

```
// - - - - 双向链表的存储结构 - - - -
typedef struct DuLNode
{
    ElemType data;           //数据域
    struct DuLNode *prior;   //直接前驱
    struct DuLNode *next;    //直接后继
} DuLNode, *DuLinkedList;
```

和单链的循环表类似，双向链表也可以有循环表，如图 2.19 (c) 所示，链表中共有两个环，图 2.19 (b) 所示为只有一个表头结点的空表。

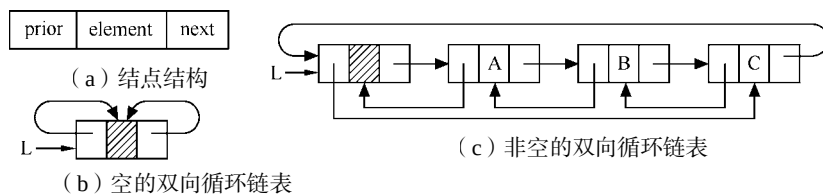


图 2.19 双向链表示例

在双向链表中，若  $d$  为指向表中某一结点的指针（即  $d$  为  $DuLinkedList$  型变量），则显然有

$$d \rightarrow \text{next} \rightarrow \text{prior} = d \rightarrow \text{prior} \rightarrow \text{next} = d$$

这个表示方式恰当地反映了这种结构的特性。

在双向链表中,有些操作(如 ListLength、GetElem 和 LocateElem 等)仅需涉及一个方向的指针,则它们的算法描述和线性链表的操作相同,但在插入、删除时有很大的不同,在双向链表中需同时修改两个方向上的指针,图 2.20 和图 2.21 分别显示了插入和删除结点时指针修改的情况。在插入结点时需要修改四个指针,在删除结点时需要修改两个指针。它们的实现分别如算法 2.13 和算法 2.14 所示,两者的时间复杂度均为  $O(n)$ 。

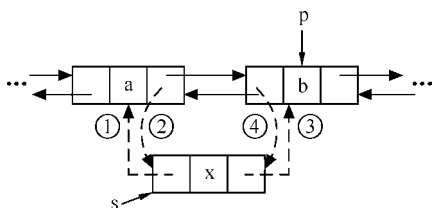


图 2.20 在双向链表中插入结点时指针的变化状况

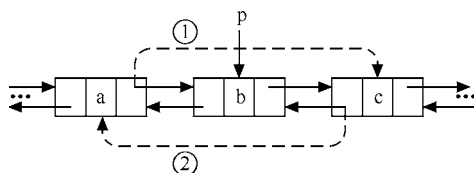


图 2.21 在双向链表中删除结点时指针的变化状况

### 算法 2.13 双向链表的插入

#### 【算法描述】

```
Status ListInsert_DuL(DuLinkList &L, int i, ElemType e)
{ // 在带头结点的双向链表 L 中第 i 个位置之前插入元素 e
    if (! (p = GetElem_DuL(L, i))) // 在 L 中确定第 i 个元素的位置指针 p
        return ERROR; // p 为 NULL 时, 第 i 个元素不存在
    s = new DuLNode; // 生成新结点 *s
    s->data = e; // 将结点 *s 数据域置为 e
    s->prior = p->prior; // 将结点 *s 插入 L 中, 此步对应图 2.20①
    p->prior->next = s; // 对应图 2.20②
    s->next = p; // 对应图 2.20③
    p->prior = s; // 对应图 2.20④
    return OK;
}
```

### 算法 2.14 双向链表的删除

#### 【算法描述】

```
Status ListDelete_DuL(DuLinkList &L, int i)
{ // 删除带头结点的双向链表 L 中的第 i 个元素
    if (! (p = GetElem_DuL(L, i))) // 在 L 中确定第 i 个元素的位置指针 p
        return ERROR; // p 为 NULL 时, 第 i 个元素不存在
    p->prior->next = p->next; // 修改被删结点的前驱结点的后继指针, 对应图 2.21①
    p->next->prior = p->prior; // 修改被删结点的后继结点的前驱指针, 对应图 2.21②
    delete p; // 释放被删结点的空间
    return OK;
}
```

## 2.6 顺序表和链表的比较

前面两节介绍了线性表的两种存储结构：顺序表和链表。在实际应用中，不能笼统地说哪种存储结构更好，由于它们各有优缺点，选用哪种存储结构，则应根据具体问题作具体分析，通常从空间性能和时间性能两个方面作比较分析。

### 2.6.1 空间性能的比较

#### (1) 存储空间的分配

顺序表的存储空间必须预先分配，元素个数扩充受一定限制，易造成存储空间浪费或空间溢出现象；而链表不需要为其预先分配空间，只要内存空间允许，链表中的元素个数就没有限制。

基于此，当线性表的长度变化较大，难以预估存储规模时，宜采用链表作为存储结构。

#### (2) 存储密度的大小

链表的每个结点除了设置数据域用来存储数据元素外，还要额外设置指针域，用来存储指示元素之间逻辑关系的指针，从存储密度上来讲，这是不经济的。所谓**存储密度**是指数据元素本身所占用的存储量和整个结点结构所占用的存储量之比，即

$$\text{存储密度} = \frac{\text{数据元素本身占用的存储量}}{\text{结点结构占用的存储量}}$$

存储密度越大，存储空间的利用率就越高。显然，顺序表的存储密度为 1，而链表的存储密度小于 1。如果每个元素数据域占据的空间较小，则指针的结构性开销就占用了整个结点的大部分空间，这样存储密度较小。例如，若单链表的结点数据均为整数，指针所占用的空间和整型量相同，则单链表的存储密度为 0.5。因此，如果不考虑顺序表中的空闲区，则顺序表的存储空间利用率为 100%，而单链表的存储空间利用率仅为 50%。

基于此，当线性表的长度变化不大，易于事先确定其大小时，为了节约存储空间，宜采用顺序表作为存储结构。

### 2.6.2 时间性能的比较

#### (1) 存取元素的效率

顺序表是由数组实现的，它是一种随机存取结构，指定任意一个位置序号  $i$ ，都可以在  $O(1)$  时间内直接存取该位置上的元素，即取值操作的效率高；而链表是一种顺序存取结构，按位置访问链表中第  $i$  个元素时，只能从表头开始依次向后遍历链表，直到找到第  $i$  个位置上的元素，时间复杂度为  $O(n)$ ，即取值操作的效率低。

基于此，若线性表的主要操作是和元素位置紧密相关的这类取值操作，很少做插入或删除时，宜采用顺序表作为存储结构。

#### (2) 插入和删除操作的效率

对于链表，在确定插入或删除的位置后，插入或删除操作无需移动数据，只需要修改指针，时间复杂度为  $O(1)$ 。而对于顺序表，进行插入或删除时，平均要移动表中近一半的结点，时间复杂度为  $O(n)$ 。尤其是当每个结点的信息量较大时，移动结点的时间开销就相当可观。

基于此，对于频繁进行插入或删除操作的线性表，宜采用链表作为存储结构。

## 2.7 线性表的应用

### 2.7.1 线性表的合并

【例 2.1】 求解一般集合的并集问题。

【问题描述】

已知两个集合  $A$  和  $B$ ，现要求一个新的集合  $A = A \cup B$ 。例如，设

$$A = (7, 5, 3, 11)$$

$$B = (2, 6, 3)$$

合并后

$$A = (7, 5, 3, 11, 2, 6)$$

【问题分析】

可以利用两个线性表 LA 和 LB 分别表示集合  $A$  和  $B$ （即线性表中的数据元素为集合中的成员），这样只需扩大线性表 LA，将存在于 LB 中而不存在于 LA 中的数据元素插入到 LA 中去。只要从 LB 中依次取得每个数据元素，并依值在 LA 中进行查访，若不存在，则插入之。

上述操作过程可用算法 2.15 来描述。具体实现时既可采用顺序形式，也可采用链表形式。

**算法 2.15** 线性表的合并

【算法步骤】

- ① 分别获取 LA 表长  $m$  和 LB 表长  $n$ 。
- ② 从 LB 中第 1 个数据元素开始，循环  $n$  次执行以下操作：
  - 从 LB 中查找第  $i$  ( $1 \leq i \leq n$ ) 个数据元素赋给  $e$ ；
  - 在 LA 中查找元素  $e$ ，如果不存在，则将  $e$  插在表 LA 的最后。

【算法描述】

```
void MergeList(List &LA, List LB)
//将所有在线性表 LB 中但不在 LA 中的数据元素插入到 LA 中
{
    m=ListLength(LA); n=ListLength(LB);           //求线性表的长度
    for (i=1; i<=n; i++)
    {
        GetElem(LB, i, e);                         //取 LB 中第 i 个数据元素赋给 e
        if (!LocateElem(LA, e))                    //LA 中不存在和 e 相同的数据元素
            ListInsert(LA, ++m, e);                 //将 e 插在 LA 的最后
    }
}
```

【算法分析】

上述算法的时间复杂度取决于抽象数据类型 List 定义中基本操作的执行时间，假设 LA 和 LB 的表长分别为  $m$  和  $n$ ，循环执行  $n$  次，则：

当采用顺序存储结构时，在每次循环中，GetElem 和 ListInsert 这两个操作的执行时间和表长无关，LocateElem 的执行时间和表长  $m$  成正比，因此，算法 2.15 的时间复杂度为  $O(m \times n)$ 。

当采用链式存储结构时，在每次循环中，ListInsert 的执行时间和表长无关，GetElem 的执行时间和表长  $n$  成正比，LocateElem 的执行时间和表长  $m$  成正比，因此，若假设  $m$  大于  $n$ ，算法 2.15 的时间复杂度也为  $O(m \times n)$ 。

## 2.7.2 有序表的合并

若线性表中的数据元素相互之间可以比较，并且数据元素在线性表中依值非递减或非递增有序排列，则称该线性表为**有序表**（Ordered List）。

【例 2.2】 求解有序集合的并集问题。

【问题描述】

**有序集合**是指集合中的元素有序排列。已知两个有序集合  $A$  和  $B$ ，数据元素按值非递减有序排列，现要求一个新的集合  $C = A \cup B$ ，使集合  $C$  中的数据元素仍按值非递减有序排列。

例如，设

$$\begin{aligned} A &= (3, 5, 8, 11) \\ B &= (2, 6, 8, 9, 11, 15, 20) \end{aligned}$$

则

$$C = (2, 3, 5, 6, 8, 8, 9, 11, 11, 15, 20)$$

【问题分析】

与例 2.1 一样，可以利用两个线性表 LA 和 LB 分别表示集合  $A$  和  $B$ ，不同的是，此例中的 LA 和 LB 有序，这样便没有必要从 LB 中依次取得每个数据元素，到 LA 中进行查访。

如果 LA 和 LB 两个表长分别记为  $m$  和  $n$ ，则合并后的新表 LC 的表长应该为  $m+n$ 。由于 LC 中的数据元素或是 LA 中的元素，或是 LB 中的元素，因此只要先设 LC 为空表，然后将 LA 或 LB 中的元素逐个插入到 LC 中即可。为使 LC 中的元素按值非递减有序排列，可设两个指针 pa 和 pb 分别指向 LA 和 LB 中的某个元素，若设 pa 当前所指的元素为 a，pb 当前所指的元素为 b，则当前应插入到 LC 中的元素 c 为

$$c = \begin{cases} a & \text{当 } a \leq b \\ b & \text{当 } a > b \end{cases}$$

显然，指针 pa 和 pb 的初值分别指向两个有序表的第一个元素，在所指元素插入 LC 之后，在 LA 或 LB 中顺序后移。

根据上述分析，分别给出有序表的顺序存储结构和链式存储结构相应合并算法的实现。

### 1. 顺序有序表的合并

#### 算法 2.16 顺序有序表的合并

【算法步骤】

- ① 创建一个表长为  $m+n$  的空表 LC。
- ② 指针 pc 初始化，指向 LC 的第一个元素。
- ③ 指针 pa 和 pb 初始化，分别指向 LA 和 LB 的第一个元素。
- ④ 当指针 pa 和 pb 均未到达相应表尾时，则依次比较 pa 和 pb 所指向的元素值，从 LA 或 LB 中“摘取”元素值较小的结点插入到 LC 的最后。
- ⑤ 如果 pb 已到达 LB 的表尾，依次将 LA 的剩余元素插入 LC 的最后。
- ⑥ 如果 pa 已到达 LA 的表尾，依次将 LB 的剩余元素插入 LC 的最后。

## 【算法描述】

```

void MergeList_Sq(SqList LA, SqList LB, SqList &LC)
{ // 已知顺序有序表 LA 和 LB 的元素按值非递减排列
  // 归并 LA 和 LB 得到新的顺序有序表 LC, LC 的元素也按值非递减排列
  LC.length = LA.length + LB.length;           // 新表长度为待合并两表的长度之和
  LC.elem = new ElemType[LC.length];           // 为合并后的新表分配一个数组空间
  pc = LC.elem;                                 // 指针 pc 指向新表的第一个元素
  pa = LA.elem; pb = LB.elem;                   // 指针 pa 和 pb 的初值分别指向两个表的第一个元素
  pa_last = LA.elem + LA.length - 1;           // 指针 pa_last 指向 LA 的最后一个元素
  pb_last = LB.elem + LB.length - 1;           // 指针 pb_last 指向 LB 的最后一个元素
  while ((pa <= pa_last) && (pb <= pb_last))    // LA 和 LB 均未到达表尾
  {
    if (*pa <= *pb) *pc++ = *pa++;             // 依次“摘取”两表中值较小的结点插入到 LC 的最后
    else *pc++ = *pb++;
  }
  while (pa <= pa_last) *pc++ = *pa++;         // LB 已到达表尾, 依次将 LA 的剩余元素插入 LC 的最后
  while (pb <= pb_last) *pc++ = *pb++;         // LA 已到达表尾, 依次将 LB 的剩余元素插入 LC 的最后
}

```

## 【算法分析】

若对算法 2.16 中第一个循环语句的循环体做如下修改：分出元素比较的第三种情况，当  $*pa == *pb$  时，只将两者中之一插入 LC，则该算法完成的操作和算法 2.15 相同，但时间复杂度却不同。在算法 2.16 中，由于 LA 和 LB 中元素依值非递减，则对 LB 中的每个元素，不需要在 LA 中从表头至表尾进行全程搜索。如果两个表长分别记为  $m$  和  $n$ ，则算法 2.16 循环最多执行的总次数为  $m + n$ 。所以算法的时间复杂度为  $O(m + n)$ 。

此算法在归并时，需要开辟新的辅助空间，所以空间复杂度也为  $O(m + n)$ ，空间复杂度较高。利用链表来实现上述归并时，不需要开辟新的存储空间，可以使空间复杂度达到最低。

## 2. 链式有序表的合并

假设头指针为 LA 和 LB 的单链表分别为线性表 LA 和 LB 的存储结构，现要归并 LA 和 LB 得到单链表 LC。因为链表结点之间的关系是通过指针指向建立起来的，所以用链表进行合并不需要另外开辟存储空间，可以直接利用原来两个表的存储空间，合并过程中只需把 LA 和 LB 两个表中的结点重新进行链接即可。

按照例 2.2 给出的合并思想，需设立 3 个指针 pa、pb 和 pc，其中 pa 和 pb 分别指向 LA 和 LB 中当前待比较插入的结点，而 pc 指向 LC 中当前最后一个结点（LC 的表头结点设为 LA 的表头结点）。指针的初值为：pa 和 pb 分别指向 LA 和 LB 表中的第一个结点，pc 指向空表 LC 中的头结点。同算法 2.16 一样，通过比较指针 pa 和 pb 所指向的元素的值，依次从 LA 或 LB 中“摘取”元素值较小的结点插入到 LC 的最后，当其中一个表变空时，只要将另一个表的剩余段链接在 pc 所指结点之后即可。

## 算法 2.17 链式有序表的合并

## 【算法步骤】

- ① 指针 pa 和 pb 初始化，分别指向 LA 和 LB 的第一个结点。
- ② LC 的结点取值为 LA 的头结点。
- ③ 指针 pc 初始化，指向 LC 的头结点。

④ 当指针 pa 和 pb 均未到达相应表尾时，则依次比较 pa 和 pb 所指向的元素值，从 LA 或 LB 中“摘取”元素值较小的结点插入到 LC 的最后。

⑤ 将非空表的剩余段插入到 pc 所指结点之后。

⑥ 释放 LB 的头结点。

#### 【算法描述】

```
void MergeList_L(LinkList &LA, LinkList &LB, LinkList &LC)
{ // 已知单链表 LA 和 LB 的元素按值非递减排列
  // 归并 LA 和 LB 得到新的单链表 LC，LC 的元素也按值非递减排列
  pa=LA->next; pb=LB->next;           // pa 和 pb 的初值分别指向两个表的第一个结点
  LC=LA;                               // 用 LA 的头结点作为 LC 的头结点
  pc=LC;                               // pc 的初值指向 LC 的头结点
  while (pa && pb)
  { // LA 和 LB 均未到达表尾，依次“摘取”两表中值较小的结点插入到 LC 的最后
    if (pa->data <= pb->data)           // “摘取” pa 所指结点
    {
      pc->next=pa;                     // 将 pa 所指结点链接到 pc 所指结点之后
      pc=pa;                           // pc 指向 pa
      pa=pa->next;                     // pa 指向下一结点
    }
    else                               // “摘取” pb 所指结点
    {
      pc->next=pb;                     // 将 pb 所指结点链接到 pc 所指结点之后
      pc=pb;                           // pc 指向 pb
      pb=pb->next;                     // pb 指向下一结点
    }
  }
  pc->next=pa?pa:pb;                   // 将非空表的剩余段插入到 pc 所指结点之后
  delete LB;                           // 释放 LB 的头结点
}
```

#### 【算法分析】

可以看出，算法 2.17 的时间复杂度和算法 2.16 相同，但空间复杂度不同。在归并两个链表为一个链表时，不需要另建新表的结点空间，而只需将原来两个链表中结点之间的关系解除，重新按元素值非递减的关系将所有结点链接成一个链表即可，所以空间复杂度为  $O(1)$ 。

## 2.8 案例分析与实现

在 2.2 节我们通过 3 个典型案例引入了线性表这种数据结构，本节结合线性表的基本操作对这 3 个案例作进一步的分析，然后给出案例中有关算法的具体实现。

#### 案例 2.1：一元多项式的运算。

#### 【案例分析】

由 2.2 节的讨论我们已知，一元多项式可以抽象成一个线性表。在计算机中，我们可以采用

数组来表示一元多项式的线性表。

利用数组  $p$  表示：数组中每个分量  $p[i]$  表示多项式每项的系数  $p_i$ ，数组分量的下标  $i$  即对应每项的指数。数组中非零的分量个数即为多项式的项数。

例如，多项式  $P(x) = 10 + 5x - 4x^2 + 3x^3 + 2x^4$  可以用表 2.1 所示的数组表示。

表 2.1 多项式的数组表示

| 指数 (下标 $i$ ) | 0  | 1 | 2  | 3 | 4 |
|--------------|----|---|----|---|---|
| 系数 $p[i]$    | 10 | 5 | -4 | 3 | 2 |

显然，利用上述方法表示一元多项式，多项式相加的算法很容易实现，只要把两个数组对应的分量项相加就可以了。

### 案例 2.2：稀疏多项式的运算。

#### 【案例分析】

由 2.2 节的讨论我们已知，稀疏多项式也可以抽象成一个线性表。结合 2.7 节介绍的两个有序表的归并方法，可以看出，稀疏多项式的相加过程和归并两个有序表的过程极其类似，不同之处仅在于，后者在比较数据元素时只出现两种情况（小于等于、大于），而多项式的相加过程在比较两个多项式指数时要考虑三种情况（等于、小于、大于）。因此，多项式相加的过程可以根据算法 2.16 和算法 2.17 改进而成。

和顺序存储结构相比，利用链式存储结构更加灵活，更适合表示一般的多项式，合并过程的空间复杂度为  $O(1)$ ，所以较为常用。本节将给出如何利用单链表的基本操作来实现多项式的相加运算。

例如，图 2.22 所示两个链表分别表示多项式  $A(x) = 7 + 3x + 9x^8 + 5x^{17}$  和多项式  $B(x) = 8x + 22x^7 - 9x^8$ 。从图中可见，每个结点表示多项式中的一项。

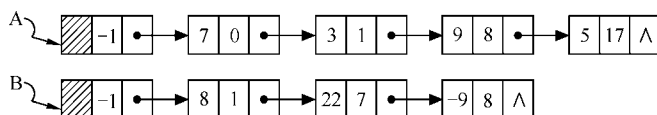


图 2.22 多项式的单链表存储结构

如何实现用这种单链表表示的多项式的加法运算呢？

根据多项式相加的运算规则：对于两个多项式中所有指数相同的项，对应系数相加，若其和不为零，则作为“和多项式”中的一项插入到“和多项式”链表中去；对于两个多项式中指数不相同的项，则将指数值较小的项插入到“和多项式”链表中去。“和多项式”链表中的结点无需生成，而应该从两个多项式的链表中摘取。图 2.22 所示的两个多项式相加的结果如图 2.23 所示，图中的长方框表示已被释放的结点。

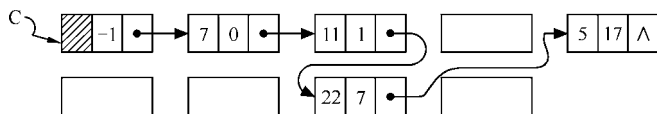


图 2.23 相加得到的和多项式

## 【案例实现】

用链表表示多项式时，每个链表结点存储多项式中的一个非零项，包括系数（coef）和指数（expn）两个数据域以及一个指针域（next）。对应的数据结构定义为：

```
typedef struct PNode
{
    float coef;           //系数
    int expn;             //指数
    struct PNode *next;   //指针域
}PNode,*Polynomial;
```

一个多项式可以表示成由这些结点链接起来的单链表，要实现多项式的相加运算，首先需要创建多项式链表。

## 1. 多项式的创建

多项式的创建方法类似于链表的创建方法，区别在于多项式链表是一个有序表，每项的位置要经过比较才能确定。首先初始化一个空链表用来表示多项式，然后逐个输入各项，通过比较，找到第一个大于该输入项指数的项，将输入项插到此项的前面，这样即可保证多项式链表的有序性。

## 算法 2.18 多项式的创建

## 【算法步骤】

- ① 创建一个只有头结点的空链表。
- ② 根据多项式的项的个数  $n$ ，循环  $n$  次执行以下操作：
  - 生成一个新结点\*s；
  - 输入多项式当前项的系数和指数赋给新结点\*s的数据域；
  - 设置一前驱指针 pre，用于指向待找到的第一个大于输入项指数的结点的前驱，pre 初值指向头结点；
  - 指针 q 初始化，指向首元结点；
  - 循链向下逐个比较链表中当前结点与输入项指数，找到第一个大于输入项指数的结点\*q；
  - 将输入项结点\*s 插入到结点\*q 之前。

## 【算法描述】

```
void CreatePolyn(Polynomial &P,int n)
{//输入 m 项的系数和指数，建立表示多项式的有序链表 P
    P=new PNode;
    P->next=NULL;           //先建立一个带头结点的单链表
    for(i=1;i<=n;++i)       //依次输入 n 个非零项
    {
        s=new PNode;       //生成新结点
        cin>>s->coef>>s->expn; //输入系数和指数
        pre=P;             //pre 用于保存 q 的前驱，初值为头结点
        q=P->next;         //q 初始化，指向首元结点
        while (q&&q->expn<s->expn) //通过比较指数找到第一个大于输入项指数的项*q
        {
            pre=q;
            q=q->next;
        }
        s->next=q;          //将输入项 s 插入到 q 和其前驱结点 pre 之间
        pre->next=s;
    }
```

```

    }
    //for
}

```

### 【算法分析】

创建一个项数为  $n$  的有序多项式链表, 需要执行  $n$  次循环逐个输入各项, 而每次循环又都需要从前向后比较输入项与各项的指数。在最坏情况下, 第  $n$  次循环需要作  $n$  次比较, 因此, 时间复杂度为  $O(n^2)$ 。

### 2. 多项式的相加

创建两个多项式链表后, 便可以进行多项式的加法运算了。假设头指针为 Pa 和 Pb 的单链表分别为多项式 A 和 B 的存储结构, 指针 p1 和 p2 分别指向 A 和 B 中当前进行比较的某个结点, 则逐一比较两个结点中的指数项, 对于指数相同的项, 对应系数相加, 若其和不为零, 则将插入到“和多项式”链表中去; 对于指数不相同的项, 则通过比较将指数值较小的项插入到“和多项式”链表中去。

#### 算法 2.19 多项式的相加

##### 【算法步骤】

- ① 指针 p1 和 p2 初始化, 分别指向 Pa 和 Pb 的首元结点。
- ② p3 指向和多项式的当前结点, 初值为 Pa 的头结点。
- ③ 当指针 p1 和 p2 均未到达相应表尾时, 则循环比较 p1 和 p2 所指结点对应的指数值 (p1->expn 与 p2->expn), 有下列 3 种情况:

- 当 p1->expn 等于 p2->expn 时, 则将两个结点中的系数相加, 若和不为零, 则修改 p1 所指结点的系数值, 同时删除 p2 所指结点, 若和为零, 则删除 p1 和 p2 所指结点;
- 当 p1->expn 小于 p2->expn 时, 则应摘取 p1 所指结点插入到“和多项式”链表中去;
- 当 p1->expn 大于 p2->expn 时, 则应摘取 p2 所指结点插入到“和多项式”链表中去。

- ④ 将非空多项式的剩余段插入到 p3 所指结点之后。

- ⑤ 释放 Pb 的头结点。

##### 【算法描述】

```

void AddPolyn(Polynomial &Pa, Polynomial &Pb)
{ //多项式加法: Pa=Pa+Pb, 利用两个多项式的结点构成“和多项式”
    p1=Pa->next; p2=Pb->next;           //p1 和 p2 初值分别指向 Pa 和 Pb 的首元结点
    p3=Pa;                               //p3 指向和多项式的当前结点, 初值为 Pa
    while (p1&& p2)                      //p1 和 p2 均非空
    {
        if (p1->expn==p2->expn)           //指数相等
        {
            sum=p1->coef+p2->coef;         //sum 保存两项的系数和
            if (sum!=0)                   //系数和不为 0
            {
                p1->coef=sum;              //修改 Pa 当前结点的系数值为两项系数的和
                p3->next=p1; p3=p1;        //将修改后的 Pa 当前结点链在 p3 之后, p3 指向 p1
                p1=p1->next;               //p1 指向后一项
                r=p2; p2=p2->next; delete r; //删除 Pb 当前结点, p2 指向后一项
            }
        }
        else                               //系数和为 0
    }
}

```

```

    {
        r=p1; p1=p1->next; delete r; //删除 Pa 当前结点, p1 指向后一项
        r=p2; p2=p2->next; delete r; //删除 Pb 当前结点, p2 指向后一项
    }
}
else if(p1->expn<p2->expn) //Pa 当前结点的指数值小
{
    p3->next=p1; //将 p1 链在 p3 之后
    p3=p1; //p3 指向 p1
    p1=p1->next; //p1 指向后一项
}
else //Pb 当前结点的指数值小
{
    p3->next=p2; //将 p2 链在 p3 之后
    p3=p2; //p3 指向 p2
    p2=p2->next; //p2 指向后一项
}
} //while
p3->next=p1?p1:p2; //插入非空多项式的剩余段
delete Pb; //释放 Pb 的头结点
}

```

### 【算法分析】

假设两个多项式的项数分别为  $m$  和  $n$ , 则同算法 2.17 一样, 该算法的时间复杂度为  $O(m+n)$ , 空间复杂度为  $O(1)$ 。

对于两个一元多项式减法和乘法的运算, 都可以利用多项式加法的算法来实现。减法运算比较简单, 只需要先对要减的多项式的每项系数进行取反, 然后再调用加法运算 AddPolyn 即可。多项式的乘法运算可以分解为一系列的加法运算。假设  $A(x)$  和  $B(x)$  为式 (2-1) 的多项式, 则

$$\begin{aligned}
 M(x) &= A(x) \times B(x) \\
 &= A(x) \times [b_1x^{e_1} + b_2x^{e_2} + \cdots + b_nx^{e_n}] \\
 &= \sum_{i=1}^n b_i A(x)x^{e_i}
 \end{aligned}$$

其中, 每一项都是一个一元多项式。

多项式相加的例子说明, 对于一些有规律的数学运算, 借助链表实现是一种解决问题的途径。

### 案例 2.3: 图书信息管理系统。

#### 【案例分析】

把图书表抽象成一个线性表, 每本图书 (包括 ISBN、书名、定价) 作为线性表中的一个元素。在图书信息管理系统中要求实现查找、插入、删除、修改、排序和计数总计 6 个功能, 具体分析如下。

(1) 对于查找、插入、删除这 3 个功能的算法, 本章已分别给出了线性表利用顺序存储结构和链式存储结构表示时相应的算法描述。

(2) 对于修改功能, 可以通过调用查找算法, 找到满足条件的图书进行修改即可。

(3) 对于排序功能, 如果在没有时间复杂度限制的情况下, 可以采用读者熟悉的冒泡排序来

完成；如果图书数目较多，对排序算法的时间效率要求较高，在学完第 8 章的内部排序算法后，可以选取一种较高效的排序算法来实现，例如，快速排序。

（4）对于计数功能，如果采取顺序存储结构，线性表的长度是它的属性，可以直接通过返回 `length` 的值实现图书个数的统计功能，时间复杂度是  $O(1)$ ；如果采取链式存储结构，则需要通过从首元结点开始，附设一个计数器进行计数，一直“数”到最后一个结点，时间复杂度是  $O(n)$ 。

在实现图书信息管理系统时，具体采取哪种存储结构，可以根据实际情况而定。如果图书数据较多，需要频繁地进行插入和删除操作，则宜采取链表表示；反之，如果图书数据个数变化不大，很少进行插入和删除操作，则宜采取顺序表表示。

此案例中所涉及的算法比较基础，但非常重要，读者可以分别采用顺序表和链表实现此案例的相应功能，作为本章内容的实验题目来完成。

## 2.9 小 结

线性表是整个数据结构课程的重要基础，本章主要内容如下。

（1）线性表的逻辑结构特性是指数据元素之间存在着线性关系，在计算机中表示这种关系的两类不同的存储结构是顺序存储结构（顺序表）和链式存储结构（链表）。

（2）对于顺序表，元素存储的相邻位置反映出其逻辑上的线性关系，可借助数组来表示。给定数组的下标，便可以存取相应的元素，可称为随机存取结构。而对于链表，是依靠指针来反映其线性逻辑关系的，链表结点的存取都要从头指针开始，顺链而行，所以不属于随机存取结构，可称之为顺序存取结构。不同的特点使得顺序表和链表有不同的适用情况，表 2.2 分别从空间、时间和适用情况 3 方面对二者进行了比较。

表 2.2 顺序表和链表的比较

| 存储结构<br>比较项目 |       | 顺序表                                                   | 链表                              |
|--------------|-------|-------------------------------------------------------|---------------------------------|
| 空间           | 存储空间  | 预先分配，会导致空间闲置或溢出现象                                     | 动态分配，不会出现存储空间闲置或溢出现象            |
|              | 存储密度  | 不用为表示结点间的逻辑关系而增加额外的存储开销，存储密度等于 1                      | 需要借助指针来体现元素间的逻辑关系，存储密度小于 1      |
| 时间           | 存取元素  | 随机存取，按位置访问元素的时间复杂度为 $O(1)$                            | 顺序存取，按位置访问元素时间复杂度为 $O(n)$       |
|              | 插入、删除 | 平均移动约表中一半元素，时间复杂度为 $O(n)$                             | 不需移动元素，确定插入、删除位置后，时间复杂度为 $O(1)$ |
| 适用情况         |       | ① 表长变化不大，且能事先确定变化的范围<br>② 很少进行插入或删除操作，经常按元素位置序号访问数据元素 | ① 长度变化较大<br>② 频繁进行插入或删除操作       |

（3）对于链表，除了常用的单链表外，在本章还讨论了两种不同形式的链表，即循环链表和

双向链表，它们有不同的应用场合。表 2.3 对三者的几项有差别的基本操作进行了比较。

## 单链表、循环链表和双向链表的比较

| 操作名称<br>链表名称       | 查找表头结点                  | 查找表尾结点                           | 查找结点 *p 的前驱结点                      |
|--------------------|-------------------------|----------------------------------|------------------------------------|
| 带头结点的单链表<br>L      | L->next<br>时间复杂度 $O(1)$ | 从 L->next 依次向后遍历<br>时间复杂度 $O(n)$ | 通过 p->next 无法找到其前驱                 |
| 带头结点仅设头指针 L 的循环单链表 | L->next<br>时间复杂度 $O(1)$ | 从 L->next 依次向后遍历<br>时间复杂度 $O(n)$ | 通过 p->next 可以找到其前驱<br>时间复杂度 $O(n)$ |
| 带头结点仅设尾指针 R 的循环单链表 | R->next<br>时间复杂度 $O(1)$ | R<br>时间复杂度 $O(1)$                | 通过 p->next 可以找到其前驱<br>时间复杂度 $O(n)$ |
| 带头结点的双向循环链表 L      | L->next<br>时间复杂度 $O(1)$ | L->prior<br>时间复杂度 $O(1)$         | p->prior<br>时间复杂度 $O(1)$           |

学习完本章后，应熟练掌握顺序表和链表的查找、插入和删除算法、链表的创建算法，并能够设计出线性表应用的常用算法，比如线性表的合并等。要求能够从时间和空间复杂度的角度比较两种存储结构的不同特点及其适用场合，明确它们各自的优缺点。

## 习 题

### 1. 选择题

(1) 顺序表中第一个元素的存储地址是 100，每个元素的长度为 2，则第 5 个元素的地址是 ( )。

- A. 110  
B. 108  
C. 100  
D. 120

(2) 在含  $n$  个结点的顺序表中, 算法的时间复杂度是  $O(1)$  的操作是 ( )。

- A. 访问第  $i$  个结点 ( $1 \leq i \leq n$ ) 和求第  $i$  个结点的直接前驱 ( $2 \leq i \leq n$ )
- B. 在第  $i$  个结点后插入一个新结点 ( $1 \leq i \leq n$ )
- C. 删除第  $i$  个结点 ( $1 \leq i \leq n$ )
- D. 将  $n$  个结点从小到大排序

(3) 在一个有 127 个元素的顺序表中插入一个新元素并保持原来顺序不变, 平均要移动的元素个数为 ( )。

- A. 8  
B. 63.5  
C. 63  
D. 7

(4) 链接存储的存储结构所占存储空间 ( )。

- A. 分为两部分, 一部分存放结点值, 另一部分存放表示结点间关系的指针  
B. 只有一部分, 存放结点值  
C. 只有一部分, 存储表示结点间关系的指针  
D. 分两部分, 一部分存放结点值, 另一部分存放结点所占单元数

(5) 线性表若采用链式存储结构, 要求内存中可用存储单元的地址 ( )。

- A. 必须是连续的  
B. 部分地址必须是连续的  
C. 一定是不连续的  
D. 连续或不连续都可以
- (6) 线性表 L 在 ( ) 情况下适用于使用链式结构实现。  
A. 需经常修改 L 中的结点的值  
B. 需不断对 L 进行删除、插入  
C. L 中含有大量的结点  
D. L 中结点结构复杂
- (7) 单链表的存储密度 ( )。  
A. 大于 1  
B. 等于 1  
C. 小于 1  
D. 不能确定
- (8) 将两个各有  $n$  个元素的有序表归并成一个有序表, 其最少的比较次数是 ( )。  
A.  $n$   
B.  $2n - 1$   
C.  $2n$   
D.  $n - 1$
- (9) 在一个长度为  $n$  的顺序表中, 在第  $i$  个元素 ( $1 \leq i \leq n + 1$ ) 之前插入一个新元素时需向后移动 ( ) 个元素。  
A.  $n - i$   
B.  $n - i + 1$   
C.  $n - i - 1$   
D.  $i$
- (10) 线性表  $L = (a_1, a_2, \dots, a_n)$ , 下列陈述正确的是 ( )。  
A. 每个元素都有一个直接前驱和一个直接后继  
B. 线性表中至少有一个元素  
C. 表中诸元素的排列必须是由小到大或由大到小  
D. 除第一个和最后一个元素外, 其余每个元素都有一个且仅有一个直接前驱和直接后继
- (11) 创建一个包括  $n$  个结点的有序单链表的时间复杂度是 ( )。  
A.  $O(1)$   
B.  $O(n)$   
C.  $O(n^2)$   
D.  $O(n \log_2 n)$
- (12) 以下陈述错误的是 ( )。  
A. 求表长、定位这两种运算在采用顺序存储结构时实现的效率不比采用链式存储结构时实现的效率低  
B. 顺序存储的线性表可以随机存取  
C. 由于顺序存储要求连续的存储区域, 所以在存储管理上不够灵活  
D. 线性表的链式存储结构优于顺序存储结构
- (13) 在单链表中, 要将  $s$  所指结点插入到  $p$  所指结点之后, 其语句应为 ( )。  
A.  $s \rightarrow \text{next} = p + 1; p \rightarrow \text{next} = s;$   
B.  $(*p). \text{next} = s; (*s). \text{next} = (*p). \text{next};$   
C.  $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s \rightarrow \text{next};$   
D.  $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$
- (14) 在双向链表存储结构中, 删除  $p$  所指结点时修改指针的操作为 ( )。  
A.  $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}; p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next};$   
B.  $p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next}; p \rightarrow \text{next} \rightarrow \text{prior} = p;$   
C.  $p \rightarrow \text{prior} \rightarrow \text{next} = p; p \rightarrow \text{prior} = p \rightarrow \text{prior} \rightarrow \text{prior};$   
D.  $p \rightarrow \text{prior} = p \rightarrow \text{next} \rightarrow \text{next}; p \rightarrow \text{next} = p \rightarrow \text{prior} \rightarrow \text{prior};$

(15) 在双向循环链表中, 在  $p$  指针所指的结点后插入  $q$  所指向的新结点, 其修改指针的操作是 ( )。

- A.  $p \rightarrow \text{next} = q; q \rightarrow \text{prior} = p; p \rightarrow \text{next} \rightarrow \text{prior} = q; q \rightarrow \text{next} = q;$
- B.  $p \rightarrow \text{next} = q; p \rightarrow \text{next} \rightarrow \text{prior} = q; q \rightarrow \text{prior} = p; q \rightarrow \text{next} = p \rightarrow \text{next};$
- C.  $q \rightarrow \text{prior} = p; q \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} \rightarrow \text{prior} = q; p \rightarrow \text{next} = q;$
- D.  $q \rightarrow \text{prior} = p; q \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = q; p \rightarrow \text{next} \rightarrow \text{prior} = q;$

## 2. 算法设计题

(1) 将两个递增的有序链表合并为一个递增的有序链表。要求结果链表仍使用原来两个链表的存储空间, 不另外占用其他的存储空间。表中不允许有重复的数据。

(2) 将两个非递减的有序链表合并为一个非递增的有序链表。要求结果链表仍使用原来两个链表的存储空间, 不另外占用其他的存储空间。表中允许有重复的数据。

(3) 已知两个链表  $A$  和  $B$  分别表示两个集合, 其元素递增排列。请设计一个算法, 用于求出  $A$  与  $B$  的交集, 并存放在  $A$  链表中。

(4) 已知两个链表  $A$  和  $B$  分别表示两个集合, 其元素递增排列。请设计算法求出两个集合  $A$  和  $B$  的差集 (即仅由在  $A$  中出现而不在  $B$  中出现的元素所构成的集合), 并以同样的形式存储, 同时返回该集合的元素个数。

(5) 设计算法将一个带头结点的单链表  $A$  分解为两个具有相同结构的链表  $B$  和  $C$ , 其中  $B$  表的结点为  $A$  表中值小于零的结点, 而  $C$  表的结点为  $A$  表中值大于零的结点 (链表  $A$  中的元素为非零整数, 要求  $B$ 、 $C$  表利用  $A$  表的结点)。

(6) 设计一个算法, 通过一趟遍历确定长度为  $n$  的单链表中值最大的结点。

(7) 设计一个算法, 将链表中所有结点的链接方向“原地”逆转, 即要求仅利用原表的存储空间, 换句话说, 要求算法的空间复杂度为  $O(1)$ 。

(8) 设计一个算法, 删除递增有序链表中值大于  $\text{mink}$  且小于  $\text{maxk}$  的所有元素 ( $\text{mink}$  和  $\text{maxk}$  是给定的两个参数, 其值可以和表中的元素相同, 也可以不同)。

(9) 已知  $p$  指向双向循环链表中的一个结点, 其结点结构为  $\text{data}$ 、 $\text{prior}$ 、 $\text{next}$  三个域, 写出算法  $\text{change}(p)$ , 交换  $p$  所指向的结点及其前驱结点的顺序。

(10) 已知长度为  $n$  的线性表  $A$  采用顺序存储结构, 请写一个时间复杂度为  $O(n)$ 、空间复杂度为  $O(1)$  的算法, 该算法可删除线性表中所有值为  $\text{item}$  的数据元素。

# 第3章

## 栈和队列

栈和队列是两种重要的线性结构。从数据结构角度看，栈和队列也是线性表，其特殊性在于栈和队列的基本操作是线性表操作的子集，它们是操作受限的线性表，因此，可称为限定性的数据结构。但从数据类型角度看，它们是和线性表不相同的两类重要的抽象数据类型。本章除了讨论栈和队列的定义、表示方法和实现外，还将给出一些应用的例子。

### 3.1 栈和队列的定义和特点

#### 3.1.1 栈的定义和特点

栈（stack）是限定仅在表尾进行插入或删除操作的线性表。因此，对栈来说，表尾端有其特殊含义，称为**栈顶**（top），相应地，表头端称为**栈底**（bottom）。不含元素的空表称为**空栈**。

假设栈  $S = (a_1, a_2, \dots, a_n)$ ，则称  $a_1$  为栈底元素， $a_n$  为栈顶元素。栈中元素按  $a_1, a_2, \dots, a_n$  的次序进栈，退栈的第一个元素应为栈顶元素。换句话说，栈的修改是按后进先出的原则进行的，如图 3.1（a）所示。因此，栈又称为**后进先出**（Last In First Out, LIFO）的线性表，它的这个特点可用图 3.1（b）所示的铁路调度站形象地表示。

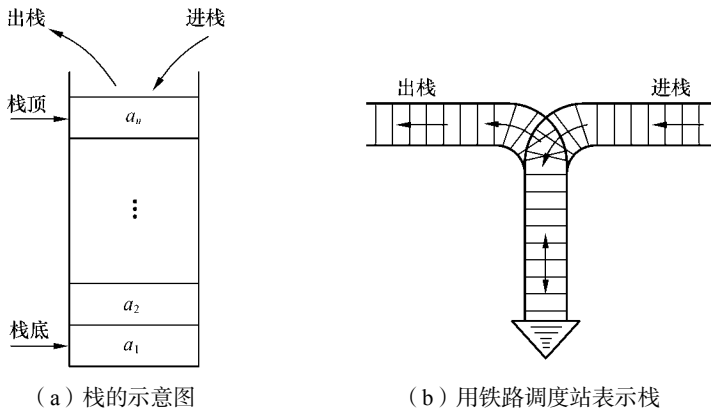


图 3.1 栈

在日常生活中，还有很多类似栈的例子。例如，洗干净的盘子总是逐个往上叠放在已经洗好的盘子上面，而用时从上往下逐个取用。栈的操作特点正是上述实际应用的抽象。在程序设计中，

如果需要按照保存数据时相反的顺序来使用数据，则可以利用栈来实现。

### 3.1.2 队列的定义和特点

和栈相反，队列（queue）是一种先进先出（First In First Out, FIFO）的线性表。它只允许在表的一端进行插入，而在另一端删除元素。这和日常生活中的排队是一致的，最早进入队列的元素最早离开。在队列中，允许插入的一端称为队尾（rear），允许删除的一端则称为队头（front）。假设队列为  $q = (a_1, a_2, \dots, a_n)$ ，那么， $a_1$  就是队头元素， $a_n$  则是队尾元素。队列中的元素是按照  $a_1, a_2, \dots, a_n$  的顺序进入的，退出队列也只能按照这个次序依次退出，也就是说，只有在  $a_1, a_2, \dots, a_{n-1}$  都离开队列之后， $a_n$  才能退出队列。图 3.2 所示为队列的示意图。

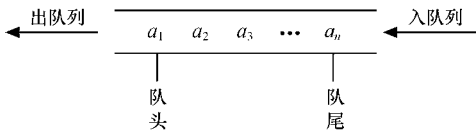


图 3.2 队列的示意图

队列在程序设计中也经常出现。一个最典型的例子就是操作系统中的作业排队。在允许多道程序运行的计算机系统中，同时有几个作业运行。如果运行的结果都需要通过通道输出，那就要按请求输入的先后次序排队。每当通道传输完毕可以接受新的输出任务时，队头的作业先从队列中退出做输出操作。凡是申请输出的作业都从队尾进入队列。

## 3.2 案例引入

### 案例 3.1：数制的转换。

十进制数  $N$  和其他  $d$  进制数的转换是计算机实现计算的基本问题，其解决方法很多，其中一个简单算法基于下列原理：

$$N = (N \div d) \times d + N \bmod d \quad (\text{其中, } \div \text{ 为整除运算, } \bmod \text{ 为求余运算})$$

例如： $(1348)_{10} = (2504)_8$ ，其运算过程如下：

| $N$  | $N \div 8$ | $N \bmod 8$ |
|------|------------|-------------|
| 1348 | 168        | 4           |
| 168  | 21         | 0           |
| 21   | 2          | 5           |
| 2    | 0          | 2           |

假设现要编制一个满足下列要求的程序：对于输入的任意一个非负十进制整数，输出与其等值的八进制数。上述计算过程是从低位到高位顺序产生八进制数的各个数位；而输出过程应从高位到低位进行，恰好和计算过程相反，因而我们可以使用栈来解决这个问题。在计算过程中依次将得到的余数压入栈中，计算完毕后，再依次弹出栈中的余数就是数制转换的结果。

### 案例 3.2：括号匹配的检验。

假设表达式中允许包含两种括号：圆括号和方括号，其嵌套的顺序随意，即  $([ ] ( ))$  或  $( [ ( ] )$  等均为正确的格式， $[ ( ]$  或  $( [ ( ) )$  或  $(( [ ] ))$  均为不正确的格式。检验括号是否匹配的方法可用“期待的急迫程度”这个概念来描述。例如，考虑下列括号序列：

[ ( [ ] [ ] ) ]  
1 2 3 4 5 6 7 8

当计算机接受了第一个括号后，它期待着与其匹配的第八个括号的出现，然而等来的却是第二个括号，显然第二个括号的期待急迫性高于第一个括号，此时第一个括号“[”只能暂时靠边，而迫切等待与第二个括号相匹配的、第七个括号“)”的出现。类似地，因等来的是第三个括号“[”，其期待匹配的程度较第二个括号更急迫，则第二个括号也只能靠边，让位于第三个括号。在接受了第四个括号之后，第三个括号的期待得到满足，消解之后，第二个括号的期待匹配就成为当前最急迫的任务了，……，依次类推。可见，这个处理过程恰与栈的特点相吻合。每读入一个括号，若是右括号，则或者使置于栈顶的最急迫的期待得以消解，或者是不合法的情况；若是左括号，则作为一个新的更急迫的期待压入栈中，自然使原有的在栈中的所有未消解的期待的急迫性都降了一级。

#### 案例 3.3：表达式求值。

表达式求值是程序设计语言编译中的一个最基本问题，其实现是栈应用的又一个典型例子。“算符优先法”，是一种简单直观、广为使用的表达式求值算法。

要把一个表达式翻译成正确求值的一个机器指令序列，或者直接对表达式求值，首先要能够正确解释表达式。算符优先法就是根据算术四则运算规则确定的运算优先关系，实现对表达式的编译或解释执行的。

在表达式计算中先出现的运算符不一定先运算，具体运算顺序是需要通过运算符优先关系的比较，确定合适的运算时机，而运算时机的确定是可以借助栈来完成的。将扫描到的不能进行运算的运算数和运算符先分别压入运算数栈和运算符栈中，在条件满足时再分别从栈中弹出进行运算。

上述 3 个应用实例都是借助栈的后进先出的特性来处理问题的，在日常生活中，符合先进先出特性的应用更为常见。

#### 案例 3.4：舞伴问题。

假设在周末舞会上，男士们和女士们进入舞厅时，各自排成一队。跳舞开始时，依次从男队和女队的队头各出一人配成舞伴。若两队初始人数不相同，则较长的那一队中未配对者等待下一轮舞曲。现要求写一算法模拟上述舞伴配对问题。

先入队的男士或女士应先出队配成舞伴，因此该问题具有典型的先进先出特性，可用队列作为算法的数据结构。

从上面的应用案例可以看出，不论是借助栈还是队列来解决问题，最基本的操作都是“入”和“出”。对于栈，在栈顶插入元素的操作称作“入栈”，删除栈顶元素的操作称作“出栈”；对于队列，在队尾插入元素的操作称作“入队”，在队头删除元素的操作称作“出队”。和线性表一样，栈和队列的存储结构也包括顺序和链式两种。

本章后续章节将依次给出不同存储结构表示的栈和队列的基本操作，并介绍栈的一个非常重要的应用--在程序设计语言中来实现递归，借助栈的基本操作，读者可以深刻理解递归的处理机制。本章最后将利用栈和队列给出上述四个案例的具体实现。

## 3.3 栈的表示和操作的实现

### 3.3.1 栈的类型定义

栈的基本操作除了入栈和出栈外，还有栈的初始化、栈空的判定，以及取栈顶元素等。下面

给出栈的抽象数据类型定义：

**ADT Stack** {

数据对象：  $D = \{a_i | a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0\}$

数据关系：  $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i=2, \dots, n \}$

约定  $a_n$  端为栈顶， $a_1$  端为栈底。

基本操作：

InitStack(&S)

操作结果：构造一个空栈 S。

DestroyStack(&S)

初始条件：栈 S 已存在。

操作结果：栈 S 被销毁。

ClearStack(&S)

初始条件：栈 S 已存在。

操作结果：将 S 清为空栈。

StackEmpty(S)

初始条件：栈 S 已存在。

操作结果：若栈 S 为空栈，则返回 true，否则返回 false。

StackLength(S)

初始条件：栈 S 已存在。

操作结果：返回 S 的元素个数，即栈的长度。

GetTop(S)

初始条件：栈 S 已存在且非空。

操作结果：返回 S 的栈顶元素，不修改栈顶指针。

Push(&S, e)

初始条件：栈 S 已存在。

操作结果：插入元素 e 为新的栈顶元素。

Pop(&S, &e)

初始条件：栈 S 已存在且非空。

操作结果：删除 S 的栈顶元素，并用 e 返回其值。

StackTraverse(S)

初始条件：栈 S 已存在且非空。

操作结果：从栈底到栈顶依次对 S 的每个数据元素进行访问。

}ADT Stack

本书在以后各章中引用的栈大多为如上定义的数据类型，栈的数据元素类型在应用程序内定义。

和线性表类似，栈也有两种存储表示方法，分别称为顺序栈和链栈。

### 3.3.2 顺序栈的表示和实现

顺序栈是指利用顺序存储结构实现的栈，即利用一组地址连续的存储单元依次存放自栈底到栈顶的数据元素，同时附设指针 top 指示栈顶元素在顺序栈中的位置。通常习惯的做法是：以 top = 0 表示空栈，鉴于 C 语言中数组的下标约定从 0 开始，则当以 C 语言作描述语言时，如此设定会带来很大不便，因此另设指针 base 指示栈底元素在顺序栈中的位置。当 top 和 base 的值相等时，表示空栈。顺序栈的定义如下：

```
// - - - - 顺序栈的存储结构 - - - -
#define MAXSIZE 100          // 顺序栈存储空间的初始分配量
typedef struct
```

```
{
    SElemType *base;        //栈底指针
    SElemType *top;         //栈顶指针
    int stacksize;          //栈可用的最大容量
}SqStack;
```



说明

(1) `base` 为栈底指针, 初始化完成后, 栈底指针 `base` 始终指向栈底的位置, 若 `base` 的值为 `NULL`, 则表明栈结构不存在。 `top` 为栈顶指针, 其初值指向栈底。每当插入新的栈顶元素时, 指针 `top` 增 1; 删除栈顶元素时, 指针 `top` 减 1。因此, 栈空时, `top` 和 `base` 的值相等, 都指向栈底; 栈非空时, `top` 始终指向栈顶元素的上一个位置。

(2) `stacksize` 指示栈可使用的最大容量, 后面算法 3.1 的初始化操作为顺序栈动态分配 `MAXSIZE` 大小的数组空间, 将 `stacksize` 置为 `MAXSIZE`。

图 3.3 所示为顺序栈中数据元素和栈指针之间的对应关系。

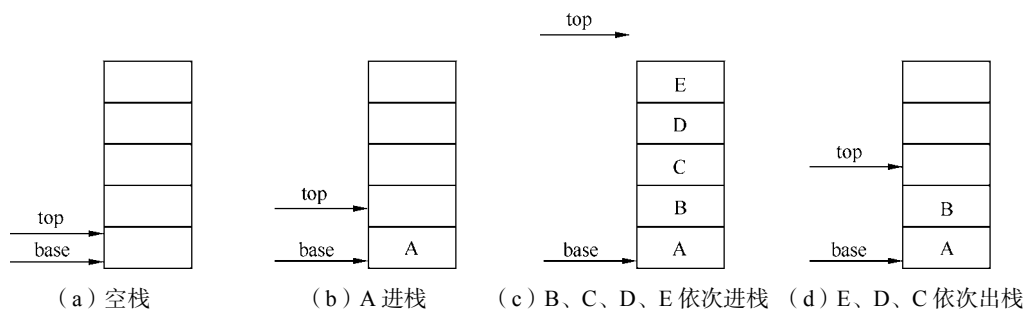


图 3.3 栈中元素和栈指针之间的关系

由于顺序栈的插入和删除只在栈顶进行, 因此顺序栈的基本操作比顺序表要简单得多, 以下给出顺序栈部分操作的实现。

### 1. 初始化

顺序栈的初始化操作就是为顺序栈动态分配一个预定义大小的数组空间。

#### 算法 3.1 顺序栈的初始化

##### 【算法步骤】

① 为顺序栈动态分配一个最大容量为 `MAXSIZE` 的数组空间, 使 `base` 指向这段空间的基地址, 即栈底。

② 栈顶指针 `top` 初始为 `base`, 表示栈为空。

③ `stacksize` 置为栈的最大容量 `MAXSIZE`

##### 【算法描述】

```
Status InitStack(SqStack &S)
{ //构造一个空栈 S
    S.base=new SElemType[MAXSIZE]; //为顺序栈动态分配一个最大容量为 MAXSIZE 的数组空间
    if(!S.base) exit(OVERFLOW);    //存储分配失败
    S.top=S.base;                   //top 初始为 base, 空栈
    S.stacksize=MAXSIZE;           //stacksize 置为栈的最大容量 MAXSIZE
    return OK;
}
```

## 2. 入栈

入栈操作是指在栈顶插入一个新的元素。

### 算法 3.2 顺序栈的入栈

【算法步骤】

- ① 判断栈是否满，若满则返回 ERROR。
- ② 将新元素压入栈顶，栈顶指针加 1。

【算法描述】

```
Status Push(SqStack &S, SElemType e)
{ // 插入元素 e 为新的栈顶元素
    if (S.top == S.base) return ERROR; // 栈满
    *S.top++ = e; // 元素 e 压入栈顶，栈顶指针加 1
    return OK;
}
```

## 3. 出栈

出栈操作是将栈顶元素删除。

### 算法 3.3 顺序栈的出栈

【算法步骤】

- ① 判断栈是否空，若空则返回 ERROR。
- ② 栈顶指针减 1，栈顶元素出栈。

【算法描述】

```
Status Pop(SqStack &S, SElemType &e)
{ 删除 S 的栈顶元素，用 e 返回其值
    if (S.top == S.base) return ERROR; // 栈空
    e = *--S.top; // 栈顶指针减 1，将栈顶元素赋给 e
    return OK;
}
```

## 4. 取栈顶元素

当栈非空时，此操作返回当前栈顶元素的值，栈顶指针保持不变。

### 算法 3.4 取顺序栈的栈顶元素

【算法描述】

```
SElemType GetTop(SqStack S)
{ // 返回 S 的栈顶元素，不修改栈顶指针
    if (S.top != S.base) // 栈非空
        return *(S.top - 1); // 返回栈顶元素的值，栈顶指针不变
}
```

由于顺序栈和顺序表一样，受到最大空间容量的限制，虽然可以在“满员”时重新分配空间扩大容量，但工作量较大，应该尽量避免。因此在应用程序无法预先估计栈可能达到的最大容量时，还是应该使用下面介绍的链栈。

### 3.3.3 链栈的表示和实现

链栈是指采用链式存储结构实现的栈。通常链栈用单链表来表示，如图 3.4 所示。链栈的结

点结构与单链表的结构相同, 在此用 StackNode 表示, 定义如下:

```
// - - - - 链栈的存储结构- - - -
typedef struct StackNode
{
    ElemType    data;
    struct StackNode *next;
}StackNode, *LinkStack;
```

由于栈的主要操作是在栈顶插入和删除, 显然以链表的头部作为栈顶是最方便的, 而且没必要像单链表那样为了操作方便附加一个头结点。

下面给出链栈部分操作的实现。

### 1. 初始化

链栈的初始化操作就是构造一个空栈, 因为没必要设头结点, 所以直接将栈顶指针置空即可。

#### 算法 3.5 链栈的初始化

【算法描述】

```
Status InitStack(LinkStack &S)
{ //构造一个空栈 S, 栈顶指针置空
    S=NULL;
    return OK;
}
```

### 2. 入栈

和顺序栈的入栈操作不同的是, 链栈在入栈前不需要判断栈是否满, 只需要为入栈元素动态分配一个结点空间, 如图 3.5 所示。

#### 算法 3.6 链栈的入栈

【算法步骤】

- ① 为入栈元素 e 分配空间, 用指针 p 指向。
- ② 将新结点数据域置为 e。
- ③ 将新结点插入栈顶。
- ④ 修改栈顶指针为 p。

【算法描述】

```
Status Push(LinkStack &S, SElemType e)
{ //在栈顶插入元素 e
    p=new StackNode;           //生成新结点
    p->data=e;                  //将新结点数据域置为 e
    p->next=S;                  //将新结点插入栈顶
    S=p;                        //修改栈顶指针为 p
    return OK;
}
```

### 3. 出栈

和顺序栈一样, 链栈在出栈前也需要判断栈是否为空, 不同的是, 链栈在出栈后需要释放出栈元素的栈顶空间, 如图 3.6 所示。

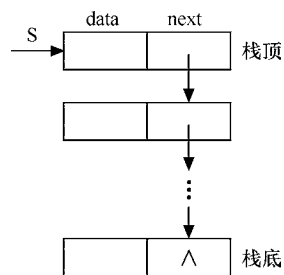


图 3.4 链栈示意图

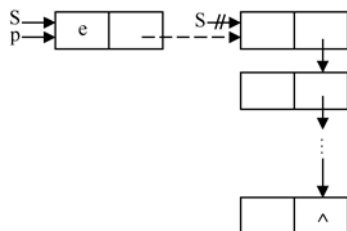


图 3.5 链栈的入栈过程

### 算法 3.7 链栈的出栈

#### 【算法步骤】

- ① 判断栈是否为空，若空则返回 ERROR。
- ② 将栈顶元素赋给 e。
- ③ 临时保存栈顶元素的空间，以备释放。
- ④ 修改栈顶指针，指向新的栈顶元素。
- ⑤ 释放原栈顶元素的空间。

#### 【算法描述】

```

Status Pop(LinkStack &S, SElemType &e)
{ //删除 S 的栈顶元素，用 e 返回其值
    if (S == NULL) return ERROR;           //栈空
    e = S->data;                             //将栈顶元素赋给 e
    p = S;                                   //用 p 临时保存栈顶元素空间，以备释放
    S = S->next;                             //修改栈顶指针
    delete p;                               //释放原栈顶元素的空间
    return OK;
}

```

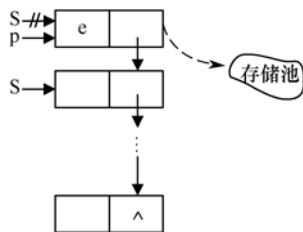


图 3.6 链栈的出栈过程

### 4. 取栈顶元素

与顺序栈一样，当栈非空时，此操作返回当前栈顶元素的值，栈顶指针 S 保持不变。

### 算法 3.8 取链栈的栈顶元素

#### 【算法描述】

```

SElemType GetTop(LinkStack S)
{ //返回 S 的栈顶元素，不修改栈顶指针
    if (S != NULL)                //栈非空
        return S->data;           //返回栈顶元素的值，栈顶指针不变
}

```

## 3.4 栈与递归

栈有一个重要应用是在程序设计语言中实现递归。递归是算法设计中最常用的手段，它通常把一个大型复杂问题的描述和求解变得简洁和清晰。因此递归算法常常比非递归算法更易设计，尤其是当问题本身或所涉及的数据结构是递归定义的时候，使用递归方法更加合适。为使读者增强理解和设计递归算法的能力，本节将介绍栈在递归算法的内部实现中所起的作用。

### 3.4.1 采用递归算法解决的问题

所谓递归是指，若在一个函数、过程或者数据结构定义的内部又直接（或间接）出现定义本身的应用，则称它们是递归的，或者是递归定义的。在以下三种情况下，常常使用递归的方法。

#### 1. 定义是递归的

有很多数学函数是递归定义的，如大家熟悉的阶乘函数

$$\text{Fact}(n) = \begin{cases} 1 & \text{若 } n = 0 \\ n * \text{Fact}(n-1) & \text{若 } n > 0 \end{cases} \quad (3-1)$$

二阶 Fibonacci 数列

$$\text{Fib}(n) = \begin{cases} 1 & \text{若 } n = 1 \text{ 或 } n = 2 \\ \text{Fib}(n-1) + \text{Fib}(n-2) & \text{其他情形} \end{cases} \quad (3-2)$$

对于 (3-1) 中的阶乘函数, 可以使用递归过程来求解。

```
long Fact(long n)
{
    if (n==0) return 1;           //递归终止的条件
    else return n*Fact(n-1);      //递归步骤
}
```

图 3.7 所示为主程序调用函数 Fact(4) 的执行过程。在函数过程体中, else 语句以参数 3、2、1、0 执行递归调用。最后一次递归调用的函数因参数 n 为 0 执行 if 语句, 递归终止, 逐步返回, 返回时依次计算 1\*1、2\*1、3\*2、4\*6, 最后将计算结果 24 返回给主程序。

类似地, 可写出 Fibonacci 数列的递归程序:

```
long Fib(long n)
{
    if (n==1||n==2) return 1;    //递归终止的条件
    else return Fib(n-1)+ Fib(n-2); //递归步骤
}
```

对于类似这种的复杂问题, 若能够分解成几个相对简单且解法相同或类似的子问题来求解, 便称作递归求解。例如, 在图 3.7 中, 计算 4! 时先计算 3!, 然后再进一步分解进行求解, 这种分解-求解的策略叫做“分治法”。

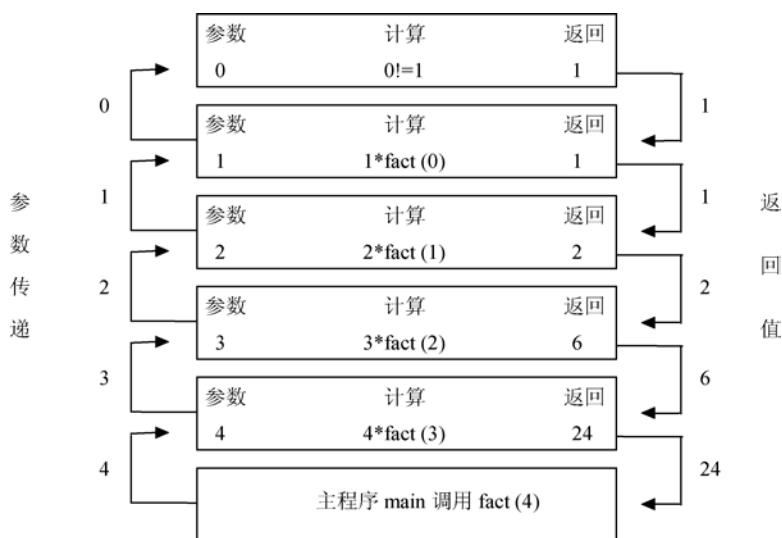


图 3.7 求解 4! 的过程

采取“分治法”进行递归求解的问题需要满足以下三个条件。

(1) 能将一个问题转变成一个新问题，而新问题与原问题的解法相同或类同，不同的仅是处理的对象，并且这些处理对象更小且变化有规律。

(2) 可以通过上述转化而使问题简化。

(3) 必须有一个明确的递归出口，或称递归的边界。

“分治法”求解递归问题算法的一般形式为：

```
void p(参数表)
{
    if(递归结束条件成立)可直接求解;           //递归终止的条件
    else p(较小的参数);                         //递归步骤
}
```

可见，上述阶乘函数和 Fibonacci 数列的递归过程均与此一般形式相对应。

## 2. 数据结构是递归的

某些数据结构本身具有递归的特性，则它们的操作可递归地描述。

例如，对于链表，其结点 LNode 的定义由数据域 data 和指针域 next 组成，而指针域 next 是一种指向 LNode 类型的指针，即 LNode 的定义中又用到了其自身，所以链表是一种递归的数据结构。

对于递归的数据结构，相应算法采用递归的方法来实现特别方便。链表的创建和链表结点的遍历输出都可以采用递归的方法。算法 3.9 是从前向后遍历输出链表结点的递归算法，调用此递归函数前，参数 p 指向单链表的首元结点，在递归过程中，p 不断指向后继结点，直到 p 为 NULL 时递归结束。显然，这个问题满足上述给出的采用“分治法”进行递归求解的问题需要满足的三个条件。

### 算法 3.9 遍历输出链表中各个结点的递归算法

#### 【算法步骤】

- ① 如果 p 为 NULL，递归结束返回。
- ② 否则输出 p->data，p 指向后继结点继续递归。

#### 【算法描述】

```
void TraverseList(LinkList p)
{
    if(p==NULL) return;           //递归终止
    else
    {
        cout<<p->data<<end;       //输出当前结点的数据域
        TraverseList(p->next);     //p 指向后继结点继续递归
    }
}
```

在递归算法中，如果当递归结束条件成立，只执行 return 操作时，“分治法”求解递归问题算法的一般形式可以简化为：

```
void p(参数表)
{
    if(递归结束条件不成立)
        p(较小的参数);
}
```

因此, 算法 3.9 可以简化为:

```
void TraverseList(LinkList p)
{
    if(p)
    {
        cout<<p->data<<end;
        TraverseList(p->next);
    }
}
```

后面章节要介绍的广义表、二叉树等也是典型的具有递归特性的数据结构, 其相应算法也可采用递归的方法来实现。

### 3. 问题的解法是递归的

还有一类问题, 虽然问题本身没有明显的递归结构, 但用递归求解比迭代求解更简单, 如 Hanoi 塔问题、八皇后问题、迷宫问题等。

【例 3.1】  $n$  阶 Hanoi 塔问题。

【问题描述】

假设有 3 个分别命名为 A、B 和 C 的塔座, 在塔座 A 上插有  $n$  个直径大小各不相同, 依从小到大编号为 1, 2,  $\dots$ ,  $n$  的圆盘 (如图 3.8 所示)。现要求将塔座 A 上的  $n$  个圆盘移至塔座 C 上, 并仍按同样顺序叠排, 圆盘移动时必须遵循下列规则:

- (1) 每次只能移动一个圆盘;
- (2) 圆盘可以插在 A、B 和 C 中的任一塔座上;
- (3) 任何时刻都不能将一个较大的圆盘压在较小的圆盘之上。

【问题分析】

如何实现移动圆盘的操作呢? 可以用分治求解的递归方法来解决这个问题。设 A 柱上最初的盘子总数为  $n$ , 则当  $n=1$  时, 只要将编号为 1 的圆盘从塔座 A 直接移至塔座 C 上即可; 否则, 执行以下三步:

- (1) 用 C 柱做过渡, 将 A 柱上的  $(n-1)$  个盘子移到 B 柱上;
- (2) 将 A 柱上最后一个盘子直接移到 C 柱上;
- (3) 用 A 柱做过渡, 将 B 柱上的  $(n-1)$  个盘子移到 C 柱上。

具体移动过程如图 3.8 所示, 图中  $n=4$ 。

根据这种解法, 如何将  $n-1$  个圆盘从一个塔座移至另一个塔座的问题是一个和原问题具有相同特征属性的问题, 只是问题的规模小 1, 因此可以用同样的方法求解。

为了便于描述算法, 将搬动操作定义为  $\text{move}(A, n, C)$ , 是指将编号为  $n$  的圆盘从 A 移到 C, 同时设一个初值为 0 的全局变量  $m$ , 对搬动进行计数:

```
int m=0;
void move(char A,int n,char C)
{
    cout<<"+m<<"<<n<<"<<A<<"<<C<<endl;}
```

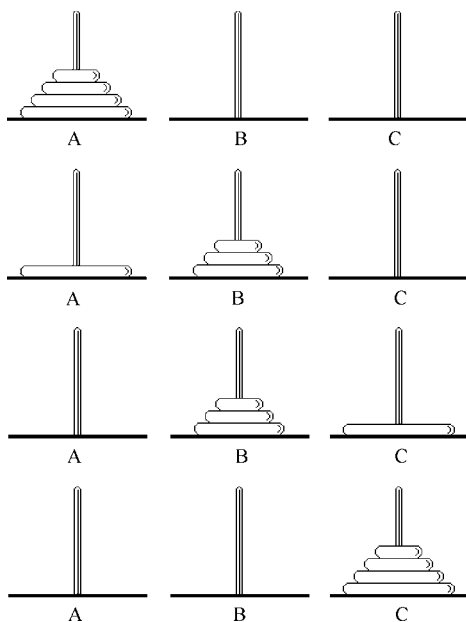


图 3.8 Hanoi 塔问题

### 算法 3.10 Hanoi 塔问题的递归算法

#### 【算法步骤】

- ① 如果  $n=1$ ，则直接将编号为 1 的圆盘从 A 移到 C，递归结束。
- ② 否则：
  - 递归，将 A 上编号为 1 至  $n-1$  的圆盘移到 B，C 做辅助塔；
  - 直接将编号为  $n$  的圆盘从 A 移到 C；
  - 递归，将 B 上编号为 1 至  $n-1$  的圆盘移到 C，A 做辅助塔。

#### 【算法描述】

```
void Hanoi(int n,char A,char B,char C)
{ //将塔座 A 上的 n 个圆盘按规则搬到 C 上, B 做辅助塔
    if(n==1) move(A,1,C);          //将编号为 1 的圆盘从 A 移到 C
    else
    {
        Hanoi(n-1,A,C,B);          //将 A 上编号为 1 至 n-1 的圆盘移到 B, C 做辅助塔
        move(A,n,C);               //将编号为 n 的圆盘从 A 移到 C
        Hanoi(n-1,B,A,C);          //将 B 上编号为 1 至 n-1 的圆盘移到 C, A 做辅助塔
    }
}
```

## 3.4.2 递归过程与递归工作栈

一个递归函数，在函数的执行过程中，需多次进行自我调用。那么，这个递归函数是如何执行的？先看任意两个函数之间进行调用的情形。

与汇编语言程序设计中主程序和子程序之间的链接及信息交换相类似，在高级语言编制的程序中，调用函数和被调用函数之间的链接及信息交换需通过栈来进行。

通常，当在一个函数的运行期间调用另一个函数时，在运行被调用函数之前，系统需先完成 3 件事：

- (1) 将所有的实参、返回地址等信息传递给被调用函数保存；
- (2) 为被调用函数的局部变量分配存储区；
- (3) 将控制转移到被调函数的入口。

而从被调用函数返回调用函数之前，系统也应完成 3 件工作：

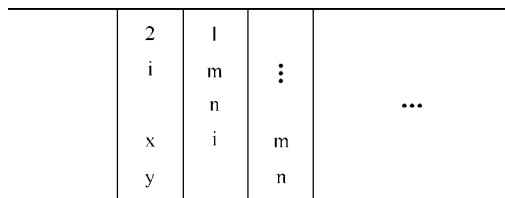
- (1) 保存被调函数的计算结果；
- (2) 释放被调函数的数据区；
- (3) 依照被调函数保存的返回地址将控制转移到调用函数。

当有多个函数构成嵌套调用时，按照“后调用先返回”的原则，上述函数之间的信息传递和控制转移必须通过“栈”来实现，即系统将整个程序运行时所需的数据空间安排在一个栈中，每当调用一个函数时，就为它在栈顶分配一个存储区，每当从一个函数退出时，就释放它的存储区，则当前正运行的函数的数据区必在栈顶。

例如，在图 3.9 (c) 所示的主函数 main 中调用了函数 first，而在函数 first 中又调用了函数 second，则图 3.9 (a) 所示为当前正在执行函数 second 中某个语句时栈的状态，而图 3.9 (b) 展示从函数 second 退出之后正执行函数 first 中某个语句时栈的状态（图中以语句标号表示返回地址）。

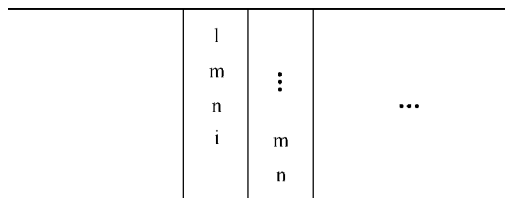
一个递归函数的运行过程类似于多个函数的嵌套调用，只是调用函数和被调用函数是同一

个函数，因此，和每次调用相关的一个重要概念是递归函数运行的“层次”。假设调用该递归函数的主函数为第 0 层，则从主函数调用递归函数为进入第 1 层；从第  $i$  层递归调用本函数为进入“下一层”，即第  $i+1$  层。反之，退出第  $i$  层递归应返回至“上一层”，即第  $i-1$  层。为了保证递归函数正确执行，系统需设立一个“递归工作栈”作为整个递归函数运行期间使用的数据存储区。每一层递归所需信息构成一个工作记录，其中包括所有的实参、所有的局部变量，以及上一层的返回地址。每进入一层递归，就产生一个新的工作记录压入栈顶。每退出一层递归，就从栈顶弹出一个工作记录，则当前执行层的工作记录必是递归工作栈栈顶的工作记录，称这个记录为“活动记录”。



▲  
栈  
顶

(a)



▲  
栈  
顶

(b)

```
void first(int s, int t);
void second(int d);
void main() {
    int m, n;
    ...
    first(m, n);
    1: ...
}
```

```
int first (int s, int t){
    int i;
    ...
    second(i);
    2: ...
}
```

```
int second (int d) {
    int x, y;
    ...
}
```

(c)

图 3.9 主函数 main 执行期间运行栈的状态

下面以图 3.10 所示的阶乘函数 Fact(4)为例，介绍递归过程中递归工作栈和活动记录的使用。主函数调用 Fact(4)，当函数运行结束后，控制返回到 RetLoc1，在此处  $n$  被赋为 24（即  $4!$ ）：

```
void main()
{
    long n;           //调用 Fact (4)时记录进栈
    n=Fact(4);        //返回地址 RetLoc1 在赋值语句
    RetLoc1 _____
}
```

为说明方便起见，将阶乘函数算法改写为：

```
long Fact(long n)
{
    long temp;
    if (n==0) return 1;    //活动记录退栈
    else temp=n*Fact(n-1); //活动记录进栈
    RetLoc2 _____    //返回地址 RetLoc2 在计算语句
```

```
        return temp;           //活动记录退栈
    }
```

这里暂忽略局部变量 temp 的入栈和出栈情况。RetLoc2 是递归调用 Fact ( n-1)的返回地址，当 Fact(n-1)结束后，返回到 RetLoc2，在此处计算  $n*(n-1)!$ ，然后将结果赋给临时变量 temp。

主函数执行后依次启动了 5 个函数调用。图 3.10 所示为每次函数调用时活动记录的进栈情况。主程序外部调用 Fact(4)的活动记录在栈底，Fact (1)调用 Fact (0)进栈的活动记录在栈顶。

递归结束条件出现于函数 Fact(0)的内部，执行 Fact(0)引起了返回语句的执行。退出栈顶的活动记录，返回地址返回到上一层 Fact(1)的调用递归处 RetLoc2，继续执行语句  $temp=1*1$ ，接着执行  $return\ temp$  又引起新的退栈操作。此退栈过程直至 Fact(4)执行完毕后，将控制权转移给 main 为止，其过程如图 3.11 所示。

| 调用者      | 参数表 | 返回地址 |
|----------|-----|------|
| Fact (1) | 0   |      |
| Fact (2) | 1   |      |
| Fact (3) | 2   |      |
| Fact (4) | 3   |      |
| main ( ) | 4   |      |

图 3.10 求解 4!活动记录进栈过程

| 退栈的活动记录 |   |        | 退栈后执行的计算                                                       |
|---------|---|--------|----------------------------------------------------------------|
| 0       | R | etLoc2 | $temp=1*1$ ; //从 Fact (0)返回 1<br>$return\ temp$ ; // $temp=1$  |
| 1       | R | etLoc2 | $temp=2*1$ ; //从 Fact (1)返回 1<br>$return\ temp$ ; // $temp=2$  |
| 2       | R | etLoc2 | $temp=3*2$ ; //从 Fact (2)返回 2<br>$return\ temp$ ; // $temp=6$  |
| 3       | R | etLoc2 | $temp=4*6$ ; //从 Fact (3)返回 6<br>$return\ temp$ ; // $temp=24$ |
| 4       | R | etLoc1 | $n= Fact (4)$ //从 Fact (4)返回 24                                |

图 3.11 求解 4!活动记录退栈过程

### 3.4.3 递归算法的效率分析

#### 1. 时间复杂度的分析

在算法分析中，当一个算法中包含递归调用时，其时间复杂度的分析可以转化为一个递归方程求解。实际上，这个问题是数学上求解渐近阶的问题，而递归方程的形式多种多样，其求解方法也不一而足。迭代法是求解递归方程的一种常用方法，其基本步骤是迭代地展开递归方程的右端，使之成为一个非递归的和式，然后通过和对式的估计来达到对方程左端（即方程的解）的估计。

下面以阶乘的递归函数 Fact(n)为例，说明通过迭代法求解递归方程来计算时间复杂度的方法。

设 Fact(n)的执行时间是  $T(n)$ 。此递归函数中语句  $if(n==0)\ return\ 1$ ;的执行时间是  $O(1)$ ，递归调用 Fact(n-1)的执行时间是  $T(n-1)$ ，所以  $else\ return\ n*Fact(n-1)$ ;的执行时间是  $O(1)+T(n-1)$ 。其中，设两数相乘和赋值操作的执行时间为  $O(1)$ ，则对某常数 C、D 有如下递归方程：

$$T(n)=\begin{cases} D & n=0 \\ C+T(n-1) & n\geq 1 \end{cases}$$

设  $n > 2$ , 利用上式对  $T(n-1)$  展开, 即在上式中用  $n-1$  代替  $n$  得到

$$T(n-1) = C + T(n-2)$$

再代入  $T(n) = C + T(n-1)$  中, 有

$$T(n) = 2C + T(n-2)$$

同理, 当  $n > 3$  时有

$$T(n) = 3C + T(n-3)$$

依次类推, 当  $n > i$  时有

$$T(n) = iC + T(n-i)$$

最后, 当  $i = n$  时有

$$T(n) = nC + T(0) = nC + D$$

求得递归方程的解为:  $T(n) = O(n)$

采用这种方法计算 Fibonacci 数列和 Hanoi 塔问题递归算法的时间复杂度均为  $O(2^n)$ 。

## 2. 空间复杂度的分析

递归函数在执行时, 系统需设立一个“递归工作栈”存储每一层递归所需的信息, 此工作栈是递归函数执行的辅助空间, 因此, 分析递归算法的空间复杂度需要分析工作栈的大小。

对于递归算法, 空间复杂度

$$S(n) = O(f(n))$$

其中,  $f(n)$  为“递归工作栈”中工作记录的个数与问题规模  $n$  的函数关系。

根据这种分析方法不难得到, 前面讨论的阶乘问题、Fibonacci 数列问题、Hanoi 塔问题的递归算法的空间复杂度均为  $O(n)$ 。

### 3.4.4 利用栈将递归转换为非递归的方法

通过上述讨论, 可以看出递归程序在执行时需要系统提供隐式栈这种数据结构来实现, 对于一般的递归过程, 仿照递归算法执行过程中递归工作栈的状态变化可直接写出相应的非递归算法。这种利用栈消除递归过程的步骤如下。

- (1) 设置一个工作栈存放递归工作记录 (包括实参、返回地址及局部变量等)。
- (2) 进入非递归调用入口 (即被调程序开始处) 将调用程序传来的实在参数和返回地址入栈 (递归程序不可以作为主程序, 因而可认为初始是被某个调用程序调用)。
- (3) 进入递归调用入口: 当不满足递归结束条件时, 逐层递归, 将实参、返回地址及局部变量入栈, 这一过程可用循环语句来实现——模拟递归分解的过程。
- (4) 递归结束条件满足, 将到达递归出口的给定常数作为当前的函数值。
- (5) 返回处理: 在栈不空的情况下, 反复退出栈顶记录, 根据记录中的返回地址进行题意的操作, 即逐层计算当前函数值, 直至栈空为止——模拟递归求值过程。

通过以上步骤, 可将任何递归算法改写成非递归算法。但改写后的非递归算法和原来比较起来, 结构不够清晰, 可读性差, 有的还需要经过一系列的优化, 这里不再举例详述, 具体示例参见 5.5.1 节中二叉树中序遍历的非递归算法。

由于递归函数结构清晰, 程序易读, 而且其正确性容易得到证明, 因此, 利用允许递归调用的语言 (如 C 语言) 进行程序设计时, 给用户编制程序和调试程序带来很大方便。因为对这样一类递归问题编程时, 不需用户自己而由系统来管理递归工作栈。

## 3.5 队列的表示和操作的实现

### 3.5.1 队列的类型定义

队列的操作与栈的操作类似，不同的是，删除是在表的头部（即队头）进行。

下面给出队列的抽象数据类型定义：

```

ADT Queue {
    数据对象： $D = \{a_i \mid a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0\}$ 
    数据关系： $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2, \dots, n \}$ 
        约定其中  $a_1$  端为队列头， $a_n$  端为队列尾。

    基本操作：
        InitQueue (&Q)
            操作结果：构造一个空队列 Q。
        DestroyQueue (&Q)
            初始条件：队列 Q 已存在。
            操作结果：队列 Q 被销毁，不再存在。
        ClearQueue (&Q)
            初始条件：队列 Q 已存在。
            操作结果：将 Q 清为空队列。
        QueueEmpty (Q)
            初始条件：队列 Q 已存在。
            操作结果：若 Q 为空队列，则返回 true，否则返回 false。
        QueueLength (Q)
            初始条件：队列 Q 已存在。
            操作结果：返回 Q 的元素个数，即队列的长度。
        GetHead (Q)
            初始条件：Q 为非空队列。
            操作结果：返回 Q 的队头元素。
        EnQueue (&Q, e)
            初始条件：队列 Q 已存在。
            操作结果：插入元素 e 为 Q 的新的队尾元素。
        DeQueue (&Q, &e)
            初始条件：Q 为非空队列。
            操作结果：删除 Q 的队头元素，并用 e 返回其值。
        QueueTraverse (Q)
            初始条件：Q 已存在且非空。
            操作结果：从队头到队尾，依次对 Q 的每个数据元素访问。
} ADT Queue

```

和栈类似，在本书后面内容中引用的队列都是如上定义的队列类型，队列的数据元素类型在应用程序内定义。

### 3.5.2 循环队列——队列的顺序表示和实现

队列也有两种存储表示，顺序表示和链式表示。

和顺序栈相类似, 在队列的顺序存储结构中, 除了用一组地址连续的存储单元依次存放从队列头到队列尾的元素之外, 尚需附设两个整型变量 `front` 和 `rear` 分别指示队列头元素及队列尾元素的位置 (后面分别称为头指针和尾指针)。队列的顺序存储结构表示如下:

```
// - - - - 队列的顺序存储结构 - - - -
#define MAXQSIZE 100      // 队列可能达到的最大长度
typedef struct
{
    QElemType *base;       // 存储空间的基础地址
    int front;              // 头指针
    int rear;              // 尾指针
} SqQueue;
```

为了在 C 语言中描述方便起见, 在此约定: 初始化创建空队列时, 令 `front = rear = 0`, 每当插入新的队列尾元素时, 尾指针 `rear` 增 1; 每当删除队列头元素时, 头指针 `front` 增 1。因此, 在非空队列中, 头指针始终指向队列头元素, 而尾指针始终指向队列尾元素的下一个位置, 如图 3.12 所示。

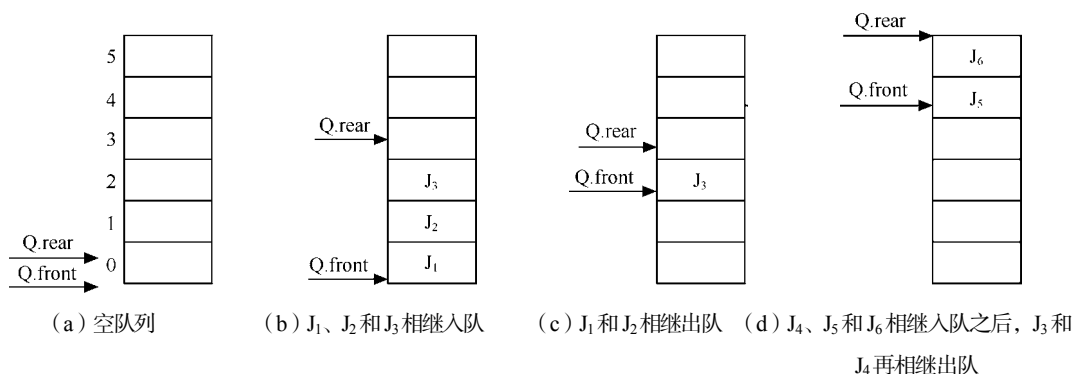


图 3.12 顺序分配的队列中头、尾指针和元素之间的关系

假设当前队列分配的最大空间为 6, 则当队列处于图 3.12 (d) 所示的状态时不可再继续插入新的队尾元素, 否则会出现溢出现象, 即因数组越界而导致程序的非法操作错误。事实上, 此时队列的实际可用空间并未占满, 所以这种现象称为“假溢出”。这是由“队尾入队, 队头出队”这种受限制的操作造成的。

怎样解决这种“假溢出”问题呢? 一个较巧妙的办法是将顺序队列变为一个环状的空间, 如图 3.13 所示, 称之为循环队列。

头、尾指针以及队列元素之间的关系不变, 只是在循环队列中, 头、尾指针“依环状增 1”的操作可用“模”运算来实现。通过取模, 头指针和尾指针就可以在顺序表空间内以头尾衔接的方式“循环”移动。

在图 3.14 (a) 中, 队头元素是  $J_5$ , 在元素  $J_6$  入队之前, 在 `Q.rear` 的值为 5, 当元素  $J_6$  入队之后, 通过“模”运算,  $Q.rear = (Q.rear + 1) \% 6$ , 得到 `Q.rear` 的值为 0, 而不会出现图 3.12 (d) 的“假溢出”状态。

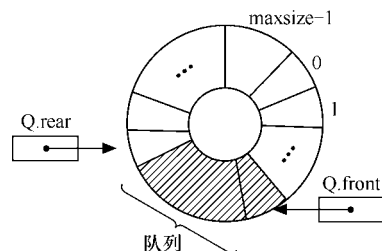
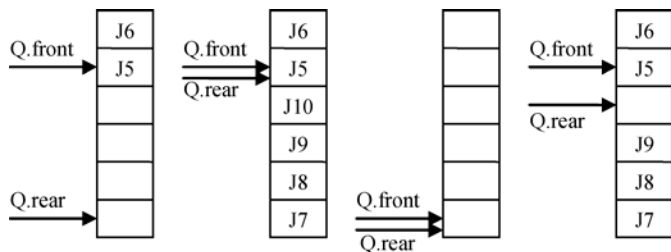


图 3.13 循环队列示意图



(a) 一般情况 (b) 队列空间被占满 (c) 空队 (d) 呈“满”状态的循环队列

图 3.14 循环队列中头、尾指针和元素之间的关系

在图 3.14 (b) 中,  $J_7$ 、 $J_8$ 、 $J_9$ 、 $J_{10}$  相继入队, 则队列空间均被占满, 此时头、尾指针相同。

在图 3.14 (c) 中, 若  $J_5$  和  $J_6$  相继从图 3.14 (a) 所示的队列中出队, 使队列此时呈“空”的状态, 头、尾指针的值也是相同的。

由此可见, 对于循环队列不能以头、尾指针的值是否相同来判别队列空间是“满”还是“空”。在这种情况下, 如何区别队满还是队空呢?

通常有以下两种处理方法。

(1) 少用一个元素空间, 即队列空间大小为  $m$  时, 有  $m-1$  个元素就认为是队满。这样判断队空的条件不变, 即当头、尾指针的值相同时, 则认为队空; 而当尾指针在循环意义上加 1 后是等于头指针, 则认为队满。因此, 在循环队列中队空和队满的条件是:

队空的条件:  $Q.front == Q.rear$

队满的条件:  $(Q.rear + 1) \% MAXQSIZE == Q.front$

如图 3.14 (d) 所示, 当  $J_7$ 、 $J_8$ 、 $J_9$  进入图 3.14 (a) 所示的队列后,  $(Q.rear + 1) \% MAXQSIZE$  的值等于  $Q.front$ , 此时认为队满。

(2) 另设一个标志位以区别队列是“空”还是“满”。具体描述参考本章习题中算法设计题的第 (7) 题, 由读者自行设计完成。

下面给出用第一种方法实现循环队列的操作, 循环队列的类型定义同前面给出的顺序队列的类型定义。

### 1. 初始化

循环队列的初始化操作就是动态分配一个预定义大小为  $MAXQSIZE$  的数组空间。

#### 算法 3.11 循环队列的初始化

##### 【算法步骤】

- ① 为队列分配一个最大容量为  $MAXSIZE$  的数组空间,  $base$  指向数组空间的首地址。
- ② 头指针和尾指针置为零, 表示队列为空。

##### 【算法描述】

```
Status InitQueue(SqQueue &Q)
{ //构造一个空队列 Q
    Q.base = new QElemType[MAXQSIZE] //为队列分配一个最大容量为 MAXSIZE 的数组空间
    if (!Q.base) exit(OVERFLOW);      //存储分配失败
    Q.front = Q.rear = 0;               //头指针和尾指针置为零, 队列为空
    return OK;
}
```

## 2. 求队列长度

对于非循环队列, 尾指针和头指针的差值便是队列长度, 而对于循环队列, 差值可能为负数, 所以需要将差值加上 MAXQSIZE, 然后与 MAXQSIZE 求余。

### 算法 3.12 求循环队列的长度

#### 【算法描述】

```
int QueueLength(SqQueue Q)
{ // 返回 Q 的元素个数, 即队列的长度
    return (Q.rear - Q.front + MAXQSIZE) % MAXQSIZE;
}
```

## 3. 入队

入队操作是指在队尾插入一个新的元素。

### 算法 3.13 循环队列的入队

#### 【算法步骤】

- ① 判断队列是否满, 若满则返回 ERROR。
- ② 将新元素插入队尾。
- ③ 队尾指针加 1。

#### 【算法描述】

```
Status EnQueue(SqQueue &Q, QElemType e)
{ // 插入元素 e 为 Q 的新的队尾元素
    if ((Q.rear + 1) % MAXQSIZE == Q.front) // 尾指针在循环意义上加 1 后等于头指针, 表明队满
        return ERROR;
    Q.base[Q.rear] = e; // 新元素插入队尾
    Q.rear = (Q.rear + 1) % MAXQSIZE; // 队尾指针加 1
    return OK;
}
```

## 4. 出队

出队操作是将队头元素删除。

### 算法 3.14 循环队列的出队

#### 【算法步骤】

- ① 判断队列是否为空, 若空则返回 ERROR。
- ② 保存队头元素。
- ③ 队头指针加 1。

#### 【算法描述】

```
Status DeQueue(SqQueue &Q, QElemType &e)
{ // 删除 Q 的队头元素, 用 e 返回其值
    if (Q.front == Q.rear) return ERROR; // 队空
    e = Q.base[Q.front]; // 保存队头元素
    Q.front = (Q.front + 1) % MAXQSIZE; // 队头指针加 1
    return OK;
}
```

## 5. 取队头元素

当队列非空时, 此操作返回当前队头元素的值, 队头指针保持不变。

算法 3.15 取循环队列的队头元素

【算法描述】

```
SElemType GetHead(SqQueue Q)
{ //返回 Q 的队头元素，不修改队头指针
    if(Q.front!=Q.rear)           //队列非空
        return Q.base[Q.front];   //返回队头元素的值，队头指针不变
}
```

由上述分析可见，如果用户的应用程序中设有循环队列，则必须为它设定一个最大队列长度；若用户无法预估所用队列的最大长度，则宜采用链队。

3.5.3 链队——队列的链式表示和实现

链队是指采用链式存储结构实现的队列。通常链队用单链表来表示，如图 3.15 所示。一个链队显然需要两个分别指示队头和队尾的指针（分别称为头指针和尾指针）才能唯一确定。这里和线性表的单链表一样，为了操作方便起见，给链队添加一个头结点，并令头指针始终指向头结点。队列的链式存储结构表示如下：

```
// - - - - 队列的链式存储结构 - - - -
typedef struct QNode
{
    QElemType data;
    struct QNode *next;
}QNode, *QueuePtr;
typedef struct
{
    QueuePtr front;           //队头指针
    QueuePtr rear;           //队尾指针
}LinkQueue;
```

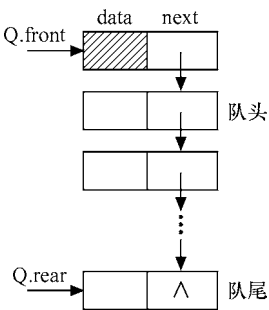


图 3.15 链队列示意图

链队的操作即为单链表插入和删除操作的特殊情况，只是需进一步修改尾指针或头指针。下面给出链队初始化、入队、出队操作的实现。

1. 初始化

链队的初始化操作就是构造一个只有一个头结点的空队，如图 3.16（a）所示。

算法 3.16 链队的初始化

【算法步骤】

- ① 生成新结点作为头结点，队头和队尾指针指向此结点。
- ② 头结点的指针域置空。

【算法描述】

```
Status InitQueue(LinkQueue &Q)
{ //构造一个空队列 Q
    Q.front=Q.rear=new QNode;   //生成新结点作为头结点，队头和队尾指针指向此结点
    Q.front->next=NULL;         //头结点的指针域置空
    return OK;
}
```

## 2. 入队

和循环队列的入队操作不同的是,链队在入队前不需要判断队是否满,需要为入队元素动态分配一个结点空间,如图 3.16 (b) 和 (c) 所示。

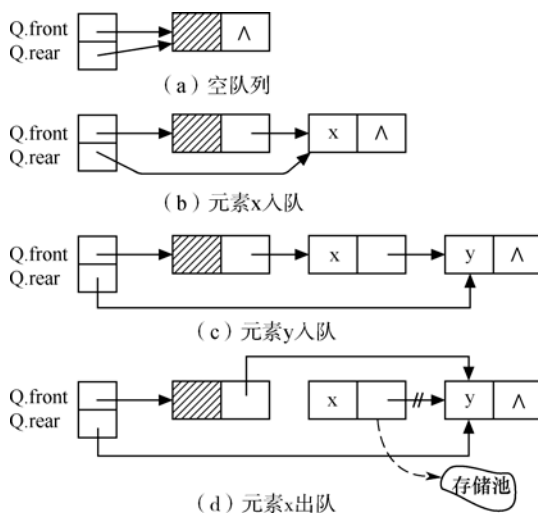


图 3.16 队列运算指针变化状况

### 算法 3.17 链队的入队

#### 【算法步骤】

- ① 为入队元素分配结点空间,用指针  $p$  指向。
- ② 将新结点数据域置为  $e$ 。
- ③ 将新结点插入到队尾。
- ④ 修改队尾指针为  $p$ 。

#### 【算法描述】

```
Status EnQueue(LinkQueue &Q, QElemType e)
{ // 插入元素 e 为 Q 的新的队尾元素
    p = new QNode;                // 为入队元素分配结点空间, 用指针 p 指向
    p->data = e;                  // 将新结点数据域置为 e
    p->next = NULL; Q.rear->next = p; // 将新结点插入到队尾
    Q.rear = p;                  // 修改队尾指针
    return OK;
}
```

## 3. 出队

和循环队列一样,链队在出队前也需要判断队列是否为空,不同的是,链队在出队后需要释放出队头元素的所占空间,如图 3.16 (d) 所示。

### 算法 3.18 链队的出队

#### 【算法步骤】

- ① 判断队列是否为空,若空则返回 ERROR。

- ② 临时保存队头元素的空间，以备释放。
- ③ 修改队头指针，指向下一个结点。
- ④ 判断出队元素是否为最后一个元素，若是，则将队尾指针重新赋值，指向头结点。
- ⑤ 释放原队头元素的空间。

#### 【算法描述】

```
Status DeQueue(LinkQueue &Q, QElemType &e)
{ // 删除 Q 的队头元素，用 e 返回其值
    if (Q.front == Q.rear) return ERROR;           // 若队列空，则返回 ERROR
    p = Q.front->next;                             // p 指向队头元素
    e = p->data;                                     // e 保存队头元素的值
    Q.front->next = p->next;                         // 修改头指针
    if (Q.rear == p) Q.rear = Q.front;             // 最后一个元素被删，队尾指针指向头结点
    delete p;                                       // 释放原队头元素的空间
    return OK;
}
```

需要注意的是，在链队出队操作时还要考虑当队列中最后一个元素被删后，队列尾指针也丢失了，因此需对队尾指针重新赋值（指向头结点）。

#### 4. 取队头元素

与循环队列一样，当队列非空时，此操作返回当前队头元素的值，队头指针保持不变。

#### 算法 3.19 取链队的队头元素。

#### 【算法描述】

```
SElemType GetHead(LinkQueue Q)
{ // 返回 Q 的队头元素，不修改队头指针
    if (Q.front != Q.rear)                       // 队列非空
        return Q.front->next->data;               // 返回队头元素的值，队头指针不变
}
```

## 3.6 案例分析与实现

在 3.2 节我们引入了 3 个有关栈应用的案例和一个有关队列应用的案例。本节对这四个案例作进一步的分析，然后分别利用栈和队列的基本操作给出案例中相关算法的具体实现。

#### 案例 3.1：数制的转换。

#### 【案例分析】

当将一个十进制整数  $N$  转换为八进制数时，在计算过程中，把  $N$  与 8 求余得到的八进制数的各位依次进栈，计算完毕后将栈中的八进制数依次出栈输出，输出结果就是待求得的八进制数。

#### 【案例实现】

在具体实现时，栈可以采用顺序存储表示也可以采用链式存储表示。

#### 算法 3.20 数制的转换

#### 【算法步骤】

- ① 初始化一个空栈  $S$ 。

- ② 当十进制数  $N$  非零时, 循环执行以下操作:
- 把  $N$  与 8 求余得到的八进制数压入栈  $S$ ;
  - $N$  更新为  $N$  与 8 的商。
- ③ 当栈  $S$  非空时, 循环执行以下操作:
- 弹出栈顶元素  $e$ ;
  - 输出  $e$ 。

#### 【算法描述】

```
void conversion(int N)
{ // 对于任意一个非负十进制数, 打印输出与其等值的八进制数
    InitStack(S); // 初始化空栈 S
    while(N) // 当 N 非零时, 循环
    {
        Push(S, N%8); // 把 N 与 8 求余得到的八进制数压入栈 S
        N=N/8; // N 更新为 N 与 8 的商
    }
    while(!StackEmpty(S)) // 当栈 S 非空时, 循环
    {
        Pop(S, e); // 弹出栈顶元素 e
        cout<<e; // 输出 e
    }
}
```

#### 【算法分析】

显然, 该算法的时间和空间复杂度均为  $O(\log_8 n)$ 。

这是利用栈的后进先出特性的最简单的例子。在这个例子中, 栈的操作是单调的, 即先一味地入栈, 然后一味地出栈。也许, 有的读者会提出疑问: 用数组直接实现不是更简单吗? 但仔细分析上述算法不难看出, 栈的引入简化了程序设计的问题, 划分了不同的关注层次, 使思考范围缩小了。而用数组不仅掩盖了问题的本质, 还要分散精力去考虑数组下标增减等细节问题。

在实际利用栈的问题中, 入栈和出栈操作大都不是单调的, 而是交错进行的。下面的案例 3.2 和 3.3 都属于这种情况。

#### 案例 3.2: 括号匹配的检验。

##### 【案例分析】

检验算法借助一个栈, 每当读入一个左括号, 则直接入栈, 等待相匹配的同类右括号; 每当读入一个右括号, 若与当前栈顶的左括号类型相同, 则二者匹配, 将栈顶的左括号出栈, 直到表达式扫描完毕。

在处理过程中, 还要考虑括号不匹配出错的情况。例如, 当出现  $(([ ]))$  这种情况时, 由于前面入栈的左括号均已和后面出现的右括号相匹配, 栈已空, 因此最后扫描的右括号不能得到匹配; 出现  $([ ( ])$  这种错误, 当表达式扫描结束时, 栈中还有一个左括号没有匹配; 出现  $(( ))$  这种错误显然是栈顶的左括号和最后的右括号不匹配。

##### 【案例实现】

#### 算法 3.21 括号的匹配

##### 【算法步骤】

- ① 初始化一个空栈  $S$ 。

② 设置一标记性变量 `flag`，用来标记匹配结果以控制循环及返回结果，1 表示正确匹配，0 表示错误匹配，`flag` 初值为 1。

③ 扫描表达式，依次读入字符 `ch`，如果表达式没有扫描完毕或 `flag` 非零，则循环执行以下操作：

- 若 `ch` 是左括号 “[” 或 “(”，则将其压入栈；
- 若 `ch` 是右括号 “)” 或 “)”，则根据当前栈顶元素的值分情况考虑：若栈非空且栈顶元素是 “(”，则正确匹配，否则错误匹配，`flag` 置为 0；
- 若 `ch` 是右括号 “]”，则根据当前栈顶元素的值分情况考虑：若栈非空且栈顶元素是 “[”，则正确匹配，否则错误匹配，`flag` 置为 0。

④ 退出循环后，如果栈空且 `flag` 值为 1，则匹配成功，返回 `true`，否则返回 `false`。

#### 【算法描述】

```
Status Matching()
{ // 检验表达式中所含括号是否正确匹配，如果匹配，则返回 true，否则返回 false
  // 表达式以 “#” 结束
  InitStack(S); // 初始化空栈
  flag=1; // 标记匹配结果以控制循环及返回结果
  cin>>ch; // 读入第一个字符
  while(ch!='#' && flag) // 假设表达式以 “#” 结尾
  {
    switch(ch)
    {
      case '['||'(': // 若是左括号，则将其压入栈
        Push(S,ch);
        break;
      case ')': // 若是 “)”，则根据当前栈顶元素的值分情况考虑
        if(!StackEmpty(S) && GetTop(S)=='(')
          Pop(S,x); // 若栈非空且栈顶元素是 “(”，则正确匹配
        else flag=0; // 若栈空或栈顶元素不是 “(”，则错误失败
        break;
      case ']': // 若是 “]”，则根据当前栈顶元素的值分情况考虑
        if(!StackEmpty(S) && GetTop(S)=='[')
          Pop(S,x); // 若栈非空且栈顶元素是 “[”，则正确匹配
        else flag=0; // 若栈空或栈顶元素不是 “[”，则错误匹配
        break;
    } // switch
    cin>>ch; // 继续读入下一个字符
  } // while
  if(StackEmpty(S) && flag) return true; // 匹配成功
  else return false; // 匹配失败
}
```

#### 【算法分析】

此算法从头到尾扫描表达式中每个字符，若表达式的字符串长度为  $n$ ，则此算法的时间复杂度为  $O(n)$ 。算法在运行时所占用的辅助空间主要取决于  $S$  栈的大小，显然， $S$  栈的空间大小不会超过  $n$ ，所以此算法的空间复杂度也同为  $O(n)$ 。

## 案例 3.3：表达式求值。

## 【案例分析】

任何一个表达式都是由操作数（operand）、运算符（operator）和界限符（delimiter）组成的，统称它们为单词。一般地，操作数既可以是常数，也可以是被说明为变量或常量的标识符；运算符可以分为算术运算符、关系运算符和逻辑运算符 3 类；基本界限符有左右括号和表达式结束符等。为了叙述的简洁，在此仅讨论简单算术表达式的求值问题，这种表达式只含加、减、乘、除 4 种运算符。读者不难将它推广到更一般的表达式上。

下面把运算符和界限符统称为算符。

我们知道，算术四则运算遵循以下 3 条规则：

- (1) 先乘除，后加减；
- (2) 从左算到右；
- (3) 先括号内，后括号外。

根据上述 3 条运算规则，在运算的每一步中，任意两个相继出现的算符 $\theta_1$ 和 $\theta_2$ 之间的优先关系，至多是下面 3 种关系之一：

$$\begin{aligned}\theta_1 < \theta_2 & \quad \theta_1 \text{ 的优先权低于 } \theta_2 \\ \theta_1 = \theta_2 & \quad \theta_1 \text{ 的优先权等于 } \theta_2 \\ \theta_1 > \theta_2 & \quad \theta_1 \text{ 的优先权高于 } \theta_2\end{aligned}$$

表 3.1 定义了算符之间的这种优先关系。

表 3.1 算符间的优先关系

| $\theta_1 \backslash \theta_2$ | + | - | * | / | ( | ) | # |
|--------------------------------|---|---|---|---|---|---|---|
| +                              | > | > | < | < | < | > | > |
| -                              | > | > | < | < | < | > | > |
| *                              | > | > | > | > | < | > | > |
| /                              | > | > | > | > | < | > | > |
| (                              | < | < | < | < | < | = |   |
| )                              | > | > | > | > |   | > | > |
| #                              | < | < | < | < | < |   | = |

由规则（1），先进行乘除运算，后进行加减运算，所以有“+” < “\*”；“+” < “/”；“\*” > “+”；“/” > “+”等。

由规则（2），运算遵循左结合性，当两个运算符相同时，先出现的运算符优先级高，所以有“+” > “+”；“-” > “-”；“\*” > “\*”；“/” > “/”。

由规则（3），括号内的优先级高，+、-、\*和/为 $\theta_1$ 时的优先性均低于“（”但高于“）”。

表中的“（” = “）”表示当左右括号相遇时，括号内的运算已经完成。为了便于实现，假设每个表达式均以“#”开始，以“#”结束。所以“#” = “#”表示整个表达式求值完毕。“）”与“（”、“#”与“）”以及“（”与“#”之间无优先关系，这是因为表达式中不允许它们相继出现，一旦遇到这种情况，则可以认为出现了语法错误。在下面的讨论中，我们暂假定所输入的表达式不会出现语法错误。

## 【案例实现】

为实现算符优先算法，可以使用两个工作栈，一个称做 OPTR，用以寄存运算符；另一个称

作 OPND，用以寄存操作数或运算结果。

### 算法 3.22 表达式求值

#### 【算法步骤】

- ① 初始化 OPTR 栈和 OPND 栈，将表达式起始符“#”压入 OPTR 栈。
- ② 扫描表达式，读入第一个字符 ch，如果表达式没有扫描完毕至“#”或 OPTR 的栈顶元素不为“#”时，则循环执行以下操作：
  - 若 ch 不是运算符，则压入 OPND 栈，读入下一字符 ch；
  - 若 ch 是运算符，则根据 OPTR 的栈顶元素和 ch 的优先级比较结果，做不同的处理：
    - 若是小于，则 ch 压入 OPTR 栈，读入下一字符 ch；
    - 若是大于，则弹出 OPTR 栈顶的运算符，从 OPND 栈弹出两个数，进行相应运算，结果压入 OPND 栈；
    - 若是等于，则 OPTR 的栈顶元素是“(”且 ch 是“)”，这时弹出 OPTR 栈顶的“(”，相当于括号匹配成功，然后读入下一字符 ch。
- ③ OPND 栈顶元素即为表达式求值结果，返回此元素。

#### 【算法描述】

```
char EvaluateExpression()
{ //算术表达式求值的算符优先算法，设 OPTR 和 OPND 分别为运算符栈和操作数栈
    InitStack(OPND); //初始化 OPND 栈
    InitStack(OPTR); //初始化 OPTR 栈
    Push(OPTR, '#'); //将表达式起始符“#”压入 OPTR 栈
    cin>>ch;
    while(ch!='#'||GetTop(OPTR)!='#') //表达式没有扫描完毕或 OPTR 的栈顶元素不为“#”
    {
        if(!In(ch)){Push(OPND,ch);cin>>ch;} //ch 不是运算符则进 OPND 栈
        else
            switch(Precede(GetTop(OPTR),ch)) //比较 OPTR 的栈顶元素和 ch 的优先级
            {
                case '<':
                    Push(OPTR,ch);cin>>ch; //当前字符 ch 压入 OPTR 栈，读入下一字符 ch
                    break;
                case '>':
                    Pop(OPTR,theta); //弹出 OPTR 栈顶的运算符
                    Pop(OPND,b);Pop(OPND,a); //弹出 OPND 栈顶的两个运算数
                    Push(OPND,Operate(a,theta,b)); //将运算结果压入 OPND 栈
                    break;
                case '=':
                    Pop(OPTR,x);cin>>ch; //弹出 OPTR 栈顶的“(”，读入下一字符 ch
                    break;
            }
    } //switch
    return GetTop(OPND); //OPND 栈顶元素即为表达式求值结果
}
```

算法调用的三个函数需要读者自行补充完成。其中函数 In 是判定读入的字符 ch 是否为运算符，Precede 是判定运算符栈的栈顶元素与读入的运算符之间优先关系的函数，Operate 为进行二元运算的函数。

另外需要特别说明的是，上述算法中的操作数只能是一位数，因为这里使用的 OPND 栈是字符栈，如果要进行多位数的运算，则需要将 OPND 栈改为数栈，读入的数字字符拼成数之后再入栈。读者可以改进此算法，使之能完成多位数的运算。

### 【算法分析】

同算法 3.21 一样，此算法从头到尾扫描表达式中每个字符，若表达式的字符串长度为  $n$ ，则此算法的时间复杂度为  $O(n)$ 。算法在运行时所占用的辅助空间主要取决于 OPTR 栈和 OPND 栈的大小，显然，它们的空间大小之和不会超过  $n$ ，所以此算法的空间复杂度也同为  $O(n)$ 。

### 【例 3.2】 算法表达式的求值过程。

利用算法 3.22 对算术表达式  $3*(7-2)$  进行求值，给出其求值的具体过程。

在表达式两端先增加“#”，改写为

#3\*(7-2)#

具体操作过程如表 3.2 所示。

表 3.2 算术表达式  $3*(7-2)$  的求值过程

| 步骤 | OPTR 栈 | OPND 栈 | 读入字符     | 主要操作                               |
|----|--------|--------|----------|------------------------------------|
| 1  | #      |        | 3*(7-2)# | Push(OPND, '3')                    |
| 2  | #      | 3      | *(7-2)#  | Push(OPTR, '*')                    |
| 3  | #*     | 3      | (7-2)#   | Push(OPTR, '(')                    |
| 4  | #*(    | 3      | 7-2)#    | Push(OPND, '7')                    |
| 5  | #*(    | 3 7    | -2)#     | Push(OPTR, '-')                    |
| 6  | #*(-   | 3 7    | 2)#      | Push(OPND, '2')                    |
| 7  | #*(-   | 3 7 2  | )#       | Push(OPND, Operate('7', '-', '2')) |
| 8  | #*(    | 3 5    | )#       | Pop(OPTR){消去一对括号}                  |
| 9  | #*     | 3 5    | #        | Push(OPND, Operate('3', '*', '5')) |
| 10 | #      | 15     | #        | return(GetTop(OPND))               |

在高级语言的编译处理过程中，实际上不只是表达式求值可以借助栈来实现，高级语言中一般语法成分的分析都可以借助栈来实现，在编译原理后续课程中会涉及栈在语法、语义等分析算法中的应用。

### 案例 3.4：舞伴问题。

#### 【案例分析】

对于舞伴配对问题，先入队的男士或女士先出队配成舞伴，因此设置两个队列分别存放男士和女士入队者。假设男士和女士的记录存放在一个数组中作为输入，然后依次扫描该数组的各元素，并根据性别来决定是进入男队还是女队。当这两个队列构造完成之后，依次将两队当前的队头元素出队来配成舞伴，直至某队列变空为止。此时，若某队仍有等待配对者，则输出此队列中排在队头的等待者的姓名，此人将是下一轮舞曲开始时第一个可获得舞伴的人。

#### 【案例实现】

算法中有关数据结构的定义如下：

```
// - - - - 跳舞者个人信息 - - - -
typedef struct
```

```

{
    char name[20];                //姓名
    char sex;                     //性别, 'F'表示女性, 'M'表示男性
}Person;
//- - - - 队列的顺序存储结构- - - -
#define MAXQSIZE 100             //队列可能达到的最大长度
typedef struct
{
    Person *base;                //队列中数据元素类型为 Person
    int front;                   //头指针
    int rear;                    //尾指针
}SqQueue;
SqQueue Mdancers, Fdancers;      //分别存放男士和女士入队者队列

```

### 算法 3.23 舞伴问题

#### 【算法步骤】

- ① 初始化 Mdancers 队列和 Fdancers 队列。
- ② 反复循环, 依次将舞者根据其性别插入 Mdancers 队列或 Fdancers 队列。
- ③ 当 Mdancers 队列和 Fdancers 队列均为非空时, 反复循环, 依次输出男女舞伴的姓名。
- ④ 如果 Mdancers 队列为空而 Fdancers 队列非空, 则输出 Fdancers 队列的队头女士的姓名。
- ⑤ 如果 Fdancers 队列为空而 Mdancers 队列非空, 则输出 Mdancers 队列的队头男士的姓名。

#### 【算法描述】

```

void DancePartner(Person dancer[], int num)
{ //结构数组 dancer 中存放跳舞的男女, num 是跳舞的人数。
    InitQueue(Mdancers);          //男士队列初始化
    InitQueue(Fdancers);         //女士队列初始化
    for(i=0; i<num; i++)         //依次将舞者根据其性别入队
    {
        p=dancer[i];
        if(p.sex=='F') EnQueue(Fdancers, p); //插入女队
        else EnQueue(Mdancers, p);          //插入男队
    }
    cout<<"The dancing partners are:\n";
    while(!QueueEmpty(Fdancers) && !QueueEmpty(Mdancers))
    { //依次输出男女舞伴的姓名
        DeQueue(Fdancers, p);              //女士出队
        cout<<p.name<<" ";                //输出出队女士姓名
        DeQueue(Mdancers, p);              //男士出队
        cout<<p.name<<endl;               //输出出队男士姓名
    }
    if(!QueueEmpty(Fdancers))              //女士队列非空, 输出队头女士的姓名
    {
        p=GetHead(Fdancers)               //取女士队头
        cout<<"The first woman to get a partner is: "<< p.name<<endl;
    }
    else if(!QueueEmpty(Mdancers))         //男士队列非空, 输出队头男士的姓名

```

```

{
    p=GetHead(Mdancers)           //取男士队头
    cout<<"The first man to get a partner is: "<< p.name<<endl;
}
}

```

### 【算法分析】

若跳舞者人数总计为  $n$ ，则此算法的时间复杂度为  $O(n)$ 。空间复杂度取决于 Mdancers 队列和 Fdancers 队列的长度，二者长度之和不会超过  $n$ ，因此空间复杂度也同样为  $O(n)$ 。

队列在程序设计中也很多应用，凡是符合先进先出原则的数学模型，都可以用队列。最典型的例子是操作系统中用来解决主机与外设之间速度不匹配问题或多个用户引起的资源竞争问题。

例如，一个局域网上有一台共享的网络打印机，网上每个用户都可以将数据发送给网络打印机进行打印。为了保证能够正常打印，操作系统为网络打印机生成一个“作业队列”，每个申请打印的“作业”应按先后的顺序排队，打印机从作业队列中逐个提取作业进行打印。

这方面的例子很多，在操作系统等后续课程中会涉及大量队列这种数据结构的应用。

在实际应用中，队列应用的例子更是常见，通常用以模拟排队情景。例如，拿汽车加油站来说，通常的结构基本上是：入口和出口为单行道，加油车道可能有若干条。每辆车加油都要经过三段路程，第一段是在入口处排队等候进入加油车道；第二段是在加油车道排队等候加油；第三段是在进入出口处排队等候离开。实际上，这三段都是队列结构。若用算法模拟这个过程，总共需要设置的队列个数应该为加油车道数加上 2。

## 3.7 小 结

本章介绍了两种特殊的线性表：栈和队列，主要内容如下。

(1) 栈是限定仅在表尾进行插入或删除的线性表，又称为后进先出的线性表。栈有两种存储表示，顺序表示（顺序栈）和链式表示（链栈）。栈的主要操作是进栈和出栈，对于顺序栈的进栈和出栈操作要注意判断栈满或栈空。

(2) 队列是一种先进先出的线性表。它只允许在表的一端进行插入，而在另一端删除元素。队列也有两种存储表示，顺序表示（循环队列）和链式表示（链队）。队列的主要操作是进队和出队，对于顺序的循环队列的进队和出队操作要注意判断队满或队空。凡是涉及队头或队尾指针的修改都要将其对 MAXQSIZE 求模。

(3) 栈和队列是在程序设计中被广泛使用的两种数据结构，其具体的应用场景都是与其表示方法和运算规则相互联系的。表 3.3 分别从逻辑结构、存储结构和运算规则 3 方面对二者进行了比较。

表 3.3 栈和队列的比较

| 栈和队列<br>比较项目 | 栈                     | 队列                    |
|--------------|-----------------------|-----------------------|
| 逻辑结构         | 和线性表一样，数据元素之间存在一对一的关系 | 和线性表一样，数据元素之间存在一对一的关系 |



```

{ //n 大于等于 0
    if (n <= 0) return 1;
    else return n * fact(n-1);
}

```

则计算  $\text{fact}(n)$  需要调用该函数的次数为 ( )。

- A.  $n+1$
- B.  $n-1$
- C.  $n$
- D.  $n+2$

(6) 栈在 ( ) 中有所应用。

- A. 递归调用
- B. 函数调用
- C. 表达式求值
- D. 前三个选项都有

(7) 为解决计算机主机与打印机间速度不匹配问题, 通常设一个打印数据缓冲区。主机将要输出的数据依次写入该缓冲区, 而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是 ( )。

- A. 队列
- B. 栈
- C. 线性表
- D. 有序表

(8) 设栈  $S$  和队列  $Q$  的初始状态为空, 元素  $e_1$ 、 $e_2$ 、 $e_3$ 、 $e_4$ 、 $e_5$  和  $e_6$  依次进入栈  $S$ , 一个元素出栈后即进入  $Q$ , 若 6 个元素出队的序列是  $e_2$ 、 $e_4$ 、 $e_3$ 、 $e_6$ 、 $e_5$  和  $e_1$ , 则栈  $S$  的容量至少应该是 ( )。

- A. 2
- B. 3
- C. 4
- D. 6

(9) 若一个栈以向量  $V[1..n]$  存储, 初始栈顶指针  $\text{top}$  设为  $n+1$ , 则元素  $x$  进栈的正确操作是 ( )。

- A.  $\text{top}++$ ;  $V[\text{top}]=x$ ;
- B.  $V[\text{top}]=x$ ;  $\text{top}++$ ;
- C.  $\text{top}--$ ;  $V[\text{top}]=x$ ;
- D.  $V[\text{top}]=x$ ;  $\text{top}--$ ;

(10) 设计一个判别表达式中左、右括号是否配对出现的算法, 采用 ( ) 数据结构最佳。

- A. 线性表的顺序存储结构
- B. 队列
- C. 线性表的链式存储结构
- D. 栈

(11) 用链接方式存储的队列, 在进行删除运算时 ( )。

- A. 仅修改头指针
- B. 仅修改尾指针
- C. 头、尾指针都要修改
- D. 头、尾指针可能都要修改

(12) 循环队列存储在数组  $A[0..m]$  中, 则入队时的操作为 ( )。

- A.  $\text{rear}=\text{rear}+1$
- B.  $\text{rear}=(\text{rear}+1)\%(m-1)$
- C.  $\text{rear}=(\text{rear}+1)\%m$
- D.  $\text{rear}=(\text{rear}+1)\%(m+1)$

(13) 最大容量为  $n$  的循环队列, 队尾指针是  $\text{rear}$ , 队头是  $\text{front}$ , 则队空的条件是 ( )。

- A.  $(\text{rear}+1)\%n == \text{front}$
- B.  $\text{rear} == \text{front}$
- C.  $\text{rear}+1 == \text{front}$
- D.  $(\text{rear}-1)\%n == \text{front}$

(14) 栈和队列的共同点是 ( )。

- A. 都是先进先出
- B. 都是先进后出
- C. 只允许在端点处插入和删除元素
- D. 没有共同点

(15) 一个递归算法必须包括 ( )。

A. 递归部分

B. 终止条件和递归部分

C. 迭代部分

D. 终止条件和迭代部分

## 2. 算法设计题

(1) 将编号为 0 和 1 的两个栈存放于一个数组空间  $V[m]$  中, 栈底分别处于数组的两端。当第 0 号栈的栈顶指针  $\text{top}[0]$  等于 -1 时该栈为空; 当第 1 号栈的栈顶指针  $\text{top}[1]$  等于  $m$  时, 该栈为空。两个栈均从两端向中间增长 (见图 3.17)。试编写双栈初始化, 判断栈空、栈满、进栈和出栈等算法的函数。双栈数据结构的定义如下:

```
typedef struct
{
    int top[2], bot[2];           // 栈顶和栈底指针
    SElemType *V;                // 栈数组
    int m;                       // 栈最大可容纳元素个数
} DblStack;
```

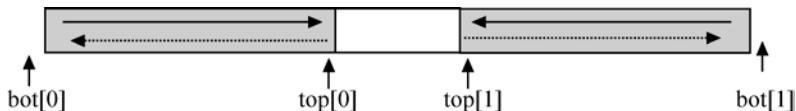


图 3.17 双栈结构的表示

(2) 回文是指正读反读均相同的字符序列, 如“abba”和“abdba”均是回文, 但“good”不是回文。试写一个算法判定给定的字符序列是否为回文。(提示: 将一半字符入栈)

(3) 设从键盘输入一整数的序列:  $a_1, a_2, a_3, \dots, a_n$ , 试编写算法实现: 用栈结构存储输入的整数, 当  $a_i \neq -1$  时, 将  $a_i$  进栈; 当  $a_i = -1$  时, 输出栈顶整数并出栈。算法应对异常情况 (入栈满等) 给出相应的信息。

(4) 从键盘上输入一个后缀表达式, 试编写算法计算表达式的值。规定: 逆波兰表达式的长度不超过一行, 以“\$”作为输入结束, 操作数之间用空格分隔, 操作符只可能有 +、-、\*、/ 四种运算。例如:  $23434 + 2 * \$$ 。

(5) 假设以 I 和 O 分别表示入栈和出栈操作。栈的初态和终态均为空, 入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列, 称可以操作的序列为合法序列, 否则称为非法序列。

① 下面所示的序列中哪些是合法的?

A. IOIOIOIO    B. IOOIOIOIO    C. IIOIOIOIO    D. IIIIOIOIO

② 通过对①的分析, 写出一个算法, 判定所给的操作序列是否合法。若合法, 返回 true, 否则返回 false (假定被判定的操作序列已存入一维数组中)。

(6) 假设以带头结点的循环链表表示队列, 并且只设一个指针指向队尾元素结点 (注意: 不设头指针), 试编写相应的置空队列、判断队列是否为空、入队和出队等算法。

(7) 假设以数组  $Q[m]$  存放循环队列中的元素, 同时设置一个标志 tag, 以  $\text{tag} == 0$  和  $\text{tag} == 1$  来区别在队头指针 (front) 和队尾指针 (rear) 相等时, 队列状态为“空”还是“满”。试编写与此结构相应的插入 (enqueue) 和删除 (dequeue) 算法。

(8) 如果允许在循环队列的两端都可以进行插入和删除操作。要求:

① 写出循环队列的类型定义;

② 写出“从队尾删除”和“从队头插入”的算法。

(9) 已知 Ackermann 函数定义如下:

$$Ack(m, n) = \begin{cases} n+1 & \text{当时} = 0 \\ Ack(m-1, 1) & \text{当时} \neq 0, n = 0 \\ Ack(m-1, Ack(m, n-1)) & \text{当时} \neq 0, n \neq 0 \end{cases}$$

① 写出计算  $Ack(m, n)$  的递归算法, 并根据此算法给出  $Ack(2, 1)$  的计算过程。

② 写出计算  $Ack(m, n)$  的非递归算法。

(10) 已知  $f$  为单链表的表头指针, 链表中存储的都是整型数据, 试写出实现下列运算的递归算法:

① 求链表中的最大整数;

② 求链表的结点个数;

③ 求所有整数的平均值。

## 第4章

# 串、数组和广义表

计算机上的非数值处理的对象大部分是字符串数据，字符串一般简称为串。串是一种特殊的线性表，其特殊性体现在数据元素是一个字符，也就是说，串是一种内容受限的线性表。由于现今使用的计算机硬件结构是面向数值计算的需要而设计的，在处理字符串数据时比处理整数和浮点数要复杂得多。而且，在不同类型的应用中，所处理的字符串具有不同的特点，要有效地实现字符串的处理，就必须根据具体情况使用合适的存储结构。本章的第一部分主要讨论串的定义、存储结构和基本操作，重点讨论串的模式匹配算法。

本章后两部分讨论的多维数组和广义表可以看成是线性表的一种扩充，即线性表的数据元素自身又是一个数据结构。高级语言都支持数组，但在高级语言中，重点介绍数组的使用，而本章重点介绍数组的内部实现，并介绍对于一些特殊的二维数组如何实现压缩存储。最后介绍广义表的基本概念和存储结构。

## 4.1 串 的 定 义

**串 (string) (或 字符串)** 是由零个或多个字符组成的有限序列，一般记为

$$s = "a_1 a_2 \cdots a_n" \quad (n \geq 0)$$

其中， $s$  是串的名，用双引号括起来的字符序列是串的值； $a_i (1 \leq i \leq n)$  可以是字母、数字或其他字符；串中字符的数目  $n$  称为串的**长度**。零个字符的串称为**空串** (null string)，其长度为零。

串中任意个连续的字符组成的子序列称为该串的**子串**。包含子串的串相应地称为**主串**。通常称字符在序列中的序号为该字符在串中的**位置**。子串在主串中的位置则以子串的第一个字符在主串中的位置来表示。

例如，假设  $a$ 、 $b$ 、 $c$ 、 $d$  为如下的 4 个串：

$$a = \text{"BEI"}, b = \text{"JING"}$$

$$c = \text{"BEIJING"}, d = \text{"BEI JING"}$$

则它们的长度分别为 3、4、7 和 8；并且  $a$  和  $b$  都是  $c$  和  $d$  的子串， $a$  在  $c$  和  $d$  中的位置都是 1，而  $b$  在  $c$  中的位置是 4，在  $d$  中的位置则是 5。

称两个串是**相等**的，当且仅当这两个串的值相等。也就是说，只有当两个串的长度相等，并且各个对应位置的字符都相等时才相等。例如，上例中的串  $a$ 、 $b$ 、 $c$  和  $d$  彼此都不相等。

在各种应用中，空格常常是串的字符集合中的一个元素；因而可以出现在其他字符中间。由

一个或多个空格组成的串“ ”称为**空格串** (blank string, 请注意: 此处不是空串), 其长度为串中空格的个数。为了清楚起见, 以后我们用符号“Ø”来表示“空串”。

## 4.2 案例引入

字符串在实际中有极为广泛的应用, 在文字编辑、信息检索、语言编译等软件系统中, 字符串均是重要的操作对象; 在网络入侵检测、计算机病毒特征码匹配以及 DNA 序列匹配等应用中, 都需要进行串匹配, 也称模式匹配。

### 案例 4.1: 病毒感染检测。

医学研究者最近发现了某些新病毒, 通过对这些病毒的分析, 得知它们的 DNA 序列都是环状的。现在研究者已收集了大量的病毒 DNA 和人的 DNA 数据, 想快速检测出这些人是否感染了相应的病毒。为了方便研究, 研究者将人的 DNA 和病毒 DNA 均表示成由一些字母组成的字符串序列, 然后检测某种病毒 DNA 序列是否在患者的 DNA 序列中出现过, 如果出现过, 则此人感染了该病毒, 否则没有感染。例如, 假设病毒的 DNA 序列为 baa, 患者 1 的 DNA 序列为 aaabbbba, 则感染, 患者 2 的 DNA 序列为 babbba, 则未感染。(注意, 人的 DNA 序列是线性的, 而病毒的 DNA 序列是环状的)

研究者将待检测的数据保存在一个文本文件中, 文件格式和内容规定如下 (图 4.1 截取了部分数据)。

文件有 num+1 行, 第一行有一个整数 num, 表示有 num 个待检测的任务 (num≤300)。

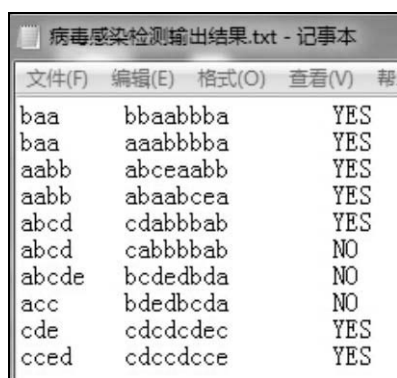
接下来每行 i (2≤i≤num+1) 对应一个任务, 每行有两个数据, 用空格分隔, 第一个数据表示病毒的 DNA 序列 (长度≤6000), 第二个数据表示人的 DNA 序列 (长度≤10000)。

要求将检测结果输出到文件中, 文件中包括 num 行, 每行有三个数据, 用空格分隔, 前两个数据分别表示输入文件中对应病毒的 DNA 序列和人的 DNA 序列, 如果该人感染了对应的病毒, 该行第三个数据则为“YES”, 否则为“NO”。图 4.1 数据对应的输出结果如图 4.2 所示。



| 病毒 DNA 序列 | 人的 DNA 序列 |
|-----------|-----------|
| 10        |           |
| baa       | bbaabbbba |
| baa       | aaabbbba  |
| aabb      | abceaabb  |
| aabb      | abaabcea  |
| abcd      | cdabbbab  |
| abcd      | cabbbbab  |
| abcde     | bcdedbda  |
| acc       | bdedbcda  |
| cde       | cdcdcdcc  |
| cced      | cdccdcce  |

图 4.1 病毒感染检测输入数据



| 病毒 DNA 序列 | 人的 DNA 序列 | 检测结果 |
|-----------|-----------|------|
| 10        |           |      |
| baa       | bbaabbbba | YES  |
| baa       | aaabbbba  | YES  |
| aabb      | abceaabb  | YES  |
| aabb      | abaabcea  | YES  |
| abcd      | cdabbbab  | YES  |
| abcd      | cabbbbab  | NO   |
| abcde     | bcdedbda  | NO   |
| acc       | bdedbcda  | NO   |
| cde       | cdcdcdcc  | YES  |
| cced      | cdccdcce  | YES  |

图 4.2 病毒感染检测输出结果

这个案例中要处理的操作对象便是字符串, 将病毒的 DNA 序列看作是子串, 患者的 DNA 序列看作是主串, 检测任务的实质就是看子串是否在主串中出现过, 即 4.3.3 小节讨论的字符串的模式匹配算法。但因为此案例中病毒的 DNA 序列是环状的, 这样需要对传统模式匹配算法进行改进。案例的具体分析与实现将在 4.6 节给出。

## 4.3 串的类型定义、存储结构及其运算

### 4.3.1 串的抽象类型定义

串的逻辑结构和线性表极为相似，区别仅在于串的数据对象约束为字符集。然而，串的基本操作和线性表有很大差别。在线性表的基本操作中，大多以“单个元素”作为操作对象，例如，在线性表中查找某个元素，求取某个元素，在某个位置上插入一个元素或删除一个元素等；而在串的基本操作中，通常以“串的整体”作为操作对象，例如，在串中查找某个子串，求取一个子串，在串的某个位置上插入一个子串，以及删除一个子串等。

串的抽象数据类型的定义如下：

**ADT String{**

**数据对象：** $D=\{a_i | a_i \in \text{CharacterSet}, i=1, 2, \dots, n, n \geq 0\}$

**数据关系：** $R1=\{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i=2, \dots, n \}$

**基本操作：**

**StrAssign(&T, chars)**

初始条件：chars 是字符串常量。

操作结果：生成一个其值等于 chars 的串 T。

**StrCopy(&T, S)**

初始条件：串 S 存在。

操作结果：由串 S 复制得串 T。

**StrEmpty(S)**

初始条件：串 S 存在。

操作结果：若 S 为空串，则返回 true，否则返回 false。

**StrCompare(S, T)**

初始条件：串 S 和 T 存在。

操作结果：若  $S > T$ ，则返回值  $> 0$ ；若  $S = T$ ，则返回值  $= 0$ ；若  $S < T$ ，则返回值  $< 0$ 。

**StrLength(S)**

初始条件：串 S 存在。

操作结果：返回 S 的元素个数，称为串的长度。

**ClearString(&S)**

初始条件：串 S 存在。

操作结果：将 S 清为空串。

**Concat(&T, S1, S2)**

初始条件：串 S1 和 S2 存在。

操作结果：用 T 返回由 S1 和 S2 联接而成的新串。

**SubString(&Sub, S, pos, len)**

初始条件：串 S 存在， $1 \leq \text{pos} \leq \text{StrLength}(S)$  且  $0 \leq \text{len} \leq \text{StrLength}(S) - \text{pos} + 1$ 。

操作结果：用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。

**Index(S, T, pos)**

初始条件：串 S 和 T 存在，T 是非空串， $1 \leq \text{pos} \leq \text{StrLength}(S)$ 。

操作结果：若主串 S 中存在和串 T 值相同的子串，则返回它在主串 S 中第 pos 个字符之后第一次出现的位置；否则函数值为 0。

**Replace(&S, T, V)**

初始条件: 串  $S$ ,  $T$  和  $V$  存在,  $T$  是非空串。

操作结果: 用  $V$  替换主串  $S$  中出现的所有与  $T$  相等的非重叠的子串。

`StrInsert(&S, pos, T)`

初始条件: 串  $S$  和  $T$  存在,  $1 \leq \text{pos} \leq \text{StrLength}(S) + 1$ 。

操作结果: 在串  $S$  的第  $\text{pos}$  个字符之前插入串  $T$ 。

`StrDelete(&S, pos, len)`

初始条件: 串  $S$  存在,  $1 \leq \text{pos} \leq \text{StrLength}(S) - \text{len} + 1$ 。

操作结果: 从串  $S$  中删除第  $\text{pos}$  个字符起长度为  $\text{len}$  的子串。

`DestroyString(&S)`

初始条件: 串  $S$  存在。

操作结果: 串  $S$  被销毁。

**}ADT String**

对于串的基本操作集可以有不同的定义方法, 读者在使用高级程序设计语言中的串类型时, 应以该语言的参考手册为准。

## 4.3.2 串的存储结构

与线性表类似, 串也有两种基本存储结构: 顺序存储和链式存储。但考虑到存储效率和算法的方便性, 串多采用顺序存储结构。

### 1. 串的顺序存储

类似于线性表的顺序存储结构, 用一组地址连续的存储单元存储串值的字符序列。按照预定义的大小, 为每个定义的串变量分配一个固定长度的存储区, 则可用定长数组如下描述:

```
// - - - - 串的定长顺序存储结构- - - -
#define MAXLEN 255           //串的最大长度
typedef struct{
    char ch[MAXLEN+1];       //存储串的一维数组
    int length;               //串的当前长度
}SString;
```

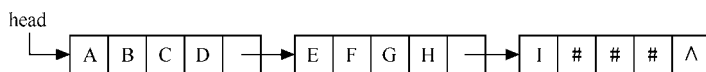
其中,  $\text{MAXLEN}$  表示串的最大长度,  $\text{ch}$  是存储字符串的一维数组, 每个分量存储一个字符,  $\text{length}$  表示字符串的当前长度。为了便于说明问题, 本章后面算法描述当中所用到的顺序存储的字符串都是从下标为 1 的数组分量开始存储的, 下标为 0 的分量闲置不用。

这种定义方式是静态的, 在编译时刻就确定了串空间的大小。而多数情况下, 串的操作是以串的整体形式参与的, 串变量之间的长度相差较大, 在操作中串值长度的变化也较大, 这样为串变量设定固定大小的空间不尽合理。因此最好是根据实际需要, 在程序执行过程中动态地分配和释放字符数组空间。在 C 语言中, 存在一个称之为“堆”(Heap)的自由存储区, 可以为每个新产生的串动态分配一块实际串长所需的存储空间, 若分配成功, 则返回一个指向起始地址的指针, 作为串的基址, 同时为了以后处理方便, 约定串长也作为存储结构的一部分。这种字符串的存储方式也称为串的堆式顺序存储结构, 定义如下:

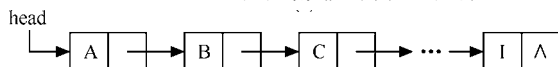
```
// - - - - 串的堆式顺序存储结构- - - -
typedef struct{
    char *ch;                 //若是非空串, 则按串长分配存储区, 否则 ch 为 NULL
    int length;               //串的当前长度
}HString;
```

## 2. 串的链式存储

顺序串的插入和删除操作不方便,需要移动大量的字符。因此,可采用单链表方式存储串。由于串结构的特殊性——结构中的每个数据元素是一个字符,则在用链表存储串值时,存在一个“结点大小”的问题,即每个结点可以存放一个字符,也可以存放多个字符。例如,图 4.3 (a) 所示为结点大小为 4 (即每个结点存放 4 个字符)的链表,图 4.3 (b) 所示为结点大小为 1 的链表。当结点大小大于 1 时,由于串长不一定是结点大小的整倍数,则链表中的最后一个结点不一定全被串值占满,此时通常补上“#”或其他非串值字符(通常“#”不属于串的字符集,是一个特殊的符号)。



(a) 结点大小为 4 的链表



(b) 结点大小为 1 的链表

图 4.3 串值的链表存储方式

为了便于进行串的操作,当以链表存储串值时,除头指针外,还可附设一个尾指针指示链表中的最后一个结点,并给出当前串的长度。称如此定义的串存储结构为块链结构,说明如下:

```
// - - - - 串的链式存储结构 - - - -
#define CHUNKSIZE 80                //可由用户定义的块大小
typedef struct Chunk{
    char ch[CHUNKSIZE];
    struct Chunk *next;
}Chunk;
typedef struct{
    Chunk *head,*tail;               //串的头和尾指针
    int length;                      //串当前长度
}LString;
```

在链式存储方式中,结点大小的选择直接影响着串处理的效率。在各种串的处理系统中,所处理的串往往很长或很多,如一本书的几百万个字符,情报资料的成千上万个条目,这就要求考虑串值的存储密度。

显然,存储密度小(如结点大小为 1 时),运算处理方便,然而,存储占用量大。如果在串处理过程中需进行内、外存交换的话,则会因为内、外存交换操作过多而影响处理的总效率。应该看到,串的字符集的大小也是一个重要因素。一般来说,字符集小,则字符的机内编码就短,这也影响串值存储方式的选取。

串值的链式存储结构对某些串操作,如联接操作等,有一定方便之处,但总地说来,不如顺序存储结构灵活,它占用存储量大且操作复杂。此外,串值在链式存储结构时,串操作的实现和线性表在链表存储结构中的操作类似,故在此不作详细讨论。下面 4.3.3 小节的模式匹配算法是采用串的定长顺序存储结构实现的。

### 4.3.3 串的模式匹配算法

子串的定位运算通常称为串的模式匹配或串匹配。此运算的应用非常广泛,比如在搜索引擎、

拼写检查、语言翻译、数据压缩等应用中,都需要进行串匹配。

串的模式匹配设有两个字符串  $S$  和  $T$ , 设  $S$  为主串, 也称正文串; 设  $T$  为子串, 也称为模式。在主串  $S$  中查找与模式  $T$  相匹配的子串, 如果匹配成功, 确定相匹配的子串中的第一个字符在主串  $S$  中出现的位置。

著名的模式匹配算法有 BF 算法和 KMP 算法, 下面详细介绍这两种算法。

### 1. BF 算法

最简单直观的模式匹配算法是 BF (Brute-Force) 算法。

#### 算法 4.1 BF 算法

模式匹配不一定是从主串的第一个位置开始, 可以指定主串中查找的起始位置  $\text{pos}$ 。如果采用字符串顺序存储结构, 可以写出不依赖于其他串操作的匹配算法。

##### 【算法步骤】

- ① 分别利用计数指针  $i$  和  $j$  指示主串  $S$  和模式  $T$  中当前正待比较的字符位置,  $i$  初值为  $\text{pos}$ ,  $j$  初值为 1。
- ② 如果两个串均未比较到串尾, 即  $i$  和  $j$  均分别小于等于  $S$  和  $T$  的长度时, 则循环执行以下操作:
  - $S[i].\text{ch}$  和  $T[j].\text{ch}$  比较, 若相等, 则  $i$  和  $j$  分别指示串中下个位置, 继续比较后续字符;
  - 若不等, 指针后退重新开始匹配, 从主串的下一个字符 ( $i=i-j+2$ ) 起再重新和模式的第一个字符 ( $j=1$ ) 比较。
- ③ 如果  $j > T.\text{length}$ , 说明模式  $T$  中的每个字符依次和主串  $S$  中的一个连续的字符序列相等, 则匹配成功, 返回和模式  $T$  中第一个字符相等的字符在主串  $S$  中的序号 ( $i-T.\text{length}$ ); 否则称匹配不成功, 返回 0。

##### 【算法描述】

```
int Index_BF(SString S, SString T, int pos)
{ // 返回模式 T 在主串 S 中第 pos 个字符开始第一次出现的位置。若不存在, 则返回值为 0
  // 其中, T 非空,  $1 \leq \text{pos} \leq S.\text{length}$ 
  i=pos; j=1; // 初始化
  while(i <= S.length && j <= T.length) // 两个串均未比较到串尾
  {
    if(S[i].ch==T[j].ch){++i; ++j;} // 继续比较后继字符
    else{i=i-j+2; j=1;} // 指针后退重新开始匹配
  }
  if(j > T.length) return i-T.length; // 匹配成功
  else return 0; // 匹配失败
}
```

图 4.4 展示了模式  $T = \text{"abcac"}$  和主串  $S$  的匹配过程 ( $\text{pos} = 1$ )。

##### 【算法分析】

BF 算法的匹配过程易于理解, 且在某些应用场合效率也较高。在匹配成功的情况下, 考虑以下两种极端情况。

(1) 最好情况下, 每趟不成功的匹配都发生在模式串的第一个字符与主串中相应字符的比较。例如:

$S = \text{"aaaaaba"}$   
 $T = \text{"ba"}$

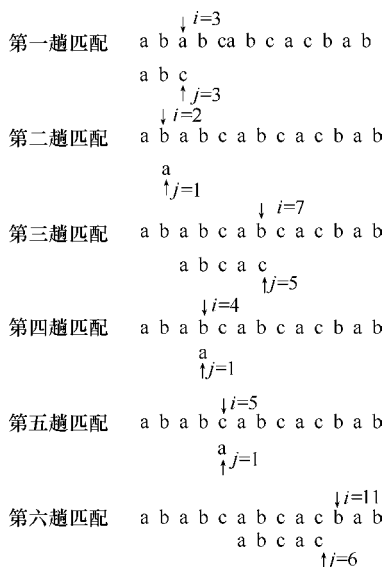


图 4.4 BF 算法的匹配过程

设主串的长度为  $n$ ，子串的长度为  $m$ ，假设从主串的第  $i$  个位置开始与模式串匹配成功，则在前  $i-1$  趟匹配中字符总共比较了  $i-1$  次；若第  $i$  趟成功的字符比较次数为  $m$ ，则总比较次数为  $i-1+m$ 。对于成功匹配的主串，其起始位置由 1 到  $n-m+1$ ，假定这  $n-m+1$  个起始位置上的匹配成功概率相等，则最好的情况下匹配成功的平均比较次数为

$$\sum_{i=1}^{n-m+1} p_i (i-1+m) = \frac{1}{n-m+1} \sum_{i=1}^{n-m+1} i-1+m = \frac{1}{2}(n+m)$$

即最好情况下的平均时间复杂度是  $O(n+m)$ 。

(2) 最坏情况下，每趟不成功的匹配都发生在模式串的最后一个字符与主串中相应字符的比较。

例如：

$S = \text{"aaaaaab"}$

$T = \text{"aab"}$

假设从主串的第  $i$  个位置开始与模式串匹配成功，则在前  $i-1$  趟匹配中字符总共比较了  $(i-1) \times m$  次；若第  $i$  趟成功的字符比较次数为  $m$ ，则总比较次数  $i \times m$ 。因此最坏情况下匹配成功的平均比较次数为

$$\sum_{i=1}^{n-m+1} p_i (i \times m) = \frac{1}{n-m+1} \sum_{i=1}^{n-m+1} i \times m = \frac{1}{2} m \times (n-m+2)$$

即最坏情况下的平均时间复杂度是  $O(n \times m)$ 。

BF 算法思路直观简明。但当匹配失败时，主串的指针  $i$  总是回溯到  $i-j+2$  位置，模式串的指针总是恢复到首字符位置  $j=1$ ，因此，算法时间复杂度高。下面将介绍另一种改进的模式匹配算法。

## 2. KMP 算法

这种改进算法是由 Knuth、Morris 和 Pratt 同时设计实现的，因此简称 KMP 算法。此算

法可以在  $O(n + m)$  的时间数量级上完成串的模式匹配操作。其改进在于：每当一趟匹配过程中出现字符比较不等时，不需回溯  $i$  指针，而是利用已经得到的“部分匹配”的结果将模式向右“滑动”尽可能远的一段距离后，继续进行比较。下面先从具体例子看起。

回顾图 4.4 中的匹配过程示例，在第三趟的匹配中，当  $i = 7, j = 5$  字符比较不等时，又从  $i = 4, j = 1$  重新开始比较。然后，经仔细观察可发现， $i = 4$  和  $j = 1, i = 5$  和  $j = 1$ ，以及  $i = 6$  和  $j = 1$  这 3 次比较都是不必进行的。因为从第三趟部分匹配的结果就可得出，主串中第 4 个、第 5 个和第 6 个字符必然是“b”、“c”和“a”（即模式串中第 2 个、第 3 个和第 4 个字符）。因为模式中的第一个字符是“a”，因此它无需再和这 3 个字符进行比较，而仅需将模式向右滑动 3 个字符的位置继续进行  $i = 7, j = 2$  时的字符比较即可。同理，在第一趟匹配中出现字符不等时，仅需将模式向右移动两个字符的位置继续进行  $i = 3, j = 1$  时的字符比较。由此，在整个匹配的过程中， $i$  指针没有回溯，如图 4.5 所示。

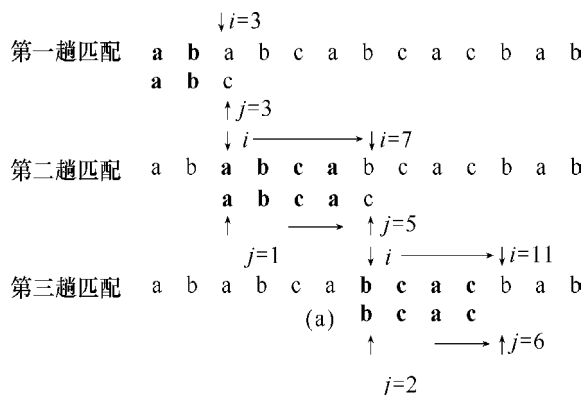


图 4.5 K MP 算法的匹配过程

现在讨论一般情况。假设主串为“ $s_1s_2 \cdots s_n$ ”，模式串为“ $t_1t_2 \cdots t_m$ ”，从上例的分析可知，为了实现改进算法，需要解决下述问题：当匹配过程中产生“失配”（即  $s_i \neq t_j$ ）时，模式串“向右滑动”可行的距离多远，换句话说，当主串中第  $i$  个字符与模式中第  $j$  个字符“失配”（即比较不等）时，主串中第  $i$  个字符（ $i$  指针不回溯）应与模式中哪个字符再比较？

假设此时应与模式中第  $k$  ( $k < j$ ) 个字符继续比较，则模式中前  $k-1$  个字符的子串必须满足下列关系式 (4-1)，且不可能存在  $k' > k$  满足下列关系式 (4-1)

$$t_1t_2 \cdots t_{k-1} = s_{i-k+1}s_{i-k+2} \cdots s_{i-1} \quad (4-1)$$

而已经得到的“部分匹配”的结果是

$$t_{j-k+1}t_{j-k+2} \cdots t_{j-1} = s_{i-k+1}s_{i-k+2} \cdots s_{i-1} \quad (4-2)$$

由式 (4-1) 和式 (4-2) 推得下列等式

$$t_1t_2 \cdots t_{k-1} = t_{j-k+1}t_{j-k+2} \cdots t_{j-1} \quad (4-3)$$

反之，若模式串中存在满足式 (4-3) 的两个子串，则当匹配过程中，主串中第  $i$  个字符与模式中第  $j$  个字符比较不等时，仅需将模式向右滑动至模式中第  $k$  个字符和主串中第  $i$  个字符对齐，此时，模式中头  $k-1$  个字符的子串“ $t_1t_2 \cdots t_{k-1}$ ”必定与主串中第  $i$  个字符之前长度为  $k-1$  的子串“ $s_{i-k+1}s_{i-k+2} \cdots s_{i-1}$ ”相等，由此，匹配仅需从模式中第  $k$  个字符与主串中第  $i$  个字符开始，依次向后进行比较。

若令  $\text{next}[j] = k$ , 则  $\text{next}[j]$  表明当模式中第  $j$  个字符与主串中相应字符“失配”时, 在模式中需重新和主串中该字符进行比较的字符的位置。由此可引出模式串的  $\text{next}$  函数的定义:

$$\text{next}[j] = \begin{cases} 0 & j=1 \text{ (与比较不等时, 下一步进行与的比较)} \\ \text{Max} \{k | 1 < k \text{ 且有 } \cdots t_1 t_2 \cdots t_{k-1} = t_{j-k+1} t_{j-k+2} \cdots t_{j-1} \} & \\ 1 & k=1 \text{ (不存在相同子串, 下一步进行与的比较)} \end{cases} \quad (4-4)$$

由此定义可推出模式串的  $\text{next}$  函数值, 如图 4.6 所示。

在求得模式的  $\text{next}$  函数之后, 匹配可如下进行: 假设以指针  $i$  和  $j$  分别指示主串和模式中正在比较的字符, 令  $i$  的初值为  $\text{pos}$ ,  $j$  的初值为 1。若在匹配过程中  $s_i = t_j$ , 则  $i$  和  $j$  分别增 1, 否则,  $i$  不变, 而  $j$  退到  $\text{next}[j]$  的位置再比较, 若相等, 则指针各自增 1, 否则  $j$  再退到下一个  $\text{next}$  值的位置,

| $j$              | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|------------------|---|---|---|---|---|---|---|---|
| 模式串              | a | b | a | a | b | c | a | c |
| $\text{next}[j]$ | 0 | 1 | 1 | 2 | 2 | 3 | 1 | 2 |

图 4.6 模式串的  $\text{next}$  函数值

依次类推, 直至下列两种可能: 一种是  $j$  退到某个  $\text{next}$  值 ( $\text{next}[\text{next}[\cdots \text{next}[j] \cdots]]$ ) 时字符比较相等, 则指针各自增 1, 继续进行匹配; 另一种是  $j$  退到值为零 (即模式的第一个字符“失配”), 则此时需将模式继续向右滑动一个位置, 即从主串的下一个字符  $s_{i+1}$  起和模式重新开始匹配。图 4.7 所示正是上述匹配过程的一个例子。

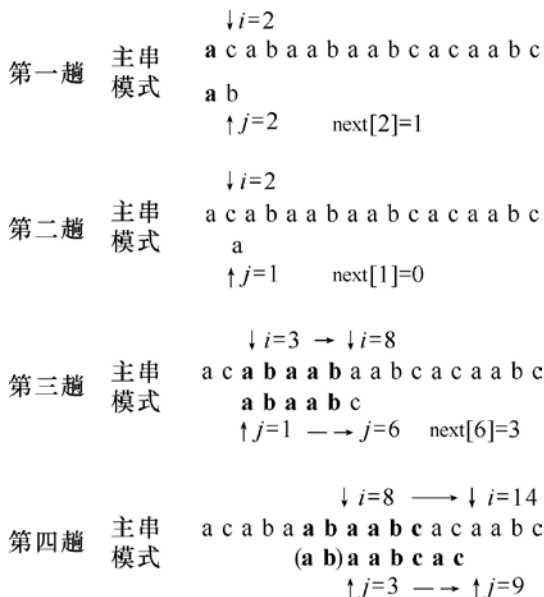


图 4.7 利用模式的  $\text{next}$  函数进行匹配的过程示例

KMP 算法如算法 4.2 所示, 它在形式上和算法 4.1 极为相似。不同之处仅在于: 当匹配过程中产生“失配”时, 指针  $i$  不变, 指针  $j$  退回到  $\text{next}[j]$  所指示的位置上重新进行比较, 并且当指针  $j$  退至零时, 指针  $i$  和指针  $j$  需同时增 1。即若主串的第  $i$  个字符和模式的第 1 个字符不等, 应从主串的第  $i+1$  个字符起重新进行匹配。

#### 算法 4.2 KMP 算法

##### 【算法描述】

```
int Index_KMP(SString S, SString T, int pos)
```

```

    { //利用模式串 T 的 next 函数求 T 在主串 S 中第 pos 个字符之后的位置
      //其中, T 非空,  $1 \leq \text{pos} \leq \text{S.length}$ 
      i=pos; j=1;
      while(i <= S.length && j <= S.length) //两个串均未比较到串尾
      {
          if(j==0 || S[i]==T[j]) {++i; ++j;} //继续比较后继字符
          else j=next[j]; //模式串向右移动
      }
      if(j>T[0]) return i-T[0]; //匹配成功
      else return 0; //匹配失败
    }

```

KMP 算法是在已知模式串的 next 函数值的基础上执行的, 那么, 如何求得模式串的 next 函数值呢?

从上述讨论可见, 此函数值仅取决于模式串本身, 而和相匹配的主串无关, 可从分析其定义出发用递推的方法求得 next 函数值。

由定义得知:

$$\text{next}[1] = 0 \quad (4-5)$$

设  $\text{next}[j] = k$ , 这表明在模式串中存在下列关系:

$$“t_1 t_2 \cdots t_{k-1}” = “t_{j-k+1} t_{j-k+2} \cdots t_{j-1}” \quad (4-6)$$

其中  $k$  为满足  $1 < k < j$  的某个值, 并且不可能存在  $k' > k$  满足等式 (4-7)。此时  $\text{next}[j+1] = ?$  可能有以下两种情况。

(1) 若  $t_k = t_j$ , 则表明在模式串中

$$“t_1 t_2 \cdots t_k” = “t_{j-k+1} t_{j-k+2} \cdots t_j” \quad (4-7)$$

并且不可能存在  $k' > k$  满足等式 (4-7), 这就是说  $\text{next}[j+1] = k+1$ , 即

$$\text{next}[j+1] = \text{next}[j] + 1 \quad (4-8)$$

(2) 若  $t_k \neq t_j$ , 则表明在模式串中

$$“t_1 t_2 \cdots t_k” \neq “t_{j-k+1} t_{j-k+2} \cdots t_j”$$

此时可把求 next 函数值的问题看成是一个模式匹配的问题, 整个模式串既是主串又是模式串, 而当前在匹配的过程中, 已有  $t_{j-k+1} = t_1, t_{j-k+2} = t_2, \cdots, t_{j-1} = t_{k-1}$ , 则当  $t_j \neq t_k$  时应将模式向右滑动至以模式中的第  $\text{next}[k]$  个字符和主串中的第  $j$  个字符相比较。若  $\text{next}[k] = k'$ , 且  $t_j = t_{k'}$ , 则说明在主串中第  $j+1$  个字符之前存在一个长度为  $k'$  (即  $\text{next}[k]$ ) 的最长子串, 和模式串中从首字符起长度为  $k'$  的子串相等, 即

$$“t_1 t_2 \cdots t_{k'}” = “t_{j-k'+1} t_{j-k'+2} \cdots t_j” \quad (1 < k' < k < j) \quad (4-9)$$

这就是说  $\text{next}[j+1] = k'+1$ , 即

$$\text{next}[j+1] = \text{next}[k] + 1 \quad (4-10)$$

同理, 若  $t_j \neq t_{k'}$ , 则将模式继续向右滑动直至将模式中第  $\text{next}[k']$  个字符和  $t_j$  对齐,  $\cdots$ , 依次类推, 直至  $t_j$  和模式中某个字符匹配成功或者不存在任何  $k'$  ( $1 < k' < j$ ) 满足等式 (4-9), 则

$$\text{next}[j+1] = 1 \quad (4-11)$$

例如, 图 4.8 中的模式串, 已求得前 6 个字符的 next 函数值, 现求  $\text{next}[7]$ , 因为  $\text{next}[6] = 3$ ,

又  $t_6 \neq t_3$ , 则需比较  $t_6$  和  $t_1$  (因为  $\text{next}[3]=1$ ), 这相当于将子串模式向右滑动。由于  $t_6 \neq t_1$ , 而且  $\text{next}[1]=0$ , 所以  $\text{next}[7]=1$ , 而因为  $t_7=t_1$ , 则  $\text{next}[8]=2$ 。

根据上述分析所得结果 (式 (4-5)、式 (4-8)、式 (4-10) 和式 (4-11)), 仿照 KMP 算法, 可得到求  $\text{next}$  函数值的算法, 如算法 4.3 所示。

#### 算法 4.3 计算 $\text{next}$ 函数值

##### 【算法描述】

```
void get_next(SString T, int next[])
{ // 求模式串 T 的 next 函数值并存入数组 next
    i=1; next[1]=0; j=0;
    while (i < T[0])
    {
        if (j==0 || T[i]==T[j]) { ++i; ++j; next[i]=j; }
        else j=next[j];
    }
}
```

算法 4.3 的时间复杂度为  $O(m)$ 。通常, 模式串的长度  $m$  比主串的长度  $n$  要小得多, 因此, 对整个匹配算法来说, 所增加的这点时间是值得的。

最后, 要说明以下两点。

(1) 虽然 BF 算法的时间复杂度是  $O(n \times m)$ , 但在一般情况下, 其实际的执行时间近似于  $O(n + m)$ , 因此至今仍被采用。KMP 算法仅当模式与主串之间存在许多“部分匹配”的情况下, 才显得比 BF 算法快得多。但是 KMP 算法的最大特点是指示主串的指针不需回溯, 整个匹配过程中, 对主串仅需从头至尾扫描一遍, 这对处理从外设输入的庞大文件很有效, 可以边读入边匹配, 而无需回头重读。

(2) 前面定义的  $\text{next}$  函数在某些情况下尚有缺陷。例如模式“aaaab”在和主串“aaabaaaab”匹配时, 当  $i=4$ 、 $j=4$  时  $s.ch[4] \neq t.ch[4]$ , 由  $\text{next}[j]$  的指示还需进行  $i=4$ 、 $j=3$ ,  $i=4$ 、 $j=2$ ,  $i=4$ 、 $j=1$  这 3 次比较。实际上, 因为模式中第 1~3 个字符和第 4 个字符都相等, 因此不需要再和主串中第 4 个字符相比较, 而可以将模式连续向右滑动 4 个字符的位置直接进行  $i=5$ 、 $j=1$  时的字符比较。这就是说, 若按上述定义得到  $\text{next}[j]=k$ , 而模式中  $t_j=t_k$ , 则当主串中字符  $s_i$  和  $t_j$  比较不等时, 不需要再和  $t_k$  进行比较, 而直接和  $T_{\text{next}[k]}$  进行比较, 换句话说, 此时的  $\text{next}[j]$  应和  $\text{next}[k]$  相同。由此可得计算  $\text{next}$  函数修正值的算法如算法 4.4 所示,  $\text{next}$  函数修正值的计算结果如图 4.9 所示。此时匹配算法不变。

#### 算法 4.4 计算 $\text{next}$ 函数修正值

##### 【算法描述】

```
void get_nextval(SString T, int nextval[])
{ // 求模式串 T 的 next 函数修正值并存入数组 nextval
    i=1; nextval[1]=0; j=0;
    while (i < T[0])
    {
        if (j==0 || T[i]==T[j])
```

|                  |   |   |   |   |   |   |   |   |
|------------------|---|---|---|---|---|---|---|---|
| $j$              | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 模式               | a | b | a | a | b | c |   | a |
| $\text{next}[j]$ | 0 | 1 | 1 | 2 | 2 | 3 | 1 | 2 |

(a b a)  
(a)

图 4.8 模式串的  $\text{next}$  函数值

|                     |   |   |   |   |   |
|---------------------|---|---|---|---|---|
| $j$                 | 1 | 2 | 3 | 4 | 5 |
| 模式串                 | a | a | a | a | b |
| $\text{next}[j]$    | 0 | 1 | 2 | 3 | 4 |
| $\text{nextval}[j]$ | 0 | 0 | 0 | 0 | 4 |

图 4.9  $\text{next}$  函数修正值

```

{
    ++i; ++j;
    if (T[i] != T[j]) nextval[i] = j;
    else nextval[i] = nextval[j];
}
else j = nextval[j];
}
}

```

## 4.4 数 组

### 4.4.1 数组的类型定义

数组是由类型相同的数据元素构成的有序集合, 每个元素称为数组元素, 每个元素受  $n$  ( $n \geq 1$ ) 个线性关系的约束, 每个元素在  $n$  个线性关系中的序号  $i_1, i_2, \dots, i_n$  称为该元素的下标, 可以通过下标访问该数据元素。因为数组中每个元素处于  $n$  ( $n \geq 1$ ) 个关系中, 故称该数组为  $n$  维数组。数组可以看成是线性表的推广, 其特点是结构中的元素本身可以是具有某种结构的数据, 但属于同一数据类型。

例如, 一维数组可以看成是一个线性表, 二维数组可以看成数据元素是线性表的线性表。图 4.10 (a) 所示的二维数组可以看成是一个线性表

$$A_{m \times n} = \begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0, n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1, n-1} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{m-1, 0} & a_{m-1, 1} & a_{m-1, 2} & \cdots & a_{m-1, n-1} \end{bmatrix}, \quad A_{m \times n} = \begin{bmatrix} \begin{bmatrix} a_{00} \\ a_{10} \\ \vdots \\ a_{m-1, 0} \end{bmatrix} & \begin{bmatrix} a_{01} \\ a_{11} \\ \vdots \\ a_{m-1, 1} \end{bmatrix} & \cdots & \begin{bmatrix} a_{0, n-1} \\ a_{1, n-1} \\ \vdots \\ a_{m-1, n-1} \end{bmatrix} \end{bmatrix}$$

(a) 矩阵形式表示

(b) 列向量的一维数组

$$A_{m \times n} = ((a_{00} a_{01} \cdots a_{0, n-1}), (a_{10} a_{11} \cdots a_{1, n-1}), \dots, (a_{m-1, 0} a_{m-1, 1} \cdots a_{m-1, n-1}))$$

(c) 行向量的一维数组

图 4.10 二维数组图例

$$A = (a_0, a_1, \dots, a_p) \quad (p = m-1 \text{ 或 } n-1)$$

其中每个数据元素  $a_j$  是一个列向量形式的线性表

$$a_j = (a_{0j}, a_{1j}, \dots, a_{m-1, j}) \quad 0 \leq j \leq n-1$$

(见图 4.10 (b)) 或者  $a_i$  是一个行向量形式的线性表

$$a_i = (a_{i0}, a_{i1}, \dots, a_{i, n-1}) \quad 0 \leq i \leq m-1$$

(见图 4.10 (c))。在 C 语言中, 一个二维数组类型可以定义为其分量类型为一维数组类型的一维数组类型, 也就是说,

```
typedef E IemType A rray2[m][n];
```

等价于

```
typedef E IemType Array1[n];
```

```
typedef A rray1 A rray2[m];
```

同理，一个  $n$  维数组类型可以定义为其数据元素为  $n-1$  维数组类型的一维数组类型。

数组一旦被定义，它的维数和维界就不再改变。因此，除了结构的初始化和销毁之外，数组只有存取元素和修改元素值的操作。

抽象数据类型数组可形式地定义为：

**ADT Array{**

数据对象：  $j_i=0, \dots, b_i-1, i=1, 2, \dots, n,$

$D=\{a_{j_1 j_2 \dots j_n} \mid n(>0) \text{ 称为数组的维数, } b_i \text{ 是数组第 } i \text{ 维的长度,}$

$j_i \text{ 是数组元素的第 } i \text{ 维下标, } a_{j_1 j_2 \dots j_n} \in \text{ElemSet}\}$

数据关系：  $R=\{R_1, R_2, \dots, R_n\}$

基本操作：

InitArray(&A, n, bound<sub>1</sub>, ..., bound<sub>n</sub>)

操作结果：若维数  $n$  和各维长度合法，则构造相应的数组  $A$ ，并返回 OK。

DestroyArray(&A)

操作结果：销毁数组  $A$ 。

Value(A, &e, index<sub>1</sub>, ..., index<sub>n</sub>)

初始条件：  $A$  是  $n$  维数组，  $e$  为元素变量，随后是  $n$  个下标值。

操作结果：若各下标不超界，则  $e$  赋值为所指定的  $A$  的元素值，并返回 OK。

Assign(&A, e, index<sub>1</sub>, ..., index<sub>n</sub>)

初始条件：  $A$  是  $n$  维数组，  $e$  为元素变量，随后是  $n$  个下标值。

操作结果：若下标不超界，则将  $e$  的值赋给所指定的  $A$  的元素，并返回 OK。

**} ADT Array**

## 4.4.2 数组的顺序存储

由于数组一般不做插入或删除操作，也就是说，一旦建立了数组，则结构中的数据元素个数和元素之间的关系就不再发生变动。因此，采用顺序存储结构表示数组比较合适。

由于存储单元是一维的结构，而数组可能是多维的结构，则用一组连续存储单元存放数组的数据元素就有次序约定问题。例如图 4.10 (a) 所示的二维数组可以看成如图 4.10 (b) 所示的一维数组，也可看成如图 4.10 (c) 所示的一维数组。对应地，对二维数组可有两种存储方式：一种是以列序为主序的存储方式，如图 4.11 (a) 所示；一种是以行序为主序的存储方式，如图 4.11 (b) 所示。在扩展 Basic、Pascal、Java 和 C 语言中，用的都是以行序为主序的存储结构，而在 FORTRAN 语言中，用的是以列序为主序的存储结构。

由此，对于数组，一旦规定了其维数和各维的长度，便可为它分配存储空间。反之，只要给出一组下标便可求得相应数组元素的存储位置。下面仅用以行序为主序的存储结构为例予以说明。

假设每个数据元素占  $L$  个存储单元，则二维数组  $A[0..m-1, 0..n-1]$  (即下标从 0 开始，共有  $m$  行  $n$  列) 中任一元素  $a_{ij}$  的存储位置可由下式确定

$$\text{LOC}(i, j) = \text{LOC}(0, 0) + (n \times i + j)L \quad (4-12)$$

式中， $\text{LOC}(i, j)$  是  $a_{ij}$  的存储位置； $\text{LOC}(0, 0)$  是  $a_{00}$  的存储位置，即二维数组  $A$  的起始存储位置，也称为基地址或基址。

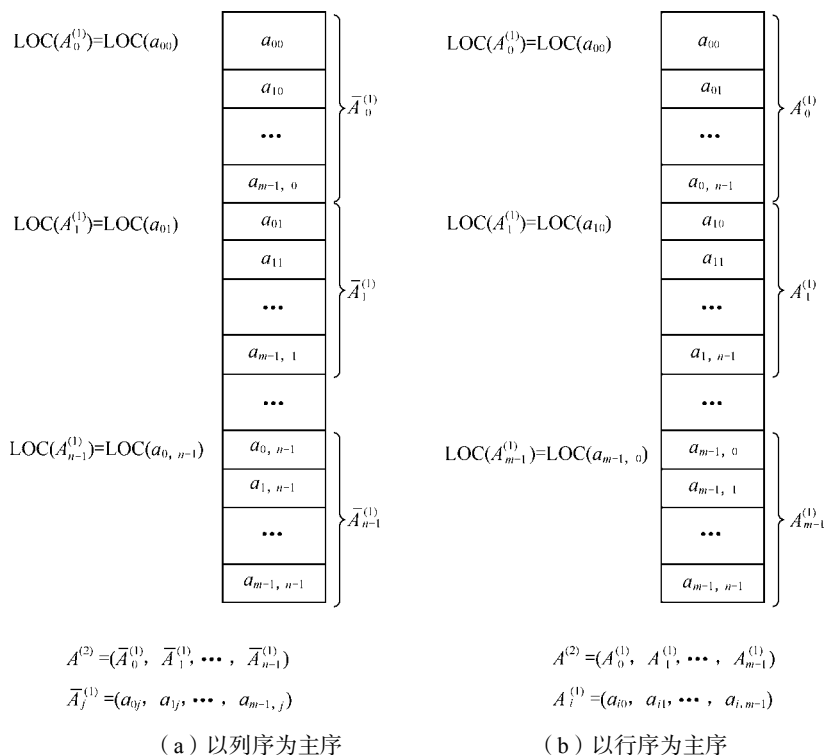


图 4.11 二维数组的两种存储方式

将式 (4-12) 推广到一般情况, 可得到  $n$  维数组  $A[0..b_1-1, 0..b_2-1, \dots, 0..b_n-1]$  的数据元素存储位置的计算公式:

$$\begin{aligned} \text{LOC}(j_1, j_2, \dots, j_n) &= \text{LOC}(0, 0, \dots, 0) + (b_2 \times \dots \times b_n \times j_1 + b_3 \times \dots \times b_n \times j_2 \\ &\quad + \dots + b_n \times j_{n-1} + j_n)L \\ &= \text{LOC}(0, 0, \dots, 0) + \left( \sum_{i=1}^{n-1} j_i \prod_{k=i+1}^n b_k + j_n \right) L \end{aligned}$$

可缩写成

$$\text{LOC}(j_1, j_2, \dots, j_n) = \text{LOC}(0, 0, \dots, 0) + \sum_{i=1}^n c_i j_i \quad (4-13)$$

其中,  $c_n = L$ ,  $c_{i-1} = b_i \times c_i$ ,  $1 \leq i \leq n$ 。

式 (4-13) 称为  $n$  维数组的映像函数。容易看出, 数组元素的存储位置是其下标的线性函数, 一旦确定了数组各维的长度,  $c_i$  就是常数。由于计算各个元素存储位置的时间相等, 所以存取数组中任一元素的时间也相等, 即数组是一种随机存取结构。

### 4.4.3 特殊矩阵的压缩存储

矩阵是很多科学与工程计算问题中研究的数学对象, 矩阵用二维数组来表示是最自然的方法。但是, 在数值分析中经常出现一些阶数很高的矩阵, 同时在矩阵中有很多值相同的元素或者是零元素。有时为了节省存储空间, 可以对这类矩阵进行压缩存储。所谓压缩存储, 是指为多个值相

同的元只分配一个存储空间，对零元不分配空间。

假若值相同的元素或者零元素在矩阵中的分布有一定规律，则称此类矩阵为**特殊矩阵**。特殊矩阵主要包括对称矩阵、三角矩阵和对角矩阵等，下面我们重点讨论这三种特殊矩阵的压缩存储。

### 1. 对称矩阵

若  $n$  阶矩阵  $A$  中的元满足下述性质

$$a_{ij} = a_{ji} \quad 1 \leq i, j \leq n$$

则称为  $n$  阶对称矩阵。

对于对称矩阵，可以为每一对对称元分配一个存储空间，则可将  $n^2$  个元压缩存储到  $n(n+1)/2$  个元的空间中，不失一般性，可以行序为主序存储其下三角（包括对角线）中的元。

假设以一维数组  $sa[n(n+1)/2]$  作为  $n$  阶对称矩阵  $A$  的存储结构，则  $sa[k]$  和矩阵元  $a_{ij}$  之间存在着——对应的关系：

$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1 & \text{当 } i \geq j \\ \frac{j(j-1)}{2} + i - 1 & \text{当 } i < j \end{cases} \quad (4-14)$$

对于任意给定的一组下标  $(i, j)$ ，均可在  $sa$  中找到矩阵元  $a_{ij}$ ；反之，对所有的  $k = 0, 1, 2, \dots, \frac{n(n+1)}{2} - 1$ ，都能确定  $sa[k]$  中的元在矩阵中的位置  $(i, j)$ 。由此，称  $sa[n(n+1)/2]$  为  $n$  阶对称矩阵  $A$  的压缩存储（见图 4.12）。

|          |          |          |          |     |                    |     |                      |
|----------|----------|----------|----------|-----|--------------------|-----|----------------------|
| $a_{11}$ | $a_{21}$ | $a_{22}$ | $a_{31}$ | ... | $a_{n,1}$          | ... | $a_{n,n}$            |
| $k=0$    | 1        | 2        | 3        |     | $\frac{n(n-1)}{2}$ |     | $\frac{n(n+1)}{2}-1$ |

图 4.12 对称矩阵的压缩存储

### 2. 三角矩阵

以主对角线划分，三角矩阵有上三角矩阵和下三角矩阵两种。上三角矩阵是指矩阵下三角（不包括对角线）中的元均为常数  $c$  或零的  $n$  阶矩阵，下三角矩阵与之相反。对三角矩阵进行压缩存储时，除了和对称矩阵一样，只存储其上（下）三角中的元素之外，再加一个存储常数  $c$  的存储空间即可。

#### (1) 上三角矩阵

$sa[k]$  和矩阵元  $a_{ij}$  之间的对应关系为

$$k = \begin{cases} \frac{(i-1)(2n-i+2)}{2} + (j-i) & \text{当 } i \leq j \\ \frac{n(n+1)}{2} & \text{当 } i > j \end{cases} \quad (4-15)$$

#### (2) 下三角矩阵

$sa[k]$  和矩阵元  $a_{ij}$  之间的对应关系为

$$k = \begin{cases} \frac{i(i-1)}{2} + j - 1 & \text{当 } \geq j \\ \frac{n(n+1)}{2} & \text{当 } < j \end{cases} \quad (4-16)$$

### 3. 对角矩阵

对角矩阵所有的非零元都集中在以主对角线为中心的带状区域中，即除了主对角线上和直接在对角线上、下方若干条对角线上的元之外，所有其他的元皆为零，如图 4.13 所示。对这种矩阵，也可按某个原则（或以行为主，或以对角线的顺序）将其压缩存储到一维数组上。

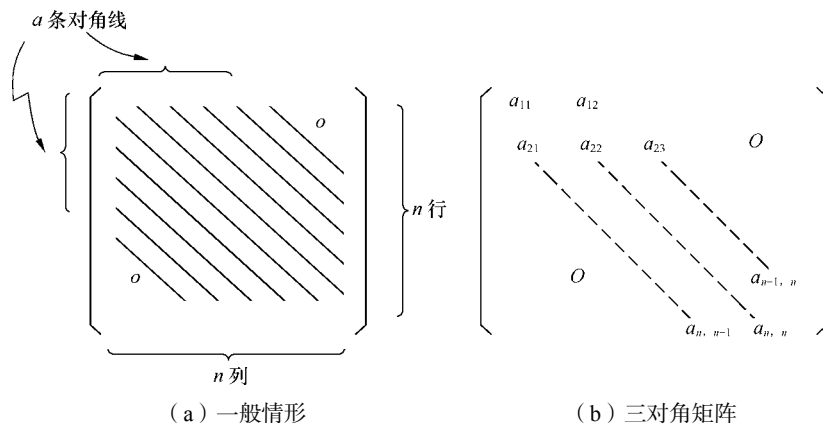


图 4.13 对角矩阵

在上述这些特殊矩阵中，非零元的分布都有一个明显的规律，从而可将其压缩存储到一维数组中，并找到每个非零元在一维数组中的对应关系。

然而，在实际应用中还经常会遇到另一类矩阵，其非零元较零元少，且分布没有一定规律，称之为**稀疏矩阵**。这类矩阵的压缩存储就要比特殊矩阵复杂，在此不做讨论。

## 4.5 广 义 表

### 4.5.1 广义表的定义

顾名思义，广义表是线性表的推广，也称为列表。广泛地用于人工智能等领域的表处理语言 LISP 语言，把广义表作为基本的数据结构，就连程序也表示为一系列的广义表。

广义表一般记作

$$LS = (a_1, a_2, \dots, a_n)$$

其中， $LS$  是广义表  $(a_1, a_2, \dots, a_n)$  的名称， $n$  是其长度。在线性表的定义中， $a_i$  ( $1 \leq i \leq n$ ) 只限于是单个元素。而在广义表的定义中， $a_i$  可以是单个元素，也可以是广义表，分别称为广义表  $LS$  的原子和子表。习惯上，用大写字母表示广义表的名称，用小写字母表示原子。

显然，广义表的定义是一个递归的定义，因为在描述广义表时又用到了广义表的概念。下面列举一些广义表的例子。

- (1)  $A = ()$ —— $A$  是一个空表, 其长度为零。
- (2)  $B = (e)$ —— $B$  只有一个原子  $e$ , 其长度为 1。
- (3)  $C = (a, (b, c, d))$ —— $C$  的长度为 2, 两个元素分别为原子  $a$  和子表  $(b, c, d)$ 。
- (4)  $D = (A, B, C)$ —— $D$  的长度为 3, 3 个元素都是广义表。显然, 将子表的值代入后, 则有  $D = (( ), (e), (a, (b, c, d)))$ 。
- (5)  $E = (a, E)$ ——这是一个递归的表, 其长度为 2。 $E$  相当于一个无限的广义表  $E = (a, (a, (a, \dots)))$ 。

从上述定义和例子可推出广义表的如下 3 个重要结论。

(1) 广义表的元素可以是子表, 而子表的元素还可以是子表……由此, 广义表是一个多层次的结构, 可以用图形象地表示。例如, 图 4.14 表示的是广义表  $D$ , 图中以圆圈表示广义表, 以方块表示原子。

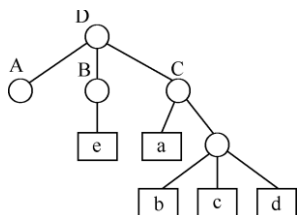


图 4.14 广义表的图形表示

(2) 广义表可为其他广义表所共享。例如在上述例子中, 广义表  $A$ 、 $B$  和  $C$  为  $D$  的子表, 则在  $D$  中可以不必列出子表的值, 而是通过子表的名称来引用。

(3) 广义表可以是一个递归的表, 即广义表也可以是其本身的一个子表。例如, 表  $E$  就是一个递归的表。

由于广义表的结构比较复杂, 其各种运算的实现也不如线性表简单, 其中, 最重要的两个运算如下。

(1) 取表头  $\text{GetHead}(LS)$ : 取出的表头为非空广义表的第一个元素, 它可以是一个单原子, 也可以是一个子表。

(2) 取表尾  $\text{GetTail}(LS)$ : 取出的表尾为除去表头之外, 由其余元素构成的表。即表尾一定是一个广义表。

例如:

$\text{GetHead}(B) = e, \quad \text{GetTail}(B) = ()$ ,

$\text{GetHead}(D) = A, \quad \text{GetTail}(D) = (B, C)$ ,

由于  $(B, C)$  为非空广义表, 则可继续分解得到:

$\text{GetHead}(B, C) = B, \quad \text{GetTail}(B, C) = (C)$ ,

值得提醒的是, 广义表  $()$  和  $(( ))$  不同。前者为空表, 长度  $n = 0$ ; 后者长度  $n = 1$ , 可分解得到其表头、表尾均为空表  $()$ 。

## 4.5.2 广义表的存储结构

由于广义表中的数据元素可以有不同的结构 (或是原子, 或是列表), 因此难以用顺序存储结构表示, 通常采用链式存储结构。常用的链式存储结构有两种, 头尾链表的存储结构和扩展线性链表的存储结构。

## 1. 头尾链表的存储结构

由于广义表中的数据元素可能为原子或广义表，由此需要两种结构的结点：一种是表结点，用以表示广义表；一种是原子结点，用以表示原子。从上节得知：若广义表不空，则可分解成表头和表尾，因此，一对确定的表头和表尾可唯一确定广义表。一个表结点可由 3 个域组成：标志域、指示表头的指针域和指示表尾的指针域。而原子结点只需两个域：标志域和值域。如图 4.15 所示，其中 tag 是标志域，值为 1 时表明结点是子表，值为 0 时表明结点是原子。



图 4.15 头尾链表表示的结点结构

其形式定义说明如下：

```
// - - - - 广义表的头尾链表存储表示 - - - -
typedef enum{ATOM,LIST} ElemTag; //ATOM==0: 原子; LIST==1: 子表
typedef struct GLNode
{
    ElemTag tag; //公共部分，用于区分原子结点和表结点
    union //原子结点和表结点的联合部分
    {
        AtomType atom; //atom 是原子结点的值域，AtomType 由用户定义
        struct{struct GLNode*hp, *tp;}ptr; //ptr 是表结点的指针域，ptr.hp 和 ptr.tp 分别指向表头和表尾
    };
}*GList; //广义表类型
```

上节中曾列举了广义表的例子，它们的存储结构如图 4.16 所示，在这种存储结构中有以下几种情况。

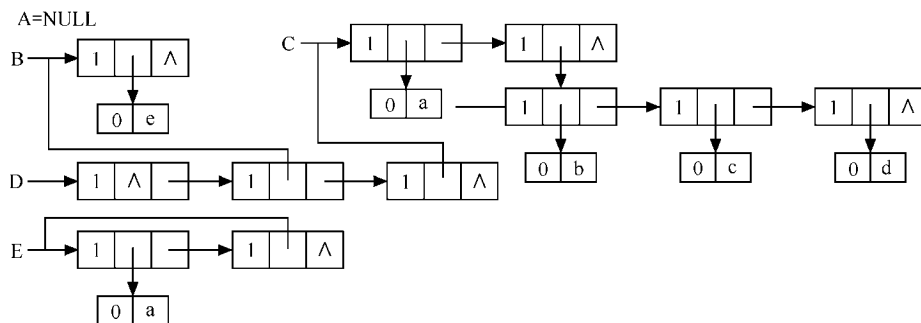


图 4.16 头尾链表表示的存储结构示例

(1) 除空表的表头指针为空外，对任何非空广义表，其表头指针均指向一个表结点，且该结

点中的 **hp** 域指示广义表表头（或为原子结点，或为表结点），**tp** 域指向广义表表尾（除非表尾为空，则指针为空，否则必为表结点）。

（2）容易分清列表中原子和子表所在层次。如在广义表 **D** 中，原子 **a** 和 **e** 在同一层次上，而 **b**、**c** 和 **d** 在同一层次且比 **a** 和 **e** 低一层，**B** 和 **C** 是同一层的子表。

（3）最高层的表结点个数即为广义表的长度。

以上 3 个特点在某种程度上给广义表的操作带来方便。

## 2. 扩展线性链表的存储结构

在这种结构中，无论是原子结点还是表结点均由三个域组成，其结点结构如图 4.17 所示。



图 4.17 扩展线性链表表示的结点结构

4.5.1 小节中广义表例子所对应的这种表示法的存储结构，如图 4.18 所示。

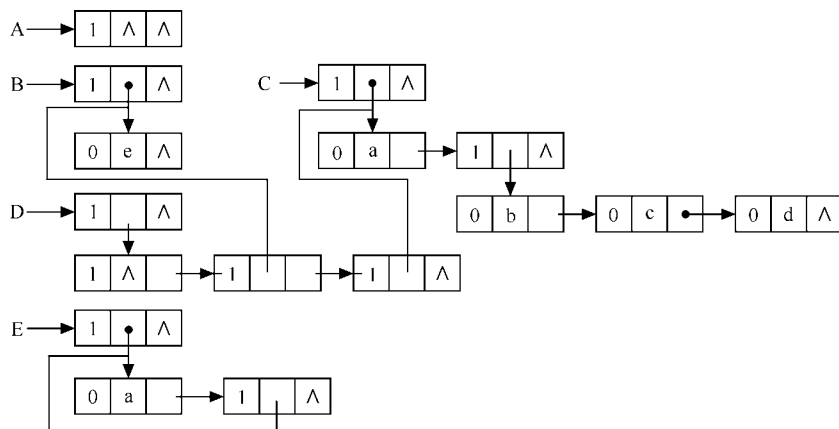


图 4.18 扩展线性链表表示的存储结构示例

## 4.6 案例分析与实现

### 案例 4.1：病毒感染检测。

#### 【案例分析】

因为患者的 DNA 和病毒 DNA 均是由一些字母组成的字符串序列，要检测某种病毒 DNA 序列是否在患者的 DNA 序列中出现过，实际上就是字符串的模式匹配问题。可以利用 BF 算法，也可以利用更高效的 KMP 算法。但与一般的模式匹配问题不同的是，此案例中病毒的 DNA 序列是环状的，这样需要对传统的 BF 算法或 KMP 算法进行改进。

下面给出利用 BF 算法实现检测的方案。

#### 【案例实现】

对于每一个待检测的任务，假设病毒 DNA 序列的长度是  $m$ ，因为病毒 DNA 序列是环状的，

为了线性取到每个可行的长度为  $m$  的模式串, 可将存储病毒 DNA 序列的字符串长度扩大为  $2m$ , 将病毒 DNA 序列连续存储两次。然后循环  $m$  次, 依次取得每个长度为  $m$  的环状字符串, 将此字符串作为模式串, 将人的 DNA 序列作为主串, 调用 BF 算法进行模式匹配。只要匹配成功, 即可中止循环, 表明该人感染了对应的病毒; 否则, 循环  $m$  次结束循环时, 可通过 BF 算法的返回值判断该人是否感染了对应的病毒。

#### 算法 4.5 病毒感染检测

##### 【算法步骤】

- ① 从文件中读取待检测的任务数  $num$ 。
- ② 根据  $num$  个数依次检测每对病毒 DNA 和人的 DNA 是否匹配, 循环  $num$  次, 执行以下操作:
  - 从文件中分别读取一对病毒 DNA 序列和人的 DNA 序列;
  - 设置一个标志性变量  $flag$ , 用来标识是否匹配成功, 初始为 0, 表示未匹配;
  - 病毒 DNA 序列的长度是  $m$ , 将存储病毒 DNA 序列的字符串长度扩大为  $2m$ , 将病毒 DNA 序列连续存储两次;
  - 循环  $m$  次, 重复执行以下操作:
    - 依次取得每个长度为  $m$  的病毒 DNA 环状字符串;
    - 将此字符串作为模式串, 将人的 DNA 序列作为主串, 调用 BF 算法进行模式匹配, 将匹配结果返回赋值给  $flag$ ;
    - 若  $flag$  非 0, 表示匹配成功, 中止循环, 表明该人感染了对应的病毒。
  - 退出循环时, 判断  $flag$  的值, 若  $flag$  非 0, 输出 “YES”, 否则, 输出 “NO”。

##### 【算法描述】

```
void Virus_detection()
{ // 利用 BF 算法实现病毒检测
    ifstream inFile("病毒感染检测输入数据.txt");
    ofstream outFile("病毒感染检测输出结果.txt");
    inFile >> num; // 读取待检测的任务数
    while (num--) // 依次检测每对病毒 DNA 和人的 DNA 是否匹配
    {
        inFile >> Virus.ch+1; // 读取病毒 DNA 序列, 字符串从下标 1 开始存放
        inFile >> Person.ch+1; // 读取人的 DNA 序列
        Vir=Virus.ch; // 将病毒 DNA 临时暂存在 Vir 中, 以备输出
        flag=0; // 用来标识是否匹配, 初始为 0, 匹配后为非 0
        m=Virus.length; // 病毒 DNA 序列的长度是 m
        for (i=m+1, j=1; j<=m; j++)
            Virus.ch[i++]=Virus.ch[j]; // 将病毒字符串的长度扩大 2 倍
        Virus.ch[2*m+1]='\0'; // 添加结束符号
        for (i=0; i<m; i++) // 依次取得每个长度为 m 的病毒 DNA 环状字符串 temp
        {
            for (j=1; j<=m; j++) temp.ch[j]=Virus.ch[i+j];
            temp.ch[m+1]='\0'; // 添加结束符号
            flag=Index_BF(Person, temp, 1); // 模式匹配
            if (flag) break; // 匹配即可退出循环
        }
        // for
    }
}
```

```

        if(flag) outFile<<Vir+1<<"    "<<Person.ch+1<<" "<<"YES"<<endl;
        else    outFile<<Vir+1<<"    "<<Person.ch+1<<" "<<"NO"<<endl;
    }
    //while
}

```

### 【算法分析】

对于每一个待检测的任务而言,该算法都需要执行  $m$  次模式匹配,假设人的 DNA 序列长度为  $n$ ,由于 BF 算法的时间复杂度为  $O(m*n)$ ,因此,对于每一个待检测的任务,时间复杂度都为  $O(m*m*n)$ 。如果待检测的任务个数为  $num$ ,则上述算法的时间复杂度为  $O(num*m*m*n)$ ,时间复杂度较高。利用 KMP 算法完成模式匹配将有效地提高匹配效率,读者可以模仿该算法,实现利用 KMP 算法完成检测的方案。

## 4.7 小 结

本章介绍了三种数据结构:串、数组和广义表,主要内容如下。

(1) 串是内容受限的线性表,它限定了表中的元素为字符。串有两种基本存储结构:顺序存储和链式存储,但多采用顺序存储结构。串的常用算法是模式匹配算法,主要有 BF 算法和 KMP 算法。BF 算法实现简单,但存在回溯,效率低,时间复杂度为  $O(m \times n)$ 。KMP 算法对 BF 算法进行改进,消除回溯,提高了效率,时间复杂度为  $O(m+n)$ 。

(2) 多维数组可以看成是线性表的推广,其特点是结构中的元素本身可以是具有某种结构的数据,但属于同一数据类型。一个  $n$  维数组实质上是  $n$  个线性表的组合,其每一维都是一个线性表。数组一般采用顺序存储结构,故存储多维数组时,应先将其确定转换为一维结构,有按“行”转换和按“列”转换两种。科学与工程计算中的矩阵通常用二维数组来表示,为了节省存储空间,对于几种常见形式的特殊矩阵,比如对称矩阵、三角矩阵和对角矩阵,在存储时可进行压缩存储,即为多个值相同的元只分配一个存储空间,对零元不分配空间。

(3) 广义表是另外一种线性表的推广形式,表中的元素可以是称为原子的单个元素,也可以是一个子表,所以线性表可以看成广义表的特例。广义表的结构相当灵活,在某种前提下,它可以兼容线性表、数组、树和有向图等各种常用的数据结构。广义表的常用操作有取表头和取表尾。广义表通常采用链式存储结构:头尾链表的存储结构和扩展线性链表的存储结构。

学习完本章后,要求掌握串的存储方法,理解串的两种模式匹配算法,尤其是 KMP 算法。明确数组和广义表这两种数据结构的特点,掌握数组存储时地址计算方法,掌握几种特殊矩阵的压缩存储方法,了解广义表的两种链式存储结构。

## 习 题

### 1. 选择题

(1) 串是一种特殊的线性表,其特殊性体现在 ( )。

- A. 可以顺序存储  
B. 数据元素是单个字符  
C. 可以链式存储  
D. 数据元素可以是多个字符

(2) 下列关于串的叙述中, 不正确的是 ( )。

- A. 串是字符的有限序列  
B. 空串是由空格构成的串  
C. 模式匹配是串的一种重要运算  
D. 串既可以采用顺序存储, 也可以采用链式存储

(3) 串 “ababaaababaa” 的 next 数组为 ( )。

- A. 012345678999  
B. 012121111212  
C. 011234223456  
D. 0123012322345

(4) 串 “ababaabab” 的 nextval 为 ( )。

- A. 010104101  
B. 010102101  
C. 010100011  
D. 010101011

(5) 串的长度是指 ( )。

- A. 串中所含不同字母的个数  
B. 串中所含字符的个数  
C. 串中所含不同字符的个数  
D. 串中所含非空格字符的个数

(6) 假设以行序为主序存储二维数组  $A = \text{array}[1..100, 1..100]$ , 设每个数据元素占 2 个存储单元, 基地址为 10, 则  $\text{LOC}[5, 5] = ( )$ 。

- A. 808  
B. 818  
C. 1010  
D. 1020

(7) 设有数组  $A[i, j]$ , 数组的每个元素长度为 3 字节,  $i$  的值为  $1 \sim 8$ ,  $j$  的值为  $1 \sim 10$ , 数组从内存首地址 BA 开始顺序存放, 当用以列为主存放时, 元素  $A[5, 8]$  的存储首地址为 ( )。

- A.  $BA + 141$   
B.  $BA + 180$   
C.  $BA + 222$   
D.  $BA + 225$

(8) 设有一个 10 阶的对称矩阵  $A$ , 采用压缩存储方式, 以行序为主存储,  $a_{11}$  为第一元素, 其存储地址为 1, 每个元素占一个地址空间, 则  $a_{85}$  的地址为 ( )。

- A. 13  
B. 32  
C. 33  
D. 40

(9) 若对  $n$  阶对称矩阵  $A$  以行序为主序方式将其下三角形的元素 (包括主对角线上所有元素) 依次存放于一维数组  $B[1..(n(n+1))/2]$  中, 则在  $B$  中确定  $a_{ij}$  ( $i < j$ ) 的位置  $k$  的关系为 ( )。

- A.  $i \times (i-1)/2 + j$   
B.  $j \times (j-1)/2 + i$   
C.  $i \times (i+1)/2 + j$   
D.  $j \times (j+1)/2 + i$

(10) 二维数组  $A$  的每个元素是由 10 个字符组成的串, 其行下标  $i=0, 1, \dots, 8$ , 列下标  $j=1, 2, \dots, 10$ 。若  $A$  按行先存储, 元素  $A[8, 5]$  的起始地址与当  $A$  按列先存储时的元素 ( ) 的起始地址相同。设每个字符占一个字节。

- A.  $A[8, 5]$   
B.  $A[3, 10]$

C.  $A[5, 8]$ D.  $A[0, 9]$ 

(11) 设二维数组  $A[1..m, 1..n]$  (即  $m$  行  $n$  列) 按行存储在数组  $B[1..m \times n]$  中, 则二维数组元素  $A[i, j]$  在一维数组  $B$  中的下标为 ( )。

A.  $(i-1) \times n + j$ B.  $(i-1) \times n + j - 1$ C.  $i \times (j-1)$ D.  $j \times m + i - 1$ 

(12) 数组  $A[0..4, -1..-3, 5..7]$  中含有元素的个数为 ( )。

A. 55

B. 45

C. 36

D. 16

(13) 广义表  $A = (a, b, (c, d), (e, (f, g)))$ , 则  $\text{Head}(\text{Tail}(\text{Head}(\text{Tail}(\text{Tail}(A))))$  的值为 ( )。

A. (g)

B. (d)

C. c

D. d

(14) 广义表  $((a, b, c, d))$  的表头是 ( ), 表尾是 ( )。

A. a

B. ()

C. (a, b, c, d)

D. (b, c, d)

(15) 设广义表  $L = ((a, b, c))$ , 则  $L$  的长度和深度分别为 ( )。

A. 1 和 1

B. 1 和 3

C. 1 和 2

D. 2 和 3

## 2. 应用题

(1) 已知模式串  $t = \text{"abcaabbabcbab"}$ , 写出用 KMP 法求得的每个字符对应的 next 和 nextval 函数值。

(2) 设目标为  $t = \text{"abcaabbabcbabacbacba"}$ , 模式为  $p = \text{"abcabaa"}$ 。

① 计算模式  $p$  的 nextval 函数值;

② 画出利用 KMP 算法进行模式匹配时每一趟的匹配过程。

(3) 数组  $A$  中, 每个元素  $A[i, j]$  的长度均为 32 个二进制位, 行下标从  $-1 \sim 9$ , 列下标从  $1 \sim 11$ , 从首地址  $S$  开始连续存放在主存储器中, 主存储器字长为 16 位。求:

① 存放该数组所需多少单元?

② 存放数组第 4 列所有元素至少需多少单元?

③ 数组按行存放时, 元素  $A[7, 4]$  的起始地址是多少?

④ 数组按列存放时, 元素  $A[4, 7]$  的起始地址是多少?

(4) 请将香蕉 banana 用工具  $H()$ — $\text{Head}()$ ,  $T()$ — $\text{Tail}()$  从  $L$  中取出。

$L = (\text{apple}, (\text{orange}, (\text{strawberry}, (\text{banana})), \text{peach}), \text{pear})$

## 3. 算法设计题

(1) 写一个算法统计在输入字符串中各个不同字符出现的频度并将结果存入文件 (字符串中的合法字符为  $A \sim Z$  这 26 个字母和  $0 \sim 9$  这 10 个数字)。

(2) 写一个递归算法来实现字符串逆序存储, 要求不另设串存储空间。

(3) 编写算法, 实现下面函数的功能。函数  $\text{void insert}(\text{char}^*s, \text{char}^*t, \text{int pos})$  将字符串  $t$  插入到字符串  $s$  中, 插入位置为  $\text{pos}$ 。假设分配给字符串  $s$  的空间足够让字符串  $t$  插入。(说明: 不得使用任何库函数)

(4) 已知字符串  $S1$  中存放一段英文, 写出算法  $\text{format}(s1, s2, s3, n)$ , 将其按给定的长度  $n$  格式化成两端对齐的字符串  $S2$ , 其多余的字符送  $S3$ 。

(5) 设二维数组  $a[1..m, 1..n]$  含有  $m \times n$  个整数。

① 写一个算法判断  $a$  中所有元素是否互不相同? 输出相关信息 (yes/no);

② 试分析算法的时间复杂度。

(6) 设任意  $n$  个整数存放于数组  $A[1..n]$  中, 试编写算法, 将所有正数排在所有负数前面 (要求: 算法时间复杂度为  $O(n)$ )。



树的结构定义是一个递归的定义,即在树的定义中又用到树的定义,它道出了树的固有特性。树还可有其他的表示形式,如图 5.2 所示为图 5.1 (b) 中树的各种表示。其中 (a) 是以嵌套集合 (即是一些集合的集体,对于其中任何两个集合,或者不相交,或者一个包含另一个) 的形式表示的 (b) 是以广义表的形式表示的,根作为由子树森林组成的表的名字写在表的左边; (c) 用的是凹入表示法 (类似书的编目)。表示方法的多样化,正说明了树结构在日常生活及计算机程序设计中的重要性。一般来说,分等级的分类方案都可用层次结构来表示,也就是说,都可由一个树结构来表示。

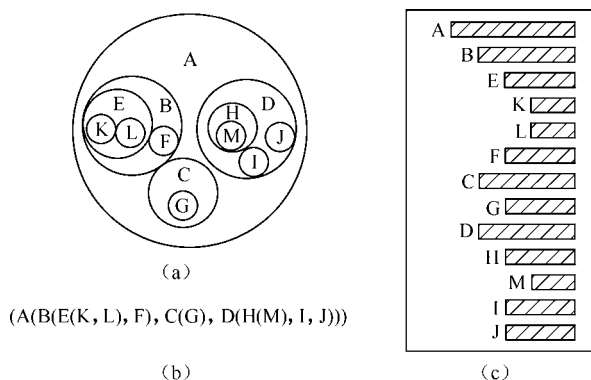


图 5.2 树的其他 3 种表示法

下面介绍树结构中的一些基本术语。

### 5.1.2 树的基本术语

(1) **结点**: 树中的一个独立单元。包含一个数据元素及若干指向其子树的分支,如图 5.1 (b) 中的 A、B、C、D 等。(下面术语中均以图 5.1 (b) 为例来说明)

(2) **结点的度**: 结点拥有的子树数称为结点的度。例如, A 的度为 3, C 的度为 1, F 的度为 0。

(3) **树的度**: 树的度是树内各结点度的最大值。图 5.1 (b) 所示的树的度为 3。

(4) **叶子**: 度为 0 的结点称为叶子或终端结点。结点 K、L、F、G、M、I、J 都是树的叶子。

(5) **非终端结点**: 度不为 0 的结点称为非终端结点或分支结点。除根结点之外,非终端结点也称为内部结点。

(6) **双亲和孩子**: 结点的子树的根称为该结点的孩子,相应地,该结点称为孩子的双亲。例如, B 的双亲为 A, B 的孩子有 E 和 F。

(7) **兄弟**: 同一个双亲的孩子之间互称兄弟。例如, H、I 和 J 互为兄弟。

(8) **祖先**: 从根到该结点所经分支上的所有结点。例如, M 的祖先为 A、D 和 H。

(9) **子孙**: 以某结点为根的子树中的任一结点都称为该结点的子孙。如 B 的子孙为 E、K、L 和 F。

(10) **层次**: 结点的层次从根开始定义起,根为第一层,根的孩子为第二层。树中任一结点的层次等于其双亲结点的层次加 1。

(11) **堂兄弟**: 双亲在同一层的结点互为堂兄弟。例如, 结点 G 与 E、F、H、I、J 互为

堂兄弟。

(12) **树的深度**：树中结点的最大层次称为树的深度或高度。图 5.1 (b) 所示的树的深度为 4。

(13) **有序树和无序树**：如果将树中结点的各子树看成从左至右是有次序的（即不能互换），则称该树为有序树，否则称为无序树。在有序树中最左边的子树的根称为第一个孩子，最右边的称为最后一个孩子。

(14) **森林**：是  $m$  ( $m \geq 0$ ) 棵互不相交的树的集合。对树中每个结点而言，其子树的集合即为森林。由此，也可以用森林和树相互递归的定义来描述树。

就逻辑结构而言，任何一棵树都是一个二元组  $Tree = (root, F)$ ，其中  $root$  是数据元素，称作树的根结点； $F$  是  $m$  ( $m \geq 0$ ) 棵树的森林， $F = (T_1, T_2, \dots, T_m)$ ，其中  $T_i = (r_i, F_i)$  称作根  $root$  的第  $i$  棵子树；当  $m \neq 0$  时，在树根和其子树森林之间存在下列关系：

$$RF = \{ \langle root, r_i \mid r_i = 1, 2, \dots, m \rangle \}$$

这个定义将有助于得到森林和树与二叉树之间转换的递归定义。

### 5.1.3 二叉树的定义

**二叉树 (Binary Tree)** 是  $n$  ( $n \geq 0$ ) 个结点所构成的集合，它或为空树 ( $n = 0$ )；或为非空树，对于非空树  $T$ ：

- (1) 有且仅有一个称之为根的结点；
- (2) 除根结点以外的其余结点分为两个互不相交的子集  $T_1$  和  $T_2$ ，分别称为  $T$  的左子树和右子树，且  $T_1$  和  $T_2$  本身又都是二叉树。

二叉树与树一样具有递归性质，二叉树与树的区别主要有以下两点：

- (1) 二叉树每个结点至多只有两棵子树（即二叉树中不存在度大于 2 的结点）；
- (2) 二叉树的子树有左右之分，其次序不能任意颠倒。

二叉树的递归定义表明二叉树或为空，或是由一个根结点加上两棵分别称为左子树和右子树的、互不相交的二叉树组成。由于这两棵子树也是二叉树，则由二叉树的定义，它们也可以是空树。由此，二叉树可以有 5 种基本形态，如图 5.3 所示。

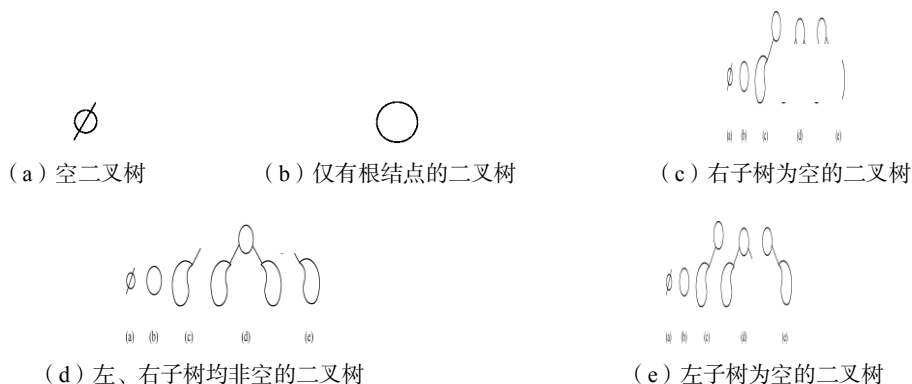


图 5.3 二叉树的 5 种基本形态

5.1.2 小节中引入的有关树的术语是都适用于二叉树。

## 5.2 案例引入

随着大数据时代的到来，如何采用有效的数据压缩技术，来节省数据文件的存储空间和网络传输时间越来越引起人们的重视。

### 案例 5.1：数据压缩问题。

在数据通信、数据压缩问题中，需要将数据文件转换成由二进制字符 0、1 组成的二进制串，称之为编码。

假设待压缩的数据为“abcdabcdaaaaabbbdd”，文件中只包含 a、b、c、d 四种字符，如果采用等长编码，每个字符编码取两位即可，表 5.1（a）所示为一种等长编码方案。上述数据为 18 个字符，其编码总长度为 36 位。但这并非最优的编码方案，因为每个字符出现的频率不同，如果在编码时考虑字符出现的频率，使频率高的字符采用尽可能短的编码，频率低的字符采用稍长的编码，来构造一种不等长编码，则会获得更好的空间效率，这也是文件压缩技术的核心思想。如表 5.1（b）所示为一种不等长编码方案，采用这种编码方案时，其编码总长度为 35 位。但对于不等长编码，如果设计得不合理，便会给解码带来困难。例如，对于如表 5.1（c）所示的另一种不等长编码方案，上述数据编码后为“0010111110010111110000001010111111”。但是，这样的编码数据无法翻译，例如，传送过去的字符串中前 4 个字符的子串“0010”就可有不同的译法，或是“aba”，或是“ac”。因此，若要设计长短不等的编码，必须满足一个条件：任何一个字符的编码都不是另一个字符的编码的前缀。表 5.1（b）所示的编码方案便满足这个条件。

那么如何设计有效的用于数据压缩的二进制编码呢？我们可以利用一种特殊的树结构——哈夫曼树来设计。表 5.1（b）所示的编码是以字符 a、b、c、d 在数据串“abcdabcdaaaaabbbdd”中出现的次数 7、5、2、4 为权值，构造如图 5.4 所示的哈夫曼树，其 4 个叶子结点分别表示 a、b、c、d 这 4 个字符，且约定左分支标记为 0，右分支标记为 1，则根结点到每个叶子结点路径上的 0、1 序列即为相应字符的编码。由图 5.4 所得 a、b、c、d 的二进制编码分别为 0、10、110 和 111。

有关哈夫曼树的构造及其编码设计的具体内容，将在 5.7 节详细介绍。

表 5.1

3 种不同的编码方案

| (a) 等长编码方案 |    | (b) 不等长编码方案 1 |     | (c) 不等长编码方案 2 |     |
|------------|----|---------------|-----|---------------|-----|
| 字符         | 编码 | 字符            | 编码  | 字符            | 编码  |
| a          | 00 | a             | 0   | a             | 0   |
| b          | 01 | b             | 10  | b             | 01  |
| c          | 10 | c             | 110 | c             | 010 |
| d          | 11 | d             | 111 | d             | 111 |

### 案例 5.2：利用二叉树求解表达式的值。

一般情况下，一个表达式由一个运算符和两个操作数构成，两个操作数之间有次序之分，并且操作数本身也可以是表达式，这个结构类似于二叉树，因此可以利用二叉树来表示表达式。

以二叉树表示表达式的递归定义如下：

（1）若表达式为数或简单变量，则相应二叉树中仅有一个根结点，其数据域存放该表达式信息；

(2) 若表达式为“第一操作数 运算符 第二操作数”的形式,则相应的二叉树中以左子树表示第一操作数,右子树表示第二操作数,根结点的数据域存放运算符(若为一元运算符,则左子树为空),其中,操作数本身又为表达式。

如图 5.5 所示的二叉树表示表达式  $a + b * (c - d) - e / f$ 。在二叉树中表达式中并无括号,但其结构却有效地表达了其运算符间的运算次序。利用二叉树的遍历等操作,可实现表达式的求值运算,有关内容将在 5.8 节详细介绍。

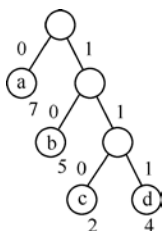
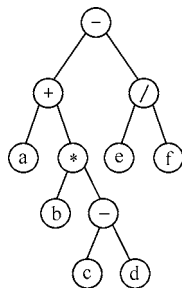


图 5.4 哈夫曼树及编码示例

图 5.5 表达式  $(a + b * (c - d) - e / f)$  的二叉树

## 5.3 树和二叉树的抽象数据类型定义

根据树的结构定义,加上树的一组基本操作就构成了树的抽象数据类型定义:

**ADT Tree{**

**数据对象 D:** D 是具有相同特性的数据元素的集合。

**数据关系 R:** 若 D 为空集,则称为空树;

若 D 仅含一个数据元素,则 R 为空集,否则  $R = \{H\}$ , H 是如下二元关系:

(1) 在 D 中存在唯一的称为根的数据元素 root,它在关系 H 下无前驱;

(2) 若  $D - \{\text{root}\} \neq \Phi$ ,则存在  $D - \{\text{root}\}$  的一个划分  $D_1, D_2, \dots, D_m$  ( $m > 0$ ),对任意  $j \neq k$  ( $1 \leq j, k \leq m$ ) 有  $D_j \cap D_k = \Phi$ ,且对任意的  $i$  ( $1 \leq i \leq m$ ),唯一存在数据元素  $x_i \in D_i$ ,有  $\langle \text{root}, x_i \rangle \in H$ ;

(3) 对应于  $D - \{\text{root}\}$  的划分,  $H - \{\langle \text{root}, x_i \rangle, \dots, \langle \text{root}, x_m \rangle\}$  有唯一的一个划分  $H_1, H_2, \dots, H_m$  ( $m > 0$ ),对任意  $j \neq k$  ( $1 \leq j, k \leq m$ ) 有  $H_j \cap H_k = \Phi$ ,且对任意  $i$  ( $1 \leq i \leq m$ ),  $H_i$  是  $D_i$  上的二元关系,  $(D_i, \{H_i\})$  是一棵符合本定义棵树,称为根 root 的子树。

**基本操作 P:**

InitTree(&T)

操作结果:构造空树 T。

DestroyTree(&T)

初始条件:树 T 存在。

操作结果:销毁树 T。

CreateTree(&T,definition)

初始条件:definition 给出树 T 的定义。

操作结果:按 definition 构造树 T。

ClearTree(&T)

初始条件:树 T 存在。

操作结果:将树 T 清为空树。

TreeEmpty(T)

初始条件: 树 T 存在。

操作结果: 若 T 为空树, 则返回 true, 否则 false。

TreeDepth(T)

初始条件: 树 T 存在。

操作结果: 返回 T 的深度。

Root(T)

初始条件: 树 T 存在。

操作结果: 返回 T 的根。

Value(T, cur\_e)

初始条件: 树 T 存在, cur\_e 是 T 中某个结点。

操作结果: 返回 cur\_e 的值。

Assign(T, cur\_e, value)

初始条件: 树 T 存在, cur\_e 是 T 中某个结点。

操作结果: 结点 cur\_e 赋值为 value。

Parent(T, cur\_e);

初始条件: 树 T 存在, cur\_e 是 T 中某个结点。

操作结果: 若 cur\_e 是 T 的非根结点, 则返回它的双亲, 否则函数值为“空”。

LeftChild(T, cur\_e)

初始条件: 树 T 存在, cur\_e 是 T 中某个结点。

操作结果: 若 cur\_e 是 T 的非叶子结点, 则返回它的最左孩子, 否则返回“空”。

RightSibling(T, cur\_e)

初始条件: 树 T 存在, cur\_e 是 T 中某个结点。

操作结果: 若 cur\_e 有右兄弟, 则返回它的右兄弟, 否则函数值为“空”。

InsertChild(&T, p, i, c)

初始条件: 树 T 存在, p 指向 T 中某个结点,  $1 \leq i \leq p$  所指结点的度 + 1, 非空树 c 与 T 不相交。

操作结果: 插入 c 为 T 中 p 所指结点的第 i 棵子树。

DeleteChild(&T, p, i)

初始条件: 树 T 存在, p 指向 T 中某个结点,  $1 \leq i \leq p$  所指结点的度。

操作结果: 删除 T 中 p 所指结点的第 i 棵子树。

TraverseTree(T)

初始条件: 树 T 存在。

操作结果: 按某种次序对 T 的每个结点访问一次。

} ADT Tree

二叉树的抽象数据类型定义如下:

ADT BinaryTree{

数据对象 D: D 是具有相同特性的数据元素的集合。

数据关系 R:

若  $D = \Phi$ , 则  $R = \Phi$ , 称 BinaryTree 为空二叉树;

若  $D \neq \Phi$ , 则  $R = \{H\}$ , H 是如下二元关系:

(1) 在 D 中存在唯一的称为根的数据元素 root, 它在关系 H 下无前驱;

(2) 若  $D - \{root\} \neq \Phi$ , 则存在  $D - \{root\} = \{D_1, D_r\}$ , 且  $D_1 \cap D_r = \Phi$ ;

(3) 若  $D_1 \neq \Phi$ , 则  $D_1$  中存在唯一的元素  $x_1$ ,  $\langle root, x_1 \rangle \in H$ , 且存在  $D_1$  上的关系  $H_1 \subset H$ ; 若  $D_r \neq \Phi$ , 则  $D_r$  中存在唯一的元素  $x_r$ ,  $\langle root, x_r \rangle \in H$ , 且存在  $D_r$  上的关系  $H_r \subset H$ ;  $H = \{\langle root, x_1 \rangle, \langle root, x_r \rangle, H_1, H_r\}$ ;

(4)  $(D_L, \{H_L\})$  是一棵符合本定义的二叉树, 称为根的左子树,  $(D_R, \{H_R\})$  是一棵符合本定义的二叉树, 称为根的右子树。

**基本操作 P:**

`InitBiTree(&T)`

操作结果: 构造空二叉树  $T$ 。

`DestroyBiTree(&T)`

初始条件: 二叉树  $T$  存在。

操作结果: 销毁二叉树  $T$ 。

`CreateBiTree(&T, definition)`

初始条件: `definition` 给出二叉树  $T$  的定义。

操作结果: 按 `definition` 构造二叉树  $T$ 。

`ClearBiTree(&T)`

初始条件: 二叉树  $T$  存在。

操作结果: 将二叉树  $T$  清为空树。

`BiTreeEmpty(T)`

初始条件: 二叉树  $T$  存在。

操作结果: 若  $T$  为空二叉树, 则返回 `true`, 否则 `false`。

`BiTreeDepth(T)`

初始条件: 二叉树  $T$  存在。

操作结果: 返回  $T$  的深度。

`Root(T)`

初始条件: 二叉树  $T$  存在。

操作结果: 返回  $T$  的根。

`Value(T, e)`

初始条件: 二叉树  $T$  存在,  $e$  是  $T$  中某个结点。

操作结果: 返回  $e$  的值。

`Assign(T, &e, value)`

初始条件: 二叉树  $T$  存在,  $e$  是  $T$  中某个结点。

操作结果: 结点  $e$  赋值为 `value`。

`Parent(T, e)`

初始条件: 二叉树  $T$  存在,  $e$  是  $T$  中某个结点。

操作结果: 若  $e$  是  $T$  的非根结点, 则返回它的双亲, 否则返回“空”。

`LeftChild(T, e)`

初始条件: 二叉树  $T$  存在,  $e$  是  $T$  中某个结点。

操作结果: 返回  $e$  的左孩子。若  $e$  无左孩子, 则返回“空”。

`RightChild(T, e)`

初始条件: 二叉树  $T$  存在,  $e$  是  $T$  中某个结点。

操作结果: 返回  $e$  的右孩子。若  $e$  无右孩子, 则返回“空”。

`LeftSibling(T, e)`

初始条件: 二叉树  $T$  存在,  $e$  是  $T$  中某个结点。

操作结果: 返回  $e$  的左兄弟。若  $e$  是  $T$  的左孩子或无左兄弟, 则返回“空”。

`RightSibling(T, e)`

初始条件: 二叉树  $T$  存在,  $e$  是  $T$  中某个结点。

操作结果: 返回  $e$  的右兄弟。若  $e$  是  $T$  的右孩子或无右兄弟, 则返回“空”。

`InsertChild(&T, p, LR, c)`

初始条件: 二叉树  $T$  存在,  $p$  指向  $T$  中某个结点,  $LR$  为 0 或 1, 非空二叉树  $c$  与  $T$  不相交且右子树为空。

操作结果: 根据  $LR$  为 0 或 1, 插入  $c$  为  $T$  中  $p$  所指结点的左或右子树。 $p$  所指结点的原有左或右子树则成为  $c$  的右子树。

```

DeleteChild(&T,p,LR)
    初始条件: 二叉树 T 存在, p 指向 T 中某个结点, LR 为 0 或 1。
    操作结果: 根据 LR 为 0 或 1, 删除 T 中 p 所指结点的左或右子树。
PreOrderTraverse(T)
    初始条件: 二叉树 T 存在。
    操作结果: 先序遍历 T, 对每个结点访问一次。
InOrderTraverse(T)
    初始条件: 二叉树 T 存在。
    操作结果: 中序遍历 T, 对每个结点访问一次。
PostOrderTraverse(T)
    初始条件: 二叉树 T 存在。
    操作结果: 后序遍历 T, 对每个结点访问一次。
LevelOrderTraverse(T)
    初始条件: 二叉树 T 存在。
    操作结果: 层序遍历 T, 对每个结点访问一次。
} ADT BinaryTree

```

## 5.4 二叉树的性质和存储结构

### 5.4.1 二叉树的性质

二叉树具有下列重要特性:

**性质 1** 在二叉树的第  $i$  层上至多有  $2^{i-1}$  个结点 ( $i \geq 1$ )。

**证明:** 利用归纳法容易证得此性质。

$i = 1$  时, 只有一个根结点。显然,  $2^{i-1} = 2^0 = 1$  是对的。

现在假定对所有的  $j (1 \leq j < i)$ , 命题成立, 即第  $j$  层上至多有  $2^{j-1}$  个结点。那么, 可以证明  $j = i$  时命题也成立。

由归纳假设: 第  $i-1$  层上至多有  $2^{i-2}$  个结点。由于二叉树每个结点的度至多为 2, 故在第  $i$  层上的最大结点数为第  $i-1$  层上的最大结点数的 2 倍, 即  $2 \times 2^{i-2} = 2^{i-1}$ 。

**性质 2** 深度为  $k$  的二叉树至多有  $2^k - 1$  个结点 ( $k \geq 1$ )。

**证明:** 由性质 1 可见, 深度为  $k$  的二叉树的最大结点数为

$$\sum_{i=1}^k (\text{第 } i \text{ 层上的最大结点数}) = \sum_{i=1}^k 2^{i-1} = 2^k - 1$$

**性质 3** 对任何一棵二叉树  $T$ , 如果其终端结点数为  $n_0$ , 度为 2 的结点数为  $n_2$ , 则  $n_0 = n_2 + 1$ 。

**证明:** 设  $n_1$  为二叉树  $T$  中度为 1 的结点数。因为二叉树中所有结点的度均小于或等于 2, 所以其结点总数为

$$n = n_0 + n_1 + n_2 \quad (5-1)$$

再看二叉树中的分支数。除了根结点外, 其余结点都有一个分支进入, 设  $B$  为分支总数, 则  $n = B + 1$ 。由于这些分支是由度为 1 或 2 的结点射出的, 所以又有  $B = n_1 + 2n_2$ 。

于是得

$$n = n_1 + 2n_2 + 1 \quad (5-2)$$

由式 (5-1) 和式 (5-2) 得

$$n_0 = n_2 + 1$$

现在介绍两种特殊形态的二叉树，它们是满二叉树和完全二叉树。

**满二叉树**：深度为  $k$  且含有  $2^k - 1$  个结点的二叉树。图 5.6 (a) 所示是一棵深度为 4 的满二叉树。

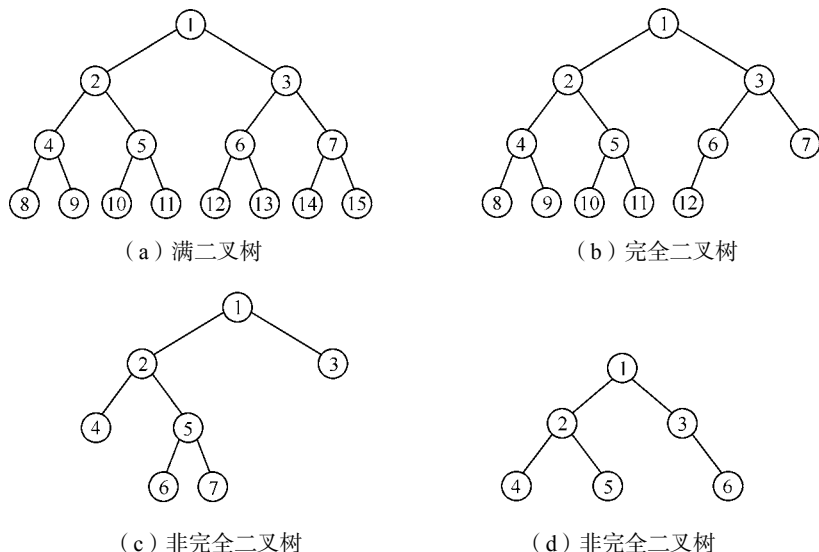


图 5.6 特殊形态的二叉树

满二叉树的特点是：每一层上的结点数都是最大结点数，即每一层  $i$  的结点数都具有最大值  $2^{i-1}$ 。

可以对满二叉树的结点进行连续编号，约定编号从根结点起，自上而下，自左至右。由此可引出完全二叉树的定义。

**完全二叉树**：深度为  $k$  的，有  $n$  个结点的二叉树，当且仅当其每一个结点都与深度为  $k$  的满二叉树中编号从 1 至  $n$  的结点一一对应时，称之为完全二叉树。图 5.6 (b) 所示为一棵深度为 4 的完全二叉树。

完全二叉树的特点是：

- (1) 叶子结点只可能在层次最大的两层上出现；
- (2) 对任一结点，若其右分支下的子孙的最大层次为  $l$ ，则其左分支下的子孙的最大层次必为  $l$  或  $l+1$ 。图 5.6 中 (c) 和 (d) 不是完全二叉树。

完全二叉树在很多场合下出现，下面的性质 4 和性质 5 是完全二叉树的两个重要特性。

**性质 4** 具有  $n$  个结点的完全二叉树的深度为  $\lfloor \log_2 n \rfloor + 1$ <sup>①</sup>。

**证明**：假设深度为  $k$ ，则根据性质 2 和完全二叉树的定义有

$$2^{k-1} - 1 < n \leq 2^k - 1 \quad \text{或} \quad 2^{k-1} \leq n < 2^k$$

于是  $k-1 \leq \log_2 n < k$ ，因为  $k$  是整数，所以  $k = \lfloor \log_2 n \rfloor + 1$ 。

**性质 5** 如果对一棵有  $n$  个结点的完全二叉树（其深度为  $\lfloor \log_2 n \rfloor + 1$ ）的结点按层序编号（从第 1 层到第  $\lfloor \log_2 n \rfloor + 1$  层，每层从左到右），则对任一结点  $i$  ( $1 \leq i \leq n$ )，有

- (1) 如果  $i = 1$ ，则结点  $i$  是二叉树的根，无双亲；如果  $i > 1$ ，则其双亲  $\text{PARENT}(i)$  是结

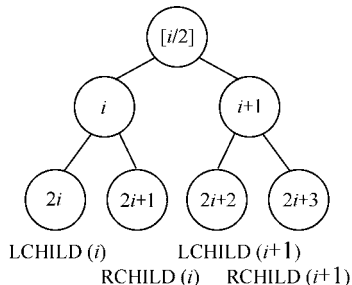
① 符号  $\lfloor x \rfloor$  表示不大于  $x$  的最大整数，反之， $\lceil x \rceil$  表示不小于  $x$  的最小整数。

点 $\lfloor i/2 \rfloor$ 。

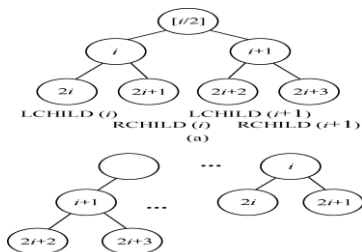
(2) 如果  $2i > n$ , 则结点  $i$  无左孩子 (结点  $i$  为叶子结点); 否则其左孩子  $LCHILD(i)$  是结点  $2i$ 。

(3) 如果  $2i + 1 > n$ , 则结点  $i$  无右孩子; 否则其右孩子  $RCHILD(i)$  是结点  $2i + 1$ 。

在此省略证明过程, 读者可由图 5.7 直观地看出性质 5 所描述的结点与编号的对应关系。



(a) 结点  $i$  和  $i + 1$  在同一层上



(b) 结点  $i$  和  $i + 1$  不在同一层上

图 5.7 完全二叉树中结点  $i$  和  $i + 1$  的左、右孩子

## 5.4.2 二叉树的存储结构

类似线性表, 二叉树的存储结构也可采用顺序存储和链式存储两种方式。

### 1. 顺序存储结构

//-----二叉树的顺序存储表示-----

```
#define MAXTSIZE 100
```

//二叉树的最大结点数

```
typedef TElemType SqBiTree[MAXTSIZE];
```

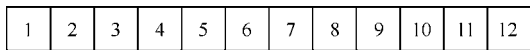
//0 号单元存储根结点

```
SqBiTree bt;
```

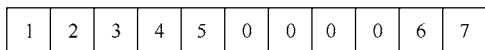
顺序存储结构使用一组地址连续的存储单元来存储数据元素, 为了能够在存储结构中反映出结点之间的逻辑关系, 必须将二叉树中的结点依照一定的规律安排在这组单元中。

对于完全二叉树, 只要从根起按层序存储即可, 依次自上而下、自左至右存储结点元素, 即将完全二叉树上编号为  $i$  的结点元素存储在如上定义的一维数组中下标为  $i-1$  的分量中。例如, 图 5.8 (a) 所示为图 5.6 (b) 所示完全二叉树的顺序存储结构。

对于一般二叉树, 则应将其每个结点与完全二叉树上的结点相对照, 存储在一维数组的相应分量中, 图 5.6 (c) 所示二叉树的顺序存储结构如图 5.8 (b) 所示, 图中以 “0” 表示不存在此结点。



(a) 完全二叉树



(b) 一般二叉树

图 5.8 二叉树的顺序存储结构

由此可见, 这种顺序存储结构仅适用于完全二叉树。因为, 在最坏的情况下, 一个深度为  $k$  且只有  $k$  个结点的单支树 (树中不存在度为 2 的结点) 却需要长度为  $2^k - 1$  的一维数组。这造成了存储空间的极大浪费, 所以对于一般二叉树, 更适合采取下面的链式存储结构。

### 2. 链式存储结构

设计不同的结点结构可构成不同形式的链式存储结构。由二叉树的定义得知, 二叉树的结点 (见图 5.9 (a)) 由一个数据元素和分别指向其左、右子树的两个分支构成, 则表示二叉树的链表

中的结点至少包含 3 个域：数据域和左、右指针域，如图 5.9 (b) 所示。有时，为了便于找到结点的双亲，还可在结点结构中增加一个指向其双亲结点的指针域，如图 5.9 (c) 所示。利用这两种结点结构所得二叉树的存储结构分别称之为二叉链表和三叉链表，如图 5.10 所示。链表的头指针指向二叉树的根结点。容易证得，在含有  $n$  个结点的二叉链表中有  $n+1$  个空链域。在 5.5 节中将会看到可以利用这些空链域存储其他有用信息，从而得到另一种链式存储结构——线索链表。



图 5.9 二叉树的结点及其存储结构

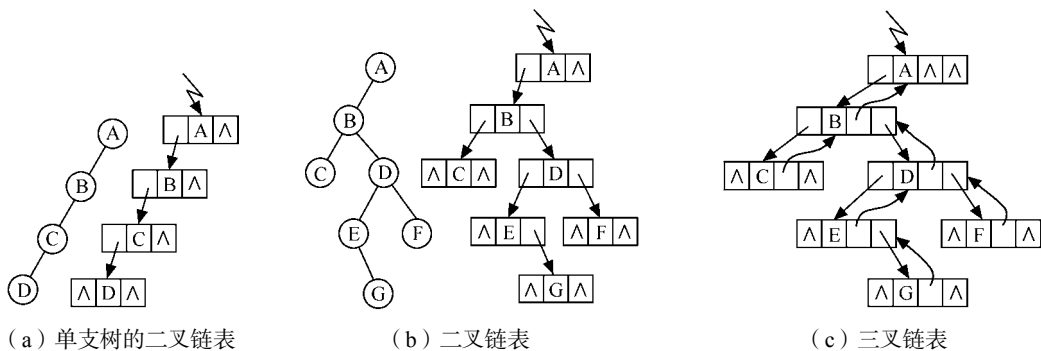


图 5.10 链表存储结构

在不同的存储结构中，实现二叉树的操作方法也不同，如找结点  $x$  的双亲  $PARENT(T, e)$ ，在三叉链表中很容易实现，而在二叉链表中则需从根指针出发巡查。由此，在具体应用中采用什么存储结构，除根据二叉树的形态之外还应考虑需进行何种操作。读者可试以 5.4.1 小节中定义的各种操作对以上定义的各种存储结构进行比较。在下一节的二叉树遍历及其应用的算法均采用以下定义的二叉链表形式实现。

```
// - - - - 二叉树的二叉链表存储表示 - - - -
typedef struct BiTNode{
    TElemType data;                // 结点数据域
    struct BiTNode *lchild, *rchild; // 左右孩子指针
}BiTNode, *BiTree;
```

## 5.5 遍历二叉树和线索二叉树

在二叉树的一些应用中，常常要求在树中查找具有某种特征的结点，或者是对树中的全部结点逐一进行处理，这就提出了一个遍历二叉树的问题。线索二叉树是在第一次遍历时将结点的前驱、后继信息存储下来，便于再次遍历二叉树。

## 5.5.1 遍历二叉树

### 1. 遍历二叉树算法描述

**遍历二叉树** (traversing binary tree) 是指按某条搜索路径巡访树中每个结点, 使得每个结点均被访问一次, 而且仅被访问一次。访问的含义很广, 可以是对结点做各种处理, 包括输出结点的信息, 对结点进行运算和修改等。遍历二叉树是二叉树最基本的操作, 也是二叉树其他各种操作的基础, 遍历的实质是对二叉树进行线性化的过程, 即遍历的结果是将非线性结构的树中结点排成一个线性序列。由于二叉树的每个结点都可能有两棵子树, 因而需要寻找一种规律, 以便使二叉树上的结点能排列在一个线性队列上, 从而便于遍历。

回顾二叉树的递归定义可知, 二叉树是由 3 个基本单元组成: 根结点、左子树和右子树。因此, 若能依次遍历这三部分, 便是遍历了整个二叉树。假如从 L、D、R 分别表示遍历左子树、访问根结点和遍历右子树, 则可有 DLR、LDR、LRD、DRL、RDL、RLD 这 6 种遍历二叉树的方案。若限定先左后右, 则只有前 3 种情况, 分别称之为先 (根) 序遍历、中 (根) 序遍历和后 (根) 序遍历。基于二叉树的递归定义, 可得下述遍历二叉树的递归算法定义。

先序遍历二叉树的操作定义如下:

若二叉树为空, 则空操作; 否则

- (1) 访问根结点;
- (2) 先序遍历左子树;
- (3) 先序遍历右子树。

中序遍历二叉树的操作定义如下:

若二叉树为空, 则空操作; 否则

- (1) 中序遍历左子树;
- (2) 访问根结点;
- (3) 中序遍历右子树。

后序遍历二叉树的操作定义如下:

若二叉树为空, 则空操作; 否则

- (1) 后序遍历左子树;
- (2) 后序遍历右子树;
- (3) 访问根结点。

例如, 图 5.5 所示的二叉树表示下述表达式

$$a + b * (c - d) - e / f$$

若先序遍历此二叉树, 按访问结点的先后次序将结点排列起来, 可得到二叉树的先序序列为

$$- + a * b - cd / ef \quad (5-3)$$

类似地, 中序遍历此二叉树, 可得此二叉树的中序序列为

$$a + b * c - d - e / f \quad (5-4)$$

后序遍历此二叉树, 可得此二叉树的后序序列为

$$abcd - * + ef / - \quad (5-5)$$

从表达式来看, 以上 3 个序列 (5-3)、(5-4) 和 (5-5) 恰好为表达式的前缀表示 (波兰式)、中缀表示和后缀表示 (逆波兰式)。

算法 5.1 给出了中序遍历二叉树基本操作的递归算法在二叉链表上的实现, 算法将结点的访

问简化成数据的输出。

### 算法 5.1 中序遍历的递归算法

#### 【算法描述】

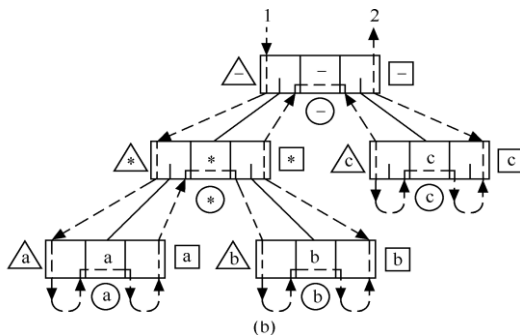
```
void InOrderTraverse(BiTree T)
{ //中序遍历二叉树 T 的递归算法
    if (T)                                //若二叉树非空
    {
        InOrderTraverse(T->lchild);      //中序遍历左子树
        cout<<T->data;                    //访问根结点
        InOrderTraverse(T->rchild);       //中序遍历右子树
    }
}
```

只要改变输出语句的顺序,读者便可类似地实现先序遍历和后序遍历的递归算法,此处不再一一列举。

从上述二叉树遍历的定义可知,3种遍历算法不同处仅在于访问根结点和遍历左、右子树的先后关系。如果在算法中暂且抹去和递归无关的 `cout` 语句,则3个遍历算法完全相同。由此,从递归执行过程的角度来看先序、中序和后序遍历也是完全相同的。图 5.11 (b) 中用带箭头的虚线表示了这3种遍历算法的递归执行过程。其中,向下的箭头表示更深一层的递归调用,向上的箭头表示从递归调用退出返回;虚线旁三角形、圆形和方形内的字符分别表示在先序、中序和后序遍历二叉树过程中访问结点时输出的信息。例如,由于中序遍历中访问结点是在遍历左子树之后、遍历右子树之前进行,则带圆形的字符标在向左递归返回和向右递归调用之间。由此,只要沿虚线从1出发到2结束,将沿途所见的三角形(或圆形、或方形)内的字符记下,便得到遍历二叉树的先序(或中序、或后序)序列。例如,从图 5.11 (b) 分别可得图 5.11 (a) 所示表达式的前缀表示 ( $-*abc$ )、中缀表示 ( $a*b-c$ ) 和后缀表示 ( $ab*c-$ )。



(a) 表达式  $(a * b - c)$  的二叉树



(b) 遍历的递归执行过程

图 5.11 3 种遍历过程示意图

根据前面 3.4.4 小节的内容,可利用栈将递归算法改写成非递归算法,如算法 5.2 所示。例如,从中序遍历递归算法执行过程中递归工作栈的状态可见:

(1) 工作记录中包含两项,其一是递归调用的语句编号,其二是指向根结点的指针,则当栈顶记录中的指针非空时,应遍历左子树,即指向左子树根的指针进栈;

(2) 若栈顶记录中的指针值为空,则应退至上一层,若是从左子树返回,则应访问当前层(即栈顶记录)中指针所指的根结点;

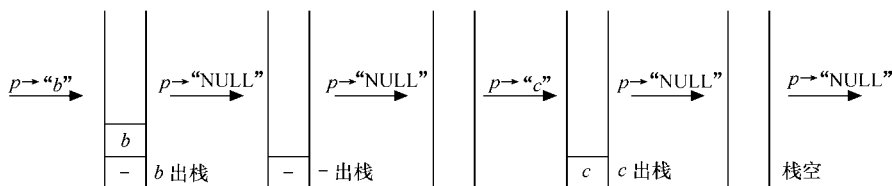
### 【算法步骤】

- ① 初始化一个空栈 S，指针 p 指向根结点。
- ② 申请一个结点空间 q，用来存放栈顶弹出的元素。
- ③ 当 p 非空或者栈 S 非空时，循环执行以下操作：
  - 如果 p 非空，则将 p 进栈，p 指向该结点的左孩子；
  - 如果 p 为空，则弹出栈顶元素并访问，将 p 指向该结点的右孩子。

```
void InOrderTraverse(BiTree T)
{ //中序遍历二叉树 T 的非递归算法
    InitStack(S); p=T;
    q=new BiTNode;
    while(p || !StackEmpty(S))
    {
        if(p) //p 非空
        {
            Push(S,p); //根指针进栈
            p=p->lchild; //根指针进栈，遍历左子树
        }
        else //p 为空
        {
            Pop(S,q); //退栈
            cout<<q->data; //访问根结点
            p=q->rchild; //遍历右子树
        }
    } // while
}
```

Diagram illustrating the execution of the algorithm for the expression  $(a * a) * (a * a)$ . The diagram shows a sequence of stack operations:

- Initial state: Stack contains  $-$ .
- Operation  $p \rightarrow "-"$ : Push  $-$ . Stack contains  $-$ .
- Operation  $p \rightarrow "*"$ : Push  $*$ . Stack contains  $-$ ,  $*$ .
- Operation  $p \rightarrow "a"$ : Push  $a$ . Stack contains  $-$ ,  $*$ ,  $a$ .
- Operation  $p \rightarrow "NULL"$ : Pop  $a$  (labeled  $a$  出栈). Stack contains  $-$ ,  $*$ .
- Operation  $p \rightarrow "NULL"$ : Pop  $*$  (labeled  $*$  出栈). Stack contains  $-$ .



124

## 【算法分析】

无论是递归还是非递归遍历二叉树，因为每个结点被访问一次，则不论按哪一种次序进行遍历，对含  $n$  个结点的二叉树，其时间复杂度均为  $O(n)$ 。所需辅助空间为遍历过程中栈的最大容量，即树的深度，最坏情况下为  $n$ ，则空间复杂度也为  $O(n)$ 。

二叉树的先序、中序和后序遍历是最常用的三种遍历方式。此外，还有一种按层次遍历二叉树的方式，这种方式按照“从上到下，从左到右”的顺序遍历二叉树，即先遍历二叉树第一层的结点，然后是第二层的结点，直到最底层的结点，对每一层的遍历按照从左到右的次序进行。例如，图 5.11 (a) 所示的二叉树的层次遍历序列是-\*cab。层次遍历不是一个递归过程，层次遍历算法的实现可以借助队列这种数据结构，这里不做详细讨论，算法留给读者自行完成。

## 2. 根据遍历序列确定二叉树

从前面讨论的二叉树的遍历知道，若二叉树中各结点的值均不相同，任意一棵二叉树结点的先序序列、中序序列和后序序列都是唯一的。反过来，若已知二叉树遍历的任意两种序列，能否确定这棵二叉树呢？这样确定的二叉树是否是唯一的呢？

由二叉树的先序序列和中序序列，或由其后续序列和中序序列均能唯一地确定一棵二叉树。

根据定义，二叉树的先序遍历是先访问根结点，其次再按先序遍历方式遍历根结点的左子树，最后按先序遍历方式遍历根结点的右子树。这就是说，在先序序列中，第一个结点一定是二叉树的根结点。另一方面，中序遍历是先遍历左子树，然后访问根结点，最后再遍历右子树。这样，根结点在中序序列中必然将中序序列分割成两个子序列，前一个子序列是根结点的左子树的中序序列，而后一个子序列是根结点的右子树的中序序列。根据这两个子序列，在先序序列中找到对应的左子序列和右子序列。在先序序列中，左子序列的第一个结点是左子树的根结点，右子序列的第一个结点是右子树的根结点。这样，就确定了二叉树的三个结点。同时，左子树和右子树的根结点又可以分别把左子序列和右子序列划分成两个子序列，如此递归下去，当取尽先序序列中的结点时，便可以得到一棵二叉树。

同理，由二叉树的后序序列和中序序列也可唯一地确定一棵二叉树。因为，依据后序遍历和中序遍历的定义，后序序列的最后一个结点，就如同先序序列的第一个结点一样，可将中序序列分成两个子序列，分别为这个结点左子树的中序序列和右子树的中序序列，再拿出后序序列的倒数第二个结点，并继续分割中序序列，如此递归下去，当倒着取尽后序序列中的结点时，可以得到一棵二叉树。

**【例 5.1】** 已知一棵二叉树的中序序列和后序序列分别是 BDCEAFHG 和 DECBHGFA，请画出这棵二叉树。

(1) 由后序遍历特征，根结点必在后序序列尾部，即根结点是 A；

(2) 由中序遍历特征，根结点必在其中间，而且其左部必全部是左子树子孙 (BDCE)，其右部必全部是右子树子孙 (FHG)；

(3) 继而，根据后序中的 DECB 子树可确定 B 为 A 的左孩子，根据 HGF 子串可确定 F 为 A 的右孩子；依此类推，可以唯一地确定一棵二叉树，如图 5.13 所示。

但是，由一棵二叉树的先序序列和后序序列不能唯一确定一棵二叉树，因为无法确定左右子树两部分。例如，如果有先序序列 AB，后序序列 BA，因为无法确定 B 为左子树还是右子树，所以可得到如图 5.14 所示的两棵不同的二叉树。

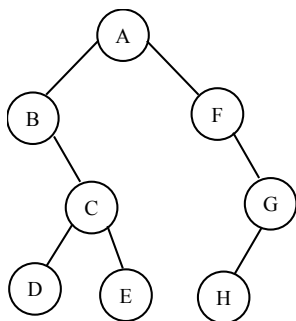


图 5.13 由中序序列和后序序列确定的二叉树

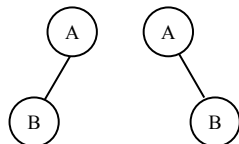


图 5.14 两棵不同的二叉树

### 3. 二叉树遍历算法的应用

“遍历”是二叉树各种操作的基础，假设访问结点的具体操作不仅仅局限于输出结点数据域的值，而把“访问”延伸到对结点的判别、计数等其他操作，可以解决一些关于二叉树的其他实际问题。如果在遍历过程中生成结点，这样便可建立二叉树的存储结构。

#### (1) 创建二叉树的存储结构——二叉链表

为简化问题，设二叉树中结点的元素均为一个单字符。假设按先序遍历的顺序建立二叉链表，T 为指向根结点的指针，对于给定的一个字符序列，依次读入字符，从根结点开始，递归创建二叉树。

#### 算法 5.3 先序遍历的顺序建立二叉链表

##### 【算法步骤】

- ① 扫描字符序列，读入字符 ch。
- ② 如果 ch 是一个“#”字符，则表明该二叉树为空树，即 T 为 NULL；否则执行以下操作：
  - 申请一个结点空间 T；
  - 将 ch 赋给 T->data；
  - 递归创建 T 的左子树；
  - 递归创建 T 的右子树；

##### 【算法描述】

```

void CreateBiTree(BiTree &T)
{ //按先序次序输入二叉树中结点的值 (一个字符)，创建二叉链表表示的二叉树 T
    cin>>ch;
    if(ch=='#') T=NULL;           //递归结束，建空树
    else                          //递归创建二叉树
    {
        T=new BiTNode;           //生成根结点
        T->data=ch;               //根结点数据域置为 ch
        CreateBiTree(T->lchild);  //递归创建左子树
        CreateBiTree(T->rchild);  //递归创建右子树
    }
    //else
}
    
```

例如，对图 5.10 (b) 所示的二叉树，读入字符的顺序为：ABC##DE#G##F### (其中#表示空树)，可建立相应的二叉链表。

## (2) 复制二叉树

复制二叉树就是利用已有的一棵二叉树复制得到另外一棵与其完全相同的二叉树。根据二叉树的特点,复制步骤如下:若二叉树不空,则首先复制根结点,这相当于二叉树先序遍历算法中访问根结点的语句;然后分别复制二叉树根结点的左子树和右子树,这相当于先序遍历中递归遍历左子树和右子树的语句。因此,复制函数的实现与二叉树先序遍历的实现非常类似。

### 算法 5.4 复制二叉树

#### 【算法步骤】

如果是空树,递归结束,否则执行以下操作:

- 申请一个新结点空间,复制根结点;
- 递归复制左子树;
- 递归复制右子树。

#### 【算法描述】

```
void Copy(BiTree T,BiTree &NewT)
{//复制一棵和 T 完全相同的二叉树
    if(T==NULL)                                //如果是空树,递归结束
    {
        NewT=NULL;
        return;
    }
    else
    {
        NewT=new BiTNode;
        NewT->data=T->data;                    //复制根结点
        Copy(T->lchild,NewT->lchild);          //递归复制左子树
        Copy(T->rchild,NewT->rchild);          //递归复制右子树
    }
    //else
}
```

## (3) 计算二叉树的深度

二叉树的深度为树中结点的最大层次,二叉树的深度为左右子树深度的较大者加 1。

### 算法 5.5 计算二叉树的深度

#### 【算法步骤】

如果是空树,递归结束,深度为 0,否则执行以下操作:

- 递归计算左子树的深度记为  $m$ ;
- 递归计算右子树的深度记为  $n$ ;
- 如果  $m$  大于  $n$ ,二叉树的深度为  $m+1$ ,否则为  $n+1$ 。

#### 【算法描述】

```
int Depth(BiTree T)
{//计算二叉树 T 的深度
    if(T==NULL) return 0;                      //如果是空树,深度为 0,递归结束
    else
    {
        m=Depth(T->lchild);                    //递归计算左子树的深度记为 m
        n=Depth(T->rchild);                    //递归计算右子树的深度记为 n
        if(m>n) return (m+1);                  //二叉树的深度为 m 与 n 的较大者加 1
    }
}
```

```

        else return(n+1);
    }
}

```

显然, 计算二叉树的深度是在后序遍历二叉树的基础上进行的运算。

#### (4) 统计二叉树中结点的个数

如果是空树, 则结点个数为 0; 否则, 结点个数为左子树的结点个数加上右子树的结点个数再加上 1。

#### 算法 5.6 统计二叉树中结点的个数

##### 【算法描述】

```

int NodeCount(BiTree T)
{ //统计二叉树 T 中结点的个数
    if (T==NULL) return 0;           //如果是空树, 则结点个数为 0, 递归结束
    else return NodeCount(T->lchild)+NodeCount(T->rchild)+1;
    //否则结点个数为左子树的结点个数+右子树的结点个数+1
}

```

读者可以模仿此算法, 写出以下算法: 统计二叉树中叶结点 (度为 0) 的个数, 度为 1 的结点个数和度为 2 的结点个数。算法实现的关键是如何表示度为 0、度为 1 或度为 2 的结点。

## 5.5.2 线索二叉树

### 1. 线索二叉树的基本概念

遍历二叉树是以一定规则将二叉树中的结点排列成一个线性序列, 得到二叉树中结点的先序序列、中序序列或后序序列。这实质上是对一个非线性结构进行线性化操作, 使每个结点 (除第一个和最后一个外) 在这些线性序列中有且仅有一个直接前驱和直接后继 (在不至于混淆的情况, 后续描述中省去“直接”二字)。例如在图 5.5 所示的二叉树结点的中序序列  $a+b*c-d-e/f$  中, “c”的前驱是 “\*”, 后继是 “-”。

但是, 当以二叉链表作为存储结构时, 只能找到结点的左、右孩子信息, 而不能直接得到结点在任一序列中的前驱和后继信息, 这种信息只有在遍历的动态过程中才能得到, 为此引入线索二叉树来保存这些在动态过程中得到的有关前驱和后继的信息。

虽然可以在每个结点中增加两个指针域来存放在遍历时得到的有关前驱和后继信息, 但这样做使得结构的存储密度大大降低。由于有  $n$  个结点的二叉链表中必定存在  $n+1$  个空链域, 因此可以充分利用这些空链域来存放结点的前驱和后继信息。

试做如下规定: 若结点有左子树, 则其 lchild 域指示其左孩子, 否则令 lchild 域指示其前驱; 若结点有右子树, 则其 rchild 域指示其右孩子, 否则令 rchild 域指示其后继。为了避免混淆, 尚需改变结点结构, 增加两个标志域, 其结点形式如图 5.15 所示。

|        |      |      |      |        |
|--------|------|------|------|--------|
| lchild | LTag | data | RTag | rchild |
|--------|------|------|------|--------|

图 5.15 线索二叉树的结点形式

其中:

$$LTag = \begin{cases} 0 & \text{域指示结点的左孩子} \\ 1 & \text{lchild 域指示结点的前驱} \end{cases}$$

$$RTag = \begin{cases} 0 & \text{域指示结点的左孩子} \\ 1 & \text{域指示结点的后继} \end{cases}$$

二叉树的二叉线索类型定义如下：

```
// - - - - 二叉树的二叉线索存储表示 - - - -
typedef struct BiThrNode
{
    TElemType data;
    struct BiThrNode *lchild, *rchild;    // 左右孩子指针
    int LTag, RTag;                       // 左右标志
} BiThrNode, *BiThrTree;
```

以这种结点结构构成的二叉链表作为二叉树的存储结构，叫做**线索链表**，其中指向结点前驱和后继的指针，叫做**线索**。加上线索的二叉树称之为**线索二叉树**（Threaded Binary Tree）。对二叉树以某种次序遍历使其变为线索二叉树的过程叫做**线索化**。

例如图 5.16（a）所示为中序线索二叉树，与其对应的中序线索链表如图 5.16（b）所示。其中实线为指针（指向左、右子树），虚线为线索（指向前驱和后继）。为了方便起见，仿照线性表的存储结构，在二叉树的线索链表上也添加一个头结点，并令其 lchild 域的指针指向二叉树的根结点，其 rchild 域的指针指向中序遍历时的最后一个结点；同时，令二叉树中中序序列中第一个结点的 lchild 域指针和最后一个结点 rchild 域的指针均指向头结点。这好比为二叉树建立了一个双向线索链表，既可从第一个结点起顺后继进行遍历，也可从最后一个结点起顺前驱进行遍历。

## 2. 构造线索二叉树

由于线索二叉树构造的实质是将二叉链表中的空指针改为指向前驱或后继的线索，而前驱或后继的信息只有在遍历时才能得到，因此线索化的过程即为在遍历的过程中修改空指针的过程，可用递归算法。对二叉树按照不同的遍历次序进行线索化，可以得到不同的线索二叉树，包括先序线索二叉树、中序线索二叉树和后序线索二叉树。下面重点介绍中序线索化的算法。

为了记下遍历过程中访问结点的先后关系，附设一个指针 pre 始终指向刚刚访问过的结点，而指针 p 指向当前访问的结点，由此记录下遍历过程中访问结点的先后关系。算法 5.7 是对树中任意一个结点 p 为根的子树中序线索化的过程，算法 5.8 通过调用算法 5.7 来完成整个二叉树的中序线索化。

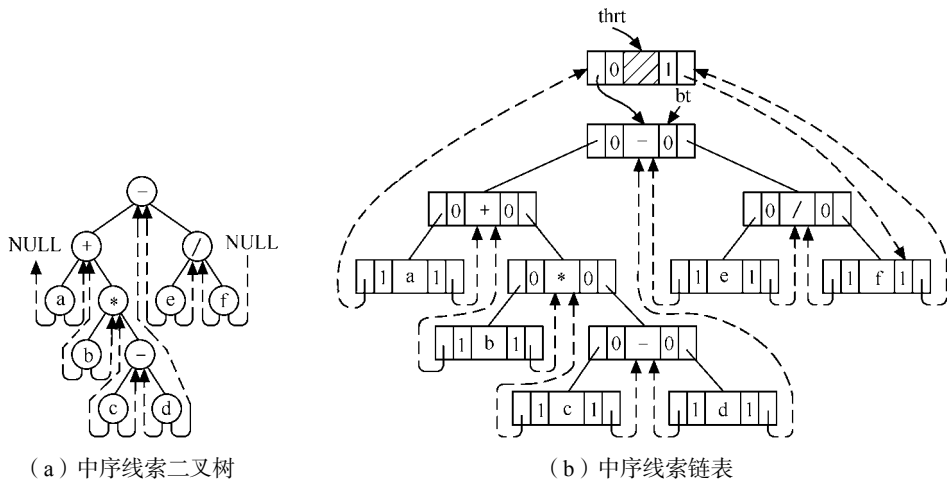


图 5.16 线索二叉树及其存储结构

### 算法 5.7 以结点 p 为根的子树中序线索化

#### 【算法步骤】

- ① 如果 p 非空, 左子树递归线索化。
- ② 如果 p 的左孩子为空, 则给 p 加上左线索, 将其 LTag 置为 1, 让 p 的左孩子指针指向 pre (前驱); 否则将 p 的 LTag 置为 0。
- ③ 如果 pre 的右孩子为空, 则给 pre 加上右线索, 将其 RTag 置为 1, 让 pre 的右孩子指针指向 p (后继); 否则将 pre 的 RTag 置为 0。
- ④ 将 pre 指向刚访问过的结点 p, 即  $pre = p$ 。
- ⑤ 右子树递归线索化。

#### 【算法描述】

```
void InThreading(BiThrTree p)
{ //pre 是全局变量, 初始化时其右孩子指针为空, 便于在树的最左点开始建线索
    if(p)
    {
        InThreading(p->lchild);           //左子树递归线索化
        if(!p->lchild)                     //p 的左孩子为空
        {
            p->LTag=1;                     //给 p 加上左线索
            p->lchild=pre;                  //p 的左孩子指针指向 pre (前驱)
        }                                 //if
        else p->LTag=0;
        if(!pre->rchild)                    //pre 的右孩子为空
        {
            pre->RTag=1;                    //给 pre 加上右线索
            pre->rchild=p;                  //pre 的右孩子指针指向 p (后继)
        }                                 //if
        else p->RTag=0;
        pre=p;                             //保持 pre 指向 p 的前驱
        InThreading(p->rchild);             //右子树递归线索化
    }
}
```

### 算法 5.8 带头结点的二叉树中序线索化

#### 【算法描述】

```
void InOrderThreading(BiThrTree &Thrt, BiThrTree T)
{ //中序遍历二叉树 T, 并将其中序线索化, Thrt 指向头结点
    Thrt=new BiThrNode;                   //建头结点
    Thrt->LTag=0;                          //头结点有左孩子, 若树非空, 则其左孩子为树根
    Thrt->RTag=1;                          //头结点的右孩子指针为右线索
    Thrt->rchild=Thrt;                     //初始化时右指针指向自己
    if(!T) Thrt->lchild=Thrt;              //若树为空, 则左指针也指向自己
    else
    {
        Thrt->lchild=T; pre=Thrt;          //头结点的左孩子指向根, pre 初值指向头结点
        InThreading(T);                    //调用算法 5.7, 对以 T 为根的二叉树进行中序线索化
        pre->rchild=Thrt;                  //算法 5.7 结束后, pre 为最右结点, pre 的右线索指向头结点
        pre->RTag=1;
    }
}
```

```

        Thrt->rchild=pre;           //头结点的右线索指向 pre
    }
}

```

### 3. 遍历线索二叉树

由于有了结点的前驱和后继信息, 线索二叉树的遍历和在指定次序下查找结点的前驱和后继算法都变得简单。因此, 若需经常查找结点在所遍历线性序列中的前驱和后继, 则采用线索链表作为存储结构。

下面分 3 种情况讨论在线索二叉树中如何查找结点的前驱和后继。

#### (1) 在中序线索二叉树中查找

##### ① 查找 p 指针所指结点的前驱:

- 若 p->LTag 为 1, 则 p 的左链指示其前驱;
- 若 p->LTag 为 0, 则说明 p 有左子树, 结点的前驱是遍历左子树时最后访问的一个结点 (左子树中最右下的结点)。

##### ② 查找 p 指针所指结点的后继:

- 若 p->RTag 为 1, 则 p 的右链指示其后继, 以图 5.16 所示的中序线索树为例来看, 结点 b 的后继为结点\*;
- 若 p->RTag 为 0, 则说明 p 有右子树。根据中序遍历的规律可知, 结点的后继应是遍历其右子树时访问的第一个结点, 即右子树中最左下的结点。例如在找结点\*的后继时, 首先沿右指针找到其右子树的根结点 -, 然后顺其左指针往下直至其左标志为 1 的结点, 即为结点\*的后继, 在图中是结点 c。

#### (2) 在先序线索二叉树中查找

##### ① 查找 p 指针所指结点的前驱:

- 若 p->LTag 为 1, 则 p 的左链指示其前驱;
- 若 p->LTag 为 0, 则说明 p 有左子树。此时 p 的前驱有两种情况: 若\*p 是其双亲的左孩子, 则其前驱为其双亲结点; 否则应是其双亲的左子树上先序遍历最后访问到的结点。

##### ② 查找 p 指针所指结点的后继:

- 若 p->RTag 为 1, 则 p 的右链指示其后继;
- 若 p->RTag 为 0, 则说明 p 有右子树。按先序遍历的规则可知, \*p 的后继必为其左子树根 (若存在) 或右子树根。

#### (3) 在后序线索二叉树中查找

##### ① 查找 p 指针所指结点的前驱:

- 若 p->LTag 为 1, 则 p 的左链指示其前驱;
- 若 p->LTag 为 0, 当 p->RTag 也为 0 时, 则 p 的右链指示其前驱; 若 p->LTag 为 0, 而 p->RTag 为 1 时, 则 p 的左链指示其前驱。

##### ② 查找 p 指针所指结点的后继情况比较复杂, 分以下情况讨论:

- 若\*p 是二叉树的根, 则其后继为空;
- 若\*p 是其双亲的右孩子, 则其后继为双亲结点;
- 若\*p 是其双亲的左孩子, 且\*p 没有右兄弟, 则其后继为双亲结点;
- 若\*p 是其双亲的左孩子, 且\*p 有右兄弟, 则其后继为双亲的右子树上按后序遍历列出的第一个结点 (即右子树中“最左下”的叶结点)。

例如, 图 5.17 所示为后序线索二叉树, 结点 B 的后继为结点 C, 结点 C 的后继为结点 D, 结点 F 的后继为结点 G, 而结点 D 的后继为结点 E。

可见, 在先序线索化树上找前驱或在后序线索化树上找后继时都比较复杂, 此时若需要, 可直接建立含 4 个指针的线索链表。

由于有了结点的前驱和后继的信息, 线索二叉树的遍历操作无需设栈, 避免了频繁的进栈、出栈, 因此在时间和空间上都较遍历二叉树节省。如果遍历某种次序的线索二叉树, 则只要从该次序下的根结点出发, 反复查找其在该次序下的后继, 直到叶子结点。下面以遍历中序线索二叉树为例介绍该算法。

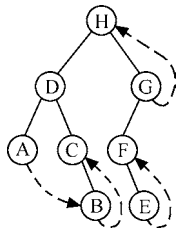


图 5.17 后序后继线索二叉树

### 算法 5.9 遍历中序线索二叉树

#### 【算法步骤】

- ① 指针 p 指向根结点。
- ② p 为非空树或遍历未结束时, 循环执行以下操作:
  - 沿左孩子向下, 到达最左下结点 \*p, 它是中序的第一个结点;
  - 访问 \*p;
  - 沿右线索反复查找当前结点 \*p 的后继结点并访问后继结点, 直至右线索为 0 或者遍历结束;
  - 转向 p 的右子树。

#### 【算法描述】

```
void InOrderTraverse_Thr(BiThrTree T)
{ //T 指向头结点, 头结点的左链 lchild 指向根结点, 可参见线索化算法 5.8。
  //中序遍历二叉线索树 T 的非递归算法, 对每个数据元素直接输出
  p=T->lchild; //p 指向根结点
  while (p!=T) //空树或遍历结束时, p==T
  {
    while (p->LTag==0) p=p->lchild; //沿左孩子向下
    cout<<p->data; //访问其左子树为空的结点
    while (p->RTag==1&&p->rchild!=T)
    {
      p=p->rchild; cout<<p->data; //沿右线索访问后继结点
    }
    p=p->rchild; //转向 p 的右子树
  }
}
```

#### 【算法分析】

遍历线索二叉树的时间复杂度为  $O(n)$ , 空间复杂度为  $O(1)$ , 这是因为线索二叉树的遍历不需要使用栈来实现递归操作。

## 5.6 树和森林

本节将讨论树的表示及其遍历操作, 并建立森林与二叉树的对应关系。

### 5.6.1 树的存储结构

在大量的应用中,人们曾使用多种形式的存储结构来表示树。这里介绍 3 种常用的表示方法。

#### 1. 双亲表示法

这种表示方法中,以一组连续的存储单元存储树的结点,每个结点除了数据域 data 外,还附设一个 parent 域用以指示其双亲结点的位置,其结点形式如图 5.18 所示。

例如,图 5.19 所示为一棵树及其双亲表示的存储结构。

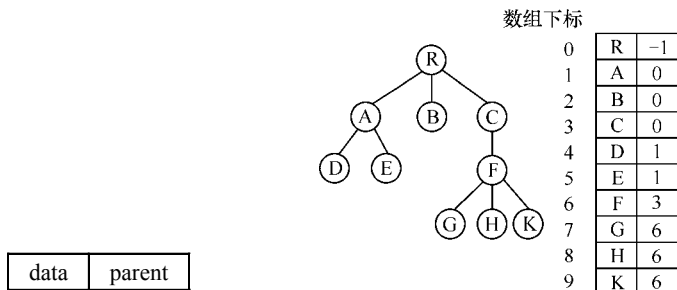


图 5.18 双亲表示法的结点形式

图 5.19 树的双亲表示法示例

这种存储结构利用了每个结点(除根以外)只有唯一的双亲的性质。在这种存储结构下,求结点的双亲十分方便,也很容易求树的根,但求结点的孩子时需要遍历整个结构。

#### 2. 孩子表示法

由于树中每个结点可能有多棵子树,则可用多重链表,即每个结点有多个指针域,其中每个指针指向一棵子树的根结点,此时链表中的结点可以有如图 5.20 所示的两种结点格式。

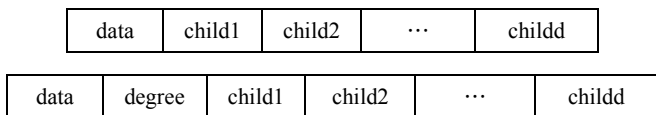


图 5.20 孩子表示法的两种结点

若采用第一种结点格式,则多重链表中的结点是同构的,其中  $d$  为树的度。由于树中很多结点的度小于  $d$ ,所以链表中有许多空链域,空间较浪费,不难推出,在一棵有  $n$  个结点度为  $k$  的树中必有  $n(k-1)+1$  个空链域。

若采用第二种结点格式,则多重链表中的结点是不同构的,其中  $d$  为结点的度,degree 域的值同  $d$ 。此时,虽能节约存储空间,但操作不方便。

另一种办法是,把每个结点的孩子结点排列起来,看成是一个线性表,且以单链表做存储结构,则  $n$  个结点有  $n$  个孩子链表(叶子的孩子链表为空表)。而  $n$  个头指针又组成一个线性表,为了便于查找,可采用顺序存储结构。

图 5.21 (a) 所示为图 5.19 中的树的孩子表示法。与双亲表示法相反,孩子表示法便于那些涉及孩子的操作的实现。可以把双亲表示法和孩子表示法结合起来,即将双亲表示和孩子链表合在一起。图 5.21 (b) 所示的就是这种存储结构的一例,它和图 5.21 (a) 表示的是同一棵树。

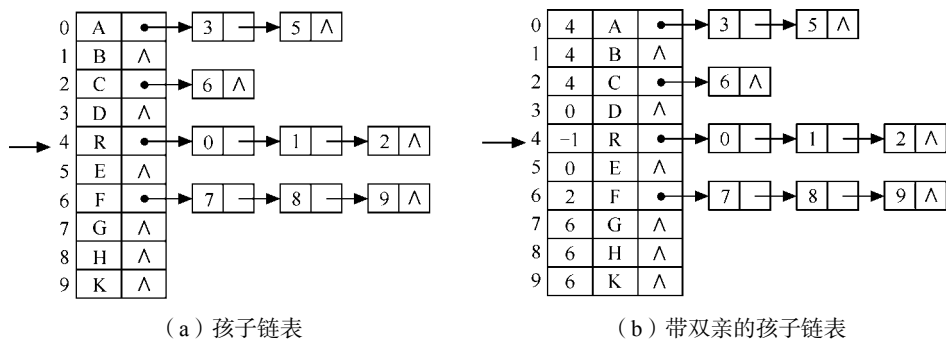


图 5.21 图 5.19 的树的另外两种表示法

### 3. 孩子兄弟法

又称二叉树表示法，或二叉链表表示法，即以二叉链表做树的存储结构。链表中结点的两个域分别指向该结点的第一个孩子结点和下一个兄弟结点，分别命名为 `firstchild` 域和 `nextsibling` 域，其结点形式如图 5.22 所示。

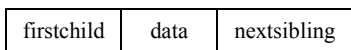


图 5.22 孩子兄弟表示法的结点

```
//- - - - -树的二叉链表（孩子-兄弟）存储表示- - - - -
typedef struct CSNode{
    ElemType data;
    struct CSNode *firstchild,*nextsibling;
}CSNode,*CSTree;
```

图 5.23 所示为图 5.19 中的树的孩子兄弟链表。利用这种存储结构便于实现各种树的操作。首先易于实现找结点孩子等的操作。例如，若要访问结点 `x` 的第  $i$  个孩子，则只要先从 `firstchild` 域找到第 1 个孩子结点，然后沿着孩子结点的 `nextsibling` 域连续走  $i-1$  步，便可找到 `x` 的第  $i$  个孩子。当然，如果为每个结点增设一个 `parent` 域，则同样能方便地实现查找双亲的操作。

这种存储结构的优点是它和二叉树的二叉链表表示完全一样，便于将一般的树结构转换为二叉树进行处理，利用二叉树的算法来实现对树的操作。因此孩子兄弟表示法是应用较为普遍的一种树的存储表示方法。

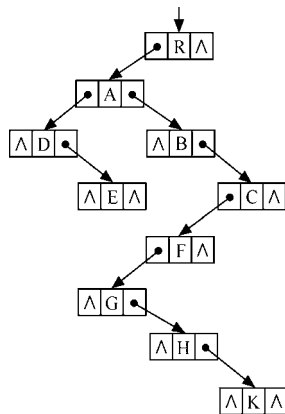


图 5.23 图 5.19 中树的二叉链表表示法

## 5.6.2 森林与二叉树的转换

从树的二叉链表表示的定义可知，任何一棵和树对应的二叉树，其根结点的右子树必空。若把森林中第二棵树的根结点看成是第一棵树的根结点的兄弟，则同样可导出森林和二叉树的对应关系。

例如，图 5.24 所示为森林与二叉树之间的对应关系。

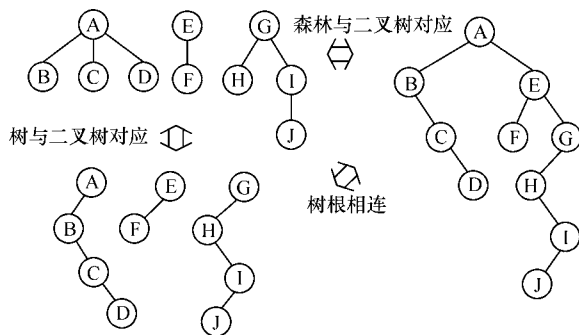


图 5.24 森林与二叉树的对应关系示例

这个一一对应的关系说明森林或树与二叉树可以相互转换。

### 1. 森林转换成二叉树

如果  $F = \{T_1, T_2, \dots, T_m\}$  是森林，则可按如下规则转换成一棵二叉树  $B = (root, LB, RB)$ 。

(1) 若  $F$  为空，即  $m = 0$ ，则  $B$  为空树；

(2) 若  $F$  非空，即  $m \neq 0$ ，则  $B$  的根  $root$  即为森林中第一棵树的根  $ROOT(T_1)$ ； $B$  的左子树  $LB$  是从  $T_1$  中根结点的子树森林  $F_1 = \{T_{11}, T_{12}, \dots, T_{1m}\}$  转换而成的二叉树；其右子树  $RB$  是从森林  $F' = \{T_2, T_3, \dots, T_m\}$  转换而成的二叉树。

### 2. 二叉树转换成森林

如果  $B = (root, LB, RB)$  是一棵二叉树，则可按如下规则转换成森林  $F = \{T_1, T_2, \dots, T_m\}$ ：

(1) 若  $B$  为空，则  $F$  为空；

(2) 若  $B$  非空，则  $F$  中第一棵树  $T_1$  的根  $ROOT(T_1)$  即为二叉树  $B$  的根  $root$ ； $T_1$  中根结点的子树森林  $F_1$  是由  $B$  的左子树  $LB$  转换而成的森林； $F$  中除  $T_1$  之外其余树组成的森林  $F' = \{T_2, T_3, \dots, T_m\}$  是由  $B$  的右子树  $RB$  转换而成的森林。

从上述递归定义容易写出相互转换的递归算法。同时，森林和树的操作亦可转换成二叉树的操作来实现。

## 5.6.3 树和森林的遍历

### 1. 树的遍历

由树结构的定义可引出两种次序遍历树的方法：一种是先根（次序）遍历树，即：先访问树的根结点，然后依次先根遍历根的每棵子树；另一种是后根（次序）遍历，即先依次后根遍历每棵子树，然后访问根结点。

例如，对图 5.19 所示的树进行先根遍历，可得树的先根序列为：

R A D E B C F G H K

若对此树进行后根遍历，则得树的后根序列为：

D E A B G H K F C R

按照森林和树相互递归的定义，可以推出森林的两种遍历方法：先序遍历和中序遍历。

### 2. 森林的遍历

(1) 先序遍历森林

若森林非空，则可按下述规则遍历：

① 访问森林中第一棵树的根结点；

- ② 先序遍历第一棵树的根结点的子树森林;
- ③ 先序遍历除去第一棵树之后剩余的树构成的森林。

#### (2) 中序遍历森林

若森林非空, 则可按下述规则遍历:

- ① 中序遍历森林中第一棵树的根结点的子树森林;
- ② 访问第一棵树的根结点;
- ③ 中序遍历除去第一棵树之后剩余的树构成的森林。

若对图 5.24 中所示的森林进行先序遍历和中序遍历, 则分别得到森林的先序序列为:

A B C D E F G H I J

中序序列为:

B C D A F E H J I G

由 5.6.2 小节森林与二叉树之间转换的规则可知, 当森林转换成二叉树时, 其第一棵树的子树森林转换成左子树, 剩余树的森林转换成右子树, 则上述森林的先序和中序遍历即为其对应的二叉树的先序和中序遍历。若对图 5.24 中所示的和森林对应的二叉树分别进行先序和中序遍历, 可得和上述相同的序列。

由此可见, 当以二叉链表做树的存储结构时, 树的先根遍历和后根遍历可借用二叉树的先序遍历和中序遍历的算法实现。

## 5.7 哈夫曼树及其应用

树结构是一种应用非常广泛的结构, 在一些特定的应用中, 树具有一些特殊特点, 利用这些特点可以解决很多工程问题。在 5.2 节提出的应用案例 5.1 可以借助一种应用很广的树——哈夫曼树来解决, 本节便以哈夫曼树为例, 说明二叉树的一个具体应用。

### 5.7.1 哈夫曼树的基本概念

**哈夫曼 (Huffman) 树** 又称**最优树**, 是一类带权路径长度最短的树, 在实际中有广泛的用途。哈夫曼树的定义, 涉及路径、路径长度、权等概念, 下面先给出这些概念的定义, 然后再介绍哈夫曼树。

(1) **路径**: 从树中一个结点到另一个结点之间的分支构成这两个结点之间的路径。

(2) **路径长度**: 路径上的分支数目称作路径长度。

(3) **树的路径长度**: 从树根到每一结点的路径长度之和。

(4) **权**: 赋予某个实体的一个量, 是对实体的某个或某些属性的数值化描述。在数据结构中, 实体有结点 (元素) 和边 (关系) 两大类, 所以对应有结点权和边权。结点权或边权具体代表什么意义, 由具体情况决定。如果在一棵树中的结点上带有权值, 则对应的就有带权树等概念。

(5) **结点的带权路径长度**: 从该结点到树根之间的路径长度与结点上权的乘积。

(6) **树的带权路径长度**: 树中所有叶子结点的带权路径长度之和, 通常记作  $WPL = \sum_{k=1}^n w_k l_k$ 。

(7) **哈夫曼树**: 假设有  $m$  个权值  $\{w_1, w_2, \dots, w_m\}$ , 可以构造一棵含  $n$  个叶子结点的二叉树, 每个叶子结点的权为  $w_i$ , 则其中带权路径长度  $WPL$  最小的二叉树称做**最优二叉树**或**哈夫曼树**。

例如,图 5.25 中所示的 3 棵二叉树,都含 4 个叶子结点 a、b、c、d,分别带权 7、5、2、4,它们的带权路径长度分别为

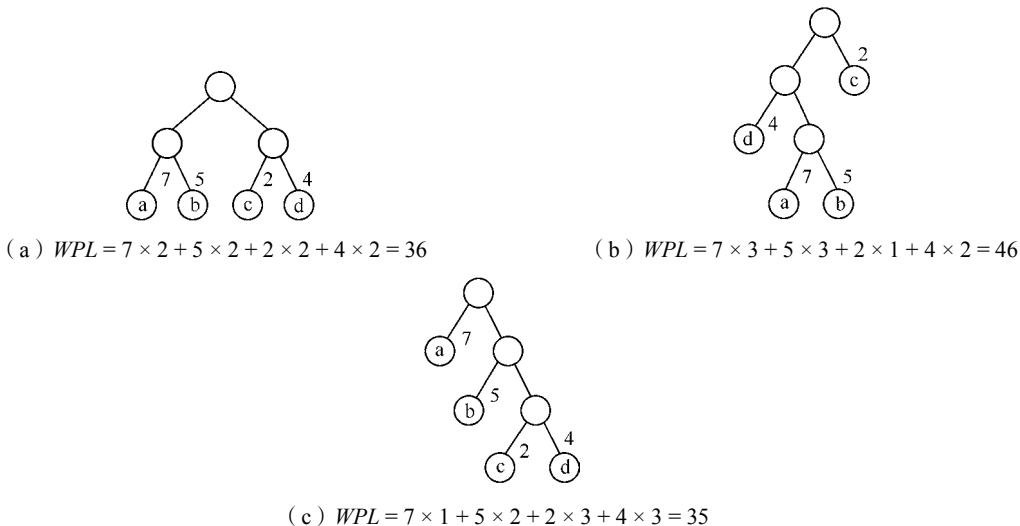


图 5.25 具有不同带权路径长度的二叉树

其中以 (c) 树的为最小。可以验证,它恰为哈夫曼树,即其带权路径长度在所有带权为 7、5、2、4 的 4 个叶子结点的二叉树中居最小。

哈夫曼树中具有不同权值的叶子结点的分布有什么特点呢?从上面的例子中,可以直观地发现,在哈夫曼树中,权值越大的结点离根结点越近。根据这个特点,哈夫曼最早给出了一个构造哈夫曼树的方法,称哈夫曼算法。

## 5.7.2 哈夫曼树的构造算法

### 1. 哈夫曼树的构造过程

(1) 根据给定的  $n$  个权值  $\{w_1, w_2, \dots, w_n\}$ , 构造  $n$  棵只有根结点的二叉树, 这  $n$  棵二叉树构成一个森林  $F$ 。

(2) 在森林  $F$  中选取两棵根结点的权值最小的树作为左右子树构造一棵新的二叉树, 且置新的二叉树的根结点的权值为其左、右子树上根结点的权值之和。

(3) 在森林  $F$  中删除这两棵树, 同时将新得到的二叉树加入  $F$  中。

(4) 重复 (2) 和 (3), 直到  $F$  只含一棵树为止。这棵树便是哈夫曼树。

在构造哈夫曼树时, 首先选择权小的, 这样保证权大的离根较近, 这样一来, 在计算树的带权路径长度时, 自然会得到最小带权路径长度, 这种生成算法是一种典型的贪心法。

例如, 图 5.26 所示为图 5.25 (c) 所示的哈夫曼树的构造过程。其中, 根结点上标注的数字是所赋的权。

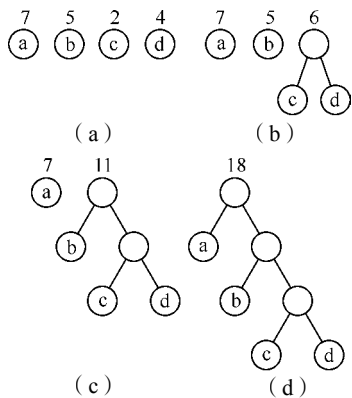


图 5.26 哈夫曼树的构造过程

## 2. 哈夫曼算法的实现

哈夫曼树是一种二叉树,当然可以采用前面介绍过的通用存储方法,而由于哈夫曼树中没有度为 1 的结点,则一棵有  $n$  个叶子结点的哈夫曼树共有  $2n-1$  个结点,可以存储在一个大小为  $2n-1$  的一维数组中。树中每个结点还要包含其双亲信息和孩子结点的信息,由此,每个结点的存储结构设计如图 5.27 所示。

| weight | parent | lchild | rchild |
|--------|--------|--------|--------|
|--------|--------|--------|--------|

图 5.27 哈夫曼树结点的形式

```
// - - - -哈夫曼树的存储表示 - - - -
typedef struct{
    int weight;                //结点的权值
    int parent,lchild,rchild;  //结点的双亲、左孩子、右孩子的下标
}HTNode,*HuffmanTree;        //动态分配数组存储哈夫曼树
```

哈夫曼树的各结点存储在由 HuffmanTree 定义的动态分配的数组中,为了实现方便,数组的 0 号单元不使用,从 1 号单元开始使用,所以数组的大小为  $2n$ 。将叶子结点集中存储在前面部分  $1\sim n$  个位置,而后面的  $n-1$  个位置存储其余非叶子结点。

### 算法 5.10 构造哈夫曼树

#### 【算法步骤】

构造哈夫曼树算法的实现可以分成两大部分。

① 初始化:首先动态申请  $2n$  个单元;然后循环  $2n-1$  次,从 1 号单元开始,依次将 1 至  $2n-1$  所有单元中的双亲、左孩子、右孩子的下标都初始化为 0;最后再循环  $n$  次,输入前  $n$  个单元中叶子结点的权值。

② 创建树:循环  $n-1$  次,通过  $n-1$  次的选择、删除与合并来创建哈夫曼树。选择是从当前森林中选择双亲为 0 且权值最小的两个树根结点  $s_1$  和  $s_2$ ;删除是指将结点  $s_1$  和  $s_2$  的双亲改为非 0;合并就是将  $s_1$  和  $s_2$  的权值和作为一个新结点的权值依次存入到数组的第  $n+1$  之后的单元中,同时记录这个新结点左孩子的下标为  $s_1$ ,右孩子的下标为  $s_2$ 。

#### 【算法描述】

```
void CreateHuffmanTree(HuffmanTree &HT,int n)
{
    //构造哈夫曼树 HT
    if(n<=1) return;
    m=2*n-1;
    HT=new HTNode[m+1];    //0 号单元未用,所以需要动态分配 m+1 个单元,HT[m] 表示根结点
    for(i=1;i<=m;++i)      //将 1~m 号单元中的双亲、左孩子、右孩子的下标都初始化为 0
        {HT[i].parent=0;HT[i].lchild=0;HT[i].rchild=0;}
    for(i=1;i<=n;++i)      //输入前 n 个单元中叶子结点的权值
        cin>>HT[i].weight;
    /* - - - - - 初始化工作结束,下面开始创建哈夫曼树 - - - - - */
    for(i=n+1;i<=m;++i)
        //通过 n-1 次的选择、删除、合并来创建哈夫曼树
        Select(HT,i-1, s1, s2);
    //在 HT[k] (1≤k≤i-1) 中选择两个其双亲域为 0 且权值最小的结点,并返回它们在 HT 中的序号 s1 和 s2
}
```

```

HT[s1].parent=i;HT[s2].parent=i;
//得到新结点 i, 从森林中删除 s1, s2, 将 s1 和 s2 的双亲域由 0 改为 i
HT[i].lchild=s1;HT[i].rchild=s2;           //s1,s2 分别作为 i 的左右孩子
HT[i].weight=HT[s1].weight+HT[s2].weight; //i 的权值为左右孩子权值之和
}                                           //for
}

```

【例 5.2】 已知  $w = (5, 29, 7, 8, 14, 23, 3, 11)$ , 利用算法 5.10 试构造一棵哈夫曼树, 计算树的带权路径长度, 并给出其构造过程中存储结构 HT 的初始状态和终结状态。

$n = 8$ , 则  $m = 15$ , 按算法 5.10 可构造一棵哈夫曼树, 如图 5.28 所示。

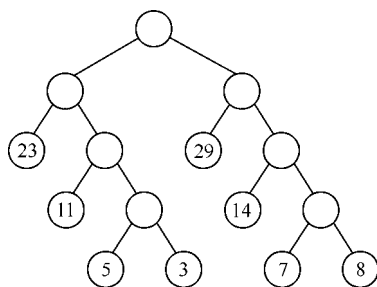


图 5.28 例 5.2 的哈夫曼树

树的带权路径长度计算如下:

$$WPL = \sum_{k=1}^n w_k l_k = 23 \times 2 + 11 \times 3 + 5 \times 4 + 3 \times 4 + 29 \times 2 + 14 \times 3 + 7 \times 4 + 8 \times 4 = 271$$

其存储结构 HT 的初始状态如表 5.2 (a) 所示, 其终结状态如表 5.2 (b) 所示。

表 5.2

例 5.2 的存储结构

(a) HT 的初态

| 结点 i | weight | parent | lchild | rchild |
|------|--------|--------|--------|--------|
| 1    | 5      | 0      | 0      | 0      |
| 2    | 29     | 0      | 0      | 0      |
| 3    | 7      | 0      | 0      | 0      |
| 4    | 8      | 0      | 0      | 0      |
| 5    | 14     | 0      | 0      | 0      |
| 6    | 23     | 0      | 0      | 0      |
| 7    | 3      | 0      | 0      | 0      |
| 8    | 11     | 0      | 0      | 0      |
| 9    | —      | 0      | 0      | 0      |
| 10   | —      | 0      | 0      | 0      |
| 11   | —      | 0      | 0      | 0      |
| 12   | —      | 0      | 0      | 0      |
| 13   | —      | 0      | 0      | 0      |
| 14   | —      | 0      | 0      | 0      |
| 15   | —      | 0      | 0      | 0      |

(b) HT 的终态

| 结点 i | weight | parent | lchild | rchild |
|------|--------|--------|--------|--------|
| 1    | 5      | 9      | 0      | 0      |
| 2    | 29     | 14     | 0      | 0      |
| 3    | 7      | 10     | 0      | 0      |
| 4    | 8      | 10     | 0      | 0      |
| 5    | 14     | 12     | 0      | 0      |
| 6    | 23     | 13     | 0      | 0      |
| 7    | 3      | 9      | 0      | 0      |
| 8    | 11     | 11     | 0      | 0      |
| 9    | 8      | 11     | 1      | 7      |
| 10   | 15     | 12     | 3      | 4      |
| 11   | 19     | 13     | 8      | 9      |
| 12   | 29     | 14     | 5      | 10     |
| 13   | 42     | 15     | 6      | 11     |
| 14   | 58     | 15     | 2      | 12     |
| 15   | 100    | 0      | 13     | 14     |

哈夫曼树在通信、编码和数据压缩等技术领域有着广泛的应用,下面讨论一个构造通信码的典型应用——哈夫曼编码。

### 5.7.3 哈夫曼编码

#### 1. 哈夫曼编码的主要思想

在 5.2 节提出的案例 5.1 中已经讨论,在进行数据压缩时,为了使压缩后的数据文件尽可能短,可采用不定长编码。其基本思想是:为出现次数较多的字符编以较短的编码。为确保对数据文件进行有效的压缩和对压缩文件进行正确的解码,可以利用哈夫曼树来设计二进制编码。哈夫曼树的具体构造过程可以根据算法 5.10,图 5.26 所示为图 5.4 所示的哈夫曼树的构造过程。在图 5.4 所示的哈夫曼树中,约定左分支标记为 0,右分支标记为 1,则根结点到每个叶子结点路径上的 0、1 序列即为相应字符的编码。

下面给出有关编码的两个概念。

(1) 前缀编码:如果在一个编码方案中,任一个编码都不是其他任何编码的前缀(最左子串),则称编码是前缀编码。例如,案例 5.1 中的第 2 种编码方案(见表 5.1(b))的编码 0, 10, 110, 111 是前缀编码,而第 3 种编码方案(见表 5.1(c))的编码 0, 01, 010, 111 就不是前缀编码。前缀编码可以保证对压缩文件进行解码时不产生二义性,确保正确解码。

(2) 哈夫曼编码:对一棵具有  $n$  个叶子的哈夫曼树,若对树中的每个左分支赋予 0,右分支赋予 1,则从根到每个叶子的路径上,各分支的赋值分别构成一个二进制串,该二进制串就称为哈夫曼编码。

哈夫曼编码满足下面的两个性质。

**性质 1** 哈夫曼编码是前缀编码。

**证明:** 哈夫曼编码是根到叶子路径上的编码序列,由树的特点知,若路径 A 是另一条路径 B 的最左部分,则 B 经过了 A,则 A 的终点一定不是叶子。而哈夫曼编码对应路径的终点一定为叶子,因此,任一哈夫曼码都不会与任意其他哈夫曼编码的前缀部分完全重叠,因此哈夫曼编码是前缀编码。

**性质 2** 哈夫曼编码是最优前缀编码。

对于包括  $n$  个字符的数据文件,分别以它们的出现次数为权值构造哈夫曼树,则利用该树对应的哈夫曼编码对文件进行编码,能使该文件压缩后对应的二进制文件的长度最短。

**证明:** 假设每种字符在数据文件中出现的次数为  $w_i$ ,其编码长度为  $l_i$ ,文件中只有  $n$  种字符,则文件总长为  $\sum_{i=1}^n w_i l_i$ 。对应到二叉树上,若置  $w_i$  为叶子结点的权,  $l_i$  恰为从根到叶子的路径长度,

则  $\sum_{i=1}^n w_i l_i$  恰为二叉树上带权路径长度。由此可见,设计文件总长最短的二进制前缀编码问题,就是以  $n$  种字符出现的频率作权设计一棵哈夫曼树的问题。而由哈夫曼树的构造方法可知,出现次数较多的字符对应的编码较短,这便直观地说明了该定理是成立的。

下面给出根据哈夫曼树构造哈夫曼编码的算法。

#### 2. 哈夫曼编码的算法实现

在构造哈夫曼树之后,求哈夫曼编码的主要思想是:依次以叶子为出发点,向上回溯至根结点为止。回溯时走左分支则生成代码 0,走右分支则生成代码 1。

由于每个哈夫曼编码是变长编码,因此使用一个指针数组来存放每个字符编码串的首地址。

```
//- - - -哈夫曼编码表的存储表示- - - -
typedef char **HuffmanCode; // 动态分配数组存储哈夫曼编码表
```

各字符的哈夫曼编码存储在由 HuffmanCode 定义的动态分配的数组 HC 中,为了实现方便,数组的 0 号单元不使用,从 1 号单元开始使用,所以数组 HC 的大小为  $n+1$ ,即编码表 HC 包括  $n+1$  行。但因为每个字符编码的长度事先不能确定,所以不能预先为每个字符分配大小合适的存储空间。为不浪费存储空间,动态分配一个长度为  $n$  (字符编码长度一定小于  $n$ ) 的一维数组 cd,用来临时存放当前正在求解的第  $i$  ( $1 \leq i \leq n$ ) 个字符的编码,当第  $i$  个字符的编码求解完毕后,根据数组 cd 的字符串长度分配 HC[i]的空间,然后将数组 cd 中的编码复制到 HC[i]中。

因为求解编码时是从哈夫曼树的叶子出发,向上回溯至根结点。所以对于每个字符,得到的编码顺序是从右向左的,故将编码向数组 cd 存放的顺序也是从后向前的,即每个字符的第 1 个编码存放在 cd[n-2]中 (cd[n-1]存放字符串结束标志 '\0'),第 2 个编码存放在 cd[n-3]中,依此类推,直到全部编码存放完毕。

**算法 5.11** 根据哈夫曼树求哈夫曼编码

【算法步骤】

① 分配存储  $n$  个字符编码的编码表空间 HC,长度为  $n+1$ ;分配临时存储每个字符编码的动态数组空间 cd,cd[n-1]置为 '\0'。

② 逐个求解  $n$  个字符的编码,循环  $n$  次,执行以下操作:

- 设置变量 start 用于记录编码在 cd 中存放的位置, start 初始时指向最后,即编码结束符位置  $n-1$ ;
- 设置变量 c 用于记录从叶子结点向上回溯至根结点所经过的结点下标, c 初始时为当前待编码字符的下标 i, f 用于记录 i 的双亲结点的下标;
- 从叶子结点向上回溯至根结点,求得字符 i 的编码,当 f 没有到达根结点时,循环执行以下操作:
  - 回溯一次 start 向前指一个位置,即  $--start$ ;
  - 若结点 c 是 f 的左孩子,则生成代码 0,否则生成代码 1,生成的代码 0 或 1 保存在 cd[start]中;
  - 继续向上回溯,改变 c 和 f 的值。
- 根据数组 cd 的字符串长度为第 i 个字符编码分配空间 HC[i],然后将数组 cd 中的编码复制到 HC[i]中。

③ 释放临时空间 cd。

【算法描述】

```
void CreatHuffmanCode(HuffmanTree HT,HuffmanCode &HC,int n)
{ //从叶子到根逆向求每个字符的哈夫曼编码,存储在编码表 HC 中
    HC=new char*[n+1]; //分配存储 n 个字符编码的编码表空间
    cd=new char[n]; //分配临时存放每个字符编码的动态数组空间
    cd[n-1]='\0'; //编码结束符
    for(i=1;i<=n;++i) //逐个字符求哈夫曼编码
```

```

{
    start=n-1;                                //start 开始时指向最后, 即编码结束符位置
    c=i;f=HT[i].parent;                       //f 指向结点 c 的双亲结点
    while(f!=0)                                //从叶子结点开始向上回溯, 直到根结点
    {
        --start;                               //回溯一次 start 向前指一个位置
        if(HT[f].lchild==c) cd[start]='0';    //结点 c 是 f 的左孩子, 则生成代码 0
        else cd[start]='1';                   //结点 c 是 f 的右孩子, 则生成代码 1
        c=f;f=HT[f].parent;                   //继续向上回溯
    }                                           //求出第 i 个字符的编码
    HC[i]=new char[n-start];                   //为第 i 个字符编码分配空间
    strcpy(HC[i],&cd[start]);                  //将求得的编码从临时空间 cd 复制到 HC 的当前行中
}                                              //for
delete cd;                                    //释放临时空间
}

```

**【例 5.3】** 已知某系统在通信联络中只可能出现 8 种字符, 其概率分别为 0.05, 0.29, 0.07, 0.08, 0.14, 0.23, 0.03, 0.11, 试设计哈夫曼编码。

根据其出现的概率可设 8 个字符的权值为:  $w=(5,29,7, 8,14,23,3,11)$ , 其对应的哈夫曼树如图 5.28 所示。将树的左分支标记为 0, 右分支标记为 1, 便得到其哈夫曼编码表如图 5.29 所示。

| HC |         |
|----|---------|
| 1  | 0 1 1 0 |
| 2  | 1 0     |
| 3  | 1 1 1 0 |
| 4  | 1 1 1 1 |
| 5  | 1 1 0   |
| 6  | 0 0     |
| 7  | 0 1 1 1 |
| 8  | 0 1 0   |

图 5.29 前缀编码表

### 3. 文件的编码和译码

#### (1) 编码

有了字符集的哈夫曼编码表之后, 对数据文件的编码过程是: 依次读入文件中的字符  $c$ , 在哈夫曼编码表 HC 中找到此字符, 将字符  $c$  转换为编码表中存放的编码串。

#### (2) 译码

对编码后的文件进行译码的过程必须借助于哈夫曼树。具体过程是: 依次读入文件的二进制码, 从哈夫曼树的根结点 (即  $HT[m]$ ) 出发, 若当前读入 0, 则走向左孩子, 否则走向右孩子。一旦到达某一叶子  $HT[i]$  时便译出相应的字符编码  $HC[i]$ 。然后重新从根出发继续译码, 直至文件结束。

具体编码和译码的算法留给读者去完成。

## 5.8 案例分析与实现

在 5.2 节引入的案例 5.1 已在 5.7 节中进行了详细的讨论, 案例 5.2 提出了可以利用二叉树来表示表达式, 本节将对案例 5.2 作进一步的分析, 给出利用表达式构建表达式树和利用表达式树求解表达式的算法。

**案例 5.2: 利用二叉树求解表达式的值。**

### 【案例分析】

对于任意一个算术表达式, 都可用二叉树来表示。表达式对应的二叉树创建后, 利用二叉树的遍历等操作, 很容易实现表达式的求值运算。因此问题的关键就是如何创建表达式树, 下面讨论由中缀表达式创建表达式树的方法。

假设运算符均为双目运算符, 则表达式对应的表达式树中叶子结点均为操作数, 分支结点均为运算符。由于创建的表达式树需要准确的表达运算次序, 因此在扫描表达式创建表达式树的过程中, 当遇到运算符时不能直接创建结点, 而应将其与前面的运算符进行优先级比较, 根据比较的结果再进行处理。这种处理方式类似于第 3 章的表达式求值算法中的运算符的比较, 可以借助一个运算符栈, 来暂存已经扫描到的还未处理的运算符。

根据表达式树与表达式对应关系的递归定义, 每两个操作数和一个运算符就可以建立一棵表达式二叉树, 而该二叉树又可以作为另一个运算符结点的一棵子树。可以另外借助一个表达式树栈, 来暂存已建立好的表达式树的根结点, 以便其作为另一个运算符结点的子树而被引用。

### 【案例实现】

为实现表达式树的创建算法, 可以使用两个工作栈, 一个称做 OPTR, 用以暂存运算符; 另一个称做 EXPT, 用以暂存已建立好的表达式树的根结点。

为了便于实现, 和第 3 章一样, 假设每个表达式均以 “#” 开始, 以 “#” 结束。

### 算法 5.12 表达式树的创建

#### 【算法步骤】

- ① 初始化 OPTR 栈和 EXPT 栈, 将表达式起始符 “#” 压入 OPTR 栈。
- ② 扫描表达式, 读入第一个字符 ch, 如果表达式没有扫描完毕至 “#” 或 OPTR 的栈顶元素不为 “#” 时, 则循环执行以下操作:
  - 若 ch 不是运算符, 则以 ch 为根创建一棵只有根结点的二叉树, 且将该树根结点压入 EXPT 栈, 读入下一字符 ch;
  - 若 ch 是运算符, 则根据 OPTR 的栈顶元素和 ch 的优先级比较结果, 做不同的处理:
    - 若是小于, 则 ch 压入 OPTR 栈, 读入下一字符 ch;
    - 若是大于, 则弹出 OPTR 栈顶的运算符, 从 EXPT 栈弹出两个表达式子树的根结点, 以该运算符为根结点, 以 EXPT 栈中弹出的第二个子树作为左子树, 以 EXPT 栈中弹出的第一个子树作为右子树, 创建一棵新二叉树, 并将该树根结点压入 EXPT 栈;
    - 若是等于, 则 OPTR 的栈顶元素是 “(” 且 ch 是 “)”, 这时弹出 OPTR 栈顶的 “(”, 相当于括号匹配成功, 然后读入下一字符 ch。

## 【算法描述】

```

void InitExpTree()
{//表达式树的创建算法
    InitStack(EXPT);           //初始化 EXPT 栈
    InitStack(OPTR);          //初始化 OPTR 栈
    Push(OPTR, '#');           //将表达式起始符“#”压入 OPTR 栈
    cin>>ch;
    while(ch!='#' || GetTop(OPTR)!='#') //表达式没有扫描完毕或 OPTR 的栈顶元素不为“#”
    {
        if(!In(ch))           //ch 不是运算符
        {
            CreateExpTree(T, NULL, NULL, ch); //以 ch 为根创建一棵只有根结点的二叉树
            Push(EXPT, T);       //将二叉树根结点 T 进 EXPT 栈
            cin>>ch;             //读入下一字符
        }
        else
        {
            switch(Precede(GetTop(OPTR), ch)) //比较 OPTR 的栈顶元素和 ch 的优先级
            {
                case '<':
                    Push(OPTR, ch); cin>>ch; //当前字符 ch 压入 OPTR 栈, 读入下一字符
                    break;
                case '>':
                    Pop(OPTR, theta); //弹出 OPTR 栈顶的运算符
                    Pop(EXPT, b); Pop(EXPT, a); //弹出 EXPT 栈顶的两个运算数
                    CreateExpTree(T, a, b, theta)
                    //以 theta 为根, a 为左子树, b 为右子树, 创建一棵二叉树
                    Push(EXPT, T); //将二叉树根结点 T 进 EXPT 栈
                    break;
                case '=':
                    //OPTR 的栈顶元素是“(”且 ch 是“)”
                    Pop(OPTR, x); cin>>ch; //弹出 OPTR 栈顶的“(”, 读入下一字符 ch
                    break;
            }
        }
    }
}
//switch
//while

```

## 【算法分析】

此算法从头到尾扫描表达式中每个字符, 若表达式的字符串长度为  $n$ , 则此算法的时间复杂度为  $O(n)$ 。算法在运行时所占用的辅助空间主要取决于 OPTR 栈和 EXPT 栈的大小, 显然, 它们的空间大小之和不会超过  $n$ , 所以此算法的空间复杂度也同为  $O(n)$ 。

## 算法 5.13 表达式树的求值

## 【算法步骤】

① 设变量 lvalue 和 rvalue 分别用以记录表达式树中左子树和右子树的值, 初始均为 0。  
 ② 如果当前结点为叶子 (结点为操作数), 则返回该结点的数值, 否则 (结点为运算符) 执行以下操作:

- 递归计算左子树的值记为 lvalue;
- 递归计算右子树的值记为 rvalue;
- 根据当前结点运算符的类型, 将 lvalue 和 rvalue 进行相应运算并返回。

## 【算法描述】

```

int EvaluateExpTree(BiTree T)
{ //遍历表达式树进行表达式求值
    lvalue=rvalue=0; //初始为 0
    if(T->lchild==NULL && T->rchild==NULL)
        return T->data-'0'; //如果结点为操作数,则返回该结点的数值
    else //如果结点为运算符
    {
        lvalue=EvaluateExpTree(T->lchild); //递归计算左子树的值记为 lvalue
        rvalue=EvaluateExpTree(T->rchild); //递归计算右子树的值记为 rvalue
        return GetValue(T->data,lvalue,rvalue); //根据当前结点运算符的类型进行相应运算
    }
}

```

## 【算法分析】

遍历表达式进行求值的过程实际上是一个后序遍历二叉树的过程,因此时间和空间复杂度均为  $O(n)$ 。

## 5.9 小 结

树和二叉树是一类具有层次关系的非线性数据结构,本章主要内容如下。

(1) 二叉树是一种最常用的树形结构,二叉树具有一些特殊的性质,而满二叉树和完全二叉树又是两种特殊形态的二叉树。

(2) 二叉树有两种存储表示:顺序存储和链式存储。顺序存储就是把二叉树的所有结点按照层次顺序存储到连续的存储单元中,这种存储更适用于完全二叉树。链式存储又称二叉链表,每个结点包括两个指针,分别指向其左孩子和右孩子。链式存储是二叉树常用的存储结构。

(3) 树的存储结构有三种:双亲表示法、孩子表示法和孩子兄弟表示法,孩子兄弟表示法是常用的表示法,任意一棵树都能通过孩子兄弟表示法转换为二叉树进行存储。森林与二叉树之间也存在相应的转换方法,通过这些转换,可以利用二叉树的操作解决一般树的有关问题。

(4) 二叉树的遍历算法是其他运算的基础,通过遍历得到了二叉树中结点访问的线性序列,实现了非线性结构的线性化。根据访问结点的次序不同可得三种遍历:先序遍历、中序遍历、后序遍历,时间复杂度均为  $O(n)$ 。

(5) 在线索二叉树中,利用二叉链表中的  $n+1$  个空指针域来存放指向某种遍历次序下的前驱结点和后继结点的指针,这些附加的指针就称为“线索”。引入二叉线索树的目的是加快查找结点前驱或后继的速度。

(6) 哈夫曼树在通信编码技术上有广泛的应用,只要构造了哈夫曼树,按分支情况在左路径上写代码 0,右路径上写代码 1,然后从上到下叶结点相应路径上的代码序列就是该叶结点的最优前缀码,即哈夫曼编码。

学习完本章后,要求掌握二叉树的性质和存储结构,熟练掌握二叉树的前、中、后序遍历算法,掌握线索化二叉树的基本概念和构造方法。熟练掌握哈夫曼树和哈夫曼编码的构造方法。能够利用树的孩子兄弟表示法将一般的树结构转换为二叉树进行存储。掌握森林与二叉树之间的相互转换方法。



C.101

D.102

(12) 若  $X$  是二叉中序线索树中一个有左孩子的结点, 且  $X$  不为根, 则  $X$  的前驱为 ( )。

A.  $X$  的双亲B.  $X$  的右子树中最左的结点C.  $X$  的左子树中最右结点D.  $X$  的左子树中最右叶结点

(13) 引入二叉线索树的目的是 ( )。

A. 加快查找结点的前驱或后继的速度

B. 为了能在二叉树中方便地进行插入与删除

C. 为了能方便地找到双亲

D. 使二叉树的遍历结果唯一

(14) 设  $F$  是一个森林,  $B$  是由  $F$  变换得的二叉树。若  $F$  中有  $n$  个非终端结点, 则  $B$  中右指针域为空的结点有 ( ) 个。

A.  $n-1$ B.  $n$ C.  $n+1$ D.  $n+2$ 

(15)  $n$  ( $n \geq 2$ ) 个权值均不相同的字符构成哈夫曼树, 关于该树的叙述中, 错误的是 ( )。

A. 该树一定是一棵完全二叉树

B. 树中一定没有度为 1 的结点

C. 树中两个权值最小的结点一定是兄弟结点

D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值

## 2. 应用题

(1) 试找出满足下列条件的二叉树。

① 先序序列与后序序列相同。

② 中序序列与后序序列相同。

③ 先序序列与中序序列相同。

④ 中序序列与层次遍历序列相同。

(2) 设一棵二叉树的先序序列:  $ABDFCEGH$ , 中序序列:  $BFDAGEHC$ 。

① 画出这棵二叉树。

② 画出这棵二叉树的后序线索树。

③ 将这棵二叉树转换成对应的树 (或森林)。

(3) 假设用于通信的电文仅由 8 个字母组成, 字母在电文中出现的频率分别为 0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10。

① 试为这 8 个字母设计哈夫曼编码。

② 试设计另一种由二进制表示的等长编码方案。

③ 对于上述实例, 比较两种方案的优缺点。

(4) 已知下列字符  $A$ 、 $B$ 、 $C$ 、 $D$ 、 $E$ 、 $F$ 、 $G$  的权值分别为 3、12、7、4、2、8、11, 试填写出其对应哈夫曼树  $HT$  存储结构的初态和终态。

## 3. 算法设计题

以二叉链表作为二叉树的存储结构, 编写以下算法:

(1) 统计二叉树的叶结点个数。

(2) 判别两棵树是否相等。

(3) 交换二叉树每个结点的左孩子和右孩子。

- (4) 设计二叉树的双序遍历算法 (双序遍历是指对于二叉树的每一个结点来说, 先访问这个结点, 再按双序遍历它的左子树, 然后再一次访问这个结点, 接下来按双序遍历它的右子树)。
- (5) 计算二叉树最大的宽度 (二叉树的最大宽度是指二叉树所有层中结点个数的最大值)。
- (6) 用按层次顺序遍历二叉树的方法, 统计树中度为 1 的结点数目。
- (7) 求任意二叉树中第一条最长的路径长度, 并输出此路径上各结点的值。
- (8) 输出二叉树中从每个叶子结点到根结点的路径。

# 第6章

## 图

图是一种比线性表和树更为复杂的数据结构。在线性表中，数据元素之间仅有线性关系，每个数据元素只有一个直接前驱和一个直接后继；在树形结构中，数据元素之间有着明显的层次关系，并且每一层中的数据元素可能和下一层中的多个元素（即其孩子结点）相关，但只能和上一层中一个元素（即其双亲结点）相关；而在图结构中，结点之间的关系可以是任意的，图中任意两个数据元素之间都可能相关。由此，图的应用极为广泛，已渗入到诸如物理、化学、电信工程、计算机科学，以及数学等其他分支中。在离散数学中，图论是专门研究图的性质的数学分支，而在数据结构中，则应用图论的知识讨论如何在计算机上实现图的操作，因此主要学习图的存储结构，以及若干图的操作的实现。

## 6.1 图的定义和基本术语

### 6.1.1 图的定义

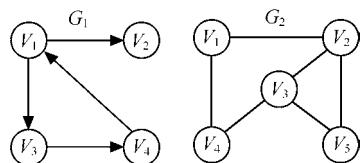
图（Graph） $G$ 由两个集合  $V$  和  $E$  组成，记为  $G=(V, E)$ ，其中  $V$  是顶点的有穷非空集合， $E$  是  $V$  中顶点偶对的有穷集合，这些顶点偶对称为边。 $V(G)$  和  $E(G)$  通常分别表示图  $G$  的顶点集和边集， $E(G)$  可以为空集。若  $E(G)$  为空，则图  $G$  只有顶点而没有边。

对于图  $G$ ，若边集  $E(G)$  为有向边的集合，则称该图为有向图；若边集  $E(G)$  为无向边的集合，则称该图为无向图。

在有向图中，顶点对  $\langle x, y \rangle$  是有序的，它称为从顶点  $x$  到顶点  $y$  的一条有向边。因此， $\langle x, y \rangle$  与  $\langle y, x \rangle$  是不同的两条边。顶点对用一对尖括号括起来， $x$  是有向边的始点， $y$  是有向边的终点。 $\langle x, y \rangle$  也称作一条弧，则  $x$  为弧尾， $y$  为弧头。

在无向图中，顶点对  $(x, y)$  是无序的，它称为与顶点  $x$  和顶点  $y$  相关联的一条边。这条边没有特定的方向， $(x, y)$  与  $(y, x)$  是同一条边。为了有别于有向图，无向图的顶点对用一对圆括号括起来。

图 6.1 分别给出了有向图和无向图的示例。



(a) 有向图  $G_1$       (b) 无向图  $G_2$

图 6.1 图的示例

### 6.1.2 图的基本术语

用  $n$  表示图中顶点数目，用  $e$  表示边的数目，下面介绍图结构中的一些基本术语。

(1) **子图**: 假设有两个图  $G = (V, E)$  和  $G' = (V', E')$ , 如果  $V' \subseteq V$  且  $E' \subseteq E$ , 则称  $G'$  为  $G$  的子图。例如, 图 6.2 所示为图 6.1 中  $G_1$  和  $G_2$  子图的一些例子。

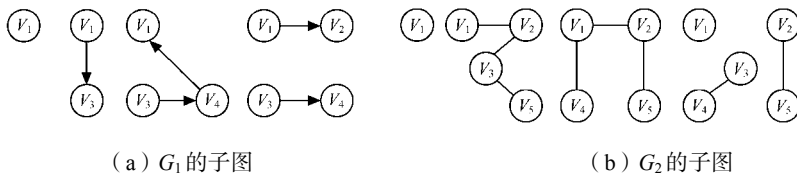


图 6.2 子图示例

(2) **无向完全图和有向完全图**: 对于无向图, 若具有  $n(n-1)/2$  条边, 则称为无向完全图。对于有向图, 若具有  $n(n-1)$  条弧, 则称为有向完全图。

(3) **稀疏图和稠密图**: 有很少条边或弧 (如  $e < n \log_2 n$ ) 的图称为稀疏图, 反之称为稠密图。

(4) **权和网**: 在实际应用中, 每条边可以标上具有某种含义的数值, 该数值称为该边上的权。这些权可以表示从一个顶点到另一个顶点的距离或耗费。这种带权的图通常称为网。

(5) **邻接点**: 对于无向图  $G$ , 如果图的边  $(v, v') \in E$ , 则称顶点  $v$  和  $v'$  互为邻接点, 即  $v$  和  $v'$  相邻接。边  $(v, v')$  依附于顶点  $v$  和  $v'$ , 或者说边  $(v, v')$  与顶点  $v$  和  $v'$  相关联。

(6) **度、入度和出度**: 顶点  $v$  的度是指和  $v$  相关联的边的数目, 记为  $TD(v)$ 。例如, 图 6.1 (b) 中  $G_2$  的顶点  $v_3$  的度是 3。对于有向图, 顶点  $v$  的度分为入度和出度。**入度**是以顶点  $v$  为头的弧的数目, 记为  $ID(v)$ ; **出度**是以顶点  $v$  为尾的弧的数目, 记为  $OD(v)$ 。顶点  $v$  的度为  $TD(v) = ID(v) + OD(v)$ 。例如, 图 6.1 中  $G_1$  的顶点  $v_1$  的入度  $ID(v_1) = 1$ , 出度  $OD(v_1) = 2$ , 度  $TD(v_1) = ID(v_1) + OD(v_1) = 3$ 。一般地, 如果顶点  $v_i$  的度记为  $TD(v_i)$ , 那么一个有  $n$  个顶点,  $e$  条边的图, 满足如下关系

$$e = \frac{1}{2} \sum_{i=1}^n TD(v_i)$$

(7) **路径和路径长度**: 在无向图  $G$  中, 从顶点  $v$  到顶点  $v'$  的路径是一个顶点序列  $(v = v_{i,0}, v_{i,1}, \dots, v_{i,m} = v')$ , 其中  $(v_{i,j-1}, v_{i,j}) \in E, 1 \leq j \leq m$ 。如果  $G$  是有向图, 则路径也是有向的, 顶点序列应满足  $\langle v_{i,j-1}, v_{i,j} \rangle \in E, 1 \leq j \leq m$ 。**路径长度**是一条路径上经过的边或弧的数目。

(8) **回路或环**: 第一个顶点和最后一个顶点相同的路径称为回路或环。

(9) **简单路径、简单回路或简单环**: 序列中顶点不重复出现的路径称为简单路径。除了第一个顶点和最后一个顶点之外, 其余顶点不重复出现的回路, 称为简单回路或简单环。

(10) **连通、连通图和连通分量**: 在无向图  $G$  中, 如果从顶点  $v$  到顶点  $v'$  有路径, 则称  $v$  和  $v'$  是连通的。如果对于图中任意两个顶点  $v_i, v_j \in V$ ,  $v_i$  和  $v_j$  都是连通的, 则称  $G$  是连通图。图 6.1 (b) 中的  $G_2$  就是一个连通图, 而图 6.3 (a) 中的  $G_3$  则是非连通图, 但  $G_3$  有 3 个连通分量, 如图 6.3 (b) 所示。所谓连通分量, 指的是无向图中的极大连通子图。

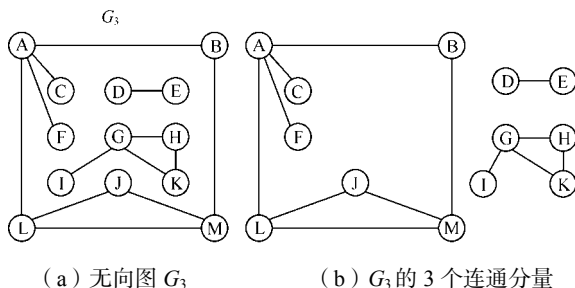


图 6.3 无向图及其连通分量

(11) **强连通图和强连通分量**: 在有向图  $G$  中, 如果对于每一对  $v_i, v_j \in V, v_i \neq v_j$ , 从  $v_i$  到  $v_j$  和从  $v_j$  到  $v_i$  都存在路径, 则称  $G$  是强连通图。有向图中的极大强连通子图称作有向图的强连通分量。例如图 6.1(a) 中的  $G_1$  不是强连通图, 但它有两个强连通分量, 如图 6.4 所示。

(12) **连通图的生成树**: 一个极小连通子图, 它含有图中全部顶点, 但只有足以构成一棵树的  $n-1$  条边, 这样的连通子图称为**连通图的生成树**。图 6.5 所示为  $G_3$  中最大连通分量的一棵生成树。如果在一棵生成树上添加一条边, 必定构成一个环, 因为这条边使得它依附的那两个顶点之间有了第二条路径。

一棵有  $n$  个顶点的生成树有且仅有  $n-1$  条边。如果一个图有  $n$  个顶点和小于  $n-1$  条边, 则是非连通图。如果它多于  $n-1$  条边, 则一定有环。但是, 有  $n-1$  条边的图不一定是生成树。

(13) **有向树和生成森林**: 有一个顶点的入度为 0, 其余顶点的入度均为 1 的有向图称为**有向树**。一个有向图的**生成森林**是由若干棵有向树组成, 含有图中全部顶点, 但只有足以构成若干棵不相交的有向树的弧。图 6.6 所示为其一例。

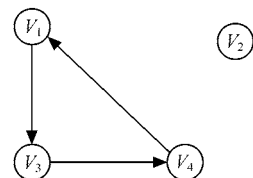


图 6.4  $G_1$  的两个强连通分量

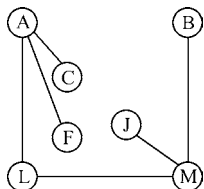


图 6.5  $G_3$  的最大连通分量的一棵生成树

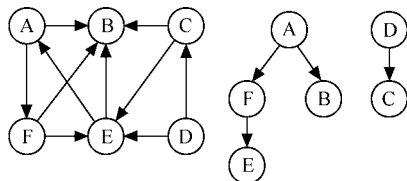


图 6.6 一个有向图及其生成森林

## 6.2 案例引入

SNS(Social Networking Services), 即社会性网络服务, 是指帮助人们建立社会性网络的互联网应用服务; 也指社会现有的已成熟普及的信息载体, 如短信 SMS 服务。SNS 的另一种常用解释是“社交网站”或“社交网”(Social Network Site)。SNS 的理论基础为“六度空间理论”。基于此理论, SNS 社区将用户关系梳理好后, 可以将海量的内容灌入 SNS 社区。例如, 喜欢电影的朋友, 把内容传递给爱好相同的朋友; 备战考研的同学, 把研究生入学考试的相关信息和自己的复习经验传递给身边其他考研的同学。这种传递的形式非常简单, 用户产生交互, 内容即通过渠道传递到真实的关系网中, 杂乱无章的内容与人群通过 SNS 社区变得有序, 如图 6.7 所示。

### 案例 6.1: 六度空间理论。

六度空间理论是一个数学领域的猜想, 又称为六度分割理论(Six Degrees of Separation)。六度空间理论是 20 世纪 60 年代由美国的心理学家米格兰姆(Stanley Milgram)提出的, 理论指出: 你和任何一个陌生人之间所间隔的人不会超过 6 个, 也就是说, 最多通过 6 个中间人你就能够认识任何一个陌生人, 如图 6.8 所示。

随着新技术的发展, 六度空间理论的应用价值受到了人们的广泛关注, 除了前面提到的微软的人立方搜索, 很多领域都运用了六度空间理论, 例如 SNS 网站、BLOG 网站、电子游戏社区、直销网络等。

六度空间理论的出现使得人们对于自身的人际关系网络的威力有了新的认识。但为什么偏偏是“六度”，而不是“七度”、“八度”或者“千百度”呢？这可能要从人际关系网络的另外一个特征——“150 定律”来寻找解释。“150 定律”指出，人类智力允许人类拥有稳定社交网络的人数是 148 人，四舍五入大约是 150 人。这样我们可以对六度空间理论做如下数学解释（并非数学证明）：若每个人平均认识 150 人，其六度便是  $150^6=11\ 390\ 625\ 000\ 000$ ，消除一些重复的结点，也远远超过了整个地球人口的若干多倍。



图 6.7 SNS 社区有序化示意图



图 6.8 六度空间理论示意图

那么，如何从理论上验证六度空间理论呢？六度空间理论的数学模型属于图结构，我们把六度空间理论中的人际关系网络图抽象成一个无向图  $G$ ，用图  $G$  中的一个顶点表示一个人，两个人“认识”与否，用代表这两个人的顶点之间是否有一条边来表示。然后利用本章所学的图的有关算法即可从理论上进行验证，本章 6.7 节将给出此案例的分析与实现。

## 6.3 图的类型定义

图是一种数据结构，加上一组基本操作，就构成了抽象数据类型。抽象数据类型图的定义如下：

ADT Graph{

**数据对象：** $V$  是具有相同特性的数据元素的集合，称为顶点集。

**数据关系：**

$R = \{VR\}$

$VR = \{ \langle v, w \rangle | v, w \in V \text{ 且 } P(v, w) \}$  表示从  $v$  到  $w$  的弧，

谓词  $P(v, w)$  定义了弧  $\langle v, w \rangle$  的意义或信息 }

**基本操作：**

CreateGraph (&G, V, VR)

初始条件： $V$  是图的顶点集， $VR$  是图中弧的集合。

操作结果：按  $V$  和  $VR$  的定义构造图  $G$ 。

DestroyGraph (&G)

初始条件：图  $G$  存在。

操作结果：销毁图  $G$ 。

LocateVex (G, u)

初始条件：图  $G$  存在， $u$  和  $G$  中顶点有相同特征。

操作结果：若  $G$  中存在顶点  $u$ ，则返回该顶点在图中的位置；否则返回其他信息。

GetVex (G, v)

初始条件：图  $G$  存在， $v$  是  $G$  中某个顶点。

操作结果：返回  $v$  的值。

```
PutVex(&G, v, value);
```

初始条件: 图  $G$  存在,  $v$  是  $G$  中某个顶点。

操作结果: 对  $v$  赋值  $value$ 。

```
FirstAdjVex(G, v)
```

初始条件: 图  $G$  存在,  $v$  是  $G$  中某个顶点。

操作结果: 返回  $v$  的第一个邻接顶点。若  $v$  在  $G$  中没有邻接顶点, 则返回“空”。

```
NextAdjVex(G, v, w)
```

初始条件: 图  $G$  存在,  $v$  是  $G$  中某个顶点,  $w$  是  $v$  的邻接顶点。

操作结果: 返回  $v$  的 (相对于  $w$  的) 下一个邻接顶点。若  $w$  是  $v$  的最后一个邻接点, 则返回“空”。

```
InsertVex(&G, v)
```

初始条件: 图  $G$  存在,  $v$  和图中顶点有相同特征。

操作结果: 在图  $G$  中增添新顶点  $v$ 。

```
DeleteVex(&G, v)
```

初始条件: 图  $G$  存在,  $v$  是  $G$  中某个顶点。

操作结果: 删除  $G$  中顶点  $v$  及其相关的弧。

```
InsertArc(&G, v, w)
```

初始条件: 图  $G$  存在,  $v$  和  $w$  是  $G$  中两个顶点。

操作结果: 在  $G$  中增添弧  $\langle v, w \rangle$ , 若  $G$  是无向图, 则还增添对称弧  $\langle w, v \rangle$ 。

```
DeleteArc(&G, v, w)
```

初始条件: 图  $G$  存在,  $v$  和  $w$  是  $G$  中两个顶点。

操作结果: 在  $G$  中删除弧  $\langle v, w \rangle$ , 若  $G$  是无向图, 则还删除对称弧  $\langle w, v \rangle$ 。

```
DFS_Traverse(G)
```

初始条件: 图  $G$  存在。

操作结果: 对图进行深度优先遍历, 在遍历过程中对每个顶点访问一次。

```
BFS_Traverse(G)
```

初始条件: 图  $G$  存在。

操作结果: 对图进行广度优先遍历, 在遍历过程中对每个顶点访问一次。

}ADT Graph

## 6.4 图的存储结构

由于图的结构比较复杂, 任意两个顶点之间都可能存在联系, 因此无法以数据元素在存储区中的物理位置来表示元素之间的关系, 即图没有顺序存储结构, 但可以借助二维数组来表示元素之间的关系, 即邻接矩阵表示法。另一方面, 由于图的任意两个顶点间都可能存在关系, 因此, 用链式存储表示图是很自然的事, 图的链式存储有多种, 有邻接表、十字链表和邻接多重表, 应根据实际需要的不同选择不同的存储结构。

### 6.4.1 邻接矩阵

#### 1. 邻接矩阵表示法

邻接矩阵 (Adjacency Matrix) 是表示顶点之间相邻关系的矩阵。设  $G(V, E)$  是具有  $n$  个顶点的图, 则  $G$  的邻接矩阵是具有如下性质的  $n$  阶方阵

$$A[i][j] = \begin{cases} 1 & \text{若 } \langle v_i, v_j \rangle \in E \\ 0 & \text{反之} \end{cases}$$

例如, 图 6.1 中所示的  $G_1$  和  $G_2$  的邻接矩阵如图 6.9 所示。

$$G_{1, \text{arcs}} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}, G_{2, \text{arcs}} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

图 6.9 图的邻接矩阵

若  $G$  是网, 则邻接矩阵可以定义为

$$A[i][j] = \begin{cases} w_{i,j} & \text{若 } \langle v_i, v_j \rangle \in E \\ \infty & \text{反之} \end{cases}$$

其中,  $w_{i,j}$  表示边上的权值;  $\infty$  表示计算机允许的、大于所有边上权值的数。例如, 图 6.10 所示为一个有向网和它的邻接矩阵。

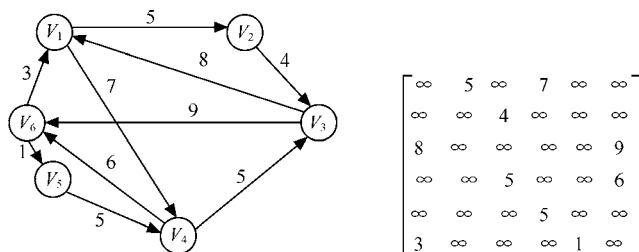


图 6.10 网及其邻接矩阵

用邻接矩阵表示法表示图, 除了一个用于存储邻接矩阵的二维数组外, 还需要用一个一维数组来存储顶点信息。其形式说明如下:

```
//-----图的邻接矩阵存储表示-----
#define MaxInt 32767 //表示极大值, 即∞
#define MVNum 100 //最大顶点数
typedef char VerTexType; //假设顶点的数据类型为字符型
typedef int ArcType; //假设边的权值类型为整型
typedef struct
{
    VerTexType vexs[MVNum]; //顶点表
    ArcType arcs[MVNum][MVNum]; //邻接矩阵
    int vexnum, arcnum; //图的当前点数和边数
}AMGraph;
```

## 2. 采用邻接矩阵表示法创建无向网

已知一个图的点和边, 使用邻接矩阵表示法来创建此图的方法比较简单, 下面以一个无向网为例来说明创建图的算法。

### 算法 6.1 采用邻接矩阵表示法创建无向网

#### 【算法步骤】

- ① 输入总顶点数和总边数。
- ② 依次输入点的信息存入顶点表中。
- ③ 初始化邻接矩阵, 使每个权值初始化为极大值。

④ 构造邻接矩阵。依次输入每条边依附的顶点和其权值，确定两个顶点在图中的位置之后，使相应边赋予相应的权值，同时使其对称边赋予相同的权值。

#### 【算法描述】

```
Status CreateUDN(AMGraph &G)
{ //采用邻接矩阵表示法，创建无向图 G
    cin>>G.vexnum>>G.arcnum;           //输入总顶点数，总边数
    for(i=0;i<G.vexnum;++i)             //依次输入点的信息
        cin>>G.vexs[i];
    for(i=0;i<G.vexnum;++i)             //初始化邻接矩阵，边的权值均置为极大值 MaxInt
        for(j=0;j<G.vexnum;++j)
            G.arcs[i][j]=MaxInt;
    for(k=0;k<G.arcnum;++k)             //构造邻接矩阵
    {
        cin>>v1>>v2>>w;                //输入一条边依附的顶点及权值
        i=LocateVex(G,v1);j=LocateVex(G,v2); //确定 v1 和 v2 在 G 中的位置，即顶点数组的下标
        G.arcs[i][j]=w;                  //边<v1, v2>的权值置为 w
        G.arcs[j][i]=G.arcs[i][j];       //置<v1, v2>的对称边<v2, v1>的权值为 w
    }
    return OK;
}
```

#### 【算法分析】

该算法的时间复杂度是  $O(n^2)$ 。

若要建立无向图，只需对上述算法做两处小的改动：一是初始化邻接矩阵时，将边的权值均初始化为 0；二是构造邻接矩阵时，将权值  $w$  改为常量值 1 即可。同样，将该算法稍做修改即可建立一个有向网或有向图。

### 3. 邻接矩阵表示法的优缺点

#### (1) 优点

- ① 便于判断两个顶点之间是否有边，即根据  $A[i][j] = 0$  或 1 来判断。
- ② 便于计算各个顶点的度。对于无向图，邻接矩阵第  $i$  行元素之和就是顶点  $i$  的度；对于有向图，第  $i$  行元素之和就是顶点  $i$  的出度，第  $i$  列元素之和就是顶点  $i$  的入度。

#### (2) 缺点

- ① 不便于增加和删除顶点。
- ② 不便于统计边的数目，需要扫描邻接矩阵所有元素才能统计完毕，时间复杂度为  $O(n^2)$ 。
- ③ 空间复杂度高。如果是有向图， $n$  个顶点需要  $n^2$  个单元存储边。如果是无向图，因其邻接矩阵是对称的，所以对规模较大的邻接矩阵可以采用压缩存储的方法，仅存储下三角（或上三角）的元素，这样需要  $n(n-1)/2$  个单元即可。但无论以何种方式存储，邻接矩阵表示法的空间复杂度均为  $O(n^2)$ ，这对于稀疏图而言尤其浪费空间。

下面介绍的邻接表将邻接矩阵的  $n$  行改成  $n$  个单链表，适合表示稀疏图。

## 6.4.2 邻接表

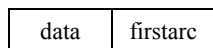
### 1. 邻接表表示法

**邻接表**（Adjacency List）是图的一种链式存储结构。在邻接表中，对图中每个顶点  $v_i$  建立一个单链表，把与  $v_i$  相邻接的顶点放在这个链表中。邻接表中每个单链表的第一个结点存放有关顶

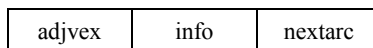
点的信息, 把这一结点看成链表的表头, 其余结点存放有关边的信息, 这样邻接表便由两部分组成: 表头结点表和边表。

(1) **表头结点表**: 由所有表头结点以顺序结构的形式存储, 以便可以随机访问任一顶点的边链表。表头结点包括**数据域** (data) 和**链域** (firstarc) 两部分, 如图 6.11 (a) 所示。其中, 数据域用于存储顶点  $v_i$  的名称或其他有关信息; 链域用于指向链表中第一个结点 (即与顶点  $v_i$  邻接的第一个邻接点)。

(2) **边表**: 由表示图中顶点间关系的  $2n$  个边链表组成。边链表中边结点包括**邻接点域** (adjvex)、**数据域** (info) 和**链域** (nextarc) 三部分, 如图 6.11 (b) 所示。其中, 邻接点域指示与顶点  $v_i$  邻接的点在图中的位置; 数据域存储和边相关的信息, 如权值等; 链域指示与顶点  $v_i$  邻接的下一条边的结点。



(a) 表头结点

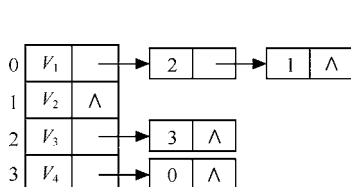


(b) 边结点

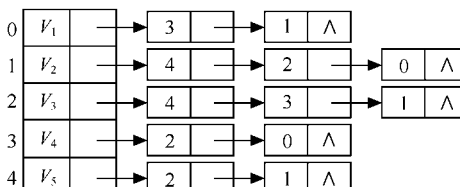
图 6.11 表头结点和边结点

例如, 图 6.12 (a) 和 (b) 所示分别为图 6.1 中  $G_1$  和  $G_2$  的邻接表。

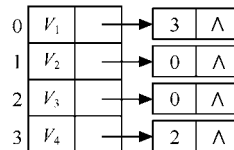
在无向图的邻接表中, 顶点  $v_i$  的度恰为第  $i$  个链表中的结点数; 而在有向图中, 第  $i$  个链表中的结点个数只是顶点  $v_i$  的出度, 为求入度, 必须遍历整个邻接表。在所有链表中, 其邻接点域的值是  $i$  的结点的个数是顶点  $v_i$  的入度。有时, 为了便于确定顶点的入度, 可以建立一个有向图的逆邻接表, 即对每个顶点  $v_i$  建立一个链接所有进入  $v_i$  的边的表, 例如, 图 6.12 (c) 所示为有向图  $G_1$  的逆邻接表。



(a)  $G_1$  的邻接表



(b)  $G_2$  的邻接表



(c)  $G_1$  的逆邻接表

图 6.12 邻接表和逆邻接表

根据上述讨论, 要定义一个邻接表, 需要先定义其存放顶点的头结点和表示边的边结点。图的邻接表存储结构说明如下:

```
// - - - -图的邻接表存储表示- - - -
#define MVNum 100 //最大顶点数
typedef struct ArcNode //边结点
{
    int adjvex; //该边所指向的顶点的位置
    struct ArcNode * nextarc; //指向下一条边的指针
    OtherInfo info; //和边相关的信息
}ArcNode;
typedef struct VNode //顶点信息
{
    VerTexType data;
```

```

    ArcNode *firstarc;           //指向第一条依附该顶点的边的指针
}VNode, AdjList[MVNum];       //AdjList 表示邻接表类型
typedef struct                //邻接表
{
    AdjList vertices;
    int vexnum, arcnum;        //图的当前顶点数和边数
}ALGraph;

```

## 2. 采用邻接表表示法创建无向图

基于上述的邻接表表示法, 要创建一个图则需要创建其相应的顶点表和边表。下面以一个无向图为例来说明采用邻接表表示法创建无向图的算法。

### 算法 6.2 采用邻接表表示法创建无向图

#### 【算法步骤】

- ① 输入总顶点数和总边数。
- ② 依次输入点的信息存入顶点表中, 使每个表头结点的指针域初始化为 NULL。
- ③ 创建邻接表。依次输入每条边依附的两个顶点, 确定这两个顶点的序号  $i$  和  $j$  之后, 将此边结点分别插入  $v_i$  和  $v_j$  对应的两个边链表的头部。

#### 【算法描述】

```

Status CreateUDG(ALGraph &G)
{ //采用邻接表表示法, 创建无向图 G
    cin>>G.vexnum>>G.arcnum;           //输入总顶点数, 总边数
    for(i=0; i<G.vexnum; ++i)           //输入各点, 构造表头结点表
    {
        cin>> G.vertices[i].data;       //输入顶点值
        G.vertices[i].firstarc=NULL;     //初始化表头结点的指针域为 NULL
    }                                     //for
    for(k=0; k<G.arcnum; ++k)           //输入各边, 构造邻接表
    {
        cin>>v1>>v2;                     //输入一条边依附的两个顶点
        i=LocateVex(G, v1); j=LocateVex(G, v2);
        //确定 v1 和 v2 在 G 中位置, 即顶点在 G.vertices 中的序号
        p1=new ArcNode;                  //生成一个新的边结点*p1
        p1->adjvex=j;                     //邻接点序号为 j
        p1->nextarc=G.vertices[i].firstarc; G.vertices[i].firstarc=p1;
        //将新结点*p1 插入顶点 vi 的边表头部
        p2=new ArcNode;                  //生成另一个对称的新的边结点*p2
        p2->adjvex=i;                     //邻接点序号为 i
        p2->nextarc=G.vertices[j].firstarc; G.vertices[j].firstarc=p2;
        //将新结点*p2 插入顶点 vj 的边表头部
    }                                     //for
    return OK;
}

```

#### 【算法分析】

该算法的时间复杂度是  $O(n + e)$ 。

建立有向图的邻接表与此类似, 只是更加简单, 每读入一个顶点对序号  $\langle i, j \rangle$ , 仅需生成一个邻接点序号为  $j$  的边表结点, 并将其插入到  $v_i$  的边链表头部即可。若要创建网的邻接表, 可以

将边的权值存储在 info 域中。



值得注意的是, 一个图的邻接矩阵表示是唯一的, 但其邻接表表示不唯一, 这是因为邻接表表示中, 各边表结点的链接次序取决于建立邻接表的算法, 以及边的输入次序。

邻接矩阵和邻接表是图的两种最常用的存储结构, 它们各有所长。与邻接矩阵相比, 邻接表有其自己的优缺点。

### 3. 邻接表表示法的优缺点

#### (1) 优点

- ① 便于增加和删除顶点。
- ② 便于统计边的数目, 按顶点表顺序扫描所有边表可得到边的数目, 时间复杂度为  $O(n + e)$ 。

③ 空间效率高。对于一个具有  $n$  个顶点  $e$  条边的图  $G$ , 若  $G$  是无向图, 则在其邻接表表示中有  $n$  个顶点表结点和  $2e$  个边表结点; 若  $G$  是有向图, 则在它的邻接表表示或逆邻接表表示中均有  $n$  个顶点表结点和  $e$  个边表结点。因此, 邻接表或逆邻接表表示的空间复杂度为  $O(n + e)$ , 适合表示稀疏图。对于稠密图, 考虑到邻接表中要附加链域, 因此常采取邻接矩阵表示法。

#### (2) 缺点

① 不便于判断顶点之间是否有边, 要判定  $v_i$  和  $v_j$  之间是否有边, 就需扫描第  $i$  个边表, 最坏情况下要耗费  $O(n)$  时间。

② 不便于计算有向图各个顶点的度。对于无向图, 在邻接表表示中顶点  $v_i$  的度是第  $i$  个边表中的结点个数。在有向图的邻接表中, 第  $i$  个边表上的结点个数是顶点  $v_i$  的出度, 但求  $v_i$  的入度较困难, 需遍历各顶点的边表。若有向图采用逆邻接表表示, 则与邻接表表示相反, 求顶点的入度容易, 而求顶点的出度较难。

下面介绍的十字链表便于求得顶点的入度和出度。

## 6.4.3 十字链表

**十字链表** (Orthogonal List) 是有向图的另一种链式存储结构。可以看成是将有向图的邻接表和逆邻接表结合起来得到的一种链表。在十字链表中, 对应于有向图中每一条弧有一个结点, 对应于每个顶点也有一个结点。这些结点的结构如图 6.13 所示。

|         |         |       |       |      |
|---------|---------|-------|-------|------|
| tailvex | headvex | hlink | tlink | info |
|---------|---------|-------|-------|------|

(a) 弧结点

|      |         |          |
|------|---------|----------|
| data | firstin | firstout |
|------|---------|----------|

(b) 顶点结点

图 6.13 弧结点和顶点结点

在弧结点中有 5 个域: 其中尾域 (tailvex) 和头域 (headvex) 分别指示弧尾和弧头这两个顶点在图中的位置, 链域 hlink 指向弧头相同的下一条弧, 而链域 tlink 指向弧尾相同的下一条弧, info 域指向该弧的相关信息。弧头相同的弧在同一链表上, 弧尾相同的弧也在同一链表上。它们的头结点即为顶点结点, 它由 3 个域组成: 其中 data 域存储和顶点相关的信息, 如顶点的名称等; firstin 和 firstout 为两个链域, 分别指向以该顶点为弧头或弧尾的第一个弧结点。例如, 图 6.14 (a) 中所示图的十字链表如图 6.14 (b) 所示。若将有向图的邻接矩阵看成是稀疏矩阵的话, 则十字链表也可以看成是邻接矩阵的链表存储结构, 在图的十字链表中, 弧结点所在的链表非循环链表, 结点之间相对位置自然形成, 不一定按顶点序号有序, 表头结点即顶点结点, 它们之间不是链接, 而是顺序存储。

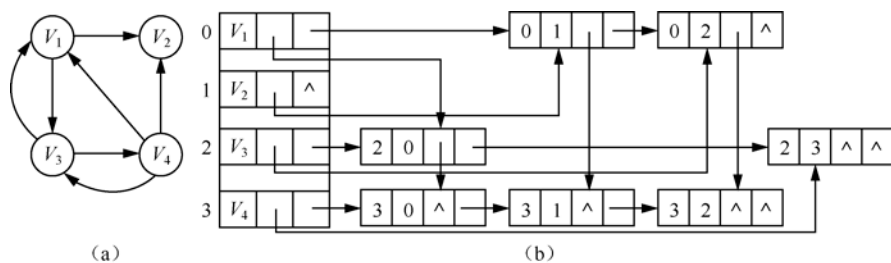


图 6.14 有向图的十字链表

有向图的十字链表存储表示的形式说明如下所示：

// - - - - 有向图的十字链表存储表示 - - - -

```
#define MAX_VERTEX_NUM 20
typedef struct ArcBox
{
    int tailvext, headvex;           //该弧的尾和头顶点的位置
    struct ArcBox *hlink, *tlink;    //分别为弧头相同和弧尾相同的弧的链域
    InfoType *info;                 //该弧相关信息的指针
}ArcBox;
typedef struct VexNode
{
    VertexType data;
    ArcBox *firstin, *firstout;      //分别指向该顶点第一条入弧和出弧
}VexNode;
typedef struct
{
    VexNode xlist[MAX_VERTEX_NUM]; //表头向量
    int vexnm, arcnum;              //有向图的当前顶点数和弧数
}OLGraph;
```

只要输入  $n$  个顶点的信息和  $e$  条弧的信息，便可建立该有向图的十字链表，读者可以模仿算法 6.2 写出采用十字链表表示法创建有向图的算法。建立十字链表的时间复杂度和建立邻接表是相同的。在十字链表中既容易找到以  $v_i$  为尾的弧，也容易找到以  $v_i$  为头的弧，因而容易求得顶点的出度和入度（或需要，可在建立十字链表的同时求出）。在某些有向图的应用中，十字链表是很有用的工具。

### 6.4.4 邻接多重表

**邻接多重表**（Adjacency Multilist）是无向图的另一种链式存储结构。虽然邻接表是无向图的一种很有效的存储结构，在邻接表中容易求得顶点和边的各种信息。但是，在邻接表中每一条边  $(v_i, v_j)$  有两个结点，分别第  $i$  个和第  $j$  个链表中，这给某些图的操作带来不便。例如在某些图的应用问题中需要对边进行某种操作，如对已被搜索过的边作记号或删除一条边等，此时需要找到表示同一条边的两个结点。因此，在进行这一类操作的无向图的问题中采用邻接多重表作存储结构更为适宜。

邻接多重表的结构和十字链表类似。在邻接多重表中，每一条边用一个结点表示，它由如图 6.15(a) 所示的 6 个域组成。其中，mark 为标志域，可用以标记该条边是否被搜索过；ivex 和 jvex 为该边依附的两个顶点在图中的位置；ilink 指向下一条依附于顶点 ivex 的边；jlink 指向下一条依

附于顶点  $jvex$  的边,  $info$  为指向和边相关的各种信息的指针域。

每一个顶点也用一个结点表示, 它由如图 6.15 (b) 所示的两个域组成。其中,  $data$  域存储和该顶点相关的信息,  $firstedge$  域指示第一条依附于该顶点的边。例如, 图 6.16 所示为无向图  $G_2$  的邻接多重表。在邻接多重表中, 所有依附于同一顶点的边串联在同一链表中, 由于每条边依附于两个顶点, 则每个边结点同时链接在两个链表中。可见, 对无向图而言, 其邻接多重表和邻接表的差别, 仅仅在于同一条边在邻接表中用两个结点表示, 而在邻接多重表中只有一个结点。因此, 除了在边结点中增加一个标志域外, 邻接多重表所需的存储量和邻接表相同。

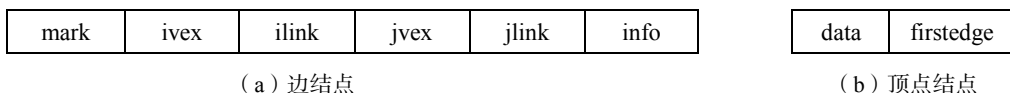


图 6.15 边结点和顶点结点

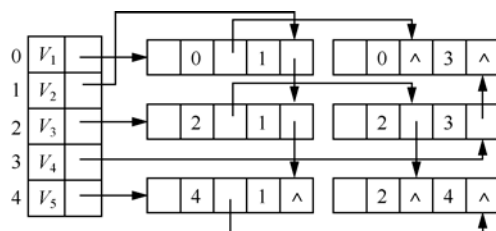


图 6.16 无向图  $G_2$  的邻接多重表

在邻接多重表上, 各种基本操作的实现亦和邻接表相似。邻接多重表的类型说明如下:

```
// - - - - 无向图的邻接多重表存储表示 - - - - -
#define MAX_VERTEX_NUM 20
typedef enum{unvisited,visited} VisitIf;
typedef struct EBox
{
    VisitIf mark;                //访问标记
    int ivex, jvex;              //该边依附的两个顶点的位置
    struct EBox *ilink, *jlink;  //分别指向依附这两个顶点的下一条边
    InfoType *info;              //该边信息指针
}EBox;
typedef struct VexBox
{
    VertexType data;
    EBox *firstedge;              //指向第一条依附该顶点的边
}VexBox;
typedef struct{
    VexBox adjmulist[MAX_VERTEX_NUM];
    int vexnum, edgenum;          //无向图的当前顶点数和边数
}AMLGraph;
```

## 6.5 图的遍历

和树的遍历类似, 图的遍历也是从图中某一顶点出发, 按照某种方法对图中所有顶点访问且

仅访问一次。图的遍历算法是求解图的连通性问题、拓扑排序和关键路径等算法的基础。

然而,图的遍历要比树的遍历复杂得多。因为图的任一顶点都可能和其余的顶点相邻接。所以在访问了某个顶点之后,可能沿着某条路径搜索之后,又回到该顶点上。例如,图 6.1 (b) 中所示的  $G_2$ , 由于图中存在回路,因此在访问了  $v_1$ 、 $v_2$ 、 $v_3$ 、 $v_4$  之后,沿着边  $\langle v_4, v_1 \rangle$  又可访问到  $v_1$ 。为了避免同一顶点被访问多次,在遍历图的过程中,必须记下每个已访问过的顶点。为此,设一个辅助数组  $visited[n]$ , 其初始值置为“false”或者 0, 一旦访问了顶点  $v_i$ , 便置  $visited[i]$  为“true”或者 1。

根据搜索路径的方向,通常有两条遍历图的路径:深度优先搜索和广度优先搜索。它们对无向图和有向图都适用。

## 6.5.1 深度优先搜索

### 1. 深度优先搜索遍历的过程

深度优先搜索 (Depth First Search, DFS) 遍历类似于树的先序遍历,是树的先序遍历的推广。对于一个连通图,深度优先搜索遍历的过程如下。

(1) 从图中某个顶点  $v$  出发,访问  $v$ 。

(2) 找出刚访问过的顶点的第一个未被访问的邻接点,访问该顶点。以该顶点为新顶点,重复此步骤,直至刚访问过的顶点没有未被访问的邻接点为止。

(3) 返回前一个访问过的且仍有未被访问的邻接点的顶点,找出该顶点的下一个未被访问的邻接点,访问该顶点。

(4) 重复步骤 (2) 和 (3),直至图中所有顶点都被访问过,搜索结束。

以图 6.17 (a) 中所示的无向图  $G_4$  为例,深度优先搜索遍历图的过程如图 6.17 (b) 所示<sup>①</sup>。具体过程如下。

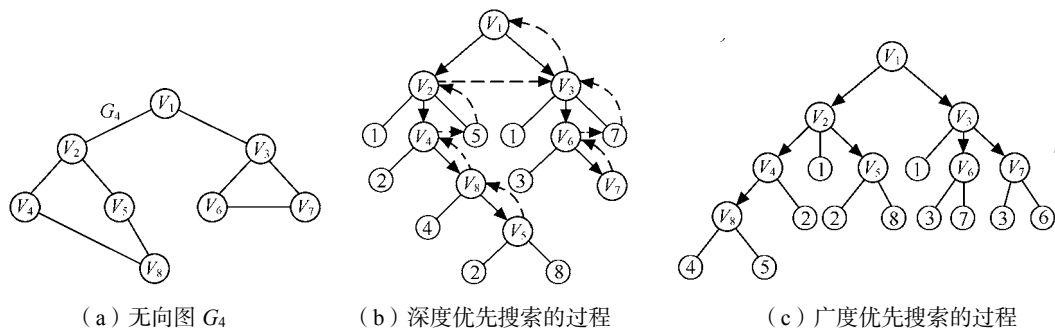


图 6.17 遍历图的过程

(1) 从顶点  $v_1$  出发,访问  $v_1$ 。

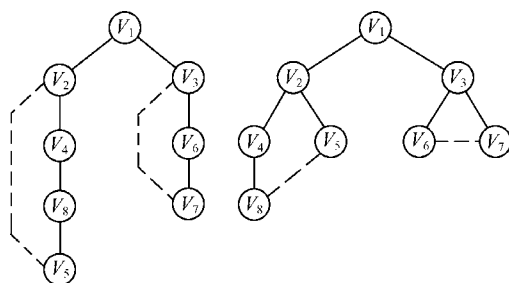
(2) 在访问了顶点  $v_1$  之后,选择第一个未被访问的邻接点  $v_2$ ,访问  $v_2$ 。以  $v_2$  为新顶点,重复此步,访问  $v_4$ 、 $v_8$ 、 $v_5$ 。在访问了  $v_5$  之后,由于  $v_5$  的邻接点都被访问,此步结束。

(3) 搜索从  $v_5$  回到  $v_8$ ,由于同样的理由,搜索继续回到  $v_4$ 、 $v_2$  直至  $v_1$ ,此时由于  $v_1$  的另一个邻接点未被访问,则搜索又从  $v_1$  到  $v_3$ ,再继续进行下去。由此,得到的顶点访问序列为:

① 图中以带箭头的粗实线表示遍历时的访问路径,以带箭头的虚线表示回溯的路径。小圆圈表示已被访问过的邻接点,大圆圈表示访问的邻接点。

$$V_1 \rightarrow V_2 \rightarrow V_4 \rightarrow V_8 \rightarrow V_5 \rightarrow V_3 \rightarrow V_6 \rightarrow V_7$$

图 6.17 (b) 中所示的所有顶点加上标有实箭头的边, 构成一棵以  $v_1$  为根的树, 称为深度优先生成树, 如图 6.18 (a) 所示。



(a)  $G_4$  的深度优先生成树 (b)  $G_4$  的广度优先生成树

图 6.18 生成树

## 2. 深度优先搜索遍历的算法实现

显然, 深度优先搜索遍历连通图是一个递归的过程。为了在遍历过程中便于区分顶点是否已被访问, 需附设访问标志数组  $visited[n]$ , 其初值为 “false”, 一旦某个顶点被访问, 则其相应的分量置为 “true”。

### 算法 6.3 深度优先搜索遍历连通图

#### 【算法步骤】

- ① 从图中某个顶点  $v$  出发, 访问  $v$ , 并置  $visited[v]$  的值为 true。
- ② 依次检查  $v$  的所有邻接点  $w$ , 如果  $visited[w]$  的值为 false, 再从  $w$  出发进行递归遍历, 直到图中所有顶点都被访问过。

#### 【算法描述】

```
bool visited[MVNum];           //访问标志数组, 其初值为 “false”
void DFS(Graph G, int v)
{ //从第 v 个顶点出发递归地深度优先遍历图 G
    cout << v; visited[v] = true;           //访问第 v 个顶点, 并置访问标志数组相应分量值为 true
    for (w = FirstAdjVex(G, v); w >= 0; w = NextAdjVex(G, v, w))
        //依次检查 v 的所有邻接点 w, FirstAdjVex(G, v) 表示 v 的第一个邻接点
        //NextAdjVex(G, v, w) 表示 v 相对于 w 的下一个邻接点, w >= 0 表示存在邻接点
        if (!visited[w]) DFS(G, w);         //对 v 的尚未访问的邻接点 w 递归调用 DFS
}
```

若是非连通图, 上述遍历过程执行之后, 图中一定还有顶点未被访问, 需要从图中另选一个未被访问的顶点作为起始点, 重复上述深度优先搜索过程, 直到图中所有顶点均被访问过为止。这样, 要实现对非连通图的遍历, 需要循环调用算法 6.3, 具体实现如算法 6.4 所示。

### 算法 6.4 深度优先搜索遍历非连通图

#### 【算法描述】

```
void DFSTraverse(Graph G)
{ //对非连通图 G 做深度优先遍历
    for (v = 0; v < G.vexnum; ++v) visited[v] = false; //访问标志数组初始化
    for (v = 0; v < G.vexnum; ++v)                     //循环调用算法 6.3
```

```

        if(!visited[v]) DFS(G,v);           //对尚未访问的顶点调用 DFS
    }

```

对于算法 6.4, 每调用一次算法 6.3 将遍历一个连通分量, 有多少次调用, 就说明图中有多少个连通分量。

在算法 6.3 中, 对于查找邻接点的操作 FirstAdjVex( $G, v$ )及 NextAdjVex( $G, v, w$ )并没有具体展开。如果图的存储结构不同, 这两个操作的实现方法不同, 时间耗费也不同。下面的算法 6.5、算法 6.6 分别用邻接矩阵和邻接表具体实现了算法 6.3 的功能。

#### 算法 6.5 采用邻接矩阵表示图的深度优先搜索遍历

##### 【算法描述】

```

void DFS_AM(AMGraph G,int v)
{//图 G 为邻接矩阵类型, 从第 v 个顶点出发深度优先搜索遍历图 G
    cout<<v;visited[v]=true;           //访问第 v 个顶点, 并置访问标志数组相应分量值为 true
    for(w=0;w<G.vexnum;w++)           //依次检查邻接矩阵 v 所在的行
        if((G.arcs[v][w]!=0)&&(!visited[w])) DFS(G,w);
    //G.arcs[v][w]!=0 表示 w 是 v 的邻接点, 如果 w 未访问, 则递归调用 DFS
}

```

#### 算法 6.6 采用邻接表表示图的深度优先搜索遍历

##### 【算法描述】

```

void DFS_AL (ALGraph G,int v)
{//图 G 为邻接表类型, 从第 v 个顶点出发深度优先搜索遍历图 G
    cout<<v;visited[v]=true;           //访问第 v 个顶点, 并置访问标志数组相应分量值为 true
    p=G.vertices[v].firstarc;           //p 指向 v 的边链表的第一个边结点
    while(p!=NULL)                       //边结点非空
    {
        w=p->adjvex;                     //表示 w 是 v 的邻接点
        if(!visited[w]) DFS(G,w);       //如果 w 未访问, 则递归调用 DFS
        p=p->nextarc;                   //p 指向下一个边结点
    }
    //while
}

```

### 3. 深度优先搜索遍历的算法分析

分析上述算法, 在遍历图时, 对图中每个顶点至多调用一次 DFS 函数, 因为一旦某个顶点被标志成已被访问, 就不再从它出发进行搜索。因此, 遍历图的过程实质上是对每个顶点查找其邻接点的过程, 其耗费的时间则取决于所采用的存储结构。当用邻接矩阵表示图时, 查找每个顶点的邻接点的时间复杂度为  $O(n^2)$ , 其中  $n$  为图中顶点数。而当以邻接表做图的存储结构时, 查找邻接点的时间复杂度为  $O(e)$ , 其中  $e$  为图中边数。由此, 当以邻接表做存储结构时, 深度优先搜索遍历图的时间复杂度为  $O(n + e)$ 。

## 6.5.2 广度优先搜索

### 1. 广度优先搜索遍历的过程

广度优先搜索 (Breadth First Search, BFS) 遍历类似于树的按层次遍历的过程。

广度优先搜索遍历的过程如下。

(1) 从图中某个顶点  $v$  出发, 访问  $v$ 。

(2) 依次访问  $v$  的各个未曾访问过的邻接点。

(3) 分别从这些邻接点出发依次访问它们的邻接点, 并使“先被访问的顶点的邻接点”先于“后被访问的顶点的邻接点”被访问。重复步骤 (3), 直至图中所有已被访问的顶点的邻接点都被访问到。

例如, 对图  $G_4$  进行广度优先搜索遍历的过程如图 6.17 (c) 所示, 具体过程如下。

(1) 从顶点  $v_1$  出发, 访问  $v_1$ 。

(2) 依次访问  $v_1$  的各个未曾访问过的邻接点  $v_2$  和  $v_3$ 。

(3) 依次访问  $v_2$  的邻接点  $v_4$  和  $v_5$ , 以及  $v_3$  的邻接点  $v_6$  和  $v_7$ , 最后访问  $v_4$  的邻接点  $v_8$ 。由于这些顶点的邻接点均已被访问, 并且图中所有顶点都被访问, 由此完成了图的遍历。得到的顶点访问序列为:

$$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5 \rightarrow v_6 \rightarrow v_7 \rightarrow v_8$$

图 6.17 (c) 中所示的所有顶点加上标有实箭头的边, 构成一棵以  $v_1$  为根的树, 称为**广度优先生成树**, 如图 6.18 (b) 所示。

## 2. 广度优先搜索遍历的算法实现

可以看出, 广度优先搜索遍历的特点是: 尽可能先对横向进行搜索。设  $x$  和  $y$  是两个相继被访问过的顶点, 若当前是以  $x$  为出发点进行搜索, 则在访问  $x$  的所有未曾被访问过的邻接点之后, 紧接着是以  $y$  为出发点进行横向搜索, 并对搜索到的  $y$  的邻接点中尚未被访问的顶点进行访问。也就是说, 先访问的顶点其邻接点亦先被访问。为此, 算法实现时需引进队列保存已被访问过的顶点。

和深度优先搜索类似, 广度优先搜索在遍历的过程中也需要一个访问标志数组。

### 算法 6.7 广度优先搜索遍历连通图

#### 【算法步骤】

- ① 从图中某个顶点  $v$  出发, 访问  $v$ , 并置  $\text{visited}[v]$  的值为  $\text{true}$ , 然后将  $v$  进队。
- ② 只要队列不空, 则重复下述操作:
  - 队头顶点  $u$  出队;
  - 依次检查  $u$  的所有邻接点  $w$ , 如果  $\text{visited}[w]$  的值为  $\text{false}$ , 则访问  $w$ , 并置  $\text{visited}[w]$  的值为  $\text{true}$ , 然后将  $w$  进队。

#### 【算法描述】

```
void BFS(Graph G, int v)
{ // 按广度优先非递归遍历连通图 G
    cout << v; visited[v] = true; // 访问第 v 个顶点, 并置访问标志数组相应分量值为 true
    InitQueue(Q); // 辅助队列 Q 初始化, 置空
    EnQueue(Q, v); // v 进队
    while (!QueueEmpty(Q)) // 队列非空
    {
        DeQueue(Q, u); // 队头元素出队并置为 u
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w))
            // 依次检查 u 的所有邻接点 w, FirstAdjVex(G, u) 表示 u 的第一个邻接点
            // NextAdjVex(G, u, w) 表示 u 相对于 w 的下一个邻接点, w >= 0 表示存在邻接点
            if (!visited[w]) // w 为 u 的尚未访问的邻接点
            {
                cout << w; visited[w] = true; // 访问 w, 并置访问标志数组相应分量值为 true
```

```

        EnQueue(Q, w);           //w 进队
    }                             //if
}                                 //while
}

```

若是非连通图，上述遍历过程执行之后，图中一定还有顶点未被访问，需要从图中另选一个未被访问的顶点作为起始点，重复上述广度优先搜索过程，直到图中所有顶点均被访问过为止。

对于非连通图的遍历，实现算法类似于算法 6.4，仅需将原算法中的 DFS 函数调用改为 BFS 函数调用。

读者可以参考算法 6.5 和算法 6.6，分别用邻接矩阵和邻接表具体实现算法 6.7 的功能。

### 3. 广度优先搜索遍历的算法分析

分析上述算法，每个顶点至多进一次队列。遍历图的过程实质上是通过边找邻接点的过程，因此广度优先搜索遍历图的时间复杂度和深度优先搜索遍历相同，即当用邻接矩阵存储时，时间复杂度为  $O(n^2)$ ；用邻接表存储时，时间复杂度为  $O(n + e)$ 。两种遍历方法的不同之处仅仅在于对顶点访问的顺序不同。

## 6.6 图的应用

现实生活中的许多问题都可以转化为图来解决。例如，如何以最小成本构建一个通信网络，如何计算地图中两地之间的最短路径，如何为复杂活动中各子任务的完成寻找一个较优的顺序等。本节将结合这些常用的实际问题，介绍图的几个常用算法，包括最小生成树，最短路径、拓扑排序和关键路径算法。

### 6.6.1 最小生成树

假设要在  $n$  个城市之间建立通信联络网，则连通  $n$  个城市只需要  $n - 1$  条线路。这时，自然会考虑这样一个问题，如何在最节省经费的前提下建立这个通信网。

在每两个城市之间都可设置一条线路，相应地都要付出一定的经济代价。 $n$  个城市之间，最多可能设置  $n(n - 1)/2$  条线路，那么，如何在这些可能的线路中选择  $n - 1$  条，以使总的耗费最少呢？

可以用连通网来表示  $n$  个城市，以及  $n$  个城市间可能设置的通信线路，其中网的顶点表示城市，边表示两城市之间的线路，赋予边的权值表示相应的代价。对于  $n$  个顶点的连通网可以建立许多不同的生成树，每一棵生成树都可以是一个通信网。最合理的通信网应该是代价之和最小的生成树。在一个连通网的所有生成树中，各边的代价之和最小的那棵生成树称为该连通网的最小代价生成树（Minimum Cost Spanning Tree），简称为最小生成树。

构造最小生成树有多种算法，其中多数算法利用了最小生成树的下列一种简称为 MST 的性质：假设  $N = (V, E)$  是一个连通网， $U$  是顶点集  $V$  的一个非空子集。若  $(u, v)$  是一条具有最小权值（代价）的边，其中  $u \in U$ ， $v \in V - U$ ，则必存在一棵包含边  $(u, v)$  的最小生成树。

可以用反证法来证明。假设网  $N$  的任何一棵最小生成树都不包含  $(u, v)$ 。设  $T$  是连通网上的一棵最小生成树，当将边  $(u, v)$  加入到  $T$  中时，由生成树的定义， $T$  中必存在一条包含  $(u, v)$  的回

路。另一方面, 由于  $T$  是生成树, 则在  $T$  上必存在另一条边  $(u', v')$ , 其中  $u' \in U$ ,  $v' \in V - U$ , 且  $u$  和  $u'$  之间、 $v$  和  $v'$  之间均有路径相通。删去边  $(u', v')$ , 便可消除上述回路, 同时得到另一棵生成树  $T'$ 。因为  $(u, v)$  的权值不高于  $(u', v')$ , 则  $T'$  的权值亦不高于  $T$ ,  $T'$  是包含  $(u, v)$  的一棵最小生成树。由此和假设矛盾。

普里姆 (Prim) 算法和克鲁斯卡尔 (Kruskal) 算法是两个利用 MST 性质构造最小生成树的算法。下面先介绍普里姆算法。

### 1. 普里姆算法

#### (1) 普里姆算法的构造过程

假设  $N = (V, E)$  是连通网,  $TE$  是  $N$  上最小生成树中边的集合。

①  $U = \{u_0\} (u_0 \in V)$ ,  $TE = \{\}$ 。

② 在所有  $u \in U$ ,  $v \in V - U$  的边  $(u, v) \in E$  中找一条权值最小的边  $(u_0, v_0)$  并入集合  $TE$ , 同时  $v_0$  并入  $U$ 。

③ 重复②, 直至  $U = V$  为止。

此时  $TE$  中必有  $n - 1$  条边, 则  $T = (V, TE)$  为  $N$  的最小生成树。

图 6.19 所示为一个连通网  $G_5$  从  $v_1$  开始构造最小生成树的例子。可以看出, 普里姆算法逐步增加  $U$  中的顶点, 可称为“加点法”。

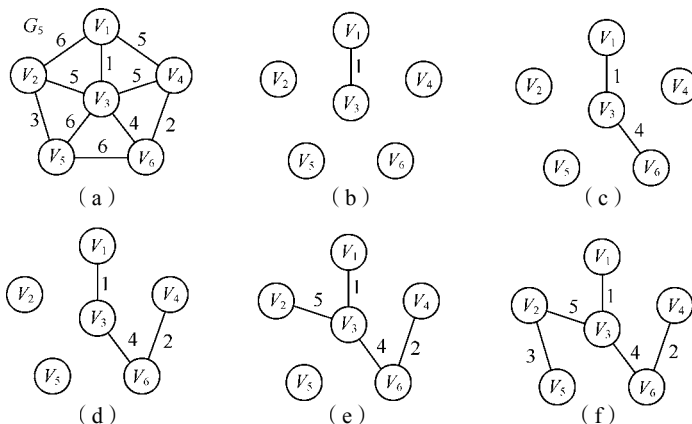


图 6.19 普里姆算法构造最小生成树的过程



每次选择最小边时, 可能存在多条同样权值的边可选, 此时任选其一即可。

#### (2) 普里姆算法的实现

假设一个无向网  $G$  以邻接矩阵形式存储, 从顶点  $u$  出发构造  $G$  的最小生成树  $T$ , 要求输出  $T$  的各条边。为实现这个算法需附设一个辅助数组 `closedge`, 以记录从  $U$  到  $V - U$  具有最小权值的边。对每个顶点  $v_i \in V - U$ , 在辅助数组中存在一个相应分量 `closedge[i-1]`, 它包括两个域: `lowcost` 和 `adjvex`, 其中 `lowcost` 存储最小边上的权值, `adjvex` 存储最小边在  $U$  中的那个顶点。显然, `closedge[i-1].lowcost = Min {cost( $u, v_i$ ) |  $u \in U$ }`, 其中 `cost( $u, v$ )` 表示赋于边  $(u, v)$  的权。

// 辅助数组的定义, 用来记录从顶点集  $U$  到  $V - U$  的权值最小的边

```

struct
{
    VerTexType adjvex;    //最小边在U中的那个顶点
    ArcType lowcost;      //最小边上的权值
}closedge[MVNum];

```

### 算法 6.8 普里姆算法

#### 【算法步骤】

- ① 首先将初始顶点  $u$  加入  $U$  中, 对其余的每一个顶点  $v_j$ , 将  $\text{closedge}[j]$  均初始化为到  $u$  的边信息。
- ② 循环  $n-1$  次, 做如下处理:
  - 从各组边  $\text{closedge}$  中选出最小边  $\text{closedge}[k]$ , 输出此边;
  - 将  $k$  加入  $U$  中;
  - 更新剩余的每组最小边信息  $\text{closedge}[j]$ , 对于  $V-U$  中的边, 新增加了一条从  $k$  到  $j$  的边, 如果新边的权值比  $\text{closedge}[j].\text{lowcost}$  小, 则将  $\text{closedge}[j].\text{lowcost}$  更新为新边的权值。

#### 【算法描述】

```

void MiniSpanTree_Prim(AMGraph G, VerTexType u)
{ //无向网G以邻接矩阵形式存储, 从顶点u出发构造G的最小生成树T, 输出T的各条边
    k=LocateVex(G,u);                //k为顶点u的下标
    for(j=0;j<G.vexnum;++j)           //对V-U的每一个顶点v_j, 初始化closedge[j]
        if(j!=k) closedge[j]={u,G.arcs[k][j]}; //{adjvex, lowcost}
    closedge[k].lowcost=0;              //初始, U={u}
    for(i=1;i<G.vexnum;++i)
    { //选择其余n-1个顶点, 生成n-1条边(n=G.vexnum)
        k=Min(closedge);
        //求出T的下一个结点: 第k个顶点, closedge[k]中存有当前最小边
        u0=closedge[k].adjvex;          //u0为最小边的一个顶点, u0∈U
        v0=G.vexs[k];                  //v0为最小边的另一个顶点, v0∈V-U
        cout<<u0<<v0;                 //输出当前的最小边(u0, v0)
        closedge[k].lowcost=0;          //第k个顶点并入U集
        for(j=0;j<G.vexnum;++j)
            if(G.arcs[k][j]<closedge[j].lowcost) //新顶点并入U后重新选择最小边
                closedge[j]={G.vexs[k],G.arcs[k][j]};
    }
}

```

#### 【算法分析】

分析算法 6.8, 假设网中有  $n$  个顶点, 则第一个进行初始化的循环语句的频度为  $n$ , 第二个循环语句的频度为  $n-1$ 。其中第二个有两个内循环: 其一是在  $\text{closedge}[v].\text{lowcost}$  中求最小值, 其频度为  $n-1$ ; 其二是重新选择具有最小权值的边, 其频度为  $n$ 。由此, 普里姆算法的时间复杂度为  $O(n^2)$ , 与网中的边数无关, 因此适用于求稠密网的最小生成树。

**【例 6.1】** 利用算法 6.8, 对图 6.19 (a) 所示的连通网  $G_5$  从顶点  $v_1$  开始构造最小生成树, 给出算法中各参量的变化。

各参量的变化如表 6.1 所示。

表 6.1

图 6.19 构造最小生成树过程中辅助数组中各分量的值

| $\begin{matrix} \text{closededge}[i] \\ i \end{matrix}$ | 0 | 1     | 2     | 3     | 4        | 5        | $U$                                | $V-U$                         | $k$ | $(u_0, v_0)$ |
|---------------------------------------------------------|---|-------|-------|-------|----------|----------|------------------------------------|-------------------------------|-----|--------------|
| adjvex                                                  |   | $v_1$ | $v_1$ | $v_1$ | $v_1$    | $v_1$    | $\{v_1\}$                          | $\{v_2, v_3, v_4, v_5, v_6\}$ | 2   | $(v_1, v_3)$ |
| lowcost                                                 | 0 | 6     | 1     | 5     | $\infty$ | $\infty$ |                                    |                               |     |              |
| adjvex                                                  |   | $v_3$ |       | $v_1$ | $v_3$    | $v_3$    | $\{v_1, v_3\}$                     | $\{v_2, v_4, v_5, v_6\}$      | 5   | $(v_3, v_6)$ |
| lowcost                                                 | 0 | 5     | 0     | 5     | 6        | 4        |                                    |                               |     |              |
| adjvex                                                  |   | $v_3$ |       | $v_6$ | $v_3$    |          | $\{v_1, v_3, v_6\}$                | $\{v_2, v_4, v_5\}$           | 3   | $(v_6, v_4)$ |
| lowcost                                                 | 0 | 5     | 0     | 2     | 6        | 0        |                                    |                               |     |              |
| adjvex                                                  |   | $v_3$ |       |       | $v_3$    |          | $\{v_1, v_3, v_6, v_4\}$           | $\{v_2, v_5\}$                | 1   | $(v_3, v_2)$ |
| lowcost                                                 | 0 | 5     | 0     | 0     | 6        | 0        |                                    |                               |     |              |
| adjvex                                                  |   |       |       |       | $v_2$    |          | $\{v_1, v_3, v_6, v_4, v_2\}$      | $\{v_5\}$                     | 4   | $(v_2, v_5)$ |
| lowcost                                                 | 0 | 0     | 0     | 0     | 3        | 0        |                                    |                               |     |              |
| adjvex                                                  |   |       |       |       |          |          | $\{v_1, v_3, v_6, v_4, v_2, v_5\}$ | $\{\}$                        |     |              |
| lowcost                                                 | 0 | 0     | 0     | 0     | 0        | 0        |                                    |                               |     |              |

初始状态时, 由于  $U = \{v_1\}$ , 则到  $V-U$  中各顶点的最小边, 即为从依附于顶点  $v_1$  的各条边中, 找到一条权值最小的边  $(v_1, v_3)$  为生成树上的第一条边, 同时将顶点  $v_3$  并入集合  $U$  中。然后修改辅助数组中的值, 首先将  $\text{closededge}[2].\text{lowcost}$  改为 0, 表明顶点  $v_3$  已并入  $U$ 。由于边  $(v_3, v_2)$  上的权值小于  $\text{closededge}[1].\text{lowcost}$ , 则需修改  $\text{closededge}[1]$  为边  $(v_3, v_2)$  及其权值。同理修改  $\text{closededge}[4]$  和  $\text{closededge}[5]$ 。依次类推, 直到  $U = V$ 。

## 2. 克鲁斯卡尔算法

### (1) 克鲁斯卡尔算法的构造过程

假设连通网  $N = (V, E)$ , 将  $N$  中的边按权值从小到大的顺序排列。

- ① 初始状态为只有  $n$  个顶点而无边的非连通图  $T = (V, \{\})$ , 图中每个顶点自成一个连通分量。
- ② 在  $E$  中选择权值最小的边, 若该边依附的顶点落在  $T$  中不同的连通分量上 (即不形成回路), 则将此边加入到  $T$  中, 否则舍去此边而选择下一条权值最小的边。
- ③ 重复②, 直至  $T$  中所有顶点都在同一连通分量上为止。

例如, 对图 6.19 (a) 所示的连通网  $G_5$ , 图 6.20 所示为依照克鲁斯卡尔算法构造一棵最小生成树的过程。权值分别为 1、2、3、4 的 4 条边由于满足上述条件, 则先后被加入到  $T$  中, 权值为 5 的两条边  $(v_1, v_4)$  和  $(v_3, v_4)$  被舍去。因为它们依附的两顶点在同一连通分量上, 它们若加入  $T$  中, 则会使  $T$  中产生回路, 而下一条权值 ( $= 5$ ) 最小的边  $(v_2, v_3)$  联结两个连通分量, 则可加入  $T$ 。由此, 构造成一棵最小生成树。

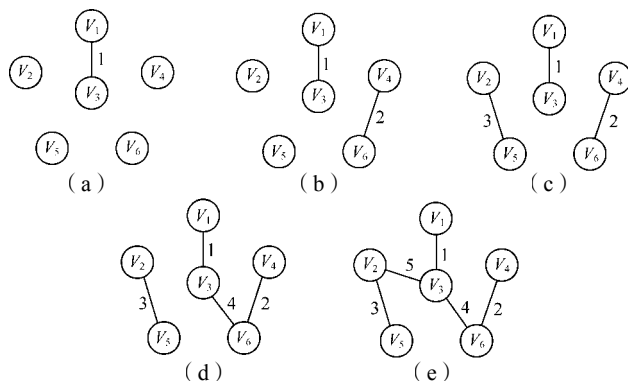


图 6.20 克鲁斯卡尔算法构造最小生成树的过程

可以看出,克鲁斯卡尔算法逐步增加生成树的边,与普里姆算法相比,可称为“加边法”。与普里姆算法一样,每次选择最小边时,可能有多条同样权值的边可选,可以任选其一。

## (2) 克鲁斯卡尔算法的实现

算法的实现要引入以下辅助的数据结构。

① 结构体数组 Edge: 存储边的信息,包括边的两个顶点信息和权值。

```
//辅助数组 Edges 的定义
struct
{
    VerTexType Head;        //边的始点
    VerTexType Tail;        //边的终点
    ArcType lowcost;        //边上的权值
}Edge[arcnum];
```

② Vexset[i]: 标识各个顶点所属的连通分量。对每个顶点  $v_i \in V$ , 在辅助数组中存在一个相应元素 Vexset[i] 表示该顶点所在的连通分量。初始时  $Vexset[i] = i$ , 表示各顶点自成一个连通分量。

```
//辅助数组 Vexset 的定义
int Vexset[MVNum];
```

## 算法 6.9 克鲁斯卡尔算法

### 【算法步骤】

① 将数组 Edge 中的元素按权值从小到大排序。

② 依次查看数组 Edge 中的边,循环执行以下操作:

- 依次从排好序的数组 Edge 中选出一条边( $U_1, U_2$ );
- 在 Vexset 中分别查找  $v_1$  和  $v_2$  所在的连通分量  $vs_1$  和  $vs_2$ , 进行判断:
  - 如果  $vs_1$  和  $vs_2$  不等,表明所选的两个顶点分属不同的连通分量,输出此边,并合并  $vs_1$  和  $vs_2$  两个连通分量;
  - 如果  $vs_1$  和  $vs_2$  相等,表明所选的两个顶点属于同一个连通分量,舍去此边而选择下一条权值最小的边。

### 【算法描述】

```
void MiniSpanTree_ Kruskal (AMGraph G)
{ //无向网 G 以邻接矩阵形式存储,构造 G 的最小生成树 T,输出 T 的各条边
    Sort (Edge); //将数组 Edge 中的元素按权值从小到大排序
    for (i=0; i<G.vexnum; ++i) //辅助数组,表示各顶点自成一个连通分量
        Vexset[i]=i;
    for (i=0; i<G.arcnum; ++i) //依次查看数组 Edge 中的边
    {
        v1=LocateVex (G, Edge[i].Head); //v1 为边的始点 Head 的下标
        v2=LocateVex (G, Edge[i].Tail); //v2 为边的终点 Tail 的下标
        vs1=Vexset[v1]; //获取边 Edge[i] 的始点所在的连通分量 vs1
        vs2=Vexset[v2]; //获取边 Edge[i] 的终点所在的连通分量 vs2
        if (vs1!=vs2) //边的两个顶点分属不同的连通分量
        {
            cout<< Edge[i].Head << Edge[i].Tail; //输出此边
            for (j=0; j<G.vexnum; ++j) //合并 vs1 和 vs2 两个分量,即两个集合统一编号
                if (Vexset[j]==vs2) Vexset[j]=vs1; //集合编号为 vs2 的都改为 vs1
        }
    }
}
```

```

    }
    }
}
//if
//for

```

### 【算法分析】

假若以第 8 章将介绍的“堆”来存放网中的边进行堆排序, 对于包含  $e$  条边的网, 上述算法排序时间是  $O(e \log_2 e)$ 。在 for 循环中最耗时的操作是合并两个不同的连通分量, 只要采取合适的数据结构, 可以证明其执行时间为  $O(\log_2 e)$ , 因此整个 for 循环的执行时间是  $O(e \log_2 e)$ , 由此, 克鲁斯卡尔算法的时间复杂度为  $O(e \log_2 e)$ , 与网中的边数有关, 与普里姆算法相比, 克鲁斯卡尔算法更适合于求稀疏网的最小生成树。

## 6.6.2 最短路径

假若要在计算机上建立一个交通咨询系统, 则可以采用图的结构来表示实际的交通网络。如图 6.21 所示, 图中顶点表示城市, 边表示城市间的交通联系。例如, 一位旅客要从 A 城到 B 城, 他希望选择一条中转次数最少的路线。假设图中每一站都需要换车, 则这个问题反映到图上就是要找一条从顶点 A 到 B 所含边的数目最少的路径。只需从顶点 A 出发对图做广度优先搜索, 一旦遇到顶点 B 就终止。由此所得的广度优先生成树上, 从根顶点 A 到顶点 B 的路径就是中转次数最少的路径, 路径上 A 与 B 之间的顶点数就是中转次数, 但是, 这只是一类最简单的图的最短路径问题。有时, 对于旅客来说, 可能更关心的是节省交通费用; 而对于司机来说, 里程和速度则是他们感兴趣的信息。为了在图上表示有关信息, 可对边赋以权, 权的值表示两城市间的距离, 或途中所需时间, 或交通费用等。此时路径长度的度量就不再是路径上边的数目, 而是路径上边的权值之和。考虑到交通图的有向性, 例如, 汽车的上山和下山, 轮船的顺水和逆水, 所花费的时间或代价就不相同, 所以交通网往往是用带权有向网表示。在带权有向网中, 习惯上称路径上的第一个顶点为源点 (Source), 最后一个顶点为终点 (Destination)。

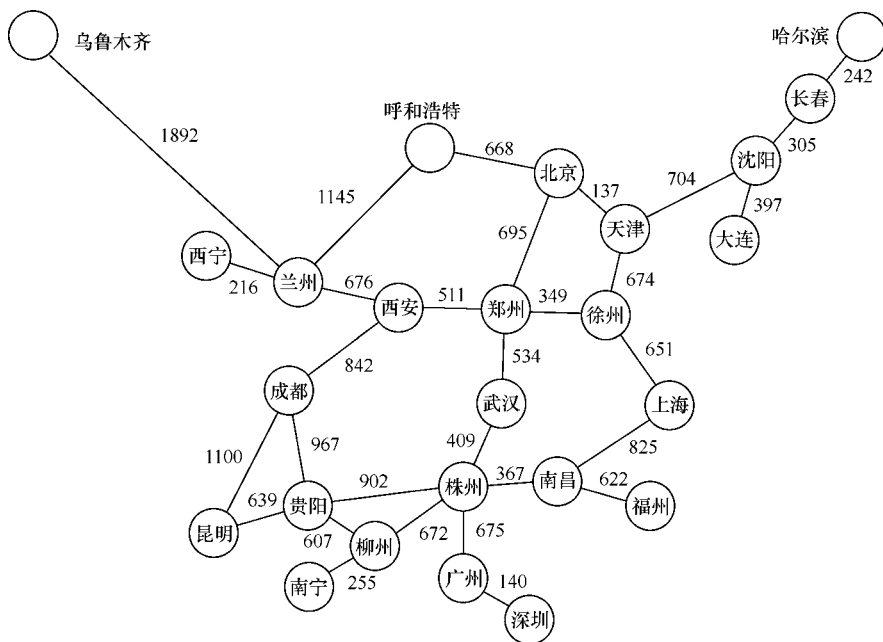


图 6.21 一个表示交通网的例图

本节主要讨论两种最常见的最短路径问题：一种是求从某个源点到其余各顶点的最短路径，另一种是求每一对顶点之间的最短路径。

### 1. 从某个源点到其余各顶点的最短路径

本节将讨论单源点的最短路径问题：给定带权有向图  $G$  和源点  $v_0$ ，求从  $v_0$  到  $G$  中其余各顶点的最短路径。迪杰斯特拉（Dijkstra）提出了一个按路径长度递增的次序产生最短路径的算法，称为迪杰斯特拉算法。

#### （1）迪杰斯特拉算法的求解过程

对于网  $N=(V, E)$ ，将  $N$  中的顶点分成两组：

第一组  $S$ ：已求出的最短路径的终点集合（初始时只包含源点  $v_0$ ）。

第二组  $V-S$ ：尚未求出的最短路径的顶点集合（初始时为  $V-\{v_0\}$ ）。

算法将按各顶点与  $v_0$  间最短路径长度递增的次序，逐个将集合  $V-S$  中的顶点加入到集合  $S$  中去。在这个过程中，总保持从  $v_0$  到集合  $S$  中各顶点的路径长度始终不大于到集合  $V-S$  中各顶点的路径长度。

这种求解方法能确保是正确的。因为，假设  $S$  为已求得最短路径的终点的集合，则可证明：下一条最短路径（设其终点为  $x$ ）或者是边  $(v, x)$ ，或者是中间只经过  $S$  中的顶点而最后到达顶点  $x$  的路径。

这可用反证法来证明。假设此路径上有一个顶点不在  $S$  中，则说明存在一条终点不在  $S$  而长度比此路径短的路径。但是，这是不可能的。因为算法是按路径长度递增的次序来产生最短路径的，故长度比此路径短的所有路径均已产生，它们的终点必定在  $S$  中，即假设不成立。

例如，图 6.22 所示的带权有向图  $G_6$  中，从  $v_0$  到其余各顶点之间的最短路径如表 6.2 所示。

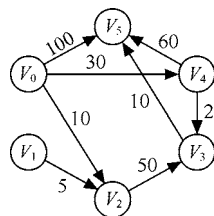


图 6.22 带权有向图  $G_6$

表 6.2 有向图  $G_6$  中从  $v_0$  到其余各点的最短路径

| 源点    | 终点    | 最短路径                   | 路径长度     |
|-------|-------|------------------------|----------|
| $v_0$ | $v_2$ | $(v_0, v_2)$           | 10       |
|       | $v_4$ | $(v_0, v_4)$           | 30       |
|       | $v_3$ | $(v_0, v_4, v_3)$      | 50       |
|       | $v_5$ | $(v_0, v_4, v_3, v_5)$ | 60       |
|       | $v_1$ | 无                      | $\infty$ |

根据迪杰斯特拉算法的求解过程，首先求出  $v_0$  到  $v_2$  的路径  $(v_0, v_2)$ ，然后按路径长度递增的次序依次得到  $v_0$  到  $v_4$  的路径  $(v_0, v_4)$ ， $v_0$  到  $v_3$  的路径  $(v_0, v_4, v_3)$ ， $v_0$  到  $v_5$  的路径  $(v_0, v_4, v_3, v_5)$ ，而从  $v_0$  到  $v_1$  没有路径。

#### （2）迪杰斯特拉算法的实现

假设用带权的邻接矩阵  $\text{arcs}$  来表示带权有向网  $G$ ， $\text{arcs}[i][j]$  表示弧  $\langle v_i, v_j \rangle$  上的权值。若  $\langle v_i, v_j \rangle$  不存在，则置  $\text{arcs}[i][j]$  为  $\infty$ ，源点为  $v_0$ 。

算法的实现要引入以下辅助的数据结构。

① 一维数组  $S[i]$ ：记录从源点  $v_0$  到终点  $v_i$  是否已被确定最短路径长度，true 表示确定，false 表示尚未确定。

② 一维数组  $\text{Path}[i]$ : 记录从源点  $v_0$  到终点  $v_i$  的当前最短路径上  $v_i$  的直接前驱顶点序号。其初值为: 如果从  $v_0$  到  $v_i$  有弧, 则  $\text{Path}[i]$  为  $v_0$ ; 否则为  $-1$ 。

③ 一维数组  $D[i]$ : 记录从源点  $v_0$  到终点  $v_i$  的当前最短路径长度。其初值为: 如果从  $v_0$  到  $v_i$  有弧, 则  $D[i]$  为弧上的权值; 否则为  $\infty$ 。

显然, 长度最短的一条最短路径必为  $(v_0, v_k)$ , 满足以下条件:

$$D[k] = \text{Min}\{D[i] | v_i \in V - S\}$$

求得顶点  $v_k$  的最短路径后, 将其加入到第一组顶点集  $S$  中。

每当加入一个新的顶点到顶点集  $S$ , 对第二组剩余的各个顶点而言, 多了一个“中转”顶点, 从而多了一个“中转”路径, 所以要对第二组剩余的各个顶点的最短路径长度进行更新。

原来  $v_0$  到  $v_i$  的最短路径长度为  $D[i]$ , 加进  $v_k$  之后, 以  $v_k$  作为中间顶点的“中转”路径长度为:  $D[k] + \text{Garcs}[k][i]$ , 若  $D[k] + \text{Garcs}[k][i] < D[i]$ , 则用  $D[k] + \text{Garcs}[k][i]$  取代  $D[i]$ 。

更新后, 再选择数组  $D$  中值最小的顶点加入到第一组顶点集  $S$  中, 如此进行下去, 直到图中所有顶点都加入到第一组顶点集  $S$  中为止。

#### 算法 6.10 迪杰斯特拉算法

##### 【算法步骤】

##### ① 初始化:

- 将源点  $v_0$  加到  $S$  中, 即  $S[v_0] = \text{true}$ ;
- 将  $v_0$  到各个终点的最短路径长度初始化为权值, 即  $D[i] = \text{Garcs}[v_0][v_i]$ , ( $v_i \in V - S$ );
- 如果  $v_0$  和顶点  $v_i$  之间有弧, 则将  $v_i$  的前驱置为  $v_0$ , 即  $\text{Path}[i] = v_0$ , 否则  $\text{Path}[i] = -1$ 。

##### ② 循环 $n - 1$ 次, 执行以下操作:

- 选择下一条最短路径的终点  $v_k$ , 使得:

$$D[k] = \text{Min}\{D[i] | v_i \in V - S\}$$

- 将  $v_k$  加到  $S$  中, 即  $S[v_k] = \text{true}$ ;
- 根据条件更新从  $v_0$  出发到集合  $V - S$  上任一顶点的最短路径的长度, 若条件  $D[k] + \text{Garcs}[k][i] < D[i]$  成立, 则更新  $D[i] = D[k] + \text{Garcs}[k][i]$ , 同时更改  $v_i$  的前驱为  $v_k$ ;  $\text{Path}[i] = k$ 。

##### 【算法描述】

```
void ShortestPath_DIJ(AMGraph G, int v0)
{ // 用 Dijkstra 算法求有向网 G 的 v0 顶点到其余顶点的最短路径
    n = G.vexnum; // n 为 G 中顶点的个数
    for (v = 0; v < n; ++v) // n 个顶点依次初始化
    {
        S[v] = false; // S 初始为空集
        D[v] = G.arcs[v0][v]; // 将 v0 到各个终点的最短路径长度初始化为弧上的权值
        if (D[v] < MaxInt) Path[v] = v0; // 如果 v0 和 v 之间有弧, 则将 v 的前驱置为 v0
        else Path[v] = -1; // 如果 v0 和 v 之间无弧, 则将 v 的前驱置为 -1
    } // for
    S[v0] = true; // 将 v0 加入 S
    D[v0] = 0; // 源点到源点的距离为 0
    /* ----- 初始化结束, 开始主循环, 每次求得 v0 到某个顶点 v 的最短路径, 将 v 加到 S 集 ----- */
    for (i = 1; i < n; ++i) // 对其余 n-1 个顶点, 依次进行计算
    {
        min = MaxInt;
        for (w = 0; w < n; ++w)
```

```

    if(!S[w]&&D[w]<min)
        {v=w;min=D[w];}           //选择一条当前的最短路径，终点为 v
    S[v]=true;                     //将 v 加入 S
    for(w=0;w<n;++w)               //更新从 v0 出发到集合 v-S 上所有顶点的最短路径长度
        if(!S[w]&&(D[v]+G.arcs[v][w]<D[w]))
        {
            D[w]=D[v]+G.arcs[v][w]; //更新 D[w]
            Path[w]=v;               //更改 w 的前驱为 v
        }
    }
}
//if
//for

```

【例 6.2】 利用算法 6.10，对图 6.22 所示的有向网  $G_6$  求解最短路径，给出算法中各参量的初始化结果和求解过程中的变化。

$G_6$  的邻接矩阵如图 6.23 所示。

$$\begin{bmatrix}
 \infty & \infty & 10 & \infty & 30 & 100 \\
 \infty & \infty & 5 & \infty & \infty & \infty \\
 \infty & \infty & \infty & 50 & \infty & \infty \\
 \infty & \infty & \infty & \infty & \infty & 10 \\
 \infty & \infty & \infty & 20 & \infty & 60 \\
 \infty & \infty & \infty & \infty & \infty & \infty
 \end{bmatrix}$$

图 6.23  $G_6$  的邻接矩阵

(1) 对图中 6 个顶点依次初始化，初始化结果如表 6.3 所示。

表 6.3 迪杰斯特拉算法初始化结果

|        | $v = 0$ | $v = 1$  | $v = 2$ | $v = 3$  | $v = 4$ | $v = 5$ |
|--------|---------|----------|---------|----------|---------|---------|
| $S$    | true    | false    | false   | false    | false   | false   |
| $D$    | 0       | $\infty$ | 10      | $\infty$ | 30      | 100     |
| $Path$ | -1      | -1       | 0       | -1       | 0       | 0       |

(2) 求解过程中各参量的变化如表 6.4 所示。

表 6.4 迪杰斯特拉算法求解过程中各参量的变化

| 终点     | 从 $v_0$ 到各终点的最短路径长度 $D$ 值和最短路径的求解过程    |                                             |                                                  |                                                       |          |
|--------|----------------------------------------|---------------------------------------------|--------------------------------------------------|-------------------------------------------------------|----------|
|        | $i = 1$                                | $i = 2$                                     | $i = 3$                                          | $i = 4$                                               | $i = 5$  |
| $v_1$  | $\infty$                               | $\infty$                                    | $\infty$                                         | $\infty$                                              | $\infty$ |
| $v_2$  | <u>10</u> ( $v_0, v_2$ )               |                                             |                                                  |                                                       |          |
| $v_3$  | $\infty$                               | 60( $v_0, v_2, v_3$ )                       | <u>50</u> ( $v_0, v_4, v_3$ )                    |                                                       |          |
| $v_4$  | 30( $v_0, v_4$ )                       | <u>30</u> ( $v_0, v_4$ )                    |                                                  |                                                       |          |
| $v_5$  | 100( $v_0, v_5$ )                      | 100( $v_0, v_5$ )                           | 90( $v_0, v_4, v_5$ )                            | <u>60</u> ( $v_0, v_4, v_3, v_5$ )                    |          |
| $v_k$  | $v_2$                                  | $v_4$                                       | $v_3$                                            | $v_5$                                                 | 无        |
| $Path$ |                                        | Path[3] = 2                                 | Path[3] = 4<br>Path[5] = 4                       | Path[5] = 3                                           |          |
| $S$    | $S[2] = \text{true}$<br>{ $v_0, v_2$ } | $S[4] = \text{true}$<br>{ $v_0, v_2, v_4$ } | $S[3] = \text{true}$<br>{ $v_0, v_2, v_4, v_3$ } | $S[5] = \text{true}$<br>{ $v_0, v_2, v_4, v_3, v_5$ } |          |

如何从表 6.4 中读取源点  $v_0$  到终点  $v_k$  的最短路径？以顶点  $k = 5$  为例：

$$Path[5] = 3 \rightarrow Path[3] = 4 \rightarrow Path[4] = 0$$

反过来排列, 得到路径 0、4、3、5, 这就是源点  $v_0$  到终点  $v_5$  的最短路径。

### 【算法分析】

算法 6.10 求解最短路径的主循环共进行  $n-1$  次, 每次执行的时间是  $O(n)$ , 所以算法的时间复杂度是  $O(n^2)$ 。如果用带权的邻接表作为有向图的存储结构, 则虽然修改  $D$  的时间可以减少, 但由于在  $D$  向量中选择最小分量的时间不变, 所以时间复杂度仍为  $O(n^2)$ 。

人们可能只希望找到从源点到某一个特定终点的最短路径, 但是, 这个问题和求源点到其他所有顶点的最短路径一样复杂, 也需要利用迪杰斯特拉算法来解决, 其时间复杂度仍为  $O(n^2)$ 。

### 2. 每一对顶点之间的最短路径

求解每一对顶点之间的最短路径有两种方法: 其一是分别以图中的每个顶点为源点共调用  $n$  次迪杰斯特拉算法; 其二是采用下面介绍的弗洛伊德 (Floyd) 算法。两种算法的时间复杂度均为  $O(n^3)$ , 但后者形式上较简单。

弗洛伊德算法仍然使用带权的邻接矩阵  $\text{arcs}$  来表示有向网  $G$ , 求从顶点  $v_i$  到  $v_j$  的最短路径。算法的实现要引入以下辅助的数据结构。

(1) 二维数组  $\text{Path}[i][j]$ : 最短路径上顶点  $v_j$  的前一顶点的序号。

(2) 二维数组  $D[i][j]$ : 记录顶点  $v_i$  和  $v_j$  之间的最短路径长度。

### 算法 6.11 弗洛伊德算法

#### 【算法步骤】

将  $v_i$  到  $v_j$  的最短路径长度初始化, 即  $D[i][j] = \text{Garc}[i][j]$ , 然后进行  $n$  次比较和更新。

① 在  $v_i$  和  $v_j$  间加入顶点  $v_0$ , 比较  $(v_i, v_j)$  和  $(v_i, v_0, v_j)$  的路径长度, 取其中较短者作为  $v_i$  到  $v_j$  的中间顶点序号不大于 0 的最短路径。

② 在  $v_i$  和  $v_j$  间加入顶点  $v_1$ , 得到  $(v_i, \dots, v_1)$  和  $(v_1, \dots, v_j)$ , 其中  $(v_i, \dots, v_1)$  是  $v_i$  到  $v_1$  的且中间顶点的序号不大于 0 的最短路径,  $(v_1, \dots, v_j)$  是  $v_1$  到  $v_j$  的且中间顶点的序号不大于 0 的最短路径, 这两条路径已在上一步中求出。比较  $(v_i, \dots, v_1, \dots, v_j)$  与上一步求出的  $v_i$  到  $v_j$  的中间顶点序号不大于 0 的最短路径, 取其中较短者作为  $v_i$  到  $v_j$  的中间顶点序号不大于 1 的最短路径。

③ 依次类推, 在  $v_i$  和  $v_j$  间加入顶点  $v_k$ , 若  $(v_i, \dots, v_k)$  和  $(v_k, \dots, v_j)$  分别是  $v_i$  到  $v_k$  和从  $v_k$  到  $v_j$  的中间顶点的序号不大于  $k-1$  的最短路径, 则将  $(v_i, \dots, v_k, \dots, v_j)$  和已经得到的从  $v_i$  到  $v_j$  且中间顶点序号不大于  $k-1$  的最短路径相比较, 其长度较短者便是从  $v_i$  到  $v_j$  的中间顶点的序号不大于  $k$  的最短路径。这样, 经过  $n$  次比较后, 最后求得的必是从  $v_i$  到  $v_j$  的最短路径。按此方法, 可以同时求得各对顶点间的最短路径。

根据上述求解过程, 图中的所有顶点对  $v_i$  和  $v_j$  间的最短路径长度对应一个  $n$  阶方阵  $D$ 。在上述  $n+1$  步中,  $D$  的值不断变化, 对应一个  $n$  阶方阵序列。

$n$  阶方阵序列可定义为:

$$D^{(-1)}, D^{(0)}, D^{(1)}, \dots, D^{(k)}, \dots, D^{(n-1)}$$

其中,

$$D^{(-1)}[i][j] = \text{Garc}[i][j]$$

$$D^{(k)}[i][j] = \text{Min}\{D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\} \quad 0 \leq k \leq n-1$$

显然,  $D^{(1)}[i][j]$  是从  $v_i$  到  $v_j$  的中间顶点的序号不大于 1 的最短路径的长度;  $D^{(k)}[i][j]$  是从  $v_i$  到  $v_j$  的中间顶点的序号不大于  $k$  的最短路径的长度;  $D^{(n-1)}[i][j]$  就是从  $v_i$  到  $v_j$  的最短路径的长度。

### 【算法描述】

```
void ShortestPath_Floyd(AMGraph G)
```

```

{ //用 Floyd 算法求有向网 G 中各对顶点 i 和 j 之间的最短路径
  for(i=0; i<G.vexnum; ++i)           //各对结点之间初始已知路径及距离
    for(j=0; j<G.vexnum; ++j)
    {
      D[i][j]=G.arcs[i][j];
      if(D[i][j]<MaxInt) Path[i][j]=i; //如果 i 和 j 之间有弧, 则将 j 的前驱置为 i
      else Path[i][j]=-1;             //如果 i 和 j 之间无弧, 则将 j 的前驱置为 -1
    }
  for(k=0; k<G.vexnum; ++k)
    for(i=0; i<G.vexnum; ++i)
      for(j=0; j<G.vexnum; ++j)
        if(D[i][k]+D[k][j]<D[i][j]) //从 i 经 k 到 j 的一条路径更短
        {
          D[i][j]=D[i][k]+D[k][j]; //更新 D[i][j]
          Path[i][j]=Path[k][j];   //更改 j 的前驱为 k
        }
      //if
    }
}

```

**【例 6.3】** 利用算法 6.11, 对图 6.24 所示有向网  $G_7$  求解最短路径, 给出每一对顶点之间的最短路径及其路径长度在求解过程中的变化。

$G_7$  的邻接矩阵如图 6.25 所示。

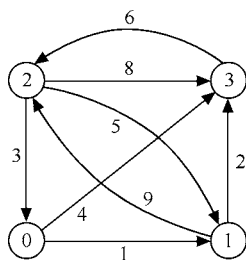


图 6.24 带权有向图  $G_7$

$$\begin{pmatrix}
 0 & 1 & 2 & 3 \\
 0 & 1 & \infty & 4 \\
 \infty & 0 & 9 & 2 \\
 3 & 5 & 0 & 8 \\
 \infty & \infty & 6 & 0
 \end{pmatrix}
 \begin{matrix}
 0 \\
 1 \\
 2 \\
 3
 \end{matrix}$$

图 6.25  $G_7$  的邻接矩阵

每一对顶点  $i$  和  $j$  之间的最短路径  $\text{Path}[i][j]$  以及其路径长度  $D[i][j]$  在求解过程中的变化如表 6.5 所示。

表 6.5 弗洛伊德算法求解过程中最短路径及其路径长度的变化

|   | $D^{(-1)}$           |          |          |    | $D^{(0)}$           |          |          |    | $D^{(1)}$           |          |    |    | $D^{(2)}$           |    |    |    | $D^{(3)}$           |    |    |    |
|---|----------------------|----------|----------|----|---------------------|----------|----------|----|---------------------|----------|----|----|---------------------|----|----|----|---------------------|----|----|----|
|   | 0                    | 1        | 2        | 3  | 0                   | 1        | 2        | 3  | 0                   | 1        | 2  | 3  | 0                   | 1  | 2  | 3  | 0                   | 1  | 2  | 3  |
| 0 | 0                    | 1        | $\infty$ | 4  | 0                   | 1        | $\infty$ | 4  | 0                   | 1        | 10 | 3  | 0                   | 1  | 10 | 3  | 0                   | 1  | 9  | 3  |
| 1 | $\infty$             | 0        | 9        | 2  | $\infty$            | 0        | 9        | 2  | $\infty$            | 0        | 9  | 2  | 12                  | 0  | 9  | 2  | 11                  | 0  | 8  | 2  |
| 2 | 3                    | 5        | 0        | 8  | 3                   | 4        | 0        | 7  | 3                   | 4        | 0  | 6  | 3                   | 4  | 0  | 6  | 3                   | 4  | 0  | 6  |
| 3 | $\infty$             | $\infty$ | 6        | 0  | $\infty$            | $\infty$ | 6        | 0  | $\infty$            | $\infty$ | 6  | 0  | 9                   | 10 | 6  | 0  | 9                   | 10 | 6  | 0  |
|   | $\text{Path}^{(-1)}$ |          |          |    | $\text{Path}^{(0)}$ |          |          |    | $\text{Path}^{(1)}$ |          |    |    | $\text{Path}^{(2)}$ |    |    |    | $\text{Path}^{(3)}$ |    |    |    |
|   | 0                    | 1        | 2        | 3  | 0                   | 1        | 2        | 3  | 0                   | 1        | 2  | 3  | 0                   | 1  | 2  | 3  | 0                   | 1  | 2  | 3  |
| 0 | -1                   | 0        | -1       | 0  | -1                  | 0        | -1       | 0  | -1                  | 0        | 1  | 1  | -1                  | 0  | 1  | 1  | -1                  | 0  | 3  | 1  |
| 1 | -1                   | -1       | 1        | 1  | -1                  | -1       | 1        | 1  | -1                  | -1       | 1  | 1  | 2                   | -1 | 1  | 1  | 2                   | -1 | 3  | 1  |
| 2 | 2                    | 2        | -1       | 2  | 2                   | 0        | -1       | 0  | 2                   | 0        | -1 | 1  | 2                   | 0  | -1 | 1  | 2                   | 0  | -1 | 1  |
| 3 | -1                   | -1       | 3        | -1 | -1                  | -1       | 3        | -1 | -1                  | -1       | 3  | -1 | 2                   | 0  | 3  | -1 | 2                   | 0  | 3  | -1 |

如何从表 6.5 中读取两个顶点之间的最短路径? 以  $\text{Path}^{(3)}$  为例, 对最短路径的读法加以说明。从  $D^{(3)}$  知, 顶点 1 到顶点 2 的最短路径长度为  $D[1][2] = 8$ , 其最短路径看  $\text{Path}[1][2] = 3$ , 表明顶点 2 的前驱是顶点 3; 再看  $\text{Path}[1][3] = 1$ , 表明顶点 3 的前驱是顶点 1。所以从顶点 1 到顶点 2 的最短路径为  $\langle 1, 3 \rangle$ ,  $\langle 3, 2 \rangle$ 。

### 6.6.3 拓扑排序

#### 1. AOV-网

一个无环的有向图称作有向无环图 (Directed Acycline Graph), 简称 DAG 图。有向无环图是描述一项工程或系统的进行过程的有效工具。通常把计划、施工过程、生产流程、程序流程等都当成一个工程。除了很小的工程外, 一般的工程都可分为若干个称做活动 (Activity) 的子工程, 而这些子工程之间, 通常受着一定条件的约束, 如其中某些子工程的开始必须在另一些子工程完成之后。

例如, 一个软件专业的学生必须学习一系列基本课程 (见表 6.6), 其中有些课程是基础课, 独立于其他课程, 如《高等数学》; 而另一些课程必须在学完作为其基础的先修课程才能开始。比如, 在《程序设计基础》和《离散数学》学完之前就不能开始学习《数据结构》。这些先决条件定义了课程之间的领先 (优先) 关系。这个关系可以用有向图更清楚地表示, 如图 6.26 所示。图中顶点表示课程, 有向弧表示先决条件。若课程  $i$  是课程  $j$  的先决条件, 则图中有弧  $\langle i, j \rangle$ 。

表 6.6 软件专业的必修课及其关系

| 课程编号     | 课程名称     | 先修课程               |
|----------|----------|--------------------|
| $C_1$    | 程序设计基础   | 无                  |
| $C_2$    | 离散数学     | $C_1$              |
| $C_3$    | 数据结构     | $C_1, C_2$         |
| $C_4$    | 汇编语言     | $C_1$              |
| $C_5$    | 高级语言程序设计 | $C_3, C_4$         |
| $C_6$    | 计算机原理    | $C_{11}$           |
| $C_7$    | 编译原理     | $C_3, C_5$         |
| $C_8$    | 操作系统     | $C_3, C_6$         |
| $C_9$    | 高等数学     | 无                  |
| $C_{10}$ | 线性代数     | $C_9$              |
| $C_{11}$ | 普通物理     | $C_9$              |
| $C_{12}$ | 数值分析     | $C_1, C_9, C_{10}$ |

这种用顶点表示活动, 用弧表示活动间的优先关系的有向图称为顶点表示活动的网 (Activity On Vertex Network), 简称 AOV-网。在网中, 若从顶点  $v_i$  到顶点  $v_j$  有一条有向路径, 则  $v_i$  是  $v_j$  的前驱;  $v_j$  是  $v_i$  的后继。若  $\langle v_i, v_j \rangle$  是网中一条弧, 则  $v_i$  是  $v_j$  的直接前驱,  $v_j$  是  $v_i$  的直接后继。

在 AOV-网中, 不应该出现有向环, 因为存在环意味着某项活动应以自己为先决条件。显然, 这是荒

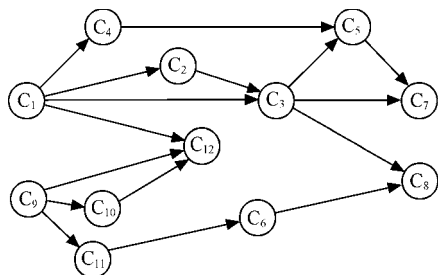


图 6.26 表示课程之间优先关系的有向图

谬的。若设计出这样的流程图,工程便无法进行。而对程序的数据流图来说,则表明存在一个死循环。因此,对给定的 AOV-网应首先判定网中是否存在环。检测的办法是对有向图的顶点进行拓扑排序,若网中所有顶点都在它的拓扑有序序列中,则该 AOV-网中必定不存在环。

所谓**拓扑排序**就是将 AOV-网中所有顶点排成一个线性序列,该序列满足:若在 AOV-网中由顶点  $v_i$  到顶点  $v_j$  有一条路径,则在该线性序列中的顶点  $v_i$  必定在顶点  $v_j$  之前。

例如,图 6.26 所示的有向图有如下两个拓扑有序序列(当然,对此图也可构造出其他的拓扑有序序列):

$C_1, C_2, C_3, C_4, C_5, C_7, C_9, C_{10}, C_{11}, C_6, C_{12}, C_8$

和

$C_9, C_{10}, C_{11}, C_6, C_1, C_{12}, C_4, C_2, C_3, C_5, C_7, C_8$

学生必须按照拓扑有序的顺序来安排学习计划,这样才能保证学习任一门课程时其先修课程已经学过。那么如何进行拓扑排序呢?

## 2. 拓扑排序的过程

(1) 在有向图中选一个无前驱的顶点且输出它。

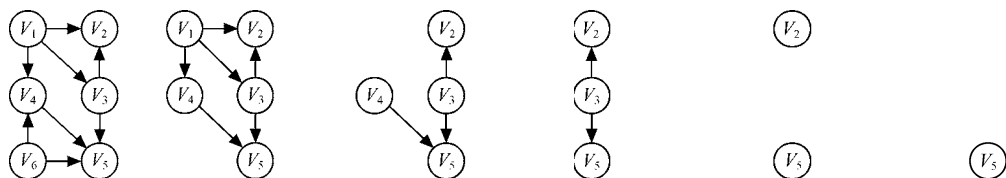
(2) 从图中删除该顶点和所有以它为尾的弧。

(3) 重复(1)和(2),直至不存在无前驱的顶点。

(4) 若此时输出的顶点数小于有向图中的顶点数,则说明有向图中存在环,否则输出的顶点序列即为一个拓扑序列。

以图 6.27 (a) 中所示的有向图为例,  $v_1$  和  $v_6$  没有前驱,则可任选一个。假设先输出  $v_6$ ,在删除  $v_6$  及弧  $\langle v_6, v_4 \rangle$ ,  $\langle v_6, v_5 \rangle$  之后,只有顶点  $v_1$  没有前驱,则输出  $v_1$  且删去  $v_1$  及弧  $\langle v_1, v_2 \rangle$ 、 $\langle v_1, v_3 \rangle$  和  $\langle v_1, v_4 \rangle$ ,之后  $v_3$  和  $v_4$  都没有前驱。依次类推,可从中任选一个继续进行。整个拓扑排序的过程如图 6.27 所示。最后得到该有向图的拓扑有序序列为

$v_6, v_1, v_4, v_3, v_2, v_5$



(a) AOV-网 (b) 输出  $v_6$  之后 (c) 输出  $v_1$  之后 (d) 输出  $v_4$  之后 (e) 输出  $v_3$  之后 (f) 输出  $v_2$  之后

图 6.27 AOV-网及其拓扑有序序列产生的过程

## 3. 拓扑排序的实现

针对上述拓扑排序的过程,可采用邻接表做有向图的存储结构。

算法的实现要引入以下辅助的数据结构。

(1) 一维数组  $\text{indegree}[i]$ : 存放各顶点入度,没有前驱的顶点就是入度为零的顶点。删除顶点及以它为尾的弧的操作,可不必真正对图的存储结构进行改变,可用弧头顶点的入度减 1 的办法来实现。

(2) 栈 S: 暂存所有入度为零的顶点,这样可以避免重复扫描数组  $\text{indegree}$  检测入度为 0 的顶点,提高算法的效率。

(3) 一维数组  $\text{topo}[i]$ : 记录拓扑序列的顶点序号。

### 算法 6.12 拓扑排序

#### 【算法步骤】

- ① 求出各顶点的入度存入数组  $\text{indegree}[i]$  中, 并将入度为 0 的顶点入栈。
- ② 只要栈不空, 则重复以下操作:
  - 将栈顶顶点  $v_i$  出栈并保存在拓扑序列数组  $\text{topo}$  中;
  - 对顶点  $v_i$  的每个邻接点  $v_k$  的入度减 1, 如果  $v_k$  的入度变为 0, 则将  $v_k$  入栈。
- ③ 如果输出顶点个数少于 AOV-网的顶点个数, 则网中存在有向环, 无法进行拓扑排序, 否则拓扑排序成功。

#### 【算法描述】

```
Status TopologicalSort(ALGraph G,int topo[])
{ //有向图 G 采用邻接表存储结构
  //若 G 无回路, 则生成 G 的一个拓扑序列 topo[] 并返回 OK, 否则 ERROR
  FindInDegree(G,indegree);           //求出各顶点的入度存入数组 indegree 中
  InitStack(S);                        //栈 S 初始化为空
  for(i=0;i<G.vexnum;++i)
    if(!indegree[i]) Push(S,i);       //入度为 0 者进栈
  m=0;                                //对输出顶点计数, 初始为 0
  while(!StackEmpty(S))                //栈 S 非空
  {
    Pop(S,i);                          //将栈顶顶点  $v_i$  出栈
    topo[m]=i;                         //将  $v_i$  保存在拓扑序列数组 topo 中
    ++m;                               //对输出顶点计数
    p=G.vertices[i].firstarc;          //p 指向  $v_i$  的第一个邻接点
    while(p!=NULL)
    {
      k=p->adjvex;                     //vk 为  $v_i$  的邻接点
      --indegree[k];                   //vi 的每个邻接点的入度减 1
      if(indegree[k]==0) Push(S,k);    //若入度减为 0, 则入栈
      p=p->nextarc;                    //p 指向顶点  $v_i$  下一个邻接点
    }
  }
  if(m<G.vexnum) return ERROR;         //该有向图有回路
  else return OK;
}
```

#### 【算法分析】

分析算法 6.12, 对有  $n$  个顶点和  $e$  条边的有向图而言, 建立求各顶点入度的时间复杂度为  $O(e)$ ; 建立零入度顶点栈的时间复杂度为  $O(n)$ ; 在拓扑排序过程中, 若有向图无环, 则每个顶点进一次栈, 出一次栈, 入度减 1 的操作在循环中总共执行  $e$  次, 所以, 总的时间复杂度为  $O(n+e)$ 。

上述拓扑排序的算法亦是下面讨论的求关键路径算法的基础。

## 6.6.4 关键路径

### 1. AOE-网

与 AOV-网相对应的是 AOE-网 (Activity On Edge), 即以边表示活动的网。AOE-网是一个带权的有向无环图, 其中, 顶点表示事件, 弧表示活动, 权表示活动持续的时间。通常, AOE-

网可用来估算工程的完成时间。

例如,图 6.28 所示为一个有 11 项活动的 AOE-网。其中有 9 个事件  $v_0, v_1, \dots, v_8$ , 每个事件表示在它之前的活动已经完成, 在它之后的活动可以开始。例如,  $v_0$  表示整个工程开始,  $v_8$  表示整个工程结束,  $v_4$  表示  $a_4$  和  $a_5$  已经完成,  $a_7$  和  $a_8$  可以开始了。与每个活动相联系的数是执行该活动所需的时间, 比如, 活动  $a_1$  需要 6 天,  $a_2$  需要 4 天等。

AOE-网在工程计划和经营管理中有广泛的应用, 针对实际的应用问题, 通常需要解决以下两个问题:

- (1) 估算完成整项工程至少需要多少时间;
- (2) 判断哪些活动是影响工程进度的关键。

工程进度控制的关键在于抓住关键活动。在一定范围内, 非关键活动的提前完成对于整个工程的进度没有直接的好处, 它的稍许拖延也不会影响整个工程的进度。工程的指挥者可以把非关键活动的人力和物力资源暂时调给关键活动, 加速其进展速度, 以使整个工程提前完工。

由于整个工程只有一个开始点和一个完成点, 故在正常的情况(无环)下, 网中只有一个入度为零的点, 称作源点, 也只有一个出度为零的点, 称作汇点。在 AOE-网中, 一条路径各弧上的权值之和称为该路径的带权路径长度(后面简称路径长度)。要估算整项工程完成的最短时间, 就是要找一条从源点到汇点的带权路径长度最长的路径, 称为关键路径(Critical Path)。关键路径上的活动叫做关键活动, 这些活动是影响工程进度的关键, 它们的提前或拖延将使整个工程提前或拖延。

例如, 在图 6.28 中,  $v_0$  是源点,  $v_8$  是汇点, 关键路径有两条:  $(v_0, v_1, v_4, v_6, v_8)$  或  $(v_0, v_1, v_4, v_7, v_8)$ , 长度均为 18。关键活动为  $(a_1, a_4, a_7, a_{10})$  或  $(a_1, a_4, a_8, a_{11})$ 。比如, 关键活动  $a_1$  需要 6 天完成, 如果  $a_1$  提前 1 天, 整个工程也可以提前 1 天完成。所以不论是估算工期, 还是研究如何加快工程进度, 主要问题就在于要找到 AOE-网的关键路径。

如何确定关键路径, 首先定义 4 个描述量。

- (1) 事件  $v_i$  的最早发生时间  $ve(i)$

进入事件  $v_i$  的每一活动都结束,  $v_i$  才可发生, 所以  $ve(i)$  是从源点到  $v_i$  的最长路径长度。

求  $ve(i)$  的值, 可根据拓扑顺序从源点开始向汇点递推。通常将工程的开始顶点事件  $v_0$  的最早发生时间定义为 0, 即

$$ve(0) = 0$$

$$ve(i) = \text{Max}\{ve(k) + w_{k,i}\} \quad \langle v_k, v_i \rangle \in T, 1 \leq i \leq n-1$$

其中,  $T$  是所有以  $v_i$  为头的弧的集合,  $w_{k,i}$  是弧  $\langle v_k, v_i \rangle$  的权值, 即对应活动  $\langle v_k, v_i \rangle$  的持续时间。

- (2) 事件  $v_i$  的最迟发生时间  $vl(i)$

事件  $v_i$  的发生不得延误  $v_i$  的每一后继事件的最迟发生时间。为了不拖延工期,  $v_i$  的最迟发生时间不得迟于其后继事件  $v_k$  的最迟发生时间减去活动  $\langle v_i, v_k \rangle$  的持续时间。

求出  $ve(i)$  后, 可根据逆拓扑顺序从汇点开始向源点递推, 求出  $vl(i)$ 。

$$vl(n-1) = ve(n-1)$$

$$vl(i) = \text{Min}\{vl(k) - w_{i,k}\} \quad \langle v_i, v_k \rangle \in S, 0 \leq i \leq n-2$$

其中,  $S$  是所有以  $v_i$  为尾的弧的集合,  $w_{i,k}$  是弧  $\langle v_i, v_k \rangle$  的权值。

- (3) 活动  $a_i = \langle v_j, v_k \rangle$  的最早开始时间  $e(i)$

只有事件  $v_j$  发生了, 活动  $a_i$  才能开始。所以, 活动  $a_i$  的最早开始时间等于事件  $v_j$  的最早发生时间  $ve(j)$ , 即

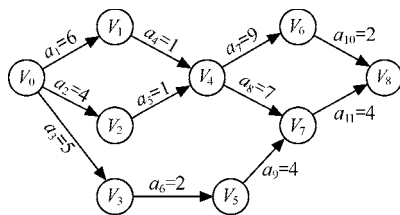


图 6.28 一个 AOE-网

$$e(i) = ve(j)$$

(4) 活动  $a_i = \langle v_j, v_k \rangle$  的最晚开始时间  $l(i)$

活动  $a_i$  的开始时间需保证不延误事件  $v_k$  的最迟发生时间。所以活动  $a_i$  的最晚开始时间  $l(i)$  等于事件  $v_k$  的最迟发生时间  $vl(k)$  减去活动  $a_i$  的持续时间  $w_{j,k}$ , 即:

$$l(i) = vl(k) - w_{j,k}$$

显然, 对于关键活动而言,  $e(i) = l(i)$ 。对于非关键活动,  $l(i) - e(i)$  的值是该工程的期限余量, 在此范围内的适度延误不会影响整个工程的工期。

一个活动  $a_i$  的最迟开始时间  $l(i)$  和其最早开始时间  $e(i)$  的差值  $l(i) - e(i)$  是该活动完成的时间余量。它是在不增加完成整个工程所需的总时间的情况下, 活动  $a_i$  可以拖延的时间。当一活动的时间余量为零时, 说明该活动必须如期完成, 否则就会拖延整个工程的进度。所以称  $l(i) - e(i) = 0$ , 即  $l(i) = e(i)$  时的活动  $a_i$  是关键活动。

## 2. 关键路径求解的过程

- (1) 对图中顶点进行排序, 在排序过程中按拓扑序列求出每个事件的最早发生时间  $ve(i)$ 。
- (2) 按逆拓扑序列求出每个事件的最迟发生时间  $vl(i)$ 。
- (3) 求出每个活动  $a_i$  的最早开始时间  $e(i)$ 。
- (4) 求出每个活动  $a_i$  的最晚开始时间  $l(i)$ 。
- (5) 找出  $e(i) = l(i)$  的活动  $a_i$ , 即为关键活动。由关键活动形成的由源点到汇点的每一条路径就是关键路径, 关键路径有可能不止一条。

**【例 6.4】** 对图 6.28 所示的 AOE-网, 计算关键路径。

计算过程如下。

(1) 计算各顶点事件  $v_i$  的最早发生时间  $ve(i)$ 。

$$ve(0) = 0$$

$$ve(1) = \text{Max}\{ve(0) + w_{0,1}\} = 6$$

$$ve(2) = \text{Max}\{ve(0) + w_{0,2}\} = 4$$

$$ve(3) = \text{Max}\{ve(0) + w_{0,3}\} = 5$$

$$ve(4) = \text{Max}\{ve(1) + w_{1,4}, ve(2) + w_{2,4}\} = 7$$

$$ve(5) = \text{Max}\{ve(3) + w_{3,5}\} = 7$$

$$ve(6) = \text{Max}\{ve(4) + w_{4,6}\} = 16$$

$$ve(7) = \text{Max}\{ve(4) + w_{4,7}, ve(5) + w_{5,7}\} = 14$$

$$ve(8) = \text{Max}\{ve(6) + w_{6,8}, ve(7) + w_{7,8}\} = 18$$

(2) 计算各顶点事件  $v_i$  的最迟发生时间  $vl(i)$ 。

$$vl(8) = ve(8) = 18$$

$$vl(7) = \text{Min}\{vl(8) - w_{7,8}\} = 14$$

$$vl(6) = \text{Min}\{vl(8) - w_{6,8}\} = 16$$

$$vl(5) = \text{Min}\{vl(7) - w_{5,7}\} = 10$$

$$vl(4) = \text{Min}\{vl(6) - w_{4,6}, vl(7) - w_{4,7}\} = 7$$

$$vl(3) = \text{Min}\{vl(5) - w_{3,5}\} = 8$$

$$vl(2) = \text{Min}\{vl(4) - w_{2,4}\} = 6$$

$$vl(1) = \text{Min}\{vl(4) - w_{1,4}\} = 6$$

$$vl(0) = \text{Min}\{vl(1) - w_{0,1}, vl(2) - w_{0,2}, vl(3) - w_{0,3}\} = 0$$

(3) 计算各活动  $a_i$  的最早开始时间  $e(i)$ 。

$$e(a_1) = ve(0) = 0$$

$$e(a_2) = ve(0) = 0$$

$$e(a_3) = ve(0) = 0$$

$$e(a_4) = ve(1) = 6$$

$$e(a_5) = ve(2) = 4$$

$$e(a_6) = ve(3) = 5$$

$$e(a_7) = ve(4) = 7$$

$$e(a_8) = ve(4) = 7$$

$$e(a_9) = ve(5) = 7$$

$$e(a_{10}) = ve(6) = 16$$

$$e(a_{11}) = ve(7) = 14$$

(4) 计算各活动  $a_i$  的最迟开始时间  $l(i)$ 。

$$l(a_{11}) = vl(8) - w_{7,8} = 14$$

$$l(a_{10}) = vl(8) - w_{6,8} = 16$$

$$l(a_9) = vl(7) - w_{5,7} = 10$$

$$l(a_8) = vl(7) - w_{4,7} = 7$$

$$l(a_7) = vl(6) - w_{4,6} = 7$$

$$l(a_6) = vl(5) - w_{3,5} = 8$$

$$l(a_5) = vl(4) - w_{2,4} = 6$$

$$l(a_4) = vl(4) - w_{1,4} = 6$$

$$l(a_3) = vl(3) - w_{0,3} = 3$$

$$l(a_2) = vl(2) - w_{0,2} = 2$$

$$l(a_1) = vl(1) - w_{0,1} = 0$$

将顶点的发生时间和活动的开始时间分别分别汇总为表 6.7 (a) 和表 6.7 (b)。由表 6.7 (b) 可以看出, 图 6.28 所示的 AOE-网有两条关键路径: 一条是由活动  $(a_1, a_4, a_7, a_{10})$  组成的关键路径, 另一条是由  $(a_1, a_4, a_8, a_{11})$  组成的关键路径, 如图 6.29 所示。

表 6.7

图 6.28 所示网的关键路径求解的中间结果

(a) 顶点的发生时间

| 顶点 $i$ | $ve(i)$ | $vl(i)$ |
|--------|---------|---------|
| 0      | 0       | 0       |
| 1      | 6       | 6       |
| 2      | 4       | 6       |
| 3      | 5       | 8       |
| 4      | 7       | 7       |
| 5      | 7       | 10      |
| 6      | 16      | 16      |
| 7      | 14      | 14      |
| 8      | 18      | 18      |

(b) 活动的开始时间

| 活动 $a_i$ | $e(i)$ | $l(i)$ | $l(i) - e(i)$ |
|----------|--------|--------|---------------|
| $a_1$    | 0      | 0      | 0             |
| $a_2$    | 0      | 2      | 2             |
| $a_3$    | 0      | 3      | 3             |
| $a_4$    | 6      | 6      | 0             |
| $a_5$    | 4      | 6      | 2             |
| $a_6$    | 5      | 8      | 3             |
| $a_7$    | 7      | 7      | 0             |
| $a_8$    | 7      | 7      | 0             |
| $a_9$    | 7      | 10     | 3             |
| $a_{10}$ | 16     | 16     | 0             |
| $a_{11}$ | 14     | 14     | 0             |

### 3. 关键路径算法的实现

由于每个事件的最早发生时间  $ve(i)$  和最迟发生时间  $vl(i)$  要在拓扑序列的基础上进行计算, 所以关键路径算法的实现要基于拓扑排序算法, 我们仍采用邻接表做有向图的存储结构。

算法的实现要引入以下辅助的数据结构。

- (1) 一维数组  $ve[i]$ : 事件  $v_i$  的最早发生时间。
- (2) 一维数组  $vl[i]$ : 事件  $v_i$  的最迟发生时间。
- (3) 一维数组  $topo[i]$ : 记录拓扑序列的顶点序号。

#### 算法 6.13 关键路径算法

##### 【算法步骤】

- ① 调用拓扑排序算法, 使拓扑序列保存在  $topo$  中。
- ② 将每个事件的最早发生时间  $ve[i]$  初始化为 0,  $ve[i] = 0$ 。
- ③ 根据  $topo$  中的值, 按从前向后的拓扑次序, 依次求每个事件的最早发生时间, 循环几次, 执行以下操作:

- 取得拓扑序列中的顶点序号  $k$ ,  $k = topo[i]$ ;
- 用指针  $p$  依次指向  $k$  的每个邻接顶点, 取得每个邻接顶点的序号  $j = p \rightarrow adjvex$ , 依次更新顶点  $j$  的最早发生时间  $ve[j]$

$if(ve[j] < ve[k] + p \rightarrow weight) \quad ve[j] = ve[k] + p \rightarrow weight;$

- ④ 将每个事件的最迟发生时间  $vl[i]$  初始化为汇点的最早发生时间,  $vl[i] = ve[n-1]$ 。
- ⑤ 根据  $topo$  中的值, 按从后向前的逆拓扑次序, 依次求每个事件的最迟发生时间, 循环  $n$  次, 执行以下操作:

- 取得拓扑序列中的顶点序号  $k$ ,  $k = topo[i]$ ;
- 用指针  $p$  依次指向  $k$  的每个邻接顶点, 取得每个邻接顶点的序号  $j = p \rightarrow adjvex$ , 依次根据  $k$  的邻接点, 更新  $k$  的最迟发生时间  $vl[k]$

$if(vl[k] > vl[j] - p \rightarrow weight) \quad vl[k] = vl[j] - p \rightarrow weight;$

- ⑥ 判断某一活动是否为关键活动, 循环  $n$  次, 执行以下操作: 对于每个顶点  $i$ , 用指针  $p$  依次指向  $i$  的每个邻接顶点, 取得每个邻接顶点的序号  $j = p \rightarrow adjvex$ , 分别计算活动  $\langle v_i, v_j \rangle$  的最早和最迟开始时间  $e$  和  $l$ ,

$e = ve[i]; l = vl[j] - p \rightarrow weight;$

如果  $e$  和  $l$  相等, 则活动  $\langle v_i, v_j \rangle$  为关键活动, 输出弧  $\langle v_i, v_j \rangle$ 。

##### 【算法描述】

```
Status CriticalPath(ALGraph G)
{ //G 为邻接表存储的有向网, 输出 G 的各项关键活动
  if(!TopologicalOrder(G, topo)) return ERROR;
  //调用拓扑排序算法, 使拓扑序列保存在 topo 中, 若调用失败, 则存在有向环, 返回 ERROR
  n=G.vexnum; //n 为顶点个数
  for(i=0; i<n; i++) //给每个事件的最早发生时间置初值 0
    ve[i]=0;
  /* - - - - - 按拓扑次序求每个事件的最早发生时间 - - - - - */
  for(i=0; i<n; i++)
  {
```

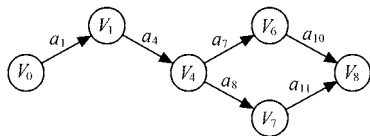


图 6.29 图 6.28 所示网的关键路径

```

        k=topo[i];                //取得拓扑序列中的顶点序号 k
        p=G.vertices[k].firstarc; //p 指向 k 的第一个邻接顶点
        while (p!=NULL)
        {
            j=p->adjvex;          //依次更新 k 的所有邻接顶点的最早发生时间
            //j 为邻接顶点的序号
            if (ve[j]<ve[k]+p->weight) //更新顶点 j 的最早发生时间 ve[j]
                ve[j]=ve[k]+p->weight;
            p=p->nextarc;          //p 指向 k 的下一个邻接顶点
        }                         //while
    }                             //for
    for(i=0;i<n;i++)              //给每个事件的最迟发生时间置初值 ve[n-1]
        vl[i]=ve[n-1];
/*- - - - -按逆拓扑次序求每个事件的最迟发生时间- - - - -*/
for(i=n-1;i>=0;i--)
{
    k=topo[i];                //取得拓扑序列中的顶点序号 k
    p=G.vertices[k].firstarc;  //p 指向 k 的第一个邻接顶点
    while (p!=NULL)           //根据 k 的邻接点, 更新 k 的最迟发生时间
    {
        j=p->adjvex;          //j 为邻接顶点的序号
        if (vl[k]>vl[j]-p->weight) //更新顶点 k 的最迟发生时间 vl[k]
            vl[k]=vl[j]-p->weight;
        p=p->nextarc;          //p 指向 k 的下一个邻接顶点
    }                         //while
}                             //for
/*- - - - -判断每一活动是否为关键活动- - - - -*/
for(i=0;i<n;i++) //每次循环针对  $v_i$  为活动开始点的所有活动
{
    p=G.vertices[i].firstarc;  //p 指向 i 的第一个邻接顶点
    while (p!=NULL)
    {
        j=p->adjvex;          //j 为 i 的邻接顶点的序号
        e=ve[i];              //计算活动< $v_i, v_j$ >的最早开始时间
        l=vl[j]-p->weight;     //计算活动< $v_i, v_j$ >的最迟开始时间
        if (e==l)              //若为关键活动, 则输出< $v_i, v_j$ >
            cout<<G.vertices[i].data <<G.vertices[j].data;
        p=p->nextarc;          //p 指向 i 的下一个邻接顶点
    } //while
}                             //for
}

```

### 【算法分析】

在算法 6.13 中, 在求每个事件的最早和最迟发生时间, 以及活动的最早和最迟开始时间时, 都要对所有顶点及每个顶点边表中所有的边结点进行检查, 由此, 求关键路径算法的时间复杂度为  $O(n+e)$ 。

实践已经证明: 用 AOE-网来估算某些工程完成的时间是非常有用的。实际上, 求关键路径的方法本身最初就是与维修和建造工程一起发展的。但是, 由于网中各项活动是互相牵涉的, 因此, 影响关键活动的因素亦是多方面的, 任何一项活动持续时间的改变都可能引起关键路径的改变。所以, 当子工程在进行过程中持续时间有所调整时, 就要重新计算关键路径。另外, 若网中有几条关键路径, 那么, 单是提高一条关键路径上关键活动的速度, 还不能导致整个工程缩短工

期, 而必须同时提高在几条关键路径上的活动速度。

## 6.7 案例分析与实现

### 案例 6.1: 六度空间理论。

#### 【案例分析】

在六度空间理论提出之后的 30 多年的时间里, 社会学家试图证明(或否定)此假设的正确性, 但是该理论从来没有得到过严谨的证明, 虽然屡屡应验, 但它只是一种假说。很多社会学家主持的验证研究, 都使用了网络时代的新型通信手段——E-mail。

比较著名的实验是 2001 年哥伦比亚大学社会学系的登肯·瓦兹主持的一项验证工程。166 个不同国家的六万多名志愿者参加了该项研究。瓦兹随机选定 18 名目标(比如一名美国的教授、一名澳大利亚警察和一名挪威兽医等), 要求志愿者选择其中的一名作为自己的目标, 并发送电子邮件给自己认为最有可能发送邮件给目标的亲友。研究取得了较好的验证成果, 瓦兹在世界最顶级的科学学术期刊《科学》杂志上发表了论文, 表明邮件要达到目标, 平均也只要经历 5~7 个人左右。

但实际上, 这种研究方式有很大的局限性和困难。第一, 使用 E-mail 保持社会关系的人群是有限的; 第二, 要记录和跟踪所有 E-mail 的走向是一项巨大的工程, 需要大量的人力和较长的时间; 第三, 验证过程与志愿者的意愿紧密相关, 志愿者可能会遗漏某些相识的人。

现代人使用电话和短信进行联络的频率远远大于使用 E-mail 的频率。由于电话和短信的通信都有一个运营商, 与 E-mail 的通信相比, 更便于跟踪。为了排除部分广告电话和广告短信, 我们可以假设任意两个人在一年内, 电话或短信相互收发两次以上即定义为两人“认识”, 这样便很容易根据电话或短信的通信信息确定两人是否存在“认识”的关系。但在实际操作中, 由于通信数据保密的原因, 我们无法获取实际的通信数据, 所以我们只能从理论上介绍并分析验证的方法。

我们把六度空间理论中的人际关系网络图抽象成一个不带权值的无向图 G, 用图 G 中的一个顶点表示一个人, 两个人“认识”与否, 用代表这两个人的顶点之间是否有一条边来表示。这样六度空间理论问题便可描述为: 在图 G 中, 任意两个顶点之间都存在一条路径长度不超过 7 的路径。

在实际验证过程中, 可以通过测试满足要求的数据达到一定的百分比(比如 99.5%)来进行验证。这样我们便把待验证六度空间理论问题描述为: 在图 G 中, 任意一个顶点到其余 99.5%以上的顶点都存在一条路径长度不超过 7 的路径。

比较简单的一种验证方案是: 利用广度优先搜索方法, 对任意一个顶点, 通过对图 G 的“7 层”遍历, 就可以统计出所有路径长度不超过 7 的顶点数, 从而得到这些顶点在所有顶点中的所占比例。

#### 【案例实现】

#### 算法 6.14 六度空间理论的验证

#### 【算法步骤】

① 完成系列初始化工作: 设变量 Visit\_Num 用来记录路径长度不超过 7 的顶点个数, 初值为 0; Start 为指定的一个起始顶点, 置 visited[Start]的值为 true, 即将 Start 标记为六度顶点的始点; 辅助队列 Q 初始化为空, 然后将 Start 进队。

② 当队列  $Q$  非空, 且循环次数小于 7 时, 循环执行以下操作 (统计路径长度不超过 7 的顶点个数):

- 队头顶点  $u$  出队;
- 依次检查  $u$  的所有邻接点  $w$ , 如果  $\text{visited}[w]$  的值为  $\text{false}$ , 则将  $w$  标记为六度顶点;
- 路径长度不超过 7 的顶点个数  $\text{Visit\_Num}$  加 1;
- 将  $w$  进队。

③ 退出循环时输出从顶点  $\text{Start}$  出发, 到其他顶点长度不超过 7 的路径的百分比。

### 【算法描述】

```
void SixDegree_BFS(Graph G,int Start)
{//通过广度优先搜索方法遍历 G 来验证六度空间理论, Start 为指定的一个起点
    Visit_Num=0;                //记录路径长度不超过 7 的顶点个数
    visited[Start]=true;        //置顶点 Start 访问标志数组相应分量值为 true
    InitQueue(Q);               //辅助队列 Q 初始化, 置空
    EnQueue(Q, Start);          //Start 进队
    for(len=1;len<=7 && !QueueEmpty(Q);len++)//统计路径长度不超过 7 的顶点个数
    {
        DeQueue(Q,u);           //队头顶点 u 出队
        for(w=FirstAdjVex(G,u);w>=0;w=NextAdjVex(G,u,w))
            //依次检查 u 的所有邻接点 w, FirstAdjVex(G,u) 表示 u 的第一个邻接点
            //NextAdjVex(G,u,w) 表示 u 相对于 w 的下一个邻接点, w>=0 表示存在邻接点
            if(!visited[w])      //w 为 u 的尚未访问的邻接点
            {
                visited[w]=true; //将 w 标记为六度顶点
                Visit_Num++;      //路径长度不超过 7 的顶点个数加 1
                EnQueue(Q,w);     //w 进队
            }                    //if
    }                            //结束至多 7 次 for 循环
    cout<<100*Visit_Num/G.vexnum;
    //输出从顶点 Start 出发, 到其他顶点长度不超过 7 的路径的百分比
}
```

### 【算法分析】

假定人际关系网络图  $G$  中有 10 亿人, 即图中的顶点个数  $n=10$  亿。根据“150 定律”, 如果平均每个人认识其他 150 个人, 则该图中边的个数  $e \approx 150 * n / 2 = 75 * 10^9$ , 该算法的时间复杂度为  $O(n+e)$ , 约为 100G, 对于现代每秒万亿次的运算速度的计算机来说, 每秒钟可以验证数个顶点, 每天可以验证数万人。算法在空间上需要借助数组  $\text{visited}$  和队列  $Q$ , 因而空间复杂度为  $O(n)$

算法 6.14 给出了利用广度优先搜索方法进行验证的方案, 实际上也可以利用求解最短路径的方法 (迪杰斯特拉算法或弗洛伊德算法) 对六度空间理论进行理论上的验证。读者可以根据算法 6.14 和最短路算法自行写出相应的验证方法。

## 6.8 小 结

图是一种复杂的非线性数据结构, 具有广泛的应用背景。本章主要内容如下。

(1) 根据不同的分类规则, 图分为多种类型: 无向图、有向图、完全图、连通图、强连通图、带权图(网)、稀疏图和稠密图等。邻接点、路径、回路、度、连通分量、生成树等是在图的算法设计中常用到的重要术语。

(2) 图的存储方式有两大类: 以边集合方式的表示法和以链接方式的表示法。其中, 以边集合方式表示的为邻接矩阵, 以链接方式表示的包括邻接表、十字链表和邻接多重表。邻接矩阵表示法借助二维数组来表示元素之间的关系, 实现起来较为简单; 邻接表、十字链表和邻接多重表都属于链式存储结构, 实现起来较为复杂。在实际应用中具体采取哪种存储表示, 可以根据图的类型和实际算法的基本思想进行选择。其中, 邻接矩阵和邻接表是两种常用的存储结构, 二者之间的比较如表 6.8 所示。

表 6.8 邻接矩阵和邻接表的比较

| 存储结构<br>比较项目 |                       | 邻接矩阵                            |                                                | 邻接表                       |                                                                 |
|--------------|-----------------------|---------------------------------|------------------------------------------------|---------------------------|-----------------------------------------------------------------|
|              |                       | 无向图                             | 有向图                                            | 无向图                       | 有向图                                                             |
| 空间           |                       | 邻接矩阵对称, 可压缩至 $n(n-1)/2$ 个单元     | 邻接矩阵不对称, 存储 $n^2$ 个单元                          | 存储 $n+2e$ 个单元             | 存储 $n+e$ 个单元                                                    |
| 时间           | 求某个顶点 $v_i$ 的度        | 扫描邻接矩阵中序号 $i$ 对应的一行, $O(1)$     | 求出度: 扫描矩阵的一行, $O(n)$ ;<br>求入度: 扫描矩阵的一列, $O(n)$ | 扫描 $v_i$ 的边表, 最坏情况 $O(n)$ | 求出度: 扫描 $v_i$ 的边表, 最坏情况 $O(n)$ ;<br>求入度: 按顶点表顺序扫描所有边表, $O(n+e)$ |
|              | 求边的数目                 | 扫描邻接矩阵, $O(n^2)$                |                                                | 按顶点表顺序扫描所有边表, $O(n+2e)$   | 按顶点表顺序扫描所有边表, $O(n+e)$                                          |
|              | 判定边 $(v_i, v_j)$ 是否存在 | 直接检查邻接矩阵 $A[i][j]$ 元素的值, $O(1)$ |                                                | 扫描 $v_i$ 的边表, 最坏情况 $O(n)$ |                                                                 |
| 适用情况         |                       | 稠密图                             |                                                | 稀疏图                       |                                                                 |

(3) 图的遍历算法是实现图的其他运算的基础, 图的遍历方法有两种: 深度优先搜索遍历和广度优先搜索遍历。深度优先搜索遍历类似于树的先序遍历, 借助于栈结构来实现(递归); 广度优先搜索遍历类似于树的层次遍历, 借助于队列结构来实现。两种遍历方法的不同之处仅仅在于对顶点访问的顺序不同, 所以时间复杂度相同。当用邻接矩阵存储时, 时间复杂度为均  $O(n^2)$ , 用邻接表存储时, 时间复杂度均为  $O(n+e)$ 。

(4) 图的很多算法与实际应用密切相关, 比较常用的算法包括构造最小生成树算法、求解最短路径算法、拓扑排序和求解关键路径算法。

① 构造最小生成树有普里姆算法和克鲁斯卡尔算法, 两者都能达到同一目的。但前者算法思想的核心是归并点, 时间复杂度是  $O(n^2)$ , 适用于稠密网; 后者是归并边, 时间复杂度是  $O(e \log_2 e)$ , 适用于稀疏网。

② 最短路径算法: 一种是迪杰斯特拉算法, 求从某个源点到其余各顶点的最短路径, 求解过程是按路径长度递增的次序产生最短路径, 时间复杂度是  $O(n^2)$ ; 另一种是弗洛伊德算法, 求每一对顶点之间的最短路径, 时间复杂度是  $O(n^3)$ , 从实现形式上来说, 这种算法比以图中的每个顶点为源点  $n$  次调用迪杰斯特拉算法更为简洁。

③ 拓扑排序和关键路径都是有向无环图的应用。拓扑排序是基于用顶点表示活动的有向图，即 AOV-网。对于不存在环的有向图，图中所有顶点一定能够排成一个线性序列，即拓扑序列，拓扑序列是不唯一的。用邻接表表示图，拓扑排序的时间复杂度为  $O(n+e)$ 。

④ 关键路径算法是基于用弧表示活动的有向图，即 AOE-网。关键路径上的活动叫做关键活动，这些活动是影响工程进度的关键，它们的提前或拖延将使整个工程提前或拖延。关键路径是不唯一的。关键路径算法的实现是在拓扑排序的基础上，用邻接表表示图，关键路径算法的时间复杂度为  $O(n+e)$ 。

学习完本章后，要求掌握图的基本概念和术语，掌握图的 4 种存储表示，明确各自的特点和适用场合，熟练掌握图的两种遍历算法，熟练掌握图在实际应用中的主要算法：最小生成树算法、最短路径算法、拓扑排序和关键路径算法。

## 习 题

### 1. 选择题

- (1) 在一个无向图中，所有顶点的度数之和等于图的边数的（ ）倍。  
A. 1/2                      B. 1                      C. 2                      D. 4
- (2) 在一个有向图中，所有顶点的入度之和等于所有顶点的出度之和的（ ）倍。  
A. 1/2                      B. 1                      C. 2                      D. 4
- (3) 具有  $n$  个顶点的有向图最多有（ ）条边。  
A.  $n$                       B.  $n(n-1)$                       C.  $n(n+1)$                       D.  $n^2$
- (4)  $n$  个顶点的连通图用邻接矩阵表示时，该矩阵至少有（ ）个非零元素。  
A.  $n$                       B.  $2(n-1)$                       C.  $n/2$                       D.  $n^2$
- (5)  $G$  是一个非连通无向图，共有 28 条边，则该图至少有（ ）个顶点。  
A. 7                      B. 8                      C. 9                      D. 10
- (6) 若从无向图的任意一个顶点出发进行一次深度优先搜索可以访问图中所有的顶点，则该图一定是（ ）图。  
A. 非连通                      B. 连通                      C. 强连通                      D. 有向
- (7) 下面（ ）适合构造一个稠密图  $G$  的最小生成树。  
A. Prim 算法                      B. Kruskal 算法                      C. Floyd 算法                      D. Dijkstra 算法
- (8) 用邻接表表示图进行广度优先遍历时，通常可借助（ ）来实现算法。  
A. 栈                      B. 队列                      C. 树                      D. 图
- (9) 用邻接表表示图进行深度优先遍历时，通常可借助（ ）来实现算法。  
A. 栈                      B. 队列                      C. 树                      D. 图
- (10) 图的深度优先遍历类似于二叉树的（ ）。  
A. 先序遍历                      B. 中序遍历                      C. 后序遍历                      D. 层次遍历
- (11) 图的广度优先遍历类似于二叉树的（ ）。  
A. 先序遍历                      B. 中序遍历                      C. 后序遍历                      D. 层次遍历
- (12) 图的 BFS 生成树的树高比 DFS 生成树的树高（ ）。  
A. 小                      B. 大                      C. 小或相等                      D. 大或相等

(13) 已知图的邻接矩阵如图 6.30 所示, 则从顶点  $v_0$  出发按深度优先遍历的结果是 ( )。

|       |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|
| $v_0$ | 0 | 1 | 1 | 1 | 1 | 0 | 1 |
| $v_1$ | 1 | 0 | 0 | 1 | 0 | 0 | 1 |
| $v_2$ | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| $v_3$ | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| $v_4$ | 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| $v_5$ | 0 | 0 | 0 | 1 | 1 | 0 | 1 |
| $v_6$ | 1 | 1 | 0 | 0 | 0 | 1 | 0 |

- A. 0 2 4 3 1 5 6  
B. 0 1 3 6 5 4 2  
C. 0 1 3 4 2 5 6  
D. 0 3 6 1 5 4 2

图 6.30 邻接矩阵

(14) 已知图的邻接表如图 6.31 所示, 则从顶点  $v_0$  出发按广度优先遍历的结果是 ( ), 按深度优先遍历的结果是 ( )。

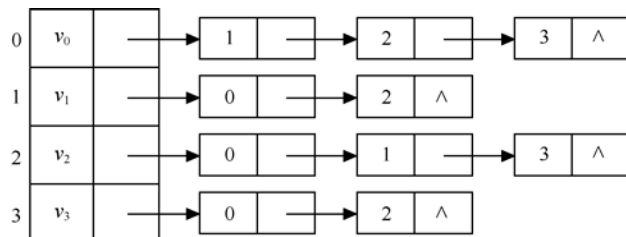


图 6.31 邻接表

- A. 0 1 3 2      B. 0 2 3 1      C. 0 3 2 1      D. 0 1 2 3
- (15) 下面的 ( ) 方法可以判断出一个有向图是否有环。
- A. 求最小生成树      B. 拓扑排序      C. 求最短路径      D. 求关键路径

## 2. 应用题

(1) 已知图 6.32 所示的有向图, 请给出:

- ① 每个顶点的入度和出度;
- ② 邻接矩阵;
- ③ 邻接表;
- ④ 逆邻接表。

(2) 已知如图 6.33 所示的无向网, 请给出:

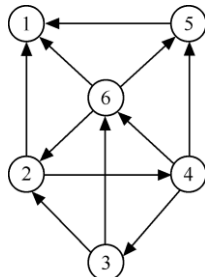


图 6.32 有向图

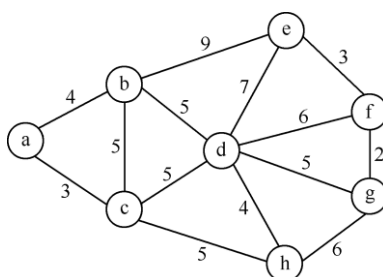


图 6.33 无向网

- ① 邻接矩阵;
- ② 邻接表;
- ③ 最小生成树。

(3) 已知图的邻接矩阵如图 6.34 所示。试分别画出自顶点 1 出发进行遍历所得的深度优先生成树和广度优先生成树。

(4) 有向网如图 6.35 所示，试用迪杰斯特拉算法求出从顶点 a 到其他各顶点间的最短路径，完成表 6.9。

|    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|----|---|---|---|---|---|---|---|---|---|----|
| 1  | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0  |
| 2  | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0  |
| 3  | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0  |
| 4  | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0  |
| 5  | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1  |
| 6  | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |
| 7  | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1  |
| 8  | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0  |
| 9  | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1  |
| 10 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0  |

图 6.34 邻接矩阵

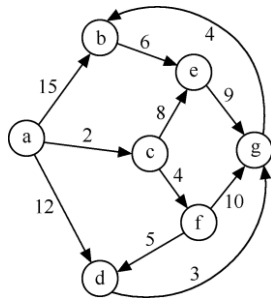


图 6.35 有向网

表 6.9

最短路径的求解

| D \ 终点 | $i=1$             | $i=2$ | $i=3$ | $i=4$ | $i=5$ | $i=6$ |
|--------|-------------------|-------|-------|-------|-------|-------|
| b      | 15<br>(a,b)       |       |       |       |       |       |
| c      | <u>2</u><br>(a,c) |       |       |       |       |       |
| d      | 12<br>(a,d)       |       |       |       |       |       |
| e      | $\infty$          |       |       |       |       |       |
| f      | $\infty$          |       |       |       |       |       |
| g      | $\infty$          |       |       |       |       |       |
| S 终点集  | {a,c}             |       |       |       |       |       |

(5) 试对图 6.36 所示的 AOE-网：

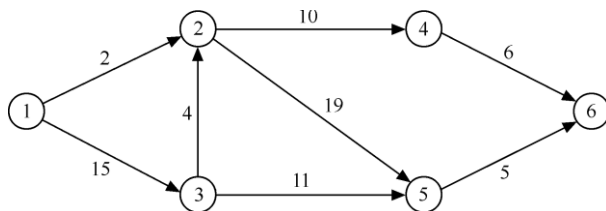


图 6.36 AOE-网

- ① 求这个工程最早可能在什么时间结束；
- ② 求每个活动的最早开始时间和最迟开始时间；
- ③ 确定哪些活动是关键活动。

### 3. 算法设计题

(1) 分别以邻接矩阵和邻接表作为存储结构, 实现以下图的基本操作:

- ① 增加一个新顶点  $v$ ,  $\text{InsertVex}(G, v)$ ;
- ② 删除顶点  $v$  及其相关的边,  $\text{DeleteVex}(G, v)$ ;
- ③ 增加一条边  $\langle v, w \rangle$ ,  $\text{InsertArc}(G, v, w)$ ;
- ④ 删除一条边  $\langle v, w \rangle$ ,  $\text{DeleteArc}(G, v, w)$ 。

(2) 一个连通图采用邻接表作为存储结构, 设计一个算法, 实现从顶点  $v$  出发的深度优先遍历的非递归过程。

(3) 设计一个算法, 求图  $G$  中距离顶点  $v$  的最短路径长度最大的一个顶点, 设  $v$  可达其余各个顶点。

(4) 试基于图的深度优先搜索策略写一算法, 判别以邻接表方式存储的有向图中是否存在由顶点  $v_i$  到顶点  $v_j$  的路径 ( $i \neq j$ )。

(5) 采用邻接表存储结构, 编写一个算法, 判别无向图中任意给定的两个顶点之间是否存在一条长度为  $k$  的简单路径。

# 第7章

## 查找

本书前几章介绍了各种线性和非线性的数据结构，并讨论了这些数据结构的相应运算。而在实际应用中，查找运算是非常常见的。面向一些数据量很大的实时系统，如订票系统、互联网上的信息检索系统等，查找效率尤其重要。本章将针对查找运算，讨论应该采用何种数据结构，使用什么样的方法，并通过对它们的效率进行分析来比较各种查找算法在不同情况下的优劣。

### 7.1 查找的基本概念

为了便于后面各节对各种查找算法的比较，首先介绍查找的概念和术语。

#### (1) 查找表

**查找表**是由同一类型的数据元素（或记录）构成的集合。由于“集合”中的数据元素之间存在着完全松散的关系，因此查找表是一种非常灵便的数据结构，可以利用其他的数据结构来实现，比如本章将要介绍的线性表、树表及散列表等。

#### (2) 关键字

**关键字**是数据元素（或记录）中某个数据项的值，用它标识一个数据元素（或记录）。若此关键字可以唯一地标识一个记录，则称此关键字为**主关键字**（对不同的记录，其主关键字均不同）。反之，称用以识别若干记录的关键字为**次关键字**。当数据元素只有一个数据项时，其关键字即为该数据元素的值。

#### (3) 查找

**查找**是指根据给定的某个值，在查找表中确定一个其关键字等于给定值的记录或数据元素。若表中存在这样的一个记录，则称**查找成功**，此时查找的结果可给出整个记录的信息，或指示该记录在查找表中的位置；若表中不存在关键字等于给定值的记录，则称**查找不成功**，此时查找的结果可给出一个“空”记录或“空”指针。

#### (4) 动态查找表和静态查找表

若在查找的同时对表做修改操作（如插入和删除），则相应的表称之为**动态查找表**，否则称之为**静态查找表**。换句话说，动态查找表的表结构本身是在查找过程中动态生成的，即在创建表时，对于给定值，若表中存在其关键字等于给定值的记录，则查找成功返回；否则插入关键字等于给定值的记录。

#### (5) 平均查找长度

为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法

在查找成功时的平均查找长度（Average Search Length, ASL）。

对于含有  $n$  个记录的表，查找成功时的平均查找长度为

$$ASL = \sum_{i=1}^n P_i C_i \quad (7-1)$$

其中， $P_i$  为查找表中第  $i$  个记录的概率，且  $\sum_{i=1}^n P_i = 1$ ；

$C_i$  为找到表中其关键字与给定值相等的第  $i$  个记录时，和给定值已进行过比较的关键字数。显然， $C_i$  随查找过程不同而不同。

由于查找算法的基本运算是关键字之间的比较操作，所以可用平均查找长度来衡量查找算法的性能。

## 7.2 线性表的查找

在查找表的组织方式中，线性表是最简单的一种。本节将介绍基于线性表的顺序查找、折半查找和分块查找。

### 7.2.1 顺序查找

**顺序查找**（Sequential Search）的查找过程为：从表的一端开始，依次将记录的关键字和给定值进行比较，若某个记录的关键字和给定值相等，则查找成功；反之，若扫描整个表后，仍未找到关键字和给定值相等的记录，则查找失败。

顺序查找方法既适用于线性表的顺序存储结构，又适用于线性表的链式存储结构。下面只介绍以顺序表作为存储结构时实现的顺序查找算法。

数据元素类型定义如下：

```
typedef struct{
    KeyType key;           //关键字域
    InfoType otherinfo;    //其他域
}ElemType;
```

顺序表的定义同第 2 章：

```
typedef struct{
    ElemType *R;           //存储空间基地址
    int length;            //当前长度
}SSTable;
```

在此定义下，顺序查找算法便与第 2 章的算法 2.2 一样。在此假设元素从 ST.R[1] 开始顺序向后存放，ST.R[0] 闲置不用，查找时从表的最后开始比较，如算法 7.1 所示。

#### 算法 7.1 顺序查找

【算法描述】

```
int Search_Seq(SSTable ST, KeyType key)
{ //在顺序表 ST 中顺序查找其关键字等于 key 的数据元素。若找到，则函数值为该元素在表中的位置，否则为 0
    for (i=ST.length; i>=1; --i)
        if (ST.R[i].key==key) return i;           //从后往前找
```

```

    return 0;
}

```

算法 7.1 在查找过程中每步都要检测整个表是否查找完毕, 即每步都要有循环变量是否满足条件  $i \geq 1$  的检测。改进这个程序, 可以免去这个检测过程。改进方法是查找之前先对  $ST.R[0]$  的关键字赋值  $key$ , 在此,  $ST.R[0]$  起到了监视哨的作用, 如算法 7.2 所示。

#### 算法 7.2 设置监视哨的顺序查找

##### 【算法描述】

```

int Search_Seq(SSTable ST,KeyType key)
{ //在顺序表 ST 中顺序查找其关键字等于 key 的数据元素。若找到, 则函数值为该元素在表中的位置, 否则为 0
    ST.R[0].key=key;                                // “哨兵”
    for(i=ST.length;ST.R[i].key!=key;--i);          // 从后往前找
    return i;
}

```

##### 【算法分析】

因此, 算法 7.2 仅是一个程序设计技巧上的改进, 即通过设置监视哨, 免去查找过程中每一步都要检测整个表是否查找完毕。然而实践证明, 这个改进能使顺序查找在  $ST.length \geq 1000$  时, 进行一次查找所需的平均时间几乎减少一半。当然, 监视哨也可设在高下标处。

算法 7.2 和算法 7.1 的时间复杂度一样, 在第 2 章已经做过分析, 即

$$ASL = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$$

算法 7.2 的时间复杂度为  $O(n)$ 。

顺序查找的优点是: 算法简单, 对表结构无任何要求, 既适用于顺序结构, 也适用于链式结构, 无论记录是否按关键字有序均可应用。其缺点是: 平均查找长度较大, 查找效率较低, 所以当  $n$  很大时, 不宜采用顺序查找。

## 7.2.2 折半查找

**折半查找** (Binary Search) 也称**二分查找**, 它是一种效率较高的查找方法。但是, 折半查找要求线性表必须采用顺序存储结构, 而且表中元素按关键字有序排列。在下面及后续的讨论中, 均假设有序表是递增有序的。

折半查找的查找过程为: 从表的中间记录开始, 如果给定值和中间记录的关键字相等, 则查找成功; 如果给定值大于或者小于中间记录的关键字, 则在表中大于或小于中间记录的那一半中查找, 这样重复操作, 直到查找成功, 或者在某一步中查找区间为空, 则代表查找失败。

折半查找每一次查找比较都使查找范围缩小一半, 与顺序查找相比, 很显然会提高查找效率。

为了标记查找过程中每一次的查找区间, 下面分别用  $low$  和  $high$  来表示当前查找区间的下界和上界,  $mid$  为区间的中间位置。

#### 算法 7.3 折半查找

##### 【算法步骤】

- ① 置查找区间初值,  $low$  为 1,  $high$  为表长。
- ② 当  $low$  小于等于  $high$  时, 循环执行以下操作:
  - $mid$  取值为  $low$  和  $high$  的中间值;

- 将给定值 `key` 与中间位置记录的关键字进行比较,若相等则查找成功,返回中间位置 `mid`;
  - 若不相等则利用中间位置记录将表对分成前、后两个子表。如果 `key` 比中间位置记录的关键字小,则 `high` 取为 `mid-1`, 否则 `low` 取为 `mid+1`。
- ③ 循环结束,说明查找区间为空,则查找失败,返回 0。

#### 【算法描述】

```
int Search_Bin(SSTable ST,KeyType key)
{ //在有序表 ST 中折半查找其关键字等于 key 的数据元素。若找到,则函数值为该元素在表中的位置,否则为 0
  low=1;high=ST.length;           //置查找区间初值
  while(low<=high)
  {
    mid=(low+high)/2;
    if(key==ST.R[mid].key) return mid;           //找到待查元素
    else if(key<ST.R[mid].key) high=mid-1;       //继续在前一子表进行查找
    else low=mid+1;                             //继续在后一子表进行查找
  }                                           //while
  return 0;                                   //表中不存在待查元素
}
```

本算法很容易理解,唯一需要注意的是,循环执行的条件是 `low<=high`, 而不是 `low<high`, 因为 `low=high` 时, 查找区间还有最后一个结点, 还要进一步比较。

算法 7.3 很容易改写成递归程序, 递归函数的参数除了 `ST` 和 `key` 之外, 还需要加上 `low` 和 `high`, 请读者自行实现折半查找的递归算法。

【例 7.1】 已知如下 11 个数据元素的有序表 (关键字即为数据元素的值):

(5, 16, 20, 27, 30, 36, 44, 55, 60, 67, 71)

请给出查找关键字为 27 和 65 的数据元素的折半查找过程。

假设指针 `low` 和 `high` 分别指示待查元素所在范围的下界和上界, 指针 `mid` 指示区间的中间位置, 即  $mid = \lfloor (low + high) / 2 \rfloor$ 。在此例中, `low` 和 `high` 的初值分别为 1 和 11, 即 [1,11] 为待查范围, `mid` 初值为 6。

查找关键字 `key = 27` 的折半查找过程如图 7.1 (a) 所示。

首先令给定值 `key=27` 与中间位置的数据元素的关键字 `ST.R[mid].key` 相比较, 因为  $36 > 27$ , 说明待查元素若存在, 必在区间  $[low, mid - 1]$  的范围内, 则令指针 `high` 指向第 `mid - 1` 个元素,  $high = 5$ , 重新求得  $mid = \lfloor (1 + 5) / 2 \rfloor = 3$ 。

然后仍以 `key` 和 `ST.R[mid].key` 相比, 因为  $20 < 27$ , 说明待查待元素若存在, 必在  $[mid + 1, high]$  范围内, 则令指针 `low` 指向第 `mid + 1` 个元素,  $low = 4$  求得 `mid` 的新值为 4, 比较 `key` 和 `ST.R[mid].key`, 因为相等, 则查找成功, 返回所查元素在表中的序号, 即指针 `mid` 的值 4。

查找关键字 `key = 65` 的折半查找过程如图 7.1 (b) 所示。

查找过程同上, 只是在图 7.1 (b) 中的最后一趟查找时, 因为 `low > high`, 查找区间不存在, 则说明表中没有关键字等于 65 的元素, 查找失败, 返回 0。

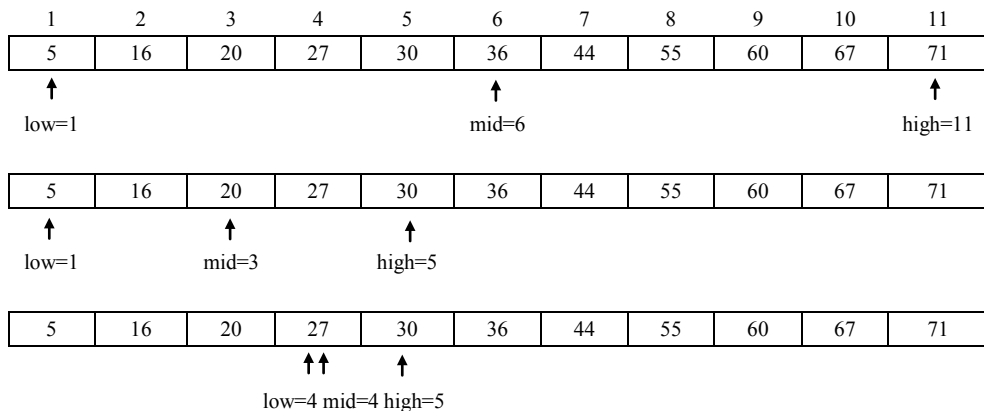
#### 【算法分析】

折半查找过程可用二叉树来描述。树中每一结点对应表中一个记录, 但结点值不是记录的关键字, 而是记录在表中的位置序号。把当前查找区间的中间位置作为根, 左子表和右子表分别作为根的左子树和右子树, 由此得到的二叉树称为折半查找的判定树。

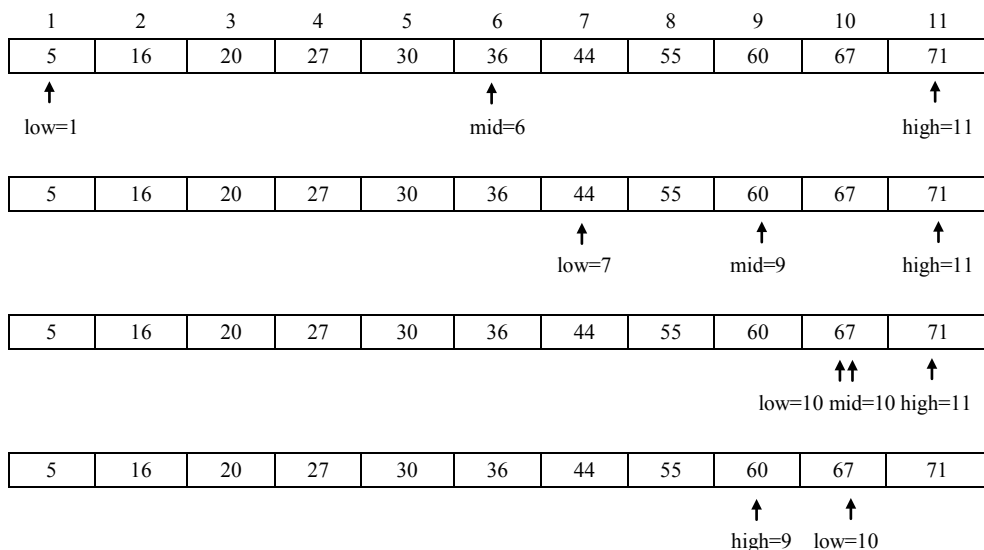
例 7.1 中的有序表对应的判定树如图 7.2 所示。从判定树上可见, 成功的折半查找恰好是走了一

条从判定树的根到被查结点的路径, 经历比较的关键字个数恰为该结点在树中的层次。例如, 查找 27 的过程经过一条从根到结点④的路径, 需要比较 3 次, 比较次数即为结点④所在的层次。图 7.2 中比较 1 次的只有一个根结点, 比较 2 次的有两个结点, 比较 3 次和 4 次的各有四个结点。假设每个记录的查找概率相同, 根据此判定树可知, 对长度为 11 的有序表进行折半查找的平均查找长度为

$$ASL = \frac{1}{11}(1 + 2 \times 2 + 3 \times 4 + 4 \times 4) = 3$$



(a) 查找 27 的过程



(b) 查找 65 的过程

图 7.1 折半查找示意图

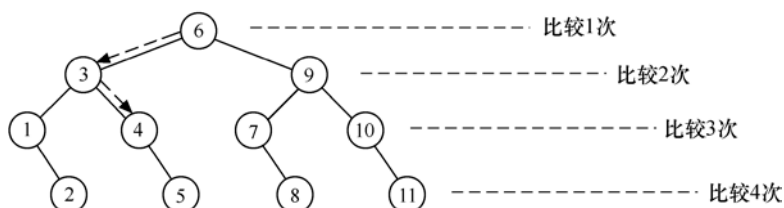


图 7.2 折半查找过程的判定树及查找 27 的过程

由此可见, 折半查找法在查找成功时进行比较的关键字个数最多不超过树的深度。而判定树的形态只与表记录个数  $n$  相关, 而与关键字的取值无关, 具有  $n$  个结点的判定树的深度为  $\lfloor \log_2 n \rfloor + 1$ 。所以, 对于长度为  $n$  的有序表, 折半查找法在查找成功时和给定值进行比较的关键字个数至多为  $\lfloor \log_2 n \rfloor + 1$ 。

如果在图 7.2 所示的判定树中所有结点的空指针域上加一个指向一个方形结点的指针, 如图 7.3 所示。并且, 称这些方形结点为判定树的外部结点 (与之相对, 称那些圆形结点为内部结点), 那么折半查找时查找失败的过程就是走了一条从根结点到外部结点的路径, 和给定值进行比较的关键字个数等于该路径上内部结点个数。例如, 查找 65 的过程即为走了一条从根到结点 9~10 的路径。因此, 折半查找在查找不成功时和给定值进行比较的关键字个数最多也不超过  $\lfloor \log_2 n \rfloor + 1$ 。

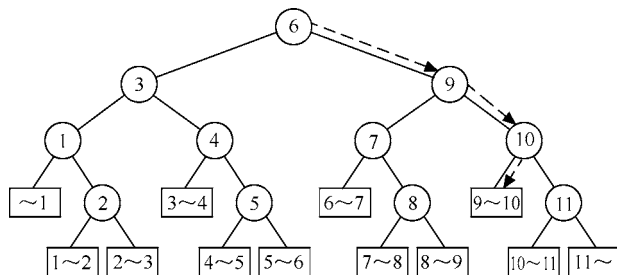


图 7.3 加上外部结点的判定树和查找 65 的过程

借助于判定树, 很容易求得折半查找的平均查找长度。为了讨论方便起见, 假定有序表的长度  $n = 2^h - 1$ , 则判定树是深度为  $h = \log_2(n + 1)$  的满二叉树。树中层次为 1 的结点有 1 个, 层次为 2 的结点有 2 个, ..., 层次为  $h$  的结点有  $2^{h-1}$  个。假设表中每个记录的查找概率相等  $\left(P_i = \frac{1}{n}\right)$ , 则查找成功时折半查找的平均查找长度为

$$\begin{aligned}
 ASL &= \sum_{i=1}^n P_i C_i \\
 &= \frac{1}{n} \sum_{j=1}^h j \cdot 2^{j-1} \\
 &= \frac{n+1}{n} \log_2(n+1) - 1
 \end{aligned} \tag{7-2}$$

当  $n$  较大时, 可有下列近似结果

$$ASL = \log_2(n + 1) - 1 \tag{7-3}$$

因此, 折半查找的时间复杂度为  $O(\log_2 n)$ 。可见, 折半查找的效率比顺序查找高, 但折半查找只适用于有序表, 且限于顺序存储结构。

折半查找的优点是: 比较次数少, 查找效率高。其缺点是: 对表结构要求高, 只能用于顺序存储的有序表。查找前需要排序, 而排序本身是一种费时的运算。同时为了保持顺序表的有序性, 对有序表进行插入和删除时, 平均比较和移动表中一半元素, 这也是一种费时的运算。因此, 折半查找不适用于数据元素经常变动的线性表。

## 7.2.3 分块查找

**分块查找** (Blocking Search) 又称**索引顺序查找**, 这是一种性能介于顺序查找和折半查找之间的一种查找方法。在此查找法中, 除表本身以外, 尚需建立一个“索引表”。例如, 图 7.4 所示为一个表及其索引表, 表中含有 18 个记录, 可分成 3 个子表 ( $R_1, R_2, \dots, R_6$ )、( $R_7, R_8, \dots, R_{12}$ )、( $R_{13}, R_{14}, \dots, R_{18}$ ), 对每个子表 (或称块) 建立一个索引项, 其中包括两项内容: 关键字项 (其值为该子表内的最大关键字) 和指针项 (指示该子表的第一个记录在表中位置)。索引表按关键字有序, 则表或者有序或者分块有序。所谓“分块有序”指的是第二个子表中所有记录的关键字均大于第一个子表中的最大关键字, 第三个子表中的所有关键字均大于第二个子表中的最大关键字, …… , 依次类推。

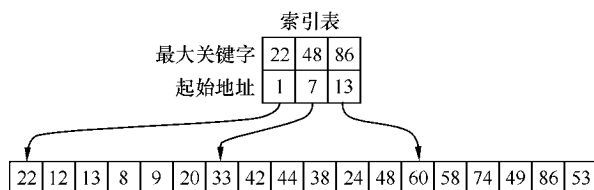


图 7.4 表及其索引表

因此, 分块查找过程需分两步进行。先确定待查记录所在的块 (子表), 然后在块中顺序查找。假设给定值  $key = 38$ , 则先将  $key$  依次和索引表中各最大关键字进行比较, 因为  $22 < key < 48$ , 则关键字为 38 的记录若存在, 必定在第二个子表中, 由于同一索引项中的指针指示第二个子表中的第一个记录是表中第 7 个记录, 则自第 7 个记录起进行顺序查找, 直到  $ST.elem[10].key = key$  为止。假如此子表中没有关键字等于  $key$  的记录 (例如:  $key = 29$  时自第 7 个记录起至第 12 个记录的关键字和  $key$  比较都不等), 则查找不成功。

由于由索引项组成的索引表按关键字有序, 则确定块的查找可以用顺序查找, 亦可用折半查找, 而块中记录是任意排列的, 则在块中只能是顺序查找。

由此, 分块查找的算法为顺序查找和折半查找两种算法的简单合成。

分块查找的平均查找长度为

$$ASL_{bs} = L_b + L_w \quad (7-4)$$

其中,  $L_b$  为查找索引表确定所在块的平均查找长度,  $L_w$  为在块中查找元素的平均查找长度。

一般情况下, 为进行分块查找, 可以将长度为  $n$  的表均匀地分成  $b$  块, 每块含有  $s$  个记录, 即  $b = \lceil n/s \rceil$ ; 又假定表中每个记录的查找概率相等, 则每块查找的概率为  $1/b$ , 块中每个记录的查找概率为  $1/s$ 。

若用顺序查找确定所在块, 则分块查找的平均查找长度为

$$\begin{aligned}
 ASL_{bs} = L_b + L_w &= \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{b+1}{2} + \frac{s+1}{2} \\
 &= \frac{1}{2} \left( \frac{n}{s} + s \right) + 1
 \end{aligned} \quad (7-5)$$

可见, 此时的平均查找长度不仅和表长  $n$  有关, 而且和每一块中的记录个数  $s$  有关。在给定  $n$  的前提下,  $s$  是可以选择的。容易证明, 当  $s$  取  $\sqrt{n}$  时,  $ASL_{bs}$  取最小值  $\sqrt{n} + 1$ 。这个值比顺序查找有了很大改进, 但远不及折半查找。

若用折半查找确定所在块, 则分块查找的平均查找长度为

$$ASL'_{bs} \approx \log_2 \left( \frac{n}{s} + 1 \right) + \frac{s}{2} \quad (7-6)$$

分块查找的优点是: 在表中插入和删除数据元素时, 只要找到该元素对应的块, 就可以在该块内进行插入和删除运算。由于块内是无序的, 故插入和删除比较容易, 无需进行大量移动。如果线性表既要快速查找又经常动态变化, 则可采用分块查找。其缺点是: 要增加一个索引表的存储空间并对初始索引表进行排序运算。

## 7.3 树表的查找

前面介绍的 3 种查找方法都是用线性表作为查找表的组织形式, 其中折半查找效率较高。但由于折半查找要求表中记录按关键字有序排列, 且不能用链表做存储结构, 因此, 当表的插入或删除操作频繁时, 为维护表的有序性, 需要移动表中很多记录。这种由移动记录引起的额外时间开销, 就会抵消折半查找的优点。所以, 线性表的查找更适用于静态查找表, 若要对动态查找表进行高效率的查找, 可采用几种特殊的二叉树作为查找表的组织形式, 在此将它们统称为树表。本节将介绍在这些树表上进行查找和修改操作的方法。

### 7.3.1 二叉排序树

**二叉排序树** (Binary Sort Tree) 又称**二叉查找树**, 它是一种对排序和查找都很有用的特殊二叉树。

#### 1. 二叉排序树的定义

二叉排序树或者是一棵空树, 或者是具有下列性质的二叉树:

- (1) 若它的左子树不空, 则左子树上所有结点的值均小于它的根结点的值;
- (2) 若它的右子树不空, 则右子树上所有结点的值均大于它的根结点的值;
- (3) 它的左、右子树也分别为二叉排序树。

二叉排序树是递归定义的。由定义可以得出二叉排序树的一个重要性质: 中序遍历一棵二叉树时可以得到一个结点值递增的有序序列。

例如, 图 7.5 所示为两棵二叉排序树。

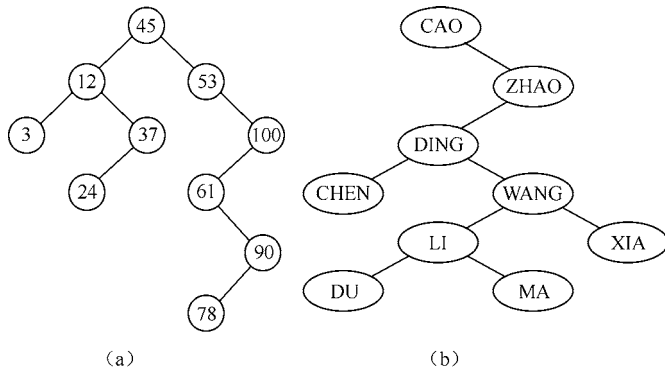


图 7.5 二叉排序树示例

若中序遍历图 7.5 (a), 则可得到一个按数值大小排序的递增序列:

3, 12, 24, 37, 45, 53, 61, 78, 90, 100

若中序遍历图 7.5 (b), 则可得到一个按字符大小排序的递增序列:

CAO, CHEN, DING, DU, LI, MA, WANG, XIA, ZHAO

在下面讨论二叉排序树的操作中, 使用二叉链表作为存储结构。因为二叉排序树的操作要根据结点的关键字域来进行, 所以下面给出了每个结点的数据域的类型定义 (包括关键字项和其他数据项)。

```
// - - - - 二叉排序树的二叉链表存储表示 - - - -
typedef struct
{
    KeyType key;                // 关键字项
    InfoType otherinfo;        // 其他数据项
} ElemType;                   // 每个结点的数据域的类型
typedef struct BSTNode
{
    ElemType data;              // 每个结点的数据域包括关键字项和其他数据项
    struct BSTNode *lchild, *rchild; // 左右孩子指针
} BSTNode, *BSTree;
```

## 2. 二叉排序树的查找

因为二叉排序树可以看成是一个有序表, 所以在二叉排序树上进行查找和折半查找类似, 也是一个逐步缩小查找范围的过程。

### 算法 7.4 二叉排序树的递归查找

#### 【算法步骤】

- ① 若二叉排序树为空, 则查找失败, 返回空指针。
- ② 若二叉排序树非空, 将给定值 `key` 与根结点的关键字 `T->data.key` 进行比较:
  - 若 `key` 等于 `T->data.key`, 则查找成功, 返回根结点地址;
  - 若 `key` 小于 `T->data.key`, 则递归查找左子树;
  - 若 `key` 大于 `T->data.key`, 则递归查找右子树。

模仿折半查找算法 7.3, 读者很容易写出二叉排序树查找的非递归算法。下面以递归形式给出此查找算法。

#### 【算法描述】

```
BSTree SearchBST(BSTree T, KeyType key)
{ // 在根指针 T 所指二叉排序树中递归地查找某关键字等于 key 的数据元素
  // 若查找成功, 则返回指向该数据元素结点的指针, 否则返回空指针
  if ((!T) || key == T->data.key) return T; // 查找结束
  else if (key < T->data.key) return SearchBST(T->lchild, key); // 在左子树中继续查找
  else return SearchBST(T->rchild, key); // 在右子树中继续查找
}
```

例如, 在图 7.5 (a) 所示的二叉排序树中查找关键字等于 100 的记录 (树中结点内的数均为记录的关键字)。首先以 `key = 100` 和根结点的关键字做比较, 因为 `key > 45`, 则查找以④为根的右子树, 此时右子树不空, 且 `key > 53`, 则继续查找以结点③为根的右子树, 由于 `key` 和③的右子树根的关键字 100 相等, 则查找成功, 返回指向结点⑩的指针值。又如在图 7.5 (a) 中

查找关键字等于 40 的记录, 和上述过程类似, 在给定值 key 与关键字 45、12 及 37 相继比较之后, 继续查找以结点 37 为根的右子树, 此时右子树为空, 则说明该树中没有待查记录, 故查找不成功, 返回指针值为 “NULL”。

### 【算法分析】

从上述的两个查找例子 (key = 100 和 key = 40) 可见, 在二叉排序树上查找其关键字等于给定值的结点的过程, 恰是走了一条从根结点到该结点的路径的过程, 和给定值比较的关键字个数等于路径长度加 1 (或结点所在层次数)。因此, 和折半查找类似, 与给定值比较的关键字个数不超过树的深度。然而, 折半查找长度为  $n$  的顺序表的判定树是唯一的, 而含有  $n$  个结点的二叉排序树却不唯一。图 7.6 中 (a) 和 (b) 的两棵二叉排序树中结点的值都相同, 但创建这两棵树的序列不同, 分别是: (45, 24, 53, 12, 37, 93) 和 (12, 24, 37, 45, 53, 93)。(a) 树的深度为 3, 而 (b) 树的深度为 6。再从平均查找长度来看, 假设 6 个记录的查找概率相等, 为  $1/6$ , 则 (a) 树的平均查找长度为

$$ASL_{(a)} = \frac{1}{6} [1 + 2 + 2 + 3 + 3 + 3] = 1.46$$

而 (b) 树的平均查找长度为

$$ASL_{(b)} = \frac{1}{6} [1 + 2 + 3 + 4 + 5 + 6] = 2.75$$

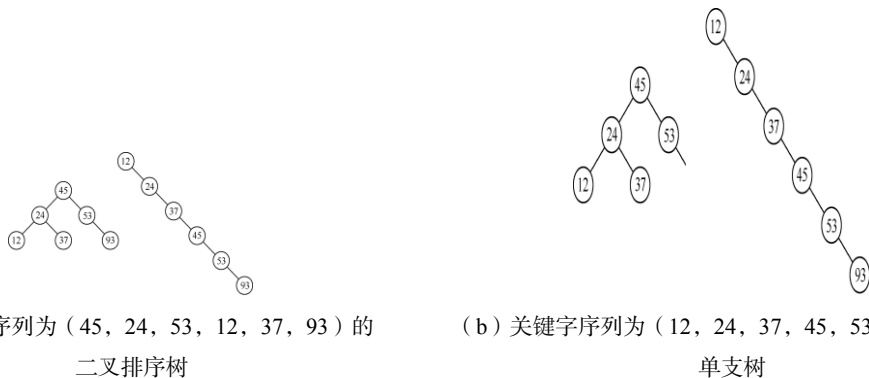


图 7.6 不同形态的二叉排序树

因此, 含有  $n$  个结点的二叉排序树的平均查找长度和树的形态有关。当先后插入的关键字有序时, 构成的二叉排序树蜕变为单支树。树的深度为  $n$ , 其平均查找长度为  $\frac{n+1}{2}$  (和顺序查找相同), 这是最差的情况。显然, 最好的情况是, 二叉排序树的形态和折半查找的判定树相似, 其平均查找长度和  $\log_2 n$  成正比。若考虑把  $n$  个结点按各种可能的次序插入到二叉排序树中, 则有  $n!$  棵二叉排序树 (其中有的形态相同)。可以证明, 综合所有可能的情况, 就平均而言, 二叉排序树的平均查找长度仍然和  $\log_2 n$  是同数量级的。

可见, 二叉排序树上的查找和折半查找相差不大。但就维护表的有序性而言, 二叉排序树更加有效, 因为无需移动记录, 只需修改指针即可完成对结点的插入和删除操作。因此, 对于需要经常进行插入、删除和查找运算的表, 采用二叉排序树比较好。

### 3. 二叉排序树的插入

二叉排序树的插入操作是以查找为基础的。要将一个关键字值为 key 的结点 \*S 插入到二叉排序

树中,则需要从根结点向下查找,当树中不存在关键字等于 key 的结点时才进行插入。新插入的结点一定是一个新添加的叶子结点,并且是查找不成功时查找路径上访问的最后一个结点的左孩子或右孩子结点。

### 算法 7.5 二叉排序树的插入

#### 【算法步骤】

- ① 若二叉排序树为空,则待插入结点 \*S 作为根结点插入到空树中。
- ② 若二叉排序树非空,则将 key 与根结点的关键字  $T \rightarrow \text{data.key}$  进行比较:
  - 若 key 小于  $T \rightarrow \text{data.key}$ ,则将 \*S 插入左子树;
  - 若 key 大于  $T \rightarrow \text{data.key}$ ,则将 \*S 插入右子树。

#### 【算法描述】

```
void InsertBST(BSTree &T, ElemType e)
{ //当二叉排序树 T 中不存在关键字等于 e.key 的数据元素时,则插入该元素
  if (!T)
  {
    //找到插入位置,递归结束
    S=new BSTNode;           //生成新结点*S
    S->data=e;                //新结点*S 的数据域置为 e
    S->lchild=S->rchild=NULL; //新结点*S 作为叶子结点
    T=S;                     //把新结点*S 链接到已找到的插入位置
  }
  else if (e.key < T->data.key)
    InsertBST(T->lchild, e); //将*S 插入左子树
  else if (e.key > T->data.key)
    InsertBST(T->rchild, e); //将*S 插入右子树
}
```

例如,在图 7.5 (a) 所示的二叉排序树上插入关键字为 55 的结点,由于插入前二叉排序树非空,故将 55 和根结点 45 进行比较,因  $55 > 45$ ,则应将 55 插入到 45 的右子树上;又和 45 的右子树的根 53 比较,因  $55 > 53$ ,则应将 55 插入到 53 的右子树上;依次类推,直至最后  $55 < 61$ ,且 61 的左子树为空,将 55 作为 61 的左孩子插入到树中。结果如图 7.7 所示。

#### 【算法分析】

二叉排序树插入的基本过程是查找,所以时间复杂度同查找一样,是  $O(\log_2 n)$ 。

### 4. 二叉排序树的创建

二叉排序树的创建是从空的二叉排序树开始的,每输入一个结点,经过查找操作,将新结点插入到当前二叉排序树的合适位置。

### 算法 7.6 二叉排序树的创建

#### 【算法步骤】

- ① 将二叉排序树 T 初始化为空树。
- ② 读入一个关键字为 key 的结点。
- ③ 如果读入的关键字 key 不是输入结束标志,则循环执行以下操作:
  - 将此结点插入二叉排序树 T 中;

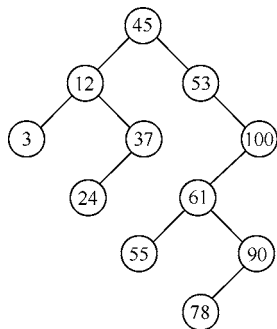


图 7.7 二叉排序树的插入

- 读入一个关键字为 key 的结点。

### 【算法描述】

```
void CreatBST(BSTree &T)
{ //依次读入一个关键字为 key 的结点, 将此结点插入二叉排序树 T 中
    T=NULL; //将二叉排序树 T 初始化为空树
    cin>>e;
    while(e.key!=ENDFLAG) //ENDFLAG 为自定义常量, 作为输入结束标志
    {
        InsertBST(T,e); //将此结点插入二叉排序树 T 中
        cin>>e;
    }
}
```

### 【算法分析】

假设有  $n$  个结点, 则需要  $n$  次插入操作, 而插入一个结点的算法时间复杂度为  $O(\log_2 n)$ , 所以创建二叉排序树算法的时间复杂度为  $O(n \log_2 n)$ 。

例如, 设关键字的输入次序为: 45, 24, 53, 45, 12, 24, 90, 按上述算法生成的二叉排序树的过程如图 7.8 所示。

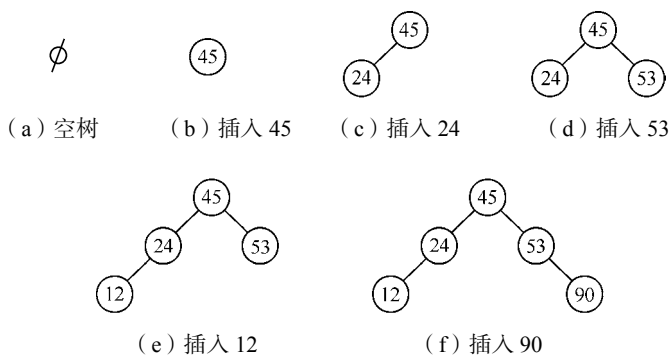


图 7.8 二叉排序树的创建过程

容易看出, 一个无序序列可以通过构造一棵二叉排序树而变成一个有序序列, 构造树的过程即为对无序序列进行排序的过程。不仅如此, 从上面的插入过程还可以看到, 每次插入的新结点都是二叉排序树上新的叶子结点, 则在进行插入操作时, 不必移动其他结点, 仅需改动某个结点的指针, 由空变为非空即可。这就相当于在一个有序序列上插入一个记录而不需要移动其他记录。

## 5. 二叉排序树的删除

被删除的结点可能是二叉排序树中的任何结点, 删除结点后, 要根据其位置不同修改其双亲结点及相关结点的指针, 以保持二叉排序树的特性。

### 算法 7.7 二叉排序树的删除

#### 【算法步骤】

首先从二叉排序树的根结点开始查找关键字为 key 的待删结点, 如果树中不存在此结点, 则不做任何操作; 否则, 假设被删结点为 \*p (指向结点的指针为 p), 其双亲结点为 \*f (指向结点的指针为 f),  $P_L$  和  $P_R$  分别表示其左子树和右子树 (见图 7.9 (a))。

不失一般性, 可设 \*p 是 \*f 的左孩子 (右孩子情况类似)。下面分 3 种情况进行讨论。

(1) 若 $*p$  结点为叶子结点, 即  $P_L$  和  $P_R$  均为空树。由于删去叶子结点不破坏整棵树的结构, 则只需修改其双亲结点的指针即可。

$f \rightarrow lchild = \text{NULL};$

(2) 若 $*p$  结点只有左子树  $P_L$  或者只有右子树  $P_R$ , 此时只要令  $P_L$  或  $P_R$  直接成为其双亲结点  $*f$  的左子树即可。

$f \rightarrow lchild = p \rightarrow lchild; \text{ (或 } f \rightarrow lchild = p \rightarrow rchild; \text{)}$

(3) 若 $*p$  结点的左子树和右子树均不空。从图 7.9 (b) 可知, 在删去 $*p$  结点之前, 中序遍历该二叉树得到的序列为  $\{\dots C_L C \dots Q_L Q S_L S P P_R F \dots\}$ , 在删去 $*p$  之后, 为保持其他元素之间的相对位置不变, 可以有两种处理方法:

① 令 $*p$  的左子树为 $*f$  的左子树, 而 $*p$  的右子树为 $*s$  的右子树, 如图 7.9 (c) 所示。

$f \rightarrow lchild = p \rightarrow lchild; s \rightarrow rchild = p \rightarrow rchild;$

② 令 $*p$  的直接前驱 (或直接后继) 替代 $*p$ , 然后再从二叉排序树中删去它的直接前驱 (或直接后继)。如图 7.9 (d) 所示, 当以直接前驱 $*s$  替代 $*p$  时, 由于 $*s$  只有左子树  $S_L$ , 则在删去 $*s$  之后, 只要令  $S_L$  为 $*s$  的双亲 $*q$  的右子树即可。

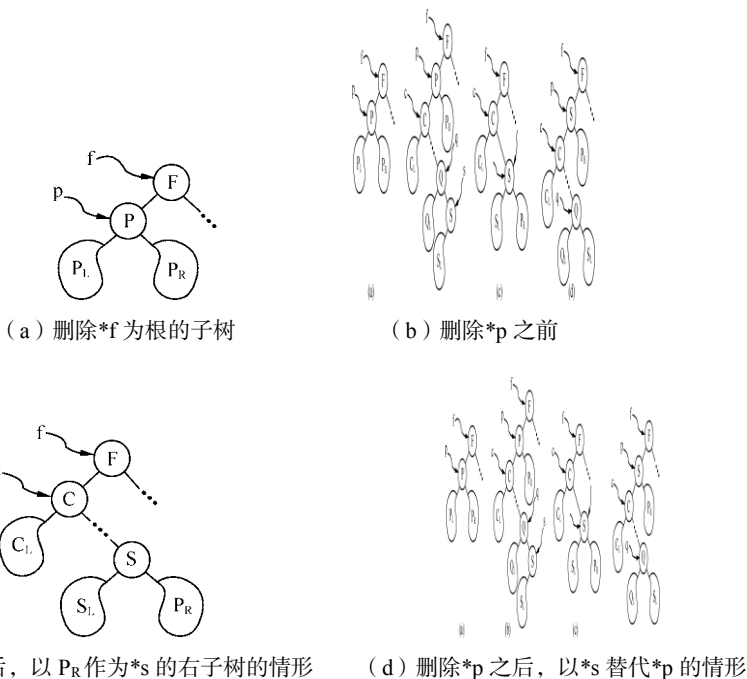


图 7.9 在二叉排序树中删除 $*p$

$p \rightarrow data = s \rightarrow data; q \rightarrow rchild = s \rightarrow lchild;$

显然, 前一种处理方法可能增加树的深度, 而后一种方法是以被删结点左子树中关键字最大的结点替代被删结点, 然后从左子树中删除这个结点。此结点一定没有右子树 (否则它就不是左子树中关键字最大的结点), 这样不会增加树的高度, 所以常采用这种处理方案。下面的算法描述即采用这种方案。

#### 【算法描述】

```
void DeleteBST(BSTree &T, KeyType key)
```

```

{ //从二叉排序树 T 中删除关键字等于 key 的结点

    p=T; f=NULL; //初始化

    /*-----下面的 while 循环从根开始查找关键字等于 key 的结点*p-----*/
    while (p)
    {
        if (p->data.key==key) break; //找到关键字等于 key 的结点*p, 结束循环
        f=p; // *f 为*p 的双亲结点
        if (p->data.key>key) p=p->lchild; //在*p 的左子树中继续查找
        else p=p->rchild; //在*p 的右子树中继续查找
    } //while
    if (!p) return; //找不到被删结点则返回

    /*----考虑 3 种情况实现 p 所指子树内部的处理: *p 左右子树均不空、无右子树、无左子树---*/
    if ((p->lchild) && (p->rchild)) //被删结点*p 左右子树均不空
    {
        q=p; s=p->lchild;
        while (s->rchild) //在*p 的左子树中继续查找其前驱结点, 即最右下结点
        {
            q=s; s=s->rchild; //向右到尽头
        }
        p->data=s->data; //s 指向被删结点的“前驱”
        if (q!=p) q->rchild=s->lchild; //重接*q 的右子树
        else q->lchild=s->lchild; //重接*q 的左子树
        delete s;
        return;
    } //if

    else if (!p->rchild) //被删结点*p 无右子树, 只需重接其左子树
    {
        q=p; p=p->lchild;
    } //else if

    else if (!p->lchild) //被删结点*p 无左子树, 只需重接其右子树
    {
        q=p; p=p->rchild;
    } //else if

    /*-----将 p 所指的子树挂接到其双亲结点*f 相应的位置-----*/
    if (!f) T=p; //被删结点为根结点
    else if (q==f->lchild) f->lchild=p; //挂接到*f 的左子树位置
    else f->rchild=p; //挂接到*f 的右子树位置
    delete q;
}

```

### 【算法分析】

同二叉排序树插入一样, 二叉排序树删除的基本过程也是查找, 所以时间复杂度仍是  $O(\log_2 n)$ 。

根据算法 7.7, 图 7.10 所示给出了二叉排序树删除的 3 种情况。

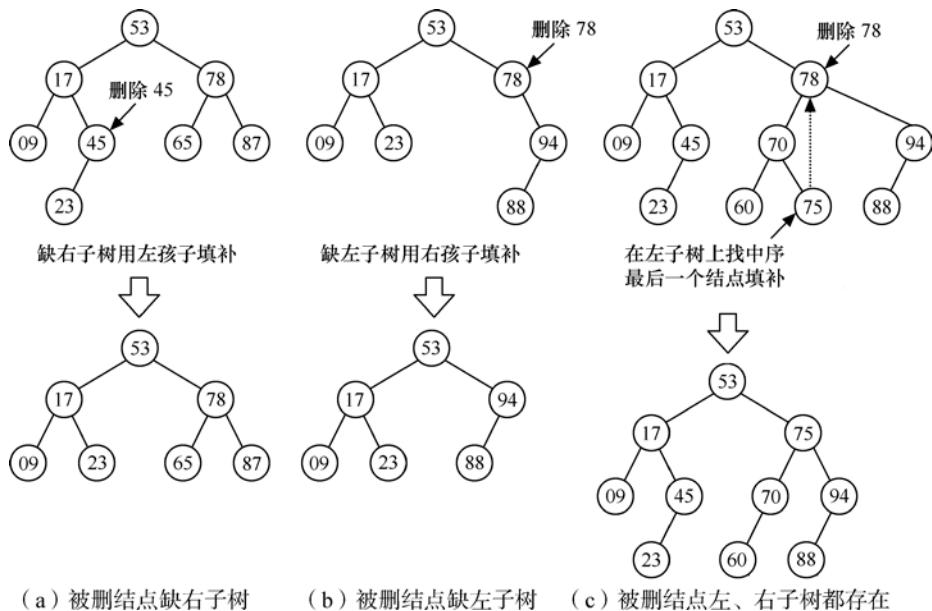


图 7.10 二叉排序树的删除

## 7.3.2 平衡二叉树

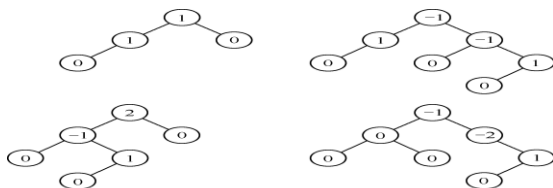
### 1. 平衡二叉树的定义

二叉排序树查找算法的性能取决于二叉树的结构，而二叉排序树的形状则取决于其数据集。如果数据呈有序排列，则二叉排序树是线性的，查找的时间复杂度为  $O(n)$ ；反之，如果二叉排序树的结构合理，则查找速度较快，查找的时间复杂度为  $O(\log_2 n)$ 。事实上，树的高度越小，查找速度越快。因此，希望二叉树的高度尽可能小。本节将讨论一种特殊类型的二叉排序树，称为平衡二叉树（Balanced Binary Tree 或 Height-Balanced Tree），因由前苏联数学家 Adelson-Velskii 和 Landis 提出，所以又称 AVL 树。

平衡二叉树或者是空树，或者是具有如下特征的二叉排序树：

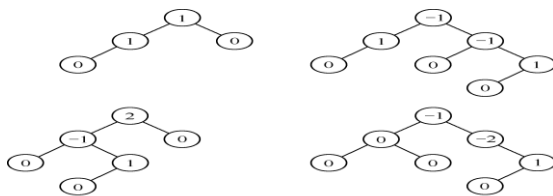
- (1) 左子树和右子树的深度之差的绝对值不超过 1；
- (2) 左子树和右子树也是平衡二叉树。

若将二叉树上结点的平衡因子（Balance Factor, BF）定义为该结点左子树和右子树的深度之差，则平衡二叉树上所有结点的平衡因子只可能是 -1、0 和 1。只要二叉树上有一个结点的平衡因子的绝对值大于 1，则该二叉树就是不平衡的。图 7.11 (a) 所示为两棵平衡二叉树，而图 7.11 (b) 所示为两棵不平衡的二叉树，结点中的值为该结点的平衡因子。



(a) 平衡二叉树

图 7.11 平衡与不平衡的二叉树及结点的平衡因子



(b) 不平衡的二叉树

图 7.11 平衡与不平衡的二叉树及结点的平衡因子 (续)

因为 AVL 树上任何结点的左右子树的深度之差都不超过 1, 则可以证明它的深度和  $\log_2 n$  是同数量级的 (其中  $n$  为结点个数)。由此, 其查找的时间复杂度是  $O(\log_2 n)$ 。

## 2. 平衡二叉树的平衡调整方法

如何创建一棵平衡二叉树呢? 插入结点时, 首先按照二叉排序树处理, 若插入结点后破坏了平衡二叉树的特性, 需对平衡二叉树进行调整。调整方法是: 找到离插入结点最近且平衡因子绝对值超过 1 的祖先结点, 以该结点为根的子树称为**最小不平衡子树**, 可将重新平衡的范围局限于这棵子树。

先看一个具体例子 (见图 7.12)。假设表中关键字序列为(13, 24, 37, 90, 53)。

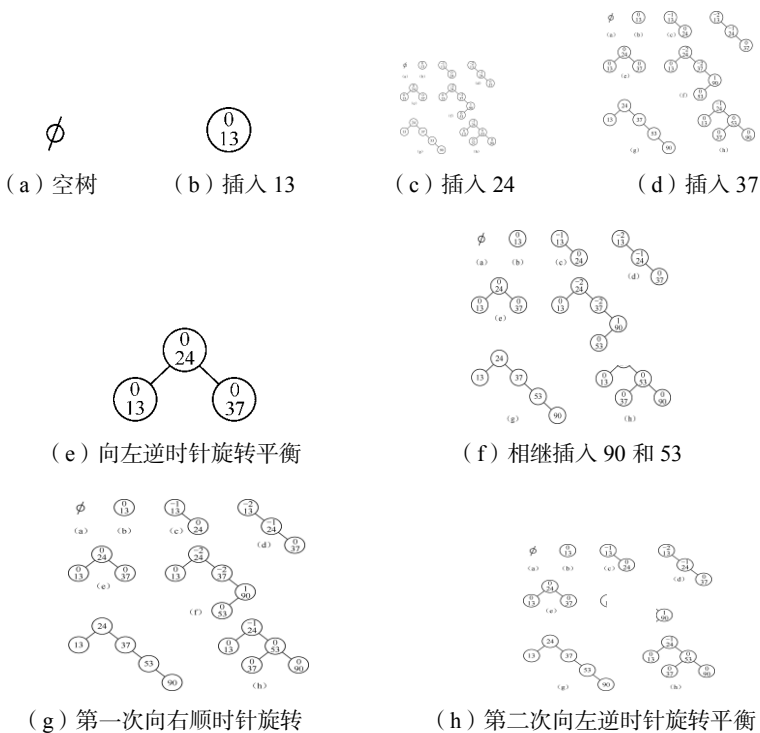


图 7.12 平衡树的生成过程

(1) 空树和 1 个结点⑬的树显然都是平衡的二叉树。在插入 24 之后仍是平衡的, 只是根结点的平衡因子 BF 由 0 变为-1, 如图 7.12 (a) ~ (c) 所示。

(2) 在继续插入 37 之后, 由于结点⑬的 BF 值由-1 变成-2, 由此出现了不平衡的现象。此时好比一根扁担出现一头重一头轻的现象, 若能将扁担的支撑点由⑬改至⑭, 扁担的两头

就平衡了。由此,可以对树做一个向左逆时针“旋转”的操作,令结点⑭为根,而结点⑬为它的左子树,此时,结点⑬和⑭的平衡因子都为 0,而且仍保持二叉排序树的特性,如图 7.12 (d)~(e)所示。

(3) 在继续插入 90 和 53 之后,结点⑳的 BF 值由-1 变成-2,排序树中出现了新的不平衡现象,需进行调整。但此时由于是结点⑬插在结点⑩的左子树上,因此不能如上做简单调整。离插入结点最近的最小不平衡子树是以结点⑳为根的子树。这时,必须以㉓作为根结点,而使㉑成为它的左子树的根,㉒成为它的右子树的根。这好比对树做了两次“旋转”操作,先向右顺时针旋转,后向左逆时针旋转(见图 7.12 (f)~(h)),使二叉排序树由不平衡转化为平衡。

一般情况下,假设最小不平衡子树的根结点为 A,则失去平衡后进行调整的规律可归纳为下列 4 种情况。

(1) LL 型: 由于在 A 左子树根结点的左子树上插入结点, A 的平衡因子由 1 增至 2,致使以 A 为根的子树失去平衡,则需进行一次向右的顺时针旋转操作,如图 7.13 所示。

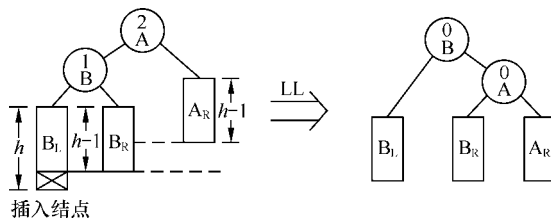


图 7.13 LL 型调整操作示意图

图 7.14 所示为两个 LL 型调整的实例。

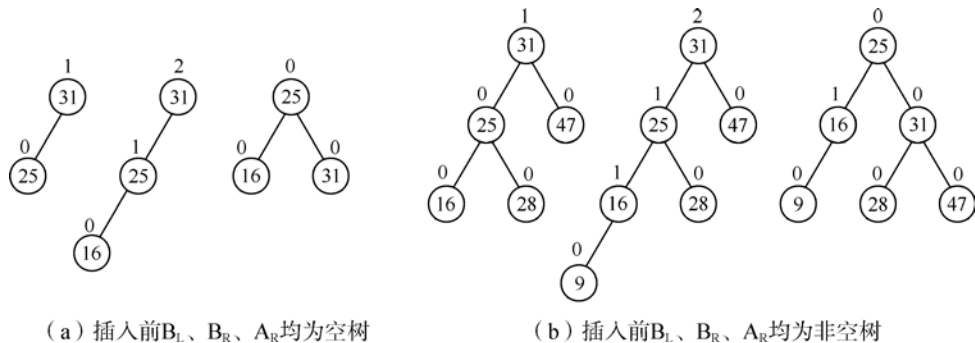


图 7.14 LL 型调整示例

(2) RR 型: 由于在 A 的右子树根结点的右子树上插入结点, A 的平衡因子由-1 变为-2,致使以 A 为根结点的子树失去平衡,则需进行一次向左的逆时针旋转操作,如图 7.15 所示。

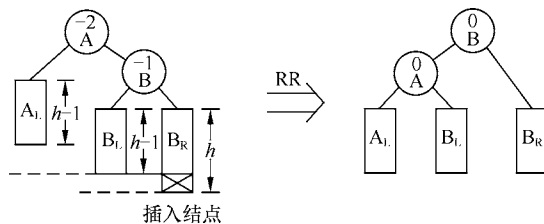


图 7.15 RR 型调整操作示意图

图 7.16 所示为两个 RR 型调整的实例。

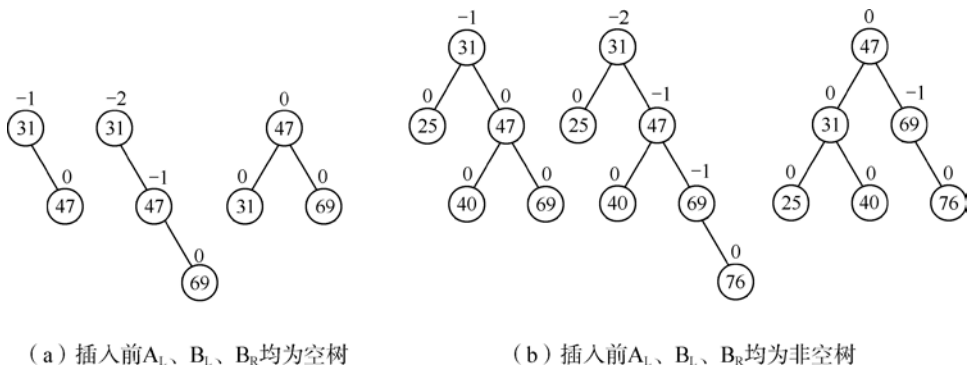


图 7.16 RR 型调整示例

(3) LR 型：由于在 A 的左子树根结点的右子树上插入结点，A 的平衡因子由 1 增至 2，致使以 A 为根结点的子树失去平衡，则需进行两次旋转操作。第一次对 B 及其右子树进行逆时针旋转，C 转上去成为 B 的根，这时变成了 LL 型，所以第二次进行 LL 型的顺时针旋转即可恢复平衡。如果 C 原来有左子树，则调整 C 的左子树为 B 的右子树，如图 7.17 所示。

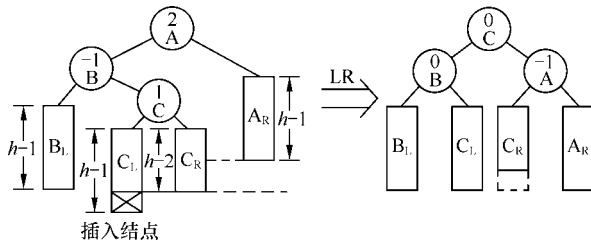


图 7.17 LR 型调整操作示意图

LR 型旋转前后 A、B、C 三个结点平衡因子的变化分为 3 种情况，图 7.18 所示为 3 种 LR 型调整的实例。

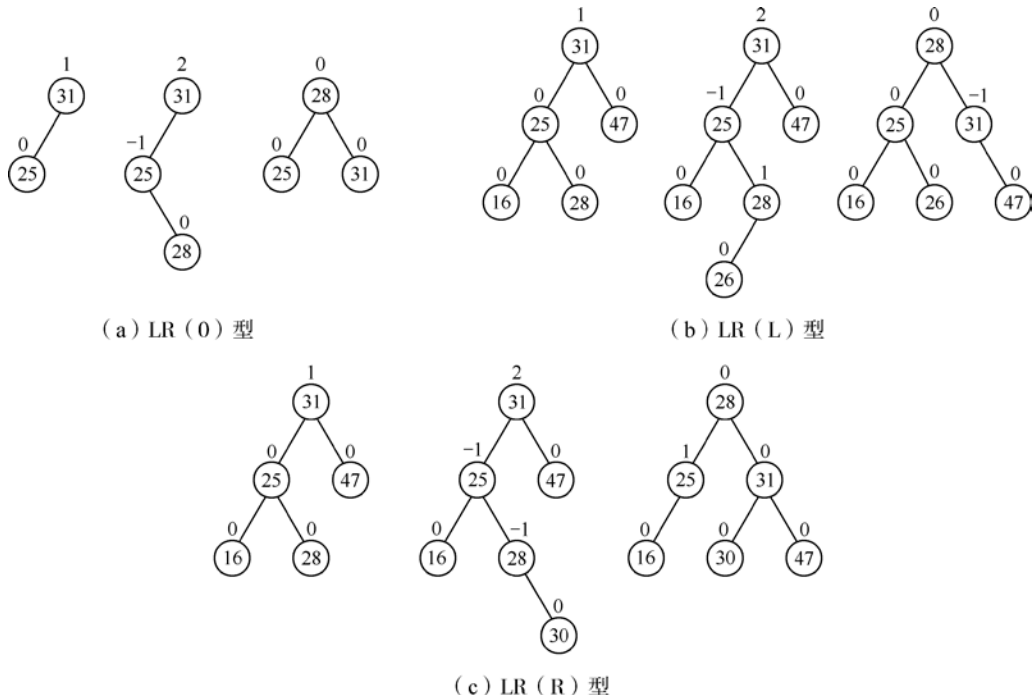


图 7.18 LR 型调整示例

(4) RL 型：由于在 A 的右子树根结点的左子树上插入结点，A 的平衡因子由 -1 变为 -2，致使以 A 为根结点的子树失去平衡，则旋转方法和 LR 型相对称，也需进行两次旋转，先顺时针右旋，再逆时针左旋，如图 7.19 所示。

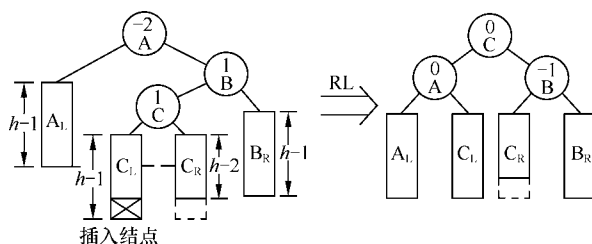


图 7.19 RL 型调整操作示意图

同 LR 型旋转类似，RL 型旋转前后 A、B、C 三个结点的平衡因子的变化也分为 3 种情况，图 7.20 所示为 3 种 RL 型调整的实例。

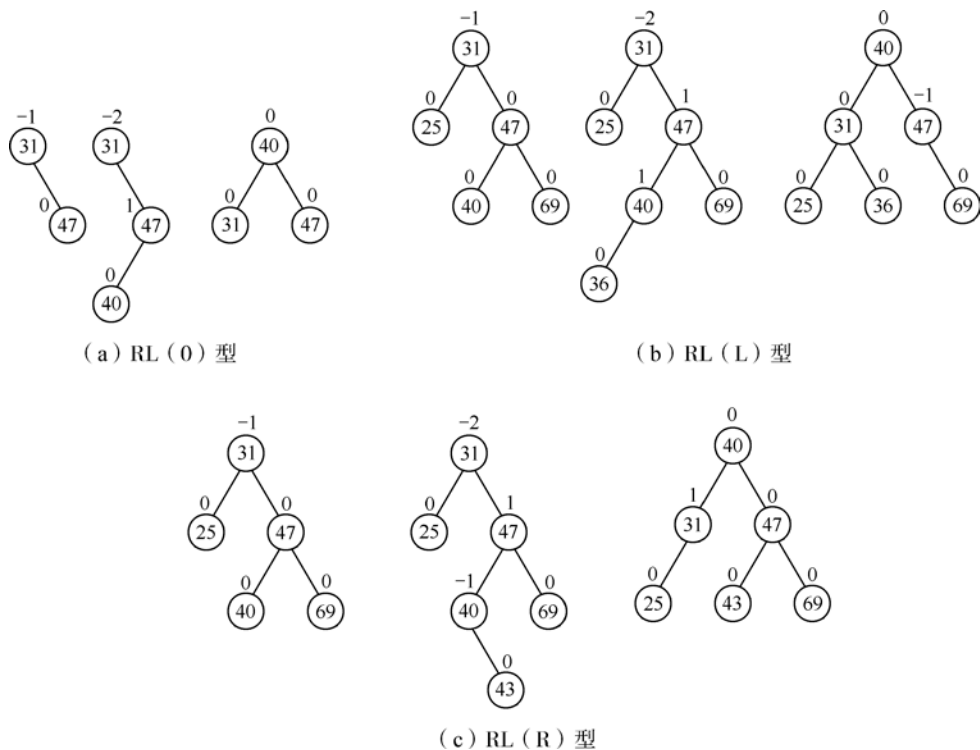


图 7.20 RL 型调整示例

上述 4 种情况中，(1) 和 (2) 对称，(3) 和 (4) 对称。旋转操作的正确性容易由“保持二叉排序树的特性：中序遍历所得关键字序列自小至大有序”证明之。同时，无论哪一种情况，在经过平衡旋转处理之后，以 B 或 C 为根的新子树为平衡二叉树，而且它们的深度和插入之前以 A 为根的子树相同。因此，当平衡的二叉排序树因插入结点而失去平衡时，仅需对最小不平衡子树进行平衡旋转处理即可。因为经过旋转处理之后的子树深度和插入之前相同，因而不影响插入路径上所有祖先结点的平衡度。

### 3. 平衡二叉树的插入

在平衡的二叉排序树 BBST 上插入一个新的数据元素  $e$  的递归算法可描述如下。

- ① 若 BBST 为空树, 则插入一个数据元素为  $e$  的新结点作为 BBST 的根结点, 树的深度增 1。
- ② 若  $e$  的关键字和 BBST 的根结点的关键字相等, 则不进行插入。
- ③ 若  $e$  的关键字小于 BBST 的根结点的关键字, 而且在 BBST 的左子树中不存在和  $e$  有相同关键字的结点, 则将  $e$  插入在 BBST 的左子树上, 并且当插入之后的左子树深度增加 (+1) 时, 分别就下列不同情况处理之:

- BBST 的根结点的平衡因子为 -1 (右子树的深度大于左子树的深度): 则将根结点的平衡因子更改为 0, BBST 的深度不变;
- BBST 的根结点的平衡因子为 0 (左、右子树的深度相等): 则将根结点的平衡因子更改为 1, BBST 的深度增 1;
- BBST 的根结点的平衡因子为 1 (左子树的深度大于右子树的深度): 若 BBST 的左子树根结点的平衡因子为 1, 则需进行单向右旋平衡处理, 并且在右旋处理之后, 将根结点和其右子树根结点的平衡因子更改为 0, 树的深度不变;
- 若 BBST 的左子树根结点的平衡因子为 -1, 则需进行先向左、后向右的双向旋转平衡处理, 并且在旋转处理之后。修改根结点和其左、右子树根结点的平衡因子, 树的深度不变。

- ④ 若  $e$  的关键字大于 BBST 的根结点的关键字, 而且在 BBST 的右子树中不存在和  $e$  有相同关键字的结点, 则将  $e$  插入在 BBST 的右子树上, 并且当插入之后的右子树深度增加 (+1) 时, 分别就不同情况处理之。其处理操作和③中所述相对称, 读者可自行补充。

## 7.3.3 B-树

前面介绍的查找方法均适用于存储在计算机内存中较小的文件, 统称为内查找法。若文件很大且存放于外存进行查找时, 这些查找方法就不适用了。内查找法都以结点为单位进行查找, 这样需要反复地进行内、外存的交换, 是很费时的。1970 年, R.Bayer 和 E.McCreight 提出了一种适用于外查找的平衡多叉树——B-树, 磁盘管理系统中的目录管理, 以及数据库系统中的索引组织多数都采用 B-树这种数据结构。

### 1. B-树的定义

一棵  $m$  阶的 B-树, 或为空树, 或为满足下列特性的  $m$  叉树:

- (1) 树中每个结点至多有  $m$  棵子树;
- (2) 若根结点不是叶子结点, 则至少有两棵子树;
- (3) 除根之外的所有非终端结点至少有  $\lceil m/2 \rceil$  棵子树;
- (4) 所有的叶子结点都出现在同一层次上, 并且不带信息, 通常称为失败结点 (失败结点不存在, 指向这些结点的指针为空。引入失败结点是为了便于分析 B-树的查找性能);
- (5) 所有的非终端结点最多有  $m-1$  个关键字, 结点的结构如图 7.21 所示。

|     |       |       |       |       |       |          |       |       |
|-----|-------|-------|-------|-------|-------|----------|-------|-------|
| $n$ | $P_0$ | $K_1$ | $P_1$ | $K_2$ | $P_2$ | $\cdots$ | $K_n$ | $P_n$ |
|-----|-------|-------|-------|-------|-------|----------|-------|-------|

图 7.21 B-树的结点结构

其中,  $K_i (i = 1, \cdots, n)$  为关键字, 且  $K_i < K_{i+1} (i = 1, \cdots, n-1)$ ;  $P_i (i = 0, \cdots, n)$  为指向子树根结点的指针, 且指针  $P_{i-1}$  所指子树中所有结点的关键字均小于  $K_i (i = 1, \cdots, n)$ ,  $P_n$  所指子树中所有结点的关键字均大于  $K_n$ ,  $n(\lceil m/2 \rceil - 1 \leq n \leq m-1)$  为关键字的个数 (或  $n+1$  为子树

个数)。

从上述定义可以看出,对任一关键字  $K_i$  而言,  $P_{i-1}$  相当于指向其“左子树”,  $P_i$  相当于指向其“右子树”。

B-树具有平衡、有序、多路的特点,图 7.22 所示为一棵 4 阶的 B-树,能很好地说明其特点。

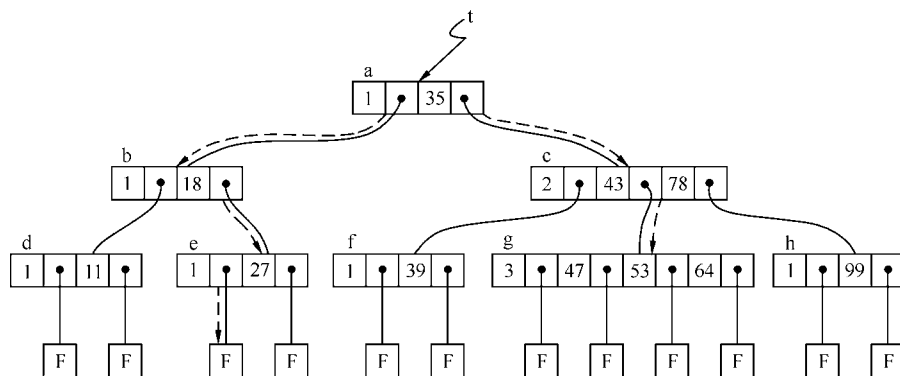


图 7.22 一棵 4 阶的 B-树

(1) 所有叶子结点均在同一层次,这体现出其平衡的特点。

(2) 树中每个结点中的关键字都是有序的,且关键字  $K_i$  “左子树”中的关键字均小于  $K_i$ ,而其“右子树”中的关键字均大于  $K_i$ ,这体现出其有序的特点。

(3) 除叶子结点外,有的结点中有一个关键字,两棵子树,有的结点中有两个关键字,三棵子树,这种 4 阶的 B-树最多有三个关键字,四棵子树,这体现出其多路的特点。

在具体实现时,为记录其双亲结点,B-树结点的存储结构通常增加一个 parent 指针,指向其双亲结点,存储结构示意图如图 7.23 所示。

|        |     |       |       |     |       |       |       |       |     |       |
|--------|-----|-------|-------|-----|-------|-------|-------|-------|-----|-------|
| parent | $n$ | $K_1$ | $K_2$ | ... | $K_n$ | $P_0$ | $P_1$ | $P_2$ | ... | $P_n$ |
|--------|-----|-------|-------|-----|-------|-------|-------|-------|-----|-------|

图 7.23 B-树结点的存储结构

## 2. B-树的查找

由 B-树的定义可知,在 B-树上进行查找的过程和二叉排序树的查找类似。

例如,在图 7.22 所示的 B-树上查找关键字 47 的过程如下:首先从根开始,根据根结点指针  $t$  找到 \*a 结点,因 \*a 结点中只有一个关键字,且  $47 > 35$ ,若查找的记录存在,则必在指针  $P_1$  所指的子树内,顺指针找到 \*c 结点,该结点有两个关键字 (43 和 78),而  $43 < 47 < 78$ ,若查找的记录存在,则必在指针  $P_1$  所指的子树中。同样,顺指针找到 \*g 结点,在该结点中顺序查找,找到关键字 47,由此,查找成功。

查找不成功的过程也类似,例如,在同一棵树中查找 23。从根开始,因为  $23 < 35$ ,则顺该结点中指针  $P_0$  找到 \*b 结点,又因为 \*b 结点中只有一个关键字 18,且  $23 > 18$ ,所以顺结点中第二个指针  $P_1$  找到 \*e 结点。同理,因为  $23 < 27$ ,则顺指针往下找,此时因指针所指为叶子结点,说明此棵 B-树中不存在关键字 23,查找因失败而告终。

由此可见,在 B-树上进行查找的过程是一个顺指针查找结点,和在结点的关键字中查找交叉进行的过程。

由于 B-树主要用做文件的索引,因此它的查找涉及外存的存取,在此略去外存的读写,只做

示意性的描述。假设结点类型定义如下:

```
#define m 3 // B-树的阶, 暂设为 3
typedef struct BTreeNode
{
    int keynum; // 结点中关键字的个数, 即结点的大小
    struct BTreeNode *parent; // 指向双亲结点
    KeyType key[m+1]; // 关键字向量, 0 号单元未用
    struct BTreeNode *ptr[m+1]; // 子树指针向量
    Record *recptr[m+1]; // 记录指针向量, 0 号单元未用
}BTreeNode, *BTree; // B-树结点和 B-树的类型
typedef struct
{
    BTreeNode *pt; // 指向找到的结点
    int i; // 1..m, 在结点中的关键字序号
    int tag; // 1: 查找成功, 0: 查找失败
}Result; // B-树的查找结果类型
```

### 算法 7.8 B-树的查找

#### 【算法步骤】

将给定值  $key$  与根结点的各个关键字  $K_1, K_2, \dots, K_j$  ( $1 \leq j \leq m-1$ ) 进行比较, 由于该关键字序列是有序的, 所以查找时可采用顺序查找, 也可采用折半查找。查找时:

- ① 若  $key = K_i$  ( $1 \leq i \leq j$ ), 则查找成功;
- ② 若  $key < K_1$ , 则顺着指针  $P_0$  所指向的子树继续向下查找;
- ③ 若  $K_i < key < K_{i+1}$  ( $1 \leq i \leq j-1$ ), 则顺着指针  $P_i$  所指向的子树继续向下查找;
- ④ 若  $key > K_j$ , 则顺着指针  $P_j$  所指向的子树继续向下查找。

如果在自上而下的查找过程中, 找到了值为  $key$  的关键字, 则查找成功; 如果直到叶子结点也未找到, 则查找失败。

#### 【算法描述】

```
Result SearchBTree(BTree T, KeyType key)
{ // 在 m 阶 B-树 T 上查找关键字 key, 返回结果 (pt, i, tag)
    // 若查找成功, 则特征值 tag=1, 指针 pt 所指结点中第 i 个关键字等于 key
    // 否则特征值 tag=0, 等于 key 的关键字应插入在指针 pt 所指结点中第 i 和第 i+1 个关键字之间
    p=T; q=NULL; found=FALSE; i=0; // 初始化, p 指向待查结点, q 指向 p 的双亲
    while (p && !found)
    {
        i=Search(p, key);
        // 在 p->key[1..keynum] 中查找 i, 使得: p->key[i] <= key < p->key[i+1]
        if (i > 0 && p->key[i] == key) found=TRUE; // 找到待查关键字
        else {q=p; p=p->ptr[i];}
    }
    if (found) return(p, i, 1); // 查找成功
    else return(q, i, 0); // 查找不成功, 返回 k 的插入位置信息
}
```

#### 【算法分析】

从算法 7.8 可见, 在 B-树上进行查找包含两种基本操作: (1) 在 B-树中找结点; (2) 在结点

中找关键字。由于 B-树通常存储在磁盘上,则前一查找操作是在磁盘上进行的(在算法 7.8 中没有体现),而后一查找操作是在内存中进行的,即在磁盘上找到指针  $p$  所指结点后,先将结点中的信息读入内存,然后再利用顺序查找或折半查找查询等于  $K$  的关键字。显然,在磁盘上进行一次查找比在内存中进行一次查找耗费时间多出很多,因此,在磁盘上进行查找的次数,即待查关键字所在结点在 B-树上的层次数,是决定 B-树查找效率的首要因素。

现考虑最坏的情况,即待查结点在 B-树的最下面一层。也就是说,含  $N$  个关键字的  $m$  阶 B-树的最大深度是多少?

先看一棵 3 阶的 B-树。按 B-树上的定义,3 阶的 B-树上所有非终端结点至多可有两个关键字,至少有一个关键字(即子树个数为 2 或 3,故又称 2-3 树)。因此,若关键字个数  $\leq 2$  时,树的深度为 2(即叶子结点层次为 2);若关键字个数  $\leq 6$  时,树的深度不超过 3。反之,若 B-树的深度为 4,则关键字的个数必须  $\geq 7$ (见图 7.24 (g)),此时,每个结点都含有可能的关键字的最小数目。

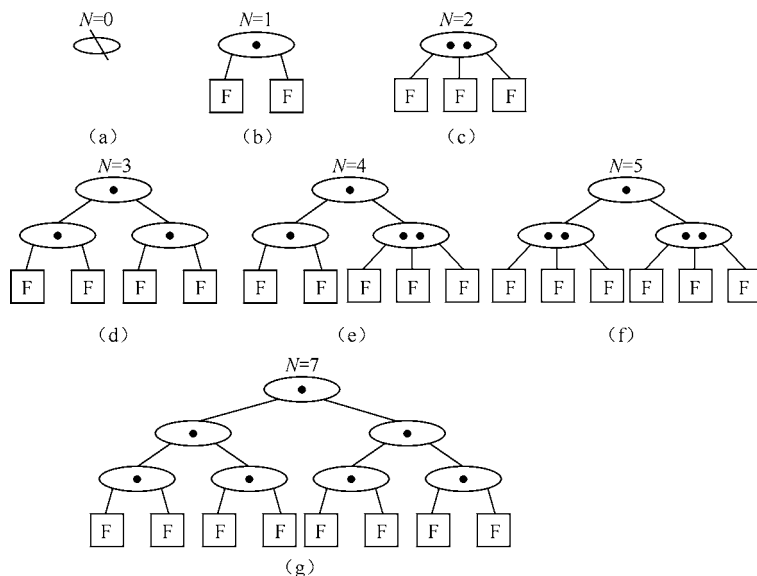


图 7.24 不同关键字数目的 B-树

一般情况的分析可类似平衡二叉树进行,先讨论深度为  $h+1$  的  $m$  阶 B-树所具有的最少结点数。

根据 B-树的定义,第一层至少有 1 个结点;第二层至少有 2 个结点;由于除根之外的每个非终端结点至少有  $\lceil m/2 \rceil$  棵子树,则第三层至少有  $2\lceil m/2 \rceil$  个结点;……;依次类推,第  $h+1$  层至少有  $2(\lceil m/2 \rceil)^{h-1}$  个结点。而  $h+1$  层的结点为叶子结点。若  $m$  阶 B-树中具有  $N$  个关键字,则叶子结点即查找不成功的结点为  $N+1$ ,由此有:

$$N+1 \geq 2 \times (\lceil m/2 \rceil)^{h-1}$$

反之

$$h \leq \log_{\lceil m/2 \rceil} \left( \frac{N+1}{2} \right) + 1$$

这就是说,在含有  $N$  个关键字的 B-树上进行查找时,从根结点到关键字所在结点的路径上涉及的结点数不超过  $\log_{\lceil m/2 \rceil} \left( \frac{N+1}{2} \right) + 1$ 。

### 3. B-树的插入

B-树是动态查找树, 因此其生成过程是从空树起, 在查找的过程中通过逐个插入关键字而得到。但由于 B-树中除根之外的所有非终端结点中的关键字个数必须大于等于  $\lceil m/2 \rceil - 1$ , 因此, 每次插入一个关键字不是在树中添加一个叶子结点, 而是首先在最低层的某个非终端结点中添加一个关键字, 若该结点的关键字个数不超过  $m - 1$ , 则插入完成, 否则表明结点已满, 要产生结点的“分裂”, 将此结点在同一层分成两个结点。一般情况下, 结点分裂方法是: 以中间关键字为界把结点一分为二, 成为两个结点, 并把中间关键字向上插入到双亲结点上, 若双亲结点已满, 则采用同样的方法继续分解。最坏的情况下, 一直分解到树根结点, 这时 B-树高度增加 1。

例如, 图 7.25 (a) 所示为 3 阶的 B-树 (图中略去 F 结点 (即叶子结点)), 假设需依次插入关键字 30、26、85 和 7。首先通过查找确定应插入的位置。由根 \*a 起进行查找, 确定 30 应插入在 \*d 结点中, 由于 \*d 中关键字数目不超过 2 (即  $m - 1$ ), 故第一个关键字插入完成。插入 30 后的 B-树如图 7.25 (b) 所示。同样, 通过查找确定关键字 26 亦应插入在 \*d 结点中。由于 \*d 中关键字的数目超过 2, 此时需将 \*d 分裂成两个结点, 关键字 26 及其前、后两个指针仍保留在 \*d 结点中, 而关键字 37 及其前、后两个指针存储到新产生的结点 \*d' 中。同时, 将关键字 30 和指示结点 \*d' 的指针插入到其双亲结点中。由于 \*b 结点中的关键字数目没有超过 2, 则插入完成。插入后的 B-树如图 7.25 (d) 所示。类似地, 在 \*g 中插入 85 之后需分裂成两个结点, 而当 70 继而插入到双亲结点中时, 由于 \*e 中关键字数目超过 2, 则再次分裂为结点 \*e 和 \*e', 如图 7.25 (g) 所示。最后在插入关键字 7 时, \*c、\*b 和 \*a 相继分裂, 并生成一个新的根结点 \*m, 如图 7.25 (h) ~ (j) 所示。

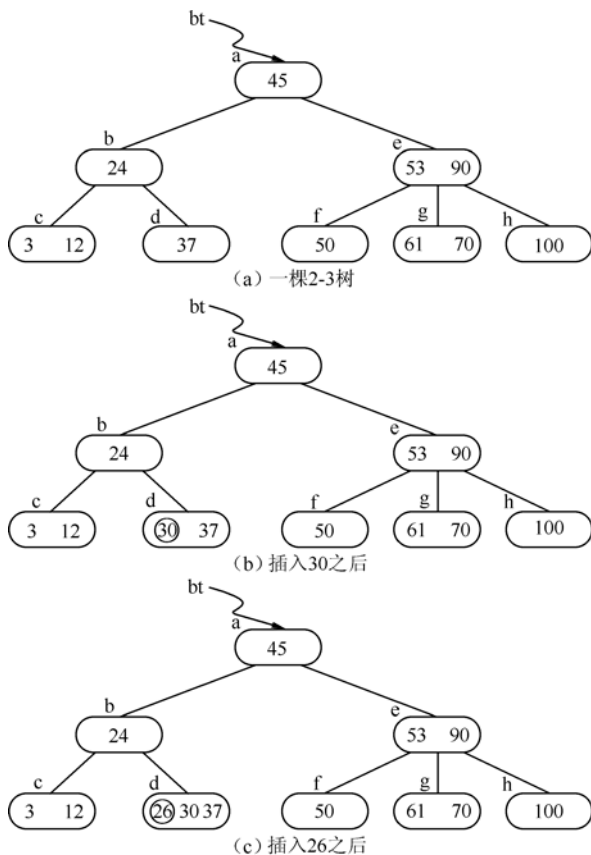


图 7.25 在 B-树中进行插入 (省略叶子结点)

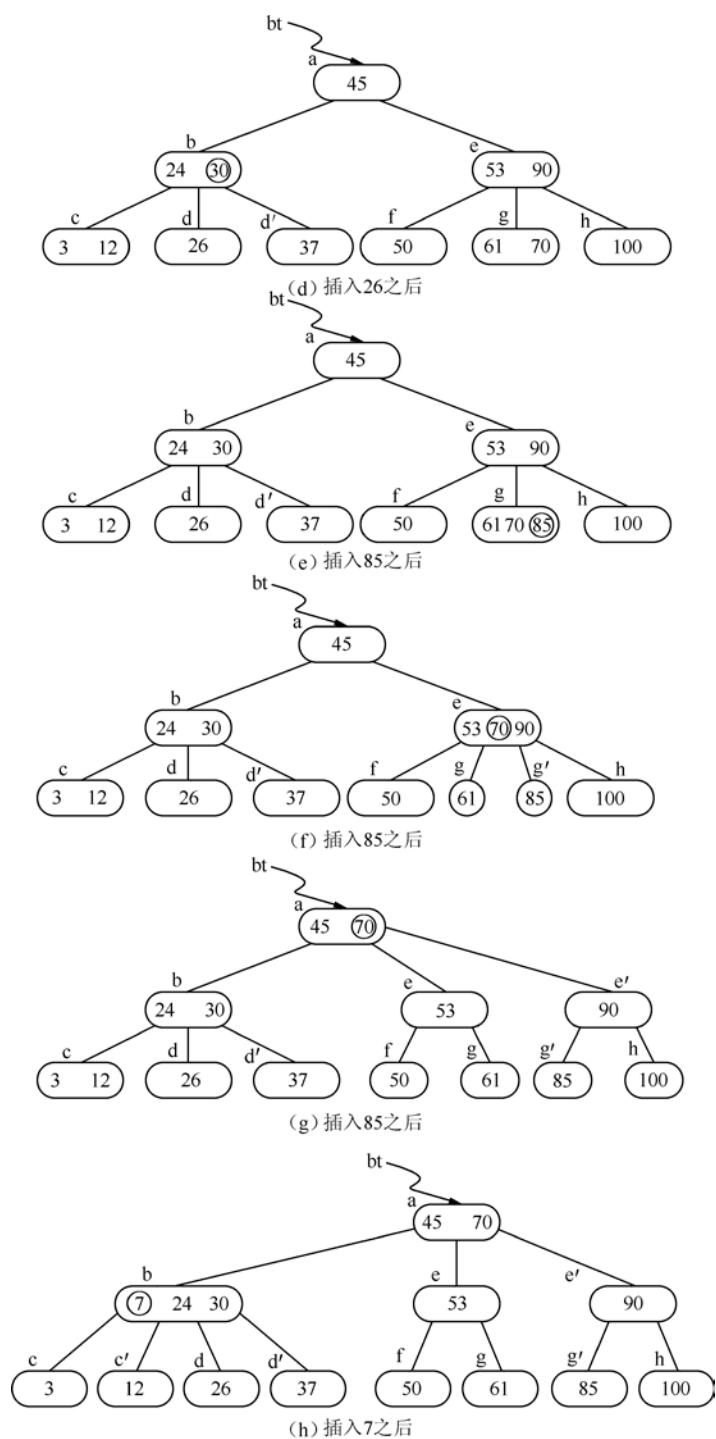
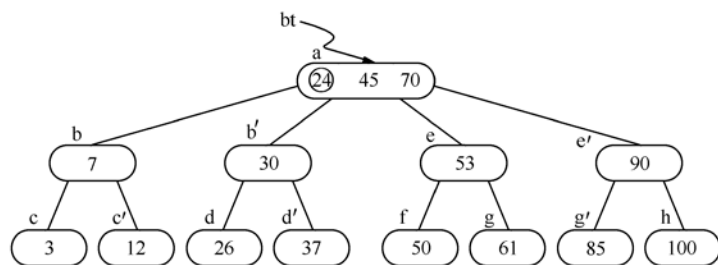
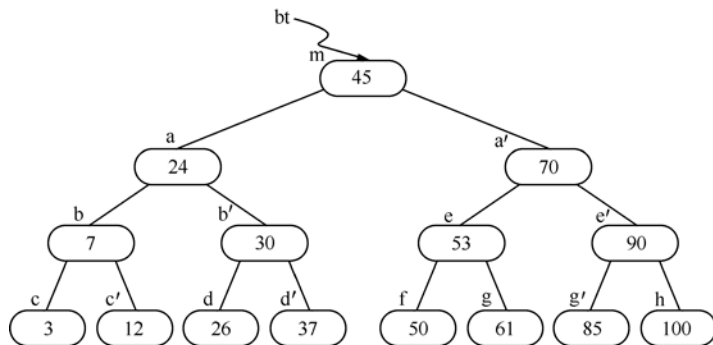


图 7.25 在 B-树中进行插入 (省略叶子结点) (续)



(i) 插入7之后



(j) 插入7之后

图 7.25 在 B-树中进行插入 (省略叶子结点) (续)

### 算法 7.9 B-树的插入

#### 【算法步骤】

① 在 B-树中查找给定关键字的记录, 若查找成功, 则插入操作失败; 否则将新记录作为空指针  $p$  插入到查找失败的叶子结点的上一层结点 (由  $q$  指向) 中。

② 若插入新记录和空指针后,  $q$  指向的结点的关键字数未超过  $m-1$ , 则插入操作成功, 否则转入步骤③。

③ 以该结点的第  $\lceil m/2 \rceil$  个关键字  $K_{\lceil m/2 \rceil}$  为拆分点, 将该结点分成 3 个部分:  $K_{\lceil m/2 \rceil}$  左边部分、 $K_{\lceil m/2 \rceil}$ 、 $K_{\lceil m/2 \rceil}$  右边部分。 $K_{\lceil m/2 \rceil}$  左边部分仍然保留在原结点中;  $K_{\lceil m/2 \rceil}$  右边部分存放在一个新创建的结点 (由  $p$  指向) 中; 关键字值为  $K_{\lceil m/2 \rceil}$  的记录和指针  $p$  插入到  $q$  的双亲结点中。因  $q$  的双亲结点增加一个新的记录, 所以必须对  $q$  的双亲结点重复②和③的操作, 依次类推, 直至由  $q$  指向的结点是根结点, 转入步骤④。

④ 由于根结点无双亲, 则由其分裂产生的两个结点的指针  $p$  和  $q$ , 以及关键字为  $K_{\lceil m/2 \rceil}$  的记录构成一个新的根结点。此时, B-树的高度增加 1。

下面算法描述中的  $q$  和  $i$  是由查找函数 `SearchBTree` 返回的信息而得。

#### 【算法描述】

```
Status InsertBTree(BTree &T, KeyType K, BTree q, int i)
{
    // 在 m 阶 B-树 T 上结点 *q 的 key[i] 与 key[i+1] 之间插入关键字 K
    // 若引起结点过大, 则沿双亲链进行必要的结点分裂调整, 使 T 仍是 m 阶 B-树
    x = K; ap = NULL; finished = FALSE;    // x 表示新插入的关键字, ap 为一个空指针
    while (q && !finished)
    {
        Insert(q, i, x, ap);                // 将 x 和 ap 分别插入到 q->key[i+1] 和 q->ptr[i+1]
```

```

    if(q->keynum<m) finished=TRUE;    //插入完成
    else                               //分裂结点*q
    {
        s= $\lceil m/2 \rceil$ ; split(q,s,ap); x=q->key[s];
        //将 q->key[s+1..m], q->ptr[s..m] 和 q->recptr[s+1..m] 移入新结点*ap
        q=q->parent;
        if(q) i=Search(q,x);    //在双亲结点*q 中查找 x 的插入位置
    }
    //else
    //while
    if(!finished)               //T 是空树 (参数 q 初值为 NULL) 或者根结点已分裂为结点*q 和*ap
        NewRoot(T,q,x,ap);    //生成含信息 (T,x,ap) 的新的根结点*T, 原 T 和 ap 为子树指针
    return OK;
}

```

#### 4. B-树的删除

$m$  阶 B-树的删除操作是在 B-树的某个结点中删除指定的关键字及其邻近的一个指针, 删除后应该进行调整使该树仍然满足 B-树的定义, 也就是要保证每个结点的关键字数目范围为  $[\lceil m/2 \rceil - 1, m]$ 。删除记录后, 结点的关键字数目如果小于  $\lceil m/2 \rceil - 1$ , 则要进行“合并”结点的操作。除了删除记录, 还要删除该记录邻近的指针。若该结点为最下层的非终端结点, 由于其指针均为空, 删除后不会影响其他结点, 可直接删除; 若该结点不是最下层的非终端结点, 邻近的指针则指向一棵子树, 不可直接删除。此时可做如下处理: 将要删除记录用其右 (左) 边邻近指针指向的子树中关键字最小 (大) 的记录 (该记录必定在最下层的非终端结点中) 替换。采取这种方法进行处理, 无论要删除的记录所在的结点是否为最下层的非终端结点, 都可归结为在最下层的非终端结点中删除记录的情况。

例如, 在图 7.25 (a) 所示的 B-树上删去 45, 可以用\*f 结点中的 50 替代 45, 然后在\*f 结点中删去 50。因此, 下面可以只讨论删除最下层非终端结点中的关键字的情形。有以下 3 种可能。

(1) 被删关键字所在结点中的关键字数目不小于  $\lceil m/2 \rceil$ , 则只需从该结点中删去该关键字  $K_i$  和相应指针  $P_i$ , 树的其他部分不变。例如, 从图 7.25 (a) 所示 B-树中删去关键字 12, 删除后的 B-树如图 7.26 (a) 所示。

(2) 被删关键字所在结点中的关键字数目等于  $\lceil m/2 \rceil - 1$ , 而与该结点相邻的右兄弟 (或左兄弟) 结点中的关键字数目大于  $\lceil m/2 \rceil - 1$ , 则需将其兄弟结点中的最小 (或最大) 的关键字上移至双亲结点中, 而将双亲结点中小于 (或大于) 且紧靠该上移关键字的关键字下移至被删关键字所在结点中。例如, 从图 7.26 (a) 中删去 50, 需将其右兄弟结点中的 61 上移至\*e 结点中, 而将\*e 结点中的 53 移至\*f, 从而使\*f 和\*g 中关键字数目均不小于  $\lceil m/2 \rceil - 1$ , 而双亲结点中的关键字数目不变, 如图 7.26 (b) 所示。

(3) 被删关键字所在结点和其相邻的兄弟结点中的关键字数目均等于  $\lceil m/2 \rceil - 1$ 。假设该结点有右兄弟, 且其右兄弟结点地址由双亲结点中的指针  $P_i$  所指, 则在删去关键字之后, 它所在结点中剩余的关键字和指针, 加上双亲结点中的关键字  $K_i$  一起, 合并到  $P_i$  所指兄弟结点中 (若没有右兄弟, 则合并至左兄弟结点中)。例如, 从图 7.26 (b) 所示 B-树中删去 53, 则应删去\*f 结点, 并将\*f 的剩余信息 (指针“空”) 和双亲\*e 结点中的 61 一起合并到右兄弟结点\*g 中, 删除后的树如图 7.26 (c) 所示。如果因此使双亲结点中关键字数目小于  $\lceil m/2 \rceil - 1$ , 则依次类推做相应处理。例如, 在图 7.26 (c) 的 B-树中删去关键字 37 之后, 双亲\*b 结点中剩余信息 (指针 c) 应和其双亲\*a 结点中关键字 45 一起合并至右兄弟结点\*e 中, 删除后的 B-树如图 7.26 (d) 所示。

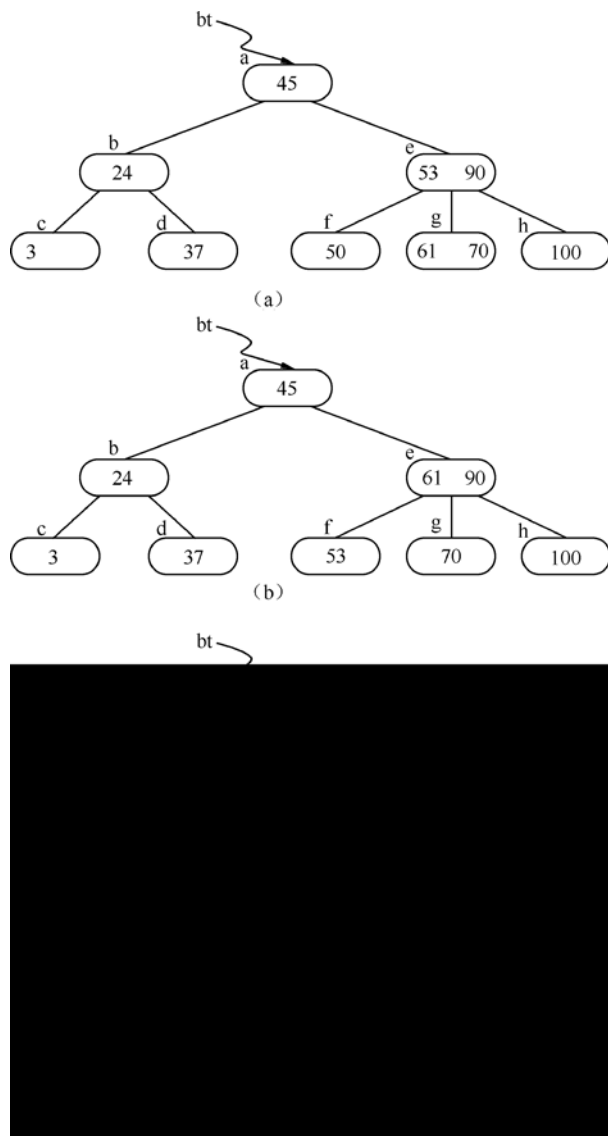


图 7.26 在 B-树中删除关键字的情形

在 B-树中删除结点的算法在此不再详述，读者可根据上述讨论自行写出此算法。

### 7.3.4 B+ 树

B+ 树是一种 B-树的变形树，更适合用于文件索引系统。严格来讲，它已不符合第 5 章中定义的了。

#### 1. B+ 树和 B-树的差异

一棵  $m$  阶的 B+ 树和  $m$  阶的 B-树的差异在于：

- (1) 有  $n$  棵子树的结点中含有  $n$  个关键字；
- (2) 所有的叶子结点中包含了全部关键字的信息，以及指向含这些关键字记录的指针，且叶子结点本身依关键字的大小自小而大顺序链接；
- (3) 所有的非终端结点可以看成是索引部分，结点中仅含有其子树（根结点）中的最大（或

最小) 关键字。

例如, 图 7.27 所示为一棵 3 阶的 B+ 树, 通常在 B+ 树上有两个头指针, 一个指向根结点, 另一个指向关键字最小的叶子结点。因此, 可以对 B+ 树进行两种查找运算: 一种是从最小关键字起顺序查找, 另一种是从根结点开始, 进行随机查找。

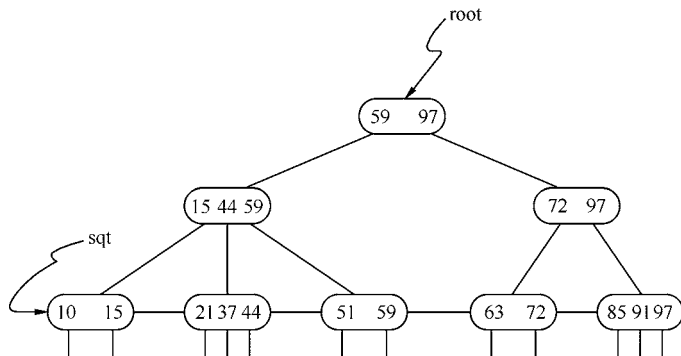


图 7.27 一棵 3 阶的 B+ 树

## 2. B+ 树的查找、插入和删除

在 B+ 树上进行随机查找、插入和删除的过程基本上与 B-树类似。

(1) 查找: 若非终端结点上的关键字等于给定值, 并不终止, 而是继续向下直到叶子结点。因此, 在 B+ 树中, 不管查找成功与否, 每次查找都是走了一条从根到叶子结点的路径。B+ 树查找的分析类似于 B-树。

B+ 树不仅能够有效地查找单个关键字, 而且更适合查找某个范围内的所有关键字。例如, 在 B+ 树上找出范围在  $[a, b]$  之间的所有关键字值。处理方法如下: 通过一次查找找出关键字  $a$ , 不管它是否存在, 都可以到达可能出现  $a$  的叶子结点, 然后在叶子结点中查找关键字值等于  $a$  或大于  $a$  的那些关键字, 对于所找到的每个关键字都有一个指针指向相应的记录, 这些记录的关键字在所需要的范围。如果在当前结点中没有发现大于  $b$  的关键字, 就可以使用当前叶子结点的最后一个指针找到下一个叶子结点, 并继续进行同样的处理, 直至在某个叶子结点中找到大于  $b$  的关键字, 才停止查找。

(2) 插入: 仅在叶子结点上进行插入, 当结点中的关键字个数大于  $m$  时要分裂成两个结点, 它们所含关键字的个数分别为  $\lfloor \frac{m+1}{2} \rfloor$  和  $\lceil \frac{m+1}{2} \rceil$ ; 并且, 它们的双亲结点中应同时包含这两个结点中的最大关键字。

(3) 删除: B+ 树的删除也仅在叶子结点进行, 当叶子结点中最大关键字被删除时, 其在非终端结点中的值可以作为一个“分界关键字”存在。若因删除而使结点中关键字的个数少于  $\lceil m/2 \rceil$  时, 其和兄弟结点的合并过程亦和 B-树类似。

# 7.4 散列表的查找

## 7.4.1 散列表的基本概念

前面讨论了基于线性表、树表结构的查找方法, 这类查找方法都是以关键字的比较为基础的。

在查找过程中只考虑各元素关键字之间的相对大小,记录在存储结构中的位置和其关键字无直接关系,其查找时间与表的长度有关,特别是当结点个数很多时,查找时要大量地与无效结点的关键字进行比较,致使查找速度很慢。如果能在元素的存储位置和其关键字之间建立某种直接关系,那么在进行搜索时,就无需做比较或做很少次的比较,按照这种关系直接由关键字找到相应的记录。这就是**散列查找法** (Hash Search) 的思想,它通过对元素的关键字值进行某种运算,直接求出元素的地址,即使用关键字到地址的直接转换方法,而不需要反复比较。因此,散列查找法又叫**杂凑法**或**散列法**。

下面给出散列法中常用的几个术语。

(1) **散列函数和散列地址**:在记录的存储位置  $p$  和其关键字  $key$  之间建立一个确定的对应关系  $H$ ,使  $p = H(key)$ ,称这个对应关系  $H$  为散列函数,  $p$  为散列地址。

(2) **散列表**:一个有限连续的地址空间,用以存储按散列函数计算得到相应散列地址的数据记录。通常散列表的存储空间是一个一维数组,散列地址是数组的下标。

(3) **冲突和同义词**:对不同的关键字可能得到同一散列地址,即  $key_1 \neq key_2$ ,而  $H(key_1) = H(key_2)$ ,这种现象称为冲突。具有相同函数值的关键字对该散列函数来说称作同义词,  $key_1$  与  $key_2$  互称为同义词。

例如,对 C 语言某些关键字集合建立一个散列表,关键字集合为

$$S_1 = \{\text{main, int, float, while, return, break, switch, case, do}\}$$

设定一个长度为 26 的散列表应该足够,散列表可定义为

char HT[26][8];

假设散列函数的值取为关键字  $key$  中第一个字母在字母表 {a, b, ..., z} 的序号 (序号范围为 0~25),即

$$H(key) = key[0] - 'a'$$

其中,设  $key$  的类型是长度为 8 的字符数组,根据此散列函数构造的散列表如表 7.1 所示。

表 7.1 关键字集合  $S_1$  对应的散列表

|   |       |      |    |   |       |     |     |     |      |     |        |        |     |       |     |    |
|---|-------|------|----|---|-------|-----|-----|-----|------|-----|--------|--------|-----|-------|-----|----|
| 0 | 1     | 2    | 3  | 4 | 5     | ... | 8   | ... | 12   | ... | 17     | 18     | ... | 22    | ... | 25 |
|   | break | case | do |   | float |     | int |     | main |     | return | switch |     | while |     |    |

假设关键字集合扩充为:

$$S_2 = S_1 + \{\text{short, default, double, static, for, struct}\}$$

如果散列函数不变,新加入的七个关键字经过计算得到:  $H(\text{short}) = H(\text{static}) = H(\text{struct}) = 18$ ,  $H(\text{default}) = H(\text{double}) = 3$ ,  $H(\text{for}) = 5$ ,而 18、3 和 5 这几个位置均已存放相应的关键字,这就发生了冲突现象,其中,switch、short、static 和 struct 称为同义词;float 和 for 称为同义词,do、default 和 double 称为同义词。

集合  $S_2$  中的关键字仅有 15 个,仔细分析这 15 个关键字的特性,应该不难构造一个散列函数避免冲突。但在实际应用中,理想化的、不产生冲突的散列函数极少存在,这是因为通常散列表中关键字的取值集合远远大于表空间的地址集。例如,高级语言的编译程序要对源程序中的标识符建立一张符号表进行管理,多数都采取散列表。在设定散列函数时,考虑的查找关键字集合应包含所有可能产生的关键字,不同的源程序中使用的标识符一般也不相同,如果此语言规定标识符为长度不超过 8 的、字母开头的字母数字串,字母区分大小写,则标识符取值集合的大小为:

$$C_{52}^1 \times C_{62}^7 \times 7! = 1.09 \times 10^{12}$$

综上所述,散列查找法主要研究以下两方面的问题:

- ### 7.4.2 散列函数的构造方法

- (1) 散列表的长度;
- (2) 关键字的长度;
- (3) 关键字的分布情况;
- (4) 计算散列函数所需的时间;
- (5) 记录的查找频率。

- (1) 函数计算要简单, 每一关键字只能有一个散列地址与之对应;
- (2) 函数的值域需在表长的范围内, 计算出的散列地址的分布应均匀, 尽可能减少冲突。

### 1. 数字分析法

例如,有 80 个记录,其关键字为 8 位十进制数。假设散列表的表长为 100,则可取两位十进制数组成散列地址,选取的原则是分析这 80 个关键字,使得到的散列地址尽量避免产生冲突。假设这 80 个关键字中的一部分如下所列:

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
|   |   |   |   | ⋮ |   |   |   |
|   | 8 | 1 | 3 | 4 | 6 | 5 | 3 |
|   | 8 | 1 | 3 | 7 | 2 | 2 | 4 |
|   | 8 | 1 | 3 | 8 | 7 | 4 | 2 |
|   | 8 | 1 | 3 | 0 | 1 | 3 | 6 |
|   | 8 | 1 | 3 | 2 | 2 | 8 | 1 |
|   | 8 | 1 | 3 | 3 | 8 | 9 | 6 |
|   | 8 | 1 | 3 | 5 | 4 | 1 | 5 |
|   | 8 | 1 | 3 | 6 | 8 | 5 | 3 |
|   | 8 | 1 | 4 | 1 | 9 | 3 | 5 |
|   |   |   |   | ⋮ |   |   |   |
| ① | ② | ③ | ④ | ⑤ | ⑥ | ⑦ | ⑧ |

---

---

---

---

221

意两位，或取其中两位与另外两位的叠加求和后舍去进位作为散列地址。

数字分析法的适用情况：事先必须明确知道所有的关键字每一位上各种数字的分布情况。

在实际应用中，例如，同一出版社出版的所有图书，其 ISBN 号的前几位都是相同的，因此，若数据表只包含同一出版社的图书，构造散列函数时可以利用这种数字分析排除 ISBN 号的前几位数字。

### 2. 平方取中法

通常在选定散列函数时不一定能知道关键字的全部情况，取其中哪几位也不一定合适，而一个数平方后的中间几位数和数的每一位都相关，如果取关键字平方后的中间几位或其组合作为散列地址，则使随机分布的关键字得到的散列地址也是随机的，具体所取的位数由表长决定。平方取中法是一种较常用的构造散列函数的方法。

例如，为源程序中的标识符建立一个散列表，假设标识符为字母开头的字母数字串。假设人为约定每个标识的内部编码规则如下：把字母在字母表中的位置序号作为该字母的内部编码，如 I 的内部编码为 09，D 的内部编码为 04，A 的内部编码为 01。数字直接用其自身作为内部编码，如 1 的内部编码为 01，2 的内部编码为 02。根据以上编码规则，可知“IDA1”的内部编码为 09040101，同理可以得到“IDB2”、“XID3”和“YID4”的内部编码。之后分别对内部编码进行平方运算，再取出第 7 位到第 9 位作为其相应标识符的散列地址，如表 7.2 所示。

表 7.2 标识符及其散列地址

| 标识符  | 内部编码     | 内部编码的平方         | 散列地址 |
|------|----------|-----------------|------|
| IDA1 | 09040101 | 081723426090201 | 426  |
| IDB2 | 09040202 | 081725252200804 | 252  |
| XID3 | 24090403 | 580347516702409 | 516  |
| YID4 | 25090404 | 629528372883216 | 372  |

平方取中法的适用情况：不能事先了解关键字的所有情况，或难于直接从关键字中找到取值较分散的几位。

### 3. 折叠法

将关键字分割成位数相同的几部分（最后一部分的位数可以不同），然后取这几部分的叠加和（舍去进位）作为散列地址，这种方法称为折叠法。根据数位叠加的方式，可以把折叠法分为移位叠加和边界叠加两种。移位叠加是将分割后每一部分的最低位对齐，然后相加；边界叠加是将两个相邻的部分沿边界来回折叠，然后对齐相加。

例如，当散列表长为 1000 时，关键字  $key = 45387765213$ ，从左到右按 3 位数一段分割，可以得到 4 个部分：453、877、652、13。分别采用移位叠加和边界叠加，求得散列地址为 995 和 914，如图 7.28 所示。

453

877

652

+ 13

---

[1]995

$H(key) = 995$

(a) 移位叠加

453

778

652

+ 31

---

[1]914

$H(key) = 914$

(b) 边界叠加

图 7.28 由折叠法求得散列地址

折叠法的适用情况：适合于散列地址的位数较少，而关键字的位数较多，且难于直接从关键字中找到取值较分散的几位。

#### 4. 除留余数法

假设散列表表长为  $m$ ，选择一个不大于  $m$  的数  $p$ ，用  $p$  去除关键字，除后所得余数为散列地址，即

$$H(key) = key \% p$$

这个方法的关键是选取适当的  $p$ ，一般情况下，可以选  $p$  为小于表长的最大质数。例如，表长  $m = 100$ ，可取  $p = 97$ 。

除留余数法计算简单，适用范围非常广，是最常用的构造散列函数的方法。它不仅可以对关键字直接取模，也可在折叠、平方取中等运算之后取模，这样能够保证散列地址一定落在散列表的地址空间中。

### 7.4.3 处理冲突的方法

选择一个“好”的散列函数可以在一定程度上减少冲突，但在实际应用中，很难完全避免发生冲突，所以选择一个有效的处理冲突的方法是散列法的另一个关键问题。创建散列表和查找散列表都会遇到冲突，两种情况下处理冲突的方法应该一致。下面以创建散列表为例，来说明处理冲突的方法。

处理冲突的方法与散列表本身的组织形式有关。按组织形式的不同，通常分两大类：开放地址法和链地址法。

#### 1. 开放地址法

开放地址法的基本思想是：把记录都存储在散列表数组中，当某一记录关键字  $key$  的初始散列地址  $H_0 = H(key)$  发生冲突时，以  $H_0$  为基础，采取合适方法计算得到另一个地址  $H_1$ ，如果  $H_1$  仍然发生冲突，以  $H_1$  为基础再求下一个地址  $H_2$ ，若  $H_2$  仍然冲突，再求得  $H_3$ 。依次类推，直至  $H_k$  不发生冲突为止，则  $H_k$  为该记录在表中的散列地址。

这种方法在寻找“下一个”空的散列地址时，原来的数组空间对所有的元素都是开放的，所以称为开放地址法。通常把寻找“下一个”空位的过程称为探测，上述方法可用如下公式表示：

$$H_i = (H(key) + d_i) \% m \quad i = 1, 2, \dots, k (k \leq m - 1)$$

其中， $H(key)$  为散列函数， $m$  为散列表表长， $d_i$  为增量序列。根据  $d_i$  取值的不同，可以分为以下 3 种探测方法。

##### (1) 线性探测法

$$d_i = 1, 2, 3, \dots, m - 1$$

这种探测方法可以将散列表假想成一个循环表，发生冲突时，从冲突地址的下一单元顺序寻找空单元，如果到最后一个位置也没找到空单元，则回到表头开始继续查找，直到找到一个空位，就把此元素放入此空位中。如果找不到空位，则说明散列表已满，需要进行溢出处理。

##### (2) 二次探测法

$$d_i = 1^2, -1^2, 2^2, -2^2, 3^2, \dots, +k^2, -k^2 (k \leq m/2)$$

## (3) 伪随机探测法

$$d_i = \text{伪随机数序列}$$

例如, 散列表的长度为 11, 散列函数  $H(key) = key \% 11$ , 假设表中已填有关键字分别为 17、60、29 的记录, 如图 7.29 (a) 所示。现有第四个记录, 其关键字为 38, 由散列函数得到散列地址为 5, 产生冲突。

若用线性探测法处理时, 得到下一个地址 6, 仍冲突; 再求下一个地址 7, 仍冲突; 直到散列地址为 8 的位置为“空”时为止, 处理冲突的过程结束, 38 填入散列表中序号为 8 的位置, 如图 7.29 (b) 所示。

若用二次探测法, 散列地址 5 冲突后, 得到下一个地址 6, 仍冲突; 再求得下一个地址 4, 无冲突, 38 填入序号为 4 的位置, 如图 7.29 (c) 所示。

若用伪随机探测法, 假设产生的伪随机数为 9, 则计算下一个散列地址为  $(5+9)\%11 = 3$ , 所以 38 填入序号为 3 的位置, 如图 7.29 (d) 所示。

|   |   |   |   |   |    |    |    |   |   |    |
|---|---|---|---|---|----|----|----|---|---|----|
| 0 | 1 | 2 | 3 | 4 | 5  | 6  | 7  | 8 | 9 | 10 |
|   |   |   |   |   | 60 | 17 | 29 |   |   |    |

(a) 插入前

|  |  |  |  |  |    |    |    |    |  |  |
|--|--|--|--|--|----|----|----|----|--|--|
|  |  |  |  |  | 60 | 17 | 29 | 38 |  |  |
|--|--|--|--|--|----|----|----|----|--|--|

(b) 线性探测法

|  |  |  |  |    |    |    |    |  |  |  |
|--|--|--|--|----|----|----|----|--|--|--|
|  |  |  |  | 38 | 60 | 17 | 29 |  |  |  |
|--|--|--|--|----|----|----|----|--|--|--|

(c) 二次探测法

|  |  |  |    |  |    |    |    |  |  |  |
|--|--|--|----|--|----|----|----|--|--|--|
|  |  |  | 38 |  | 60 | 17 | 29 |  |  |  |
|--|--|--|----|--|----|----|----|--|--|--|

(d) 伪随机探测法, 伪随机数序列为 9, ...

图 7.29 用开放地址法处理冲突时, 关键字为 38 的记录插入前后的散列表

从上述线性探测法处理的过程中可以看到一个现象: 当表中  $i, i+1, i+2$  位置上已填有记录时, 下一个散列地址为  $i, i+1, i+2$  和  $i+3$  的记录都将填入  $i+3$  的位置, 这种在处理冲突过程中发生的两个第一个散列地址不同的记录争夺同一个后继散列地址的现象称作“二次聚集”或称作“堆积”), 即在处理同义词的冲突过程中又添加了非同义词的冲突。

可以看出, 上述三种处理方法各有优缺点。线性探测法的优点是: 只要散列表未填满, 总能找到一个不发生冲突的地址。缺点是: 会产生“二次聚集”现象。而二次探测法和伪随机探测法的优点是: 可以避免“二次聚集”现象。缺点也很显然: 不能保证一定找到不发生冲突的地址。

## 2. 链地址法

链地址法的基本思想是: 把具有相同散列地址的记录放在同一个单链表中, 称为同义词链表。有  $m$  个散列地址就有  $m$  个单链表, 同时用数组  $HT[0 \cdots m-1]$  存放各个链表的头指针, 凡是散列地址为  $i$  的记录都以结点方式插入到以  $HT[i]$  为头结点的单链表中。

【例 7.2】已知一组关键字为 (19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79), 设散列函数  $H(key) = key \% 13$ , 用链地址法处理冲突, 试构造这组关键字的散列表。

由散列函数  $H(key) = key \% 13$  得知散列地址的值域为  $0 \sim 12$ , 故整个散列表有 13 个单链表组成, 用数组  $HT[0..12]$  存放各个链表的头指针。如散列地址均为 1 的同义词 14、1、27、79 构

成一个单链表，链表的头指针保存在 HT[1]中，同理，可以构造其他几个单链表，整个散列表的结构如图 7.30 所示。

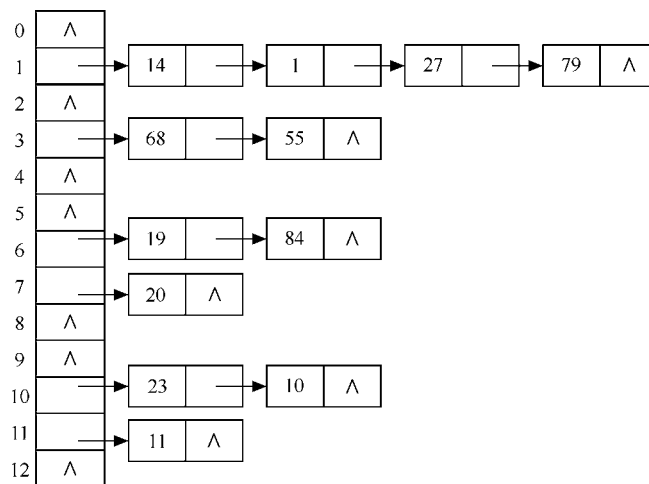


图 7.30 用链地址法处理冲突时的散列表

这种构造方法在具体实现时，依次计算各个关键字的散列地址，然后根据散列地址将关键字插入到相应的链表中。

### 7.4.4 散列表的查找

在散列表上进行查找的过程和创建散列表的过程基本一致。算法 7.10 描述了开放地址法（线性探测法）处理冲突的散列表的查找过程。

下面以开放地址法为例，给出散列表的存储表示。

```
// - - - - 开放地址法散列表的存储表示 - - - -
#define m 20 //散列表的表长
typedef struct{
    KeyType key; //关键字项
    InfoType otherinfo; //其他数据项
}HashTable[m];
```

#### 算法 7.10 散列表的查找

##### 【算法步骤】

- ① 给定待查找的关键字  $key$ ，根据造表时设定的散列函数计算  $H_0 = H(key)$ 。
- ② 若单元  $H_0$  为空，则所查元素不存在。
- ③ 若单元  $H_0$  中元素的关键字为  $key$ ，则查找成功。
- ④ 否则重复下述解决冲突的过程：
  - 按处理冲突的方法，计算下一个散列地址  $H_i$ ；
  - 若单元  $H_i$  为空，则所查元素不存在；
  - 若单元  $H_i$  中元素的关键字为  $key$ ，则查找成功。

##### 【算法描述】

```
#define NULLKEY 0 //单元为空的标记
int SearchHash(HashTable HT, KeyType key)
```

```
{//在散列表 HT 中查找关键字为 key 的元素，若查找成功，返回散列表的单元标号，否则返回-1
H0=H(key); //根据散列函数 H (key) 计算散列地址
if (HT[H0].key==NULLKEY) return -1; //若单元 H0 为空，则所查元素不存在
else if (HT[H0].key==key) return H0; //若单元 H0 中元素的关键字为 key，则查找成功
else
{
    for (i=1;i<m;++i)
    {
        Hi=(H0+i)%m; //按照线性探测法计算下一个散列地址 Hi
        if (HT[Hi].key==NULLKEY) return -1; //若单元 Hi 为空，则所查元素不存在
        else if (HT[Hi].key==key) return Hi; //若单元 Hi 中元素的关键字为 key，则查找成功
    }
    return -1; //for
} //else
}
```

【算法分析】

从散列表的查找过程可见：

(1) 虽然散列表在关键字与记录的存储位置之间建立了直接映像，但由于“冲突”的产生，使得散列表的查找过程仍然是一个给定值和关键字进行比较的过程。因此，仍需以平均查找长度作为衡量散列表查找效率的量度。

(2) 查找过程中需和给定值进行比较的关键字的个数取决于三个因素：散列函数、处理冲突的方法和散列表的装填因子。

散列表的装填因子  $\alpha$  定义为

$$\alpha = \frac{\text{表中填入的记录数}}{\text{散列表的长度}}$$

$\alpha$  标志散列表的装满程度。直观地看， $\alpha$  越小，发生冲突的可能性就越小；反之， $\alpha$  越大，表中已填入的记录越多，再填记录时，发生冲突的可能性就越大，则查找时，给定值需与之进行比较的关键字的个数也就越多。

(3) 散列函数的“好坏”首先影响出现冲突的频繁程度。但一般情况下认为：凡是“均匀的”散列函数，对同一组随机的关键字，产生冲突的可能性相同，假如所设定的散列函数是“均匀”的，则影响平均查找长度的因素只有两个——处理冲突的方法和装填因子  $\alpha$ 。

表 7.3 给出了在等概率情况下，采用几种不同方法处理冲突时，得到的散列表查找成功和查找失败时的平均查找长度，证明过程从略。

表 7.3 用几种不同方法处理冲突时散列表的平均查找长度

| 处理冲突的方法         | 平均查找长度                                         |                                                    |
|-----------------|------------------------------------------------|----------------------------------------------------|
|                 | 查找成功                                           | 查找失败                                               |
| 线性探测法           | $\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$ | $\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$ |
| 二次探测法<br>伪随机探测法 | $-\frac{1}{\alpha}\ln(1-\alpha)$               | $\frac{1}{1-\alpha}$                               |
| 链地址法            | $1+\frac{\alpha}{2}$                           | $\alpha+e^{-\alpha}$                               |

(4) 从表 7.3 可以看出，散列表的平均查找长度是  $\alpha$  的函数，而不是记录个数  $n$  的函数。由

此,在设计散列表时,不管  $n$  多大,总可以选择合适的  $\alpha$  以便将平均查找长度限定在一个范围内。

对于一个具体的散列表,通常采用直接计算的方法求其平均查找长度,下面通过具体示例说明。

**【例 7.3】** 对于例 7.2 中的关键字(19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79), 仍设散列函数为  $H(key)=key \% 13$ , 用线性探测法处理冲突。设表长为 16, 试构造这组关键字的散列表, 并计算查找成功和查找失败时的平均查找长度。

依次计算各个关键字的散列地址, 如果没有冲突, 将关键字直接存放在相应的散列地址所对应的单元中; 否则, 用线性探测法处理冲突, 直到找到相应的存储单元中。

如对于前三个关键字进行计算,  $H(19)=6$ ,  $H(14)=1$ ,  $H(23)=10$ , 所得散列地址均没有冲突, 直接填入所在单元。

而对于第四个关键字,  $H(1)=1$ , 发生冲突, 根据线性探测法, 求得下一个地址  $(1+1)\%16=2$ , 没有冲突, 所以 1 填入序号为 2 的单元。

同理, 可依次填入其他关键字。对于最后一个关键字 79,  $H(79)=1$ , 发生冲突, 用线性探测法处理冲突, 后面的地址 2~8 均有冲突, 最终 79 填入 9 号单元。

最终构造结果如表 7.4 所示, 表中最后一行的数字表示放置该关键字时所进行的关键字比较次数。

表 7.4 用线性探测法处理冲突时的散列表

| 散列地址 | 0 | 1  | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|------|---|----|---|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 关键字  |   | 14 | 1 | 68 | 27 | 55 | 19 | 20 | 84 | 79 | 23 | 11 | 10 |    |    |    |
| 比较次数 |   | 1  | 2 | 1  | 4  | 3  | 1  | 1  | 3  | 9  | 1  | 1  | 3  |    |    |    |

要查找一个关键字  $key$ , 根据算法 7.10, 首先用散列函数计算  $H_0=H(key)$ , 然后进行比较, 比较的次数和创建散列表时放置此关键字的比较次数是相同的。

例如, 查找 19 时, 计算散列函数  $H(19)=6$ ,  $HT[6].key$  非空且值为 19, 查找成功, 关键字比较次数为 1 次。

同样, 当查找关键字 14、68、20、23、11 时, 均需比较 1 次即查找成功。

当查找关键字 1 时, 计算散列函数  $H(1)=1$ ,  $HT[1].key$  非空且值为 14 $\neq$ 1, 用线性探测法处理冲突, 计算下一个地址为  $(1+1)\%16=2$ ,  $HT[2].key$  非空且值为 1, 查找成功, 关键字比较次数为 2 次。

当查找关键字 55, 84, 10 时, 需比较 3 次; 当查找 27 时, 需比较 4 次; 而查找 79 时, 需要比较 9 次才能查找成功。

在记录的查找概率相等的前提下, 这组关键字采用线性探测法处理散列表冲突时, 查找成功时的平均查找长度为

$$ASL_{\text{succ}} = \frac{1}{12}(1 \cdot 6 + 2 + 3 \cdot 3 + 4 + 9) = 2$$

查找失败时有两种情况:

(1) 单元为空;

(2) 按处理冲突的方法探测一遍后仍未找到。假设散列函数的取值个数为  $r$ , 则 0 到  $r-1$  相当于  $r$  个查找失败的入口, 从每个入口进入后, 直到确定查找失败为止, 其关键字的比较次数就是与该入口对应的查找失败的查找长度。

在例 7.3 中, 散列函数的取值个数为 13, 即总共有 13 个查找失败的入口 (0 到 12), 对每个

入口依次进行计算。

假设待查找的关键字不在表中, 若计算散列函数  $H(key) = 0$ ,  $HT[0].key$  为空, 比较 1 次即确定查找失败。若  $H(key) = 1$ ,  $HT[1].key$  非空, 则依次向后比较, 直到  $HT[13].key$  为空, 总共比较 13 次才能确定查找失败。类似地, 对  $H(key) = 2, 3, \dots, 12$  进行分析, 可得查找失败的平均查找长度为

$$ASL_{\text{unsucc}} = \frac{1}{13}(1+1+3+1+1+9+8+7+6+5+4+3+2) = 7$$

在例 7.2 中, 采用链地址法处理冲突时, 对于图 7.30 中所示的每个单链表中的第 1 个结点的关键字 (如 14、68、19、20、23、11), 查找成功时只需比较 1 次, 而对于第 2 个结点的关键字 (如 1、55、84、10), 查找成功时需比较 2 次, 第 3 个结点的关键字 27 需比较 3 次, 第 4 个结点的关键字 79 则需比较 4 次才能查找成功。这时, 查找成功时的平均查找长度为

$$ASL_{\text{succ}} = \frac{1}{12}(1 \cdot 6 + 2 \cdot 4 + 3 + 4) = 1.75$$

采用链地址法处理冲突时, 待查的关键字不在表中, 若计算散列函数  $H(key) = 0$ ,  $HT[0]$  的指针域为空, 比较 1 次即确定查找失败。若  $H(key) = 1$ ,  $HT[1]$  所指的单链表包括 4 个结点, 所以需要比较 5 次才能确定失败。类似地, 对  $H(key) = 2, 3, \dots, 12$  进行分析, 可得查找失败的平均查找长度为

$$ASL_{\text{unsucc}} = \frac{1}{13}(1+5+1+3+1+1+3+2+1+1+3+2+1) = 1.9$$

容易看出, 线性探测法在处理冲突的过程中易产生记录的二次聚集, 使得散列地址不相同的记录又产生新的冲突; 而链地址法处理冲突不会发生类似情况, 因为散列地址不同的记录在不同的链表中, 所以链地址法的平均查找长度小于开放地址法。另外, 由于链地址法的结点空间是动态申请的, 无需事先确定表的容量, 因此更适用于表长不确定的情况。同时, 易于实现插入和删除操作。

通过上面的示例, 可以看出, 在查找概率相等的前提下, 直接计算查找成功的平均查找长度可以采用以下公式

$$ASL_{\text{succ}} = \frac{1}{n} \sum_{i=1}^n C_i \quad (7-7)$$

其中,  $n$  为散列表中记录的个数,  $C_i$  为成功查找第  $i$  个记录所需的比较次数。

而直接计算查找失败的平均查找长度可以采用以下公式

$$ASL_{\text{unsucc}} = \frac{1}{r} \sum_{i=1}^r C_i \quad (7-8)$$

其中,  $r$  为散列函数取值的个数,  $C_i$  为散列函数取值为  $i$  时查找失败的比较次数。

## 7.5 小 结

查找是数据处理中经常使用的一种操作。本章主要介绍了对查找表的查找, 查找表实际上仅

仅是一个集合，为了提高查找效率，将查找表组织成不同的数据结构，主要包括 3 种不同结构的查找表：线性表、树表和散列表。

（1）线性表的查找。主要包括顺序查找、折半查找和分块查找，3 者之间的比较详见表 7.5。

表 7.5 顺序查找、折半查找和分块查找的比较

| 查找方法<br>比较项目 | 顺序查找                   | 折半查找             | 分块查找                        |
|--------------|------------------------|------------------|-----------------------------|
| 查找时间复杂度      | $O(n)$                 | $O(\log_2 n)$    | 与确定所在块的查找方法有关               |
| 特点           | 算法简单，对表结构无任何要求，但查找效率较低 | 对表结构要求较高，查找效率较高  | 对表结构有一定要求，查找效率介于折半查找和顺序查找之间 |
| 适用情况         | 任何结构的线性表，不经常做插入和删除     | 有序的顺序表，不经常做插入和删除 | 块间有序、块内无序的顺序表，经常做插入和删除      |

（2）树表的查找。树表的结构主要包括二叉排序树、平衡二叉树、B-树和 B+ 树。

① 二叉排序树的查找过程与折半查找过程类似，二者之间的比较详见表 7.6。

表 7.6 折半查找和二叉排序树查找的比较

| 查找方法<br>比较项目 | 折半查找                        | 二叉排序树的查找                            |
|--------------|-----------------------------|-------------------------------------|
| 查找时间复杂度      | $O(\log_2 n)$               | $O(\log_2 n)$                       |
| 特点           | 数据结构采用有序的顺序表，插入和删除操作需移动大量元素 | 数据结构采用树的二叉链表表示，插入和删除操作无需移动元素，只需修改指针 |
| 适用情况         | 不经常做插入和删除的静态查找表             | 经常做插入和删除的动态查找表                      |

② 二叉排序树在形态均匀时性能最好，而形态为单支树时其查找性能则退化为与顺序查找相同，因此，二叉排序树最好是一棵平衡二叉树。平衡二叉树的平衡调整方法就是确保二叉排序树在任何情况下的深度均为  $O(\log_2 n)$ ，平衡调整方法分为 4 种：LL 型、RR 型、LR 型和 RL 型。

③ B-树是一种平衡的多叉查找树，是一种在外存文件系统中常用的动态索引技术。在 B-树上进行查找的过程和二叉排序树类似，是一个顺指针查找结点和在结点内的关键字中查找交叉进行的过程。为了确保 B-树的定义，在 B-树中插入一个关键字，可能产生结点的“分裂”，而删除一个关键字，可能产生结点的“合并”。

④ B+ 树是一种 B-树的变型树，更适合做文件系统的索引。在 B+ 树上进行随机查找、插入和删除的过程基本上与 B-树类似，但具体实现细节又有所区别。

（3）散列表的查找。散列表也属线性结构，但它和线性表的查找有着本质的区别。它不是以关键字比较为基础进行查找的，而是通过一种散列函数把记录的关键字和它在表中的位置建立起对应关系，并在存储记录发生冲突时采用专门的处理冲突的方法。这种方式构造的散列表，不仅平均查找长度和记录总数无关，而且可以通过调节装填因子，把平均查找长度控制在所需的范围内。

散列查找法主要研究两方面的问题：如何构造散列函数，以及如何处理冲突。

① 构造散列函数的方法很多，除留余数法是最常用的构造散列函数的方法。它不仅可以对关

键字直接取模，也可在折叠、平方取中等运算之后取模。

② 处理冲突的方法通常分为两大类：开放地址法和链地址法，二者之间的差别类似于顺序表和单链表的差别，二者的比较详见表 7.7。

表 7.7 开放地址法和链地址法的比较

| 处理方法<br>比较项目 |       | 开放地址法             | 链地址法               |
|--------------|-------|-------------------|--------------------|
| 空间           |       | 无指针域，存储效率较高       | 附加指针域，存储效率较低       |
| 时<br>间       | 查找    | 有二次聚集现象，查找效率较低    | 无二次聚集现象，查找效率较高     |
|              | 插入、删除 | 不易实现              | 易于实现               |
| 适用情况         |       | 表的大小固定，适于表长无变化的情况 | 结点动态生成，适于表长经常变化的情况 |

学习完本章后，要求掌握顺序查找、折半查找和分块查找的方法，掌握描述折半查找过程的判定树的构造方法。掌握二叉排序树的构造和查找方法，平衡二叉树的 4 种平衡调整方法。理解 B-和 B+ 树的特点、基本操作和二者的区别。熟练掌握散列表的构造方法。明确各种不同查找方法之间的区别和各自的适用情况，能够按定义计算各种查找方法在等概率情况下查找成功的平均查找长度。

## 习 题

### 1. 选择题

- (1) 对  $n$  个元素的表做顺序查找时，若查找每个元素的概率相同，则平均查找长度为（ ）。
  - A.  $(n-1)/2$
  - B.  $n/2$
  - C.  $(n+1)/2$
  - D.  $n$
- (2) 适用于折半查找的表的存储方式，以及元素排列要求为（ ）。
  - A. 链接方式存储，元素无序
  - B. 链接方式存储，元素有序
  - C. 顺序方式存储，元素无序
  - D. 顺序方式存储，元素有序
- (3) 如果要求一个线性表既能较快的查找，又能适应动态变化的要求，最好采用（ ）查找法。
  - A. 顺序查找
  - B. 折半查找
  - C. 分块查找
  - D. 哈希查找
- (4) 折半查找有序表（4, 6, 10, 12, 20, 30, 50, 70, 88, 100）。若查找表中元素 58，则它将依次与表中（ ）比较大小，查找结果是失败。
  - A. 20, 70, 30, 50
  - B. 30, 88, 70, 50
  - C. 20, 50
  - D. 30, 88, 50
- (5) 对 22 个记录的有序表作折半查找，当查找失败时，至少需要比较（ ）次关键字。
  - A. 3
  - B. 4
  - C. 5
  - D. 6
- (6) 折半查找与二叉排序树的时间性能（ ）。
  - A. 相同
  - B. 完全不同
  - C. 有时不相同
  - D. 数量级都是  $O(\log_2 n)$

(7) 分别以下列序列构造二叉排序树, 与用其他三个序列所构造的结果不同的是 ( )。

- A. (100, 80, 90, 60, 120, 110, 130)      B. (100, 120, 110, 130, 80, 60, 90)  
C. (100, 60, 80, 90, 120, 110, 130)      D. (100, 80, 60, 90, 120, 130, 110)

(8) 在平衡二叉树中插入一个结点后造成了不平衡, 设最低的不平衡结点为 A, 并已知 A 的左孩子的平衡因子为 0, 右孩子的平衡因子为 1, 则应作 ( ) 型调整以使其平衡。

- A. LL      B. LR  
C. RL      D. RR

(9) 下列关于  $m$  阶 B-树的说法错误的是 ( )。

- A. 根结点至多有  $m$  棵子树  
B. 所有叶子都在同一层次上  
C. 非叶结点至少有  $m/2$  ( $m$  为偶数) 或  $m/2 + 1$  ( $m$  为奇数) 棵子树  
D. 根结点中的数据是有序的

(10) 下面关于 B-树和 B+ 树的叙述中, 不正确的是 ( )。

- A. B-树和 B+ 树都是平衡的多叉树  
B. B-树和 B+ 树都可用于文件的索引结构  
C. B-树和 B+ 树都能有效地支持顺序检索  
D. B-树和 B+ 树都能有效地支持随机检索

(11)  $m$  阶 B-树是一棵 ( )。

- A.  $m$  叉排序树      B.  $m$  叉平衡排序树  
C.  $m - 1$  叉平衡排序树      D.  $m + 1$  叉平衡排序树

(12) 下面关于散列查找的说法, 正确的是 ( )。

- A. 散列函数构造的越复杂越好, 因为这样随机性好, 冲突小  
B. 除留余数法是所有散列函数中最好的  
C. 不存在特别好与坏的散列函数, 要视情况而定  
D. 散列表的平均查找长度有时也和记录总数有关

(13) 下面关于散列查找的说法, 不正确的是 ( )。

- A. 采用链地址法处理冲突时, 查找任何一个元素的时间都相同  
B. 采用链地址法处理冲突时, 若插入规定总是在链首, 则插入任一个元素的时间是相同的  
C. 用链地址法处理冲突, 不会引起二次聚集现象  
D. 用链地址法处理冲突, 适合表长不确定的情况

(14) 设散列表长为 14, 散列函数是  $H(key) = key \% 11$ , 表中已有数据的关键字为 15, 38, 61, 84 共四个, 现要将关键字为 49 的元素加到表中, 用二次探测法解决冲突, 则放入的位置是 ( )。

- A. 3      B. 5  
C. 8      D. 9

(15) 采用线性探测法处理冲突, 可能要探测多个位置, 在查找成功的情况下, 所探测的这些位置上的关键字 ( )。

- A. 不一定是同义词      B. 一定都是同义词  
C. 一定都不是同义词      D. 都相同

## 2. 应用题

(1) 假定对有序表: (3, 4, 5, 7, 24, 30, 42, 54, 63, 72, 87, 95) 进行折半查找, 试回答下列问题。

- ① 画出描述折半查找过程的判定树。
- ② 若查找元素 54, 需依次与哪些元素比较?
- ③ 若查找元素 90, 需依次与哪些元素比较?
- ④ 假定每个元素的查找概率相等, 求查找成功时的平均查找长度。

(2) 在一棵空的二叉排序树中依次插入关键字序列为 12, 7, 17, 11, 16, 2, 13, 9, 21, 4, 请画出所得到的二叉排序树。

(3) 已知如下所示长度为 12 的表: (Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec)。

① 试按表中元素的顺序依次插入一棵初始为空的二叉排序树, 画出插入完成之后的二叉排序树, 并求其在等概率的情况下查找成功的平均查找长度。

② 若对表中元素先进行排序构成有序表, 求在等概率的情况下对此有序表进行折半查找时查找成功的平均查找长度。

③ 按表中元素顺序构造一棵平衡二叉排序树, 并求其在等概率的情况下查找成功的平均查找长度。

(4) 对图 7.31 所示的 3 阶 B-树, 依次执行下列操作, 画出各步操作的结果。

- ① 插入 90
- ② 插入 25
- ③ 插入 45
- ④ 删除 60

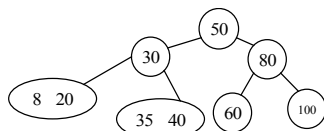


图 7.31 3 阶 B-树

(5) 设散列表的地址范围为 0~17, 散列函数为:

$H(key) = key \% 16$ 。用线性探测法处理冲突, 输入关键字序列: (10, 24, 32, 17, 31, 30, 46, 47, 40, 63, 49), 构造散列表, 试回答下列问题:

- ① 画出散列表的示意图。
- ② 若查找关键字 63, 需要依次与哪些关键字进行比较?
- ③ 若查找关键字 60, 需要依次与哪些关键字进行比较?
- ④ 假定每个关键字的查找概率相等, 求查找成功时的平均查找长度。

(6) 设有一组关键字 (9, 1, 23, 14, 55, 20, 84, 27), 采用散列函数:  $H(key) = key \% 7$ , 表长为 10, 用开放地址法的二次探测法处理冲突。要求: 对该关键字序列构造散列表, 并计算查找成功的平均查找长度。

(7) 设散列函数  $H(K) = 3K \% 11$ , 散列地址空间为 0~10, 对关键字序列 (32, 13, 49, 24, 38, 21, 4, 12), 按下述两种解决冲突的方法构造散列表, 并分别求出等概率下查找成功时和查找失败时的平均查找长度  $ASL_{succ}$  和  $ASL_{unsucc}$ 。

- ① 线性探测法。
- ② 链地址法。

## 3. 算法设计题

(1) 试写出折半查找的递归算法。

(2) 试写一个判别给定二叉树是否为二叉排序树的算法。

(3) 已知二叉排序树采用二叉链表存储结构, 根结点的指针为 T, 链结点的结构为 (lchild,

data, rchild), 其中 lchild、rchild 分别指向该结点左、右孩子的指针, data 域存放结点的数据信息。请写出递归算法, 从小到大输出二叉排序树中所有数据值  $\geq x$  的结点的数据。要求先找到第一个满足条件的结点后, 再依次输出其他满足条件的结点。

(4) 已知二叉树  $T$  的结点形式为(llink, data, count, rlink), 在树中查找值为  $X$  的结点, 若找到, 则记数(count)加 1; 否则, 作为一个新结点插入树中, 插入后仍为二叉排序树, 写出其非递归算法。

(5) 假设一棵平衡二叉树的每个结点都标明了平衡因子  $b$ , 试设计一个算法, 求平衡二叉树的高度。

(6) 分别写出在散列表中插入和删除关键字为  $K$  的一个记录的算法, 设散列函数为  $H$ , 解决冲突的方法为链地址法。

# 第 8 章

## 排序

排序是计算机程序设计中的一种重要操作，在很多领域中都有广泛的应用。如各种升学考试的录取工作，日常生活的各类竞赛活动等都离不开排序。排序的一个主要目的是便于查找，从第 7 章的讨论中容易看出，有序的顺序表可以采用查找效率较高的折半查找法，又如创建树表（无论是二叉排序树还是 B-树）的过程本身就是一个排序的过程。

人们设计了大量的排序算法以满足不同的需求。例如，著名计算机科学家 D.E.Knuth 在他的巨著《计算机程序设计艺术》第三卷《排序和查找》中，就给出了 25 种排序方法，并且指出，这只不过是现有排序方法的一小部分。本章仅讨论几种典型的、常用的排序方法。读者在学习本章内容时应注意，除了掌握算法本身以外，更重要的是了解该算法在进行排序时所依据的原则，以利于学习和创造更加新的算法。

## 8.1 基本概念和排序方法概述

### 8.1.1 排序的基本概念

#### 1. 排序

**排序**（Sorting）是按关键字的非递减或非递增顺序对一组记录重新进行排列的操作。确切描述如下。

假设含  $n$  个记录的序列为

$$\{R_1, R_2, \dots, R_n\} \quad (8-1)$$

其相应的关键字序列为

$$\{K_1, K_2, \dots, K_n\}$$

需确定  $1, 2, \dots, n$  的一种排列  $p_1, p_2, \dots, p_n$ ，使其相应的关键字满足如下的非递减（或非递增）关系

$$K_{p_1} \leq K_{p_2} \leq \dots \leq K_{p_n} \quad (8-2)$$

即使式（8-1）的序列成为一个按关键字有序的序列

$$\{R_{p_1}, R_{p_2}, \dots, R_{p_n}\} \quad (8-3)$$

这样一种操作称为排序。

#### 2. 排序的稳定性

当排序记录中的关键字  $K_i$ （ $i = 1, 2, \dots, n$ ）都不相同时，则任何一个记录的无序序列经排序后得到的结果唯一；反之，当待排序的序列中存在两个或两个以上关键字相等的记录时，则排序

所得的结果不唯一。假设  $K_i = K_j$  ( $1 \leq i \leq n, 1 \leq j \leq n, i \neq j$ )，且在排序前的序列中  $R_i$  领先于  $R_j$  (即  $i < j$ )。若在排序后的序列中  $R_i$  仍领先于  $R_j$ ，则称所用的排序方法是**稳定的**；反之，若可能使排序后的序列中  $R_j$  领先于  $R_i$ ，则称所用的排序方法是**不稳定的**。注意，排序算法的稳定性是针对所有记录而言的。也就是说，在所有的待排序记录中，只要有一组关键字的实例不满足稳定性要求，则该排序方法就是不稳定的。虽然稳定的排序方法和不稳定的排序方法排序结果不同，但不能说不稳定的排序方法就不好，各有各的适用场合。

### 3. 内部排序和外部排序

由于待排序记录的数量不同，使得排序过程中数据所占用的存储设备会有所不同。根据在排序过程中记录所占用的存储设备，可将排序方法分为两大类：一类是**内部排序**，指的是待排序记录全部存放在计算机内存中进行排序的过程；另一类是**外部排序**，指的是待排序记录的数量很大，以致内存一次不能容纳全部记录，在排序过程中尚需对外存进行访问的排序过程。本章首先介绍各种常用内部排序的方法，最后介绍外部排序的基本过程。

## 8.1.2 内部排序方法的分类

内部排序的方法很多，但就其全面性能而言，很难提出一种被认为是最好的方法，每一种方法都有各自的优缺点，适合在不同的环境（如记录的初始排列状态等）下使用。

内部排序的过程是一个逐步扩大记录的有序序列长度的过程。在排序的过程中，可以将排序记录区分为两个区域：有序序列区和无序序列区。

使有序区中记录的数目增加一个或几个的操作称为一趟排序。

根据逐步扩大记录有序序列长度的原则不同，可以将内部排序分为以下几类。

(1) 插入类：将无序子序列中的一个或几个记录“插入”到有序序列中，从而增加记录的有序子序列的长度。主要包括直接插入排序、折半插入排序和希尔排序。

(2) 交换类：通过“交换”无序序列中的记录从而得到其中关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。主要包括冒泡排序和快速排序。

(3) 选择类：从记录的无序子序列中“选择”关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。主要包括简单选择排序、树形选择排序和堆排序。

(4) 归并类：通过“归并”两个或两个以上的记录有序子序列，逐步增加记录有序序列的长度。2-路归并排序是最为常见的归并排序方法。

(5) 分配类：是唯一一类不需要进行关键字之间比较的排序方法，排序时主要利用分配和收集两种基本操作来完成。基数排序是主要的分配类排序方法。

## 8.1.3 待排序记录的存储方式

(1) 顺序表：记录之间的次序关系由其存储位置决定，实现排序需要移动记录。

(2) 链表：记录之间的次序关系由指针指示，实现排序不需要移动记录，仅需修改指针即可。这种排序方式称为链表排序。

(3) 待排序记录本身存储在一组地址连续的存储单元内，同时另设一个指示各个记录存储位置的地址向量，在排序过程中不移动记录本身，而移动地址向量中这些记录的“地址”，在排序结束之后再按照地址向量中的值调整记录的存储位置。这种排序方式称为地址排序。

在本章的讨论中，除基数排序外，待排序记录均按上述第一种方式存储，且为了讨论方便，设记录的关键字均为整数。

即在以后讨论的大部分算法中，待排序记录的数据类型定义为：

```
#define MAXSIZE 20                //顺序表的最大长度
typedef int KeyType;              //定义关键字类型为整型
typedef struct{
    KeyType key;                  //关键字项
    InfoType otherinfo;          //其他数据项
}RedType;                        //记录类型
typedef struct{
    RedType r[MAXSIZE+1];        //r[0]闲置或用做哨兵单元
    int length;                  //顺序表长度
}SqList;                         //顺序表类型
```

### 8.1.4 排序算法效率的评价指标

前面已指出，就排序方法的全面性能而言，很难提出一种被认为是最好的方法。目前，评价排序算法好坏的标准主要有两点。

#### (1) 执行时间

对于排序操作，时间主要消耗在关键字之间的比较和记录的移动上（这里，只考虑以顺序表方式存储待排序记录），排序算法的时间复杂度由这两个指标决定。因此可以认为，高效的排序算法的比较次数和移动次数都应该尽可能的少。

#### (2) 辅助空间

空间复杂度由排序算法所需的辅助空间决定。辅助空间是除了存放待排序记录占用的空间之外，执行算法所需要的其他存储空间。理想的空间复杂度为  $O(1)$ ，即算法执行期间所需要的辅助空间与待排序的数据量无关。

在以下各节讨论各种排序算法时，将给出有关算法的关键字比较次数和记录的移动次数。有的排序算法其执行时间不仅依赖于待排序的记录个数，还取决于待排序序列的初始状态。因此，对这样的算法，本书还将给出其最好、最坏和平均情况下的 3 种时间性能评价。

在讨论排序算法的平均执行时间时，均假定待排序记录初始状态是随机分布的，即出现各种排列情况的概率是相等的。同时假定各种排序的结果均是按关键字非递减排序。

## 8.2 插入排序

插入排序的基本思想是：每一趟将一个待排序的记录，按其关键字的大小插入到已经排好序的一组记录的适当位置上，直到所有待排序记录全部插入为止。

例如，打扑克牌在抓牌时要保证抓过的牌有序排列，则每抓一张牌，就插入到合适的位置，直到抓完牌为止，即可得到一个有序序列。

可以选择不同的方法在已排好序的记录中寻找插入位置。根据查找方法的不同，有多种插入排序方法，这里仅介绍三种方法：直接插入排序、折半插入排序和希尔排序。

### 8.2.1 直接插入排序

直接插入排序 (Straight Insertion Sort) 是一种最简单的排序方法，其基本操作是将一条记录

插入到已排好序的有序表中，从而得到一个新的、记录数量增 1 的有序表。

### 算法 8.1 直接插入排序

#### 【算法步骤】

① 设待排序的记录存放在数组  $r[1..n]$  中， $r[1]$  是一个有序序列。

② 循环  $n-1$  次，每次使用顺序查找法，查找  $r[i]$  ( $i = 2, \dots, n$ ) 在已排好序的序列  $r[1 \dots i-1]$  中的插入位置，然后将  $r[i]$  插入表长为  $i-1$  的有序序列  $r[1 \dots i-1]$ ，直到将  $r[n]$  插入表长为  $n-1$  的有序序列  $r[1 \dots n-1]$ ，最后得到一个表长为  $n$  的有序序列。

【例 8.1】 已知待排序记录的关键字序列为  $\{49, 38, 65, 97, 76, 13, 27, \overline{49}\}$ ，请给出用直接插入排序法进行排序的过程。

直接插入排序过程如图 8.1 所示，其中 ( ) 中为已排好序的记录的关键字。

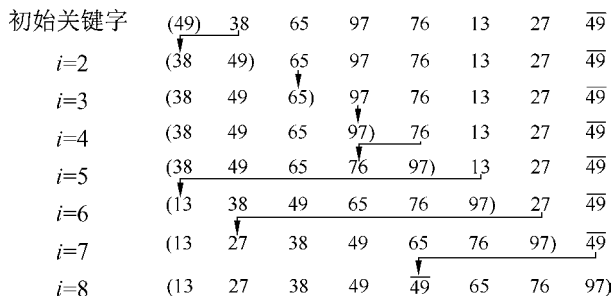


图 8.1 直接插入排序过程

在具体实现  $r[i]$  向前面的有序序列插入时，有两种方法：一种是将  $r[i]$  与  $r[1], r[2], \dots, r[i-1]$  从前向后顺序比较；另一种是将  $r[i]$  与  $r[i-1], r[i-2], \dots, r[1]$  从后向前顺序比较。这里采用后一种方法，和顺序查找类似，为了在查找插入位置的过程中避免数组下标出界，在  $r[0]$  处设置监视哨。在自  $i-1$  起往前查找插入位置的过程中，可以同时后移记录。

#### 【算法描述】

```
void InsertSort(Sqlist &L)
{ //对顺序表 L 做直接插入排序
    for(i=2; i<=L.length; ++i)
        if(L.r[i].key<L.r[i-1].key) // “<”，需将 r[i] 插入有序子表
        {
            L.r[0]=L.r[i]; //将待插入的记录暂存到监视哨中
            L.r[i]=L.r[i-1]; //r[i-1] 后移
            for(j=i-2; L.r[0].key<L.r[j].key; --j) //从后向前寻找插入位置
                L.r[j+1]=L.r[j]; //记录逐个后移，直到找到插入位置
            L.r[j+1]=L.r[0]; //将 r[0] 即原 r[i]，插入到正确位置
        } //if
}
```

#### 【算法分析】

##### (1) 时间复杂度

从时间来看，排序的基本操作为：比较两个关键字的大小和移动记录。

对于其中的某一趟插入排序，算法 8.1 中内层的 for 循环次数取决于待插记录的关键字与前  $i-1$  个记录的关键字之间的关系。其中，在最好情况（正序：待排序序列中记录按关键字非递减有序

排列)下,比较 1 次,不移动;在最坏情况(逆序:待排序序列中记录按关键字非递增有序排列)下,比较  $i$  次(依次同前面的  $i-1$  个记录进行比较,并和哨兵比较 1 次),移动  $i+1$  次(前面的  $i-1$  个记录依次向后移动,另外开始时将待插入的记录移动到监视哨中,最后找到插入位置,又从监视哨中移过去)。

对于整个排序过程需执行  $n-1$  趟,最好情况下,总的比较次数达最小值  $n-1$ ,记录不需移动;最坏情况下,总的关键字比较次数  $KCN$  和记录移动次数  $RMN$  均达到最大值,分别为

$$KCN = \sum_{i=2}^n i = (n+2)(n-1)/2 \approx n^2/2$$

$$RMN = \sum_{i=2}^n (i+1) = (n+4)(n-1)/2 \approx n^2/2$$

若待排序序列中出现各种可能排列的概率相同,则可取上述最好情况和最坏情况的平均情况。在平均情况下,直接插入排序关键字的比较次数和记录移动次数均约为  $n^2/4$ 。

由此,直接插入排序的时间复杂度为  $O(n^2)$ 。

## (2) 空间复杂度

直接插入排序只需要一个记录的辅助空间  $r[0]$ ,所以空间复杂度为  $O(1)$ 。

### 【算法特点】

- (1) 稳定排序。
- (2) 算法简便,且容易实现。
- (3) 也适用于链式存储结构,只是在单链表上无需移动记录,只需修改相应的指针。
- (4) 更适用于初始记录基本有序(正序)的情况,当初始记录无序, $n$  较大时,此算法时间复杂度较高,不宜采用。

## 8.2.2 折半插入排序

直接插入排序采用顺序查找法查找当前记录在已排好序的序列中的插入位置,从 7.2 节的讨论中可知,这个“查找”操作可利用“折半查找”来实现,由此进行的插入排序称之为折半插入排序(Binary Insertion Sort)。

### 算法 8.2 折半插入排序

#### 【算法步骤】

- ① 设待排序的记录存放在数组  $r[1 \cdots n]$  中, $r[1]$  是一个有序序列。
- ② 循环  $n-1$  次,每次使用折半查找法,查找  $r[i]$  ( $i=2, \cdots, n$ ) 在已排好序的序列  $r[1 \cdots i-1]$  中的插入位置,然后将  $r[i]$  插入表长为  $i-1$  的有序序列  $r[1 \cdots i-1]$ ,直到将  $r[n]$  插入表长为  $n-1$  的有序序列  $r[1 \cdots n-1]$ ,最后得到一个表长为  $n$  的有序序列。

#### 【算法描述】

```
void BInsertSort(SqList &L)
{ //对顺序表 L 做折半插入排序
    for (i=2; i<=L.length; ++i)
    {
        L.r[0]=L.r[i]; //将待插入的记录暂存到监视哨中
        low=1; high=i-1; //置查找区间初值
        while (low<=high) //在 r[low..high] 中折半查找插入的位置
```

```

{
    m=(low+high)/2;                //折半
    if(L.r[0].key<L.r[m].key) high=m-1;    //插入点在前一子表
    else low=m+1;                //插入点在后一子表
}
for(j=i-1;j>=high+1; --j) L.r[j+1]=L.r[j];    //记录后移
L.r[high+1]=L.r[0];                //将 r[0]即原 r[i], 插入到正确位置
}
//for
}

```

### 【算法分析】

#### (1) 时间复杂度

从时间上比较, 折半查找比顺序查找快, 所以就平均性能来说, 折半插入排序优于直接插入排序。

折半插入排序所需要的关键字比较次数与待排序序列的初始排列无关, 仅依赖于记录的个数。不论初始序列情况如何, 在插入第  $i$  个记录时, 需要经过  $\lfloor \log_2 i \rfloor + 1$  次比较, 才能确定它应插入的位置。所以当记录的初始排列为正序或接近正序时, 直接插入排序比折半插入排序执行的关键字比较次数要少。

折半插入排序的对象移动次数与直接插入排序相同, 依赖于对象的初始排列。

在平均情况下, 折半插入排序仅减少了关键字间的比较次数, 而记录的移动次数不变。因此, 折半插入排序的时间复杂度仍为  $O(n^2)$ 。

#### (2) 空间复杂度

折半插入排序所需附加存储空间和直接插入排序相同, 只需要一个记录的辅助空间  $r[0]$ , 所以空间复杂度为  $O(1)$ 。

### 【算法特点】

#### (1) 稳定排序。

#### (2) 因为要进行折半查找, 所以只能用于顺序结构, 不能用于链式结构。

#### (3) 适合初始记录无序、 $n$ 较大时的情况。

## 8.2.3 希尔排序

希尔排序 (Shell's Sort) 又称“缩小增量排序” (Diminishing Increment Sort), 是插入排序的一种, 因 D.L.Shell 于 1959 年提出而得名。

直接插入排序, 当待排序的记录个数较少且待排序序列的关键字基本有序时, 效率较高。希尔排序基于以上两点, 从“减少记录个数”和“序列基本有序”两个方面对直接插入排序进行了改进。

### 算法 8.3 希尔排序

#### 【算法步骤】

希尔排序实质上是采用分组插入的方法。先将整个待排序记录序列分割成几组, 从而减少参与直接插入排序的数据量, 对每组分别进行直接插入排序, 然后增加每组的数据量, 重新分组。这样当经过几次分组排序后, 整个序列中的记录“基本有序”时, 再对全体记录进行一次直接插入排序。

希尔对记录的分组, 不是简单地“逐段分割”, 而是将相隔某个“增量”的记录分成一组。

① 第一趟取增量  $d_1$  ( $d_1 < n$ ) 把全部记录分成  $d_1$  个组, 所有间隔为  $d_1$  的记录分在同一组, 在各个组中进行直接插入排序。

② 第二趟取增量  $d_2$  ( $d_2 < d_1$ ), 重复上述的分组和排序。

③ 依次类推, 直到所取的增量  $d_t = 1$  ( $d_t < d_{t-1} < \dots < d_2 < d_1$ ), 所有记录在同一组中进行直接插入排序为止。

【例 8.2】 已知待排序记录的关键字序列为 {49, 38, 65, 97, 76, 13, 27,  $\overline{49}$ , 55, 04}, 请给出用希尔排序法进行排序的过程。

希尔排序过程如图 8.2 所示。

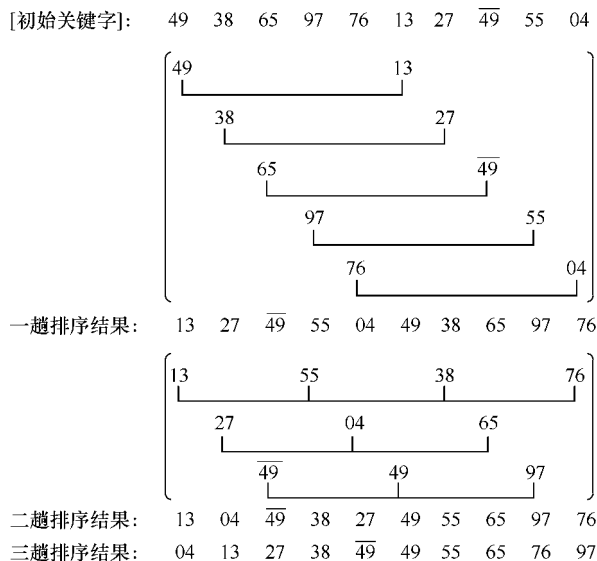


图 8.2 希尔排序过程

(1) 第一趟取增量  $d_1 = 5$ , 所有间隔为 5 的记录分在同一组, 全部记录分成 5 组, 在各个组中分别进行直接插入排序, 排序结果如图 8.2 的第 7 行所示。

(2) 第二趟取增量  $d_2 = 3$ , 所有间隔为 3 的记录分在同一组, 全部记录分成 3 组, 在各个组中分别进行直接插入排序, 排序结果如图 8.2 的第 11 行所示。

(3) 第三趟取增量  $d_3 = 1$ , 对整个序列进行一趟直接插入排序, 排序完成。

希尔排序的算法实现如算法 8.3 所示。预设好的增量序列保存在数组  $dt[0 \dots t-1]$  中, 整个希尔排序算法需执行  $t$  趟。从上述排序过程可见, 算法 8.1 中的直接插入排序可以看成一趟增量是 1 的希尔排序, 所以可以通过改写算法 8.1, 得到一趟希尔排序算法 ShellInsert。在 ShellInsert 中, 具体改写主要有两处:

(1) 前后记录位置的增量是  $dk$ , 而不是 1;

(2)  $r[0]$  只是暂存单元, 不是哨兵。当  $j \leq 0$  时, 插入位置已找到。

【算法描述】

```
void ShellInsert(SqList &L, int dk)
{ // 对顺序表 L 做一趟增量是 dk 的希尔插入排序
    for (i = dk + 1; i <= L.length; ++i)
    {
        if (L.r[i].key < L.r[i - dk].key) // 需将 L.r[i] 插入有序增量子表
        {
            L.r[0] = L.r[i]; // 暂存在 L.r[0]
            for (j = i - dk; j > 0 && L.r[0].key < L.r[j].key; j -= dk)
```

```

        L.r[j+dk]=L.r[j];           //记录后移，直到找到插入位置
        L.r[j+dk]=L.r[0];           //将 r[0] 即原 r[i]，插入到正确位置
    }                                //if

void ShellSort(SqList &L,int dt[],int t)
{//按增量序列 dt[0..t-1]对顺序表 L 作 t 趟希尔排序
    for(k=0;k<t;++k)
        ShellInsert(L,dt[k]);      //一趟增量为 dt[t] 的希尔插入排序
}

```

### 【算法分析】

#### (1) 时间复杂度

当增量大于 1 时，关键字较小的记录就不是一步一步地挪动，而是跳跃式地移动，从而使得在进行最后一趟增量为 1 的插入排序中，序列已基本有序，只要做记录的少量比较和移动即可完成排序，因此希尔排序的时间复杂度较直接插入排序低。但要具体分析，则是一个复杂的问题，因为希尔排序的时间复杂度是所取“增量”序列的函数，这涉及一些数学上尚未解决的难题。因此，到目前为止尚未有人求得一种最好的增量序列，但大量的研究已得出一些局部的结论。如有人指出，当增量序列为  $dt[k]=2^{t-k+1}-1$  时，希尔排序的时间复杂度为  $O(n^{3/2})$ ，其中  $t$  为排序趟数， $1 \leq k \leq t \leq \lfloor \log_2(n+1) \rfloor$ 。还有人在大量的实验基础上推出：当  $n$  在某个特定范围内，希尔排序所需的比较和移动次数约为  $n^{1.3}$ ，当  $n \rightarrow \infty$  时，可减少到  $n(\log_2 n)^2$ 。

#### (2) 空间复杂度

从空间来看，希尔排序和前面两种排序方法一样，也只需要一个辅助空间  $r[0]$ ，空间复杂度为  $O(1)$ 。

### 【算法特点】

(1) 记录跳跃式地移动导致排序方法是不稳定的。

(2) 只能用于顺序结构，不能用于链式结构。

(3) 增量序列可以有各种取法，但应该使增量序列中的值没有除 1 之外的公因子，并且最后一个增量值必须等于 1。

(4) 记录总的比较次数和移动次数都比直接插入排序要少， $n$  越大时，效果越明显。所以适合初始记录无序、 $n$  较大时的情况。

## 8.3 交换排序

交换排序的基本思想是：两两比较待排序记录的关键字，一旦发现两个记录不满足次序要求时则进行交换，直到整个序列全部满足要求为止。本节首先介绍基于简单交换思想实现的冒泡排序，然后给出另一种在此基础上进行改进的排序方法——快速排序。

### 8.3.1 冒泡排序

**冒泡排序** (Bubble Sort) 是一种最简单的交换排序方法，它通过两两比较相邻记录的关键字，如果发生逆序，则进行交换，从而使关键字小的记录如气泡一般逐渐往上“漂浮”（左移），或者

使关键字大的记录如石块一样逐渐向下“坠落”(右移)。

### 算法 8.4 冒泡排序

#### 【算法步骤】

① 设待排序的记录存放在数组  $r[1 \cdots n]$  中。首先将第一个记录的关键字和第二个记录的关键字进行比较,若为逆序(即  $L.r[1].key > L.r[2].key$ ),则交换两个记录。然后比较第二个记录和第三个记录的关键字。依次类推,直至第  $n-1$  个记录和第  $n$  个记录的关键字进行过比较为止。上述过程称作第一趟起泡排序,其结果使得关键字最大的记录被安置到最后一个记录的位置上。

② 然后进行第二趟起泡排序,对前  $n-1$  个记录进行同样操作,其结果是使关键字次大的记录被安置到第  $n-1$  个记录的位置上。

③ 重复上述比较和交换过程 第  $i$  趟是从  $L.r[1]$  到  $L.r[n-i+1]$  依次比较相邻两个记录的关键字,并在“逆序”时交换相邻记录,其结果是这  $n-i+1$  个记录中关键字最大的记录被交换到第  $n-i+1$  的位置上。直到在某一趟排序过程中没有进行过交换记录的操作,说明序列已全部达到排序要求,则完成排序。

【例 8.3】 已知待排序记录的关键字序列为  $\{49, 38, 65, 97, 76, 13, 27, \overline{49}\}$ , 请给出用冒泡排序法进行排序的过程。

冒泡排序过程如图 8.3 所示,算法如算法 8.4 所示。

|       |           |           |           |           |           |           |
|-------|-----------|-----------|-----------|-----------|-----------|-----------|
| 49    | 38        | 38        | 38        | 38        | 13        | <b>13</b> |
| 38    | 49        | 49        | 49        | 13        | 27        | <b>27</b> |
| 65    | 65        | 65        | 13        | 27        | 38        | <b>38</b> |
| 97    | 76        | 13        | 27        | 49        | <b>49</b> |           |
| 76    | 13        | 27        | 49        | <b>49</b> |           |           |
| 13    | 27        | <b>49</b> | <b>65</b> |           |           |           |
| 27    | <b>49</b> | <b>76</b> |           |           |           |           |
| 49    | <b>97</b> |           |           |           |           |           |
| 初始关键字 | 第一趟排序后    | 第二趟排序后    | 第三趟排序后    | 第四趟排序后    | 第五趟排序后    | 第六趟排序后    |

图 8.3 冒泡排序过程

待排序的记录总共有 8 个,但算法在第六趟排序过程中没有进行过交换记录的操作,则完成排序。

#### 【算法描述】

```
void BubbleSort(SqList &L)
{ // 对顺序表 L 做冒泡排序
    m=L.length-1; flag=1; // flag 用来标记某一趟排序是否发生交换
    while ((m>0) && (flag==1))
    {
        flag=0; // flag 置为 0, 如果本趟排序没有发生交换, 则不会执行下一趟排序
        for (j=1; j<=m; j++)
            if (L.r[j].key>L.r[j+1].key)
            {
```

```

        flag=1;                                //flag 置为 1, 表示本趟排序发生了交换
        t=L.r[j]; L.r[j]=L.r[j+1]; L.r[j+1]=t; //交换前后两个记录
    }                                           //if
    --m;
}                                              //while
}                                              //BubbleSort

```

### 【算法分析】

#### (1) 时间复杂度

最好情况（初始序列为正序）：只需进行一趟排序，在排序过程中进行  $n-1$  次关键字间的比较，且不移动记录。

最坏情况（初始序列为逆序）：需进行  $n-1$  趟排序，总的关键字比较次数  $KCN$  和记录移动次数  $RMN$ （每次交换都要移动 3 次记录）分别为

$$KCN = \sum_{i=n}^2 (i-1) = n(n-1)/2 \approx n^2/2$$

$$RMN = 3 \sum_{i=n}^2 (i-1) = 3n(n-1)/2 \approx 3n^2/2$$

所以，在平均情况下，冒泡排序关键字的比较次数和记录移动次数分别约为  $n^2/4$  和  $3n^2/4$ ，时间复杂度为  $O(n^2)$ 。

#### (2) 空间复杂度

冒泡排序只有在两个记录交换位置时需要一个辅助空间用做暂存记录，所以空间复杂度为  $O(1)$ 。

### 【算法特点】

(1) 稳定排序。

(2) 可用于链式存储结构。

(3) 移动记录次数较多，算法平均时间性能比直接插入排序差。当初始记录无序， $n$  较大时，此算法不宜采用。

## 8.3.2 快速排序

**快速排序**（Quick Sort）是由冒泡排序改进而得的。在冒泡排序过程中，只对相邻的两个记录进行比较，因此每次交换两个相邻记录时只能消除一个逆序。如果能通过两个（不相邻）记录的一次交换，消除多个逆序，则会大大加快排序的速度。快速排序方法中的一次交换可能消除多个逆序。

### 算法 8.5 快速排序

#### 【算法步骤】

在待排序的  $n$  个记录中任取一个记录（通常取第一个记录）作为枢轴（或支点），设其关键字为 **pivotkey**。经过一趟排序后，把所有关键字小于 **pivotkey** 的记录交换到前面，把所有关键字大于 **pivotkey** 的记录交换到后面，结果将待排序记录分成两个子表，最后将枢轴放置在分界处的位置。然后，分别对左、右子表重复上述过程，直至每一子表只有一个记录时，排序完成。

其中，一趟快速排序的具体步骤如下。

① 选择待排序表中的第一个记录作为枢轴, 将枢轴记录暂存在  $r[0]$  的位置上。附设两个指针  $low$  和  $high$ , 初始时分别指向表的下界和上界 (第一趟时,  $low = 1$ ;  $high = L.length$ )。

② 从表的最右侧位置依次向左搜索, 找到第一个关键字小于枢轴关键字  $pivotkey$  的记录, 将其移到  $low$  处。具体操作是: 当  $low < high$  时, 若  $high$  所指记录的关键字大于等于  $pivotkey$ , 则向左移动指针  $high$  (执行操作  $--high$ ); 否则将  $high$  所指记录移到  $low$  所指记录。

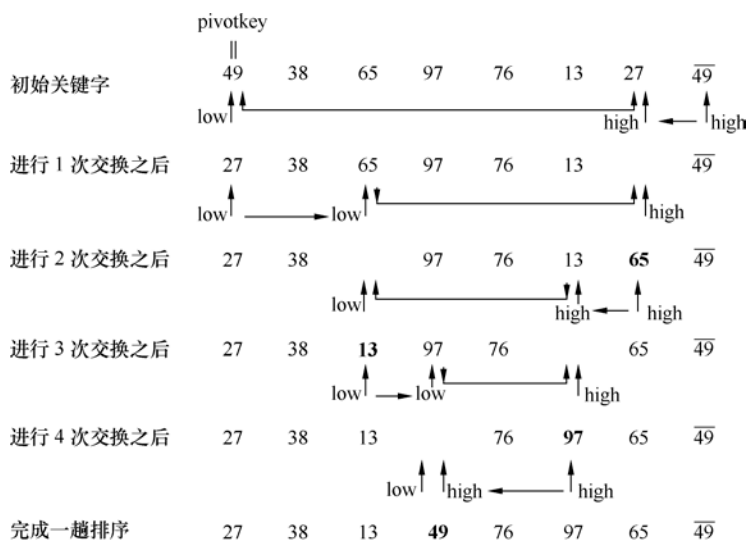
③ 然后再从表的最左侧位置, 依次向右搜索找到第一个关键字大于  $pivotkey$  的记录和枢轴记录交换。具体操作是: 当  $low < high$  时, 若  $low$  所指记录的关键字小于等于  $pivotkey$ , 则向右移动指针  $low$  (执行操作  $++low$ ); 否则将  $low$  所指记录与枢轴记录交换。

④ 重复步骤②和③, 直至  $low$  与  $high$  相等为止。此时  $low$  或  $high$  的位置即为枢轴在此趟排序中的最终位置, 原表被分成两个子表。

在上述过程中, 记录的交换都是与枢轴之间发生, 每次交换都要移动 3 次记录, 可以先将枢轴记录暂存在  $r[0]$  的位置上, 排序过程中只移动要与枢轴交换的记录, 即只做  $r[low]$  或  $r[high]$  的单向移动, 直至一趟排序结束后再将枢轴记录移至正确位置上。

【例 8.4】 已知待排序记录的关键字序列为  $\{49, 38, 65, 97, 76, 13, 27, \overline{49}\}$ , 请给出用快速排序法进行排序的过程。

第一趟快速排序过程如图 8.4 (a) 所示, 整个快速排序的过程如图 8.4 (b) 所示。



(a) 第一趟快速排序过程

|        |                                                                                            |
|--------|--------------------------------------------------------------------------------------------|
| 初始状态   | $\{49 \quad 38 \quad 65 \quad 97 \quad 76 \quad 13 \quad 27 \quad \overline{49}\}$         |
| 一趟排序结果 | $\{27 \quad 38 \quad 13\} \quad 49 \quad \{76 \quad 97 \quad 65 \quad \overline{49}\}$     |
| 二趟排序结果 | $\{13\} \quad 27 \quad \{38\} \quad 49 \quad \{76 \quad 97 \quad 65 \quad \overline{49}\}$ |
| 三趟排序结果 | $13 \quad 27 \quad 38 \quad 49 \quad \{\overline{49} \quad 65\} \quad 76 \quad \{97\}$     |
| 四趟排序结果 | $13 \quad 27 \quad 38 \quad 49 \quad \overline{49} \quad \{65\} \quad 76 \quad 97$         |

(b) 快速排序的全过程

图 8.4 快速排序过程

由上述可知, 整个快速排序的过程可递归进行, 其递归树如图 8.5 所示。快速排序的算法实现如算法 8.5 所示。其中, 算法 Partition 完成一趟快速排序, 返回枢轴的位置。若待排序序列长

度大于1 ( $low < high$ ), 算法 QuickSort 调用 Partition 获取枢轴位置, 然后递归执行分别对分割所得的两个子表进行排序。若待排序序列中只有一个记录, 递归结束, 排序完成。

### 【算法描述】

```
int Partition(SqList &L, int low, int high)
{ //对顺序表 L 中的子表 r[low..high] 进行一趟排序, 返回枢轴位置
    L.r[0]=L.r[low]; //用子表的第一个记录做枢轴记录
    pivotkey=L.r[low].key; //枢轴记录关键字保存在 pivotkey 中
    while(low<high) //从表的两端交替地向中间扫描
    {
        while(low<high&&L.r[high].key>=pivotkey) --high;
        L.r[low]=L.r[high]; //将比枢轴记录小的记录移到低端
        while(low<high&&L.r[low].key<=pivotkey) ++low;
        L.r[high]=L.r[low]; //将比枢轴记录大的记录移到高端
    } //while
    L.r[low]=L.r[0]; //枢轴记录到位
    return low; //返回枢轴位置
}
```

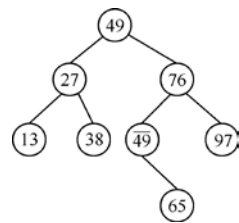


图 8.5 快速排序的递归树

```
void QSort(SqList &L, int low, int high)
{ //调用前置初值: low=1; high=L.length;
    //对顺序表 L 中的子序列 L.r[low..high] 做快速排序
    if(low<high) { //长度大于 1
        pivotloc=Partition(L, low, high); //将 L.r[low..high] 一分为二, pivotloc 是枢轴位置
        QSort(L, low, pivotloc-1); //对左子表递归排序
        QSort(L, pivotloc+1, high); //对右子表递归排序
    }
}

void QuickSort(SqList &L)
{ //对顺序表 L 做快速排序
    QSort(L, 1, L.length);
}
```

### 【算法分析】

#### (1) 时间复杂度

从快速排序算法的递归树可知, 快速排序的趟数取决于递归树的深度。

最好情况: 每一趟排序后都能将记录序列均匀地分割成两个长度大致相等的子表, 类似折半查找。在  $n$  个元素的序列中, 对枢轴定位所需时间为  $O(n)$ 。若设  $T(n)$  是对  $n$  个元素的序列进行排序所需的时间, 而且每次对枢轴正确定位后, 正好把序列划分为长度相等的两个子表, 此时, 设  $Cn$  是一个常数, 表示  $n$  个元素进行一趟快速排序的时间, 则总的排序时间为

$$\begin{aligned}
 T(n) &= Cn + 2T(n/2) \\
 &\leq n + 2T(n/2) \\
 &\leq n + 2(n/2 + 2T(n/4)) = 2n + 4T(n/4) \\
 &\leq 2n + 4(n/4 + 2T(n/8)) = 3n + 8T(n/8) \\
 &\dots\dots
 \end{aligned}$$

$$\leq kn + 2^k T(n/2^k)$$

$$\because k = \log_2 n$$

$$\therefore T(n) \leq n \log_2 n + n T(1) \approx O(n \log_2 n)$$

最坏情况：在待排序序列已经排好序的情况下，其递归树成为单支树，每次划分只得到一个比上一次少一个记录的子序列。这样，必须经过  $n-1$  趟才能将所有记录定位，而且第  $i$  趟需要经过  $n-i$  次比较。这样，总的关键字比较次数  $KCN$  为

$$KCN = \sum_{i=1}^{n-1} n-i = n(n-1)/2 \approx n^2/2$$

这种情况下，快速排序的速度已经蜕化到简单排序的水平。枢轴记录的合理选择可避免这种最坏情况的出现，如利用“三者取中”的规则：比较当前表中第一个记录、最后一个记录和中间一个记录的关键字，取关键字居中的记录作为枢轴记录，事先调换到第一个记录的位置。

理论上可以证明，平均情况下，快速排序的时间复杂度为  $O(n \log_2 n)$ 。

## (2) 空间复杂度

快速排序是递归的，执行时需要有一个栈来存放相应的数据。最大递归调用次数与递归树的深度一致，所以最好情况下的空间复杂度为  $O(\log_2 n)$ ，最坏情况下为  $O(n)$ 。

### 【算法特点】

- (1) 记录非顺次的移动导致排序方法是不稳定的。
- (2) 排序过程中需要定位表的下界和上界，所以适合用于顺序结构，很难用于链式结构。
- (3) 当  $n$  较大时，在平均情况下快速排序是所有内部排序方法中速度最快的一种，所以其适合初始记录无序、 $n$  较大时的情况。

## 8.4 选择排序

选择排序的基本思想是：每一趟从待排序的记录中选出关键字最小的记录，按顺序放在已排序的记录序列的最后，直到全部排完为止。本节首先介绍一种简单选择排序方法，然后给出另一种改进的选择排序方法——堆排序。

### 8.4.1 简单选择排序

简单选择排序 (Simple Selection Sort) 也称作直接选择排序。

#### 算法 8.6 简单选择排序

##### 【算法步骤】

- ① 设待排序的记录存放在数组  $r[1 \cdots n]$  中。第一趟从  $r[1]$  开始，通过  $n-1$  次比较，从  $n$  个记录中选出关键字最小的记录，记为  $r[k]$ ，交换  $r[1]$  和  $r[k]$ 。
- ② 第二趟从  $r[2]$  开始，通过  $n-2$  次比较，从  $n-1$  个记录中选出关键字最小的记录，记为  $r[k]$ ，交换  $r[2]$  和  $r[k]$ 。
- ③ 依次类推，第  $i$  趟从  $r[i]$  开始，通过  $n-i$  次比较，从  $n-i+1$  个记录中选出关键字最小的记录，记为  $r[k]$ ，交换  $r[i]$  和  $r[k]$ 。
- ④ 经过  $n-1$  趟，排序完成。

【例 8.5】 已知待排序记录的关键字序列为 {49,38,65,97, $\overline{49}$ ,13,27,76}, 给出用简单选择排序法进行排序的过程。

简单选择排序过程如图 8.6 所示, 其中()中为已排好序记录的关键字。

|        |             |                           |                 |                 |                 |    |    |    |
|--------|-------------|---------------------------|-----------------|-----------------|-----------------|----|----|----|
| 初始关键字  | 49          | 38                        | 65              | 97              | $\overline{49}$ | 13 | 27 | 76 |
| 一趟排序结果 | ( 13)       | 38                        | 65              | 97              | $\overline{49}$ | 49 | 27 | 76 |
| 二趟排序结果 | ( 13 27)    | 65                        | 97              | $\overline{49}$ | 49              | 38 | 76 |    |
| 三趟排序结果 | ( 13 27 38) | 97                        | $\overline{49}$ | 49              | 65              | 76 |    |    |
| 四趟排序结果 | ( 13 27 38  | $\overline{49}$ )         | 97              | 49              | 65              | 76 |    |    |
| 五趟排序结果 | ( 13 27 38  | $\overline{49}$ 49)       | 97              | 65              | 76              |    |    |    |
| 六趟排序结果 | ( 13 27 38  | $\overline{49}$ 49 65)    | 97              | 76              |                 |    |    |    |
| 七趟排序结果 | ( 13 27 38  | $\overline{49}$ 49 65 76) | 97              |                 |                 |    |    |    |

图 8.6 简单选择排序过程

### 【算法描述】

```
void SelectSort(Sqlist &L)
{//对顺序表 L 做简单选择排序
    for(i=1;i<L.length;++i){                //在 L.r[i..L.length] 中选择关键字最小的记录
        k=i;
        for(j=i+1;j<=L.length;++j)
            if(L.r[j].key<L.r[k].key) k=j;    //k 指向此趟排序中关键字最小的记录
        if(k!=i)
            {t=L.r[i]; L.r[i]=L.r[k]; L.r[k]=t;} //交换 r[i] 与 r[k]
    }                                          //for
}
```

### 【算法分析】

#### (1) 时间复杂度

简单选择排序过程中, 所需进行记录移动的次数较少。最好情况(正序): 不移动; 最坏情况(逆序): 移动  $3(n-1)$  次。

然而, 无论记录的初始排列如何, 所需进行的关键字间的比较次数相同, 均为

$$KCN = \sum_{i=1}^{n-1} n-i = n(n-1)/2 \approx n^2/2$$

因此, 简单选择排序的时间复杂度也是  $O(n^2)$ 。

#### (2) 空间复杂度

同冒泡排序一样, 只有在两个记录交换时需要一个辅助空间, 所以空间复杂度为  $O(1)$ 。

### 【算法特点】

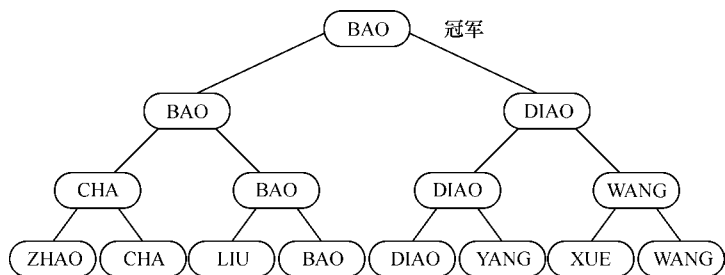
(1) 就选择排序方法本身来讲, 它是一种稳定的排序方法, 但图 8.6 所表现出来的现象是不稳定的, 这是因为上述实现选择排序的算法采用“交换记录”的策略所造成的, 改变这个策略, 可以写出不产生“不稳定现象”的选择排序算法。

(2) 可用于链式存储结构。

(3) 移动记录次数较少, 当每一记录占用的空间较多时, 此方法比直接插入排序快。

从上述可见, 选择排序的主要操作是进行关键字间的比较, 因此改进简单选择排序应从如何减少“比较”出发考虑。显然, 在  $n$  个关键字中选出最小值, 至少要进行  $n-1$  次比较, 然而, 继续在

剩余的  $n-1$  个关键字中选择次小值并非一定要进行  $n-2$  次比较, 若能利用前  $n-1$  次比较所得信息, 则可减少以后各趟选择排序中所用的比较次数。实际上, 体育比赛中的锦标赛便是一种选择排序。例如, 在 8 个运动员中决出前 3 名至多需要 11 场比赛, 而不是  $7+6+5=18$  场比赛 (它的前提是, 若乙胜丙, 甲胜乙, 则认为甲必能胜丙)。例如, 图 8.7 (a) 中最低层的叶子结点中 8 个选手之间经过第一轮 4 场比赛之后选拔出 4 个优胜者 “CHA”、“BAO”、“DIAO” 和 “WANG”, 然后经过两场半决赛和一场决赛之后, 选拔出冠军 “BAO”。显然, 按照锦标赛的传递关系, 亚军只能产生于分别在决赛、半决赛和第一轮比赛中输给冠军的选手中。由此, 在经过 “CHA” 和 “LIU”、“CHA” 和 “DIAO” 的两场比赛之后, 选拔出亚军 “CHA”, 同理, 选拔季军的比赛只要在 “ZHAO”、“LIU” 和 “DIAO” 3 个选手之间进行即可。按照这种锦标赛的思想可导出树形选择排序。



的思想进行选择排序的方法。首先对  $n$  个记录的关键字进行两两比较, 然后在其中  $\left\lceil \frac{n}{2} \right\rceil$  个较小者之间再进行两两比较, 如此重复, 直至选出最小关键字的记录为止。这个过程可用一棵有  $n$  个叶子结点的完全二叉树表示。例如, 图 8.8 (a) 中的二叉树表示从 8 个关键字中选出最小关键字的过程。8 个叶子结点中依次存放排序之前的 8 个关键字, 每个非终端结点中的关键字均等于其左、右孩子结点中较小的关键字, 则根结点中的关键字即为叶子结点中的最小关键字。在输出最小关键字之后, 根据关系的可传递性, 欲选出次小关键字, 仅需将叶子结点中的最小关键字 (13) 改为“最大值”, 然后从该叶子结点开始, 和其左 (或右) 兄弟的关键字进行比较, 修改从叶子结点到根的路径上各结点的关键字, 则根结点的关键字即为次小关键字。同理, 可依次选出从小到大的所有关键字 (参见图 8.8 (b) 和 (c))。由于含有  $n$  个叶子结点的完全二叉树的深度为  $\lceil \log_2 n \rceil + 1$ , 则在树形选择排序中, 除了最小关键字之外, 每选择一个次小关键字仅需进行  $\lceil \log_2 n \rceil$  次比较, 因此, 它的时间复杂度为  $O(n \log_2 n)$ 。但是, 这种排序方法尚有辅助存储空间较多、和“最大值”进行多余的比较等缺点。为了改进这个缺点, 威洛姆斯 (J. Williams) 在 1964 提年出了另一种形式的选择排序——堆排序。

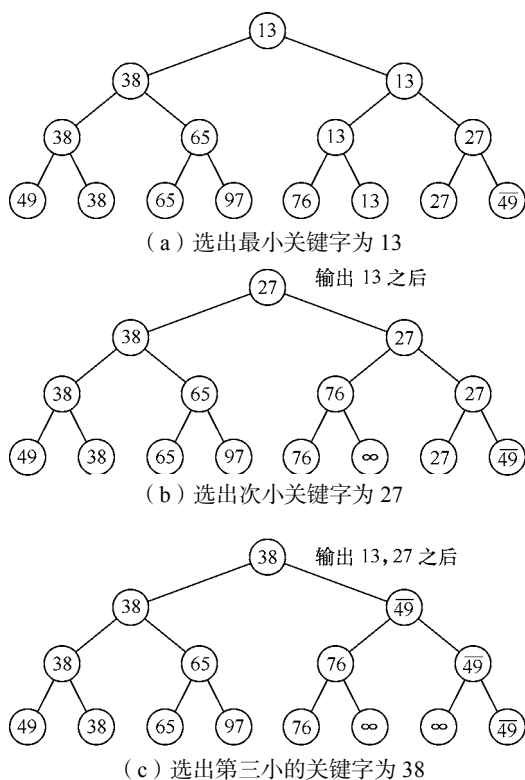


图 8.8 树形选择排序示例

### 8.4.3 堆排序

**堆排序 (Heap Sort)** 是一种树形选择排序, 在排序过程中, 将待排序的记录  $r[1..n]$  看成是一棵完全二叉树的顺序存储结构, 利用完全二叉树中双亲结点和孩子结点之间的内在关系, 在当前无序的序列中选择关键字最大 (或最小) 的记录。

首先给出堆的定义。

$n$  个元素的序列  $\{k_1, k_2, \dots, k_n\}$  称之为堆, 当且仅当满足以下条件时:

$$(1) \ k_i \geq k_{2i} \text{ 且 } k_i \geq k_{2i+1} \quad \text{或} \quad (2) \ k_i \leq k_{2i} \text{ 且 } k_i \leq k_{2i+1} \quad (1 \leq i \leq \lfloor n/2 \rfloor)$$

若将和此序列对应的一维数组 (即以一维数组做此序列的存储结构) 看成是一个完全二叉树, 则堆实质上是满足如下性质的完全二叉树: 树中所有非终端结点的值均不大于 (或不小于) 其左、右孩子结点的值。

例如, 关键字序列  $\{96, 83, 27, 38, 11, 09\}$  和  $\{12, 36, 24, 85, 47, 30, 53, 91\}$  分别满足条件 (1) 和条件 (2), 故它们均为堆, 对应的完全二叉树分别如图 8.9 (a) 和 (b) 所示。显然, 在这两种堆中, 堆顶元素 (或完全二叉树的根) 必为序列中  $n$  个元素的最大值 (或最小值), 分别称之为大根堆和小根堆。

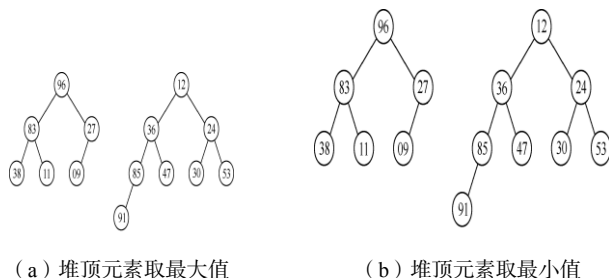


图 8.9 堆的示例

堆排序利用了大根堆 (或小根堆) 堆顶记录的关键字最大 (或最小) 这一特征, 使得当前无序的序列中选择关键字最大 (或最小) 的记录变得简单。下面讨论用大根堆进行排序, 堆排序的步骤如下。

① 按堆的定义将待排序序列  $r[1..n]$  调整为大根堆 (这个过程称为建初堆), 交换  $r[1]$  和  $r[n]$ , 则  $r[n]$  为关键字最大的记录。

② 将  $r[1..n-1]$  重新调整为堆, 交换  $r[1]$  和  $r[n-1]$ , 则  $r[n-1]$  为关键字次大的记录。

③ 循环  $n-1$  次, 直到交换了  $r[1]$  和  $r[2]$  为止, 得到了一个非递减的有序序列  $r[1..n]$ 。

同样, 可以通过构造小根堆得到一个非递增的有序序列。

由此, 实现堆排序需要解决如下两个问题。

(1) 建初堆: 如何将一个无序序列建成一个堆?

(2) 调整堆: 去掉堆顶元素, 在堆顶元素改变之后, 如何调整剩余元素成为一个新的堆?

因为建初堆要用到调整堆的操作, 所以下面先讨论调整堆的实现。

### 1. 调整堆

先看一个例子, 图 8.10 (a) 是个堆, 将堆顶元素 97 和堆中最后一个元素 38 交换后, 如图 8.10 (b) 所示。由于此时除根结点外, 其余结点均满足堆的性质, 由此仅需自上至下进行一条路径上的结点调整即可。首先以堆顶元素 38 和其左、右子树根结点的值进行比较, 由于左子树根结点的值大于右子树根结点的值且大于根结点的值, 则将 38 和 76 交换; 由于 38 替代了 76 之后破坏了左子树的“堆”, 则需进行和上述相同的调整, 直至叶子结点, 调整后的状态如图 8.10 (c) 所示。重复上述过程, 将堆顶元素 76 和堆中最后一个元素 27 交换且调整, 得到如图 8.10 (d) 所示新的堆。

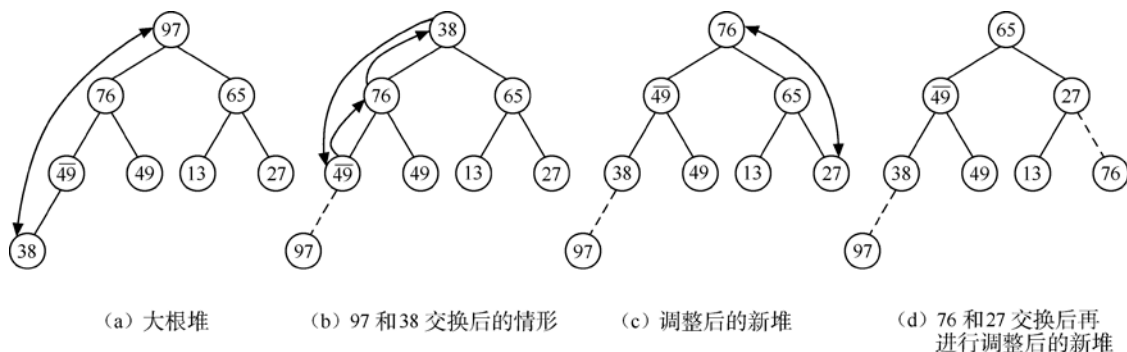


图 8.10 堆顶元素改变后调整堆的过程

上述过程就像过筛子一样，把较小的关键字逐层筛下去，而将较大的关键字逐层选上来。因此，称此方法为“筛选法”。

假设  $r[s+1..m]$  已经是堆的情况下，按“筛选法”将  $r[s..m]$  调整为以  $r[s]$  为根的堆，算法实现如下。

#### 算法 8.7 筛选法调整堆

##### 【算法步骤】

从  $r[2s]$  和  $r[2s+1]$  中选出关键字较大者，假设  $r[2s]$  的关键字较大，比较  $r[s]$  和  $r[2s]$  的关键字。

① 若  $r[s].key \geq r[2s].key$ ，说明以  $r[s]$  为根的子树已经是堆，不必做任何调整。

② 若  $r[s].key < r[2s].key$ ，交换  $r[s]$  和  $r[2s]$ 。交换后，以  $r[2s+1]$  为根的子树仍是堆，如果以  $r[2s]$  为根的子树不是堆，则重复上述过程，将以  $r[2s]$  为根的子树调整为堆，直至进行到叶子结点为止。

##### 【算法描述】

```
void HeapAdjust(SqList &L, int s, int m)
{//假设  $r[s+1..m]$  已经是堆，将  $r[s..m]$  调整为以  $r[s]$  为根的大根堆
    rc=L.r[s];
    for(j=2*s; j<=m; j*=2)                //沿 key 较大的孩子结点向下筛选
    {
        if(j<m&&L.r[j].key<L.r[j+1].key) ++j; //j 为 key 较大的记录的下标
        if(rc.key>=L.r[j].key) break;          //rc 应插入在位置 s 上
        L.r[s]=L.r[j]; s=j;
    }
    L.r[s]=rc;                             //插入
}
```

## 2. 建初堆

要将一个无序序列调整为堆，就必须将其所对应的完全二叉树中以每一结点为根的子树都调整为堆。显然，只有一个结点的树必是堆，而在完全二叉树中，所有序号大于  $\lfloor n/2 \rfloor$  的结点都是叶子，因此以这些结点为根的子树均已堆。这样，只需利用筛选法，从最后一个分支结点  $\lfloor n/2 \rfloor$  开始，依次将序号为  $\lfloor n/2 \rfloor$ 、 $\lfloor n/2 \rfloor - 1 \cdots 1$  的结点作为根的子树都调整为堆即可。

#### 算法 8.8 建初堆

##### 【算法步骤】

对于无序序列  $r[1 \cdots n]$ ，从  $i = n/2$  开始，反复调用筛选法  $\text{HeapAdjust}(L, i, n)$ ，依次将以  $r[i]$ ， $r[i-1]$ ， $\cdots$ ， $r[1]$  为根的子树调整为堆。

## 【算法描述】

```

void CreatHeap(SqList &L)
{ //把无序序列 L.r[1..n] 建成大根堆
    n=L.length;
    for (i=n/2; i>0; --i)           //反复调用 HeapAdjust
        HeapAdjust(L, i, n);
}

```

【例 8.6】 已知无序序列为 {49, 38, 65, 97, 76, 13, 27, 49}，用“筛选法”将其调整为一个大根堆，给出建堆的过程。

从图 8.11 (a) 所示的无序序列的最后一个非终端结点开始筛选，即从第 4 个元素 97 开始，由于  $97 > 49$ ，则无需交换。同理，第 3 个元素 65 不小于其左、右子树根的值，仍无需交换。而第 2 个元素  $38 < 97$ ，被筛选之后序列的状态如图 8.11 (b) 所示，然后对根元素 49 筛选之后得到图 8.11 (c) 所示的大根堆。

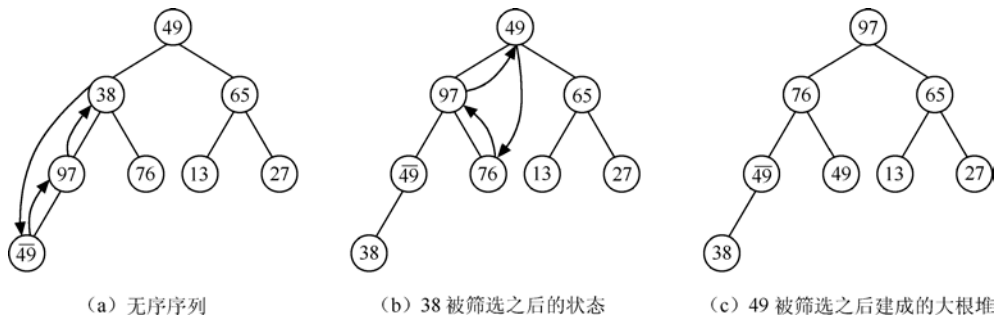


图 8.11 建初堆的过程

## 3. 堆排序算法的实现

根据前面堆排序算法步骤的描述，可知堆排序就是将无序序列建成初堆以后，反复进行交换和堆调整。在建初堆和调整堆算法实现的基础上，下面给出堆排序算法的实现。

## 算法 8.9 堆排序

## 【算法描述】

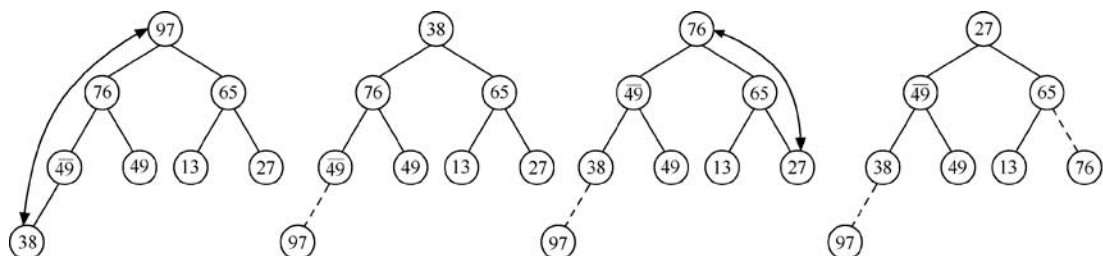
```

void HeapSort(SqList &L)
{ //对顺序表 L 进行堆排序
    CreatHeap(L);           //把无序序列 L.r[1..L.length] 建成大根堆
    for (i=L.length; i>1; --i)
    {
        x=L.r[1];           //将堆顶记录和当前未经排序子序列 L.r[1..i] 中最后一个记录互换
        L.r[1]=L.r[i];
        L.r[i]=x;
        HeapAdjust(L, 1, i-1); //将 L.r[1..i-1] 重新调整为大根堆
    }
    //for
}

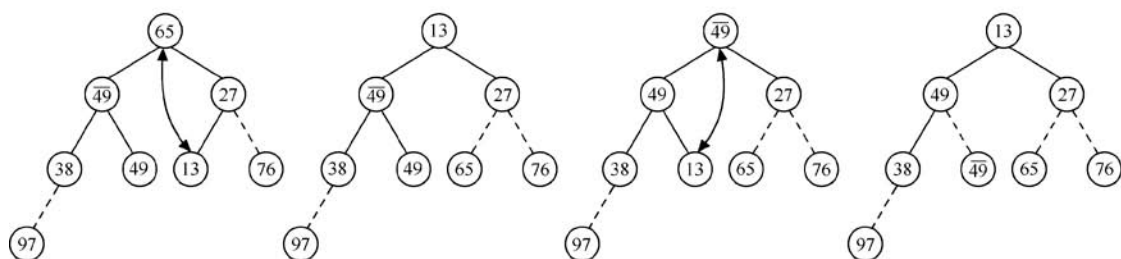
```

【例 8.7】 已知待排序记录的关键字序列为 {49, 38, 65, 97, 76, 13, 27, 49}，给出用堆排序法进行排序的过程。

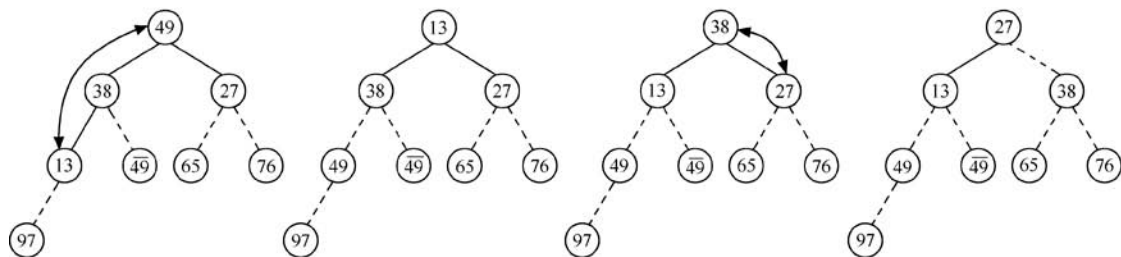
首先将无序序列建初堆，过程如图 8.11 所示。在初始大根堆的基础上，反复交换堆顶元素和最后一个元素，然后重新调整堆，直至最后得到一个有序序列，整个堆排序过程如图 8.12 所示。



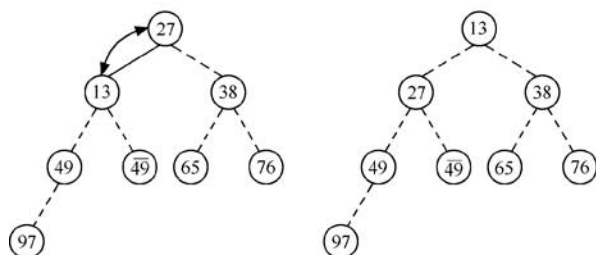
(a) 初始大根堆 $r[1..8]$  (b) 第一趟排序的交换操作之后 (c) 第一趟重调整堆 $r[1..7]$ 之后 (d) 第二趟排序的交换操作之后



(e) 第二趟重调整堆 $r[1..6]$ 之后 (f) 第三趟排序的交换操作之后 (g) 第三趟重调整堆 $r[1..5]$ 之后 (h) 第四趟排序的交换操作之后



(i) 第四趟重调整堆 $r[1..4]$ 之后 (j) 第五趟排序的交换操作之后 (k) 第五趟重调整堆 $r[1..3]$ 之后 (l) 第六趟排序的交换操作之后



(m) 第六趟重调整堆 $r[1..2]$ 之后 (n) 第七趟排序的交换操作之后

图 8.12 堆排序的过程

### 【算法分析】

#### (1) 时间复杂度

堆排序的运行时间主要耗费在建初堆和调整堆时进行的反复“筛选”上。

设有  $n$  个记录的初始序列所对应的完全二叉树的深度为  $h$ ，建初堆时，每个非终端结点都要自上而下进行“筛选”。由于第  $i$  层上的结点数小于等于  $2^{i-1}$ ，且第  $i$  层结点最大下移的深度为  $h-i$ ，

每下移一层要做两次比较, 所以建初堆时关键字总的比较次数为

$$\sum_{i=h-1}^1 2^{i-1} \cdot 2(h-i) = \sum_{i=h-1}^1 2^i \cdot (h-i) = \sum_{j=1}^{h-1} 2^{h-j} \cdot j \leq 2n \sum_{j=1}^{h-1} j/2^j \approx 4n$$

调整建新堆时要做  $n-1$  次“筛选”, 每次“筛选”都要将根结天下移到合适的位置。 $n$  个结点的完全二叉树的深度为  $\lfloor \log_2 n \rfloor + 1$ , 则重建堆时关键字总的比较次数不超过

$$2(\lfloor \log_2(n-1) \rfloor + \lfloor \log_2(n-2) \rfloor + \cdots + \log_2 2) < 2n(\lfloor \log_2 n \rfloor)$$

由此, 堆排序在最坏的情况下, 其时间复杂度也为  $O(n \log_2 n)$ 。

实验研究表明, 平均性能接近于最坏性能。

## (2) 空间复杂度

仅需一个记录大小供交换用的辅助存储空间, 所以空间复杂度为  $O(1)$ 。

## 【算法特点】

(1) 是不稳定排序。

(2) 只能用于顺序结构, 不能用于链式结构。

(3) 初始建堆所需的比较次数较多, 因此记录数较少时不宜采用。堆排序在最坏情况下时间复杂度为  $O(n \log_2 n)$ , 相对于快速排序最坏情况下的  $O(n^2)$  而言是一个优点, 当记录较多时较为高效。

# 8.5 归 并 排 序

**归并排序** (Merging Sort) 就是将两个或两个以上的有序表合并成一个有序表的过程。将两个有序表合并成一个有序表的过程称为 **2-路归并**, 2-路归并最为简单和常用。下面以 2-路归并为例, 介绍归并排序算法。

归并排序算法的思想是:

假设初始序列含有  $n$  个记录, 则可看成是  $n$  个有序的子序列, 每个子序列的长度为 1, 然后两两归并, 得到  $\lceil n/2 \rceil$  个长度为 2 或 1 的有序子序列; 再两两归并, …… , 如此重复, 直至得到一个长度为  $n$  的有序序列为止。

**【例 8.8】** 已知待排序记录的关键字序列为 {49, 38, 65, 97, 76, 13, 27}, 给出用 2-路归并排序法进行排序的过程。

2-路归并排序的过程如图 8.13 所示。

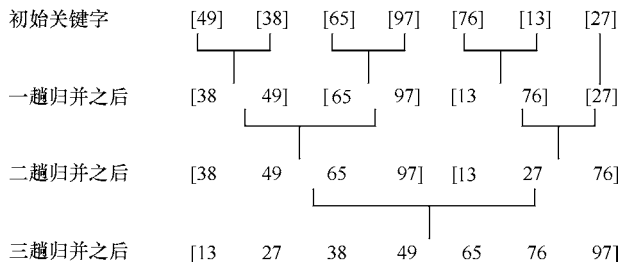


图 8.13 2-路归并排序过程

2-路归并排序中的核心操作是, 将待排序序列中前后相邻的两个有序序列归并为一个有序序列, 其算法类似于第 2 章的算法 2.16。

### 算法 8.10 相邻两个有序子序列的归并

#### 【算法步骤】

设两个有序表存放在同一数组中相邻的位置上： $R[\text{low}..\text{mid}]$ 和 $R[\text{mid}+1..\text{high}]$ ，每次分别从两个表中取出一个记录进行关键字的比较，将较小者放入 $T[\text{low}..\text{high}]$ 中，重复此过程，直至其中一个表为空，最后将另一非空表中余下的部分直接复制到 $T$ 中。

#### 【算法描述】

```
void Merge(RedType R[], RedType &T[], int low, int mid, int high)
{ //将有序表 R[low..mid] 和 R[mid+1..high] 归并为有序表 T[low..high]
    i=low; j=mid+1; k=low;
    while(i<=mid&&j<=high)                //将 R 中记录由小到大并入 T 中
    {
        if(R[i].key<=R[j].key) T[k]=R[i++];
        else T[k]=R[j++];
    }
    while(i<=mid) T[k++]=R[i++];           //将剩余的 R[low..mid] 复制到 T 中
    while(j<=high) T[k++]=R[j++];         //将剩余的 R[j..high] 复制到 T 中
}
```

假设每个子序列的长度为 $h$ ，则一趟归并排序需调用 $\lceil n/2h \rceil$ 次算法 merge 进行两两归并，得到前后相邻、长度为 $2h$ 的有序段，整个归并排序需进行 $\lceil \log_2 n \rceil$ 趟。

与快速排序类似，2-路归并排序也可以利用划分为子序列的方法递归实现。首先把整个待排序序列划分为两个长度大致相等的子序列，对这两个子序列分别递归地进行排序，然后再把它们归并。

### 算法 8.11 归并排序

#### 【算法步骤】

2-路归并排序将 $R[\text{low}..\text{high}]$ 中的记录归并排序后放入 $T[\text{low}..\text{high}]$ 中。当序列长度等于1时，递归结束，否则：

- ① 将当前序列一分为二，求出分裂点 $\text{mid} = \lfloor (\text{low}+\text{high})/2 \rfloor$ ；
- ② 对子序列 $R[\text{low}..\text{mid}]$ 递归，进行归并排序，结果放入 $S[\text{low}..\text{mid}]$ 中；
- ③ 对子序列 $R[\text{mid}+1..\text{high}]$ 递归，进行归并排序，结果放入 $S[\text{mid}+1..\text{high}]$ 中；
- ④ 调用算法 Merge，将有序的两个子序列 $S[\text{low}..\text{mid}]$ 和 $S[\text{mid}+1..\text{high}]$ 归并为一个有序的序列 $T[\text{low}..\text{high}]$ 。

#### 【算法描述】

```
void MSort(RedType R[], RedType &T[], int low, int high)
{ //R[low..high] 归并排序后放入 T[low..high] 中
    if(low==high) T[low]=R[low];
    else
    {
        mid=(low+high)/2;    //将当前序列一分为二，求出分裂点 mid
        MSort(R, S, low, mid); //对子序列 R[low..mid] 递归归并排序，结果放入 S[low..mid]
        MSort(R, S, mid+1, high); //对子序列 R[mid+1..high] 递归归并排序，结果放入 S[mid+1..high]
        Merge(S, T, low, mid, high); //将 S[low..mid] 和 S[mid+1..high] 归并到 T[low..high]
    }
    //else
}

void MergeSort(Sqlist &L)
{ //对顺序表 L 做归并排序
    MSort(L.r, L.r, 1, L.length);
}
```

### 【算法分析】

#### (1) 时间复杂度

当有  $n$  个记录时, 需进行  $\lceil \log_2 n \rceil$  趟归并排序, 每一趟归并, 其关键字比较次数不超过  $n$ , 元素移动次数都是  $n$ , 因此, 归并排序的时间复杂度为  $O(n \log_2 n)$ 。

#### (2) 空间复杂度

用顺序表实现归并排序时, 需要和待排序记录个数相等的辅助存储空间, 所以空间复杂度为  $O(n)$ 。

### 【算法特点】

#### (1) 是稳定排序。

#### (2) 可用于链式结构, 且不需要附加存储空间, 但递归实现时仍需要开辟相应的递归工作栈。

## 8.6 基数排序

前述各类排序方法都是建立在关键字比较的基础上, 而分配类排序不需要比较关键字的大小, 它是根据关键字中各位的值, 通过对待排序记录进行若干趟“分配”与“收集”来实现排序的, 是一种借助于多关键字排序的思想对单关键字排序的方法。**基数排序 (Radix Sorting)** 是典型的分配类排序。

### 8.6.1 多关键字的排序

先看一个具体例子。

已知扑克牌中 52 张牌面的次序关系为

$\clubsuit 2 < \clubsuit 3 < \dots < \clubsuit A < \diamond 2 < \diamond 3 < \dots < \diamond A < \heartsuit 2 < \heartsuit 3 < \dots < \heartsuit A < \spadesuit 2 < \spadesuit 3 < \dots < \spadesuit A$

每一张牌有两个“关键字”: 花色 ( $\clubsuit < \diamond < \heartsuit < \spadesuit$ ) 和面值 ( $2 < 3 < \dots < A$ ), 且“花色”的地位高于“面值”, 在比较任意两张牌面的大小时, 必须先比较“花色”, 若“花色”相同, 则再比较面值。

由此, 将扑克牌整理成如上所述次序关系时, 有以下两种排序法。

(1) **最高位优先法**: 先按不同“花色”分成有次序的 4 堆, 每一堆的牌均具有相同的“花色”, 然后分别对每一堆按“面值”大小整理有序。

(2) **最低位优先法**: 这是一种“分配”与“收集”交替进行的方法。先按不同“面值”分成 13 堆, 然后将这 13 堆牌自小至大叠在一起 (“3”在“2”之上, “4”在“3”之上, ……, 最上面的是 4 张“A”), 然后将每堆按照面值的次序收集到一起。再重新对这些牌按不同“花色”分成 4 堆, 最后将这 4 堆牌按花色的次序再收集到一起 ( $\clubsuit$  在最下面,  $\spadesuit$  在最上面), 此时同样得到一副满足如上次序关系的牌, 如图 8.14 所示。

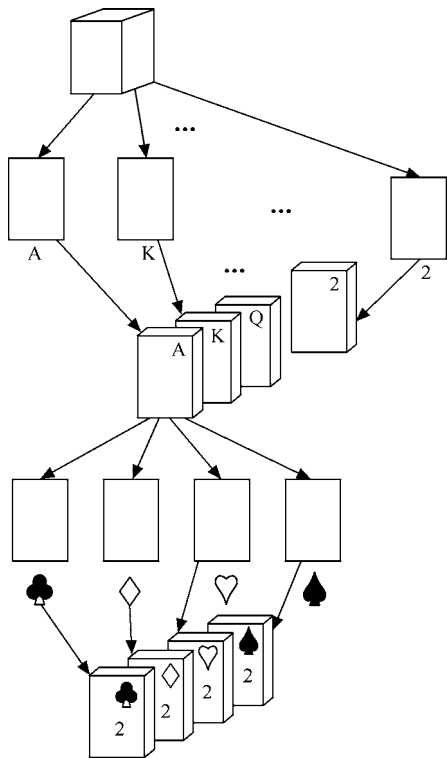


图 8.14 扑克牌的一种洗牌过程

## 8.6.2 链式基数排序

基数排序的思想类似于上述“最低位优先法”的洗牌过程，是借助“分配”和“收集”两种操作对单逻辑关键字进行排序的一种内部排序方法。有的逻辑关键字可以看成由若干个关键字复合而成的。例如，若关键字是数值，且其值都在  $0 \leq K \leq 999$  范围内，则可把每一个十进制数字看成一个关键字，即可认为  $K$  由 3 个关键字 ( $K^0, K^1, K^2$ ) 组成，其中  $K^0$  是百位数， $K^1$  是十位数， $K^2$  是个位数；又若关键字  $K$  是由 5 个字母组成的单词，则可看成是由 5 个关键字 ( $K^0, K^1, K^2, K^3, K^4$ ) 组成，其中  $K^{j-1}$  是（自左至右的）第  $j$  个字母。由于如此分解而得的每个关键字  $K^j$  都在相同的范围内（对数字， $0 \leq K^j \leq 9$ ，对字母‘A’ $\leq K^j \leq$ ‘Z’），故可以按照“分配”和“收集”的方法进行排序。

假设记录的逻辑关键字由  $d$  个“关键字”组成，每个关键字可能取  $rd$  个值。只要从最低数位关键字起，按关键字的不同值将序列中记录“分配”到  $rd$  个队列中后再“收集”之，如此重复  $d$  次完成排序。按这种方法实现排序称之为**基数排序**，其中“基”指的是  $rd$  的取值范围，在上述两种关键字的情况下， $rd$  分别为“10”和“26”。

具体实现时，一般采用链式基数排序。

先看一个具体例子。首先以链表存储  $n$  个待排记录，并令表头指针指向第一个记录，如图 8.15 (a) 所示，然后通过以下三趟“分配”和“收集”操作来完成排序。

第一趟分配对最低数位关键字（个位数）进行，改变记录的指针值将链表中的记录分配至 10 个链队列中去，每个队列中的记录关键字的个位数相等，如图 8.15 (b) 所示，其中  $f[i]$  和  $e[i]$  分别为第  $i$  个队列的头指针和尾指针；第一趟收集是改变所有非空队列的队尾记录的指针域，令其指向下一个非空队列的队头记录，重新将 10 个队列中的记录链成一个链表，如图 8.15 (c) 所示。

第二趟分配和第二趟收集是对十位数进行的，其过程和个位数相同。分配和收集结果分别如图 8.15 (d) 和图 8.15 (e) 所示。

第三趟分配和第三趟收集是对百位数进行的，过程同上，分配和收集结果分别如图 8.15 (f) 和图 8.15 (g) 所示。至此排序完毕。

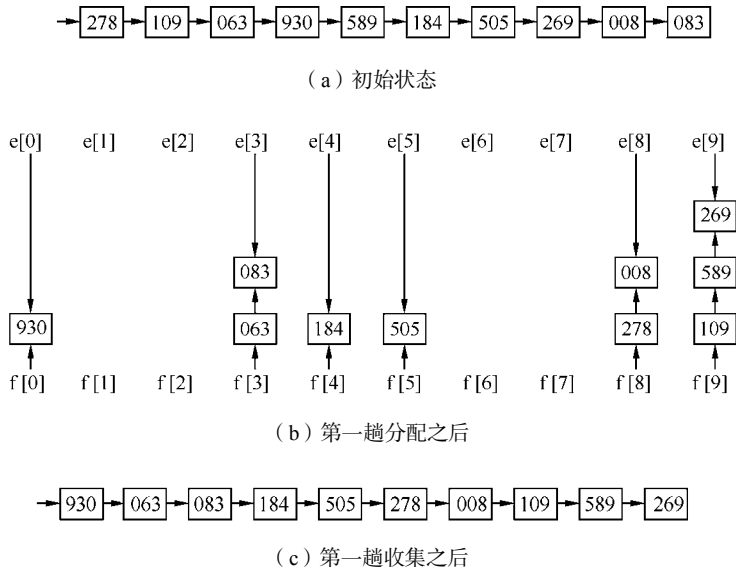
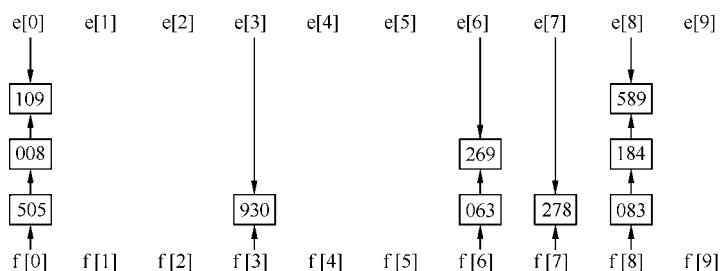
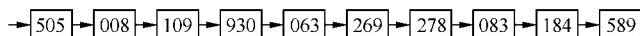


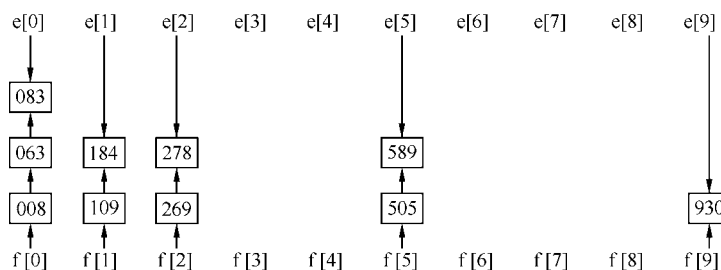
图 8.15 链式基数排序过程



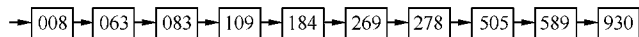
(d) 第二趟分配之后



(e) 第二趟收集之后



(f) 第三趟分配之后



(g) 第三趟收集之后的有序序列

图 8.15 链式基数排序过程 (续)

算法实现时采用静态链表，以便于更有效地存储和重排记录。相关数据类型的定义如下：

```
#define MAXNUM_KEY 8 //关键字项数的最大值
#define RADIX 10 //关键字基数，此时是十进制整数的基数
#define MAX_SPACE 10000
typedef struct
{
    KeysType keys[MAXNUM_KEY]; //关键字
    InfoType otheritems; //其他数据项
    int next;
}SLCell; //静态链表的结点类型
typedef struct
{
    SLCell r[MAX_SPACE]; //静态链表的可利用空间，r[0]为头结点
    int keynum; //记录的当前关键字个数
    int recnum; //静态链表的当前长度
}SLList; //静态链表类型
typedef int ArrType[RADIX] //指针数组类型
```

## 算法 8.12 基数排序

### 【算法描述】

```

void Distribute(SLCell &r,int i,ArrType &f,ArrType &e)
{//静态链表 L 的 r 域中记录已按 (keys[0], ..., keys[i-1]) 有序
//本算法按第 i 个关键字 keys[i] 建立 RADIX 个子表, 使同一子表中记录的 keys[i] 相同
//f[0..RADIX-1] 和 e[0..RADIX-1] 分别指向各子表中第一个和最后一个记录
    for(j=0;j<RADIX;++j) f[j]=0;           //各子表初始化为空表
    for(p=r[0].next;p;p=r[p].next)
    {
        j=ord(r[p].keys[i]);                //ord 将记录中第 i 个关键字映射到 [0..RADIX-1]
        if(!f[j]) f[j]=p;
        else r[e[j]].next=p;
        e[j]=p;                             //将 p 所指的结点插入第 j 个子表中
    }
    //for

void Collect (SLCell &r, int i, ArrType f, ArrType e)
{//本算法按 keys[i] 自小至大地将 f[0..RADIX-1] 所指各子表依次链接成一个链表
//e[0..RADIX-1] 为各子表的尾指针
    for(j=0;!f[j]; j=succ(j));              //找第一个非空子表, succ 为求后继函数
    r[0].next=f[j]; t=e[j];                 //r[0].next 指向第一个非空子表中第一个结点
    while(j<RADIX)
    {
        for(j=succ(j);j<RADIX-1&&!f[j];j=succ(j)); //找下一个非空子表
        if(f[j]) {r[t].next=f[j]; t=e[j];}       //链接两个非空子表
    }
    //while
    r[t].next=0;                               //t 指向最后一个非空子表中的最后一个结点
}

void RadixSort(SLList &L)
{//L 是采用静态链表表示的顺序表
//对 L 做基数排序, 使得 L 成为按关键字自小到大的有序静态链表, L.r[0] 为头结点
    for(i=0;i<L.recnum;++i) L.r[i].next=i+1;
    L.r[L.recnum].next = 0;                   //将 L 改造为静态链表
    for(i=0;i<L.keynum;++i) {                 //按最低位优先依次对各关键字进行分配和收集
        Distribute(L.r,i,f,e);                //第 i 趟分配
        Collect(L.r,i,f,e);                   //第 i 趟收集
    }
    //for
}

```

### 【算法分析】

#### (1) 时间复杂度

对于  $n$  个记录 (假设每个记录含  $d$  个关键字, 每个关键字的取值范围为  $rd$  个值) 进行链式基数排序时, 每一趟分配的时间复杂度为  $O(n)$ , 每一趟收集的时间复杂度为  $O(rd)$ , 整个排序需进行  $d$  趟分配和收集, 所以时间复杂度为  $O(d(n + rd))$ 。

#### (2) 空间复杂度

所需辅助空间为  $2rd$  个队列指针, 另外由于需用链表做存储结构, 则相对于其他以顺序结构存储记录的排序方法而言, 还增加了  $n$  个指针域的空间, 所以空间复杂度为  $O(n + rd)$ 。

## 【算法特点】

- (1) 是稳定排序。
- (2) 可用于链式结构, 也可用于顺序结构。
- (3) 时间复杂度可以突破基于关键字比较一类方法的下界  $O(n\log_2 n)$ , 达到  $O(n)$ 。
- (4) 基数排序使用条件有严格的要求: 需要知道各级关键字的主次关系和各级关键字的取值范围。

## 8.7 外部排序

前面讨论的都是内部排序的方法, 即整个排序过程全部是在内存中完成的, 并不涉及数据的内外存交换问题。但如果待排序的记录数目很大, 无法一次性调入内存, 整个排序过程就必须借用外存分批调入内存才能完成。

### 8.7.1 外部排序的基本方法

外部排序基本上由两个相对独立的阶段组成。首先, 按可用内存大小, 将外存上含  $n$  个记录的文件分成若干长度为  $l$  的子文件或段 (segment), 依次读入内存并利用有效的内部排序方法对它们进行排序, 并将排序后得到的有序子文件重新写入外存, 通常称这些有序子文件为归并段或顺串 (run); 然后, 对这些归并段进行逐趟归并, 使归并段 (有序的子文件) 逐渐由小至大, 直至得到整个有序文件为止。显然, 第一阶段所涉及的内部排序的工作在前几节已经讨论过。本节主要讨论第二阶段即归并的过程。先从一个具体例子来看外排中的归并是如何进行的?

假设有一个含 10 000 个记录的文件, 首先通过 10 次内部排序得到 10 个初始归并段  $R_1 \sim R_{10}$ , 其中每一段都含 1 000 个记录。然后对它们作如图 8.16 所示的两两归并, 直至得到一个有序文件为止。

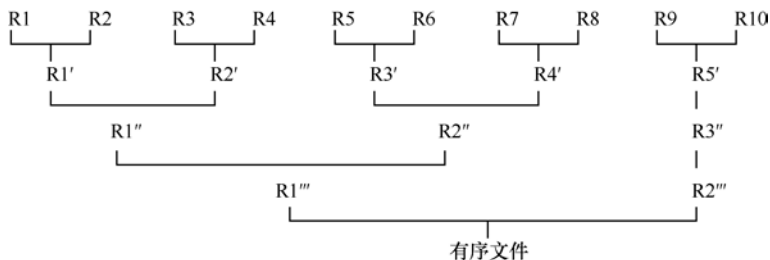


图 8.16 2-路平衡归并

从图 8.16 可见, 由 10 个初始归并段到一个有序文件, 共进行了 4 趟归并, 每一趟从  $m$  个归并段得到  $\lceil m/2 \rceil$  个归并段。这种归并方法称为 **2-路平衡归并**。

将两个有序段归并成一个有序段的过程, 若在内存进行, 则很简单, 算法 8.10 的 merge 过程便可实现此归并。但是, 在外部排序中实现两两归并时, 不仅要调用 merge 过程, 而且要进行外存的读/写, 这是由于我们不可能将两个有序段及归并结果段同时存放在内存中的缘故。我们知道, 对外存上信息的读/写是以“物理块”为单位的。假设在上例中每个物理块可以容纳 200 个记录, 则每一趟归并需进行 50 次“读”和 50 次“写”, 4 趟归并加上内部排序时所需进行的读/写使得

在外排中总共需进行 500 次的读/写。

一般情况下，

外部排序所需总的时间=内部排序（产生初始归并段）所需的时间（ $m \times t_{IS}$ ）+

外存信息读写的时间（ $d \times t_{IO}$ ）+

内部归并所需的时间（ $s \times ut_{mg}$ ）

(8-4)

其中， $t_{IS}$ 是为得到一个初始归并段进行内部排序所需时间的均值； $t_{IO}$ 是进行一次外存读/写时间的均值； $ut_{mg}$ 是对  $u$  个记录进行内部归并所需时间； $m$  为经过内部排序之后得到的初始归并段的个数； $s$  为归并的趟数； $d$  为总的读/写次数。由此，上例 10 000 个记录利用 2-路归并进行外排所需总的时间为：

$$10 \times t_{IS} + 500 \times t_{IO} + 4 \times 10\,000 t_{mg}$$

其中， $t_{IO}$  取决于所用的外存设备，显然， $t_{IO}$  较  $t_{mg}$  要大得多。因此，提高外排的效率应主要着眼于减少外存信息读写的次数  $d$ 。

下面来分析  $d$  和“归并过程”的关系。若对上例中所得的 10 个初始归并段进行 5-路平衡归并（即每一趟将 5 个或 5 个以下的有序子文件归并成一个有序子文件），则从图 8.17 可见，仅需进行二趟归并，外排时总的读/写次数便减至  $2 \times 100 + 100 = 300$ ，比 2-路归并减少了 200 次的读/写。

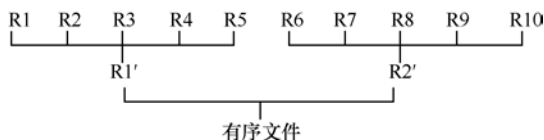


图 8.17 5-路平衡归并

可见，对同一文件而言，进行外排时所需读/写外存的次数和归并的趟数  $s$  成正比。而在一般情况下，对  $m$  个初始归并段进行  $k$ -路平衡归并时，归并的趟数

$$s = \lceil \log_k m \rceil \quad (8-5)$$

此时，为了减少归并趟数  $s$ ，可以从以下两个方面进行改进：

- (1) 增加归并段的个数  $k$ ；
- (2) 减少初始归并段的个数  $m$ 。

其中，“多路平衡归并”的方法是通过增加归并段的个数来减少对数据的扫描趟数；“置换-选择”的方法是在扫描一遍的前提下，能够得到更长的初始归并段，从而减少了初始归并段的个数。

下面分别就这两个方面讨论之。

## 8.7.2 多路平衡归并的实现

从式 (8-5) 得知，增加  $k$  可以减少  $s$ ，从而减少外存读/写的次数。但是，从下面的讨论中又可发现，单纯增加  $k$  将导致增加内部归并的时间  $ut_{mg}$ 。那么，如何解决这个矛盾呢？

先看 2-路归并。令  $u$  个记录分布在两个归并段上，按 merge 过程进行归并。每得到归并后的一个记录，仅需一次比较即可，则得到含  $u$  个记录的归并段需进行  $u-1$  次比较。

再看  $k$ -路归并。令  $u$  个记录分布在  $k$  个归并段上，显然，归并后的第一个记录应是  $k$  个归并段中关键字最小的记录，即应从每个归并段的第一个记录的相互比较中选出最小者，这需要进行

$k-1$  次比较。同理, 每得到归并后的有序段中的一个记录, 都要进行  $k-1$  次比较。显然, 为得到含  $u$  个记录的归并段需进行  $(u-1)(k-1)$  次比较。由此, 对  $n$  个记录的文件进行外排时, 在内部归并过程中进行的总的比较次数为  $s(k-1)(n-1)$ 。假设所得初始归并段为  $m$  个, 则由式 (8-5) 可得内部归并过程中进行比较的总的次数为

$$\lceil \log_k m \rceil (k-1)(n-1)t_{mg} = \left\lceil \frac{\log_2 m}{\log_2 k} \right\rceil (k-1)(n-1)t_{mg} \quad (8-6)$$

由于  $\frac{k-1}{\log_2 k}$  随  $k$  的增长而增长, 则内部归并时间亦随  $k$  的增长而增长。这将抵消由于增大  $k$  而减少外存信息读写时间所得效益, 这是我们所不希望的。然而, 若在进行  $k$ -路归并时利用“败者树”(Tree of Loser), 则可使在  $k$  个记录中选出关键字最小的记录时仅需进行  $\lceil \log_2 k \rceil$  次比较, 从而使总的归并时间由式 (8-6) 变为  $\lceil \log_2 m \rceil (n-1)t_{mg}$ , 显然, 这个式子和  $k$  无关, 它不再随  $k$  的增长而增长。

那么, 什么是“败者树”? 它是树形选择排序的一种变形。相对地, 我们可称图 8.7 和图 8.8 中的二叉树为“胜者树”, 因为每个非终端结点均表示其左、右孩子结点中的“胜者”。反之, 若在双亲结点中记下刚进行完的这场比赛中的败者, 而让胜者去参加更高一层的比赛, 便可得到一棵“败者树”。例如, 图 8.18 (a) 所示为一棵实现 5-路归并的败者树  $ls[0..4]$ , 图中方形结点表示叶子结点 (也可看成是外结点), 分别为 5 个归并段中当前参加归并选择的记录的关键字; 败者树中根结点  $ls[1]$  的双亲结点  $ls[0]$  为“冠军”, 在此指示各归并段中的最小关键字记录为第 3 段中的当前记录; 结点  $ls[3]$  指示  $b1$  和  $b2$  两个叶子结点中的败者即  $b2$ , 而胜者  $b1$  和  $b3$  ( $b3$  是叶子结点  $b3$ 、 $b4$  和  $b0$  经过两场比赛后选出的获胜者) 进行比较, 结点  $ls[1]$  则指示它们中的败者为  $b1$ 。在选得最小关键字的记录之后, 只要修改叶子结点  $b3$  中的值, 使其为同一归并段中的下一个记录的关键字, 然后从该结点向上和双亲结点所指示的关键字进行比较, 败者留在该双亲结点, 胜者继续向上直至树根的双亲。如图 8.18 (b) 所示, 当第 3 个归并段中第 2 个记录参加归并时, 选得的最小关键字记录为第 1 个归并段中的记录。为防止在归并过程中某个归并段变空, 可以在每个归并段中附加一个关键字为最大值的记录。当选出的“冠军”记录的关键字为最大值时, 表明此次归并已完成。由于实现  $k$ -路归并的败者树的深度为  $\lceil \log_2 k \rceil + 1$ , 则在  $k$  个记录中选择最小关键字仅需进行  $\lceil \log_2 k \rceil$  次比较。败者树的初始化也容易实现, 只要先令所有的非终端结点指向一个含最小关键字的叶子结点, 然后从各个叶子结点出发调整非终端结点为新的败者即可。

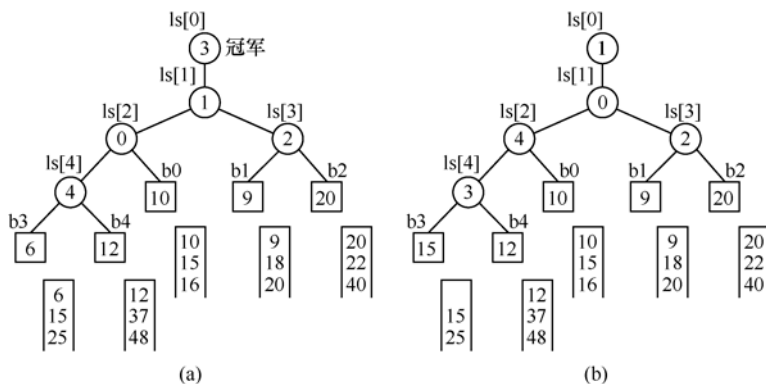


图 8.18 实现 5-路归并的败者树

最后要提及一点,  $k$  值的选择并非越大越好, 如何选择合适的  $k$  是一个需要综合考虑的问题。

### 8.7.3 置换-选择排序

由 8-5 式得知, 归并的趟数  $s$  不仅和  $k$  成反比, 也和  $m$  成反比, 因此, 减少  $m$  是减少  $s$  的另一条途径。然而, 我们从 8.7.1 小节的讨论中也得知,  $m$  是外部文件经过内部排序之后得到的初始归并段的个数, 显然,  $m = \lceil n/l \rceil$ , 其中  $n$  为外部文件中的记录数,  $l$  为初始归并段中的记录数。回顾前面讨论的各种内排方法, 在内排过程中移动记录和对关键字进行比较都是在内存中进行的。因此, 用这些方法进行内部排序得到的各个初始归并段的长度  $l$  (除最后一段外) 都相同, 且完全依赖于进行内部排序时可用内存工作区的大小, 则  $m$  也随其而限定。由此, 若要减小  $m$ , 即增加  $l$ , 就必须探索新的排序方法。

**置换-选择排序** (Replacement-Selection Sorting) 是在树形选择排序的基础上得来的, 它的特点是: 在整个排序 (得到所有初始归并段) 的过程中, 选择最小 (或最大) 关键字和输入、输出交叉或平行进行。

先从具体例子谈起。已知初始文件含有 24 个记录, 它们的关键字分别为 51, 49, 39, 46, 38, 29, 14, 61, 15, 30, 1, 48, 52, 3, 63, 27, 4, 13, 89, 24, 46, 58, 33, 76。假设内存工作区可容纳 6 个记录, 则按前面讨论的选择排序可求得如下 4 个初始归并段:

RUN1: 29, 38, 39, 46, 49, 51

RUN2: 1, 14, 15, 30, 48, 61

RUN3: 3, 4, 13, 27, 52, 63

RUN4: 24, 33, 46, 58, 76, 89

若按置换-选择排序进行排序, 则可求得如下 3 个初始归并段:

RUN1: 29, 38, 39, 46, 49, 51, 61

RUN2: 1, 3, 14, 15, 27, 30, 48, 52, 63, 89

RUN3: 4, 13, 24, 33, 46, 58, 76

假设初始待排文件为输入文件 FI, 初始归并段文件为输出文件 FO, 内存工作区为 WA, FO 和 WA 的初始状态为空, 并设内存工作区的容量可容纳  $w$  个记录, 则置换-选择排序的操作过程为:

- ① 从 FI 输入  $w$  个记录到工作区 WA。
  - ② 从 WA 中选出其中关键字取最小值得记录, 记为 MINIMAX 记录。
  - ③ 将 MINIMAX 记录输入到 FO 中去。
  - ④ 若 FI 不空, 则从 FI 输入下一个记录到 WA 中。
  - ⑤ 从 WA 中所有关键字比 MINIMAX 记录的关键字大的记录中选出最小关键字记录, 作为新的 MINIMAN 记录。
  - ⑥ 重复③~⑤, 直至 WA 中选不出新的 MIMIMAX 记录为止, 由此得到一个初始归并段, 输出一个归并段的结束标志到 FO 中去。
  - ⑦ 重复②~⑥, 直至 WA 为空。由此得到所有初始归并段。
- 例如, 以上所举之例的置换-选择过程如表 8.1 所示。

表 8.1

置换-选择的过程

| FO                         | WA                | FI                                                  |
|----------------------------|-------------------|-----------------------------------------------------|
| 空                          | 空                 | 51,49,39,46,38,29,14,61,15,30,1,48,52,3,63,27, 4... |
| 空                          | 51,49,39,46,38,29 | 14,61,15,30,1,48,52,3,63,27,4,...                   |
| 29                         | 51,49,39,46,38    | 14,61,15,30,1,48,52,3,63,27,4,...                   |
| 29                         | 51,49,39,46,38,14 | 61,15,30,1,48,52,3,63,27,4,...                      |
| 29,38                      | 51,49,39,46, ,14  | 61,15,30,1,48,52,3,63,27,4,...                      |
| 29,38                      | 51,49,39,46,61,14 | 15,30,1,48,52,3,63,27,4,...                         |
| 29,38,39                   | 51,49, , 46,61,14 | 15,30,1,48,52,3,63,27,4,...                         |
| 29,38,39                   | 51,49,15,46,61,14 | 30,1,48,52,3,63,27,4,...                            |
| 29,38,39,46                | 51,49,15, , 61,14 | 30,1,48,52,3,63,27,4,...                            |
| 29,38,39,46                | 51,49,15,30,61,14 | 1,48,52,3,63,27,4,...                               |
| 29,38,39,46,49             | 51, ,15,30,61,14  | 1,48,52,3,63,27,4,...                               |
| 29,38,39,46,49             | 51,1,15,30,61,14  | 48,52,3,63,27,4,...                                 |
| 29,38,39,46,49,51          | ,1,15,30,61,14    | 48,52,3,63,27,4,...                                 |
| 29,38,39,46,49,51          | 48,1,15,30,61,14  | 52,3,63,27,4,...                                    |
| 29,38,39,46,49,51,61       | 48,1,15,30, ,14   | 52,3,63,27,4,...                                    |
| 29,38,39,46,49,51,61       | 48,1,15,30,52 ,14 | 3,63,27,4,...                                       |
| 29,38,39,46,49,51,61, *    | 48,1,15,30,52,14  | 3,63,27,4,...                                       |
| 29,38,39,46,49,51,61, *, 1 | 48, ,15,30,52,14  | 3,63,27,4,...                                       |
| 29,38,39,46,49,51,61, *, 1 | 48,3,15,30,52,14  | 63,27,4,...                                         |
| ...                        | ...               | ...                                                 |

在 WA 中选择 MINIMAX 记录的过程需利用“败者树”来实现。关于“败者树”本身，上节已有详细讨论，在此仅就置换-选择排序中的实现细节加以说明。

(1) 内存工作区中的记录作为败者树的外部结点，而败者树中根结点的双亲结点指示工作区中关键字最小的记录。



(2) 为了便于选出 MINIMAX 记录，为每个记录附设一个所在归并段的序号，在进行关键字的比较时，先比较段号，段号小者为胜者；段号相同的则关键字小的为胜者。

(3) 败者树的建立可从设工作区中所有记录的段号均为“零”开始，然后从 FI 逐个输入  $w$  个记录到工作区时，自上而下调整败者树。由于这些记录的段号为“1”，则它们对于“零”段的记录而言均为败者，从而逐个填充到败者树的各结点中去。

下面利用败者树对前面例子进行置换-选择排序时的局部状况进行说明，如图 8.19 所示。其中，内存工作区的存储结构定义如下：

```
// - - - - 内存工作区的存储结构- - - -
typedef struct
{
    RcdType rec;           //记录
    KeyType key;           //从记录中抽取的关键字
    int rnum;              //所属归并段的段号
} RcdNode, WorkArea[w];   //内存工作区，容量为 w
WorkArea wa;
```

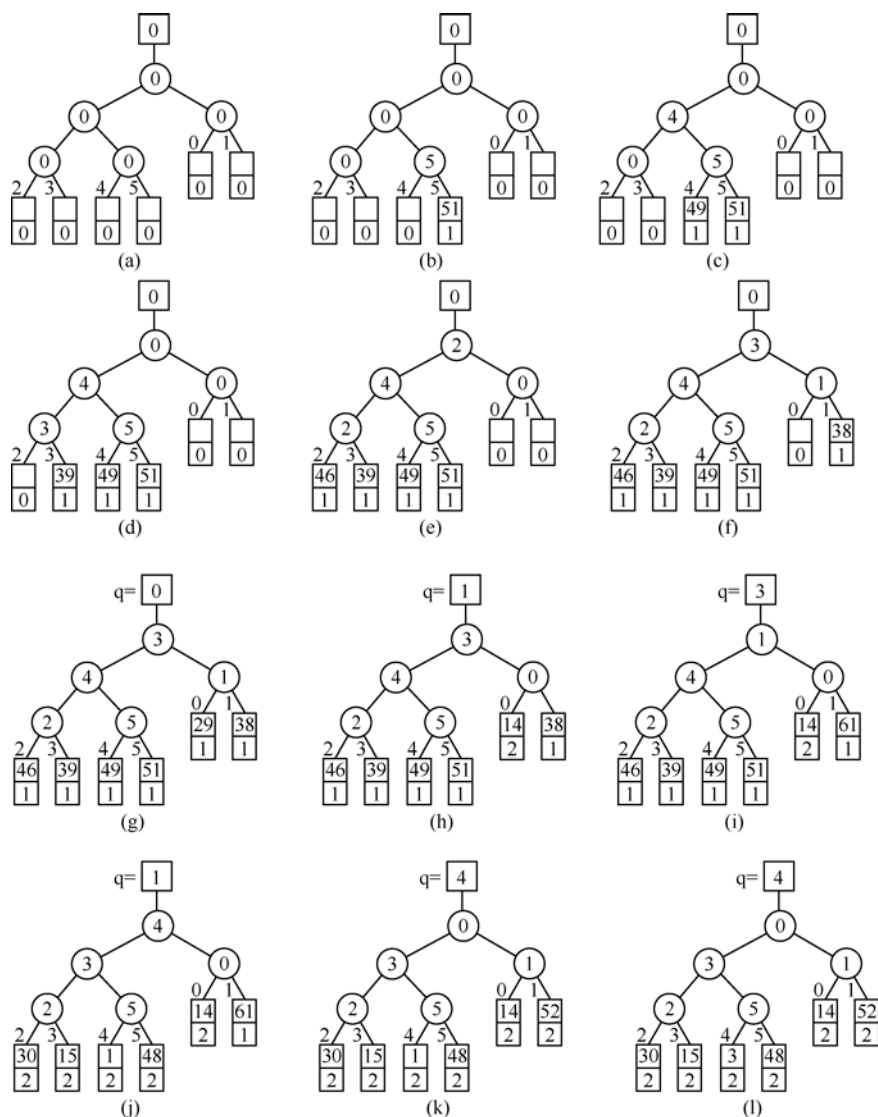


图 8.19 置换-选择过程中的败者树

(a) ~ (g) 建立败者树, 选出最小关键字记录  $wa[0]$  (h) ~ (l) 选好新的 MINIMAX 记录

图 8.19 (a) ~ (g) 显示了败者树建立过程中的状态变化状况。最后得到最小关键字的记录为  $wa[0]$ , 之后, 输出  $wa[0].rec$ , 并从 FI 中输入下一个记录至  $wa[0]$ 。由于它的关键字小于刚刚输出的记录的关键字, 则设此新输入的记录的段号为 2 (如图 8.19 (h) 所示), 而由于在输出  $wa[1]$  之后新输入的关键字较  $wa[1].key$  大, 则该新输入的记录的段号仍为 1 (如图 8.19 (i) 所示)。图 8.19 (j) 所示为在输出 6 个记录之后选得的 MINIMAX 记录为  $wa[1]$  时的败者树。图 8.19 (k) 表明在该记录  $wa[1]$  之后, 由于输入的下一条记录的关键字较小, 其段号亦为 2, 致使工作区中的所有记录的段号均为 2。由此败者树选出的新的 MINIMAX 记录的段号大于当前生成的归并段的序号。这说明该段已结束, 而此新的 MINIMAX 记录应是下一归并段中的第一个记录。

从上述可见, 由置换-选择排序所得初始归并段的长度不等。且可证明, 当输入文件中记录的关键字为随机数时, 所得初始归并段的平均长度为内存工作区大小  $w$  的两倍。这个证明是

E.F.Moore 在 1961 年从置换-选择排序和扫雪机的类比中得出的。

假设一台扫雪机在环形路上等速进行扫雪, 又下雪的速度也是均衡的 (即每小时落到地面上的雪量相等), 雪均匀地落在扫雪机的前、后路面上, 边下雪边扫雪。显然, 在某个时刻之后, 整个系统达到平衡状态, 路面上的积雪总量不变。且在任何时刻, 整个路面上的积雪都形成一个均匀的斜面, 紧靠扫雪机前端的积雪最厚, 其深度为  $h$ , 而在扫雪机刚扫过的路面上的积雪深度为零。若将环形路伸展开来, 路面积雪状态如图 8.20 所示。假设此刻路面积雪的总体积为  $w$ , 环形路一圈的长度为  $l$ , 由于扫雪机在任何时刻扫走的雪的深度为  $h$ , 则扫雪机在环形路上走一圈扫掉的积雪体积为  $lh$  即  $2w$ 。

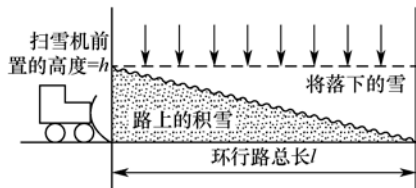


图 8.20 环形路上扫雪机系统平衡时的状态

将置换-选择排序与此类比, 工作区中的记录好比路面的积雪, 输出的 MINIMAX 记录好比扫走的雪, 新输入的记录好比新下的雪, 当关键字为随机数时, 新记录的关键字比 MINIMAX 大或小的概率相等。若大, 则属当前归并段 (好比落在扫雪机前面的积雪, 在这一圈中将被扫走); 若小, 则属下一归并段 (好比落在扫雪机后面的积雪, 在下一圈中才能扫走)。由此, 得到一个初始归并段好比扫雪机走一圈。假设工作区的容量为  $w$ , 则置换-选择所得初始归并段长度的期望值便为  $2w$ 。

容易看出, 若不计输入、输出的时间, 则对  $n$  个记录的文件而言, 生成所有初始归并段所需时间为  $O(n \log_2 w)$ 。

## 8.7.4 最佳归并树

这一节要讨论的问题是, 由置换-选择生成所得的初始归并段, 其各段长度不等对平衡归并有何影响?

假设由置换-选择得到 9 个初始归并段, 其长度 (即记录数) 依次为 9, 30, 12, 18, 3, 17, 2, 6, 24。现作 3-路平衡归并, 其归并树 (表示归并过程的图) 如图 8.21 所示, 图中每个圆圈表示一个初始归并段, 圆圈中数字表示归并段的长度。假设每个记录占一个物理块, 则两趟归并所需对外存进行读/写次数为

$$(9+30+12+18+3+17+2+6+24) \times 2 \times 2 = 484$$

若将初始归并段的长度看成是归并树中叶子结点的权, 则此 3 叉树的带权路径长度的两倍恰为 484。显然, 归并方案不同, 树的带权路径长度 (或外存读/写次数) 亦不同。回顾在第 5 章中曾讨论了有  $n$  个叶子结点的带权路径长度最短的二叉树称哈夫曼树, 同理, 存在有  $n$  个叶子结点的带权路径长度最短的 3 叉、4 叉、...、 $k$  叉树, 亦称哈夫曼树。因此, 若对长度不等的  $m$  个初始归并段, 构造一棵哈夫曼树作为归并树, 便可使在进行外部归并时所需对外存进行读/写次数达最少。例如, 对上述 9 个初始归并段可构造一棵如图 8.22 所示的归并树, 按此树进行归并, 仅需对外存进行 446 次读/写, 这棵归并树便称做**最佳归并树**。

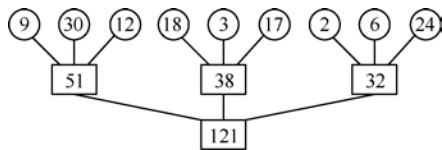


图 8.21 3-路平衡归并的归并树

图 8.22 的哈夫曼树是一棵真正的 3 叉树, 即树中只有度为 3 或 0 的结点。假若只有 8 个初始归并段, 例如, 在前面例子中少了一个长度为 30 的归并段。如果在设计归并方案时, 缺额的归并段留在最后, 即除了最后一次作 2-路归并外, 其他各次归并仍都是 3 路归并, 容易看出此归并方

案的外存读/写次数为 386。显然,这不是最佳方案。正确的做法是,当初始归并段的数目不足时,需附加长度为零的“虚段”,按照哈夫曼树构成的原则,权为零的叶子应离树根最远,因此,这个只有 8 个初始归并段的归并树应如图 8.23 所示。

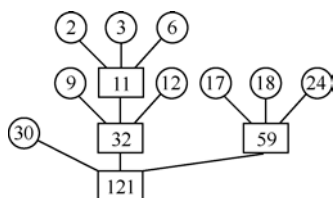


图 8.22 3-路平衡归并的最佳归并树图

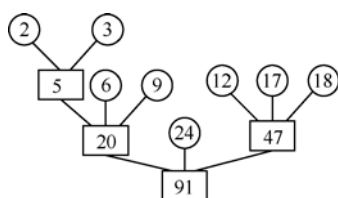


图 8.23 8 个归并段的最佳归并树

那么,如何判断附加虚段的数目?当3叉树中只有度为3或0的结点时,必有 $n_3 = (n_0 - 1)/2$ ,其中, $n_3$ 是度为3的结点数, $n_0$ 是度为0的结点数。由于 $n_3$ 必为整数,则 $(n_0 - 1) \text{ MOD } 2 = 0$ 。也就是说,对3-路归并而言,只有当初始归并段的个数为偶数时,才需加1个虚段。

在一般情况下,对 $k$ -路归并而言,容易推算得到,若 $(m-1) \text{ MOD } (k-1) = 0$ ,则不需加虚段,否则需附加 $k - (m-1) \text{ MOD } (k-1) - 1$ 个虚段。换句话说,第一次归并为 $(m-1) \text{ MOD } (k-1) + 1$ 路归并。

若按最佳归并树的归并方案进行磁盘归并排序,需在内存建立一张载有归并段的长度和它在磁盘上的物理位置的索引表。

## 8.8 小 结

本章介绍了内部排序和外部排序的方法,内部排序是外部排序的基础,必须通过内部排序产生初始归并段之后,才能进行外部排序。

对于内部排序,总计介绍了5类9种较常用的排序方法,下面从时间复杂度、空间复杂度和稳定性几个方面对这些内部排序方法做比较,结果如表 8.2 所示。

表 8.2 各种内部排序方法的比较

| 排序方法   | 时间复杂度           |                 |                 | 空间复杂度         | 稳定性 |
|--------|-----------------|-----------------|-----------------|---------------|-----|
|        | 最好情况            | 最坏情况            | 平均情况            |               |     |
| 直接插入排序 | $O(n)$          | $O(n^2)$        | $O(n^2)$        | $O(1)$        | 稳定  |
| 折半插入排序 | $O(n \log_2 n)$ | $O(n^2)$        | $O(n^2)$        | $O(1)$        | 稳定  |
| 希尔排序   |                 |                 | $O(n^{1.3})$    | $O(1)$        | 不稳定 |
| 冒泡排序   | $O(n)$          | $O(n^2)$        | $O(n^2)$        | $O(1)$        | 稳定  |
| 简单选择排序 | $O(n^2)$        | $O(n^2)$        | $O(n^2)$        | $O(1)$        | 稳定  |
| 快速排序   | $O(n \log_2 n)$ | $O(n^2)$        | $O(n \log_2 n)$ | $O(\log_2 n)$ | 不稳定 |
| 堆排序    | $O(n \log_2 n)$ | $O(n \log_2 n)$ | $O(n \log_2 n)$ | $O(1)$        | 不稳定 |
| 归并排序   | $O(n \log_2 n)$ | $O(n \log_2 n)$ | $O(n \log_2 n)$ | $O(n)$        | 稳定  |
| 基数排序   | $O(d(n + rd))$  | $O(d(n + rd))$  | $O(d(n + rd))$  | $O(n + rd)$   | 稳定  |

从表 8.2 的时间复杂度的平均情况来看,直接插入排序、折半插入排序、冒泡排序和简单选择排序的速度较慢,而其他排序方法的速度较快。从算法实现的角度来看,速度较慢的算法实现

过程比较简单,称之为简单的排序方法;而速度较快的算法可以看作是对某一排序算法的改进,称之为先进的排序方法,但这些算法实现过程比较复杂。总的来看,各种排序算法各有优缺点,没有哪一种是绝对最优的。在使用时需根据不同情况适当选用,甚至可将多种方法结合起来使用。一般综合考虑以下因素:

- (1) 待排序的记录个数;
- (2) 记录本身的大小;
- (3) 关键字的结构及初始状态;
- (4) 对排序稳定性的要求;
- (5) 存储结构。

根据这些因素和表 8.2 所做的比较,可以得出以下几点结论。

(1) 当待排序的记录个数  $n$  较小时,  $n^2$  和  $n\log_2 n$  的差别不大,可选用简单的排序方法。而当关键字基本有序时,可选用直接插入排序或冒泡排序,排序速度很快,其中直接插入排序最为简单常用、性能最佳。

(2) 当  $n$  较大时,应该选用先进的排序方法。对于先进的排序方法,从平均时间性能而言,快速排序最佳,是目前基于比较的排序方法中最好的方法。但在最坏情况下,即当关键字基本有序时,快速排序的递归深度为  $n$ ,时间复杂度为  $O(n^2)$ ,空间复杂度为  $O(n)$ 。堆排序和归并排序不会出现快速排序的最坏情况,但归并排序的辅助空间较大。这样,当  $n$  较大时,具体选用的原则是:

- ① 当关键字分布随机,稳定性不做要求时,可采用快速排序;
- ② 当关键字基本有序,稳定性不做要求时,可采用堆排序;
- ③ 当关键字基本有序,内存允许且要求排序稳定时,可采用归并排序。

(3) 可以将简单的排序方法和先进的排序方法结合使用。例如,当  $n$  较大时,可以先将待排序序列划分成若干子序列,分别进行直接插入排序,然后再利用归并排序,将有序子序列合并成一个完整的有序序列。或者,在快速排序中,当划分子区间的长度小于某值时,可以转而调用直接插入排序算法。

(4) 基数排序的时间复杂度也可写成  $O(d \cdot n)$ 。因此,它最适用于  $n$  值很大而关键字较小的序列。若关键字也很大,而序列中大多数记录的“最高位关键字”均不同,则亦可先按“最高位关键字”不同将序列分成若干“小”的子序列,而后进行直接插入排序。但基数排序使用条件有严格的要求:需要知道各级关键字的主次关系和各级关键字的取值范围,即只适用于像整数和字符这类有明显结构特征的关键字,当关键字的取值范围为无穷集合时,则无法使用基数排序。

(5) 从方法的稳定性来比较,基数排序是稳定的内排方法,所有时间复杂度为  $O(n^2)$  的简单排序法也是稳定的,然而,快速排序、堆排序和希尔排序等时间性能较好的排序方法都是不稳定的。

一般来说,如果排序过程中的“比较”是在“相邻的两个记录关键字”间进行的,则排序方法是稳定的。值得提出的是,稳定性是由方法本身决定的,对不稳定的排序方法而言,不管其描述形式如何,总能举出一个说明不稳定的实例来。反之,对稳定的排序方法,可能有的描述形式会引起不稳定,但总能找到一种不引起不稳定的描述形式。由于大多数情况下排序是按记录的主关键字进行的,则所用的排序方法是否稳定无关紧要。若排序按记录的次关键字进行,则必须采用稳定的排序方法。

(6) 在本章讨论的排序方法中,多数是采用顺序表实现的。若记录本身信息量较大,为避免

移动记录耗费大量时间,可采用链式存储结构。比如直接插入排序、归并排序都易于在链表上实现。但像折半插入排序、希尔排序、快速排序和堆排序,却难于在链表上实现。

对于外部排序,常用的方法是归并方法,这种方法主要由两个独立的阶段组成:第一,把待排序的文件划分成若干个子文件;第二,逐趟归并子文件,最后形成对整个文件的排序。为减少归并中外存读写的次数,提高外排序的效率,一般通过增大归并路数和减少初始归并段个数两种方案对归并算法进行改进,其中,“多路平衡归并”的方法可以增加归并段的个数,“置换-选择”的方法可以减少初始归并段的个数。

学完本章后,要求掌握与排序相关的基本概念,如:关键字比较次数、数据移动次数、稳定性、内部排序、外部排序。深刻理解各种内部排序方法的基本思想、特点、实现方法及其性能分析,能从时间、空间、稳定性各个方面对各种排序方法做综合比较,并能加以灵活应用。掌握外部排序方法中败者树的建立及归并方法,掌握置换-选择排序的过程和最佳归并树的构造方法。

## 习 题

### 1. 选择题

(1) 从未排序序列中依次取出元素与已排序序列(初始时空)中的元素进行比较,将其放入已排序序列的正确位置上的方法,这种排序方法称为( )。

- A. 归并排序
- B. 冒泡排序
- C. 插入排序
- D. 选择排序

(2) 从未排序序列中挑选元素,并将其依次插入已排序序列(初始时空)末端的方法,称为( )。

- A. 归并排序
- B. 冒泡排序
- C. 插入排序
- D. 选择排序

(3) 对  $n$  个不同的关键字由小到大进行冒泡排序,在下列( )情况下比较的次数最多。

- A. 从小到大排列好的
- B. 从大到小排列好的
- C. 元素无序
- D. 元素基本有序

(4) 对  $n$  个不同的排序码进行冒泡排序,在元素无序的情况下比较的次数为( )。

- A.  $n+1$
- B.  $n$
- C.  $n-1$
- D.  $n(n-1)/2$

(5) 快速排序在下列( )情况下最易发挥其长处。

- A. 被排序的数据中含有多个相同排序码
- B. 被排序的数据已基本有序
- C. 被排序的数据完全无序
- D. 被排序的数据中的最大值和最小值相差悬殊

(6) 对  $n$  个关键字做快速排序,在最坏情况下,算法的时间复杂度是( )。

- A.  $O(n)$
- B.  $O(n^2)$
- C.  $O(n\log_2 n)$
- D.  $O(n^3)$

(7) 若一组记录的排序码为(46, 79, 56, 38, 40, 84),则利用快速排序的方法,以第一个记录为基准得到的一次划分结果为( )。

- A. 38, 40, 46, 56, 79, 84                      B. 40, 38, 46, 79, 56, 84  
C. 40, 38, 46, 56, 79, 84                      D. 40, 38, 46, 84, 56, 79
- (8) 下列关键字序列中, ( ) 是堆。  
A. 16, 72, 31, 23, 94, 53                      B. 94, 23, 31, 72, 16, 53  
C. 16, 53, 23, 94, 31, 72                      D. 16, 23, 53, 31, 94, 72
- (9) 堆是一种 ( ) 排序。  
A. 插入                      B. 选择                      C. 交换                      D. 归并
- (10) 堆的形状是一棵 ( )。  
A. 二叉排序树                      B. 满二叉树                      C. 完全二叉树                      D. 平衡二叉树
- (11) 若一组记录的排序码为 (46, 79, 56, 38, 40, 84), 则利用堆排序的方法建立的初始堆为 ( )。  
A. 79, 46, 56, 38, 40, 84  
B. 84, 79, 56, 38, 40, 46  
C. 84, 79, 56, 46, 40, 38  
D. 84, 56, 79, 40, 46, 38
- (12) 下述几种排序方法中, 要求内存最大的是 ( )。  
A. 希尔排序                      B. 快速排序                      C. 归并排序                      D. 堆排序
- (13) 下述几种排序方法中, ( ) 是稳定的排序方法。  
A. 希尔排序                      B. 快速排序                      C. 归并排序                      D. 堆排序
- (14) 数据表中有 10000 个元素, 如果仅要求求出其中最大的 10 个元素, 则采用 ( ) 算法最节省时间。  
A. 冒泡排序                      B. 快速排序  
C. 简单选择排序                      D. 堆排序
- (15) 下列排序算法中, 不能保证每趟排序至少能将一个元素放到其最终的位置上的排序方法是 ( )。  
A. 希尔排序                      B. 快速排序                      C. 冒泡排序                      D. 堆排序

## 2. 应用题

(1) 设待排序的关键字序列为 {12, 2, 16, 30, 28, 10, 16\*, 20, 6, 18}, 试分别写出使用以下排序方法, 每趟排序结束后关键字序列的状态。

- ① 直接插入排序
- ② 折半插入排序
- ③ 希尔排序 (增量选取 5、3 和 1)
- ④ 冒泡排序
- ⑤ 快速排序
- ⑥ 简单选择排序
- ⑦ 堆排序
- ⑧ 二路归并排序

(2) 给出如下关键字序列 {321, 156, 57, 46, 28, 7, 331, 33, 34, 63}, 试按链式基数排序方法, 列出每一趟分配和收集的过程。

(3) 对输入文件 (101, 51, 19, 61, 3, 71, 31, 17, 19, 100, 55, 20, 9, 30, 50, 6, 90);

当  $k=6$  时, 使用置换-选择算法, 写出建立的初始败者树及生成的初始归并段。

### 3. 算法设计题

(1) 试以单链表为存储结构, 实现简单选择排序算法。

(2) 有  $n$  个记录存储在带头结点的双向链表中, 现用双向冒泡排序法对其按升序进行排序, 请写出这种排序的算法。(注: 双向冒泡排序即相邻两趟排序向相反方向冒泡)

(3) 设有顺序放置的  $n$  个桶, 每个桶中装有一粒砾石, 每粒砾石的颜色是红、白、蓝之一。要求重新安排这些砾石, 使得所有红色砾石在前, 所有白色砾石居中, 所有蓝色砾石在后, 重新安排时对每粒砾石的颜色只能看一次, 并且只允许交换操作来调整砾石的位置。

(4) 编写算法, 对  $n$  个关键字取整数值的记录序列进行整理, 以使所有关键字为负值的记录排在关键字为非负值的记录之前, 要求:

① 采用顺序存储结构, 至多使用一个记录的辅助存储空间;

② 算法的时间复杂度为  $O(n)$ 。

(5) 借助于快速排序的算法思想, 在一组无序的记录中查找给定关键字值等于  $key$  的记录。设此组记录存放于数组  $r[1..n]$  中。若查找成功, 则输出该记录在  $r$  数组中的位置及其值, 否则显示 “not find” 信息。请简要说明算法思想并编写算法。

(6) 有一种简单的排序算法, 叫做计数排序。这种排序算法对一个待排序的表进行排序, 并将排序结果存放到另一个新的表中。必须注意的是, 表中所有待排序的关键字互不相同, 计数排序算法针对表中的每个记录, 扫描待排序的表一趟, 统计表中有多少个记录的关键字比该记录的关键字小。假设针对某一个记录, 统计出的计数值为  $c$ , 那么, 这个记录在新的有序表中的合适的存放位置即为  $c$ 。

① 给出适用于计数排序的顺序表定义;

② 编写实现计数排序的算法;

③ 对于有  $n$  个记录的表, 关键字比较次数是多少?

④ 与简单选择排序相比较, 这种方法是否更好? 为什么?

- [1] 严蔚敏, 吴伟民. 数据结构 (C 语言版). 北京: 清华大学出版社, 2011.
- [2] 严蔚敏, 陈文博. 数据结构及应用算法教程 (修订版). 北京: 清华大学出版社, 2011.
- [3] 陈越, 何钦铭, 徐镜春, 魏宝刚, 杨棣. 数据结构. 北京: 高等教育出版社, 2012.
- [4] 耿国华. 数据结构—用 C 语言描述. 北京: 高等教育出版社, 2011.
- [5] 王红梅, 胡明, 王涛. 数据结构 (C++ 版). 北京: 清华大学出版社, 2011.

Data Structure (2nd Edition)

# 数据结构 (C语言版) (第2版)

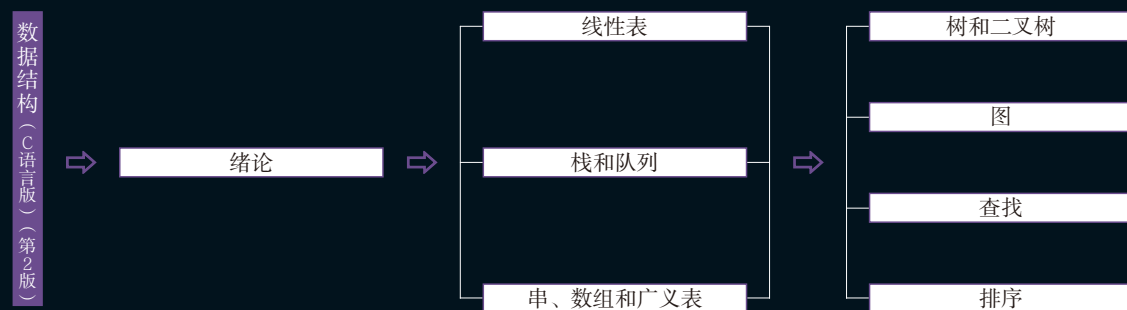
本书在选材与编排上,贴近当前普通高等院校“数据结构”课程的现状和发展趋势,符合最新研究生考试大纲,内容难度适中,突出实用性和应用性。全书共8章,内容包括绪论,线性表,栈和队列,串、数组和广义表,树和二叉树,图,查找和排序。全书采用类C语言作为数据结构和算法的描述语言。

本版教材主要做了以下方面的修订。

- 采用“案例驱动”的编写模式。书中结合实际应用,将各章按照“案例引入——数据结构及其操作——案例分析与实现”的案例驱动思路来展开。每章使用一个有趣的“问题案例”开头,由该案例逐步引入新的数据结构,然后给出该数据结构的存储表示及各种基本操作的实现,之后进一步分析此案例,最终利用该数据结构来实现此案例。
- 算法讲解更加细致。新版教材中对每个算法思想进行详细阐述,将用文字描述的算法步骤与用类C语言表述的算法描述一一对应。
- 优化教材内容。参考计算机专业最新的全国统考考研大纲,增加了大纲近两年新增的考点内容,如分块查找、外部排序等,有助于考研学生复习备考使用。

**严蔚敏** 清华大学教授,长期从事数据结构教学和教材建设,和吴伟民合作编著的《数据结构》曾获“第二届普通高等学校优秀教材全国特等奖”和“1996年度国家科学技术进步奖三等奖”,是目前国内数据结构教学领域的经典教材。

## 本教材的结构框图



免费提供

PPT等教学相关资料



人民邮电出版社  
教学服务与资源网  
[www.ptpedu.com.cn](http://www.ptpedu.com.cn)

教材服务热线: 010-81055256

反馈/投稿/推荐信箱: 315@ptpress.com.cn

本书将提供电子教案,可到人民邮电出版社网站<http://www.ptpress.com.cn>下载。

ISBN 978-7-115-37950-4



9 787115 379504 >